

COMPUTER GRAPHICS
A Multi-Volume Research-Level Study Plan
Version 4: Concrete Research and Programming 18-Volume Edition

From zero prerequisites to research apprenticeship, with a parallel mathematical-animation pathway

Designed for approximately 15-20 hours/week

Includes exact chapter/section guidance, current online courses/tutorials, topic-specific derivations, P1-P7 implementation laboratories, seven-problem exercise sets, concrete F1-F6 mini-research projects, self-contained G1-G6 mastery practicals, 18 volume-level programming capstones, paper-reading sequences, and research apprenticeship.

Resource links verified: 12 August 2026

How to Use This Expanded Plan

- This document is a curriculum, not a reading list. Never postpone implementation until after the theory: graphics understanding comes from the loop mathematics -> algorithm -> code -> image/animation -> measurement -> failure analysis.
- Each of the 144 modules is written as a self-contained roughly 1-3 week unit, with shorter or longer pacing allowed by background. A module contains the exact primary readings from the 67-page master plan, an ordered tutorial path, a checklist of definitions/results, derivations, implementation tasks, a problem set, research-paper reading and a mastery checkpoint.

How to use the checkpoints. Research sections are now executable mini-projects rather than templates: each states the actual stress case, competing methods, code extension, measurement set, artifact package, mechanism to test, and go/no-go rule. Mastery checkpoints are self-contained: they repeat the actual derivation, micro-oracle, clean-room build, deliberate bug, transfer experiment, and method-selection defense so you do not need to chase P1-P4 references months later.

- At 15-20 h/week, a normal unit should consume roughly 4-6 h reading/derivation, 5-8 h implementation, 2-4 h problems/experiments, 1-2 h paper reading, and optionally 1-2 h mathematical animation.
- If you already know a topic, use the mastery gate rather than the calendar. Skip a module only if you can derive the core result, implement a reduced version, and explain the numerical/representational failure modes.
- Beginning with Volumes 5-8, read primary papers every week. Beginning with Volume 13, paper reading and reproduction should take at least one third of study time.
- Volume 17 (Mathematical Animation) is deliberately parallel. Use it to visualize concepts from the main curriculum: barycentric coordinates, aliasing, ray intersections, path tracing, cotangent Laplacians, finite elements, pressure projection, NeRF volume rendering and more.

Editorial audit for Version 4. All 144 units were checked for prerequisite order, topic placement, reading relevance, programming specificity, research-test relevance, and cross-topic leakage. Each unit now has seven programming tasks (P1-P7), a concrete F1-F6 research project, and a self-contained G1-G6 practical. Major executable resources were rechecked against their official pages on 12 August 2026. The document avoids using SIGGRAPH program pages as implementation manuals: they are discovery indexes only; implementation tasks point to primary papers, books, or

official software documentation. A second pass also tightened the remaining vague advanced reading assignments and expanded the final programming portfolio into one concrete capstone per volume.

What Counts as Research-Level Understanding?

- You can derive the governing continuous model and explain the discretization used by a graphics algorithm.
- You can implement a reduced but correct version and build tests that isolate correctness, performance and numerical stability.
- You can distinguish representation error, discretization error, optimization failure, estimator variance, data insufficiency and implementation bugs.
- You can read a SIGGRAPH/TOG/SGP paper and reconstruct its assumptions, objective, algorithm, baselines and evaluation without relying on the authors' prose alone.
- You can reproduce at least one nontrivial result, design an ablation, and turn a repeated failure into a falsifiable research question.
- You can communicate the idea mathematically and visually, including a clean diagram or animation that preserves the actual invariants of the problem.

Exercise design in this edition. The problem sets are deliberately heterogeneous. A typical unit contains a derivation, a hand-worked boundary case, a reference implementation, a controlled comparison, an adversarial stress test, a reading reconstruction, and a small research study. The point is not to accumulate solved exercises; it is to learn to predict, implement, falsify, diagnose, and communicate the behavior of the specific graphics method in that unit.

Programming Exercise Workflow and Repository Layout

Use one long-lived repository for the curriculum. A practical layout is ``src/`` for reusable algorithms, ``tests/`` for hand-oracle and regression cases, ``experiments/`` for one-command sweeps, ``configs/`` for machine-readable parameters, ``assets/`` for small public/test scenes, ``results/raw/`` for immutable CSV/JSON/logs, ``results/figures/`` for regenerated plots, and ``notes/`` for derivations and paper reconstructions. Do not manually edit reported numbers.

For each unit, P1-P2 establishes correctness; P3 and P7 establish failure detection; P4 and P6 establish research-grade comparison; P5 establishes reproducible evidence. Keep a slow/reference implementation whenever possible. An optimization is not accepted until it passes exactly the same oracle and regression suite as the reference path.

Recommended languages/tools by stage: Python/NumPy/SciPy for early numerical prototypes and plots; modern C++/Eigen plus graphics APIs for systems/real-time/geometry work; CUDA or the relevant GPU API when performance is the subject; Mitsuba/Dr.Jit for differentiable rendering; PyTorch for neural/generative units; Manim for Volume 17. These are defaults, not requirements: change tools only when the research question benefits.

Three Pacing Modes

Mode	Weekly time	Calendar	How to use
Deliberate	10 h	5-6 years	Preserve all implementation; reduce paper count in early volumes, not coding.
Standard	15-20 h	3.5-4.5 years	Recommended path in this document.
Accelerated	25-30 h	2.5-3 years	Assumes strong mathematics/programming and sustained project time.

The 18 Volumes

Vol.	Title	Modules	Duration	Role
1	Mathematical Foundations for Computer Graphics	9	16-20 weeks	Build the linear algebra, geometry, probability, analysis,

Vol.	Title	Modules	Duration	Role
				optimization, and numerical-method toolkit used throughout rendering, geometry processing, simulation, and inverse graphics.
2	Programming, Numerical Computing, and GPU Systems	7	12-16 weeks	Acquire the implementation fluency needed to turn graphics mathematics into reliable CPU/GPU systems.
3	Images, Sampling, Color, and Visual Perception	6	10-12 weeks	Understand the image as a sampled signal and the human visual system as the final observer.
4	Rasterization and Real-Time Rendering	8	12-16 weeks	Master the modern real-time graphics pipeline from triangle coverage to physically based shading and GPU effects.
5	Ray Tracing and Rendering Systems	6	12-14 weeks	Develop a renderer from first principles and understand the architecture of a production-quality ray tracer.
6	Physically Based Rendering and Light Transport	8	18-22 weeks	Reach a research-capable understanding of radiometry, BSDFs, Monte Carlo transport, participating media, and advanced path-space methods.
7	Geometric Modeling, CAGD, and Shape Representations	7	14-18 weeks	Learn the representations used to construct and edit shapes: meshes, splines, subdivision, implicit surfaces, SDFs, and constructive models.
8	Geometry Processing	9	18-22 weeks	Build the

Vol.	Title	Modules	Duration	Role
	and Discrete Differential Geometry			mathematical and algorithmic foundation for research on meshes and surfaces.
9	Animation, Kinematics, and Character Motion	7	12-16 weeks	Understand how motion is represented, authored, interpolated, constrained, and controlled before physics enters.
10	Physics-Based Simulation I: Particles, Rigid Bodies, Solids, Cloth, and Contact	8	18-22 weeks	Develop a principled simulation toolkit for deformable and rigid objects, including numerical stability and robust contact.
11	Physics-Based Simulation II: Fluids, Continuum Methods, MPM, and Multiphysics	8	18-24 weeks	Learn grid, particle, and hybrid continuum simulation methods used in modern graphics research.
12	Cameras, Optics, Computational Imaging, and Displays	7	14-18 weeks	Connect graphics with physical image formation, computational photography, inverse imaging, and immersive display systems.
13	Differentiable Rendering and Inverse Graphics	8	16-20 weeks	Learn to differentiate image formation, formulate inverse problems, and use rendering as a differentiable computational layer.
14	Neural Rendering and Neural Scene Representations	9	18-22 weeks	Understand radiance fields, Gaussian splatting, implicit neural representations, dynamic scenes, editing, and relighting as graphics representations.
15	Generative 3D/4D and Machine Learning for	8	18-24 weeks	Study generative models as tools for

Vol.	Title	Modules	Duration	Role
	Graphics			creating geometry, appearance, scenes, motion, and physically plausible visual content.
16	Visualization, Interaction, VR/AR, Fabrication, and Special Topics	7	12-18 weeks	Broaden from image synthesis to visual computing: communicating data, interactive systems, immersive interfaces, and graphics-to-physical pipelines.
17	Mathematical Animation and Visual Explanation	13	16-24 weeks, then ongoing	Learn mathematical animation from scratch as both a hobby and a way to sharpen geometric intuition and communicate research.
18	Research Apprenticeship, Paper Reproduction, and Open Problems	9	12-18 months overlapping Vols. 8-17	Convert broad knowledge into research ability: reading, reproduction, benchmarking, identifying gaps, and proposing defensible new work.

Dependency Structure

- Foundation block: Volumes 1-3, with Volume 17 in parallel from the first month.
- Classical graphics core: Volumes 4-6. Do not replace these with neural rendering; modern work becomes much clearer after the classical rendering equation, sampling, visibility and systems material.
- Geometry/animation/simulation: Volumes 7-11. Volume 8 is mathematically deep and Volume 10-11 are numerically deep.
- Imaging and inverse problems: Volumes 12-13.
- Neural/generative graphics: Volumes 14-15.
- Breadth and synthesis: Volumes 16-18. Volume 18 begins informally after Volume 6 and becomes the dominant mode at the end.

Four-Year Quarter-by-Quarter Master Schedule

Quarter	Study focus	Knowledge target	Required output
Year 1 Q1	Vol. 1 modules 1-5 + Vol. 2 module 1 + Vol. 17 modules 1-3	Linear algebra, transforms, calculus, probability, Python/Manim.	Graphics mathematics laboratory + first 60-90 second mathematical animation.
Year 1 Q2	Finish Vol. 1; Vol. 2 modules 2-4; Vol. 3 modules 1-3	Numerical methods, C++, robustness,	Aliasing/color laboratory; reproducible C++/Python

Quarter	Study focus	Knowledge target	Required output
		images/sampling/color.	utilities.
Year 1 Q3	Vol. 2 modules 5-7; Vol. 3 modules 4-6; Vol. 4 modules 1-4	GPU/shaders, perception, rasterization, textures and shadows.	Software rasterizer + first GPU real-time viewer.
Year 1 Q4	Vol. 4 modules 5-8; Vol. 5	Real-time PBR, temporal methods, hybrid rendering, ray tracing systems.	Real-time PBR renderer + BVH ray tracer.
Year 2 Q1	Vol. 6	Radiometry, BSDFs, path tracing, MIS, media, advanced transport.	Research-grade path tracer with convergence/variance analysis.
Year 2 Q2	Vol. 7 + Vol. 8 modules 1-3	CAGD, shape representations, DDG foundations and Laplacians.	Shape modeling sandbox + first geometry-processing operators.
Year 2 Q3	Finish Vol. 8; Vol. 9 modules 1-4	Parameterization, geodesics, deformation, topology; animation/IK/skinning.	Geometry workbench + animated articulated character.
Year 2 Q4	Finish Vol. 9; Vol. 10 modules 1-5	Motion, crowds, integration, particles, rigid bodies, collision/contact.	Motion/control project + rigid/cloth/contact simulator.
Year 3 Q1	Vol. 10 modules 6-8; Vol. 11 modules 1-4	FEM, shells, fracture; Eulerian and particle fluids.	Deformable FEM demo + smoke/FLIP/APIC comparison.
Year 3 Q2	Finish Vol. 11; Vol. 12	MPM, coupling, high-performance simulation; cameras/optics/imaging/displays.	Continuum simulation capstone + computational imaging notebook.
Year 3 Q3	Vol. 13	Differentiable rendering and inverse graphics.	Synthetic inverse-rendering benchmark; first full paper reproduction.
Year 3 Q4	Vol. 14 modules 1-5	NeRF, acceleration, 3DGS, surface representations.	NeRF vs 3DGS controlled benchmark.
Year 4 Q1	Finish Vol. 14; Vol. 15 modules 1-4	Relighting, dynamics, editing, compression; diffusion and 3D generation.	Editing/relighting experiment + generative 3D validity benchmark.
Year 4 Q2	Finish Vol. 15; Vol. 16	Materials, avatars, physics-aware generation; visualization/VR/fabrication breadth.	One specialization prototype + second paper reproduction.
Year 4 Q3	Vol. 18 modules 1-6; Vol. 17 capstone	Benchmarking, ablation, reproducible research; visual explanation.	Systematic failure study + 6-10 minute rigorous explainer.
Year 4 Q4	Vol. 18 modules 7-9	Literature map, writing, research problem selection and original extension.	SIGGRAPH-style research report, code release, presentation and next-year research agenda.

Specialization Branches After the Classical Core

Rendering / inverse graphics: Prioritize Vols. 6, 12-14, 18. Deepen Monte Carlo estimators, differentiable visibility, materials, spectral rendering and inverse problems.

Geometry processing / modeling: Prioritize Vols. 7-8, then 13/15 where geometry meets optimization and learning. Add SGP papers continuously.

Physics-based simulation: Prioritize Vols. 10-11 and numerical analysis; read SIGGRAPH/TOG simulation papers every week and reproduce canonical benchmarks.

Neural / generative graphics: Do Vols. 5-6 before Vols. 14-15; specialize in representations, 3D consistency, editing, dynamics, compression or physically constrained generation.

Computational imaging / displays: Prioritize Vols. 6, 12-13, then optics/display papers; strengthen Fourier optics and inverse-problem mathematics.

Mathematical visualization: Run Vol. 17 throughout; pair with whichever scientific specialization you choose and develop reusable visualization primitives for research communication.

Current Online Lecture Spine: Berkeley CS184 Spring 2026

Use CS184 as a broad lecture spine, but not as the only source. The Spring 2026 schedule is especially useful because it aligns naturally with the first eleven volumes.

Foundations / sampling: Lecture 3 Sampling and Aliasing; Lectures 4-5 Transforms

Real-time core: Lecture 2 Drawing Triangles; Lectures 4-5 Transforms; Lecture 6 Texture Mapping; Lectures 7-8 Rasterization Pipeline

Geometry: Lectures 9-11 Bezier Curves and Surfaces; Lecture 12 Mesh Representations and Geometry Processing

Ray tracing / PBR: Lecture 13 Ray Tracing and Acceleration Structures; Lectures 14-15 Radiometry, Photometry and Monte Carlo Integration; Lectures 16-18 Global Illumination and Path Tracing; Lecture 18 Material Modeling

Color: Lectures 19-20 Introduction to Color Science

Animation / simulation: Lectures 21-23 Introduction to Animation; Lecture 24 Introduction to Physical Simulation; Lecture 25 Fluid Simulation

VR: Lectures 26-27 Virtual Reality

Course page: <https://cs184.eecs.berkeley.edu/sp26/>

For each lecture: watch/read the slides after attempting the derivation or algorithm yourself; then solve the discussion worksheet when available. Use course projects as additional implementation prompts, but retain your own independent codebase.

Volume 1. Mathematical Foundations for Computer Graphics

Purpose. Build the linear algebra, geometry, probability, analysis, optimization, and numerical-method toolkit used throughout rendering, geometry processing, simulation, and inverse graphics.

Suggested duration. 16-20 weeks

Prerequisites. No graphics prerequisites. High-school algebra is enough to begin; calculus and programming may be learned concurrently.

Typical module pace. 1.8-2.2 weeks per module (9 modules total).

Primary references for this volume

- Marschner & Shirley, Fundamentals of Computer Graphics, 5e (primary).
- Trefethen & Bau, Numerical Linear Algebra (selected lectures).
- Nocedal & Wright, Numerical Optimization (selected chapters).
- Keenan Crane, DDG notes (for geometry bridge).

Online spine

Berkeley CS184/284A Spring 2026 course: <https://cs184.eecs.berkeley.edu/sp26/>

PBRT 4th edition - full online book: <https://pbr-book.org/4ed/contents>

Keenan Crane - Discrete Differential Geometry: <https://www.cs.cmu.edu/~kmc Crane/Projects/DDG/>

Canonical paper track

- Trefethen & Bau, Numerical Linear Algebra (selected lecture-style chapters rather than a paper).
- Shoemake (1985), Animating Rotation with Quaternion Curves.
- Cook, Porter & Carpenter (1984), Distributed Ray Tracing (for probability/sampling motivation).
- Kajiya (1986), The Rendering Equation (read after Monte Carlo foundations).

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Volume 1, Unit 1. Vectors, affine spaces, and coordinates

Recommended time. 1.8-2.2 weeks (about 27-44 focused hours depending on background).

What you are learning. Vectors versus points; bases, coordinates, affine combinations, barycentric coordinates, frames, dual vectors, normals and covectors.

A. Reading order

1. Read the primary assignment first: FCG 5e §§2.4, 2.8-2.9 and Chs. 6-7 (vectors, interpolation/triangles, linear algebra, transformations). Use Gortler, Foundations of 3D Computer Graphics, as a deeper optional treatment of affine/projective geometry.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Berkeley CS184 lectures on transforms; Cornell CS4620 transform material.
3. Keep the most useful executable/reference page open while coding: Fundamentals of Computer Graphics 5e companion site - <https://www.cs.cornell.edu/~srm/fcg5/>

B. Definitions, results and algorithms to know

- Linear independence, span, basis, affine subspace, dot/cross product, Gram matrix, barycentric coordinates
- why normals transform by inverse transpose
- Precision requirement: distinguish and justify direct linear solve vs area/volume formulas vs projected barycentric coordinates; vectors vs points vs covectors under transforms. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive barycentric coordinates for a simplex from an affine-coordinate system, then prove that the coordinates are invariant under any nonsingular affine map; separately derive the inverse-transpose rule for normals from preservation of tangent-plane orthogonality.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use nearly collinear/degenerate simplices and coordinate magnitudes spanning at least 10^8 ; test affine invariance and the stability of barycentric/frame conversion. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Derive barycentric coordinates three ways; implement robust 2D/3D coordinate-frame conversion; numerically test normal transformation under nonuniform scale. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For triangle $A=(0,0)$, $B=(2,0)$, $C=(0,1)$ and $P=(0.5,0.25)$, compute barycentric coordinates by two methods. Apply affine map $x \rightarrow [[2,1],[0,3]]x + (4,-2)$ and verify the barycentrics are unchanged; then transform tangent $(1,0)$ and normal $(0,1)$ correctly and check orthogonality. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use nearly collinear/degenerate simplices and coordinate magnitudes spanning at least 10^8 ; test affine invariance and the stability of barycentric/frame conversion. Save the input and config of each qualitatively different failure as a named regression case.

- P4 - Research-grade programming extension. Write a property-based test generator for random affine transforms and simplices; assert barycentric invariance and tangent-normal orthogonality, and automatically shrink failing cases.
- P5 - Machine-readable evidence. Export barycentric sum-to-one/reconstruction error, tangent-normal dot product, affine-invariance error, and condition number of the simplex solve to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: direct linear solve vs area/volume formulas vs projected barycentric coordinates; vectors vs points vs covectors under transforms. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use nearly collinear/degenerate simplices and coordinate magnitudes spanning at least 10^8 ; test affine invariance and the stability of barycentric/frame conversion." without hand-editing code between methods. Record barycentric sum-to-one/reconstruction error, tangent-normal dot product, affine-invariance error, and condition number of the simplex solve.
- P7 - Deliberate bug and regression drill. Intentionally transform normals with the model matrix instead of the inverse transpose under nonuniform scale. Before running, predict the first observable failure in barycentric sum-to-one/reconstruction error, tangent-normal dot product, affine-invariance error, and condition number of the simplex solve. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive barycentric coordinates for a simplex from an affine-coordinate system, then prove that the coordinates are invariant under any nonsingular affine map; separately derive the inverse-transpose rule for normals from preservation of tangent-plane orthogonality. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For triangle $A=(0,0)$, $B=(2,0)$, $C=(0,1)$ and $P=(0.5,0.25)$, compute barycentric coordinates by two methods. Apply affine map $x \rightarrow [[2,1],[0,3]]x + (4,-2)$ and verify the barycentrics are unchanged; then transform tangent $(1,0)$ and normal $(0,1)$ correctly and check orthogonality. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Derive barycentric coordinates three ways; implement robust 2D/3D coordinate-frame conversion; numerically test normal transformation under nonuniform scale. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Write a property-based test generator for random affine transforms and simplices; assert barycentric invariance and tangent-normal orthogonality, and automatically shrink failing cases. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare direct linear solve vs area/volume formulas vs projected barycentric coordinates; vectors vs points vs covectors under transforms. Use the same analytic input, numerical precision and scale; for sampling/optimization comparisons, match sample or iteration budget and stopping criteria. Record barycentric sum-to-one/reconstruction error, tangent-normal dot product, affine-invariance error, and condition number of the simplex solve. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use nearly collinear/degenerate simplices and coordinate magnitudes spanning at least 10^8 ; test affine invariance and the stability of barycentric/frame conversion. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG 5e §§2.4, 2.8-2.9 and Chs. 6-7 (vectors, interpolation/triangles, linear algebra, transformations). Use Gortler, Foundations of 3D Computer Graphics, as a deeper optional treatment of affine/projective geometry. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Vectors, affine spaces, and coordinates. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among direct linear solve vs area/volume formulas vs projected barycentric coordinates; vectors vs points vs covectors under transforms. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using barycentric sum-to-one/reconstruction error, tangent-normal dot product, affine-invariance error, and condition

number of the simplex solve. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among direct linear solve vs area/volume formulas vs projected barycentric coordinates; vectors vs points vs covectors under transforms is reproducible across at least three controlled parameter regimes and agrees with an analytic conditioning/error/stability prediction expressed in barycentric sum-to-one/reconstruction error, tangent-normal dot product, affine-invariance error, and condition number of the simplex solve; isolate numerical precision from mathematical failure.

F. Concrete research project - Vectors, affine spaces, and coordinates

F1. Falsifiable question. Under the controlled failure regime "Use nearly collinear/degenerate simplices and coordinate magnitudes spanning at least 10^8 ; test affine invariance and the stability of barycentric/frame conversion.", which alternative in direct linear solve vs area/volume formulas vs projected barycentric coordinates; vectors vs points vs covectors under transforms is preferable, and is the change explained by conditioning, scaling, approximation error, and the invariants of the mathematical formulation? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Write a property-based test generator for random affine transforms and simplices; assert barycentric invariance and tangent-normal orthogonality, and automatically shrink failing cases. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use double precision for the reference path; include a hand-solvable case, then at least 6 logarithmically or geometrically spaced conditioning/scale levels. Where feasible, add an mpmath/high-precision oracle for the two hardest cases. The central adversarial condition is: Use nearly collinear/degenerate simplices and coordinate magnitudes spanning at least 10^8 ; test affine invariance and the stability of barycentric/frame conversion.

F4. Required evidence. Measure barycentric sum-to-one/reconstruction error, tangent-normal dot product, affine-invariance error, and condition number of the simplex solve. Deliver 'oracle.md', 'sweep.csv', one error/conditioning plot, and a minimal 'tests/' case reproducing the first failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Vectors, affine spaces, and coordinates. If a stable failure/crossover is found among direct linear solve vs area/volume formulas vs projected barycentric coordinates; vectors vs points vs covectors under transforms, implement one adaptive or corrected variant that directly targets conditioning, scaling, approximation error, and the invariants of the mathematical formulation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Vectors, affine spaces, and coordinates. Using the measured barycentric sum-to-one/reconstruction error, tangent-normal dot product, affine-invariance error, and condition number of the simplex solve, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Vectors, affine spaces, and coordinates

- G1. Closed-book derivation/oracle. Derive barycentric coordinates for a simplex from an affine-coordinate system, then prove that the coordinates are invariant under any nonsingular affine map; separately derive the inverse-transpose rule for normals from preservation of tangent-plane orthogonality. Then solve this exact micro-case from scratch without opening your implementation: For triangle $A=(0,0)$, $B=(2,0)$, $C=(0,1)$ and $P=(0.5,0.25)$, compute barycentric coordinates by two methods. Apply affine map $x \rightarrow [[2,1],[0,3]]x+(4,-2)$ and verify the barycentrics are unchanged; then transform tangent $(1,0)$ and normal $(0,1)$ correctly and check orthogonality. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Derive barycentric coordinates three ways; implement robust 2D/3D coordinate-frame conversion; numerically test normal transformation under nonuniform scale. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally transform normals with the model matrix instead of the inverse transpose under nonuniform scale. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.

- G4. Transfer experiment. On an input/configuration not used during development, execute: Write a property-based test generator for random affine transforms and simplices; assert barycentric invariance and tangent-normal orthogonality, and automatically shrink failing cases. Then apply this stress without retuning: Use nearly collinear/degenerate simplices and coordinate magnitudes spanning at least 10^8 ; test affine invariance and the stability of barycentric/frame conversion. Report barycentric sum-to-one/reconstruction error, tangent-normal dot product, affine-invariance error, and condition number of the simplex solve and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among direct linear solve vs area/volume formulas vs projected barycentric coordinates; vectors vs points vs covectors under transforms for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured barycentric sum-to-one/reconstruction error, tangent-normal dot product, affine-invariance error, and condition number of the simplex solve; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Vectors, affine spaces, and coordinates. Using barycentric sum-to-one/reconstruction error, tangent-normal dot product, affine-invariance error, and condition number of the simplex solve, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 1, Unit 2. Matrices, eigenproblems, and decompositions

Recommended time. 1.8-2.2 weeks (about 27-44 focused hours depending on background).

What you are learning. Matrix maps, determinants, eigenpairs, SVD, polar decomposition, QR, least squares, conditioning, sparse matrices.

A. Reading order

1. Read the primary assignment first: FCG Ch. 6; numerical linear algebra supplement: Trefethen & Bau, selected lectures 1-11, 24-31.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: MIT/OpenCourseWare-style linear algebra lectures as needed; Eigen documentation for implementation.
3. Keep the most useful executable/reference page open while coding: Fundamentals of Computer Graphics 5e companion site - <https://www.cs.cornell.edu/~srm/fcg5/>

B. Definitions, results and algorithms to know

- Spectral theorem for symmetric matrices
- normal equations
- pseudoinverse
- SVD as best rank-k approximation
- polar decomposition $R S$
- Precision requirement: distinguish and justify normal equations vs Householder QR vs SVD; direct polar decomposition vs SVD-based polar decomposition. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the normal equations for least squares and show why their condition number is squared; then derive the SVD solution and explain, using singular directions, why it is more reliable near rank deficiency.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Create matrices with clustered singular values and condition numbers from 10^2 to 10^{12} ; compare normal equations, QR, SVD, and polar decomposition errors. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement least squares with normal equations and QR; compare conditioning; implement PCA on point clouds; implement polar decomposition and use it to extract rotation. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Solve $Ax=b$ for $A=\text{diag}(1,10^{-3},10^{-6})$ after rotating A by known orthogonal matrices. Compute the condition number, compare

normal-equation and SVD reasoning on paper, and predict which component loses accuracy first. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Create matrices with clustered singular values and condition numbers from 10^2 to 10^{12} ; compare normal equations, QR, SVD, and polar decomposition errors. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Generate matrices with prescribed singular values, then plot forward solution error and residual versus condition number for normal equations, QR, and SVD.
- P5 - Machine-readable evidence. Export relative solution error, residual norm, orthogonality defect, singular-value error, and runtime versus prescribed condition number to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: normal equations vs Householder QR vs SVD; direct polar decomposition vs SVD-based polar decomposition. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Create matrices with clustered singular values and condition numbers from 10^2 to 10^{12} ; compare normal equations, QR, SVD, and polar decomposition errors." without hand-editing code between methods. Record relative solution error, residual norm, orthogonality defect, singular-value error, and runtime versus prescribed condition number.
- P7 - Deliberate bug and regression drill. Intentionally solve least squares through the normal equations in a near-rank-deficient case and silently compare only the residual, not forward error. Before running, predict the first observable failure in relative solution error, residual norm, orthogonality defect, singular-value error, and runtime versus prescribed condition number. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the normal equations for least squares and show why their condition number is squared; then derive the SVD solution and explain, using singular directions, why it is more reliable near rank deficiency. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. Solve $Ax=b$ for $A=\text{diag}(1,10^{-3},10^{-6})$ after rotating A by known orthogonal matrices. Compute the condition number, compare normal-equation and SVD reasoning on paper, and predict which component loses accuracy first. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Implement least squares with normal equations and QR; compare conditioning; implement PCA on point clouds. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Generate matrices with prescribed singular values, then plot forward solution error and residual versus condition number for normal equations, QR, and SVD. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare normal equations vs Householder QR vs SVD; direct polar decomposition vs SVD-based polar decomposition. Use the same analytic input, numerical precision and scale; for sampling/optimization comparisons, match sample or iteration budget and stopping criteria. Record relative solution error, residual norm, orthogonality defect, singular-value error, and runtime versus prescribed condition number. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Create matrices with clustered singular values and condition numbers from 10^2 to 10^{12} ; compare normal equations, QR, SVD, and polar decomposition errors. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: FCG Ch. 6; numerical linear algebra supplement: Trefethen & Bau, selected lectures 1-11, 24-31. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Matrices, eigenproblems, and decompositions. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among normal equations vs Householder QR vs SVD; direct polar decomposition vs SVD-based polar decomposition. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using relative solution error, residual norm, orthogonality defect, singular-value error, and runtime versus prescribed condition number.

Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among normal equations vs Householder QR vs SVD; direct polar decomposition vs SVD-based polar decomposition is reproducible across at least three controlled parameter regimes and agrees with an analytic conditioning/error/stability prediction expressed in relative solution error, residual norm, orthogonality defect, singular-value error, and runtime versus prescribed condition number; isolate numerical precision from mathematical failure.

F. Concrete research project - Matrices, eigenproblems, and decompositions

F1. Falsifiable question. Under the controlled failure regime "Create matrices with clustered singular values and condition numbers from 10^2 to 10^{12} ; compare normal equations, QR, SVD, and polar decomposition errors.", which alternative in normal equations vs Householder QR vs SVD; direct polar decomposition vs SVD-based polar decomposition is preferable, and is the change explained by conditioning, local linearization, and loss/parameter scaling? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Generate matrices with prescribed singular values, then plot forward solution error and residual versus condition number for normal equations, QR, and SVD. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use double precision for the reference path; include a hand-solvable case, then at least 6 logarithmically or geometrically spaced conditioning/scale levels. Where feasible, add an mpmath/high-precision oracle for the two hardest cases. The central adversarial condition is: Create matrices with clustered singular values and condition numbers from 10^2 to 10^{12} ; compare normal equations, QR, SVD, and polar decomposition errors.

F4. Required evidence. Measure relative solution error, residual norm, orthogonality defect, singular-value error, and runtime versus prescribed condition number. Deliver `oracle.md`, `sweep.csv`, one error/conditioning plot, and a minimal `tests/` case reproducing the first failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Matrices, eigenproblems, and decompositions. If a stable failure/crossover is found among normal equations vs Householder QR vs SVD; direct polar decomposition vs SVD-based polar decomposition, implement one adaptive or corrected variant that directly targets conditioning, local linearization, and loss/parameter scaling; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Matrices, eigenproblems, and decompositions. Using the measured relative solution error, residual norm, orthogonality defect, singular-value error, and runtime versus prescribed condition number, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Matrices, eigenproblems, and decompositions

- G1. Closed-book derivation/oracle. Derive the normal equations for least squares and show why their condition number is squared; then derive the SVD solution and explain, using singular directions, why it is more reliable near rank deficiency. Then solve this exact micro-case from scratch without opening your implementation: Solve $Ax=b$ for $A=\text{diag}(1,10^{-3},10^{-6})$ after rotating A by known orthogonal matrices. Compute the condition number, compare normal-equation and SVD reasoning on paper, and predict which component loses accuracy first. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement least squares with normal equations and QR; compare conditioning; implement PCA on point clouds; implement polar decomposition and use it to extract rotation. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally solve least squares through the normal equations in a near-rank-deficient case and silently compare only the residual, not forward error. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Generate matrices with prescribed singular values, then plot forward solution error and residual versus condition number for normal

equations, QR, and SVD. Then apply this stress without retuning: Create matrices with clustered singular values and condition numbers from 10^2 to 10^{12} ; compare normal equations, QR, SVD, and polar decomposition errors. Report relative solution error, residual norm, orthogonality defect, singular-value error, and runtime versus prescribed condition number and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method-selection defense. Using your own measured results, choose among normal equations vs Householder QR vs SVD; direct polar decomposition vs SVD-based polar decomposition for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured relative solution error, residual norm, orthogonality defect, singular-value error, and runtime versus prescribed condition number; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Matrices, eigenproblems, and decompositions. Using relative solution error, residual norm, orthogonality defect, singular-value error, and runtime versus prescribed condition number, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 1, Unit 3. Rigid, affine, projective geometry

Recommended time. 1.8-2.2 weeks (about 27-44 focused hours depending on background).

What you are learning. Homogeneous coordinates, rigid motions, quaternions, projective transformations, camera matrices and perspective division.

A. Reading order

1. Read the primary assignment first: FCG Ch. 7-8; Real-Time Rendering (RTR) 4e Ch. 4; optional multiple-view geometry notes later in Vol. 12.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 transforms/viewing lectures.
3. Keep the most useful executable/reference page open while coding: Fundamentals of Computer Graphics 5e companion site - <https://www.cs.cornell.edu/~srm/fcg5/>

B. Definitions, results and algorithms to know

- $SE(3)$, $SO(3)$, affine transform, projective equivalence
- quaternion composition and slerp
- projective camera model
- Precision requirement: distinguish and justify Euler angles vs rotation matrices vs quaternions; affine interpolation vs projectively correct constructions. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive homogeneous-coordinate representations of rigid, affine, and perspective transformations; derive the perspective divide and the location of vanishing points, and compare quaternion and matrix composition for rotations.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Compose hundreds of transforms including near- 180° rotations and extreme translations/scales; check drift, handedness mistakes, and interpolation behavior. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement quaternion slerp; build look-at and perspective matrices from first principles; derive vanishing points and visualize projective transformations. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Project the eight vertices of a unit cube with a 60-degree pinhole camera, then apply a 2D homography to one face. Locate a vanishing point for two parallel 3D edges and show explicitly why affine interpolation after perspective divide is wrong. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Compose hundreds of transforms including near-180° rotations and extreme translations/scales; check drift, handedness mistakes, and interpolation behavior. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build an interactive projective-geometry notebook that drags 3D points/lines and shows vanishing points, homogeneous coordinates, and decomposition drift after repeated transform composition.
- P5 - Machine-readable evidence. Export orthogonality/determinant drift, reprojection error, angular interpolation speed, and vanishing-point error to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: Euler angles vs rotation matrices vs quaternions; affine interpolation vs projectively correct constructions. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Compose hundreds of transforms including near-180° rotations and extreme translations/scales; check drift, handedness mistakes, and interpolation behavior." without hand-editing code between methods. Record orthogonality/determinant drift, reprojection error, angular interpolation speed, and vanishing-point error.
- P7 - Deliberate bug and regression drill. Intentionally perform the perspective divide before clipping and allow quaternion interpolation to cross the antipodal long arc. Before running, predict the first observable failure in orthogonality/determinant drift, reprojection error, angular interpolation speed, and vanishing-point error. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive homogeneous-coordinate representations of rigid, affine, and perspective transformations; derive the perspective divide and the location of vanishing points, and compare quaternion and matrix composition for rotations. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Project the eight vertices of a unit cube with a 60-degree pinhole camera, then apply a 2D homography to one face. Locate a vanishing point for two parallel 3D edges and show explicitly why affine interpolation after perspective divide is wrong. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement quaternion slerp; build look-at and perspective matrices from first principles; derive vanishing points and visualize projective transformations. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build an interactive projective-geometry notebook that drags 3D points/lines and shows vanishing points, homogeneous coordinates, and decomposition drift after repeated transform composition. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare Euler angles vs rotation matrices vs quaternions; affine interpolation vs projectively correct constructions. Use the same analytic input, numerical precision and scale; for sampling/optimization comparisons, match sample or iteration budget and stopping criteria. Record orthogonality/determinant drift, reprojection error, angular interpolation speed, and vanishing-point error. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Compose hundreds of transforms including near-180° rotations and extreme translations/scales; check drift, handedness mistakes, and interpolation behavior. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 7-8; Real-Time Rendering (RTR) 4e Ch. 4; optional multiple-view geometry notes later in Vol. 12. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Rigid, affine, projective geometry. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among Euler angles vs rotation matrices vs quaternions; affine interpolation vs projectively correct constructions. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using orthogonality/determinant drift, reprojection error, angular interpolation speed, and vanishing-point error.

Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among Euler angles vs rotation matrices vs quaternions; affine interpolation vs projectively correct constructions is reproducible across at least three controlled parameter regimes and agrees with an analytic conditioning/error/stability prediction expressed in orthogonality/determinant drift, reprojection error, angular interpolation speed, and vanishing-point error; isolate numerical precision from mathematical failure.

F. Concrete research project - Rigid, affine, projective geometry

F1. Falsifiable question. Under the controlled failure regime "Compose hundreds of transforms including near-180° rotations and extreme translations/scales; check drift, handedness mistakes, and interpolation behavior.", which alternative in Euler angles vs rotation matrices vs quaternions; affine interpolation vs projectively correct constructions is preferable, and is the change explained by conditioning, scaling, approximation error, and the invariants of the mathematical formulation? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build an interactive projective-geometry notebook that drags 3D points/lines and shows vanishing points, homogeneous coordinates, and decomposition drift after repeated transform composition. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use double precision for the reference path; include a hand-solvable case, then at least 6 logarithmically or geometrically spaced conditioning/scale levels. Where feasible, add an mpmath/high-precision oracle for the two hardest cases. The central adversarial condition is: Compose hundreds of transforms including near-180° rotations and extreme translations/scales; check drift, handedness mistakes, and interpolation behavior.

F4. Required evidence. Measure orthogonality/determinant drift, reprojection error, angular interpolation speed, and vanishing-point error. Deliver `oracle.md`, `sweep.csv`, one error/conditioning plot, and a minimal `tests/` case reproducing the first failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Rigid, affine, projective geometry. If a stable failure/crossover is found among Euler angles vs rotation matrices vs quaternions; affine interpolation vs projectively correct constructions, implement one adaptive or corrected variant that directly targets conditioning, scaling, approximation error, and the invariants of the mathematical formulation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Rigid, affine, projective geometry. Using the measured orthogonality/determinant drift, reprojection error, angular interpolation speed, and vanishing-point error, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Rigid, affine, projective geometry

- G1. Closed-book derivation/oracle. Derive homogeneous-coordinate representations of rigid, affine, and perspective transformations; derive the perspective divide and the location of vanishing points, and compare quaternion and matrix composition for rotations. Then solve this exact micro-case from scratch without opening your implementation: Project the eight vertices of a unit cube with a 60-degree pinhole camera, then apply a 2D homography to one face. Locate a vanishing point for two parallel 3D edges and show explicitly why affine interpolation after perspective divide is wrong. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement quaternion slerp; build look-at and perspective matrices from first principles; derive vanishing points and visualize projective transformations. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally perform the perspective divide before clipping and allow quaternion interpolation to cross the antipodal long arc. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.

- G4. Transfer experiment. On an input/configuration not used during development, execute: Build an interactive projective-geometry notebook that drags 3D points/lines and shows vanishing points, homogeneous coordinates, and decomposition drift after repeated transform composition. Then apply this stress without retuning: Compose hundreds of transforms including near-180° rotations and extreme translations/scales; check drift, handedness mistakes, and interpolation behavior. Report orthogonality/determinant drift, reprojection error, angular interpolation speed, and vanishing-point error and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among Euler angles vs rotation matrices vs quaternions; affine interpolation vs projectively correct constructions for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured orthogonality/determinant drift, reprojection error, angular interpolation speed, and vanishing-point error; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Rigid, affine, projective geometry. Using orthogonality/determinant drift, reprojection error, angular interpolation speed, and vanishing-point error, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 1, Unit 4. Calculus and differential calculus

Recommended time. 1.8-2.2 weeks (about 27-44 focused hours depending on background).

What you are learning. Multivariable derivatives, Jacobians, gradients, Hessians, chain rule, implicit differentiation, constrained optimization.

A. Reading order

1. Read the primary assignment first: Use a standard multivariable-calculus reference for gradients, Jacobians, chain rule, implicit differentiation, and constrained derivatives; use FCG §2.5 only for basic integration. Revisit these derivatives concretely in Vol. 13 with Dr.Jit/Mitsuba.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Autodiff tutorials in PyTorch/JAX; Dr.Jit quickstart in Vol. 13.
3. Keep the most useful executable/reference page open while coding: Fundamentals of Computer Graphics 5e companion site - <https://www.cs.cornell.edu/~srm/fcg5/>

B. Definitions, results and algorithms to know

- Directional derivative
- Jacobian-vector and vector-Jacobian products
- implicit function theorem intuition
- Lagrange multipliers
- Precision requirement: distinguish and justify analytic derivatives vs central finite differences vs automatic differentiation near smooth and near-singular configurations. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive Jacobian-vector and vector-Jacobian products for a composed graphics map, then differentiate a ray-sphere intersection and identify where the derivative becomes ill-conditioned or discontinuous.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Gradient-check a near-tangent ray/surface intersection and a normalized vector near zero norm; compare analytic, autodiff, and finite differences. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Differentiate ray-sphere intersection analytically; compare finite differences/autodiff; derive gradients of signed distance to primitive shapes. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Differentiate $y=x/||x||$ at $x=(3,4)$ and verify the Jacobian by central differences for $h=10^{-1}, \dots, 10^{-8}$. Separately differentiate the positive ray-sphere intersection distance with respect to sphere radius in a non-tangent configuration. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Gradient-check a near-tangent ray/surface intersection and a normalized vector near zero norm; compare analytic, autodiff, and finite differences. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement a gradient-check harness that sweeps finite-difference step size and reports truncation/roundoff regimes for ray intersections and normalized-vector derivatives.
- P5 - Machine-readable evidence. Export relative gradient error versus finite-difference step, derivative magnitude near tangency/singularity, and autodiff/analytic runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: analytic derivatives vs central finite differences vs automatic differentiation near smooth and near-singular configurations. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Gradient-check a near-tangent ray/surface intersection and a normalized vector near zero norm; compare analytic, autodiff, and finite differences." without hand-editing code between methods. Record relative gradient error versus finite-difference step, derivative magnitude near tangency/singularity, and autodiff/analytic runtime.
- P7 - Deliberate bug and regression drill. Intentionally differentiate a normalized vector as if normalization were constant, then choose one fixed finite-difference step for all scales. Before running, predict the first observable failure in relative gradient error versus finite-difference step, derivative magnitude near tangency/singularity, and autodiff/analytic runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive Jacobian-vector and vector-Jacobian products for a composed graphics map, then differentiate a ray-sphere intersection and identify where the derivative becomes ill-conditioned or discontinuous. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Differentiate $y = x / |x|$ at $x = (3, 4)$ and verify the Jacobian by central differences for $h = 10^{-1}, \dots, 10^{-8}$. Separately differentiate the positive ray-sphere intersection distance with respect to sphere radius in a non-tangent configuration. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Differentiate ray-sphere intersection analytically; compare finite differences/autodiff; derive gradients of signed distance to primitive shapes. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement a gradient-check harness that sweeps finite-difference step size and reports truncation/roundoff regimes for ray intersections and normalized-vector derivatives. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare analytic derivatives vs central finite differences vs automatic differentiation near smooth and near-singular configurations. Use the same analytic input, numerical precision and scale; for sampling/optimization comparisons, match sample or iteration budget and stopping criteria. Record relative gradient error versus finite-difference step, derivative magnitude near tangency/singularity, and autodiff/analytic runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Gradient-check a near-tangent ray/surface intersection and a normalized vector near zero norm; compare analytic, autodiff, and finite differences. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Use a standard multivariable-calculus reference for gradients, Jacobians, chain rule, implicit differentiation, and constrained derivatives; use FCG §2.5 only for basic integration. Revisit these derivatives concretely in Vol. 13 with Dr.Jit/Mitsuba. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Calculus and differential calculus. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among analytic derivatives vs central finite differences vs automatic differentiation near smooth and near-singular configurations. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using relative gradient error versus finite-difference step, derivative magnitude near tangency/singularity, and

autodiff/analytic runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among analytic derivatives vs central finite differences vs automatic differentiation near smooth and near-singular configurations is reproducible across at least three controlled parameter regimes and agrees with an analytic conditioning/error/stability prediction expressed in relative gradient error versus finite-difference step, derivative magnitude near tangency/singularity, and autodiff/analytic runtime; isolate numerical precision from mathematical failure.

F. Concrete research project - Calculus and differential calculus

- F1. Falsifiable question. Under the controlled failure regime "Gradient-check a near-tangent ray/surface intersection and a normalized vector near zero norm; compare analytic, autodiff, and finite differences.", which alternative in analytic derivatives vs central finite differences vs automatic differentiation near smooth and near-singular configurations is preferable, and is the change explained by conditioning, local linearization, and loss/parameter scaling? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement a gradient-check harness that sweeps finite-difference step size and reports truncation/roundoff regimes for ray intersections and normalized-vector derivatives. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use double precision for the reference path; include a hand-solvable case, then at least 6 logarithmically or geometrically spaced conditioning/scale levels. Where feasible, add an mpmath/high-precision oracle for the two hardest cases. The central adversarial condition is: Gradient-check a near-tangent ray/surface intersection and a normalized vector near zero norm; compare analytic, autodiff, and finite differences.
- F4. Required evidence. Measure relative gradient error versus finite-difference step, derivative magnitude near tangency/singularity, and autodiff/analytic runtime. Deliver `oracle.md`, `sweep.csv`, one error/conditioning plot, and a minimal `tests/` case reproducing the first failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Calculus and differential calculus. If a stable failure/crossover is found among analytic derivatives vs central finite differences vs automatic differentiation near smooth and near-singular configurations, implement one adaptive or corrected variant that directly targets conditioning, local linearization, and loss/parameter scaling; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Calculus and differential calculus. Using the measured relative gradient error versus finite-difference step, derivative magnitude near tangency/singularity, and autodiff/analytic runtime, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Calculus and differential calculus

- G1. Closed-book derivation/oracle. Derive Jacobian-vector and vector-Jacobian products for a composed graphics map, then differentiate a ray-sphere intersection and identify where the derivative becomes ill-conditioned or discontinuous. Then solve this exact micro-case from scratch without opening your implementation: Differentiate $y = x / \|x\|$ at $x = (3, 4)$ and verify the Jacobian by central differences for $h = 10^{-1}, \dots, 10^{-8}$. Separately differentiate the positive ray-sphere intersection distance with respect to sphere radius in a non-tangent configuration. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Differentiate ray-sphere intersection analytically; compare finite differences/autodiff; derive gradients of signed distance to primitive shapes. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally differentiate a normalized vector as if normalization were constant, then choose one fixed finite-difference step for all scales. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.

- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement a gradient-check harness that sweeps finite-difference step size and reports truncation/roundoff regimes for ray intersections and normalized-vector derivatives. Then apply this stress without retuning: Gradient-check a near-tangent ray/surface intersection and a normalized vector near zero norm; compare analytic, autodiff, and finite differences. Report relative gradient error versus finite-difference step, derivative magnitude near tangency/singularity, and autodiff/analytic runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among analytic derivatives vs central finite differences vs automatic differentiation near smooth and near-singular configurations for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured relative gradient error versus finite-difference step, derivative magnitude near tangency/singularity, and autodiff/analytic runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Calculus and differential calculus. Using relative gradient error versus finite-difference step, derivative magnitude near tangency/singularity, and autodiff/analytic runtime, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 1, Unit 5. Probability, random variables, and Monte Carlo

Recommended time. 1.8-2.2 weeks (about 27-44 focused hours depending on background).

What you are learning. Discrete/continuous probability, expectations, transformations, importance sampling, variance, stratification, quasi-Monte Carlo.

A. Reading order

1. Read the primary assignment first: FCG Ch. 2.10-2.12 and Ch. 13; PBRT 4e Ch. 2 and Ch. 8.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 Monte Carlo lectures; PBRT online chapter examples.
3. Keep the most useful executable/reference page open while coding: Fundamentals of Computer Graphics 5e companion site - <https://www.cs.cornell.edu/~srm/fcg5/>

B. Definitions, results and algorithms to know

- Law of the unconscious statistician
- change of variables
- unbiasedness
- variance
- importance sampling estimator
- MIS intuition
- Precision requirement: distinguish and justify uniform, stratified, low-discrepancy, and importance sampling under matched sample counts. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Starting from expectation, derive an unbiased Monte Carlo estimator, its variance, and the change-of-variables/importance-sampling estimator; identify the exact support condition required to avoid invalid estimates.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use both a sharply peaked integrand and a discontinuous integrand; include a deliberately mismatched proposal and inspect heavy-tailed estimator errors. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Write Monte Carlo estimators for 1D/2D integrals; compare uniform, stratified, importance, Halton, Sobol; plot convergence and variance. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Estimate $\int_0^1 x^2 dx$ with $N=4$ samples using uniform random, four-bin stratified, and importance sampling with $p(x)=2x$. Compute each

estimator and its sample variance by hand before running code. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Use both a sharply peaked integrand and a discontinuous integrand; include a deliberately mismatched proposal and inspect heavy-tailed estimator errors. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement a reusable estimator interface that records sample values, PDFs, effective sample size, confidence intervals, and contribution histograms for each sampling strategy.
- P5 - Machine-readable evidence. Export bias, empirical variance/RMSE, confidence-interval coverage, effective sample size, and contribution-tail quantiles to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: uniform, stratified, low-discrepancy, and importance sampling under matched sample counts. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use both a sharply peaked integrand and a discontinuous integrand; include a deliberately mismatched proposal and inspect heavy-tailed estimator errors." without hand-editing code between methods. Record bias, empirical variance/RMSE, confidence-interval coverage, effective sample size, and contribution-tail quantiles.
- P7 - Deliberate bug and regression drill. Intentionally change the sampling distribution without dividing by the corresponding PDF/Jacobian. Before running, predict the first observable failure in bias, empirical variance/RMSE, confidence-interval coverage, effective sample size, and contribution-tail quantiles. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Starting from expectation, derive an unbiased Monte Carlo estimator, its variance, and the change-of-variables/importance-sampling estimator; identify the exact support condition required to avoid invalid estimates. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Estimate $\int_0^1 x^2 dx$ with $N=4$ samples using uniform random, four-bin stratified, and importance sampling with $p(x)=2x$. Compute each estimator and its sample variance by hand before running code. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Write Monte Carlo estimators for 1D/2D integrals; compare uniform, stratified, importance, Halton, Sobol; plot convergence and variance. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement a reusable estimator interface that records sample values, PDFs, effective sample size, confidence intervals, and contribution histograms for each sampling strategy. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare uniform, stratified, low-discrepancy, and importance sampling under matched sample counts. Use the same analytic input, numerical precision and scale; for sampling/optimization comparisons, match sample or iteration budget and stopping criteria. Record bias, empirical variance/RMSE, confidence-interval coverage, effective sample size, and contribution-tail quantiles. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use both a sharply peaked integrand and a discontinuous integrand; include a deliberately mismatched proposal and inspect heavy-tailed estimator errors. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 2.10-2.12 and Ch. 13; PBRT 4e Ch. 2 and Ch. 8. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Probability, random variables, and Monte Carlo. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among uniform, stratified, low-discrepancy, and importance sampling under matched sample counts. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using bias, empirical variance/RMSE, confidence-interval coverage, effective sample size, and contribution-tail quantiles. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results,

and a final decision using this criterion: Advance only if the ranking among uniform, stratified, low-discrepancy, and importance sampling under matched sample counts is reproducible across at least three controlled parameter regimes and agrees with an analytic conditioning/error/stability prediction expressed in bias, empirical variance/RMSE, confidence-interval coverage, effective sample size, and contribution-tail quantiles; isolate numerical precision from mathematical failure.

F. Concrete research project - Probability, random variables, and Monte Carlo

F1. Falsifiable question. Under the controlled failure regime "Use both a sharply peaked integrand and a discontinuous integrand; include a deliberately mismatched proposal and inspect heavy-tailed estimator errors.", characterize the ranking or regime transition across uniform, stratified, low-discrepancy, and importance sampling under matched sample counts. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by sampling support/PDF correctness, estimator bias and variance, tail behavior, and confidence-interval validity. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement a reusable estimator interface that records sample values, PDFs, effective sample size, confidence intervals, and contribution histograms for each sampling strategy. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use double precision for the reference path; include a hand-solvable case, then at least 6 logarithmically or geometrically spaced conditioning/scale levels. Where feasible, add an mpmath/high-precision oracle for the two hardest cases. The central adversarial condition is: Use both a sharply peaked integrand and a discontinuous integrand; include a deliberately mismatched proposal and inspect heavy-tailed estimator errors.

F4. Required evidence. Measure bias, empirical variance/RMSE, confidence-interval coverage, effective sample size, and contribution-tail quantiles. Deliver `oracle.md`, `sweep.csv`, one error/conditioning plot, and a minimal `tests/` case reproducing the first failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Probability, random variables, and Monte Carlo. If a stable failure/crossover is found among uniform, stratified, low-discrepancy, and importance sampling under matched sample counts, implement one adaptive or corrected variant that directly targets sampling support/PDF correctness, estimator bias and variance, tail behavior, and confidence-interval validity; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Probability, random variables, and Monte Carlo. Using the measured bias, empirical variance/RMSE, confidence-interval coverage, effective sample size, and contribution-tail quantiles, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Probability, random variables, and Monte Carlo

- G1. Closed-book derivation/oracle. Starting from expectation, derive an unbiased Monte Carlo estimator, its variance, and the change-of-variables/importance-sampling estimator; identify the exact support condition required to avoid invalid estimates. Then solve this exact micro-case from scratch without opening your implementation: Estimate $\int_0^1 x^2 dx$ with $N=4$ samples using uniform random, four-bin stratified, and importance sampling with $p(x)=2x$. Compute each estimator and its sample variance by hand before running code. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Write Monte Carlo estimators for 1D/2D integrals; compare uniform, stratified, importance, Halton, Sobol; plot convergence and variance. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally change the sampling distribution without dividing by the corresponding PDF/Jacobian. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement a reusable estimator interface that records sample values, PDFs, effective sample size, confidence intervals, and contribution histograms for each sampling strategy. Then apply this stress without retuning: Use both a sharply peaked

integrand and a discontinuous integrand; include a deliberately mismatched proposal and inspect heavy-tailed estimator errors. Report bias, empirical variance/RMSE, confidence-interval coverage, effective sample size, and contribution-tail quantiles and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across uniform, stratified, low-discrepancy, and importance sampling under matched sample counts for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured bias, empirical variance/RMSE, confidence-interval coverage, effective sample size, and contribution-tail quantiles; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Probability, random variables, and Monte Carlo. Using bias, empirical variance/RMSE, confidence-interval coverage, effective sample size, and contribution-tail quantiles, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 1, Unit 6. Fourier analysis and sampling

Recommended time. 1.8-2.2 weeks (about 27-44 focused hours depending on background).

What you are learning. Convolution, Fourier transform, frequency domain, Nyquist-Shannon sampling, aliasing, reconstruction filters.

A. Reading order

1. Read the primary assignment first: FCG Ch. 10; PBRT Ch. 8.1 and 8.8; RTR Ch. 12 selections.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 sampling/aliasing lecture.
3. Keep the most useful executable/reference page open while coding: Fundamentals of Computer Graphics 5e companion site - <https://www.cs.cornell.edu/~srm/fcg5/>

B. Definitions, results and algorithms to know

- Convolution theorem
- sampling as multiplication by a comb
- spectral replication
- bandlimit
- reconstruction
- Precision requirement: distinguish and justify box, tent, Gaussian, Lanczos/sinc-like, and anisotropic reconstruction on signals with known spectra. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the sampling theorem for a band-limited 1D signal from spectral replication, then derive how convolution with a reconstruction filter modifies the spectrum and predict aliasing for a chirp or rotated checkerboard.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use rotated checkerboards and chirps under minification so different regions cross Nyquist at different rates; inspect spatial and Fourier-domain artifacts. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create an aliasing lab for 1D signals and checkerboards; implement box, tent, Gaussian and sinc-like reconstruction; compare spectra. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Sample $f(x) = \sin(2\pi 3x) + 0.5 \sin(\pi 7x)$ at 10, 16, and 32 samples per unit interval. Predict every aliased frequency and sketch the spectra before verifying them with a DFT. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use rotated checkerboards and chirps under minification so different regions cross Nyquist at different rates; inspect spatial and Fourier-domain artifacts. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create a 2D resampling workbench that shows the image and log-magnitude spectrum side by side while changing sampling rate, prefilter, and reconstruction kernel.

- P5 - Machine-readable evidence. Export alias energy outside the intended band, image RMSE to supersampling, edge/ringing overshoot, and samples/filter taps per pixel to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: box, tent, Gaussian, Lanczos/sinc-like, and anisotropic reconstruction on signals with known spectra. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use rotated checkerboards and chirps under minification so different regions cross Nyquist at different rates; inspect spatial and Fourier-domain artifacts." without hand-editing code between methods. Record alias energy outside the intended band, image RMSE to supersampling, edge/ringing overshoot, and samples/filter taps per pixel.
- P7 - Deliberate bug and regression drill. Intentionally downsample a high-frequency signal without a prefilter and then attribute the aliased energy to the reconstruction kernel. Before running, predict the first observable failure in alias energy outside the intended band, image RMSE to supersampling, edge/ringing overshoot, and samples/filter taps per pixel. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the sampling theorem for a band-limited 1D signal from spectral replication, then derive how convolution with a reconstruction filter modifies the spectrum and predict aliasing for a chirp or rotated checkerboard. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Sample $f(x) = \sin(2\pi 3x) + 0.5 \sin(2\pi 7x)$ at 10, 16, and 32 samples per unit interval. Predict every aliased frequency and sketch the spectra before verifying them with a DFT. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Create an aliasing lab for 1D signals and checkerboards; implement box, tent, Gaussian and sinc-like reconstruction; compare spectra. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create a 2D resampling workbench that shows the image and log-magnitude spectrum side by side while changing sampling rate, prefilter, and reconstruction kernel. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare box, tent, Gaussian, Lanczos/sinc-like, and anisotropic reconstruction on signals with known spectra. Use the same analytic input, numerical precision and scale; for sampling/optimization comparisons, match sample or iteration budget and stopping criteria. Record alias energy outside the intended band, image RMSE to supersampling, edge/ringing overshoot, and samples/filter taps per pixel. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use rotated checkerboards and chirps under minification so different regions cross Nyquist at different rates; inspect spatial and Fourier-domain artifacts. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 10; PBRT Ch. 8.1 and 8.8; RTR Ch. 12 selections. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Fourier analysis and sampling. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among box, tent, Gaussian, Lanczos/sinc-like, and anisotropic reconstruction on signals with known spectra. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using alias energy outside the intended band, image RMSE to supersampling, edge/ringing overshoot, and samples/filter taps per pixel. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among box, tent, Gaussian, Lanczos/sinc-like, and anisotropic reconstruction on signals with known spectra is reproducible across at least three controlled parameter regimes and agrees with an analytic conditioning/error/stability prediction expressed in alias energy outside the intended band, image RMSE to supersampling, edge/ringing overshoot, and samples/filter taps per pixel; isolate numerical precision from mathematical failure.

F. Concrete research project - Fourier analysis and sampling

F1. Falsifiable question. Under the controlled failure regime "Use rotated checkerboards and chirps under minification so different regions cross Nyquist at different rates; inspect spatial and Fourier-domain artifacts.", characterize the ranking or regime transition across box, tent, Gaussian, Lanczos/sinc-like, and anisotropic reconstruction on signals with known spectra. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by spectral support, prefiltering, aliasing, and reconstruction-kernel frequency response. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create a 2D resampling workbench that shows the image and log-magnitude spectrum side by side while changing sampling rate, prefilter, and reconstruction kernel. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use double precision for the reference path; include a hand-solvable case, then at least 6 logarithmically or geometrically spaced conditioning/scale levels. Where feasible, add an mpmath/high-precision oracle for the two hardest cases. The central adversarial condition is: Use rotated checkerboards and chirps under minification so different regions cross Nyquist at different rates; inspect spatial and Fourier-domain artifacts.

F4. Required evidence. Measure alias energy outside the intended band, image RMSE to supersampling, edge/ringing overshoot, and samples/filter taps per pixel. Deliver `oracle.md`, `sweep.csv`, one error/conditioning plot, and a minimal `tests/` case reproducing the first failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Fourier analysis and sampling. If a stable failure/crossover is found among box, tent, Gaussian, Lanczos/sinc-like, and anisotropic reconstruction on signals with known spectra, implement one adaptive or corrected variant that directly targets spectral support, prefiltering, aliasing, and reconstruction-kernel frequency response; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Fourier analysis and sampling. Using the measured alias energy outside the intended band, image RMSE to supersampling, edge/ringing overshoot, and samples/filter taps per pixel, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Fourier analysis and sampling

- G1. Closed-book derivation/oracle. Derive the sampling theorem for a band-limited 1D signal from spectral replication, then derive how convolution with a reconstruction filter modifies the spectrum and predict aliasing for a chirp or rotated checkerboard. Then solve this exact micro-case from scratch without opening your implementation: Sample $f(x) = \sin(2\pi 3x) + 0.5 \sin(2\pi 7x)$ at 10, 16, and 32 samples per unit interval. Predict every aliased frequency and sketch the spectra before verifying them with a DFT. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create an aliasing lab for 1D signals and checkerboards; implement box, tent, Gaussian and sinc-like reconstruction; compare spectra. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally downsample a high-frequency signal without a prefilter and then attribute the aliased energy to the reconstruction kernel. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create a 2D resampling workbench that shows the image and log-magnitude spectrum side by side while changing sampling rate, prefilter, and reconstruction kernel. Then apply this stress without retuning: Use rotated checkerboards and chirps under minification so different regions cross Nyquist at different rates; inspect spatial and Fourier-domain artifacts. Report alias energy outside the intended band, image RMSE to supersampling, edge/ringing overshoot, and samples/filter taps per pixel and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across box, tent, Gaussian, Lanczos/sinc-like, and anisotropic reconstruction on signals with known spectra for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained

case of your choice. Cite the measured alias energy outside the intended band, image RMSE to supersampling, edge/ringing overshoot, and samples/filter taps per pixel; state one counterexample that would change the diagnosis or ranking.

- G6. Pass condition for Fourier analysis and sampling. Using alias energy outside the intended band, image RMSE to supersampling, edge/ringing overshoot, and samples/filter taps per pixel, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 1, Unit 7. ODEs, PDEs, and variational principles

Recommended time. 1.8-2.2 weeks (about 27-44 focused hours depending on background).

What you are learning. Initial-value problems, explicit/implicit integration, stability, boundary-value problems, energy minimization, weak forms.

A. Reading order

1. Read the primary assignment first: Physics-Based Simulation online book, foundations/time integration sections; Bridson fluid notes later; Sifakis-Barbic FEM course later.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Stanford CS348C introductory simulation lectures; Physics-Based Simulation examples.
3. Keep the most useful executable/reference page open while coding: Fundamentals of Computer Graphics 5e companion site - <https://www.cs.cornell.edu/~srm/fcg5/>

B. Definitions, results and algorithms to know

- Forward/backward Euler
- symplectic Euler
- CFL intuition
- Euler-Lagrange equations
- weak form
- finite differences and FEM concept
- Precision requirement: distinguish and justify explicit Euler, symplectic Euler, Verlet/midpoint, and backward Euler across nonstiff and stiff regimes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive explicit Euler, backward Euler, and symplectic Euler for the harmonic oscillator; compute their update matrices/eigenvalues and use them to explain stability and long-time energy behavior.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Sweep timestep through a stability boundary on both a simple oscillator and a stiff system; run long enough to reveal energy drift and phase error. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement harmonic oscillator with four integrators; measure energy drift; derive Poisson equation from an energy; solve 1D Poisson by finite differences. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For $q''+q=0$ with $q(0)=1$, $v(0)=0$ and $dt=0.5$, take four steps of explicit Euler, symplectic Euler, and implicit Euler by hand. Track energy and compare with the exact solution. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Sweep timestep through a stability boundary on both a simple oscillator and a stiff system; run long enough to reveal energy drift and phase error. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Add an eigenvalue/stability-region plot for each integrator and verify that the observed oscillator/stiff-system behavior agrees with the update-matrix prediction.
- P5 - Machine-readable evidence. Export spectral radius/stability boundary, phase error, relative energy drift, Poisson residual, and convergence with timestep/grid size to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.

- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: explicit Euler, symplectic Euler, Verlet/midpoint, and backward Euler across nonstiff and stiff regimes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Sweep timestep through a stability boundary on both a simple oscillator and a stiff system; run long enough to reveal energy drift and phase error." without hand-editing code between methods. Record spectral radius/stability boundary, phase error, relative energy drift, Poisson residual, and convergence with timestep/grid size.
- P7 - Deliberate bug and regression drill. Intentionally use explicit Euler on the stiffest test at the same timestep as backward/symplectic methods and interpret instability as a modeling failure. Before running, predict the first observable failure in spectral radius/stability boundary, phase error, relative energy drift, Poisson residual, and convergence with timestep/grid size. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive explicit Euler, backward Euler, and symplectic Euler for the harmonic oscillator; compute their update matrices/eigenvalues and use them to explain stability and long-time energy behavior. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For $q''+q=0$ with $q(0)=1$, $v(0)=0$ and $dt=0.5$, take four steps of explicit Euler, symplectic Euler, and implicit Euler by hand. Track energy and compare with the exact solution. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement harmonic oscillator with four integrators; measure energy drift; derive Poisson equation from an energy. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Add an eigenvalue/stability-region plot for each integrator and verify that the observed oscillator/stiff-system behavior agrees with the update-matrix prediction. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare explicit Euler, symplectic Euler, Verlet/midpoint, and backward Euler across nonstiff and stiff regimes. Use the same analytic input, numerical precision and scale; for sampling/optimization comparisons, match sample or iteration budget and stopping criteria. Record spectral radius/stability boundary, phase error, relative energy drift, Poisson residual, and convergence with timestep/grid size. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Sweep timestep through a stability boundary on both a simple oscillator and a stiff system; run long enough to reveal energy drift and phase error. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Physics-Based Simulation online book, foundations/time integration sections; Bridson fluid notes later; Sifakis-Barbic FEM course later. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on ODEs, PDEs, and variational principles. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among explicit Euler, symplectic Euler, Verlet/midpoint, and backward Euler across nonstiff and stiff regimes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using spectral radius/stability boundary, phase error, relative energy drift, Poisson residual, and convergence with timestep/grid size. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among explicit Euler, symplectic Euler, Verlet/midpoint, and backward Euler across nonstiff and stiff regimes is reproducible across at least three controlled parameter regimes and agrees with an analytic conditioning/error/stability prediction expressed in spectral radius/stability boundary, phase error, relative energy drift, Poisson residual, and convergence with timestep/grid size; isolate numerical precision from mathematical failure.

F. Concrete research project - ODEs, PDEs, and variational principles

- F1. Falsifiable question. Under the controlled failure regime "Sweep timestep through a stability boundary on both a simple oscillator and a stiff system; run long enough to reveal energy drift and phase error.", characterize the ranking

or regime transition across explicit Euler, symplectic Euler, Verlet/midpoint, and backward Euler across nonstiff and stiff regimes. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by conditioning, scaling, approximation error, and the invariants of the mathematical formulation. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Add an eigenvalue/stability-region plot for each integrator and verify that the observed oscillator/stiff-system behavior agrees with the update-matrix prediction. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use double precision for the reference path; include a hand-solvable case, then at least 6 logarithmically or geometrically spaced conditioning/scale levels. Where feasible, add an mpmath/high-precision oracle for the two hardest cases. The central adversarial condition is: Sweep timestep through a stability boundary on both a simple oscillator and a stiff system; run long enough to reveal energy drift and phase error.

F4. Required evidence. Measure spectral radius/stability boundary, phase error, relative energy drift, Poisson residual, and convergence with timestep/grid size. Deliver `oracle.md`, `sweep.csv`, one error/conditioning plot, and a minimal `tests/` case reproducing the first failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for ODEs, PDEs, and variational principles. If a stable failure/crossover is found among explicit Euler, symplectic Euler, Verlet/midpoint, and backward Euler across nonstiff and stiff regimes, implement one adaptive or corrected variant that directly targets conditioning, scaling, approximation error, and the invariants of the mathematical formulation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for ODEs, PDEs, and variational principles. Using the measured spectral radius/stability boundary, phase error, relative energy drift, Poisson residual, and convergence with timestep/grid size, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for ODEs, PDEs, and variational principles

- G1. Closed-book derivation/oracle. Derive explicit Euler, backward Euler, and symplectic Euler for the harmonic oscillator; compute their update matrices/eigenvalues and use them to explain stability and long-time energy behavior. Then solve this exact micro-case from scratch without opening your implementation: For $q''+q=0$ with $q(0)=1$, $v(0)=0$ and $dt=0.5$, take four steps of explicit Euler, symplectic Euler, and implicit Euler by hand. Track energy and compare with the exact solution. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement harmonic oscillator with four integrators; measure energy drift; derive Poisson equation from an energy; solve 1D Poisson by finite differences. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use explicit Euler on the stiffest test at the same timestep as backward/symplectic methods and interpret instability as a modeling failure. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Add an eigenvalue/stability-region plot for each integrator and verify that the observed oscillator/stiff-system behavior agrees with the update-matrix prediction. Then apply this stress without retuning: Sweep timestep through a stability boundary on both a simple oscillator and a stiff system; run long enough to reveal energy drift and phase error. Report spectral radius/stability boundary, phase error, relative energy drift, Poisson residual, and convergence with timestep/grid size and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across explicit Euler, symplectic Euler, Verlet/midpoint, and backward Euler across nonstiff and stiff regimes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your

choice. Cite the measured spectral radius/stability boundary, phase error, relative energy drift, Poisson residual, and convergence with timestep/grid size; state one counterexample that would change the diagnosis or ranking.

- G6. Pass condition for ODEs, PDEs, and variational principles. Using spectral radius/stability boundary, phase error, relative energy drift, Poisson residual, and convergence with timestep/grid size, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 1, Unit 8. Differential geometry essentials

Recommended time. 1.8-2.2 weeks (about 27-44 focused hours depending on background).

What you are learning. Curves/surfaces, tangent spaces, metrics, curvature, geodesics, differential forms at an intuitive and computational level.

A. Reading order

1. Read the primary assignment first: Keenan Crane, Discrete Differential Geometry (DDG) course: smooth curves, smooth surfaces, exterior algebra/calculus lectures.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CMU DDG videos and notes.
3. Keep the most useful executable/reference page open while coding: Fundamentals of Computer Graphics 5e companion site - <https://www.cs.cornell.edu/~srm/fcg5/>

B. Definitions, results and algorithms to know

- First/second fundamental forms, Gaussian/mean curvature, Gauss map, geodesic, exterior derivative and Stokes theorem concept
- Precision requirement: distinguish and justify analytic curvature/geodesic quantities vs discrete estimates under mesh refinement, irregularity, and noise. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive curvature of a plane curve from the unit tangent, define the first and second fundamental forms for a surface patch, and derive Gaussian/mean curvature from the shape operator on a sphere or cylinder.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Estimate curvature/geodesic quantities on analytic shapes, then add positional noise and increasingly skinny triangles while holding the underlying surface fixed. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Compute curvature of sampled plane curves; numerically estimate normals/curvatures on meshes; animate parallel transport on a sphere. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Approximate a unit circle by regular polygons with 6 and 12 vertices. Compute turning angles and integrated curvature. Then use a coarse triangulated unit sphere to check that total angle defect approaches 4π . The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Estimate curvature/geodesic quantities on analytic shapes, then add positional noise and increasingly skinny triangles while holding the underlying surface fixed. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build a mesh-resolution/noise sweep on sphere, cylinder, and torus meshes; export curvature error versus edge length and triangle aspect ratio.
- P5 - Machine-readable evidence. Export curvature/geodesic error to analytic truth, convergence rate with edge length, sensitivity to triangle aspect ratio/noise to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: analytic curvature/geodesic quantities vs discrete estimates under mesh refinement, irregularity, and noise. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Estimate curvature/geodesic quantities on analytic shapes, then add positional noise and increasingly skinny triangles while holding the underlying surface fixed." without hand-editing code between methods. Record curvature/geodesic error to analytic truth, convergence rate with edge length, sensitivity to triangle aspect ratio/noise.

- P7 - Deliberate bug and regression drill. Intentionally flip a subset of triangle orientations and compute curvature without detecting the resulting sign/orientation inconsistency. Before running, predict the first observable failure in curvature/geodesic error to analytic truth, convergence rate with edge length, sensitivity to triangle aspect ratio/noise. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive curvature of a plane curve from the unit tangent, define the first and second fundamental forms for a surface patch, and derive Gaussian/mean curvature from the shape operator on a sphere or cylinder. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Approximate a unit circle by regular polygons with 6 and 12 vertices. Compute turning angles and integrated curvature. Then use a coarse triangulated unit sphere to check that total angle defect approaches 4π . Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Compute curvature of sampled plane curves; numerically estimate normals/curvatures on meshes; animate parallel transport on a sphere. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build a mesh-resolution/noise sweep on sphere, cylinder, and torus meshes; export curvature error versus edge length and triangle aspect ratio. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare analytic curvature/geodesic quantities vs discrete estimates under mesh refinement, irregularity, and noise. Use the same analytic input, numerical precision and scale; for sampling/optimization comparisons, match sample or iteration budget and stopping criteria. Record curvature/geodesic error to analytic truth, convergence rate with edge length, sensitivity to triangle aspect ratio/noise. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Estimate curvature/geodesic quantities on analytic shapes, then add positional noise and increasingly skinny triangles while holding the underlying surface fixed. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Keenan Crane, Discrete Differential Geometry (DDG) course: smooth curves, smooth surfaces, exterior algebra/calculus lectures. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Differential geometry essentials. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among analytic curvature/geodesic quantities vs discrete estimates under mesh refinement, irregularity, and noise. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using curvature/geodesic error to analytic truth, convergence rate with edge length, sensitivity to triangle aspect ratio/noise. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among analytic curvature/geodesic quantities vs discrete estimates under mesh refinement, irregularity, and noise is reproducible across at least three controlled parameter regimes and agrees with an analytic conditioning/error/stability prediction expressed in curvature/geodesic error to analytic truth, convergence rate with edge length, sensitivity to triangle aspect ratio/noise; isolate numerical precision from mathematical failure.

F. Concrete research project - Differential geometry essentials

- F1. Falsifiable question. Under the controlled failure regime "Estimate curvature/geodesic quantities on analytic shapes, then add positional noise and increasingly skinny triangles while holding the underlying surface fixed.", which alternative in analytic curvature/geodesic quantities vs discrete estimates under mesh refinement, irregularity, and noise is preferable, and is the change explained by conditioning, scaling, approximation error, and the invariants of the mathematical formulation? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build a mesh-resolution/noise sweep on sphere, cylinder, and torus meshes; export curvature error versus edge length and triangle aspect ratio. Add a single

experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use double precision for the reference path; include a hand-solvable case, then at least 6 logarithmically or geometrically spaced conditioning/scale levels. Where feasible, add an mpmath/high-precision oracle for the two hardest cases. The central adversarial condition is: Estimate curvature/geodesic quantities on analytic shapes, then add positional noise and increasingly skinny triangles while holding the underlying surface fixed.

F4. Required evidence. Measure curvature/geodesic error to analytic truth, convergence rate with edge length, sensitivity to triangle aspect ratio/noise. Deliver `oracle.md`, `sweep.csv`, one error/conditioning plot, and a minimal `tests/` case reproducing the first failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Differential geometry essentials. If a stable failure/crossover is found among analytic curvature/geodesic quantities vs discrete estimates under mesh refinement, irregularity, and noise, implement one adaptive or corrected variant that directly targets conditioning, scaling, approximation error, and the invariants of the mathematical formulation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Differential geometry essentials. Using the measured curvature/geodesic error to analytic truth, convergence rate with edge length, sensitivity to triangle aspect ratio/noise, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Differential geometry essentials

- G1. Closed-book derivation/oracle. Derive curvature of a plane curve from the unit tangent, define the first and second fundamental forms for a surface patch, and derive Gaussian/mean curvature from the shape operator on a sphere or cylinder. Then solve this exact micro-case from scratch without opening your implementation: Approximate a unit circle by regular polygons with 6 and 12 vertices. Compute turning angles and integrated curvature. Then use a coarse triangulated unit sphere to check that total angle defect approaches 4π . You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Compute curvature of sampled plane curves; numerically estimate normals/curvatures on meshes; animate parallel transport on a sphere. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally flip a subset of triangle orientations and compute curvature without detecting the resulting sign/orientation inconsistency. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Build a mesh-resolution/noise sweep on sphere, cylinder, and torus meshes; export curvature error versus edge length and triangle aspect ratio. Then apply this stress without retuning: Estimate curvature/geodesic quantities on analytic shapes, then add positional noise and increasingly skinny triangles while holding the underlying surface fixed. Report curvature/geodesic error to analytic truth, convergence rate with edge length, sensitivity to triangle aspect ratio/noise and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among analytic curvature/geodesic quantities vs discrete estimates under mesh refinement, irregularity, and noise for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured curvature/geodesic error to analytic truth, convergence rate with edge length, sensitivity to triangle aspect ratio/noise; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Differential geometry essentials. Using curvature/geodesic error to analytic truth, convergence rate with edge length, sensitivity to triangle aspect ratio/noise, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 1, Unit 9. Optimization for graphics

Recommended time. 1.8-2.2 weeks (about 27-44 focused hours depending on background).

What you are learning. Gradient descent, Newton/Gauss-Newton, line search, constraints, robust losses, stochastic optimization, nonlinear least squares.

A. Reading order

1. Read the primary assignment first: Nocedal & Wright selected Ch. 2-3, 10; Physics-Based Simulation contact/optimization sections; inverse-rendering tutorials later.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Mitsuba gradient-based optimization tutorial (defer code to Vol. 13).
3. Keep the most useful executable/reference page open while coding: Fundamentals of Computer Graphics 5e companion site - <https://www.cs.cornell.edu/~srm/fcg5/>

B. Definitions, results and algorithms to know

- First- and second-order conditions
- Gauss-Newton
- KKT intuition
- proximal/penalty/barrier idea
- Precision requirement: distinguish and justify gradient descent, Gauss-Newton, Levenberg-Marquardt, and robust losses across well- and poorly-observed inverse problems. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive gradient descent, Gauss-Newton, and Levenberg-Marquardt updates for a nonlinear least-squares objective; explain the local quadratic model and construct a case where the Hessian/Jacobian approximation is misleading.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use a symmetric or weakly observed inverse problem and at least 30 initializations; map attraction basins and compare L2 versus robust losses. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Fit a camera pose by nonlinear least squares; implement Gauss-Newton for curve fitting; compare L2 vs Huber losses. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Fit line $y=ax+b$ to points (0,0), (1,1),(2,2),(3,20). Compute one gradient/Gauss-Newton step for L2 and compare the influence of the outlier with a Huber loss. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use a symmetric or weakly observed inverse problem and at least 30 initializations; map attraction basins and compare L2 versus robust losses. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create basin-of-attraction maps for a two-parameter inverse problem over at least 30 initializations and compare L2, Huber, and LM damping schedules.
- P5 - Machine-readable evidence. Export parameter error, objective value, gradient norm, iteration count, success basin fraction, and robustness to initialization/outliers to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: gradient descent, Gauss-Newton, Levenberg-Marquardt, and robust losses across well- and poorly-observed inverse problems. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use a symmetric or weakly observed inverse problem and at least 30 initializations; map attraction basins and compare L2 versus robust losses." without hand-editing code between methods. Record parameter error, objective value, gradient norm, iteration count, success basin fraction, and robustness to initialization/outliers.
- P7 - Deliberate bug and regression drill. Intentionally apply a Gauss-Newton step with an inconsistent residual/Jacobian convention and accept every step without a damping or line-search check. Before running, predict the first observable failure in parameter error, objective value, gradient norm, iteration count, success basin fraction, and robustness to initialization/outliers. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive gradient descent, Gauss-Newton, and Levenberg-Marquardt updates for a nonlinear least-squares objective; explain the local quadratic model and construct a case where the Hessian/Jacobian approximation is misleading. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Fit line $y=ax+b$ to points (0,0),(1,1),(2,2),(3,20). Compute one gradient/Gauss-Newton step for L2 and compare the influence of the outlier with a Huber loss. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Fit a camera pose by nonlinear least squares; implement Gauss-Newton for curve fitting; compare L2 vs Huber losses. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create basin-of-attraction maps for a two-parameter inverse problem over at least 30 initializations and compare L2, Huber, and LM damping schedules. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare gradient descent, Gauss-Newton, Levenberg-Marquardt, and robust losses across well- and poorly-observed inverse problems. Use the same analytic input, numerical precision and scale; for sampling/optimization comparisons, match sample or iteration budget and stopping criteria. Record parameter error, objective value, gradient norm, iteration count, success basin fraction, and robustness to initialization/outliers. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use a symmetric or weakly observed inverse problem and at least 30 initializations; map attraction basins and compare L2 versus robust losses. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Nocedal & Wright selected Ch. 2-3, 10; Physics-Based Simulation contact/optimization sections; inverse-rendering tutorials later. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Optimization for graphics. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among gradient descent, Gauss-Newton, Levenberg-Marquardt, and robust losses across well- and poorly-observed inverse problems. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using parameter error, objective value, gradient norm, iteration count, success basin fraction, and robustness to initialization/outliers. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among gradient descent, Gauss-Newton, Levenberg-Marquardt, and robust losses across well- and poorly-observed inverse problems is reproducible across at least three controlled parameter regimes and agrees with an analytic conditioning/error/stability prediction expressed in parameter error, objective value, gradient norm, iteration count, success basin fraction, and robustness to initialization/outliers; isolate numerical precision from mathematical failure.

F. Concrete research project - Optimization for graphics

- F1. Falsifiable question. Under the controlled failure regime "Use a symmetric or weakly observed inverse problem and at least 30 initializations; map attraction basins and compare L2 versus robust losses.", characterize the ranking or regime transition across gradient descent, Gauss-Newton, Levenberg-Marquardt, and robust losses across well- and poorly-observed inverse problems. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by conditioning, local linearization, and loss/parameter scaling. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create basin-of-attraction maps for a two-parameter inverse problem over at least 30 initializations and compare L2, Huber, and LM damping schedules. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use double precision for the reference path; include a hand-solvable case, then at least 6 logarithmically or geometrically spaced conditioning/scale levels. Where feasible, add an mpmath/high-precision oracle

for the two hardest cases. The central adversarial condition is: Use a symmetric or weakly observed inverse problem and at least 30 initializations; map attraction basins and compare L2 versus robust losses.

F4. Required evidence. Measure parameter error, objective value, gradient norm, iteration count, success basin fraction, and robustness to initialization/outliers. Deliver `oracle.md`, `sweep.csv`, one error/conditioning plot, and a minimal `tests/` case reproducing the first failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Optimization for graphics. If a stable failure/crossover is found among gradient descent, Gauss-Newton, Levenberg-Marquardt, and robust losses across well- and poorly-observed inverse problems, implement one adaptive or corrected variant that directly targets conditioning, local linearization, and loss/parameter scaling; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Optimization for graphics. Using the measured parameter error, objective value, gradient norm, iteration count, success basin fraction, and robustness to initialization/outliers, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Optimization for graphics

- G1. Closed-book derivation/oracle. Derive gradient descent, Gauss-Newton, and Levenberg-Marquardt updates for a nonlinear least-squares objective; explain the local quadratic model and construct a case where the Hessian/Jacobian approximation is misleading. Then solve this exact micro-case from scratch without opening your implementation: Fit line $y=ax+b$ to points (0,0),(1,1),(2,2),(3,20). Compute one gradient/Gauss-Newton step for L2 and compare the influence of the outlier with a Huber loss. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Fit a camera pose by nonlinear least squares; implement Gauss-Newton for curve fitting; compare L2 vs Huber losses. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally apply a Gauss-Newton step with an inconsistent residual/Jacobian convention and accept every step without a damping or line-search check. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create basin-of-attraction maps for a two-parameter inverse problem over at least 30 initializations and compare L2, Huber, and LM damping schedules. Then apply this stress without retuning: Use a symmetric or weakly observed inverse problem and at least 30 initializations; map attraction basins and compare L2 versus robust losses. Report parameter error, objective value, gradient norm, iteration count, success basin fraction, and robustness to initialization/outliers and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across gradient descent, Gauss-Newton, Levenberg-Marquardt, and robust losses across well- and poorly-observed inverse problems for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured parameter error, objective value, gradient norm, iteration count, success basin fraction, and robustness to initialization/outliers; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Optimization for graphics. Using parameter error, objective value, gradient norm, iteration count, success basin fraction, and robustness to initialization/outliers, Pass when the hand/high-precision oracle is reproduced, the predicted conditioning/refinement trend appears, and any loss of accuracy is quantitatively explained rather than hidden by residual-only checks. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 1 Closeout: Project, Review and Mastery Gate

Volume project. Build a graphics mathematics laboratory containing barycentric coordinates, transformations, quaternion interpolation, Monte Carlo sampling, Fourier aliasing, ODE integration, sparse Poisson solves, curvature estimation and nonlinear least squares. Every notebook must contain a derivation and a numerical verification.

Cumulative review

- Build a dependency sheet linking affine/projective geometry, probability, Fourier analysis, ODE/PDE ideas, differential geometry and optimization to later graphics algorithms.
- Re-derive one Monte Carlo estimator, one sampling/Fourier result and one variational/time-integration result from scratch; verify each with a numerical oracle.
- Run a conditioning experiment showing when a mathematically equivalent formulation becomes numerically inferior.
- Package a reusable math-test notebook/library containing gradient checks, Monte Carlo confidence intervals, transform tests and convergence plots.
- Write a synthesis explaining which mathematical assumptions later graphics systems most often violate or approximate.

Mastery gate before moving on

- Derive affine/projective transforms, a Monte Carlo estimator, one time-integration stability statement, and one curvature/optimization relation without notes.
- Solve one small inverse problem with your own Jacobian/least-squares implementation and pass a numerical gradient check.
- Explain how conditioning, discretization, and representation choices interact across at least three Volume 1 units.

Volume 2. Programming, Numerical Computing, and GPU Systems

Purpose. Acquire the implementation fluency needed to turn graphics mathematics into reliable CPU/GPU systems.

Suggested duration. 12-16 weeks

Prerequisites. Volume 1 may be taken concurrently. For C++, begin once basic Python and linear algebra are comfortable.

Typical module pace. 1.7-2.3 weeks per module (7 modules total).

Primary references for this volume

- Fundamentals of Computer Graphics, Ch. 12 and 17.
- Real-Time Rendering, 4e, Ch. 2-6 and GPU-focused sections.
- PBRT 4e, Ch. 7 and 15 for data structures/GPU wavefront engineering.
- Eigen, NumPy/SciPy and graphics API official documentation.

Online spine

Berkeley CS184/284A Spring 2026 course: <https://cs184.eecs.berkeley.edu/sp26/>

Real-Time Rendering 4e resources: <https://www.realtimerendering.com/>

Canonical paper track

- Aila & Laine (2009), Understanding the Efficiency of Ray Traversal on GPUs.
- Laine et al. / vendor technical reports on GPU execution and divergence; pair with current GPU guides.
- PBRT 4e Chapter 15 as a research-engineering case study in wavefront GPU rendering.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Volume 2, Unit 1. Python scientific stack and reproducible experiments

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. Python, NumPy/SciPy, Matplotlib, notebooks, environments, profiling, deterministic experiments.

A. Reading order

1. Read the primary assignment first: NumPy/SciPy documentation; selected FCG implementation examples.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Official NumPy quickstart; pytest basics; profiling tutorials.
3. Keep the most useful executable/reference page open while coding: NumPy documentation - <https://numpy.org/doc/> ; SciPy documentation - <https://docs.scipy.org/doc/scipy/>

B. Definitions, results and algorithms to know

- Array broadcasting, vectorization, floating-point reproducibility, profiling, testing
- Precision requirement: distinguish and justify vectorized NumPy/SciPy implementation vs naïve Python loops, with identical seeds/data and a clean-environment rerun. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Take one small numerical graphics experiment and derive a reproducibility contract: deterministic inputs, seeded randomness, environment capture, numerical tolerances, and a data/figure pipeline that can be regenerated from raw results.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Rerun the same experiment in a fresh environment and a second process/machine; require every reported output to regenerate from one documented command. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Build a small image-processing library with tests, benchmarks, and command-line reproducibility. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Compute pairwise squared distances for four 2D points both with nested Python loops and vectorized broadcasting; verify the two 4x4 matrices exactly, then make the same calculation reproducible from a fixed seed-generated point set. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Rerun the same experiment in a fresh environment and a second process/machine; require every reported output to regenerate from one documented command. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Package one experiment as a CLI with config file, seed, environment lock, tests, benchmark command, and script that regenerates every figure from raw results.
- P5 - Machine-readable evidence. Export bitwise/tolerance reproducibility, clean-environment success, test coverage of numerical kernels, runtime, and figure regeneration success to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: vectorized NumPy/SciPy implementation vs naïve Python loops, with identical seeds/data and a clean-environment rerun. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Rerun the same experiment in a fresh environment and a second process/machine; require every reported output to regenerate from one documented command." without hand-editing code between methods. Record bitwise/tolerance reproducibility, clean-environment success, test coverage of numerical kernels, runtime, and figure regeneration success.
- P7 - Deliberate bug and regression drill. Intentionally reuse global mutable RNG/state and accidentally benchmark a view/copy with different memory layout while claiming identical data and seeds. Before running, predict the first observable failure in bitwise/tolerance reproducibility, clean-environment success, test coverage of numerical kernels, runtime, and figure regeneration success. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Take one small numerical graphics experiment and derive a reproducibility contract: deterministic inputs, seeded randomness, environment capture, numerical tolerances, and a data/figure pipeline that can be regenerated from raw results. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Compute pairwise squared distances for four 2D points both with nested Python loops and vectorized broadcasting; verify the two 4x4 matrices exactly, then make the same calculation reproducible from a

fixed seed-generated point set. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Build a small image-processing library with tests, benchmarks, and command-line reproducibility. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Package one experiment as a CLI with config file, seed, environment lock, tests, benchmark command, and script that regenerates every figure from raw results. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare vectorized NumPy/SciPy implementation vs naïve Python loops, with identical seeds/data and a clean-environment rerun. Use the same hardware, compiler/interpreter version, optimization flags, input data and thread/GPU configuration; repeat timings and report median plus a tail statistic. Record bitwise/tolerance reproducibility, clean-environment success, test coverage of numerical kernels, runtime, and figure regeneration success. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Rerun the same experiment in a fresh environment and a second process/machine; require every reported output to regenerate from one documented command. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: NumPy/SciPy documentation; selected FCG implementation examples. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Python scientific stack and reproducible experiments. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among vectorized NumPy/SciPy implementation vs naïve Python loops, with identical seeds/data and a clean-environment rerun. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using bitwise/tolerance reproducibility, clean-environment success, test coverage of numerical kernels, runtime, and figure regeneration success. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if vectorized NumPy/SciPy implementation vs naïve Python loops, with identical seeds/data and a clean-environment rerun remain correctly matched and the performance/correctness crossover in bitwise/tolerance reproducibility, clean-environment success, test coverage of numerical kernels, runtime, and figure regeneration success survives repeated runs on stated hardware/build settings; a faster incorrect or differently configured implementation does not count.

F. Concrete research project - Python scientific stack and reproducible experiments

F1. Falsifiable question. Under the controlled failure regime "Rerun the same experiment in a fresh environment and a second process/machine; require every reported output to regenerate from one documented command.", which alternative in vectorized NumPy/SciPy implementation vs naïve Python loops, with identical seeds/data and a clean-environment rerun is preferable, and is the change explained by data layout, ownership/lifetime, precision, synchronization, and hardware execution cost? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Package one experiment as a CLI with config file, seed, environment lock, tests, benchmark command, and script that regenerates every figure from raw results. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Benchmark at least 5 problem sizes after warm-up; use at least 20 timed repeats per configuration, report median plus dispersion, and keep correctness tests enabled. Record CPU/GPU model, compiler, optimization flags, driver/runtime, and thread/block configuration. The central adversarial condition is: Rerun the same experiment in a fresh environment and a second process/machine; require every reported output to regenerate from one documented command.

F4. Required evidence. Measure bitwise/tolerance reproducibility, clean-environment success, test coverage of numerical kernels, runtime, and figure regeneration success. Deliver a one-command benchmark, 'bench.csv', profiler/counter capture, hardware/software manifest, and a regression test for the adversarial case. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Python scientific stack and reproducible experiments. If a stable failure/crossover is found among vectorized NumPy/SciPy implementation vs naïve Python loops, with identical seeds/data and a clean-environment rerun, implement one adaptive or corrected variant that directly targets data layout, ownership/lifetime, precision, synchronization, and hardware execution cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Python scientific stack and reproducible experiments. Using the measured bitwise/tolerance reproducibility, clean-environment success, test coverage of numerical kernels, runtime, and figure regeneration success, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Python scientific stack and reproducible experiments

- G1. Closed-book derivation/oracle. Take one small numerical graphics experiment and derive a reproducibility contract: deterministic inputs, seeded randomness, environment capture, numerical tolerances, and a data/figure pipeline that can be regenerated from raw results. Then solve this exact micro-case from scratch without opening your implementation: Compute pairwise squared distances for four 2D points both with nested Python loops and vectorized broadcasting; verify the two 4x4 matrices exactly, then make the same calculation reproducible from a fixed seed-generated point set. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Build a small image-processing library with tests, benchmarks, and command-line reproducibility. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally reuse global mutable RNG/state and accidentally benchmark a view/copy with different memory layout while claiming identical data and seeds. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Package one experiment as a CLI with config file, seed, environment lock, tests, benchmark command, and script that regenerates every figure from raw results. Then apply this stress without retuning: Rerun the same experiment in a fresh environment and a second process/machine; require every reported output to regenerate from one documented command. Report bitwise/tolerance reproducibility, clean-environment success, test coverage of numerical kernels, runtime, and figure regeneration success and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among vectorized NumPy/SciPy implementation vs naïve Python loops, with identical seeds/data and a clean-environment rerun for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured bitwise/tolerance reproducibility, clean-environment success, test coverage of numerical kernels, runtime, and figure regeneration success; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Python scientific stack and reproducible experiments. Using bitwise/tolerance reproducibility, clean-environment success, test coverage of numerical kernels, runtime, and figure regeneration success, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 2, Unit 2. Modern C++ for graphics

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. RAI, references, value semantics, templates, STL, smart pointers, build systems, Eigen.

A. Reading order

1. Read the primary assignment first: C++ reference plus Eigen getting-started material; geometry-central sample project conventions.

2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Geometry Central load/navigate mesh tutorial; libigl build tutorial.

3. Keep the most useful executable/reference page open while coding: Eigen documentation - <https://eigen.tuxfamily.org/>; cppreference - <https://en.cppreference.com/>

B. Definitions, results and algorithms to know

- Ownership and lifetime
- contiguous data
- iterators
- compile-time polymorphism
- numerical types
- Precision requirement: distinguish and justify value semantics, unique ownership, shared ownership, and non-owning views in a mesh/scene workload. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Trace ownership and lifetime for a scene graph containing shared materials and unique geometry; compare value semantics, unique ownership, shared ownership, and non-owning views, and justify a design that avoids leaks and dangling references.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use sanitizers plus copy/move/lifetime stress tests on large mesh/scene objects; intentionally trigger invalid ownership patterns and confirm they are detected. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement vector/matrix/mesh classes only once, then replace them with Eigen/libigl to understand library design tradeoffs. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Trace construction, move, copy, and destruction for a Mesh object owning one `std::vector<Vertex>` as it is returned from a function and inserted into a `vector<Mesh>`. Predict which operations should occur before using compiler instrumentation. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use sanitizers plus copy/move/lifetime stress tests on large mesh/scene objects; intentionally trigger invalid ownership patterns and confirm they are detected. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Compile the same scene/mesh container under AddressSanitizer/UBSan; add move/copy stress tests and a microbenchmark of AoS versus SoA access for one graphics kernel.
- P5 - Machine-readable evidence. Export sanitizer-clean execution, allocations/copies, cache/memory throughput, construction/update time, and peak memory to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: value semantics, unique ownership, shared ownership, and non-owning views in a mesh/scene workload. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use sanitizers plus copy/move/lifetime stress tests on large mesh/scene objects; intentionally trigger invalid ownership patterns and confirm they are detected." without hand-editing code between methods. Record sanitizer-clean execution, allocations/copies, cache/memory throughput, construction/update time, and peak memory.
- P7 - Deliberate bug and regression drill. Intentionally return a non-owning Eigen/span/view into a temporary or reallocated buffer, creating a dangling reference that survives small tests. Before running, predict the first observable failure in sanitizer-clean execution, allocations/copies, cache/memory throughput, construction/update time, and peak memory. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Trace ownership and lifetime for a scene graph containing shared materials and unique geometry; compare value semantics, unique ownership, shared ownership, and non-owning views, and justify a design that avoids leaks and dangling references. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. Trace construction, move, copy, and destruction for a Mesh object owning one `std::vector<Vertex>` as it is returned from a function and inserted into a `vector<Mesh>`. Predict which operations should occur before using compiler instrumentation. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement vector/matrix/mesh classes only once, then replace them with Eigen/libigl to understand library design tradeoffs. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Compile the same scene/mesh container under AddressSanitizer/UBSan; add move/copy stress tests and a microbenchmark of AoS versus SoA access for one graphics kernel. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare value semantics, unique ownership, shared ownership, and non-owning views in a mesh/scene workload. Use the same hardware, compiler/interpreter version, optimization flags, input data and thread/GPU configuration; repeat timings and report median plus a tail statistic. Record sanitizer-clean execution, allocations/copies, cache/memory throughput, construction/update time, and peak memory. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use sanitizers plus copy/move/lifetime stress tests on large mesh/scene objects; intentionally trigger invalid ownership patterns and confirm they are detected. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: C++ reference plus Eigen getting-started material; geometry-central sample project conventions. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Modern C++ for graphics. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among value semantics, unique ownership, shared ownership, and non-owning views in a mesh/scene workload. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using sanitizer-clean execution, allocations/copies, cache/memory throughput, construction/update time, and peak memory. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if value semantics, unique ownership, shared ownership, and non-owning views in a mesh/scene workload remain correctly matched and the performance/correctness crossover in sanitizer-clean execution, allocations/copies, cache/memory throughput, construction/update time, and peak memory survives repeated runs on stated hardware/build settings; a faster incorrect or differently configured implementation does not count.

F. Concrete research project - Modern C++ for graphics

- F1. Falsifiable question. Under the controlled failure regime "Use sanitizers plus copy/move/lifetime stress tests on large mesh/scene objects; intentionally trigger invalid ownership patterns and confirm they are detected.", characterize the ranking or regime transition across value semantics, unique ownership, shared ownership, and non-owning views in a mesh/scene workload. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by data layout, ownership/lifetime, precision, synchronization, and hardware execution cost. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Compile the same scene/mesh container under AddressSanitizer/UBSan; add move/copy stress tests and a microbenchmark of AoS versus SoA access for one graphics kernel. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Benchmark at least 5 problem sizes after warm-up; use at least 20 timed repeats per configuration, report median plus dispersion, and keep correctness tests enabled. Record CPU/GPU model, compiler, optimization flags, driver/runtime, and thread/block configuration. The central adversarial condition is: Use sanitizers plus copy/move/lifetime stress tests on large mesh/scene objects; intentionally trigger invalid ownership patterns and confirm they are detected.
- F4. Required evidence. Measure sanitizer-clean execution, allocations/copies, cache/memory throughput, construction/update time, and peak memory. Deliver a one-command benchmark, 'bench.csv', profiler/counter capture, hardware/software manifest, and a regression test for the adversarial case. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Modern C++ for graphics. If a stable failure/crossover is found among value semantics, unique ownership, shared ownership, and non-owning views in a mesh/scene workload, implement one adaptive or corrected variant that directly targets data layout, ownership/lifetime, precision, synchronization, and hardware execution cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Modern C++ for graphics. Using the measured sanitizer-clean execution, allocations/copies, cache/memory throughput, construction/update time, and peak memory, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Modern C++ for graphics

- G1. Closed-book derivation/oracle. Trace ownership and lifetime for a scene graph containing shared materials and unique geometry; compare value semantics, unique ownership, shared ownership, and non-owning views, and justify a design that avoids leaks and dangling references. Then solve this exact micro-case from scratch without opening your implementation: Trace construction, move, copy, and destruction for a Mesh object owning one `std::vector<Vertex>` as it is returned from a function and inserted into a `vector<Mesh>`. Predict which operations should occur before using compiler instrumentation. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement vector/matrix/mesh classes only once, then replace them with Eigen/libigl to understand library design tradeoffs. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally return a non-owning Eigen/span/view into a temporary or reallocated buffer, creating a dangling reference that survives small tests. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Compile the same scene/mesh container under AddressSanitizer/UBSan; add move/copy stress tests and a microbenchmark of AoS versus SoA access for one graphics kernel. Then apply this stress without retuning: Use sanitizers plus copy/move/lifetime stress tests on large mesh/scene objects; intentionally trigger invalid ownership patterns and confirm they are detected. Report sanitizer-clean execution, allocations/copies, cache/memory throughput, construction/update time, and peak memory and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across value semantics, unique ownership, shared ownership, and non-owning views in a mesh/scene workload for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured sanitizer-clean execution, allocations/copies, cache/memory throughput, construction/update time, and peak memory; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Modern C++ for graphics. Using sanitizer-clean execution, allocations/copies, cache/memory throughput, construction/update time, and peak memory, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 2, Unit 3. Data structures and spatial indexing

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. Arrays, hash tables, trees, graphs, half-edge meshes, BVHs, kd-trees, grids, octrees.

A. Reading order

1. Read the primary assignment first: FCG Ch. 12; PBRT Ch. 7; geometry-central mesh data structures.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 mesh representations and acceleration lectures.

3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Asymptotic cost, locality, traversal invariants, Morton codes intuition
- Precision requirement: distinguish and justify uniform grid, k-d tree, median BVH, and SAH BVH on uniform, clustered, elongated, and duplicated geometry. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive expected query/build costs for a uniform grid, k-d tree, and BVH under contrasting spatial distributions; explain which assumptions fail for clustered, elongated, or duplicated geometry.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Benchmark uniform, clustered, elongated, duplicated, and nearly degenerate spatial distributions; include both coherent and incoherent query patterns. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement a half-edge mesh validator and a median-split BVH; benchmark brute-force vs BVH ray intersections. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Place eight points at $x=\{0,0.1,0.2,0.3,10,10.1,10.2,10.3\}$. Draw a uniform grid, balanced k-d split, median BVH-like split, and a cost-aware split; count candidate checks for two queries near each cluster. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Benchmark uniform, clustered, elongated, duplicated, and nearly degenerate spatial distributions; include both coherent and incoherent query patterns. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement a benchmark driver that generates four spatial distributions and two query-coherence regimes, then records build cost, query cost, memory, and tree depth statistics.
- P5 - Machine-readable evidence. Export build time, query/traversal time, memory, tree depth/overlap, and scaling across distribution/coherence regimes to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: uniform grid, k-d tree, median BVH, and SAH BVH on uniform, clustered, elongated, and duplicated geometry. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Benchmark uniform, clustered, elongated, duplicated, and nearly degenerate spatial distributions; include both coherent and incoherent query patterns." without hand-editing code between methods. Record build time, query/traversal time, memory, tree depth/overlap, and scaling across distribution/coherence regimes.
- P7 - Deliberate bug and regression drill. Intentionally forget to update spatial bounds/keys after moving or duplicating geometry, so queries become stale only after dynamic edits. Before running, predict the first observable failure in build time, query/traversal time, memory, tree depth/overlap, and scaling across distribution/coherence regimes. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive expected query/build costs for a uniform grid, k-d tree, and BVH under contrasting spatial distributions; explain which assumptions fail for clustered, elongated, or duplicated geometry. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Place eight points at $x=\{0,0.1,0.2,0.3,10,10.1,10.2,10.3\}$. Draw a uniform grid, balanced k-d split, median BVH-like split, and a cost-aware split; count candidate checks for two queries near each cluster. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement a half-edge mesh validator and a median-split BVH; benchmark brute-force vs BVH ray intersections. Before optimizing, make the smallest case agree with Problem 2. Then

complete this deeper task: Implement a benchmark driver that generates four spatial distributions and two query-coherence regimes, then records build cost, query cost, memory, and tree depth statistics. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare uniform grid, k-d tree, median BVH, and SAH BVH on uniform, clustered, elongated, and duplicated geometry. Use the same hardware, compiler/interpreter version, optimization flags, input data and thread/GPU configuration; repeat timings and report median plus a tail statistic. Record build time, query/traversal time, memory, tree depth/overlap, and scaling across distribution/coherence regimes. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Benchmark uniform, clustered, elongated, duplicated, and nearly degenerate spatial distributions; include both coherent and incoherent query patterns. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: FCG Ch. 12; PBRT Ch. 7; geometry-central mesh data structures. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Data structures and spatial indexing. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among uniform grid, k-d tree, median BVH, and SAH BVH on uniform, clustered, elongated, and duplicated geometry. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using build time, query/traversal time, memory, tree depth/overlap, and scaling across distribution/coherence regimes. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if uniform grid, k-d tree, median BVH, and SAH BVH on uniform, clustered, elongated, and duplicated geometry remain correctly matched and the performance/correctness crossover in build time, query/traversal time, memory, tree depth/overlap, and scaling across distribution/coherence regimes survives repeated runs on stated hardware/build settings; a faster incorrect or differently configured implementation does not count.

F. Concrete research project - Data structures and spatial indexing

F1. Falsifiable question. Under the controlled failure regime "Benchmark uniform, clustered, elongated, duplicated, and nearly degenerate spatial distributions; include both coherent and incoherent query patterns.", characterize the ranking or regime transition across uniform grid, k-d tree, median BVH, and SAH BVH on uniform, clustered, elongated, and duplicated geometry. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement a benchmark driver that generates four spatial distributions and two query-coherence regimes, then records build cost, query cost, memory, and tree depth statistics. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Benchmark at least 5 problem sizes after warm-up; use at least 20 timed repeats per configuration, report median plus dispersion, and keep correctness tests enabled. Record CPU/GPU model, compiler, optimization flags, driver/runtime, and thread/block configuration. The central adversarial condition is: Benchmark uniform, clustered, elongated, duplicated, and nearly degenerate spatial distributions; include both coherent and incoherent query patterns.

F4. Required evidence. Measure build time, query/traversal time, memory, tree depth/overlap, and scaling across distribution/coherence regimes. Deliver a one-command benchmark, `bench.csv`, profiler/counter capture, hardware/software manifest, and a regression test for the adversarial case. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Data structures and spatial indexing. If a stable failure/crossover is found among uniform grid, k-d tree, median BVH, and SAH BVH on uniform, clustered, elongated, and duplicated geometry, implement one adaptive or corrected variant that directly targets memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Data structures and spatial indexing. Using the measured build time, query/traversal time, memory, tree depth/overlap, and scaling across distribution/coherence regimes, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Data structures and spatial indexing

- G1. Closed-book derivation/oracle. Derive expected query/build costs for a uniform grid, k-d tree, and BVH under contrasting spatial distributions; explain which assumptions fail for clustered, elongated, or duplicated geometry. Then solve this exact micro-case from scratch without opening your implementation: Place eight points at $x=\{0,0.1,0.2,0.3,10,10.1,10.2,10.3\}$. Draw a uniform grid, balanced k-d split, median BVH-like split, and a cost-aware split; count candidate checks for two queries near each cluster. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement a half-edge mesh validator and a median-split BVH; benchmark brute-force vs BVH ray intersections. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally forget to update spatial bounds/keys after moving or duplicating geometry, so queries become stale only after dynamic edits. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement a benchmark driver that generates four spatial distributions and two query-coherence regimes, then records build cost, query cost, memory, and tree depth statistics. Then apply this stress without retuning: Benchmark uniform, clustered, elongated, duplicated, and nearly degenerate spatial distributions; include both coherent and incoherent query patterns. Report build time, query/traversal time, memory, tree depth/overlap, and scaling across distribution/coherence regimes and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across uniform grid, k-d tree, median BVH, and SAH BVH on uniform, clustered, elongated, and duplicated geometry for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured build time, query/traversal time, memory, tree depth/overlap, and scaling across distribution/coherence regimes; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Data structures and spatial indexing. Using build time, query/traversal time, memory, tree depth/overlap, and scaling across distribution/coherence regimes, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 2, Unit 4. Floating-point robustness

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. IEEE-754, rounding, cancellation, epsilon mistakes, robust predicates, interval/error bounds.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 6.8; FCG Ch. 1.5.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: PBRT online source walkthrough.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- ULPs, relative/absolute error, conditioning vs stability
- PBRT ray-offset/rounding-error ideas
- Precision requirement: distinguish and justify naïve arithmetic/epsilons vs scale-aware bounds vs higher-precision or robust predicates. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Using the standard floating-point error model, derive forward-error bounds for a subtraction-dominated geometric predicate and for a ray intersection; distinguish conditioning of the problem from stability of the implementation.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Scale and translate the same geometric predicate/intersection problem from 10^{-9} to 10^9 ; robust decisions should remain invariant. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Construct failure cases for ray intersections and orientation tests; compare naïve epsilon hacks with scale-aware bounds. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Evaluate orientation2d for $A=(0,0)$, $B=(10^8, 10^8)$, $C=(2*10^8, 2*10^8+\delta)$ as δ shrinks. Compute the exact determinant symbolically and predict when float32 and float64 will lose the sign. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Scale and translate the same geometric predicate/intersection problem from 10^{-9} to 10^9 ; robust decisions should remain invariant. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement adaptive-precision fallback for one orientation/intersection predicate and map the subset of inputs that actually require the fallback.
- P5 - Machine-readable evidence. Export decision mismatch rate versus high precision, error across scale $1e-9$ to $1e9$, fallback frequency, and runtime overhead to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: naïve arithmetic/epsilons vs scale-aware bounds vs higher-precision or robust predicates. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Scale and translate the same geometric predicate/intersection problem from 10^{-9} to 10^9 ; robust decisions should remain invariant." without hand-editing code between methods. Record decision mismatch rate versus high precision, error across scale $1e-9$ to $1e9$, fallback frequency, and runtime overhead.
- P7 - Deliberate bug and regression drill. Intentionally use one absolute epsilon for orientation/intersection tests across coordinates spanning many orders of magnitude. Before running, predict the first observable failure in decision mismatch rate versus high precision, error across scale $1e-9$ to $1e9$, fallback frequency, and runtime overhead. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Using the standard floating-point error model, derive forward-error bounds for a subtraction-dominated geometric predicate and for a ray intersection; distinguish conditioning of the problem from stability of the implementation. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Evaluate orientation2d for $A=(0,0)$, $B=(10^8, 10^8)$, $C=(2*10^8, 2*10^8+\delta)$ as δ shrinks. Compute the exact determinant symbolically and predict when float32 and float64 will lose the sign. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Construct failure cases for ray intersections and orientation tests; compare naïve epsilon hacks with scale-aware bounds. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement adaptive-precision fallback for one orientation/intersection predicate and map the subset of inputs that actually require the fallback. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare naïve arithmetic/epsilons vs scale-aware bounds vs higher-precision or robust predicates. Use the same hardware, compiler/interpreter version, optimization flags, input data and thread/GPU configuration; repeat timings and report median plus a tail statistic. Record decision mismatch rate versus high precision, error across scale $1e-9$ to $1e9$, fallback frequency, and runtime overhead. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Scale and translate the same geometric predicate/intersection problem from 10^{-9} to 10^9 ; robust decisions should remain invariant. State the predicted failure signature before execution. Deliverable: a

sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: PBRT Ch. 6.8; FCG Ch. 1.5. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Floating-point robustness. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among naïve arithmetic/epsilons vs scale-aware bounds vs higher-precision or robust predicates. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using decision mismatch rate versus high precision, error across scale $1e-9$ to $1e9$, fallback frequency, and runtime overhead. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if naïve arithmetic/epsilons vs scale-aware bounds vs higher-precision or robust predicates remain correctly matched and the performance/correctness crossover in decision mismatch rate versus high precision, error across scale $1e-9$ to $1e9$, fallback frequency, and runtime overhead survives repeated runs on stated hardware/build settings; a faster incorrect or differently configured implementation does not count.

F. Concrete research project - Floating-point robustness

F1. Falsifiable question. Under the controlled failure regime "Scale and translate the same geometric predicate/intersection problem from 10^{-9} to 10^9 ; robust decisions should remain invariant.", which alternative in naïve arithmetic/epsilons vs scale-aware bounds vs higher-precision or robust predicates is preferable, and is the change explained by data layout, ownership/lifetime, precision, synchronization, and hardware execution cost? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement adaptive-precision fallback for one orientation/intersection predicate and map the subset of inputs that actually require the fallback. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Benchmark at least 5 problem sizes after warm-up; use at least 20 timed repeats per configuration, report median plus dispersion, and keep correctness tests enabled. Record CPU/GPU model, compiler, optimization flags, driver/runtime, and thread/block configuration. The central adversarial condition is: Scale and translate the same geometric predicate/intersection problem from 10^{-9} to 10^9 ; robust decisions should remain invariant.

F4. Required evidence. Measure decision mismatch rate versus high precision, error across scale $1e-9$ to $1e9$, fallback frequency, and runtime overhead. Deliver a one-command benchmark, `bench.csv`, profiler/counter capture, hardware/software manifest, and a regression test for the adversarial case. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Floating-point robustness. If a stable failure/crossover is found among naïve arithmetic/epsilons vs scale-aware bounds vs higher-precision or robust predicates, implement one adaptive or corrected variant that directly targets data layout, ownership/lifetime, precision, synchronization, and hardware execution cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Floating-point robustness. Using the measured decision mismatch rate versus high precision, error across scale $1e-9$ to $1e9$, fallback frequency, and runtime overhead, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Floating-point robustness

- G1. Closed-book derivation/oracle. Using the standard floating-point error model, derive forward-error bounds for a subtraction-dominated geometric predicate and for a ray intersection; distinguish conditioning of the problem from stability of the implementation. Then solve this exact micro-case from scratch without opening your implementation: Evaluate orientation2d for $A=(0,0)$, $B=(10^8,10^8)$, $C=(2*10^8,2*10^8+\delta)$ as δ shrinks. Compute the exact determinant symbolically and predict when float32 and float64 will lose the sign. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Construct failure cases for ray intersections and orientation tests; compare naïve epsilon hacks with scale-aware bounds. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use one absolute epsilon for orientation/intersection tests across coordinates spanning many orders of magnitude. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement adaptive-precision fallback for one orientation/intersection predicate and map the subset of inputs that actually require the fallback. Then apply this stress without retuning: Scale and translate the same geometric predicate/intersection problem from 10^{-9} to 10^9 ; robust decisions should remain invariant. Report decision mismatch rate versus high precision, error across scale $1e-9$ to $1e9$, fallback frequency, and runtime overhead and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among naïve arithmetic/epsilons vs scale-aware bounds vs higher-precision or robust predicates for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured decision mismatch rate versus high precision, error across scale $1e-9$ to $1e9$, fallback frequency, and runtime overhead; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Floating-point robustness. Using decision mismatch rate versus high precision, error across scale $1e-9$ to $1e9$, fallback frequency, and runtime overhead, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 2, Unit 5. Graphics APIs and shader programming

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. GPU pipeline, buffers, vertex/fragment shaders, depth, blending, textures, framebuffers.

A. Reading order

1. Read the primary assignment first: FCG Ch. 17; RTR Ch. 2-6.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: LearnOpenGL or WebGPU tutorial as hands-on companion; CS184 rasterization pipeline lectures.
3. Keep the most useful executable/reference page open while coding: Khronos OpenGL/Vulkan reference material - <https://www.khronos.org/>

B. Definitions, results and algorithms to know

- SIMD/SIMT model, clip/NDC/screen spaces, depth test, interpolation, derivatives
- Precision requirement: distinguish and justify CPU reference pipeline vs GPU vertex/fragment pipeline, with intermediate clip/NDC/depth/interpolation buffers compared numerically. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the full object-to-clip-to-NDC-to-screen coordinate pipeline and perspective-correct interpolation; show algebraically why naïve linear interpolation of texture coordinates in screen space is wrong.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Create diagnostic scenes for clipping, interpolation, depth, blending, precision, and derivative behavior; compare intermediate GPU buffers against a CPU/reference calculation. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create a minimal GPU renderer: indexed mesh, camera, per-pixel lighting, texture, shadow map, postprocess pass. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For one triangle with object-space vertices, a given model-view-projection matrix, and per-vertex colors, compute clip coordinates, NDC, screen coordinates, barycentrics at one pixel, perspective-correct attributes, and depth by hand. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Create diagnostic scenes for clipping, interpolation, depth, blending, precision, and derivative behavior; compare intermediate GPU buffers against a CPU/reference calculation. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Add debug render modes for clip-space position, barycentrics, normals, depth, UV derivatives, and shadow coordinates; compare selected pixels to a CPU oracle.
- P5 - Machine-readable evidence. Export per-stage numeric mismatch to CPU oracle, frame time, draw/dispatch count, and precision/interpolation error maps to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: CPU reference pipeline vs GPU vertex/fragment pipeline, with intermediate clip/NDC/depth/interpolation buffers compared numerically. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Create diagnostic scenes for clipping, interpolation, depth, blending, precision, and derivative behavior; compare intermediate GPU buffers against a CPU/reference calculation." without hand-editing code between methods. Record per-stage numeric mismatch to CPU oracle, frame time, draw/dispatch count, and precision/interpolation error maps.
- P7 - Deliberate bug and regression drill. Intentionally mix row/column-major transform conventions or clip/depth ranges between CPU reference code and the shader pipeline. Before running, predict the first observable failure in per-stage numeric mismatch to CPU oracle, frame time, draw/dispatch count, and precision/interpolation error maps. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the full object-to-clip-to-NDC-to-screen coordinate pipeline and perspective-correct interpolation; show algebraically why naïve linear interpolation of texture coordinates in screen space is wrong. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For one triangle with object-space vertices, a given model-view-projection matrix, and per-vertex colors, compute clip coordinates, NDC, screen coordinates, barycentrics at one pixel, perspective-correct attributes, and depth by hand. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Create a minimal GPU renderer: indexed mesh, camera, per-pixel lighting, texture, shadow map, postprocess pass. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Add debug render modes for clip-space position, barycentrics, normals, depth, UV derivatives, and shadow coordinates; compare selected pixels to a CPU oracle. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare CPU reference pipeline vs GPU vertex/fragment pipeline, with intermediate clip/NDC/depth/interpolation buffers compared numerically. Use the same hardware, compiler/interpreter version, optimization flags, input data and thread/GPU configuration; repeat timings and report median plus a tail statistic. Record per-stage numeric mismatch to CPU oracle, frame time, draw/dispatch count, and precision/interpolation error maps. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Create diagnostic scenes for clipping, interpolation, depth, blending, precision, and derivative behavior; compare intermediate GPU buffers against a CPU/reference calculation. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 17; RTR Ch. 2-6. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Graphics APIs and shader programming. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among CPU reference pipeline vs GPU vertex/fragment pipeline, with intermediate clip/NDC/depth/interpolation buffers compared numerically. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using per-stage numeric mismatch to CPU oracle, frame time, draw/dispatch count, and precision/interpolation error maps. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if

CPU reference pipeline vs GPU vertex/fragment pipeline, with intermediate clip/NDC/depth/interpolation buffers compared numerically remain correctly matched and the performance/correctness crossover in per-stage numeric mismatch to CPU oracle, frame time, draw/dispatch count, and precision/interpolation error maps survives repeated runs on stated hardware/build settings; a faster incorrect or differently configured implementation does not count.

F. Concrete research project - Graphics APIs and shader programming

F1. Falsifiable question. Under the controlled failure regime "Create diagnostic scenes for clipping, interpolation, depth, blending, precision, and derivative behavior; compare intermediate GPU buffers against a CPU/reference calculation.", which alternative in CPU reference pipeline vs GPU vertex/fragment pipeline, with intermediate clip/NDC/depth/interpolation buffers compared numerically is preferable, and is the change explained by memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Add debug render modes for clip-space position, barycentrics, normals, depth, UV derivatives, and shadow coordinates; compare selected pixels to a CPU oracle. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Benchmark at least 5 problem sizes after warm-up; use at least 20 timed repeats per configuration, report median plus dispersion, and keep correctness tests enabled. Record CPU/GPU model, compiler, optimization flags, driver/runtime, and thread/block configuration. The central adversarial condition is: Create diagnostic scenes for clipping, interpolation, depth, blending, precision, and derivative behavior; compare intermediate GPU buffers against a CPU/reference calculation.

F4. Required evidence. Measure per-stage numeric mismatch to CPU oracle, frame time, draw/dispatch count, and precision/interpolation error maps. Deliver a one-command benchmark, `bench.csv`, profiler/counter capture, hardware/software manifest, and a regression test for the adversarial case. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Graphics APIs and shader programming. If a stable failure/crossover is found among CPU reference pipeline vs GPU vertex/fragment pipeline, with intermediate clip/NDC/depth/interpolation buffers compared numerically, implement one adaptive or corrected variant that directly targets memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Graphics APIs and shader programming. Using the measured per-stage numeric mismatch to CPU oracle, frame time, draw/dispatch count, and precision/interpolation error maps, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Graphics APIs and shader programming

- G1. Closed-book derivation/oracle. Derive the full object-to-clip-to-NDC-to-screen coordinate pipeline and perspective-correct interpolation; show algebraically why naïve linear interpolation of texture coordinates in screen space is wrong. Then solve this exact micro-case from scratch without opening your implementation: For one triangle with object-space vertices, a given model-view-projection matrix, and per-vertex colors, compute clip coordinates, NDC, screen coordinates, barycentrics at one pixel, perspective-correct attributes, and depth by hand. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create a minimal GPU renderer: indexed mesh, camera, per-pixel lighting, texture, shadow map, postprocess pass. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally mix row/column-major transform conventions or clip/depth ranges between CPU reference code and the shader pipeline. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Add debug render modes for clip-space position, barycentrics, normals, depth, UV derivatives, and shadow coordinates; compare

selected pixels to a CPU oracle. Then apply this stress without retuning: Create diagnostic scenes for clipping, interpolation, depth, blending, precision, and derivative behavior; compare intermediate GPU buffers against a CPU/reference calculation. Report per-stage numeric mismatch to CPU oracle, frame time, draw/dispatch count, and precision/interpolation error maps and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method-selection defense. Using your own measured results, choose among CPU reference pipeline vs GPU vertex/fragment pipeline, with intermediate clip/NDC/depth/interpolation buffers compared numerically for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured per-stage numeric mismatch to CPU oracle, frame time, draw/dispatch count, and precision/interpolation error maps; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Graphics APIs and shader programming. Using per-stage numeric mismatch to CPU oracle, frame time, draw/dispatch count, and precision/interpolation error maps, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 2, Unit 6. GPU architecture and performance

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. Warps/wavefronts, occupancy, bandwidth, latency, cache behavior, divergence, atomics.

A. Reading order

1. Read the primary assignment first: RTR Ch. 3; vendor GPU programming guides as reference.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: NVIDIA Nsight/AMD profiler tutorial appropriate to hardware.
3. Keep the most useful executable/reference page open while coding: NVIDIA CUDA documentation/learning resources - <https://developer.nvidia.com/cuda>

B. Definitions, results and algorithms to know

- Arithmetic intensity, memory coalescing, occupancy, throughput/latency distinction
- Precision requirement: distinguish and justify compute-bound, bandwidth-bound, divergent, and occupancy-limited kernels identified by predicted and measured counters. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Use a roofline-style model to derive when a graphics/GPU kernel is bandwidth-bound versus compute-bound; connect arithmetic intensity, occupancy, divergence, and memory coalescing to expected throughput.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Construct one kernel each that isolates divergence, uncoalesced memory, low occupancy, and high arithmetic intensity; verify profiler counters match the predicted bottleneck. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Profile a shader and a CUDA-style kernel; intentionally create divergence and uncoalesced accesses; explain performance counters. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a warp of eight toy lanes evaluating if $(i \bmod 3) == 0$, mark active masks on both branches. For a kernel reading consecutive versus stride-8 floats, count memory segments in a simplified 32-byte transaction model. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Construct one kernel each that isolates divergence, uncoalesced memory, low occupancy, and high arithmetic intensity; verify profiler counters match the predicted bottleneck. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Write four minimal kernels, each isolating one bottleneck; capture Nsight/appropriate profiler counters and annotate why each counter changes.
- P5 - Machine-readable evidence. Export kernel time, achieved bandwidth/FLOP rate, occupancy, branch efficiency, cache hit rate, and roofline arithmetic intensity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.

- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: compute-bound, bandwidth-bound, divergent, and occupancy-limited kernels identified by predicted and measured counters. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Construct one kernel each that isolates divergence, uncoalesced memory, low occupancy, and high arithmetic intensity; verify profiler counters match the predicted bottleneck." without hand-editing code between methods. Record kernel time, achieved bandwidth/FLOP rate, occupancy, branch efficiency, cache hit rate, and roofline arithmetic intensity.
- P7 - Deliberate bug and regression drill. Intentionally include allocation/transfer/first-launch compilation in one kernel timing but not another, then infer a kernel bottleneck from wall-clock time alone. Before running, predict the first observable failure in kernel time, achieved bandwidth/FLOP rate, occupancy, branch efficiency, cache hit rate, and roofline arithmetic intensity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Use a roofline-style model to derive when a graphics/GPU kernel is bandwidth-bound versus compute-bound; connect arithmetic intensity, occupancy, divergence, and memory coalescing to expected throughput. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a warp of eight toy lanes evaluating if $(i \bmod 3) == 0$, mark active masks on both branches. For a kernel reading consecutive versus stride-8 floats, count memory segments in a simplified 32-byte transaction model. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Profile a shader and a CUDA-style kernel; intentionally create divergence and uncoalesced accesses; explain performance counters. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Write four minimal kernels, each isolating one bottleneck; capture Nsight/appropriate profiler counters and annotate why each counter changes. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare compute-bound, bandwidth-bound, divergent, and occupancy-limited kernels identified by predicted and measured counters. Use the same hardware, compiler/interpreter version, optimization flags, input data and thread/GPU configuration; repeat timings and report median plus a tail statistic. Record kernel time, achieved bandwidth/FLOP rate, occupancy, branch efficiency, cache hit rate, and roofline arithmetic intensity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Construct one kernel each that isolates divergence, uncoalesced memory, low occupancy, and high arithmetic intensity; verify profiler counters match the predicted bottleneck. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: RTR Ch. 3; vendor GPU programming guides as reference. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on GPU architecture and performance. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among compute-bound, bandwidth-bound, divergent, and occupancy-limited kernels identified by predicted and measured counters. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using kernel time, achieved bandwidth/FLOP rate, occupancy, branch efficiency, cache hit rate, and roofline arithmetic intensity. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if compute-bound, bandwidth-bound, divergent, and occupancy-limited kernels identified by predicted and measured counters remain correctly matched and the performance/correctness crossover in kernel time, achieved bandwidth/FLOP rate, occupancy, branch efficiency, cache hit rate, and roofline arithmetic intensity survives repeated runs on stated hardware/build settings; a faster incorrect or differently configured implementation does not count.

F. Concrete research project - GPU architecture and performance

- F1. Falsifiable question. Under the controlled failure regime "Construct one kernel each that isolates divergence, uncoalesced memory, low occupancy, and high arithmetic intensity; verify profiler counters match the predicted

bottleneck.", characterize the ranking or regime transition across compute-bound, bandwidth-bound, divergent, and occupancy-limited kernels identified by predicted and measured counters. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by arithmetic intensity, memory coalescing/cache behavior, branch divergence, occupancy, and launch/measurement methodology. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Write four minimal kernels, each isolating one bottleneck; capture Nsight/appropriate profiler counters and annotate why each counter changes. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Benchmark at least 5 problem sizes after warm-up; use at least 20 timed repeats per configuration, report median plus dispersion, and keep correctness tests enabled. Record CPU/GPU model, compiler, optimization flags, driver/runtime, and thread/block configuration. The central adversarial condition is: Construct one kernel each that isolates divergence, uncoalesced memory, low occupancy, and high arithmetic intensity; verify profiler counters match the predicted bottleneck.

F4. Required evidence. Measure kernel time, achieved bandwidth/FLOP rate, occupancy, branch efficiency, cache hit rate, and roofline arithmetic intensity. Deliver a one-command benchmark, `bench.csv`, profiler/counter capture, hardware/software manifest, and a regression test for the adversarial case. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for GPU architecture and performance. If a stable failure/crossover is found among compute-bound, bandwidth-bound, divergent, and occupancy-limited kernels identified by predicted and measured counters, implement one adaptive or corrected variant that directly targets arithmetic intensity, memory coalescing/cache behavior, branch divergence, occupancy, and launch/measurement methodology; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for GPU architecture and performance. Using the measured kernel time, achieved bandwidth/FLOP rate, occupancy, branch efficiency, cache hit rate, and roofline arithmetic intensity, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for GPU architecture and performance

- G1. Closed-book derivation/oracle. Use a roofline-style model to derive when a graphics/GPU kernel is bandwidth-bound versus compute-bound; connect arithmetic intensity, occupancy, divergence, and memory coalescing to expected throughput. Then solve this exact micro-case from scratch without opening your implementation: For a warp of eight toy lanes evaluating if $(i \bmod 3) == 0$, mark active masks on both branches. For a kernel reading consecutive versus stride-8 floats, count memory segments in a simplified 32-byte transaction model. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Profile a shader and a CUDA-style kernel; intentionally create divergence and uncoalesced accesses; explain performance counters. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally include allocation/transfer/first-launch compilation in one kernel timing but not another, then infer a kernel bottleneck from wall-clock time alone. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Write four minimal kernels, each isolating one bottleneck; capture Nsight/appropriate profiler counters and annotate why each counter changes. Then apply this stress without retuning: Construct one kernel each that isolates divergence, uncoalesced memory, low occupancy, and high arithmetic intensity; verify profiler counters match the predicted bottleneck. Report kernel time, achieved bandwidth/FLOP rate, occupancy, branch efficiency, cache hit rate, and roofline arithmetic intensity and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across compute-bound, bandwidth-bound, divergent, and occupancy-limited kernels identified by predicted and measured counters for

three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured kernel time, achieved bandwidth/FLOP rate, occupancy, branch efficiency, cache hit rate, and roofline arithmetic intensity; state one counterexample that would change the diagnosis or ranking.

- G6. Pass condition for GPU architecture and performance. Using kernel time, achieved bandwidth/FLOP rate, occupancy, branch efficiency, cache hit rate, and roofline arithmetic intensity, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 2, Unit 7. Parallel patterns for graphics

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. Map/reduce, scan, sort, prefix sums, compaction, tiled algorithms, parallel BVH/raster tasks.

A. Reading order

1. Read the primary assignment first: GPU Gems/modern vendor references as optional; PBRT Ch. 15 for wavefront rendering.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: PBRT 4e GPU wavefront chapter.
3. Keep the most useful executable/reference page open while coding: NVIDIA CUDA documentation/learning resources - <https://developer.nvidia.com/cuda>

B. Definitions, results and algorithms to know

- Work/span thinking
- stable compaction
- radix sort intuition
- Precision requirement: distinguish and justify serial, parallel scan/compaction, and queue-based implementations under balanced and highly skewed workloads. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive work/span complexity for scan, compaction, and queue-based processing; explain how a stable parallel compaction can be built from prefix sums and where synchronization/memory traffic enters the cost.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Test empty/full queues, highly skewed workloads, and very large arrays; verify correctness, determinism where promised, and scaling of synchronization/memory traffic. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement parallel compaction or prefix-sum using a GPU framework; use it in a simple particle or ray queue. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For flags [1,0,1,1,0,0,1,0], perform an exclusive scan and compaction by hand. Then repeat for a segmented scan with segment starts [1,0,0,1,0,1,0,0]. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Test empty/full queues, highly skewed workloads, and very large arrays; verify correctness, determinism where promised, and scaling of synchronization/memory traffic. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Use scan/compaction to build an active-ray or particle queue; verify output order where promised and measure speedup as the active fraction varies from 1% to 100%.
- P5 - Machine-readable evidence. Export work/span proxy, throughput, synchronization/memory traffic, active-fraction scaling, and correctness/determinism to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: serial, parallel scan/compaction, and queue-based implementations under balanced and highly skewed workloads. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Test empty/full queues, highly skewed workloads, and very large arrays; verify correctness, determinism where promised, and scaling of synchronization/memory traffic." without hand-editing

code between methods. Record work/span proxy, throughput, synchronization/memory traffic, active-fraction scaling, and correctness/determinism.

- P7 - Deliberate bug and regression drill. Intentionally omit one synchronization/barrier or use an off-by-one exclusive/inclusive scan convention, causing data-dependent compaction errors. Before running, predict the first observable failure in work/span proxy, throughput, synchronization/memory traffic, active-fraction scaling, and correctness/determinism. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive work/span complexity for scan, compaction, and queue-based processing; explain how a stable parallel compaction can be built from prefix sums and where synchronization/memory traffic enters the cost. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For flags [1,0,1,1,0,0,1,0], perform an exclusive scan and compaction by hand. Then repeat for a segmented scan with segment starts [1,0,0,1,0,1,0,0]. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement parallel compaction or prefix-sum using a GPU framework; use it in a simple particle or ray queue. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Use scan/compaction to build an active-ray or particle queue; verify output order where promised and measure speedup as the active fraction varies from 1% to 100%. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare serial, parallel scan/compaction, and queue-based implementations under balanced and highly skewed workloads. Use the same hardware, compiler/interpreter version, optimization flags, input data and thread/GPU configuration; repeat timings and report median plus a tail statistic. Record work/span proxy, throughput, synchronization/memory traffic, active-fraction scaling, and correctness/determinism. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Test empty/full queues, highly skewed workloads, and very large arrays; verify correctness, determinism where promised, and scaling of synchronization/memory traffic. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: GPU Gems/modern vendor references as optional; PBRT Ch. 15 for wavefront rendering. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Parallel patterns for graphics. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among serial, parallel scan/compaction, and queue-based implementations under balanced and highly skewed workloads. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using work/span proxy, throughput, synchronization/memory traffic, active-fraction scaling, and correctness/determinism. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if serial, parallel scan/compaction, and queue-based implementations under balanced and highly skewed workloads remain correctly matched and the performance/correctness crossover in work/span proxy, throughput, synchronization/memory traffic, active-fraction scaling, and correctness/determinism survives repeated runs on stated hardware/build settings; a faster incorrect or differently configured implementation does not count.

F. Concrete research project - Parallel patterns for graphics

F1. Falsifiable question. Under the controlled failure regime "Test empty/full queues, highly skewed workloads, and very large arrays; verify correctness, determinism where promised, and scaling of synchronization/memory traffic.", characterize the ranking or regime transition across serial, parallel scan/compaction, and queue-based implementations under balanced and highly skewed workloads. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Use scan/compaction to build an active-ray or particle queue; verify output order where promised and measure speedup as the active fraction varies from 1% to 100%. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Benchmark at least 5 problem sizes after warm-up; use at least 20 timed repeats per configuration, report median plus dispersion, and keep correctness tests enabled. Record CPU/GPU model, compiler, optimization flags, driver/runtime, and thread/block configuration. The central adversarial condition is: Test empty/full queues, highly skewed workloads, and very large arrays; verify correctness, determinism where promised, and scaling of synchronization/memory traffic.

F4. Required evidence. Measure work/span proxy, throughput, synchronization/memory traffic, active-fraction scaling, and correctness/determinism. Deliver a one-command benchmark, `bench.csv`, profiler/counter capture, hardware/software manifest, and a regression test for the adversarial case. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Parallel patterns for graphics. If a stable failure/crossover is found among serial, parallel scan/compaction, and queue-based implementations under balanced and highly skewed workloads, implement one adaptive or corrected variant that directly targets memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Parallel patterns for graphics. Using the measured work/span proxy, throughput, synchronization/memory traffic, active-fraction scaling, and correctness/determinism, Pass only when all correctness tests remain green, the performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Parallel patterns for graphics

- G1. Closed-book derivation/oracle. Derive work/span complexity for scan, compaction, and queue-based processing; explain how a stable parallel compaction can be built from prefix sums and where synchronization/memory traffic enters the cost. Then solve this exact micro-case from scratch without opening your implementation: For flags `[1,0,1,1,0,0,1,0]`, perform an exclusive scan and compaction by hand. Then repeat for a segmented scan with segment starts `[1,0,0,1,0,1,0,0]`. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement parallel compaction or prefix-sum using a GPU framework; use it in a simple particle or ray queue. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally omit one synchronization/barrier or use an off-by-one exclusive/inclusive scan convention, causing data-dependent compaction errors. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Use scan/compaction to build an active-ray or particle queue; verify output order where promised and measure speedup as the active fraction varies from 1% to 100%. Then apply this stress without retuning: Test empty/full queues, highly skewed workloads, and very large arrays; verify correctness, determinism where promised, and scaling of synchronization/memory traffic. Report work/span proxy, throughput, synchronization/memory traffic, active-fraction scaling, and correctness/determinism and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across serial, parallel scan/compaction, and queue-based implementations under balanced and highly skewed workloads for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured work/span proxy, throughput, synchronization/memory traffic, active-fraction scaling, and correctness/determinism; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Parallel patterns for graphics. Using work/span proxy, throughput, synchronization/memory traffic, active-fraction scaling, and correctness/determinism, Pass only when all correctness tests remain green, the

performance conclusion survives repeated timings, and profiler/complexity evidence identifies the bottleneck; a single wall-clock number is insufficient. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 2 Closeout: Project, Review and Mastery Gate

Volume project. Build a reusable C++/Python graphics utility repository with tests, profiling, Eigen, mesh IO, a BVH and one GPU demo. The goal is engineering discipline, not a full engine.

Cumulative review

- Profile one complete graphics workload and attribute time/memory to data layout, traversal, parallelism and GPU bottlenecks.
- Demonstrate one correctness bug caused by floating-point/ownership/concurrency assumptions and preserve it as a regression test.
- Compare CPU reference and GPU implementation on identical inputs and document any precision/order differences.
- Create a reproducible benchmark CLI that records hardware/software/config and regenerates the central performance figure.
- Write a systems postmortem distinguishing algorithmic complexity, memory behavior, occupancy/divergence and implementation overhead.

Mastery gate before moving on

- Profile and correctly diagnose one CPU and one GPU bottleneck using measured counters/timings rather than intuition alone.
- Demonstrate a tested spatial structure plus a clean, reproducible experiment harness that reruns from a fresh environment.
- Diagnose one floating-point robustness bug and one memory/parallelism bug from saved failing cases.

Volume 3. Images, Sampling, Color, and Visual Perception

Purpose. Understand the image as a sampled signal and the human visual system as the final observer.

Suggested duration. 10-12 weeks

Prerequisites. Volume 1 modules on vectors, transforms, probability and Fourier analysis are helpful; basic programming from Volume 2.

Typical module pace. 1.7-2.0 weeks per module (6 modules total).

Primary references for this volume

- Fundamentals of Computer Graphics, Ch. 3, 10, 18-20.
- PBRT 4e, Ch. 4.5-4.6 and 8.
- Real-Time Rendering, 4e, Ch. 6, 8 and 12 selections.

Online spine

Berkeley CS184/284A Spring 2026 course: <https://cs184.eecs.berkeley.edu/sp26/>

PBRT 4th edition - full online book: <https://pbr-book.org/4ed/contents>

Canonical paper track

- Cook (1986), Stochastic Sampling in Computer Graphics.
- Mitchell & Netravali (1988), Reconstruction Filters in Computer Graphics.
- Reinhard et al. (2002), Photographic Tone Reproduction for Digital Images.
- Mantiuk et al. and modern HDR/perceptual display papers as follow-up reading.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Volume 3, Unit 1. Raster images and alpha

Recommended time. 1.7-2.0 weeks (about 25-40 focused hours depending on background).

What you are learning. Pixels, image coordinates, dynamic range, linear vs nonlinear values, premultiplied alpha and compositing.

A. Reading order

1. Read the primary assignment first: FCG 5e Ch. 3, especially §3.4 on alpha compositing. Do not mix tone-reproduction material into this unit; HDR/tone mapping is Unit 3.4.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 introductory image/color material.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Porter-Duff over operator
- premultiplied alpha
- linear-light arithmetic
- Precision requirement: distinguish and justify straight-alpha vs premultiplied-alpha compositing, and gamma-encoded vs linear-light filtering/compositing. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the Porter-Duff over operator in both straight-alpha and premultiplied-alpha form; prove associativity properties of premultiplied compositing and explain why filtering straight-alpha colors creates halos.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Repeatedly composite and resample thin bright edges in linear versus gamma-encoded space and with straight versus premultiplied alpha; look for halos and energy loss. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement compositing correctly in linear light; demonstrate halos from incorrect non-premultiplied processing. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Composite red RGBA=(1,0,0,0.5) over blue (0,0,1,0.5) over black using straight and premultiplied alpha. Then repeat once incorrectly in sRGB values to expose the linear-light error. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Repeatedly composite and resample thin bright edges in linear versus gamma-encoded space and with straight versus premultiplied alpha; look for halos and energy loss. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create a compositing torture test with thin bright edges, semitransparent colors, repeated resampling, and 20-layer stacks; save difference images against a correct premultiplied linear-light reference.
- P5 - Machine-readable evidence. Export linear-light pixel error, halo energy near edges, compositing associativity error, and accumulated error over repeated operations to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: straight-alpha vs premultiplied-alpha compositing, and gamma-encoded vs linear-light filtering/compositing. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Repeatedly composite and resample thin bright edges in linear versus gamma-encoded space and with straight versus premultiplied alpha; look for halos and energy loss." without hand-editing code between methods. Record linear-light pixel error, halo energy near edges, compositing associativity error, and accumulated error over repeated operations.
- P7 - Deliberate bug and regression drill. Intentionally filter/composite gamma-encoded straight-alpha colors as though they were linear premultiplied values. Before running, predict the first observable failure in linear-light pixel error, halo energy near edges, compositing associativity error, and accumulated error over repeated operations. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the Porter-Duff over operator in both straight-alpha and premultiplied-alpha form; prove associativity properties of premultiplied compositing and explain why filtering straight-alpha colors creates halos. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Composite red RGBA=(1,0,0,0.5) over blue (0,0,1,0.5) over black using straight and premultiplied alpha. Then repeat once incorrectly in sRGB values to expose the linear-light error. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement compositing correctly in linear light; demonstrate halos from incorrect non-premultiplied processing. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create a compositing torture test with thin bright edges, semitransparent colors, repeated resampling, and 20-layer stacks; save difference images against a correct premultiplied linear-light reference. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare straight-alpha vs premultiplied-alpha compositing, and gamma-encoded vs linear-light filtering/compositing. Use the same source image/signal, color space, output resolution and sampling/exposure budget; for viewer studies, use the same stimuli and participant protocol. Record linear-light pixel error, halo energy near edges, compositing associativity error, and accumulated error over repeated operations. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Repeatedly composite and resample thin bright edges in linear versus gamma-encoded space and with straight versus premultiplied alpha; look for halos and energy loss. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG 5e Ch. 3, especially §3.4 on alpha compositing. Do not mix tone-reproduction material into this unit; HDR/tone mapping is Unit 3.4. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Raster images and alpha. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among straight-alpha vs premultiplied-alpha compositing, and gamma-encoded vs linear-light filtering/compositing. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using linear-light pixel error, halo energy near edges, compositing associativity error, and accumulated error over repeated operations. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if straight-alpha vs premultiplied-alpha compositing, and gamma-encoded vs linear-light filtering/compositing show a repeatable improvement or crossover in linear-light pixel error, halo energy near edges, compositing associativity error, and accumulated error over repeated operations across multiple images/signals/scales and the effect survives linear-light/color-space and reference-quality controls relevant to the unit.

F. Concrete research project - Raster images and alpha

- F1. Falsifiable question. Under the controlled failure regime "Repeatedly composite and resample thin bright edges in linear versus gamma-encoded space and with straight versus premultiplied alpha; look for halos and energy loss.", which alternative in straight-alpha vs premultiplied-alpha compositing, and gamma-encoded vs linear-light filtering/compositing is preferable, and is the change explained by coverage/visibility conventions, precision, temporal validity, and quality-per-frame-time tradeoffs? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create a compositing torture test with thin bright edges, semitransparent colors, repeated resampling, and 20-layer stacks; save difference images against a correct premultiplied linear-light reference. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use a synthetic image with known ground truth plus at least two natural/HDR images. Save full-frame metrics and diagnostic crops/difference images; all filtering/compositing must be done in explicitly stated linear or encoded color spaces. The central adversarial condition is: Repeatedly composite and resample thin

bright edges in linear versus gamma-encoded space and with straight versus premultiplied alpha; look for halos and energy loss.

F4. Required evidence. Measure linear-light pixel error, halo energy near edges, compositing associativity error, and accumulated error over repeated operations. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Raster images and alpha. If a stable failure/crossover is found among straight-alpha vs premultiplied-alpha compositing, and gamma-encoded vs linear-light filtering/compositing, implement one adaptive or corrected variant that directly targets coverage/visibility conventions, precision, temporal validity, and quality-per-frame-time tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Raster images and alpha. Using the measured linear-light pixel error, halo energy near edges, compositing associativity error, and accumulated error over repeated operations, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Raster images and alpha

- G1. Closed-book derivation/oracle. Derive the Porter-Duff over operator in both straight-alpha and premultiplied-alpha form; prove associativity properties of premultiplied compositing and explain why filtering straight-alpha colors creates halos. Then solve this exact micro-case from scratch without opening your implementation: Composite red RGBA=(1,0,0,0.5) over blue (0,0,1,0.5) over black using straight and premultiplied alpha. Then repeat once incorrectly in sRGB values to expose the linear-light error. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement compositing correctly in linear light; demonstrate halos from incorrect non-premultiplied processing. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally filter/composite gamma-encoded straight-alpha colors as though they were linear premultiplied values. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create a compositing torture test with thin bright edges, semitransparent colors, repeated resampling, and 20-layer stacks; save difference images against a correct premultiplied linear-light reference. Then apply this stress without retuning: Repeatedly composite and resample thin bright edges in linear versus gamma-encoded space and with straight versus premultiplied alpha; look for halos and energy loss. Report linear-light pixel error, halo energy near edges, compositing associativity error, and accumulated error over repeated operations and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among straight-alpha vs premultiplied-alpha compositing, and gamma-encoded vs linear-light filtering/compositing for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured linear-light pixel error, halo energy near edges, compositing associativity error, and accumulated error over repeated operations; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Raster images and alpha. Using linear-light pixel error, halo energy near edges, compositing associativity error, and accumulated error over repeated operations, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 3, Unit 2. Sampling and reconstruction in 2D

Recommended time. 1.7-2.0 weeks (about 25-40 focused hours depending on background).

What you are learning. Aliasing, resampling, image pyramids, EWA idea, mipmapping.

A. Reading order

1. Read the primary assignment first: FCG Ch. 10-11; PBRT Ch. 8; RTR Ch. 6.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 sampling and texture mapping lectures.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Nyquist condition
- separable filtering
- prefiltering vs reconstruction
- Precision requirement: distinguish and justify isotropic mip/trilinear filtering vs anisotropic footprint filtering against a supersampled reference. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive separable image reconstruction from a 2D convolution kernel and analyze minification of a rotated sinusoid; compare isotropic filtering with an anisotropic footprint in frequency space.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Resize rotated high-frequency patterns through several scale factors; compare isotropic and anisotropic filters against a supersampled reference. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Build image resize and texture minification experiments with reference supersampling. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Downsample an 8x8 one-pixel checkerboard to 4x4 by point sampling, 2x2 box prefiltering, and a separable tent filter. Predict which outputs alias and compute one output pixel exactly for each method. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Resize rotated high-frequency patterns through several scale factors; compare isotropic and anisotropic filters against a supersampled reference. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement an elliptical/anisotropic footprint approximation and compare it with trilinear mipmapping on grazing checkerboards under equal texture-sample budgets.
- P5 - Machine-readable evidence. Export RMSE/SSIM to supersampled reference, alias energy, blur/MTF loss, texture samples per pixel, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: isotropic mip/trilinear filtering vs anisotropic footprint filtering against a supersampled reference. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Resize rotated high-frequency patterns through several scale factors; compare isotropic and anisotropic filters against a supersampled reference." without hand-editing code between methods. Record RMSE/SSIM to supersampled reference, alias energy, blur/MTF loss, texture samples per pixel, and runtime.
- P7 - Deliberate bug and regression drill. Intentionally compare reconstruction filters after downsampling without the corresponding prefilter, so aliasing is misattributed to reconstruction. Before running, predict the first observable failure in RMSE/SSIM to supersampled reference, alias energy, blur/MTF loss, texture samples per pixel, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive separable image reconstruction from a 2D convolution kernel and analyze minification of a rotated sinusoid; compare isotropic filtering with an anisotropic footprint in frequency space. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Downsample an 8x8 one-pixel checkerboard to 4x4 by point sampling, 2x2 box prefiltering, and a separable tent filter. Predict which outputs alias and compute one output pixel exactly for each

method. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Build image resize and texture minification experiments with reference supersampling. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement an elliptical/anisotropic footprint approximation and compare it with trilinear mipmapping on grazing checkerboards under equal texture-sample budgets. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare isotropic mip/trilinear filtering vs anisotropic footprint filtering against a supersampled reference. Use the same source image/signal, color space, output resolution and sampling/exposure budget; for viewer studies, use the same stimuli and participant protocol. Record RMSE/SSIM to supersampled reference, alias energy, blur/MTF loss, texture samples per pixel, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Resize rotated high-frequency patterns through several scale factors; compare isotropic and anisotropic filters against a supersampled reference. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: FCG Ch. 10-11; PBRT Ch. 8; RTR Ch. 6. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Sampling and reconstruction in 2D. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among isotropic mip/trilinear filtering vs anisotropic footprint filtering against a supersampled reference. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using RMSE/SSIM to supersampled reference, alias energy, blur/MTF loss, texture samples per pixel, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if isotropic mip/trilinear filtering vs anisotropic footprint filtering against a supersampled reference show a repeatable improvement or crossover in RMSE/SSIM to supersampled reference, alias energy, blur/MTF loss, texture samples per pixel, and runtime across multiple images/signals/scales and the effect survives linear-light/color-space and reference-quality controls relevant to the unit.

F. Concrete research project - Sampling and reconstruction in 2D

F1. Falsifiable question. Under the controlled failure regime "Resize rotated high-frequency patterns through several scale factors; compare isotropic and anisotropic filters against a supersampled reference.", which alternative in isotropic mip/trilinear filtering vs anisotropic footprint filtering against a supersampled reference is preferable, and is the change explained by projected signal bandwidth, prefiltering, reconstruction footprint, and anisotropic aliasing? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement an elliptical/anisotropic footprint approximation and compare it with trilinear mipmapping on grazing checkerboards under equal texture-sample budgets. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a synthetic image with known ground truth plus at least two natural/HDR images. Save full-frame metrics and diagnostic crops/difference images; all filtering/compositing must be done in explicitly stated linear or encoded color spaces. The central adversarial condition is: Resize rotated high-frequency patterns through several scale factors; compare isotropic and anisotropic filters against a supersampled reference.

F4. Required evidence. Measure RMSE/SSIM to supersampled reference, alias energy, blur/MTF loss, texture samples per pixel, and runtime. Deliver reference outputs plus difference/diagnostic images, 'results.csv', one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Sampling and reconstruction in 2D. If a stable failure/crossover is found among isotropic mip/trilinear filtering vs anisotropic footprint filtering against a supersampled reference, implement one adaptive or corrected variant that directly targets projected signal bandwidth, prefiltering, reconstruction footprint, and anisotropic aliasing; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Sampling and reconstruction in 2D. Using the measured RMSE/SSIM to supersampled reference, alias energy, blur/MTF loss, texture samples per pixel, and runtime, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Sampling and reconstruction in 2D

- G1. Closed-book derivation/oracle. Derive separable image reconstruction from a 2D convolution kernel and analyze minification of a rotated sinusoid; compare isotropic filtering with an anisotropic footprint in frequency space. Then solve this exact micro-case from scratch without opening your implementation: Downsample an 8x8 one-pixel checkerboard to 4x4 by point sampling, 2x2 box prefiltering, and a separable tent filter. Predict which outputs alias and compute one output pixel exactly for each method. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Build image resize and texture minification experiments with reference supersampling. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally compare reconstruction filters after downsampling without the corresponding prefilter, so aliasing is misattributed to reconstruction. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement an elliptical/anisotropic footprint approximation and compare it with trilinear mipmapping on grazing checkerboards under equal texture-sample budgets. Then apply this stress without retuning: Resize rotated high-frequency patterns through several scale factors; compare isotropic and anisotropic filters against a supersampled reference. Report RMSE/SSIM to supersampled reference, alias energy, blur/MTF loss, texture samples per pixel, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among isotropic mip/trilinear filtering vs anisotropic footprint filtering against a supersampled reference for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured RMSE/SSIM to supersampled reference, alias energy, blur/MTF loss, texture samples per pixel, and runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Sampling and reconstruction in 2D. Using RMSE/SSIM to supersampled reference, alias energy, blur/MTF loss, texture samples per pixel, and runtime, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 3, Unit 3. Color science

Recommended time. 1.7-2.0 weeks (about 25-40 focused hours depending on background).

What you are learning. Spectra, tristimulus values, CIE XYZ, RGB primaries, chromaticity, adaptation, color appearance.

A. Reading order

1. Read the primary assignment first: FCG Ch. 18; PBRT Ch. 4.5-4.6; RTR Ch. 8.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 2026 color science lectures.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Metamers
- spectral power distribution
- color matching functions
- gamut

- Precision requirement: distinguish and justify spectral-to-XYZ ground truth vs RGB approximations across metameric, narrowband, and out-of-gamut stimuli. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive tristimulus values from a spectral power distribution and color-matching functions, then show how a linear RGB space is obtained from XYZ; construct the linear-algebra explanation of metamerism.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use narrow spectral peaks, metameric pairs, and colors near/outside the output gamut; compare RGB approximations with spectral-to-XYZ integration. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Convert spectral samples to XYZ/RGB; visualize gamut and metamerism; compare RGB working spaces. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Using a toy three-wavelength spectrum and a supplied 3x3 color-matching matrix, compute XYZ and linear RGB for two different spectra that map to nearly the same tristimulus values; explain the metameric ambiguity. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use narrow spectral peaks, metameric pairs, and colors near/outside the output gamut; compare RGB approximations with spectral-to-XYZ integration. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Write a spectral notebook with several synthetic SPDs, integrate to XYZ, convert to two RGB spaces, and visualize gamut clipping and metameric pairs.
- P5 - Machine-readable evidence. Export Delta E / XYZ error, gamut clipping fraction, spectral-metamer mismatch, and adaptation/working-space sensitivity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: spectral-to-XYZ ground truth vs RGB approximations across metameric, narrowband, and out-of-gamut stimuli. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use narrow spectral peaks, metameric pairs, and colors near/outside the output gamut; compare RGB approximations with spectral-to-XYZ integration." without hand-editing code between methods. Record Delta E / XYZ error, gamut clipping fraction, spectral-metamer mismatch, and adaptation/working-space sensitivity.
- P7 - Deliberate bug and regression drill. Intentionally apply an RGB/XYZ conversion matrix in the wrong encoding/white-point convention and clip before the intended gamut mapping step. Before running, predict the first observable failure in Delta E / XYZ error, gamut clipping fraction, spectral-metamer mismatch, and adaptation/working-space sensitivity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive tristimulus values from a spectral power distribution and color-matching functions, then show how a linear RGB space is obtained from XYZ; construct the linear-algebra explanation of metamerism. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Using a toy three-wavelength spectrum and a supplied 3x3 color-matching matrix, compute XYZ and linear RGB for two different spectra that map to nearly the same tristimulus values; explain the metameric ambiguity. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Convert spectral samples to XYZ/RGB; visualize gamut and metamerism; compare RGB working spaces. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Write a spectral notebook with several synthetic SPDs, integrate to XYZ, convert to two RGB spaces, and visualize gamut clipping and metameric pairs. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare spectral-to-XYZ ground truth vs RGB approximations across metameric, narrowband, and out-of-gamut stimuli. Use the same source image/signal, color space, output resolution and sampling/exposure budget; for viewer studies, use the same stimuli and participant protocol. Record Delta E / XYZ error,

gamut clipping fraction, spectral-metamer mismatch, and adaptation/working-space sensitivity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use narrow spectral peaks, metameric pairs, and colors near/outside the output gamut; compare RGB approximations with spectral-to-XYZ integration. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: FCG Ch. 18; PBRT Ch. 4.5-4.6; RTR Ch. 8. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Color science. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among spectral-to-XYZ ground truth vs RGB approximations across metameric, narrowband, and out-of-gamut stimuli. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using Delta E / XYZ error, gamut clipping fraction, spectral-metamer mismatch, and adaptation/working-space sensitivity. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if spectral-to-XYZ ground truth vs RGB approximations across metameric, narrowband, and out-of-gamut stimuli show a repeatable improvement or crossover in Delta E / XYZ error, gamut clipping fraction, spectral-metamer mismatch, and adaptation/working-space sensitivity across multiple images/signals/scales and the effect survives linear-light/color-space and reference-quality controls relevant to the unit.

F. Concrete research project - Color science

F1. Falsifiable question. Under the controlled failure regime "Use narrow spectral peaks, metameric pairs, and colors near/outside the output gamut; compare RGB approximations with spectral-to-XYZ integration.", which alternative in spectral-to-XYZ ground truth vs RGB approximations across metameric, narrowband, and out-of-gamut stimuli is preferable, and is the change explained by sampling/color-space assumptions, signal statistics, reconstruction bias, and perceptual visibility? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Write a spectral notebook with several synthetic SPDs, integrate to XYZ, convert to two RGB spaces, and visualize gamut clipping and metameric pairs. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a synthetic image with known ground truth plus at least two natural/HDR images. Save full-frame metrics and diagnostic crops/difference images; all filtering/compositing must be done in explicitly stated linear or encoded color spaces. The central adversarial condition is: Use narrow spectral peaks, metameric pairs, and colors near/outside the output gamut; compare RGB approximations with spectral-to-XYZ integration.

F4. Required evidence. Measure Delta E / XYZ error, gamut clipping fraction, spectral-metamer mismatch, and adaptation/working-space sensitivity. Deliver reference outputs plus difference/diagnostic images, 'results.csv', one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Color science. If a stable failure/crossover is found among spectral-to-XYZ ground truth vs RGB approximations across metameric, narrowband, and out-of-gamut stimuli, implement one adaptive or corrected variant that directly targets sampling/color-space assumptions, signal statistics, reconstruction bias, and perceptual visibility; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Color science. Using the measured Delta E / XYZ error, gamut clipping fraction, spectral-metamer mismatch, and adaptation/working-space sensitivity, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Color science

- G1. Closed-book derivation/oracle. Derive tristimulus values from a spectral power distribution and color-matching functions, then show how a linear RGB space is obtained from XYZ; construct the linear-algebra explanation of

metamerism. Then solve this exact micro-case from scratch without opening your implementation: Using a toy three-wavelength spectrum and a supplied 3x3 color-matching matrix, compute XYZ and linear RGB for two different spectra that map to nearly the same tristimulus values; explain the metameric ambiguity. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Convert spectral samples to XYZ/RGB; visualize gamut and metamerism; compare RGB working spaces. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally apply an RGB/XYZ conversion matrix in the wrong encoding/white-point convention and clip before the intended gamut mapping step. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Write a spectral notebook with several synthetic SPDs, integrate to XYZ, convert to two RGB spaces, and visualize gamut clipping and metameric pairs. Then apply this stress without retuning: Use narrow spectral peaks, metameric pairs, and colors near/outside the output gamut; compare RGB approximations with spectral-to-XYZ integration. Report Delta E / XYZ error, gamut clipping fraction, spectral-metamer mismatch, and adaptation/working-space sensitivity and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among spectral-to-XYZ ground truth vs RGB approximations across metameric, narrowband, and out-of-gamut stimuli for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured Delta E / XYZ error, gamut clipping fraction, spectral-metamer mismatch, and adaptation/working-space sensitivity; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Color science. Using Delta E / XYZ error, gamut clipping fraction, spectral-metamer mismatch, and adaptation/working-space sensitivity, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 3, Unit 4. Tone mapping and HDR

Recommended time. 1.7-2.0 weeks (about 25-40 focused hours depending on background).

What you are learning. Scene-referred vs display-referred values, exposure, global/local tone mapping.

A. Reading order

1. Read the primary assignment first: FCG Ch. 20; RTR Ch. 12 selections.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: HDR imaging tutorials from graphics courses.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Dynamic range, photographic tone mapping, gradient-domain idea
- Precision requirement: distinguish and justify global Reinhard-style mapping vs local/gradient-domain approaches across exposure and display peak luminance. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a global photographic/Reinhard-style tone-mapping curve, analyze its limiting behavior and exposure dependence, and show why applying it independently to gamma-encoded channels can distort color.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use scenes containing bright speculars, deep shadows, saturated colors, and local contrast; test several display peak luminances and exposures. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement exposure, Reinhard-style, and local tone mapping; evaluate detail preservation and halos. Keep a minimal reference path even if you later optimize it.

- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Map luminances [0.01,0.1,1,10,100] with a simple Reinhard operator $L/(1+L)$ at two exposure scales. Compute ordering, local contrast ratios, and clipping if naively mapped to 8-bit. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use scenes containing bright speculars, deep shadows, saturated colors, and local contrast; test several display peak luminances and exposures. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build an HDR test gallery and compute local contrast/halo statistics while sweeping exposure and display peak luminance for at least three operators.
- P5 - Machine-readable evidence. Export local contrast preservation, clipping fraction, halo/gradient reversal count, color shift, and display-luminance sensitivity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: global Reinhard-style mapping vs local/gradient-domain approaches across exposure and display peak luminance. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use scenes containing bright speculars, deep shadows, saturated colors, and local contrast; test several display peak luminances and exposures." without hand-editing code between methods. Record local contrast preservation, clipping fraction, halo/gradient reversal count, color shift, and display-luminance sensitivity.
- P7 - Deliberate bug and regression drill. Intentionally apply exposure or local adaptation after the tone-mapping nonlinearity, or compute luminance in gamma-encoded RGB. Before running, predict the first observable failure in local contrast preservation, clipping fraction, halo/gradient reversal count, color shift, and display-luminance sensitivity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a global photographic/Reinhard-style tone-mapping curve, analyze its limiting behavior and exposure dependence, and show why applying it independently to gamma-encoded channels can distort color. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Map luminances [0.01,0.1,1,10,100] with a simple Reinhard operator $L/(1+L)$ at two exposure scales. Compute ordering, local contrast ratios, and clipping if naively mapped to 8-bit. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement exposure, Reinhard-style, and local tone mapping; evaluate detail preservation and halos. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build an HDR test gallery and compute local contrast/halo statistics while sweeping exposure and display peak luminance for at least three operators. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare global Reinhard-style mapping vs local/gradient-domain approaches across exposure and display peak luminance. Use the same source image/signal, color space, output resolution and sampling/exposure budget; for viewer studies, use the same stimuli and participant protocol. Record local contrast preservation, clipping fraction, halo/gradient reversal count, color shift, and display-luminance sensitivity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use scenes containing bright speculars, deep shadows, saturated colors, and local contrast; test several display peak luminances and exposures. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 20; RTR Ch. 12 selections. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Tone mapping and HDR. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among global Reinhard-style mapping vs local/gradient-domain approaches across exposure and display peak luminance. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using local contrast preservation, clipping fraction, halo/gradient reversal count, color shift, and display-luminance sensitivity.

Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if global Reinhard-style mapping vs local/gradient-domain approaches across exposure and display peak luminance show a repeatable improvement or crossover in local contrast preservation, clipping fraction, halo/gradient reversal count, color shift, and display-luminance sensitivity across multiple images/signals/scales and the effect survives linear-light/color-space and reference-quality controls relevant to the unit.

F. Concrete research project - Tone mapping and HDR

- F1. Falsifiable question. Under the controlled failure regime "Use scenes containing bright speculars, deep shadows, saturated colors, and local contrast; test several display peak luminances and exposures.", which alternative in global Reinhard-style mapping vs local/gradient-domain approaches across exposure and display peak luminance is preferable, and is the change explained by sampling/color-space assumptions, signal statistics, reconstruction bias, and perceptual visibility? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build an HDR test gallery and compute local contrast/halo statistics while sweeping exposure and display peak luminance for at least three operators. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use a synthetic image with known ground truth plus at least two natural/HDR images. Save full-frame metrics and diagnostic crops/difference images; all filtering/compositing must be done in explicitly stated linear or encoded color spaces. The central adversarial condition is: Use scenes containing bright speculars, deep shadows, saturated colors, and local contrast; test several display peak luminances and exposures.
- F4. Required evidence. Measure local contrast preservation, clipping fraction, halo/gradient reversal count, color shift, and display-luminance sensitivity. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Tone mapping and HDR. If a stable failure/crossover is found among global Reinhard-style mapping vs local/gradient-domain approaches across exposure and display peak luminance, implement one adaptive or corrected variant that directly targets sampling/color-space assumptions, signal statistics, reconstruction bias, and perceptual visibility; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Tone mapping and HDR. Using the measured local contrast preservation, clipping fraction, halo/gradient reversal count, color shift, and display-luminance sensitivity, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Tone mapping and HDR

- G1. Closed-book derivation/oracle. Derive a global photographic/Reinhard-style tone-mapping curve, analyze its limiting behavior and exposure dependence, and show why applying it independently to gamma-encoded channels can distort color. Then solve this exact micro-case from scratch without opening your implementation: Map luminances [0.01,0.1,1,10,100] with a simple Reinhard operator $L/(1+L)$ at two exposure scales. Compute ordering, local contrast ratios, and clipping if naively mapped to 8-bit. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement exposure, Reinhard-style, and local tone mapping; evaluate detail preservation and halos. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally apply exposure or local adaptation after the tone-mapping nonlinearity, or compute luminance in gamma-encoded RGB. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Build an HDR test gallery and compute local contrast/halo statistics while sweeping exposure and display peak luminance for at least

three operators. Then apply this stress without retuning: Use scenes containing bright speculars, deep shadows, saturated colors, and local contrast; test several display peak luminances and exposures. Report local contrast preservation, clipping fraction, halo/gradient reversal count, color shift, and display-luminance sensitivity and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method-selection defense. Using your own measured results, choose among global Reinhard-style mapping vs local/gradient-domain approaches across exposure and display peak luminance for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured local contrast preservation, clipping fraction, halo/gradient reversal count, color shift, and display-luminance sensitivity; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Tone mapping and HDR. Using local contrast preservation, clipping fraction, halo/gradient reversal count, color shift, and display-luminance sensitivity, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 3, Unit 5. Visual perception for graphics

Recommended time. 1.7-2.0 weeks (about 25-40 focused hours depending on background).

What you are learning. Contrast sensitivity, spatial frequency, color perception, motion/temporal sensitivity, picture perception.

A. Reading order

1. Read the primary assignment first: FCG Ch. 19.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Visualization/perception lectures from Munzner materials as optional.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Weber-like behavior
- contrast sensitivity function
- perceptual uniformity as an approximation
- Precision requirement: distinguish and justify pixel error metrics vs simple perceptual/contrast-aware predictors on pairs deliberately matched in RMSE. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive or reconstruct one contrast-sensitivity/perceptual-threshold model used in graphics and explain which variables are perceptual rather than radiometric; design a forced-choice test that could falsify your assumption.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Construct image pairs with equal pixel error but very different visibility due to masking/edge alignment; check whether the chosen metric ranks them sensibly. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Design psychophysics-inspired demos for Mach bands, simultaneous contrast, motion aliasing. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Construct two 1D eight-pixel error patterns with identical MSE: one distributes error uniformly, the other puts it at a sharp edge. Compute MSE and reason which error should be more visible under contrast masking/edge sensitivity. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Construct image pairs with equal pixel error but very different visibility due to masking/edge alignment; check whether the chosen metric ranks them sensibly. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement a small two-alternative forced-choice experiment (even with a few pilot viewers) comparing two artifacts matched in RMSE but different in edge alignment or masking.

- P5 - Machine-readable evidence. Export viewer/task accuracy, preference/visibility rate, metric rank correlation, and confidence intervals to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: pixel error metrics vs simple perceptual/contrast-aware predictors on pairs deliberately matched in RMSE. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Construct image pairs with equal pixel error but very different visibility due to masking/edge alignment; check whether the chosen metric ranks them sensibly." without hand-editing code between methods. Record viewer/task accuracy, preference/visibility rate, metric rank correlation, and confidence intervals.
- P7 - Deliberate bug and regression drill. Intentionally compute the pixel metric in a different color/scale domain for two methods and infer perceptual quality from that mismatch. Before running, predict the first observable failure in viewer/task accuracy, preference/visibility rate, metric rank correlation, and confidence intervals. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive or reconstruct one contrast-sensitivity/perceptual-threshold model used in graphics and explain which variables are perceptual rather than radiometric; design a forced-choice test that could falsify your assumption. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Construct two 1D eight-pixel error patterns with identical MSE: one distributes error uniformly, the other puts it at a sharp edge. Compute MSE and reason which error should be more visible under contrast masking/edge sensitivity. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Design psychophysics-inspired demos for Mach bands, simultaneous contrast, motion aliasing. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement a small two-alternative forced-choice experiment (even with a few pilot viewers) comparing two artifacts matched in RMSE but different in edge alignment or masking. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare pixel error metrics vs simple perceptual/contrast-aware predictors on pairs deliberately matched in RMSE. Use the same source image/signal, color space, output resolution and sampling/exposure budget; for viewer studies, use the same stimuli and participant protocol. Record viewer/task accuracy, preference/visibility rate, metric rank correlation, and confidence intervals. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Construct image pairs with equal pixel error but very different visibility due to masking/edge alignment; check whether the chosen metric ranks them sensibly. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 19. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Visual perception for graphics. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among pixel error metrics vs simple perceptual/contrast-aware predictors on pairs deliberately matched in RMSE. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using viewer/task accuracy, preference/visibility rate, metric rank correlation, and confidence intervals. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if pixel error metrics vs simple perceptual/contrast-aware predictors on pairs deliberately matched in RMSE show a repeatable improvement or crossover in viewer/task accuracy, preference/visibility rate, metric rank correlation, and confidence intervals across multiple images/signals/scales and the effect survives linear-light/color-space and reference-quality controls relevant to the unit.

F. Concrete research project - Visual perception for graphics

- F1. Falsifiable question. Under the controlled failure regime "Construct image pairs with equal pixel error but very different visibility due to masking/edge alignment; check whether the chosen metric ranks them sensibly.", which

alternative in pixel error metrics vs simple perceptual/contrast-aware predictors on pairs deliberately matched in RMSE is preferable, and is the change explained by sampling/color-space assumptions, signal statistics, reconstruction bias, and perceptual visibility? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement a small two-alternative forced-choice experiment (even with a few pilot viewers) comparing two artifacts matched in RMSE but different in edge alignment or masking. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a synthetic image with known ground truth plus at least two natural/HDR images. Save full-frame metrics and diagnostic crops/difference images; all filtering/compositing must be done in explicitly stated linear or encoded color spaces. The central adversarial condition is: Construct image pairs with equal pixel error but very different visibility due to masking/edge alignment; check whether the chosen metric ranks them sensibly.

F4. Required evidence. Measure viewer/task accuracy, preference/visibility rate, metric rank correlation, and confidence intervals. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Visual perception for graphics. If a stable failure/crossover is found among pixel error metrics vs simple perceptual/contrast-aware predictors on pairs deliberately matched in RMSE, implement one adaptive or corrected variant that directly targets sampling/color-space assumptions, signal statistics, reconstruction bias, and perceptual visibility; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Visual perception for graphics. Using the measured viewer/task accuracy, preference/visibility rate, metric rank correlation, and confidence intervals, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Visual perception for graphics

- G1. Closed-book derivation/oracle. Derive or reconstruct one contrast-sensitivity/perceptual-threshold model used in graphics and explain which variables are perceptual rather than radiometric; design a forced-choice test that could falsify your assumption. Then solve this exact micro-case from scratch without opening your implementation: Construct two 1D eight-pixel error patterns with identical MSE: one distributes error uniformly, the other puts it at a sharp edge. Compute MSE and reason which error should be more visible under contrast masking/edge sensitivity. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Design psychophysics-inspired demos for Mach bands, simultaneous contrast, motion aliasing. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally compute the pixel metric in a different color/scale domain for two methods and infer perceptual quality from that mismatch. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement a small two-alternative forced-choice experiment (even with a few pilot viewers) comparing two artifacts matched in RMSE but different in edge alignment or masking. Then apply this stress without retuning: Construct image pairs with equal pixel error but very different visibility due to masking/edge alignment; check whether the chosen metric ranks them sensibly. Report viewer/task accuracy, preference/visibility rate, metric rank correlation, and confidence intervals and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among pixel error metrics vs simple perceptual/contrast-aware predictors on pairs deliberately matched in RMSE for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured viewer/task accuracy, preference/visibility rate, metric rank correlation, and confidence intervals; state one counterexample that would reverse your recommendation.

- G6. Pass condition for Visual perception for graphics. Using viewer/task accuracy, preference/visibility rate, metric rank correlation, and confidence intervals, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 3, Unit 6. Denoising and image-space reconstruction

Recommended time. 1.7-2.0 weeks (about 25-40 focused hours depending on background).

What you are learning. Bilateral/joint bilateral ideas, temporal accumulation, super-resolution intuition, rendering denoisers.

A. Reading order

1. Read the primary assignment first: RTR 4e Ch. 12, then Schied et al., Spatiotemporal Variance-Guided Filtering (HPG 2017) as a concrete temporal reconstruction/denoising case. Use vendor notes only to understand current API/hardware details, not as the theory source.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Use a rendering denoising or temporal-reprojection tutorial as a practical companion.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Bias-variance tradeoff
- edge-aware filters
- temporal reprojection
- Precision requirement: distinguish and justify spatial bilateral filtering vs temporal accumulation vs feature-guided reconstruction around edges, disocclusions, and specular motion. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the minimum-mean-square linear estimator in a toy denoising problem, then analyze bias-variance tradeoffs of spatial filtering versus feature-guided reconstruction near an edge.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Include disocclusion, moving specular highlights, thin geometry, and deliberately wrong auxiliary depth/normals to separate oversmoothing from ghosting. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement bilateral filter and temporal accumulation; test ghosting/disocclusion failure modes. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For noisy samples [0.1,0.2,0.9,1.0,1.1] crossing an edge, compute one bilateral-filter output at the center using explicit spatial/range weights, then compare with a plain Gaussian average. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Include disocclusion, moving specular highlights, thin geometry, and deliberately wrong auxiliary depth/normals to separate oversmoothing from ghosting. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Add temporal reprojection, neighborhood clamping, and disocclusion rejection one at a time; produce an ablation video and per-region error curves.
- P5 - Machine-readable evidence. Export RMSE/SSIM by region, temporal flicker/ghosting error, edge preservation, disocclusion recovery time, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: spatial bilateral filtering vs temporal accumulation vs feature-guided reconstruction around edges, disocclusions, and specular motion. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Include disocclusion, moving specular highlights, thin geometry, and deliberately wrong auxiliary depth/normals to separate oversmoothing from ghosting." without hand-editing code

between methods. Record RMSE/SSIM by region, temporal flicker/ghosting error, edge preservation, disocclusion recovery time, and runtime.

- P7 - Deliberate bug and regression drill. Intentionally reuse temporal history at disocclusions/specular changes without a validity/reprojection test, producing plausible but ghosted output. Before running, predict the first observable failure in RMSE/SSIM by region, temporal flicker/ghosting error, edge preservation, disocclusion recovery time, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the minimum-mean-square linear estimator in a toy denoising problem, then analyze bias-variance tradeoffs of spatial filtering versus feature-guided reconstruction near an edge. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For noisy samples [0.1,0.2,0.9,1.0,1.1] crossing an edge, compute one bilateral-filter output at the center using explicit spatial/range weights, then compare with a plain Gaussian average. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement bilateral filter and temporal accumulation; test ghosting/disocclusion failure modes. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Add temporal reprojection, neighborhood clamping, and disocclusion rejection one at a time; produce an ablation video and per-region error curves. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare spatial bilateral filtering vs temporal accumulation vs feature-guided reconstruction around edges, disocclusions, and specular motion. Use the same source image/signal, color space, output resolution and sampling/exposure budget; for viewer studies, use the same stimuli and participant protocol. Record RMSE/SSIM by region, temporal flicker/ghosting error, edge preservation, disocclusion recovery time, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Include disocclusion, moving specular highlights, thin geometry, and deliberately wrong auxiliary depth/normals to separate oversmoothing from ghosting. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: RTR Ch. 12; recent vendor technical notes as optional. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Denoising and image-space reconstruction. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among spatial bilateral filtering vs temporal accumulation vs feature-guided reconstruction around edges, disocclusions, and specular motion. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using RMSE/SSIM by region, temporal flicker/ghosting error, edge preservation, disocclusion recovery time, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if spatial bilateral filtering vs temporal accumulation vs feature-guided reconstruction around edges, disocclusions, and specular motion show a repeatable improvement or crossover in RMSE/SSIM by region, temporal flicker/ghosting error, edge preservation, disocclusion recovery time, and runtime across multiple images/signals/scales and the effect survives linear-light/color-space and reference-quality controls relevant to the unit.

F. Concrete research project - Denoising and image-space reconstruction

F1. Falsifiable question. Under the controlled failure regime "Include disocclusion, moving specular highlights, thin geometry, and deliberately wrong auxiliary depth/normals to separate oversmoothing from ghosting.", which alternative in spatial bilateral filtering vs temporal accumulation vs feature-guided reconstruction around edges, disocclusions, and specular motion is preferable, and is the change explained by spectral support, footprint estimation, and aliasing/reconstruction error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Add temporal reprojection, neighborhood clamping, and disocclusion rejection one at a time; produce an ablation video and per-region error curves. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a synthetic image with known ground truth plus at least two natural/HDR images. Save full-frame metrics and diagnostic crops/difference images; all filtering/compositing must be done in explicitly stated linear or encoded color spaces. The central adversarial condition is: Include disocclusion, moving specular highlights, thin geometry, and deliberately wrong auxiliary depth/normals to separate oversmoothing from ghosting.

F4. Required evidence. Measure RMSE/SSIM by region, temporal flicker/ghosting error, edge preservation, disocclusion recovery time, and runtime. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Denoising and image-space reconstruction. If a stable failure/crossover is found among spatial bilateral filtering vs temporal accumulation vs feature-guided reconstruction around edges, disocclusions, and specular motion, implement one adaptive or corrected variant that directly targets spectral support, footprint estimation, and aliasing/reconstruction error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Denoising and image-space reconstruction. Using the measured RMSE/SSIM by region, temporal flicker/ghosting error, edge preservation, disocclusion recovery time, and runtime, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Denoising and image-space reconstruction

- G1. Closed-book derivation/oracle. Derive the minimum-mean-square linear estimator in a toy denoising problem, then analyze bias-variance tradeoffs of spatial filtering versus feature-guided reconstruction near an edge. Then solve this exact micro-case from scratch without opening your implementation: For noisy samples $[0.1, 0.2, 0.9, 1.0, 1.1]$ crossing an edge, compute one bilateral-filter output at the center using explicit spatial/range weights, then compare with a plain Gaussian average. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement bilateral filter and temporal accumulation; test ghosting/disocclusion failure modes. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally reuse temporal history at disocclusions/specular changes without a validity/reprojection test, producing plausible but ghosted output. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Add temporal reprojection, neighborhood clamping, and disocclusion rejection one at a time; produce an ablation video and per-region error curves. Then apply this stress without retuning: Include disocclusion, moving specular highlights, thin geometry, and deliberately wrong auxiliary depth/normals to separate oversmoothing from ghosting. Report RMSE/SSIM by region, temporal flicker/ghosting error, edge preservation, disocclusion recovery time, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among spatial bilateral filtering vs temporal accumulation vs feature-guided reconstruction around edges, disocclusions, and specular motion for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured RMSE/SSIM by region, temporal flicker/ghosting error, edge preservation, disocclusion recovery time, and runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Denoising and image-space reconstruction. Using RMSE/SSIM by region, temporal flicker/ghosting error, edge preservation, disocclusion recovery time, and runtime, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the

quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 3 Closeout: Project, Review and Mastery Gate

Volume project. Build an image laboratory demonstrating correct linear-light processing, resampling/aliasing, HDR/tonemapping, color-space conversions, denoising and perceptual effects.

Cumulative review

- Create a diagnostic image suite containing aliasing, alpha, HDR, color and temporal-denoising failure cases.
- Derive the sampling/reconstruction and color-space operations used in your image pipeline and verify them numerically.
- Run one experiment where two methods have similar RMSE but visibly/perceptually different artifacts and explain why.
- Package linear-light compositing, resampling and color-conversion tests as reusable utilities.
- Write a synthesis separating signal-processing error, display/color error and perceptual/task error.

Mastery gate before moving on

- Predict and then verify aliasing, color-space/compositing, HDR, and denoising failures on controlled images.
- Implement one reference-quality resampler and one correct linear-light/HDR pipeline with known-answer tests.
- Give a concrete case where perceptual quality disagrees with pixel error and justify a more appropriate evaluation.

Volume 4. Rasterization and Real-Time Rendering

Purpose. Master the modern real-time graphics pipeline from triangle coverage to physically based shading and GPU effects.

Suggested duration. 12-16 weeks

Prerequisites. Volumes 1-3 and basic shader/GPU programming from Volume 2.

Typical module pace. 1.5-2.0 weeks per module (8 modules total).

Primary references for this volume

- Fundamentals of Computer Graphics, Ch. 7-11 and 17.
- Real-Time Rendering, 4e, Ch. 2-12 as the main real-time spine.
- Berkeley CS184 Spring 2026 lectures 2-8 plus relevant later lectures.

Online spine

Berkeley CS184/284A Spring 2026 course: <https://cs184.eecs.berkeley.edu/sp26/>

Real-Time Rendering 4e resources: <https://www.realtimerendering.com/>

Canonical paper track

- Williams (1978), Casting Curved Shadows on Curved Surfaces (shadow maps).
- Cook & Torrance (1982), A Reflectance Model for Computer Graphics.
- Karis (2013), Real Shading in Unreal Engine 4 (course notes).
- Schied et al. / modern temporal reconstruction and real-time sampling papers; use current SIGGRAPH proceedings for updates.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Volume 4, Unit 1. Triangle rasterization

Recommended time. 1.5-2.0 weeks (about 22-40 focused hours depending on background).

What you are learning. Edge functions, barycentric interpolation, perspective-correct interpolation, depth.

A. Reading order

1. Read the primary assignment first: FCG Ch. 9; RTR Ch. 2 and Ch. 5.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 drawing triangles and rasterization pipeline.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Top-left rule intuition
- perspective correction
- z-buffer
- Precision requirement: distinguish and justify edge-function rasterization with different fill conventions, precision choices, and sample patterns on shared-edge/degenerate tests. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive edge functions and barycentric interpolation for a triangle, prove the inside/outside test under a fixed winding convention, and derive the top-left rule needed for crack-free shared edges.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Rasterize adjacent triangles sharing edges at subpixel locations plus thin/degenerate triangles; test cracks/double hits under winding and precision changes. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Write a software rasterizer with subpixel sampling, depth, and perspective-correct interpolation. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Rasterize two triangles sharing the diagonal of a 4x4 pixel grid using edge functions and the top-left rule. List exactly which sample centers belong to each triangle and verify there is neither a crack nor double coverage. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Rasterize adjacent triangles sharing edges at subpixel locations plus thin/degenerate triangles; test cracks/double hits under winding and precision changes. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement top-left fill, multisample locations, and perspective-correct interpolation in software; create a test suite of shared edges, subpixel slivers, and degenerate triangles.
- P5 - Machine-readable evidence. Export coverage mismatch, shared-edge crack/double-hit count, interpolation error, depth error, and samples/pixel cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: edge-function rasterization with different fill conventions, precision choices, and sample patterns on shared-edge/degenerate tests. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Rasterize adjacent triangles sharing edges at subpixel locations plus thin/degenerate triangles; test cracks/double hits under winding and precision changes." without hand-editing code between methods. Record coverage mismatch, shared-edge crack/double-hit count, interpolation error, depth error, and samples/pixel cost.
- P7 - Deliberate bug and regression drill. Intentionally omit a consistent top-left/shared-edge fill convention so two adjacent triangles either double-cover or leave a crack. Before running, predict the first observable failure in coverage mismatch, shared-edge crack/double-hit count, interpolation error, depth error, and samples/pixel cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive edge functions and barycentric interpolation for a triangle, prove the inside/outside test under a fixed winding convention, and derive the top-left rule needed for crack-free shared edges. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Rasterize two triangles sharing the diagonal of a 4x4 pixel grid using edge functions and the top-left rule. List exactly which sample centers belong to each triangle and verify there is neither a crack nor

double coverage. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Write a software rasterizer with subpixel sampling, depth, and perspective-correct interpolation. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement top-left fill, multisample locations, and perspective-correct interpolation in software; create a test suite of shared edges, subpixel slivers, and degenerate triangles. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare edge-function rasterization with different fill conventions, precision choices, and sample patterns on shared-edge/degenerate tests. Use the same scene, camera path, resolution and quality target; compare under either matched frame-time budget or matched samples/rays/features, and state which budget is fixed. Record coverage mismatch, shared-edge crack/double-hit count, interpolation error, depth error, and samples/pixel cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Rasterize adjacent triangles sharing edges at subpixel locations plus thin/degenerate triangles; test cracks/double hits under winding and precision changes. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: FCG Ch. 9; RTR Ch. 2 and Ch. 5. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Triangle rasterization. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among edge-function rasterization with different fill conventions, precision choices, and sample patterns on shared-edge/degenerate tests. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using coverage mismatch, shared-edge crack/double-hit count, interpolation error, depth error, and samples/pixel cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if edge-function rasterization with different fill conventions, precision choices, and sample patterns on shared-edge/degenerate tests move a quality-frame-time Pareto frontier in coverage mismatch, shared-edge crack/double-hit count, interpolation error, depth error, and samples/pixel cost on multiple scenes/camera motions at a stated fixed budget; reject gains that hide artifacts in unmeasured temporal or edge cases.

F. Concrete research project - Triangle rasterization

F1. Falsifiable question. Under the controlled failure regime "Rasterize adjacent triangles sharing edges at subpixel locations plus thin/degenerate triangles; test cracks/double hits under winding and precision changes.", characterize the ranking or regime transition across edge-function rasterization with different fill conventions, precision choices, and sample patterns on shared-edge/degenerate tests. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by coverage/visibility conventions, precision, temporal validity, and quality-per-frame-time tradeoffs. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement top-left fill, multisample locations, and perspective-correct interpolation in software; create a test suite of shared edges, subpixel slivers, and degenerate triangles. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use three scenes that separately stress geometry/coverage, lighting/material complexity, and temporal/visibility behavior. Measure at no fewer than three resolutions and report both image/task error and GPU frame time. The central adversarial condition is: Rasterize adjacent triangles sharing edges at subpixel locations plus thin/degenerate triangles; test cracks/double hits under winding and precision changes.

F4. Required evidence. Measure coverage mismatch, shared-edge crack/double-hit count, interpolation error, depth error, and samples/pixel cost. Deliver reference outputs plus difference/diagnostic images, 'results.csv', one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Triangle rasterization. If a stable failure/crossover is found among edge-function rasterization with different fill conventions, precision choices, and sample patterns on shared-edge/degenerate tests,

implement one adaptive or corrected variant that directly targets coverage/visibility conventions, precision, temporal validity, and quality-per-frame-time tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Triangle rasterization. Using the measured coverage mismatch, shared-edge crack/double-hit count, interpolation error, depth error, and samples/pixel cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Triangle rasterization

- G1. Closed-book derivation/oracle. Derive edge functions and barycentric interpolation for a triangle, prove the inside/outside test under a fixed winding convention, and derive the top-left rule needed for crack-free shared edges. Then solve this exact micro-case from scratch without opening your implementation: Rasterize two triangles sharing the diagonal of a 4x4 pixel grid using edge functions and the top-left rule. List exactly which sample centers belong to each triangle and verify there is neither a crack nor double coverage. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Write a software rasterizer with subpixel sampling, depth, and perspective-correct interpolation. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally omit a consistent top-left/shared-edge fill convention so two adjacent triangles either double-cover or leave a crack. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement top-left fill, multisample locations, and perspective-correct interpolation in software; create a test suite of shared edges, subpixel slivers, and degenerate triangles. Then apply this stress without retuning: Rasterize adjacent triangles sharing edges at subpixel locations plus thin/degenerate triangles; test cracks/double hits under winding and precision changes. Report coverage mismatch, shared-edge crack/double-hit count, interpolation error, depth error, and samples/pixel cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across edge-function rasterization with different fill conventions, precision choices, and sample patterns on shared-edge/degenerate tests for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured coverage mismatch, shared-edge crack/double-hit count, interpolation error, depth error, and samples/pixel cost; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Triangle rasterization. Using coverage mismatch, shared-edge crack/double-hit count, interpolation error, depth error, and samples/pixel cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 4, Unit 2. Transforms and clipping

Recommended time. 1.5-2.0 weeks (about 22-40 focused hours depending on background).

What you are learning. Model/view/projection, homogeneous clipping, viewport mapping, culling.

A. Reading order

1. Read the primary assignment first: FCG Ch. 7-9; RTR Ch. 4.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 transform lectures.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Clip-space inequalities

- handedness conventions
- guard bands
- Precision requirement: distinguish and justify homogeneous clipping before divide vs incorrect post-divide clipping; conventional depth vs reversed-Z under extreme near/far ratios. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a perspective projection matrix and homogeneous clipping conditions from the view frustum; work through a segment crossing multiple clip planes and explain why clipping must precede the perspective divide.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Move primitives through every frustum plane, especially near $w \approx 0$; repeat with reversed-Z and extreme near/far ratios. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement Sutherland-Hodgman clipping in homogeneous coordinates and validate against a GPU API. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Clip a triangle with one vertex behind the near plane in homogeneous coordinates. Compute the two new clip-space intersections before perspective divide; then show the incorrect result obtained by clipping in NDC. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Move primitives through every frustum plane, especially near $w \approx 0$; repeat with reversed-Z and extreme near/far ratios. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement six-plane homogeneous clipping and a depth-precision visualizer; compare conventional and reversed-Z precision for near/far ratios from 10^2 to 10^7 .
- P5 - Machine-readable evidence. Export clip correctness, screen-space reprojection error, depth quantization error, and failure rate near $w=0$ /frustum planes to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: homogeneous clipping before divide vs incorrect post-divide clipping; conventional depth vs reversed-Z under extreme near/far ratios. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Move primitives through every frustum plane, especially near $w \approx 0$; repeat with reversed-Z and extreme near/far ratios." without hand-editing code between methods. Record clip correctness, screen-space reprojection error, depth quantization error, and failure rate near $w=0$ /frustum planes.
- P7 - Deliberate bug and regression drill. Intentionally mix OpenGL- and DirectX-style depth conventions when testing reversed-Z and clipping. Before running, predict the first observable failure in clip correctness, screen-space reprojection error, depth quantization error, and failure rate near $w=0$ /frustum planes. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a perspective projection matrix and homogeneous clipping conditions from the view frustum; work through a segment crossing multiple clip planes and explain why clipping must precede the perspective divide. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Clip a triangle with one vertex behind the near plane in homogeneous coordinates. Compute the two new clip-space intersections before perspective divide; then show the incorrect result obtained by clipping in NDC. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement Sutherland-Hodgman clipping in homogeneous coordinates and validate against a GPU API. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement six-plane homogeneous clipping and a depth-precision visualizer; compare conventional and reversed-Z precision for near/far ratios from 10^2 to 10^7 . Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare homogeneous clipping before divide vs incorrect post-divide clipping; conventional depth vs reversed-Z under extreme near/far ratios. Use the same scene, camera path, resolution and

quality target; compare under either matched frame-time budget or matched samples/rays/features, and state which budget is fixed. Record clip correctness, screen-space reprojection error, depth quantization error, and failure rate near $w=0$ /frustum planes. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Move primitives through every frustum plane, especially near $w \approx 0$; repeat with reversed-Z and extreme near/far ratios. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: FCG Ch. 7-9; RTR Ch. 4. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Transforms and clipping. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among homogeneous clipping before divide vs incorrect post-divide clipping; conventional depth vs reversed-Z under extreme near/far ratios. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using clip correctness, screen-space reprojection error, depth quantization error, and failure rate near $w=0$ /frustum planes. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if homogeneous clipping before divide vs incorrect post-divide clipping; conventional depth vs reversed-Z under extreme near/far ratios move a quality-frame-time Pareto frontier in clip correctness, screen-space reprojection error, depth quantization error, and failure rate near $w=0$ /frustum planes on multiple scenes/camera motions at a stated fixed budget; reject gains that hide artifacts in unmeasured temporal or edge cases.

F. Concrete research project - Transforms and clipping

F1. Falsifiable question. Under the controlled failure regime "Move primitives through every frustum plane, especially near $w \approx 0$; repeat with reversed-Z and extreme near/far ratios.", which alternative in homogeneous clipping before divide vs incorrect post-divide clipping; conventional depth vs reversed-Z under extreme near/far ratios is preferable, and is the change explained by visibility/coverage conventions, shading-model approximation, temporal validity, and quality-per-frame-time tradeoffs? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement six-plane homogeneous clipping and a depth-precision visualizer; compare conventional and reversed-Z precision for near/far ratios from 10^2 to 10^7 . Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use three scenes that separately stress geometry/coverage, lighting/material complexity, and temporal/visibility behavior. Measure at no fewer than three resolutions and report both image/task error and GPU frame time. The central adversarial condition is: Move primitives through every frustum plane, especially near $w \approx 0$; repeat with reversed-Z and extreme near/far ratios.

F4. Required evidence. Measure clip correctness, screen-space reprojection error, depth quantization error, and failure rate near $w=0$ /frustum planes. Deliver reference outputs plus difference/diagnostic images, 'results.csv', one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Transforms and clipping. If a stable failure/crossover is found among homogeneous clipping before divide vs incorrect post-divide clipping; conventional depth vs reversed-Z under extreme near/far ratios, implement one adaptive or corrected variant that directly targets visibility/coverage conventions, shading-model approximation, temporal validity, and quality-per-frame-time tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Transforms and clipping. Using the measured clip correctness, screen-space reprojection error, depth quantization error, and failure rate near $w=0$ /frustum planes, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Transforms and clipping

- G1. Closed-book derivation/oracle. Derive a perspective projection matrix and homogeneous clipping conditions from the view frustum; work through a segment crossing multiple clip planes and explain why clipping must precede the perspective divide. Then solve this exact micro-case from scratch without opening your implementation: Clip a triangle with one vertex behind the near plane in homogeneous coordinates. Compute the two new clip-space intersections before perspective divide; then show the incorrect result obtained by clipping in NDC. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement Sutherland-Hodgman clipping in homogeneous coordinates and validate against a GPU API. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally mix OpenGL- and DirectX-style depth conventions when testing reversed-Z and clipping. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement six-plane homogeneous clipping and a depth-precision visualizer; compare conventional and reversed-Z precision for near/far ratios from 10^2 to 10^7 . Then apply this stress without retuning: Move primitives through every frustum plane, especially near $w \approx 0$; repeat with reversed-Z and extreme near/far ratios. Report clip correctness, screen-space reprojection error, depth quantization error, and failure rate near $w=0$ /frustum planes and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among homogeneous clipping before divide vs incorrect post-divide clipping; conventional depth vs reversed-Z under extreme near/far ratios for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured clip correctness, screen-space reprojection error, depth quantization error, and failure rate near $w=0$ /frustum planes; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Transforms and clipping. Using clip correctness, screen-space reprojection error, depth quantization error, and failure rate near $w=0$ /frustum planes, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 4, Unit 3. Texture mapping and filtering

Recommended time. 1.5-2.0 weeks (about 22-40 focused hours depending on background).

What you are learning. UVs, mipmaps, anisotropy, normal mapping, cube maps, procedural textures.

A. Reading order

1. Read the primary assignment first: FCG Ch. 11; RTR Ch. 6.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 texture mapping lecture.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Texture derivatives and footprint
- LOD selection
- tangent frames
- Precision requirement: distinguish and justify mipmapping/trilinear filtering vs anisotropic filtering as projected footprint eccentricity increases. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive texture-coordinate differentials from screen-space derivatives, use them to estimate a pixel footprint, and derive the level-of-detail rule for mipmapping; explain the failure under strong anisotropy.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use a high-frequency texture on a surface approaching grazing incidence; sweep LOD and anisotropy and compare against supersampling. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement mipmap generation and trilinear filtering; visualize anisotropic failure at grazing angles. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a perspective triangle with vertex w values 1,2,4 and UVs (0,0),(1,0),(0,1), compute affine and perspective-correct UV at barycentric coordinates (1/3,1/3,1/3); quantify the difference. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use a high-frequency texture on a surface approaching grazing incidence; sweep LOD and anisotropy and compare against supersampling. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Compute screen-space UV Jacobians, ellipse eccentricity, and chosen LOD/aniso level per pixel; visualize these diagnostic buffers alongside error versus supersampling.
- P5 - Machine-readable evidence. Export error to supersampled reference, alias energy, blur, footprint/LOD error, texture fetch count, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: mipmapping/trilinear filtering vs anisotropic filtering as projected footprint eccentricity increases. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use a high-frequency texture on a surface approaching grazing incidence; sweep LOD and anisotropy and compare against supersampling." without hand-editing code between methods. Record error to supersampled reference, alias energy, blur, footprint/LOD error, texture fetch count, and runtime.
- P7 - Deliberate bug and regression drill. Intentionally compute mip level from object-space derivatives instead of screen-space texture-coordinate footprints. Before running, predict the first observable failure in error to supersampled reference, alias energy, blur, footprint/LOD error, texture fetch count, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive texture-coordinate differentials from screen-space derivatives, use them to estimate a pixel footprint, and derive the level-of-detail rule for mipmapping; explain the failure under strong anisotropy. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a perspective triangle with vertex w values 1,2,4 and UVs (0,0),(1,0),(0,1), compute affine and perspective-correct UV at barycentric coordinates (1/3,1/3,1/3); quantify the difference. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement mipmap generation and trilinear filtering; visualize anisotropic failure at grazing angles. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Compute screen-space UV Jacobians, ellipse eccentricity, and chosen LOD/aniso level per pixel; visualize these diagnostic buffers alongside error versus supersampling. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare mipmapping/trilinear filtering vs anisotropic filtering as projected footprint eccentricity increases. Use the same scene, camera path, resolution and quality target; compare under either matched frame-time budget or matched samples/rays/features, and state which budget is fixed. Record error to supersampled reference, alias energy, blur, footprint/LOD error, texture fetch count, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use a high-frequency texture on a surface approaching grazing incidence; sweep LOD and anisotropy and compare against supersampling. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 11; RTR Ch. 6. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Texture mapping and filtering. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among mipmapping/trilinear filtering vs anisotropic filtering as projected footprint eccentricity increases. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using error to supersampled reference, alias energy, blur, footprint/LOD error, texture fetch count, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if mipmapping/trilinear filtering vs anisotropic filtering as projected footprint eccentricity increases move a quality-frame-time Pareto frontier in error to supersampled reference, alias energy, blur, footprint/LOD error, texture fetch count, and runtime on multiple scenes/camera motions at a stated fixed budget; reject gains that hide artifacts in unmeasured temporal or edge cases.

F. Concrete research project - Texture mapping and filtering

F1. Falsifiable question. Under the controlled failure regime "Use a high-frequency texture on a surface approaching grazing incidence; sweep LOD and anisotropy and compare against supersampling.", which alternative in mipmapping/trilinear filtering vs anisotropic filtering as projected footprint eccentricity increases is preferable, and is the change explained by spectral support, footprint estimation, and aliasing/reconstruction error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Compute screen-space UV Jacobians, ellipse eccentricity, and chosen LOD/aniso level per pixel; visualize these diagnostic buffers alongside error versus supersampling. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use three scenes that separately stress geometry/coverage, lighting/material complexity, and temporal/visibility behavior. Measure at no fewer than three resolutions and report both image/task error and GPU frame time. The central adversarial condition is: Use a high-frequency texture on a surface approaching grazing incidence; sweep LOD and anisotropy and compare against supersampling.

F4. Required evidence. Measure error to supersampled reference, alias energy, blur, footprint/LOD error, texture fetch count, and runtime. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Texture mapping and filtering. If a stable failure/crossover is found among mipmapping/trilinear filtering vs anisotropic filtering as projected footprint eccentricity increases, implement one adaptive or corrected variant that directly targets spectral support, footprint estimation, and aliasing/reconstruction error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Texture mapping and filtering. Using the measured error to supersampled reference, alias energy, blur, footprint/LOD error, texture fetch count, and runtime, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Texture mapping and filtering

- G1. Closed-book derivation/oracle. Derive texture-coordinate differentials from screen-space derivatives, use them to estimate a pixel footprint, and derive the level-of-detail rule for mipmapping; explain the failure under strong anisotropy. Then solve this exact micro-case from scratch without opening your implementation: For a perspective triangle with vertex w values 1,2,4 and UVs (0,0),(1,0),(0,1), compute affine and perspective-correct UV at barycentric coordinates (1/3,1/3,1/3); quantify the difference. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement mipmap generation and trilinear filtering; visualize anisotropic failure at grazing angles. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally compute mip level from object-space derivatives instead of screen-space texture-coordinate footprints. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.

- G4. Transfer experiment. On an input/configuration not used during development, execute: Compute screen-space UV Jacobians, ellipse eccentricity, and chosen LOD/aniso level per pixel; visualize these diagnostic buffers alongside error versus supersampling. Then apply this stress without retuning: Use a high-frequency texture on a surface approaching grazing incidence; sweep LOD and anisotropy and compare against supersampling. Report error to supersampled reference, alias energy, blur, footprint/LOD error, texture fetch count, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among mipmapping/trilinear filtering vs anisotropic filtering as projected footprint eccentricity increases for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured error to supersampled reference, alias energy, blur, footprint/LOD error, texture fetch count, and runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Texture mapping and filtering. Using error to supersampled reference, alias energy, blur, footprint/LOD error, texture fetch count, and runtime, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 4, Unit 4. Shadows and visibility

Recommended time. 1.5-2.0 weeks (about 22-40 focused hours depending on background).

What you are learning. Shadow maps, PCF, cascades, shadow acne, peter-panning, screen-space alternatives.

A. Reading order

1. Read the primary assignment first: RTR Ch. 7.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Real-Time Rendering resource links for shadow techniques.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Depth bias, percentage-closer filtering
- visibility discontinuities
- Precision requirement: distinguish and justify hard shadow maps, PCF, cascaded maps, and bias strategies across contact, thin-occluder, and long-range scenes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the projective transform used by shadow mapping, relate depth precision to near/far choices, and analytically explain shadow acne, peter-panning, and cascade discontinuities.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use contact shadows, long cascaded depth ranges, thin occluders, and moving lights/cameras; score acne, detachment, leaking, and temporal shimmer separately. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement directional-light shadow mapping with PCF and cascades; document artifacts and fixes. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Use a plane $z=0$, point light at $(0,0,10)$, and an occluding square at $z=5$. Work out shadow status for three receiver points and show how a fixed depth bias can cause acne in one scale and peter-panning in another. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use contact shadows, long cascaded depth ranges, thin occluders, and moving lights/cameras; score acne, detachment, leaking, and temporal shimmer separately. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build a scripted camera/light path that crosses cascade boundaries and contact regions; log shadow-depth bias, PCF radius, and artifact masks frame by frame.

- P5 - Machine-readable evidence. Export shadow-mask error to ray-traced reference, acne/detachment/leak pixels, cascade temporal discontinuity, and frame cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: hard shadow maps, PCF, cascaded maps, and bias strategies across contact, thin-occluder, and long-range scenes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use contact shadows, long cascaded depth ranges, thin occluders, and moving lights/cameras; score acne, detachment, leaking, and temporal shimmer separately." without hand-editing code between methods. Record shadow-mask error to ray-traced reference, acne/detachment/leak pixels, cascade temporal discontinuity, and frame cost.
- P7 - Deliberate bug and regression drill. Intentionally use one constant shadow-map depth bias across all slopes/resolutions, producing both acne and peter-panning. Before running, predict the first observable failure in shadow-mask error to ray-traced reference, acne/detachment/leak pixels, cascade temporal discontinuity, and frame cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the projective transform used by shadow mapping, relate depth precision to near/far choices, and analytically explain shadow acne, peter-panning, and cascade discontinuities. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Use a plane $z=0$, point light at $(0,0,10)$, and an occluding square at $z=5$. Work out shadow status for three receiver points and show how a fixed depth bias can cause acne in one scale and peter-panning in another. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement directional-light shadow mapping with PCF and cascades; document artifacts and fixes. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build a scripted camera/light path that crosses cascade boundaries and contact regions; log shadow-depth bias, PCF radius, and artifact masks frame by frame. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare hard shadow maps, PCF, cascaded maps, and bias strategies across contact, thin-occluder, and long-range scenes. Use the same scene, camera path, resolution and quality target; compare under either matched frame-time budget or matched samples/rays/features, and state which budget is fixed. Record shadow-mask error to ray-traced reference, acne/detachment/leak pixels, cascade temporal discontinuity, and frame cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use contact shadows, long cascaded depth ranges, thin occluders, and moving lights/cameras; score acne, detachment, leaking, and temporal shimmer separately. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: RTR Ch. 7. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Shadows and visibility. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among hard shadow maps, PCF, cascaded maps, and bias strategies across contact, thin-occluder, and long-range scenes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using shadow-mask error to ray-traced reference, acne/detachment/leak pixels, cascade temporal discontinuity, and frame cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if hard shadow maps, PCF, cascaded maps, and bias strategies across contact, thin-occluder, and long-range scenes move a quality-frame-time Pareto frontier in shadow-mask error to ray-traced reference, acne/detachment/leak pixels, cascade temporal discontinuity, and frame cost on multiple scenes/camera motions at a stated fixed budget; reject gains that hide artifacts in unmeasured temporal or edge cases.

F. Concrete research project - Shadows and visibility

- F1. Falsifiable question. Under the controlled failure regime "Use contact shadows, long cascaded depth ranges, thin occluders, and moving lights/cameras; score acne, detachment, leaking, and temporal shimmer separately.", characterize the ranking or regime transition across hard shadow maps, PCF, cascaded maps, and bias strategies across contact, thin-occluder, and long-range scenes. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by coverage/visibility conventions, precision, temporal validity, and quality-per-frame-time tradeoffs. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build a scripted camera/light path that crosses cascade boundaries and contact regions; log shadow-depth bias, PCF radius, and artifact masks frame by frame. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use three scenes that separately stress geometry/coverage, lighting/material complexity, and temporal/visibility behavior. Measure at no fewer than three resolutions and report both image/task error and GPU frame time. The central adversarial condition is: Use contact shadows, long cascaded depth ranges, thin occluders, and moving lights/cameras; score acne, detachment, leaking, and temporal shimmer separately.
- F4. Required evidence. Measure shadow-mask error to ray-traced reference, acne/detachment/leak pixels, cascade temporal discontinuity, and frame cost. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Shadows and visibility. If a stable failure/crossover is found among hard shadow maps, PCF, cascaded maps, and bias strategies across contact, thin-occluder, and long-range scenes, implement one adaptive or corrected variant that directly targets coverage/visibility conventions, precision, temporal validity, and quality-per-frame-time tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Shadows and visibility. Using the measured shadow-mask error to ray-traced reference, acne/detachment/leak pixels, cascade temporal discontinuity, and frame cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Shadows and visibility

- G1. Closed-book derivation/oracle. Derive the projective transform used by shadow mapping, relate depth precision to near/far choices, and analytically explain shadow acne, peter-panning, and cascade discontinuities. Then solve this exact micro-case from scratch without opening your implementation: Use a plane $z=0$, point light at $(0,0,10)$, and an occluding square at $z=5$. Work out shadow status for three receiver points and show how a fixed depth bias can cause acne in one scale and peter-panning in another. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement directional-light shadow mapping with PCF and cascades; document artifacts and fixes. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use one constant shadow-map depth bias across all slopes/resolutions, producing both acne and peter-panning. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Build a scripted camera/light path that crosses cascade boundaries and contact regions; log shadow-depth bias, PCF radius, and artifact masks frame by frame. Then apply this stress without retuning: Use contact shadows, long cascaded depth ranges, thin occluders, and moving lights/cameras; score acne, detachment, leaking, and temporal shimmer separately. Report shadow-mask error to ray-traced reference, acne/detachment/leak pixels, cascade temporal discontinuity, and frame cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across hard shadow maps, PCF, cascaded maps, and bias strategies across contact, thin-occluder, and long-range scenes for three

concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured shadow-mask error to ray-traced reference, acne/detachment/leak pixels, cascade temporal discontinuity, and frame cost; state one counterexample that would change the diagnosis or ranking.

- G6. Pass condition for Shadows and visibility. Using shadow-mask error to ray-traced reference, acne/detachment/leak pixels, cascade temporal discontinuity, and frame cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 4, Unit 5. Physically based shading in real time

Recommended time. 1.5-2.0 weeks (about 22-40 focused hours depending on background).

What you are learning. Microfacet BRDFs, metallic-roughness workflows, image-based lighting, split-sum approximations.

A. Reading order

1. Read the primary assignment first: RTR Ch. 8-10; PBRT Ch. 9 for deeper theory.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Official engine PBR documentation as optional practical reference.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- NDF, Fresnel, geometry term, energy conservation, reciprocity
- Precision requirement: distinguish and justify GGX/Smith/Schlick implementation vs offline PBRT/Mitsuba references over roughness, Fresnel, and grazing-angle sweeps. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Write the microfacet BRDF as Fresnel, geometry, and normal-distribution factors; derive its reciprocity/energy constraints and connect roughness to the width of the specular lobe.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Sweep incidence to grazing and roughness from nearly mirror to very rough for metals and dielectrics; compare energy and appearance against an offline path-traced reference. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement GGX + Smith + Schlick model; compare against PBRT/Mitsuba reference renders. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Evaluate a GGX/Smith/Schlick BRDF for $N=V=L$ at roughness 0.2 and 0.8 with $F_0=0.04$. Compute D, G, F and the final specular term; check limiting behavior qualitatively. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Sweep incidence to grazing and roughness from nearly mirror to very rough for metals and dielectrics; compare energy and appearance against an offline path-traced reference. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create a BRDF validation harness integrating reflected energy over the hemisphere and a grid of offline reference renders across roughness/ F_0 /incidence.
- P5 - Machine-readable evidence. Export hemispherical energy error, BRDF value error versus reference, grazing-angle error, image RMSE, and frame cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: GGX/Smith/Schlick implementation vs offline PBRT/Mitsuba references over roughness, Fresnel, and grazing-angle sweeps. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Sweep incidence to grazing and roughness from nearly mirror to very rough for metals and dielectrics; compare energy and appearance against an offline path-traced reference." without hand-editing code

between methods. Record hemispherical energy error, BRDF value error versus reference, grazing-angle error, image RMSE, and frame cost.

- P7 - Deliberate bug and regression drill. Intentionally feed perceptual roughness directly where the GGX implementation expects $\alpha = \text{roughness}^2$ and omit the geometric term at grazing angles. Before running, predict the first observable failure in hemispherical energy error, BRDF value error versus reference, grazing-angle error, image RMSE, and frame cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Write the microfacet BRDF as Fresnel, geometry, and normal-distribution factors; derive its reciprocity/energy constraints and connect roughness to the width of the specular lobe. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Evaluate a GGX/Smith/Schlick BRDF for $N=V=L$ at roughness 0.2 and 0.8 with $F0=0.04$. Compute D, G, F and the final specular term; check limiting behavior qualitatively. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement GGX + Smith + Schlick model; compare against PBRT/Mitsuba reference renders. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create a BRDF validation harness integrating reflected energy over the hemisphere and a grid of offline reference renders across roughness/ $F0$ /incidence. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare GGX/Smith/Schlick implementation vs offline PBRT/Mitsuba references over roughness, Fresnel, and grazing-angle sweeps. Use the same scene, camera path, resolution and quality target; compare under either matched frame-time budget or matched samples/rays/features, and state which budget is fixed. Record hemispherical energy error, BRDF value error versus reference, grazing-angle error, image RMSE, and frame cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Sweep incidence to grazing and roughness from nearly mirror to very rough for metals and dielectrics; compare energy and appearance against an offline path-traced reference. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: RTR Ch. 8-10; PBRT Ch. 9 for deeper theory. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Physically based shading in real time. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among GGX/Smith/Schlick implementation vs offline PBRT/Mitsuba references over roughness, Fresnel, and grazing-angle sweeps. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using hemispherical energy error, BRDF value error versus reference, grazing-angle error, image RMSE, and frame cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if GGX/Smith/Schlick implementation vs offline PBRT/Mitsuba references over roughness, Fresnel, and grazing-angle sweeps move a quality-frame-time Pareto frontier in hemispherical energy error, BRDF value error versus reference, grazing-angle error, image RMSE, and frame cost on multiple scenes/camera motions at a stated fixed budget; reject gains that hide artifacts in unmeasured temporal or edge cases.

F. Concrete research project - Physically based shading in real time

F1. Falsifiable question. Under the controlled failure regime "Sweep incidence to grazing and roughness from nearly mirror to very rough for metals and dielectrics; compare energy and appearance against an offline path-traced reference.", which alternative in GGX/Smith/Schlick implementation vs offline PBRT/Mitsuba references over roughness, Fresnel, and grazing-angle sweeps is preferable, and is the change explained by microfacet parameterization, energy/reciprocity approximations, grazing-angle behavior, and offline-reference mismatch under frame-time constraints? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create a BRDF validation harness integrating reflected energy over the hemisphere and a grid of offline reference renders across roughness/F0/incidence. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use three scenes that separately stress geometry/coverage, lighting/material complexity, and temporal/visibility behavior. Measure at no fewer than three resolutions and report both image/task error and GPU frame time. The central adversarial condition is: Sweep incidence to grazing and roughness from nearly mirror to very rough for metals and dielectrics; compare energy and appearance against an offline path-traced reference.

F4. Required evidence. Measure hemispherical energy error, BRDF value error versus reference, grazing-angle error, image RMSE, and frame cost. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Physically based shading in real time. If a stable failure/crossover is found among GGX/Smith/Schlick implementation vs offline PBRT/Mitsuba references over roughness, Fresnel, and grazing-angle sweeps, implement one adaptive or corrected variant that directly targets microfacet parameterization, energy/reciprocity approximations, grazing-angle behavior, and offline-reference mismatch under frame-time constraints; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Physically based shading in real time. Using the measured hemispherical energy error, BRDF value error versus reference, grazing-angle error, image RMSE, and frame cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Physically based shading in real time

- G1. Closed-book derivation/oracle. Write the microfacet BRDF as Fresnel, geometry, and normal-distribution factors; derive its reciprocity/energy constraints and connect roughness to the width of the specular lobe. Then solve this exact micro-case from scratch without opening your implementation: Evaluate a GGX/Smith/Schlick BRDF for $N=V=L$ at roughness 0.2 and 0.8 with $F0=0.04$. Compute D, G, F and the final specular term; check limiting behavior qualitatively. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement GGX + Smith + Schlick model; compare against PBRT/Mitsuba reference renders. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally feed perceptual roughness directly where the GGX implementation expects $\alpha = \text{roughness}^2$ and omit the geometric term at grazing angles. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create a BRDF validation harness integrating reflected energy over the hemisphere and a grid of offline reference renders across roughness/F0/incidence. Then apply this stress without retuning: Sweep incidence to grazing and roughness from nearly mirror to very rough for metals and dielectrics; compare energy and appearance against an offline path-traced reference. Report hemispherical energy error, BRDF value error versus reference, grazing-angle error, image RMSE, and frame cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among GGX/Smith/Schlick implementation vs offline PBRT/Mitsuba references over roughness, Fresnel, and grazing-angle sweeps for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured hemispherical energy error, BRDF value error versus reference, grazing-angle error, image RMSE, and frame cost; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Physically based shading in real time. Using hemispherical energy error, BRDF value error versus reference, grazing-angle error, image RMSE, and frame cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 4, Unit 6. Deferred, tiled, clustered, and forward+

Recommended time. 1.5-2.0 weeks (about 22-40 focused hours depending on background).

What you are learning. Light culling, G-buffers, transparency tradeoffs, many-light rendering.

A. Reading order

1. Read the primary assignment first: RTR Ch. 10 and resource page.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Study one modern engine frame breakdown.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Bandwidth vs shading tradeoff
- screen-space bins
- clustered frusta
- Precision requirement: distinguish and justify forward, deferred, tiled, clustered, and forward+ lighting as resolution, light count, transparency, and MSAA change. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Build an analytical cost model for forward, deferred, tiled, and clustered lighting in terms of pixels, lights, depth complexity, and G-buffer bandwidth; derive a crossover prediction before benchmarking.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Sweep resolution, light count, transparency/MSAA, and material complexity to locate crossover points between pipeline strategies. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement either tiled/clustered light culling or deferred lighting; profile with hundreds/thousands of lights. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: On an 8x8 screen divided into four tiles, place three point-light screen bounds. Build the tile-light lists by hand and count lighting evaluations for forward-all-lights versus tiled shading. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Sweep resolution, light count, transparency/MSAA, and material complexity to locate crossover points between pipeline strategies. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement two lighting paths in the same renderer and automate a sweep over resolution/light count/transparency; plot the measured crossover against your analytical cost model.
- P5 - Machine-readable evidence. Export frame time by stage, G-buffer/bandwidth use, light-list size, memory, and crossover versus lights/resolution/transparency to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: forward, deferred, tiled, clustered, and forward+ lighting as resolution, light count, transparency, and MSAA change. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Sweep resolution, light count, transparency/MSAA, and material complexity to locate crossover points between pipeline strategies." without hand-editing code between methods. Record frame time by stage, G-buffer/bandwidth use, light-list size, memory, and crossover versus lights/resolution/transparency.
- P7 - Deliberate bug and regression drill. Intentionally reconstruct view/world position from depth with the wrong projection/depth convention in only one lighting path. Before running, predict the first observable failure in frame time by stage, G-buffer/bandwidth use, light-list size, memory, and crossover versus lights/resolution/transparency. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Build an analytical cost model for forward, deferred, tiled, and clustered lighting in terms of pixels, lights, depth complexity, and G-buffer bandwidth; derive a crossover prediction before benchmarking. Then

evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. On an 8x8 screen divided into four tiles, place three point-light screen bounds. Build the tile-light lists by hand and count lighting evaluations for forward-all-lights versus tiled shading. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Implement either tiled/clustered light culling or deferred lighting; profile with hundreds/thousands of lights. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement two lighting paths in the same renderer and automate a sweep over resolution/light count/transparency; plot the measured crossover against your analytical cost model. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare forward, deferred, tiled, clustered, and forward+ lighting as resolution, light count, transparency, and MSAA change. Use the same scene, camera path, resolution and quality target; compare under either matched frame-time budget or matched samples/rays/features, and state which budget is fixed. Record frame time by stage, G-buffer/bandwidth use, light-list size, memory, and crossover versus lights/resolution/transparency. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Sweep resolution, light count, transparency/MSAA, and material complexity to locate crossover points between pipeline strategies. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: RTR Ch. 10 and resource page. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Deferred, tiled, clustered, and forward+. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among forward, deferred, tiled, clustered, and forward+ lighting as resolution, light count, transparency, and MSAA change. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using frame time by stage, G-buffer/bandwidth use, light-list size, memory, and crossover versus lights/resolution/transparency. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if forward, deferred, tiled, clustered, and forward+ lighting as resolution, light count, transparency, and MSAA change move a quality-frame-time Pareto frontier in frame time by stage, G-buffer/bandwidth use, light-list size, memory, and crossover versus lights/resolution/transparency on multiple scenes/camera motions at a stated fixed budget; reject gains that hide artifacts in unmeasured temporal or edge cases.

F. Concrete research project - Deferred, tiled, clustered, and forward+

F1. Falsifiable question. Under the controlled failure regime "Sweep resolution, light count, transparency/MSAA, and material complexity to locate crossover points between pipeline strategies.", characterize the ranking or regime transition across forward, deferred, tiled, clustered, and forward+ lighting as resolution, light count, transparency, and MSAA change. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by coverage/visibility conventions, precision, temporal validity, and quality-per-frame-time tradeoffs. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement two lighting paths in the same renderer and automate a sweep over resolution/light count/transparency; plot the measured crossover against your analytical cost model. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use three scenes that separately stress geometry/coverage, lighting/material complexity, and temporal/visibility behavior. Measure at no fewer than three resolutions and report both image/task error and GPU frame time. The central adversarial condition is: Sweep resolution, light count, transparency/MSAA, and material complexity to locate crossover points between pipeline strategies.

F4. Required evidence. Measure frame time by stage, G-buffer/bandwidth use, light-list size, memory, and crossover versus lights/resolution/transparency. Deliver reference outputs plus difference/diagnostic images, 'results.csv', one

quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Deferred, tiled, clustered, and forward+. If a stable failure/crossover is found among forward, deferred, tiled, clustered, and forward+ lighting as resolution, light count, transparency, and MSAA change, implement one adaptive or corrected variant that directly targets coverage/visibility conventions, precision, temporal validity, and quality-per-frame-time tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Deferred, tiled, clustered, and forward+. Using the measured frame time by stage, G-buffer/bandwidth use, light-list size, memory, and crossover versus lights/resolution/transparency, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Deferred, tiled, clustered, and forward+

- G1. Closed-book derivation/oracle. Build an analytical cost model for forward, deferred, tiled, and clustered lighting in terms of pixels, lights, depth complexity, and G-buffer bandwidth; derive a crossover prediction before benchmarking. Then solve this exact micro-case from scratch without opening your implementation: On an 8x8 screen divided into four tiles, place three point-light screen bounds. Build the tile-light lists by hand and count lighting evaluations for forward-all-lights versus tiled shading. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement either tiled/clustered light culling or deferred lighting; profile with hundreds/thousands of lights. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally reconstruct view/world position from depth with the wrong projection/depth convention in only one lighting path. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement two lighting paths in the same renderer and automate a sweep over resolution/light count/transparency; plot the measured crossover against your analytical cost model. Then apply this stress without retuning: Sweep resolution, light count, transparency/MSAA, and material complexity to locate crossover points between pipeline strategies. Report frame time by stage, G-buffer/bandwidth use, light-list size, memory, and crossover versus lights/resolution/transparency and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across forward, deferred, tiled, clustered, and forward+ lighting as resolution, light count, transparency, and MSAA change for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured frame time by stage, G-buffer/bandwidth use, light-list size, memory, and crossover versus lights/resolution/transparency; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Deferred, tiled, clustered, and forward+. Using frame time by stage, G-buffer/bandwidth use, light-list size, memory, and crossover versus lights/resolution/transparency, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 4, Unit 7. Temporal methods and anti-aliasing

Recommended time. 1.5-2.0 weeks (about 22-40 focused hours depending on background).

What you are learning. MSAA, TAA, jittered sampling, reprojection, history rejection, motion vectors.

A. Reading order

1. Read the primary assignment first: RTR Ch. 5/12 relevant sections; FCG Ch. 10.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Modern engine TAA notes as optional.

3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Temporal sample reuse
- disocclusion
- variance clipping intuition
- Precision requirement: distinguish and justify spatial AA, naïve temporal averaging, and motion-vector/rejection-aware TAA on motion/disocclusion/specular stress cases. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive temporal reprojection from current/previous camera matrices and motion vectors, then model the exponential/history filter and explain ghosting as a violated correspondence assumption.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use fast motion, disocclusions, thin features, specular highlights, and animated textures; log ghosting, flicker, rejection rate, and sharpness. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement basic TAA in a toy renderer and produce a systematic artifact gallery. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Track a single bright sample moving one pixel per frame across four frames. Compute naive temporal average with no reprojection, then perfect motion-vector reprojection, then one disocclusion/rejection case. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use fast motion, disocclusions, thin features, specular highlights, and animated textures; log ghosting, flicker, rejection rate, and sharpness. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Expose motion vectors, history weight, rejection mask, and clamped history; build a fixed sequence containing disocclusion, thin motion, scrolling texture, and moving highlights.
- P5 - Machine-readable evidence. Export temporal RMSE/flicker, ghost trail duration, thin-feature survival, disocclusion error, and frame cost/history memory to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: spatial AA, naïve temporal averaging, and motion-vector/rejection-aware TAA on motion/disocclusion/specular stress cases. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use fast motion, disocclusions, thin features, specular highlights, and animated textures; log ghosting, flicker, rejection rate, and sharpness." without hand-editing code between methods. Record temporal RMSE/flicker, ghost trail duration, thin-feature survival, disocclusion error, and frame cost/history memory.
- P7 - Deliberate bug and regression drill. Intentionally use motion vectors with the wrong sign/jitter convention and keep history through disocclusion. Before running, predict the first observable failure in temporal RMSE/flicker, ghost trail duration, thin-feature survival, disocclusion error, and frame cost/history memory. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive temporal reprojection from current/previous camera matrices and motion vectors, then model the exponential/history filter and explain ghosting as a violated correspondence assumption. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Track a single bright sample moving one pixel per frame across four frames. Compute naive temporal average with no reprojection, then perfect motion-vector reprojection, then one disocclusion/rejection case. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Implement basic TAA in a toy renderer and produce a systematic artifact gallery. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Expose motion vectors, history weight, rejection mask, and clamped history; build a fixed sequence containing disocclusion, thin motion, scrolling texture, and moving highlights. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare spatial AA, naïve temporal averaging, and motion-vector/rejection-aware TAA on motion/disocclusion/specular stress cases. Use the same scene, camera path, resolution and quality target; compare under either matched frame-time budget or matched samples/rays/features, and state which budget is fixed. Record temporal RMSE/flicker, ghost trail duration, thin-feature survival, disocclusion error, and frame cost/history memory. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use fast motion, disocclusions, thin features, specular highlights, and animated textures; log ghosting, flicker, rejection rate, and sharpness. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: RTR Ch. 5/12 relevant sections; FCG Ch. 10. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Temporal methods and anti-aliasing. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among spatial AA, naïve temporal averaging, and motion-vector/rejection-aware TAA on motion/disocclusion/specular stress cases. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using temporal RMSE/flicker, ghost trail duration, thin-feature survival, disocclusion error, and frame cost/history memory. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if spatial AA, naïve temporal averaging, and motion-vector/rejection-aware TAA on motion/disocclusion/specular stress cases move a quality-frame-time Pareto frontier in temporal RMSE/flicker, ghost trail duration, thin-feature survival, disocclusion error, and frame cost/history memory on multiple scenes/camera motions at a stated fixed budget; reject gains that hide artifacts in unmeasured temporal or edge cases.

F. Concrete research project - Temporal methods and anti-aliasing

- F1. Falsifiable question. Under the controlled failure regime "Use fast motion, disocclusions, thin features, specular highlights, and animated textures; log ghosting, flicker, rejection rate, and sharpness.", characterize the ranking or regime transition across spatial AA, naïve temporal averaging, and motion-vector/rejection-aware TAA on motion/disocclusion/specular stress cases. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by spectral support, footprint estimation, and aliasing/reconstruction error. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Expose motion vectors, history weight, rejection mask, and clamped history; build a fixed sequence containing disocclusion, thin motion, scrolling texture, and moving highlights. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use three scenes that separately stress geometry/coverage, lighting/material complexity, and temporal/visibility behavior. Measure at no fewer than three resolutions and report both image/task error and GPU frame time. The central adversarial condition is: Use fast motion, disocclusions, thin features, specular highlights, and animated textures; log ghosting, flicker, rejection rate, and sharpness.
- F4. Required evidence. Measure temporal RMSE/flicker, ghost trail duration, thin-feature survival, disocclusion error, and frame cost/history memory. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Temporal methods and anti-aliasing. If a stable failure/crossover is found among spatial AA, naïve temporal averaging, and motion-vector/rejection-aware TAA on motion/disocclusion/specular stress cases, implement one adaptive or corrected variant that directly targets spectral support, footprint estimation, and aliasing/reconstruction error; compare it against the strongest fixed baseline under the same protocol. If no stable

boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Temporal methods and anti-aliasing. Using the measured temporal RMSE/flicker, ghost trail duration, thin-feature survival, disocclusion error, and frame cost/history memory, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Temporal methods and anti-aliasing

- G1. Closed-book derivation/oracle. Derive temporal reprojection from current/previous camera matrices and motion vectors, then model the exponential/history filter and explain ghosting as a violated correspondence assumption. Then solve this exact micro-case from scratch without opening your implementation: Track a single bright sample moving one pixel per frame across four frames. Compute naive temporal average with no reprojection, then perfect motion-vector reprojection, then one disocclusion/rejection case. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement basic TAA in a toy renderer and produce a systematic artifact gallery. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use motion vectors with the wrong sign/jitter convention and keep history through disocclusion. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Expose motion vectors, history weight, rejection mask, and clamped history; build a fixed sequence containing disocclusion, thin motion, scrolling texture, and moving highlights. Then apply this stress without retuning: Use fast motion, disocclusions, thin features, specular highlights, and animated textures; log ghosting, flicker, rejection rate, and sharpness. Report temporal RMSE/flicker, ghost trail duration, thin-feature survival, disocclusion error, and frame cost/history memory and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across spatial AA, naïve temporal averaging, and motion-vector/rejection-aware TAA on motion/disocclusion/specular stress cases for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured temporal RMSE/flicker, ghost trail duration, thin-feature survival, disocclusion error, and frame cost/history memory; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Temporal methods and anti-aliasing. Using temporal RMSE/flicker, ghost trail duration, thin-feature survival, disocclusion error, and frame cost/history memory, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 4, Unit 8. Real-time ray tracing and hybrid rendering

Recommended time. 1.5-2.0 weeks (about 22-40 focused hours depending on background).

What you are learning. RT cores/API concepts, acceleration structures, hybrid raster/ray methods, denoising.

A. Reading order

1. Read the primary assignment first: RTR resource updates; PBRT Ch. 15 for GPU ray pipeline intuition.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Vendor ray tracing tutorials if hardware available.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- BLAS/TLAS distinction
- ray budgets
- spatiotemporal reuse idea

- Precision requirement: distinguish and justify raster-only approximation vs limited-ray hybrid shadows/reflections under matched frame-time budgets. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the split of work between rasterization and ray queries in a hybrid renderer; analyze the cost/quality effect of ray count, denoising, and visibility reuse on a simple reflection or shadow pipeline.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Hold frame time fixed while reallocating a small ray budget among shadows/reflections/GI and changing denoising strength; compare perceptual benefit. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Add one-bounce ray-traced shadow/reflection to raster renderer or simulate same algorithm on CPU. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a mirror plane and one point light, trace the single reflection ray for two pixels and compute the shadow ray. Compare the exact one-bounce result with a screen-space approximation that lacks off-screen geometry. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Hold frame time fixed while reallocating a small ray budget among shadows/reflections/GI and changing denoising strength; compare perceptual benefit. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement a frame-time allocator that varies rays per pixel and denoising history; plot shadow/reflection error versus milliseconds on at least three scenes.
- P5 - Machine-readable evidence. Export shadow/reflection error to path-traced reference, rays/pixel, denoiser error, frame time, and temporal stability to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: raster-only approximation vs limited-ray hybrid shadows/reflections under matched frame-time budgets. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Hold frame time fixed while reallocating a small ray budget among shadows/reflections/GI and changing denoising strength; compare perceptual benefit." without hand-editing code between methods. Record shadow/reflection error to path-traced reference, rays/pixel, denoiser error, frame time, and temporal stability.
- P7 - Deliberate bug and regression drill. Intentionally launch secondary rays exactly from the reconstructed surface without a robust origin offset, causing self-intersection artifacts mistaken for hybrid-method noise. Before running, predict the first observable failure in shadow/reflection error to path-traced reference, rays/pixel, denoiser error, frame time, and temporal stability. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the split of work between rasterization and ray queries in a hybrid renderer; analyze the cost/quality effect of ray count, denoising, and visibility reuse on a simple reflection or shadow pipeline. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a mirror plane and one point light, trace the single reflection ray for two pixels and compute the shadow ray. Compare the exact one-bounce result with a screen-space approximation that lacks off-screen geometry. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Add one-bounce ray-traced shadow/reflection to raster renderer or simulate same algorithm on CPU. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement a frame-time allocator that varies rays per pixel and denoising history; plot shadow/reflection error versus milliseconds on at least three scenes. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare raster-only approximation vs limited-ray hybrid shadows/reflections under matched frame-time budgets. Use the same scene, camera path, resolution and quality target; compare under either matched frame-time budget or matched samples/rays/features, and state which budget is fixed. Record shadow/reflection error to path-traced reference, rays/pixel, denoiser error, frame time, and temporal stability.

Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Hold frame time fixed while reallocating a small ray budget among shadows/reflections/GI and changing denoising strength; compare perceptual benefit. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: RTR resource updates; PBRT Ch. 15 for GPU ray pipeline intuition. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Real-time ray tracing and hybrid rendering. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among raster-only approximation vs limited-ray hybrid shadows/reflections under matched frame-time budgets. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using shadow/reflection error to path-traced reference, rays/pixel, denoiser error, frame time, and temporal stability. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if raster-only approximation vs limited-ray hybrid shadows/reflections under matched frame-time budgets move a quality-frame-time Pareto frontier in shadow/reflection error to path-traced reference, rays/pixel, denoiser error, frame time, and temporal stability on multiple scenes/camera motions at a stated fixed budget; reject gains that hide artifacts in unmeasured temporal or edge cases.

F. Concrete research project - Real-time ray tracing and hybrid rendering

F1. Falsifiable question. Under the controlled failure regime "Hold frame time fixed while reallocating a small ray budget among shadows/reflections/GI and changing denoising strength; compare perceptual benefit.", which alternative in raster-only approximation vs limited-ray hybrid shadows/reflections under matched frame-time budgets is preferable, and is the change explained by coverage/visibility conventions, precision, temporal validity, and quality-per-frame-time tradeoffs? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement a frame-time allocator that varies rays per pixel and denoising history; plot shadow/reflection error versus milliseconds on at least three scenes. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use three scenes that separately stress geometry/coverage, lighting/material complexity, and temporal/visibility behavior. Measure at no fewer than three resolutions and report both image/task error and GPU frame time. The central adversarial condition is: Hold frame time fixed while reallocating a small ray budget among shadows/reflections/GI and changing denoising strength; compare perceptual benefit.

F4. Required evidence. Measure shadow/reflection error to path-traced reference, rays/pixel, denoiser error, frame time, and temporal stability. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Real-time ray tracing and hybrid rendering. If a stable failure/crossover is found among raster-only approximation vs limited-ray hybrid shadows/reflections under matched frame-time budgets, implement one adaptive or corrected variant that directly targets coverage/visibility conventions, precision, temporal validity, and quality-per-frame-time tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Real-time ray tracing and hybrid rendering. Using the measured shadow/reflection error to path-traced reference, rays/pixel, denoiser error, frame time, and temporal stability, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Real-time ray tracing and hybrid rendering

- G1. Closed-book derivation/oracle. Derive the split of work between rasterization and ray queries in a hybrid renderer; analyze the cost/quality effect of ray count, denoising, and visibility reuse on a simple reflection or shadow pipeline. Then solve this exact micro-case from scratch without opening your implementation: For a mirror plane and one point light, trace the single reflection ray for two pixels and compute the shadow ray. Compare the exact one-bounce result with a screen-space approximation that lacks off-screen geometry. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Add one-bounce ray-traced shadow/reflection to raster renderer or simulate same algorithm on CPU. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally launch secondary rays exactly from the reconstructed surface without a robust origin offset, causing self-intersection artifacts mistaken for hybrid-method noise. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement a frame-time allocator that varies rays per pixel and denoising history; plot shadow/reflection error versus milliseconds on at least three scenes. Then apply this stress without retuning: Hold frame time fixed while reallocating a small ray budget among shadows/reflections/GI and changing denoising strength; compare perceptual benefit. Report shadow/reflection error to path-traced reference, rays/pixel, denoiser error, frame time, and temporal stability and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among raster-only approximation vs limited-ray hybrid shadows/reflections under matched frame-time budgets for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured shadow/reflection error to path-traced reference, rays/pixel, denoiser error, frame time, and temporal stability; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Real-time ray tracing and hybrid rendering. Using shadow/reflection error to path-traced reference, rays/pixel, denoiser error, frame time, and temporal stability, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 4 Closeout: Project, Review and Mastery Gate

Volume project. Build a small real-time renderer with textured meshes, PBR shading, shadow maps, environment lighting, temporal/post-processing options and a profiling report.

Cumulative review

- Build a small real-time renderer combining rasterization, texture filtering, shadows, PBR shading and temporal AA.
- Expose debug buffers for coverage, depth, derivatives, motion vectors, shadow coordinates and history rejection.
- Produce at least two quality-vs-frame-time crossover plots rather than isolated FPS numbers.
- Demonstrate one artifact caused by a violated sampling/precision/temporal assumption and fix it without hiding the failure.
- Write a rendering-pipeline dependency map explaining where approximations enter and which are scene dependent.

Mastery gate before moving on

- Render the same content through several raster/temporal/hybrid strategies and identify measured quality-performance crossover regimes.
- Pass diagnostic tests for perspective-correct interpolation, shadow bias/filtering, and GGX/IBL energy behavior.
- Explain at least three temporal/raster artifacts from intermediate buffers, not just from the final frame.

Volume 5. Ray Tracing and Rendering Systems

Purpose. Develop a renderer from first principles and understand the architecture of a production-quality ray tracer.

Suggested duration. 12-14 weeks

Prerequisites. Volumes 1-3, plus enough C++ to manage memory, data structures and testing.

Typical module pace. 2.0-2.3 weeks per module (6 modules total).

Primary references for this volume

- Fundamentals of Computer Graphics, Ch. 4 and 12.
- PBRT 4e, Ch. 1, 3, 5-7, 9-10.
- Ray Tracing in One Weekend can be used only as a warm-up, not as the main research reference.

Online spine

Berkeley CS184/284A Spring 2026 course: <https://cs184.eecs.berkeley.edu/sp26/>

PBRT 4th edition - full online book: <https://pbr-book.org/4ed/contents>

Canonical paper track

- Whitted (1980), An Improved Illumination Model for Shaded Display.
- Rubin & Whitted (1980), A 3-Dimensional Representation for Fast Rendering of Complex Scenes.
- Wald et al., BVH/ray tracing engineering papers; compare against PBRT implementation choices.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Volume 5, Unit 1. Camera rays and intersections

Recommended time. 2.0-2.3 weeks (about 30-47 focused hours depending on background).

What you are learning. Pinhole/thin-lens cameras, ray generation, intersections with spheres, triangles, boxes, curves.

A. Reading order

1. Read the primary assignment first: FCG Ch. 4; PBRT Ch. 3, 5, 6.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 ray tracing lectures; PBRT online source.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Möller-Trumbore/barycentric view
- quadratic roots
- watertightness concerns
- Precision requirement: distinguish and justify Möller-Trumbore and watertight/robust variants across grazing, shared-edge, large-coordinate, and degenerate triangles. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive primary rays from pinhole-camera intrinsics/extrinsics and derive robust ray-plane, ray-sphere, and ray-triangle intersections; identify tangent, parallel, and near-degenerate cases.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Test grazing rays, shared triangle edges, near-parallel cases, self-intersections, and scene scales from microscopic to kilometer-like coordinates. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Build a recursive CPU ray tracer with robust triangle/sphere intersections. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Intersect a ray with two triangles sharing an edge, including one hit exactly on the edge and one nearly parallel ray. Compute t and barycentrics in exact arithmetic and identify which tie-breaking convention prevents cracks. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Test grazing rays, shared triangle edges, near-parallel cases, self-intersections, and scene scales from microscopic to kilometer-like coordinates. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Add watertight/shared-edge ray-triangle regression tests and random fuzzing against a high-precision/brute-force reference.
- P5 - Machine-readable evidence. Export intersection t/barycentric error, miss/hit mismatch rate, shared-edge consistency, scale robustness, and rays/s to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: Möller-Trumbore and watertight/robust variants across grazing, shared-edge, large-coordinate, and degenerate triangles. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Test grazing rays, shared triangle edges, near-parallel cases, self-intersections, and scene scales from microscopic to kilometer-like coordinates." without hand-editing code between methods. Record intersection t/barycentric error, miss/hit mismatch rate, shared-edge consistency, scale robustness, and rays/s.
- P7 - Deliberate bug and regression drill. Intentionally normalize ray direction in one intersection path but compare t against an unnormalized reference path. Before running, predict the first observable failure in intersection t/barycentric error, miss/hit mismatch rate, shared-edge consistency, scale robustness, and rays/s. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive primary rays from pinhole-camera intrinsics/extrinsics and derive robust ray-plane, ray-sphere, and ray-triangle intersections; identify tangent, parallel, and near-degenerate cases. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Intersect a ray with two triangles sharing an edge, including one hit exactly on the edge and one nearly parallel ray. Compute t and barycentrics in exact arithmetic and identify which tie-breaking convention prevents cracks. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Build a recursive CPU ray tracer with robust triangle/sphere intersections. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Add watertight/shared-edge ray-triangle regression tests and random fuzzing against a high-precision/brute-force reference. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare Möller-Trumbore and watertight/robust variants across grazing, shared-edge, large-coordinate, and degenerate triangles. Use the same scenes, rays, sampler seeds, image resolution, compiler and hardware; establish correctness against brute force or an analytic/reference image before comparing speed. Record intersection t/barycentric error, miss/hit mismatch rate, shared-edge consistency, scale robustness, and rays/s. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Test grazing rays, shared triangle edges, near-parallel cases, self-intersections, and scene scales from microscopic to kilometer-like coordinates. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 4; PBRT Ch. 3, 5, 6. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Camera rays and intersections. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among Möller-Trumbore and watertight/robust variants across grazing, shared-edge, large-coordinate, and degenerate triangles. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using intersection t/barycentric error, miss/hit mismatch rate, shared-edge consistency, scale robustness, and rays/s. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only after all Möller-Trumbore and watertight/robust variants across grazing, shared-edge, large-coordinate, and degenerate triangles agree with the brute-

force/analytic reference, and only if the crossover in intersection t/barycentric error, miss/hit mismatch rate, shared-edge consistency, scale robustness, and rays/s persists across scene/ray distributions rather than one benchmark.

F. Concrete research project - Camera rays and intersections

F1. Falsifiable question. Under the controlled failure regime "Test grazing rays, shared triangle edges, near-parallel cases, self-intersections, and scene scales from microscopic to kilometer-like coordinates.", characterize the ranking or regime transition across Möller-Trumbore and watertight/robust variants across grazing, shared-edge, large-coordinate, and degenerate triangles. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by geometric robustness, hierarchy quality, ray coherence, and update/traversal cost. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Add watertight/shared-edge ray-triangle regression tests and random fuzzing against a high-precision/brute-force reference. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Keep analytic toy scenes as exact regression tests, then use three larger scenes with distinct geometry/instancing distributions. Separate build/update, traversal, shading, and memory costs. The central adversarial condition is: Test grazing rays, shared triangle edges, near-parallel cases, self-intersections, and scene scales from microscopic to kilometer-like coordinates.

F4. Required evidence. Measure intersection t/barycentric error, miss/hit mismatch rate, shared-edge consistency, scale robustness, and rays/s. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Camera rays and intersections. If a stable failure/crossover is found among Möller-Trumbore and watertight/robust variants across grazing, shared-edge, large-coordinate, and degenerate triangles, implement one adaptive or corrected variant that directly targets geometric robustness, hierarchy quality, ray coherence, and update/traversal cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Camera rays and intersections. Using the measured intersection t/barycentric error, miss/hit mismatch rate, shared-edge consistency, scale robustness, and rays/s, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Camera rays and intersections

- G1. Closed-book derivation/oracle. Derive primary rays from pinhole-camera intrinsics/extrinsics and derive robust ray-plane, ray-sphere, and ray-triangle intersections; identify tangent, parallel, and near-degenerate cases. Then solve this exact micro-case from scratch without opening your implementation: Intersect a ray with two triangles sharing an edge, including one hit exactly on the edge and one nearly parallel ray. Compute t and barycentrics in exact arithmetic and identify which tie-breaking convention prevents cracks. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Build a recursive CPU ray tracer with robust triangle/sphere intersections. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally normalize ray direction in one intersection path but compare t against an unnormalized reference path. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Add watertight/shared-edge ray-triangle regression tests and random fuzzing against a high-precision/brute-force reference. Then apply this stress without retuning: Test grazing rays, shared triangle edges, near-parallel cases, self-intersections, and scene scales from microscopic to kilometer-like coordinates. Report intersection t/barycentric error, miss/hit mismatch rate, shared-edge consistency, scale robustness, and rays/s and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across Möller-Trumbore and watertight/robust variants across grazing, shared-edge, large-coordinate, and degenerate triangles for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or

accuracy-constrained case of your choice. Cite the measured intersection t/barycentric error, miss/hit mismatch rate, shared-edge consistency, scale robustness, and rays/s; state one counterexample that would change the diagnosis or ranking.

- G6. Pass condition for Camera rays and intersections. Using intersection t/barycentric error, miss/hit mismatch rate, shared-edge consistency, scale robustness, and rays/s, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 5, Unit 2. Scene representation

Recommended time. 2.0-2.3 weeks (about 30-47 focused hours depending on background).

What you are learning. Primitives, transforms, instancing, materials, lights, scene graphs.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 3, 6, 7; FCG Ch. 12.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: PBRT system overview.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Object/world transforms
- primitive/shape separation
- instancing
- Precision requirement: distinguish and justify flattened scene arrays vs hierarchical scene graph with instancing and transform stacks on repeated/animated scenes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Compare scene graphs, instancing, transforms, and object/material separation by deriving the memory and update cost of several representations for repeated geometry and animated transforms.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use deep instancing, animated transform hierarchies, shared materials, and millions of repeated primitives; test update/query consistency and memory. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Design a scene file and object hierarchy; support instancing and transform stacks. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Create a scene graph with one chair mesh instanced twice under different transforms and a moving parent node. Compute both world transforms and show what data are shared versus duplicated in flattened storage. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use deep instancing, animated transform hierarchies, shared materials, and millions of repeated primitives; test update/query consistency and memory. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Serialize a scene with 1000 instances of a few meshes, then compare flattened and hierarchical memory/update costs while animating parent transforms.
- P5 - Machine-readable evidence. Export memory per instance, traversal/update time, serialization size, transform correctness, and instancing savings to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: flattened scene arrays vs hierarchical scene graph with instancing and transform stacks on repeated/animated scenes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use deep instancing, animated transform hierarchies, shared materials, and millions of repeated primitives; test update/query consistency and memory." without hand-editing code between methods. Record memory per instance, traversal/update time, serialization size, transform correctness, and instancing savings.

- P7 - Deliberate bug and regression drill. Intentionally apply an instance transform twice when traversing nested scene nodes. Before running, predict the first observable failure in memory per instance, traversal/update time, serialization size, transform correctness, and instancing savings. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Compare scene graphs, instancing, transforms, and object/material separation by deriving the memory and update cost of several representations for repeated geometry and animated transforms. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Create a scene graph with one chair mesh instanced twice under different transforms and a moving parent node. Compute both world transforms and show what data are shared versus duplicated in flattened storage. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Design a scene file and object hierarchy; support instancing and transform stacks. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Serialize a scene with 1000 instances of a few meshes, then compare flattened and hierarchical memory/update costs while animating parent transforms. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare flattened scene arrays vs hierarchical scene graph with instancing and transform stacks on repeated/animated scenes. Use the same scenes, rays, sampler seeds, image resolution, compiler and hardware; establish correctness against brute force or an analytic/reference image before comparing speed. Record memory per instance, traversal/update time, serialization size, transform correctness, and instancing savings. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use deep instancing, animated transform hierarchies, shared materials, and millions of repeated primitives; test update/query consistency and memory. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: PBRT Ch. 3, 6, 7; FCG Ch. 12. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Scene representation. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among flattened scene arrays vs hierarchical scene graph with instancing and transform stacks on repeated/animated scenes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using memory per instance, traversal/update time, serialization size, transform correctness, and instancing savings. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only after all flattened scene arrays vs hierarchical scene graph with instancing and transform stacks on repeated/animated scenes agree with the brute-force/analytic reference, and only if the crossover in memory per instance, traversal/update time, serialization size, transform correctness, and instancing savings persists across scene/ray distributions rather than one benchmark.

F. Concrete research project - Scene representation

- F1. Falsifiable question. Under the controlled failure regime "Use deep instancing, animated transform hierarchies, shared materials, and millions of repeated primitives; test update/query consistency and memory.", which alternative in flattened scene arrays vs hierarchical scene graph with instancing and transform stacks on repeated/animated scenes is preferable, and is the change explained by geometric robustness, hierarchy quality, ray coherence, and update/traversal cost? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Serialize a scene with 1000 instances of a few meshes, then compare flattened and hierarchical memory/update costs while animating parent transforms. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Keep analytic toy scenes as exact regression tests, then use three larger scenes with distinct geometry/instancing distributions. Separate build/update, traversal, shading, and memory costs. The central adversarial condition is: Use deep instancing, animated transform hierarchies, shared materials, and millions of repeated primitives; test update/query consistency and memory.

F4. Required evidence. Measure memory per instance, traversal/update time, serialization size, transform correctness, and instancing savings. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Scene representation. If a stable failure/crossover is found among flattened scene arrays vs hierarchical scene graph with instancing and transform stacks on repeated/animated scenes, implement one adaptive or corrected variant that directly targets geometric robustness, hierarchy quality, ray coherence, and update/traversal cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Scene representation. Using the measured memory per instance, traversal/update time, serialization size, transform correctness, and instancing savings, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Scene representation

- G1. Closed-book derivation/oracle. Compare scene graphs, instancing, transforms, and object/material separation by deriving the memory and update cost of several representations for repeated geometry and animated transforms. Then solve this exact micro-case from scratch without opening your implementation: Create a scene graph with one chair mesh instanced twice under different transforms and a moving parent node. Compute both world transforms and show what data are shared versus duplicated in flattened storage. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Design a scene file and object hierarchy; support instancing and transform stacks. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally apply an instance transform twice when traversing nested scene nodes. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Serialize a scene with 1000 instances of a few meshes, then compare flattened and hierarchical memory/update costs while animating parent transforms. Then apply this stress without retuning: Use deep instancing, animated transform hierarchies, shared materials, and millions of repeated primitives; test update/query consistency and memory. Report memory per instance, traversal/update time, serialization size, transform correctness, and instancing savings and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among flattened scene arrays vs hierarchical scene graph with instancing and transform stacks on repeated/animated scenes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured memory per instance, traversal/update time, serialization size, transform correctness, and instancing savings; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Scene representation. Using memory per instance, traversal/update time, serialization size, transform correctness, and instancing savings, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 5, Unit 3. Acceleration structures

Recommended time. 2.0-2.3 weeks (about 30-47 focused hours depending on background).

What you are learning. AABBs, BVHs, SAH intuition, traversal order, refit/rebuild.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 7; FCG Ch. 12.3.

2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 acceleration structure lecture.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Bounding hierarchy invariants
- SAH cost model
- Precision requirement: distinguish and justify midpoint, median, binned SAH build, rebuild, and refit strategies under coherent/incoherent rays and deformation. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the surface-area heuristic from an expected ray cost model, then work through one BVH split numerically and compare it with median/midpoint splitting on clustered geometry.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Compare rebuild versus refit on static, mildly deforming, and chaotic geometry; test coherent primary rays and incoherent secondary rays separately. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement BVH build/traversal; compare midpoint, median, approximate SAH; benchmark coherent/incoherent rays. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For four AABBs with x ranges [0,1],[1.1,2],[8,9],[9.1,10], compute a one-level SAH split cost for all three split positions and compare with the median split. Then perturb the last box toward the first cluster and recompute. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Compare rebuild versus refit on static, mildly deforming, and chaotic geometry; test coherent primary rays and incoherent secondary rays separately. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement binned SAH plus refit; record SAH cost, overlap, depth, traversal steps, and rebuild/refit crossover as vertex displacement increases.
- P5 - Machine-readable evidence. Export build/refit time, rays/s, nodes/triangle tests per ray, SAH/overlap statistics, memory, and crossover deformation magnitude to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: midpoint, median, binned SAH build, rebuild, and refit strategies under coherent/incoherent rays and deformation. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Compare rebuild versus refit on static, mildly deforming, and chaotic geometry; test coherent primary rays and incoherent secondary rays separately." without hand-editing code between methods. Record build/refit time, rays/s, nodes/triangle tests per ray, SAH/overlap statistics, memory, and crossover deformation magnitude.
- P7 - Deliberate bug and regression drill. Intentionally refit leaf boxes after deformation but forget to propagate updated bounds to all ancestors. Before running, predict the first observable failure in build/refit time, rays/s, nodes/triangle tests per ray, SAH/overlap statistics, memory, and crossover deformation magnitude. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the surface-area heuristic from an expected ray cost model, then work through one BVH split numerically and compare it with median/midpoint splitting on clustered geometry. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For four AABBs with x ranges [0,1],[1.1,2],[8,9],[9.1,10], compute a one-level SAH split cost for all three split positions and compare with the median split. Then perturb the last box toward the first cluster and recompute. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Implement BVH build/traversal; compare midpoint, median, approximate SAH; benchmark coherent/incoherent rays. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement binned SAH plus refit; record SAH cost, overlap, depth, traversal steps, and rebuild/refit crossover as vertex displacement increases. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare midpoint, median, binned SAH build, rebuild, and refit strategies under coherent/incoherent rays and deformation. Use the same scenes, rays, sampler seeds, image resolution, compiler and hardware; establish correctness against brute force or an analytic/reference image before comparing speed. Record build/refit time, rays/s, nodes/triangle tests per ray, SAH/overlap statistics, memory, and crossover deformation magnitude. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Compare rebuild versus refit on static, mildly deforming, and chaotic geometry; test coherent primary rays and incoherent secondary rays separately. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: PBRT Ch. 7; FCG Ch. 12.3. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Acceleration structures. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among midpoint, median, binned SAH build, rebuild, and refit strategies under coherent/incoherent rays and deformation. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using build/refit time, rays/s, nodes/triangle tests per ray, SAH/overlap statistics, memory, and crossover deformation magnitude. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if a repeatable deformation/coherence regime changes the rebuild/refit or split-strategy ranking, and measured overlap/SAH/traversal statistics explain the crossover on more than one scene.

F. Concrete research project - Acceleration structures

- F1. Falsifiable question. Under the controlled failure regime "Compare rebuild versus refit on static, mildly deforming, and chaotic geometry; test coherent primary rays and incoherent secondary rays separately.", characterize the ranking or regime transition across midpoint, median, binned SAH build, rebuild, and refit strategies under coherent/incoherent rays and deformation. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by bounding-volume overlap, split quality, ray coherence, rebuild/refit degradation, and update-versus-traversal cost. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement binned SAH plus refit; record SAH cost, overlap, depth, traversal steps, and rebuild/refit crossover as vertex displacement increases. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Keep analytic toy scenes as exact regression tests, then use three larger scenes with distinct geometry/instancing distributions. Separate build/update, traversal, shading, and memory costs. The central adversarial condition is: Compare rebuild versus refit on static, mildly deforming, and chaotic geometry; test coherent primary rays and incoherent secondary rays separately.
- F4. Required evidence. Measure build/refit time, rays/s, nodes/triangle tests per ray, SAH/overlap statistics, memory, and crossover deformation magnitude. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Acceleration structures. If a stable failure/crossover is found among midpoint, median, binned SAH build, rebuild, and refit strategies under coherent/incoherent rays and deformation, implement one adaptive or corrected variant that directly targets bounding-volume overlap, split quality, ray coherence, rebuild/refit degradation, and update-versus-traversal cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Acceleration structures. Using the measured build/refit time, rays/s, nodes/triangle tests per ray, SAH/overlap statistics, memory, and crossover deformation magnitude, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Acceleration structures

- G1. Closed-book derivation/oracle. Derive the surface-area heuristic from an expected ray cost model, then work through one BVH split numerically and compare it with median/midpoint splitting on clustered geometry. Then solve this exact micro-case from scratch without opening your implementation: For four AABBs with x ranges [0,1], [1.1,2],[8,9],[9.1,10], compute a one-level SAH split cost for all three split positions and compare with the median split. Then perturb the last box toward the first cluster and recompute. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement BVH build/traversal; compare midpoint, median, approximate SAH; benchmark coherent/incoherent rays. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally refit leaf boxes after deformation but forget to propagate updated bounds to all ancestors. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement binned SAH plus refit; record SAH cost, overlap, depth, traversal steps, and rebuild/refit crossover as vertex displacement increases. Then apply this stress without retuning: Compare rebuild versus refit on static, mildly deforming, and chaotic geometry; test coherent primary rays and incoherent secondary rays separately. Report build/refit time, rays/s, nodes/triangle tests per ray, SAH/overlap statistics, memory, and crossover deformation magnitude and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across midpoint, median, binned SAH build, rebuild, and refit strategies under coherent/incoherent rays and deformation for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured build/refit time, rays/s, nodes/triangle tests per ray, SAH/overlap statistics, memory, and crossover deformation magnitude; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Acceleration structures. Using build/refit time, rays/s, nodes/triangle tests per ray, SAH/overlap statistics, memory, and crossover deformation magnitude, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 5, Unit 4. Camera and film models

Recommended time. 2.0-2.3 weeks (about 30-47 focused hours depending on background).

What you are learning. Depth of field, sensor/film filtering, spherical cameras, motion blur concepts.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 5.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: PBRT camera examples.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Lens sampling
- reconstruction filter
- exposure
- Precision requirement: distinguish and justify pinhole vs thin-lens camera, different aperture shapes, and reconstruction filters against analytic depth-of-field expectations. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive pinhole projection, thin-lens depth of field, and a finite-aperture camera ray sampler; relate aperture radius, focal distance, and circle of confusion.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use extreme apertures/focus distances, motion blur, crop windows, and nontrivial reconstruction filters; verify against limiting analytic cases. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement thin-lens depth of field; compare aperture shapes; add reconstruction filters. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a pinhole/thin-lens camera with focal distance 5 and aperture radius 0.1, trace two lens samples to the same focus point for an object at $z=-5$, then repeat for an object at $z=-3$ and predict circle-of-confusion direction. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use extreme apertures/focus distances, motion blur, crop windows, and nontrivial reconstruction filters; verify against limiting analytic cases. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Render a calibrated focus chart with varying aperture/focus distance and compare measured circle-of-confusion radii with the thin-lens formula.
- P5 - Machine-readable evidence. Export circle-of-confusion error, focal-plane sharpness, aperture-shape effect, reconstruction-filter MTF/error, and sample cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: pinhole vs thin-lens camera, different aperture shapes, and reconstruction filters against analytic depth-of-field expectations. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use extreme apertures/focus distances, motion blur, crop windows, and nontrivial reconstruction filters; verify against limiting analytic cases." without hand-editing code between methods. Record circle-of-confusion error, focal-plane sharpness, aperture-shape effect, reconstruction-filter MTF/error, and sample cost.
- P7 - Deliberate bug and regression drill. Intentionally sample the aperture correctly but focus rays using the image plane instead of the intended focal plane. Before running, predict the first observable failure in circle-of-confusion error, focal-plane sharpness, aperture-shape effect, reconstruction-filter MTF/error, and sample cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive pinhole projection, thin-lens depth of field, and a finite-aperture camera ray sampler; relate aperture radius, focal distance, and circle of confusion. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a pinhole/thin-lens camera with focal distance 5 and aperture radius 0.1, trace two lens samples to the same focus point for an object at $z=-5$, then repeat for an object at $z=-3$ and predict circle-of-confusion direction. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement thin-lens depth of field; compare aperture shapes; add reconstruction filters. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Render a calibrated focus chart with varying aperture/focus distance and compare measured circle-of-confusion radii with the thin-lens formula. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare pinhole vs thin-lens camera, different aperture shapes, and reconstruction filters against analytic depth-of-field expectations. Use the same scenes, rays, sampler seeds, image resolution, compiler and hardware; establish correctness against brute force or an analytic/reference image before comparing speed. Record circle-of-confusion error, focal-plane sharpness, aperture-shape effect, reconstruction-filter MTF/error, and sample cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use extreme apertures/focus distances, motion blur, crop windows, and nontrivial reconstruction filters; verify against limiting analytic cases. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: PBRT Ch. 5. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Camera and film models. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among pinhole vs thin-lens camera, different aperture shapes, and reconstruction filters against analytic depth-of-field expectations. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using circle-of-confusion error, focal-plane sharpness, aperture-shape effect, reconstruction-filter MTF/error, and sample cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only after all pinhole vs thin-lens camera, different aperture shapes, and reconstruction filters against analytic depth-of-field expectations agree with the brute-force/analytic reference, and only if the crossover in circle-of-confusion error, focal-plane sharpness, aperture-shape effect, reconstruction-filter MTF/error, and sample cost persists across scene/ray distributions rather than one benchmark.

F. Concrete research project - Camera and film models

- F1. Falsifiable question. Under the controlled failure regime "Use extreme apertures/focus distances, motion blur, crop windows, and nontrivial reconstruction filters; verify against limiting analytic cases.", which alternative in pinhole vs thin-lens camera, different aperture shapes, and reconstruction filters against analytic depth-of-field expectations is preferable, and is the change explained by geometric robustness, scene/material abstraction, numerical precision, and build/traversal/shading engineering cost? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Render a calibrated focus chart with varying aperture/focus distance and compare measured circle-of-confusion radii with the thin-lens formula. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Keep analytic toy scenes as exact regression tests, then use three larger scenes with distinct geometry/instancing distributions. Separate build/update, traversal, shading, and memory costs. The central adversarial condition is: Use extreme apertures/focus distances, motion blur, crop windows, and nontrivial reconstruction filters; verify against limiting analytic cases.
- F4. Required evidence. Measure circle-of-confusion error, focal-plane sharpness, aperture-shape effect, reconstruction-filter MTF/error, and sample cost. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Camera and film models. If a stable failure/crossover is found among pinhole vs thin-lens camera, different aperture shapes, and reconstruction filters against analytic depth-of-field expectations, implement one adaptive or corrected variant that directly targets geometric robustness, scene/material abstraction, numerical precision, and build/traversal/shading engineering cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Camera and film models. Using the measured circle-of-confusion error, focal-plane sharpness, aperture-shape effect, reconstruction-filter MTF/error, and sample cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Camera and film models

- G1. Closed-book derivation/oracle. Derive pinhole projection, thin-lens depth of field, and a finite-aperture camera ray sampler; relate aperture radius, focal distance, and circle of confusion. Then solve this exact micro-case from scratch without opening your implementation: For a pinhole/thin-lens camera with focal distance 5 and aperture

radius 0.1, trace two lens samples to the same focus point for an object at $z=-5$, then repeat for an object at $z=-3$ and predict circle-of-confusion direction. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement thin-lens depth of field; compare aperture shapes; add reconstruction filters. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally sample the aperture correctly but focus rays using the image plane instead of the intended focal plane. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Render a calibrated focus chart with varying aperture/focus distance and compare measured circle-of-confusion radii with the thin-lens formula. Then apply this stress without retuning: Use extreme apertures/focus distances, motion blur, crop windows, and nontrivial reconstruction filters; verify against limiting analytic cases. Report circle-of-confusion error, focal-plane sharpness, aperture-shape effect, reconstruction-filter MTF/error, and sample cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among pinhole vs thin-lens camera, different aperture shapes, and reconstruction filters against analytic depth-of-field expectations for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured circle-of-confusion error, focal-plane sharpness, aperture-shape effect, reconstruction-filter MTF/error, and sample cost; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Camera and film models. Using circle-of-confusion error, focal-plane sharpness, aperture-shape effect, reconstruction-filter MTF/error, and sample cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 5, Unit 5. Materials and textures architecture

Recommended time. 2.0-2.3 weeks (about 30-47 focused hours depending on background).

What you are learning. BSDF abstraction, textures, bump/normal mapping, measured data.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 9-10.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Mitsuba material plugin examples as comparison.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Separation of geometry/material/scattering
- local frames
- Precision requirement: distinguish and justify separate texture/material/BSDF architecture vs monolithic shading code, validated on diffuse/conductor/dielectric/microfacet cases. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a clean separation between texture evaluation, material parameterization, BSDF construction, and scattering evaluation; trace one textured microfacet material from UV lookup to sampled outgoing ray.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use mixtures, nested textures, normal maps, and importance-sampled lobes; verify that evaluation, sample generation, and reported PDFs agree numerically. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create a material test scene; implement diffuse, conductor, dielectric and rough microfacet materials. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a shading point with a constant albedo texture, normal map disabled, and two BSDF lobes, write the exact sample/eval/pdf calls required

by a path tracer and check that a delta lobe is not treated like a density-valued continuous lobe. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Use mixtures, nested textures, normal maps, and importance-sampled lobes; verify that evaluation, sample generation, and reported PDFs agree numerically. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build a material unit-test scene and numerical BSDF test harness that checks sample/eval/pdf consistency, reciprocity where applicable, and energy.
- P5 - Machine-readable evidence. Export BSDF energy/reciprocity/sample-eval-pdf residuals, texture lookup correctness, reference-image error, and test coverage to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: separate texture/material/BSDF architecture vs monolithic shading code, validated on diffuse/conductor/dielectric/microfacet cases. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use mixtures, nested textures, normal maps, and importance-sampled lobes; verify that evaluation, sample generation, and reported PDFs agree numerically." without hand-editing code between methods. Record BSDF energy/reciprocity/sample-eval-pdf residuals, texture lookup correctness, reference-image error, and test coverage.
- P7 - Deliberate bug and regression drill. Intentionally return a sampled BSDF direction and value from one lobe while reporting the PDF of another lobe. Before running, predict the first observable failure in BSDF energy/reciprocity/sample-eval-pdf residuals, texture lookup correctness, reference-image error, and test coverage. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a clean separation between texture evaluation, material parameterization, BSDF construction, and scattering evaluation; trace one textured microfacet material from UV lookup to sampled outgoing ray. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation.

Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. For a shading point with a constant albedo texture, normal map disabled, and two BSDF lobes, write the exact sample/eval/pdf calls required by a path tracer and check that a delta lobe is not treated like a density-valued continuous lobe. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Create a material test scene; implement diffuse, conductor, dielectric and rough microfacet materials. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build a material unit-test scene and numerical BSDF test harness that checks sample/eval/pdf consistency, reciprocity where applicable, and energy. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare separate texture/material/BSDF architecture vs monolithic shading code, validated on diffuse/conductor/dielectric/microfacet cases. Use the same scenes, rays, sampler seeds, image resolution, compiler and hardware; establish correctness against brute force or an analytic/reference image before comparing speed. Record BSDF energy/reciprocity/sample-eval-pdf residuals, texture lookup correctness, reference-image error, and test coverage. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use mixtures, nested textures, normal maps, and importance-sampled lobes; verify that evaluation, sample generation, and reported PDFs agree numerically. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: PBRT Ch. 9-10. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Materials and textures architecture. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among separate texture/material/BSDF architecture vs monolithic shading code, validated on diffuse/conductor/dielectric/microfacet cases. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using BSDF energy/reciprocity/sample-eval-pdf residuals, texture lookup correctness, reference-image error, and test

coverage. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only after all separate texture/material/BSDF architecture vs monolithic shading code, validated on diffuse/conductor/dielectric/microfacet cases agree with the brute-force/analytic reference, and only if the crossover in BSDF energy/reciprocity/sample-eval-pdf residuals, texture lookup correctness, reference-image error, and test coverage persists across scene/ray distributions rather than one benchmark.

F. Concrete research project - Materials and textures architecture

F1. Falsifiable question. Under the controlled failure regime "Use mixtures, nested textures, normal maps, and importance-sampled lobes; verify that evaluation, sample generation, and reported PDFs agree numerically.", which alternative in separate texture/material/BSDF architecture vs monolithic shading code, validated on diffuse/conductor/dielectric/microfacet cases is preferable, and is the change explained by spectral support, footprint estimation, and aliasing/reconstruction error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build a material unit-test scene and numerical BSDF test harness that checks sample/eval/pdf consistency, reciprocity where applicable, and energy. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Keep analytic toy scenes as exact regression tests, then use three larger scenes with distinct geometry/instancing distributions. Separate build/update, traversal, shading, and memory costs. The central adversarial condition is: Use mixtures, nested textures, normal maps, and importance-sampled lobes; verify that evaluation, sample generation, and reported PDFs agree numerically.

F4. Required evidence. Measure BSDF energy/reciprocity/sample-eval-pdf residuals, texture lookup correctness, reference-image error, and test coverage. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Materials and textures architecture. If a stable failure/crossover is found among separate texture/material/BSDF architecture vs monolithic shading code, validated on diffuse/conductor/dielectric/microfacet cases, implement one adaptive or corrected variant that directly targets spectral support, footprint estimation, and aliasing/reconstruction error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Materials and textures architecture. Using the measured BSDF energy/reciprocity/sample-eval-pdf residuals, texture lookup correctness, reference-image error, and test coverage, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Materials and textures architecture

- G1. Closed-book derivation/oracle. Derive a clean separation between texture evaluation, material parameterization, BSDF construction, and scattering evaluation; trace one textured microfacet material from UV lookup to sampled outgoing ray. Then solve this exact micro-case from scratch without opening your implementation: For a shading point with a constant albedo texture, normal map disabled, and two BSDF lobes, write the exact sample/eval/pdf calls required by a path tracer and check that a delta lobe is not treated like a density-valued continuous lobe. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create a material test scene; implement diffuse, conductor, dielectric and rough microfacet materials. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally return a sampled BSDF direction and value from one lobe while reporting the PDF of another lobe. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Build a material unit-test scene and numerical BSDF test harness that checks sample/eval/pdf consistency, reciprocity where applicable,

and energy. Then apply this stress without retuning: Use mixtures, nested textures, normal maps, and importance-sampled lobes; verify that evaluation, sample generation, and reported PDFs agree numerically. Report BSDF energy/reciprocity/sample-eval-pdf residuals, texture lookup correctness, reference-image error, and test coverage and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method-selection defense. Using your own measured results, choose among separate texture/material/BSDF architecture vs monolithic shading code, validated on diffuse/conductor/dielectric/microfacet cases for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured BSDF energy/reciprocity/sample-eval-pdf residuals, texture lookup correctness, reference-image error, and test coverage; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Materials and textures architecture. Using BSDF energy/reciprocity/sample-eval-pdf residuals, texture lookup correctness, reference-image error, and test coverage, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 5, Unit 6. Renderer engineering

Recommended time. 2.0-2.3 weeks (about 30-47 focused hours depending on background).

What you are learning. Testing, deterministic sampling, image regression, performance profiling, memory layout.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 1 and utilities appendix as reference.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Read PBRT code organization.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Golden-image testing
- numerical tolerances
- reproducible seeds
- Precision requirement: distinguish and justify single-threaded reference vs optimized/parallel renderer while preserving deterministic tests and image regressions. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a dependency/dataflow graph for a small path tracer, identifying immutable scene data, per-ray state, sampler state, and output accumulation; reason about determinism, error handling, and test oracles.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Create a permanent diagnostic suite for transforms, normals, energy, PDFs/MIS, NaNs, self-intersection, motion blur, and determinism; every bug fix must add a regression case. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create unit tests for intersection/BSDF sampling and render regression tests; profile hot loops. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Render a two-sphere scene at 16x16 with a fixed deterministic sampler. Specify a pixel checksum/reference set, then predict which refactors (thread order, BVH reordering, floating-point reductions) may change bits without changing the estimator. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Create a permanent diagnostic suite for transforms, normals, energy, PDFs/MIS, NaNs, self-intersection, motion blur, and determinism; every bug fix must add a regression case. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create deterministic render tests at 16x16/64 spp with image hashes/tolerances, plus per-stage timing and a reproducible profiler trace for one hot path.
- P5 - Machine-readable evidence. Export regression-image error, deterministic rerun rate, stage timings, memory, profiler hotspot fraction, and test failure localization time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.

- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: single-threaded reference vs optimized/parallel renderer while preserving deterministic tests and image regressions. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Create a permanent diagnostic suite for transforms, normals, energy, PDFs/MIS, NaNs, self-intersection, motion blur, and determinism; every bug fix must add a regression case." without hand-editing code between methods. Record regression-image error, deterministic rerun rate, stage timings, memory, profiler hotspot fraction, and test failure localization time.
- P7 - Deliberate bug and regression drill. Intentionally share a mutable RNG across threads so image regressions change with thread count. Before running, predict the first observable failure in regression-image error, deterministic rerun rate, stage timings, memory, profiler hotspot fraction, and test failure localization time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a dependency/dataflow graph for a small path tracer, identifying immutable scene data, per-ray state, sampler state, and output accumulation; reason about determinism, error handling, and test oracles. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Render a two-sphere scene at 16x16 with a fixed deterministic sampler. Specify a pixel checksum/reference set, then predict which refactors (thread order, BVH reordering, floating-point reductions) may change bits without changing the estimator. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Create unit tests for intersection/BSDF sampling and render regression tests; profile hot loops. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create deterministic render tests at 16x16/64 spp with image hashes/tolerances, plus per-stage timing and a reproducible profiler trace for one hot path. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare single-threaded reference vs optimized/parallel renderer while preserving deterministic tests and image regressions. Use the same scenes, rays, sampler seeds, image resolution, compiler and hardware; establish correctness against brute force or an analytic/reference image before comparing speed. Record regression-image error, deterministic rerun rate, stage timings, memory, profiler hotspot fraction, and test failure localization time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Create a permanent diagnostic suite for transforms, normals, energy, PDFs/MIS, NaNs, self-intersection, motion blur, and determinism; every bug fix must add a regression case. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: PBRT Ch. 1 and utilities appendix as reference. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Renderer engineering. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among single-threaded reference vs optimized/parallel renderer while preserving deterministic tests and image regressions. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using regression-image error, deterministic rerun rate, stage timings, memory, profiler hotspot fraction, and test failure localization time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only after all single-threaded reference vs optimized/parallel renderer while preserving deterministic tests and image regressions agree with the brute-force/analytic reference, and only if the crossover in regression-image error, deterministic rerun rate, stage timings, memory, profiler hotspot fraction, and test failure localization time persists across scene/ray distributions rather than one benchmark.

F. Concrete research project - Renderer engineering

- F1. Falsifiable question. Under the controlled failure regime "Create a permanent diagnostic suite for transforms, normals, energy, PDFs/MIS, NaNs, self-intersection, motion blur, and determinism; every bug fix must add a regression

case.", which alternative in single-threaded reference vs optimized/parallel renderer while preserving deterministic tests and image regressions is preferable, and is the change explained by memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create deterministic render tests at 16x16/64 spp with image hashes/tolerances, plus per-stage timing and a reproducible profiler trace for one hot path. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Keep analytic toy scenes as exact regression tests, then use three larger scenes with distinct geometry/instancing distributions. Separate build/update, traversal, shading, and memory costs. The central adversarial condition is: Create a permanent diagnostic suite for transforms, normals, energy, PDFs/MIS, NaNs, self-intersection, motion blur, and determinism; every bug fix must add a regression case.

F4. Required evidence. Measure regression-image error, deterministic rerun rate, stage timings, memory, profiler hotspot fraction, and test failure localization time. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Renderer engineering. If a stable failure/crossover is found among single-threaded reference vs optimized/parallel renderer while preserving deterministic tests and image regressions, implement one adaptive or corrected variant that directly targets memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Renderer engineering. Using the measured regression-image error, deterministic rerun rate, stage timings, memory, profiler hotspot fraction, and test failure localization time, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Renderer engineering

- G1. Closed-book derivation/oracle. Derive a dependency/dataflow graph for a small path tracer, identifying immutable scene data, per-ray state, sampler state, and output accumulation; reason about determinism, error handling, and test oracles. Then solve this exact micro-case from scratch without opening your implementation: Render a two-sphere scene at 16x16 with a fixed deterministic sampler. Specify a pixel checksum/reference set, then predict which refactors (thread order, BVH reordering, floating-point reductions) may change bits without changing the estimator. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create unit tests for intersection/BSDF sampling and render regression tests; profile hot loops. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally share a mutable RNG across threads so image regressions change with thread count. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create deterministic render tests at 16x16/64 spp with image hashes/tolerances, plus per-stage timing and a reproducible profiler trace for one hot path. Then apply this stress without retuning: Create a permanent diagnostic suite for transforms, normals, energy, PDFs/MIS, NaNs, self-intersection, motion blur, and determinism; every bug fix must add a regression case. Report regression-image error, deterministic rerun rate, stage timings, memory, profiler hotspot fraction, and test failure localization time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among single-threaded reference vs optimized/parallel renderer while preserving deterministic tests and image regressions for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured regression-image error, deterministic rerun rate, stage timings, memory,

profiler hotspot fraction, and test failure localization time; state one counterexample that would reverse your recommendation.

- G6. Pass condition for Renderer engineering. Using regression-image error, deterministic rerun rate, stage timings, memory, profiler hotspot fraction, and test failure localization time, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 5 Closeout: Project, Review and Mastery Gate

Volume project. Build a CPU ray tracer with camera, primitives/meshes, BVH, instancing, materials/textures, depth of field, regression tests and timing benchmarks.

Cumulative review

- Package a deterministic ray-tracing core with robust intersections, instancing, BVH tests, camera/film and BSDF architecture.
- Fuzz intersections and acceleration traversal against a brute-force/high-precision reference and retain minimized failures.
- Measure BVH build/refit/traversal separately across coherent and incoherent rays.
- Run deterministic low-resolution render regression tests plus one profiler trace of a real bottleneck.
- Write a renderer-engineering memo distinguishing mathematical error, robustness error, scene-representation cost and execution cost.

Mastery gate before moving on

- Pass robust intersection and BVH tests across scale, degeneracy, deformation, and ray-coherence regimes.
- Trace a sample/pixel through camera, scene, material/texture, acceleration, and film code and explain every stored quantity.
- Maintain a renderer regression suite covering transforms/normals, self-intersection, materials/PDFs, camera/film, and determinism.

Volume 6. Physically Based Rendering and Light Transport

Purpose. Reach a research-capable understanding of radiometry, BSDFs, Monte Carlo transport, participating media, and advanced path-space methods.

Suggested duration. 18-22 weeks

Prerequisites. Volumes 1, 3 and 5. Monte Carlo probability and ray tracing are essential.

Typical module pace. 2.2-2.8 weeks per module (8 modules total).

Primary references for this volume

- PBRT 4e, Ch. 2, 4, 8-15 (primary).
- Veach thesis selections for path-space Monte Carlo and MIS.
- Real-Time Rendering Ch. 9 for real-time PBR cross-reference.

Online spine

PBRT 4th edition - full online book: <https://pbr-book.org/4ed/contents>

Berkeley CS184/284A Spring 2026 course: <https://cs184.eecs.berkeley.edu/sp26/>

Mitsuba 3 documentation: <https://mitsuba.readthedocs.io/en/stable/>

Canonical paper track

- Kajiya (1986), The Rendering Equation.
- Veach & Guibas (1995), Optimally Combining Sampling Techniques for Monte Carlo Rendering.
- Veach (1997), Robust Monte Carlo Methods for Light Transport Simulation (thesis selections).
- Jensen (1996/2001), Photon Mapping.
- Jakob et al. / modern microfacet and spectral rendering papers as appropriate.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Kajiya (1986), The Rendering Equation - derive the path-space interpretation before implementing more advanced estimators.
- Veach & Guibas (1995), Optimally Combining Sampling Techniques for Monte Carlo Rendering - use this for the MIS unit.
- Veach (1997), Robust Monte Carlo Methods for Light Transport Simulation - selected path-space and bidirectional chapters.
- PBRT 4e, Chs. 2, 4, 8-15 - executable reference for sampling, scattering, media, path tracing and GPU wavefront rendering.
- Laine, Karras & Aila (2013), Megakernels Considered Harmful: Wavefront Path Tracing on GPUs - contrast GPU execution organizations.

Volume 6, Unit 1. Radiometry and the rendering equation

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Radiance, irradiance, projected solid angle, invariance of radiance, surface transport.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 4 and 13.1; FCG Ch. 14.6-14.8.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 radiometry/global illumination lectures.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Rendering equation
- measurement units
- cosine factor
- radiance invariance along vacuum rays
- Precision requirement: distinguish and justify analytic/direct-lighting cases vs Monte Carlo estimates with units and inverse-square/cosine laws checked explicitly. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive radiance invariance along a vacuum ray and the cosine-projected area measure, then derive the rendering equation from local energy balance and identify emitted, direct, and indirect terms.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Change emitter area, distance, orientation, and scene scale while checking inverse-square/projected-area behavior and dimensional consistency. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Derive units for each term; verify inverse-square law numerically; implement direct illumination estimator. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: A unit-area Lambertian patch with $\rho=0.8$ faces a point light of intensity I at distance 2. Compute irradiance and outgoing radiance at normal incidence and at 60 degrees, keeping units explicit. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Change emitter area, distance, orientation, and scene scale while checking inverse-square/projected-area behavior and dimensional consistency. Save the input and config of each qualitatively different failure as a named regression case.

- P4 - Research-grade programming extension. Create analytic scenes (Lambertian patch, point/area light) where irradiance/radiance can be calculated; compare estimator means and units numerically.
- P5 - Machine-readable evidence. Export unit consistency, analytic irradiance/radiance relative error, estimator bias/variance, and inverse-square/cosine-law residuals to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: analytic/direct-lighting cases vs Monte Carlo estimates with units and inverse-square/cosine laws checked explicitly. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Change emitter area, distance, orientation, and scene scale while checking inverse-square/projected-area behavior and dimensional consistency." without hand-editing code between methods. Record unit consistency, analytic irradiance/radiance relative error, estimator bias/variance, and inverse-square/cosine-law residuals.
- P7 - Deliberate bug and regression drill. Intentionally mix irradiance and radiance and omit one cosine or inverse-square factor in the direct-lighting oracle. Before running, predict the first observable failure in unit consistency, analytic irradiance/radiance relative error, estimator bias/variance, and inverse-square/cosine-law residuals. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive radiance invariance along a vacuum ray and the cosine-projected area measure, then derive the rendering equation from local energy balance and identify emitted, direct, and indirect terms. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. A unit-area Lambertian patch with $\rho=0.8$ faces a point light of intensity I at distance 2. Compute irradiance and outgoing radiance at normal incidence and at 60 degrees, keeping units explicit. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Derive units for each term; verify inverse-square law numerically; implement direct illumination estimator. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create analytic scenes (Lambertian patch, point/area light) where irradiance/radiance can be calculated; compare estimator means and units numerically. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare analytic/direct-lighting cases vs Monte Carlo estimates with units and inverse-square/cosine laws checked explicitly. Use the same scenes, light/material parameters and rendering budget (samples or wall time); compare against an analytic or high-sample reference and keep estimators unbiased unless bias is the variable under study. Record unit consistency, analytic irradiance/radiance relative error, estimator bias/variance, and inverse-square/cosine-law residuals. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Change emitter area, distance, orientation, and scene scale while checking inverse-square/projected-area behavior and dimensional consistency. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: PBRT Ch. 4 and 13.1; FCG Ch. 14.6-14.8. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Radiometry and the rendering equation. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among analytic/direct-lighting cases vs Monte Carlo estimates with units and inverse-square/cosine laws checked explicitly. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using unit consistency, analytic irradiance/radiance relative error, estimator bias/variance, and inverse-square/cosine-law residuals. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if analytic/direct-lighting cases vs Monte Carlo estimates with units and inverse-square/cosine laws checked explicitly improve error/variance per sample or per time in unit consistency, analytic irradiance/radiance relative error, estimator bias/variance, and inverse-

square/cosine-law residuals across several transport regimes while retaining the intended estimator/physical correctness; explain the change from PDFs, path contributions or transport structure.

F. Concrete research project - Radiometry and the rendering equation

F1. Falsifiable question. Under the controlled failure regime "Change emitter area, distance, orientation, and scene scale while checking inverse-square/projected-area behavior and dimensional consistency.", which alternative in analytic/direct-lighting cases vs Monte Carlo estimates with units and inverse-square/cosine laws checked explicitly is preferable, and is the change explained by sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create analytic scenes (Lambertian patch, point/area light) where irradiance/radiance can be calculated; compare estimator means and units numerically. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. For stochastic estimators, use at least 10 independent seeds at 16, 64, 256, and 1024 spp (or equivalent time budgets) on three transport regimes; compare against an analytic or high-sample reference and report confidence intervals/variance tails. The central adversarial condition is: Change emitter area, distance, orientation, and scene scale while checking inverse-square/projected-area behavior and dimensional consistency.

F4. Required evidence. Measure unit consistency, analytic irradiance/radiance relative error, estimator bias/variance, and inverse-square/cosine-law residuals. Deliver instrumented renderer code, raw per-seed/per-technique statistics, contribution/PDF diagnostics where applicable, convergence/variance-per-time plots, and one scene where the nominally preferred method loses. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Radiometry and the rendering equation. If a stable failure/crossover is found among analytic/direct-lighting cases vs Monte Carlo estimates with units and inverse-square/cosine laws checked explicitly, implement one adaptive or corrected variant that directly targets sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Radiometry and the rendering equation. Using the measured unit consistency, analytic irradiance/radiance relative error, estimator bias/variance, and inverse-square/cosine-law residuals, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Radiometry and the rendering equation

- G1. Closed-book derivation/oracle. Derive radiance invariance along a vacuum ray and the cosine-projected area measure, then derive the rendering equation from local energy balance and identify emitted, direct, and indirect terms. Then solve this exact micro-case from scratch without opening your implementation: A unit-area Lambertian patch with $\rho=0.8$ faces a point light of intensity I at distance 2. Compute irradiance and outgoing radiance at normal incidence and at 60 degrees, keeping units explicit. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Derive units for each term; verify inverse-square law numerically; implement direct illumination estimator. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally mix irradiance and radiance and omit one cosine or inverse-square factor in the direct-lighting oracle. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create analytic scenes (Lambertian patch, point/area light) where irradiance/radiance can be calculated; compare estimator means and units numerically. Then apply this stress without retuning: Change emitter area, distance, orientation, and scene scale while checking inverse-square/projected-area behavior and dimensional consistency. Report unit consistency,

analytic irradiance/radiance relative error, estimator bias/variance, and inverse-square/cosine-law residuals and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method-selection defense. Using your own measured results, choose among analytic/direct-lighting cases vs Monte Carlo estimates with units and inverse-square/cosine laws checked explicitly for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured unit consistency, analytic irradiance/radiance relative error, estimator bias/variance, and inverse-square/cosine-law residuals; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Radiometry and the rendering equation. Using unit consistency, analytic irradiance/radiance relative error, estimator bias/variance, and inverse-square/cosine-law residuals, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 6, Unit 2. Scattering functions and microfacets

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. BRDF/BTDF/BSDF, Fresnel, microfacet distributions, masking-shadowing, rough dielectric/conductor.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 9; RTR Ch. 9.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Material modeling lecture from CS184.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Reciprocity
- energy conservation
- microfacet normal distribution and Smith masking
- Precision requirement: distinguish and justify Lambertian, ideal specular, dielectric, and GGX models with reciprocity, normalization, and energy tests. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the microfacet reflection model from a distribution of surface normals, including Jacobian terms between half-vector and outgoing direction; verify reciprocity and discuss energy conservation.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Sweep grazing angles, roughness extremes, IOR/metal parameters, and anisotropy; numerically test reciprocity/energy conservation where applicable. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement Lambertian, mirror, dielectric and GGX; test normalization/energy numerically. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For normal incidence and a chosen GGX roughness, compute Schlick Fresnel, one microfacet D value, and the Smith masking term. Check reciprocity by swapping incident/outgoing directions in a symmetric case. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Sweep grazing angles, roughness extremes, IOR/metal parameters, and anisotropy; numerically test reciprocity/energy conservation where applicable. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Numerically integrate each BSDF over outgoing directions for a grid of incident angles/roughness; flag energy >1 and verify sample/eval/pdf consistency.
- P5 - Machine-readable evidence. Export hemispherical reflectance, reciprocity residual, sample/eval/pdf consistency, reference BRDF error, and variance when sampled to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.

- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: Lambertian, ideal specular, dielectric, and GGX models with reciprocity, normalization, and energy tests. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Sweep grazing angles, roughness extremes, IOR/metal parameters, and anisotropy; numerically test reciprocity/energy conservation where applicable." without hand-editing code between methods. Record hemispherical reflectance, reciprocity residual, sample/eval/pdf consistency, reference BRDF error, and variance when sampled.
- P7 - Deliberate bug and regression drill. Intentionally sample a half-vector distribution but forget the half-vector-to-direction Jacobian in the PDF. Before running, predict the first observable failure in hemispherical reflectance, reciprocity residual, sample/eval/pdf consistency, reference BRDF error, and variance when sampled. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the microfacet reflection model from a distribution of surface normals, including Jacobian terms between half-vector and outgoing direction; verify reciprocity and discuss energy conservation. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For normal incidence and a chosen GGX roughness, compute Schlick Fresnel, one microfacet D value, and the Smith masking term. Check reciprocity by swapping incident/outgoing directions in a symmetric case. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement Lambertian, mirror, dielectric and GGX; test normalization/energy numerically. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Numerically integrate each BSDF over outgoing directions for a grid of incident angles/roughness; flag energy >1 and verify sample/eval/pdf consistency. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare Lambertian, ideal specular, dielectric, and GGX models with reciprocity, normalization, and energy tests. Use the same scenes, light/material parameters and rendering budget (samples or wall time); compare against an analytic or high-sample reference and keep estimators unbiased unless bias is the variable under study. Record hemispherical reflectance, reciprocity residual, sample/eval/pdf consistency, reference BRDF error, and variance when sampled. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Sweep grazing angles, roughness extremes, IOR/metal parameters, and anisotropy; numerically test reciprocity/energy conservation where applicable. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: PBRT Ch. 9; RTR Ch. 9. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Scattering functions and microfacets. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among Lambertian, ideal specular, dielectric, and GGX models with reciprocity, normalization, and energy tests. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using hemispherical reflectance, reciprocity residual, sample/eval/pdf consistency, reference BRDF error, and variance when sampled. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if Lambertian, ideal specular, dielectric, and GGX models with reciprocity, normalization, and energy tests improve error/variance per sample or per time in hemispherical reflectance, reciprocity residual, sample/eval/pdf consistency, reference BRDF error, and variance when sampled across several transport regimes while retaining the intended estimator/physical correctness; explain the change from PDFs, path contributions or transport structure.

F. Concrete research project - Scattering functions and microfacets

- F1. Falsifiable question. Under the controlled failure regime "Sweep grazing angles, roughness extremes, IOR/metal parameters, and anisotropy; numerically test reciprocity/energy conservation where applicable.", characterize the

ranking or regime transition across Lambertian, ideal specular, dielectric, and GGX models with reciprocity, normalization, and energy tests. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Numerically integrate each BSDF over outgoing directions for a grid of incident angles/roughness; flag energy >1 and verify sample/eval/pdf consistency. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. For stochastic estimators, use at least 10 independent seeds at 16, 64, 256, and 1024 spp (or equivalent time budgets) on three transport regimes; compare against an analytic or high-sample reference and report confidence intervals/variance tails. The central adversarial condition is: Sweep grazing angles, roughness extremes, IOR/metal parameters, and anisotropy; numerically test reciprocity/energy conservation where applicable.

F4. Required evidence. Measure hemispherical reflectance, reciprocity residual, sample/eval/pdf consistency, reference BRDF error, and variance when sampled. Deliver instrumented renderer code, raw per-seed/per-technique statistics, contribution/PDF diagnostics where applicable, convergence/variance-per-time plots, and one scene where the nominally preferred method loses. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Scattering functions and microfacets. If a stable failure/crossover is found among Lambertian, ideal specular, dielectric, and GGX models with reciprocity, normalization, and energy tests, implement one adaptive or corrected variant that directly targets sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Scattering functions and microfacets. Using the measured hemispherical reflectance, reciprocity residual, sample/eval/pdf consistency, reference BRDF error, and variance when sampled, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Scattering functions and microfacets

- G1. Closed-book derivation/oracle. Derive the microfacet reflection model from a distribution of surface normals, including Jacobian terms between half-vector and outgoing direction; verify reciprocity and discuss energy conservation. Then solve this exact micro-case from scratch without opening your implementation: For normal incidence and a chosen GGX roughness, compute Schlick Fresnel, one microfacet D value, and the Smith masking term. Check reciprocity by swapping incident/outgoing directions in a symmetric case. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement Lambertian, mirror, dielectric and GGX; test normalization/energy numerically. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally sample a half-vector distribution but forget the half-vector-to-direction Jacobian in the PDF. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Numerically integrate each BSDF over outgoing directions for a grid of incident angles/roughness; flag energy >1 and verify sample/eval/pdf consistency. Then apply this stress without retuning: Sweep grazing angles, roughness extremes, IOR/metal parameters, and anisotropy; numerically test reciprocity/energy conservation where applicable. Report hemispherical reflectance, reciprocity residual, sample/eval/pdf consistency, reference BRDF error, and variance when sampled and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across Lambertian, ideal specular, dielectric, and GGX models with reciprocity, normalization, and energy tests for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured hemispherical reflectance, reciprocity residual, sample/eval/pdf consistency,

reference BRDF error, and variance when sampled; state one counterexample that would change the diagnosis or ranking.

- G6. Pass condition for Scattering functions and microfacets. Using hemispherical reflectance, reciprocity residual, sample/eval/pdf consistency, reference BRDF error, and variance when sampled, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 6, Unit 3. Monte Carlo path tracing

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Path construction, throughput, Russian roulette, next-event estimation.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 2 and Ch. 13.2-13.4.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 path tracing lectures.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Unbiased path tracing estimator
- path throughput
- RR preserves expectation
- Precision requirement: distinguish and justify naïve recursive path tracing vs next-event estimation/Russian roulette variants on diffuse, glossy, and indirect-only scenes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the path-integral form of light transport and the recursive path-tracing estimator; derive path throughput and Russian-roulette weighting, and state unbiasedness conditions.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use a tiny bright light, glossy chains, indirect-dominated interiors, and high-dynamic-range environments; classify which path families dominate variance. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement unidirectional path tracer; isolate direct/indirect contributions; run variance studies. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a path camera->diffuse A->diffuse B->area light, assign numeric BSDFs, cosines, PDFs, and emitted radiance and compute the path contribution and throughput step by step. Then add Russian roulette at B. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use a tiny bright light, glossy chains, indirect-dominated interiors, and high-dynamic-range environments; classify which path families dominate variance. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Instrument path length, throughput, direct/indirect contribution, and firefly tails; produce convergence curves for diffuse box, glossy indirect, and small-light scenes.
- P5 - Machine-readable evidence. Export RMSE/variance versus spp/time, contribution-tail quantiles, average path length, firefly frequency, and bias checks to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: naïve recursive path tracing vs next-event estimation/Russian roulette variants on diffuse, glossy, and indirect-only scenes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use a tiny bright light, glossy chains, indirect-dominated interiors, and high-dynamic-range environments; classify which path families dominate variance." without hand-editing code between methods. Record RMSE/variance versus spp/time, contribution-tail quantiles, average path length, firefly frequency, and bias checks.

- P7 - Deliberate bug and regression drill. Intentionally apply Russian roulette without dividing surviving path throughput by the survival probability. Before running, predict the first observable failure in RMSE/variance versus spp/time, contribution-tail quantiles, average path length, firefly frequency, and bias checks. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the path-integral form of light transport and the recursive path-tracing estimator; derive path throughput and Russian-roulette weighting, and state unbiasedness conditions. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a path camera->diffuse A->diffuse B->area light, assign numeric BSDFs, cosines, PDFs, and emitted radiance and compute the path contribution and throughput step by step. Then add Russian roulette at B. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement unidirectional path tracer; isolate direct/indirect contributions; run variance studies. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Instrument path length, throughput, direct/indirect contribution, and firefly tails; produce convergence curves for diffuse box, glossy indirect, and small-light scenes. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare naïve recursive path tracing vs next-event estimation/Russian roulette variants on diffuse, glossy, and indirect-only scenes. Use the same scenes, light/material parameters and rendering budget (samples or wall time); compare against an analytic or high-sample reference and keep estimators unbiased unless bias is the variable under study. Record RMSE/variance versus spp/time, contribution-tail quantiles, average path length, firefly frequency, and bias checks. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use a tiny bright light, glossy chains, indirect-dominated interiors, and high-dynamic-range environments; classify which path families dominate variance. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: PBRT Ch. 2 and Ch. 13.2-13.4. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Monte Carlo path tracing. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among naïve recursive path tracing vs next-event estimation/Russian roulette variants on diffuse, glossy, and indirect-only scenes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using RMSE/variance versus spp/time, contribution-tail quantiles, average path length, firefly frequency, and bias checks. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if naïve recursive path tracing vs next-event estimation/Russian roulette variants on diffuse, glossy, and indirect-only scenes improve error/variance per sample or per time in RMSE/variance versus spp/time, contribution-tail quantiles, average path length, firefly frequency, and bias checks across several transport regimes while retaining the intended estimator/physical correctness; explain the change from PDFs, path contributions or transport structure.

F. Concrete research project - Monte Carlo path tracing

- F1. Falsifiable question. Under the controlled failure regime "Use a tiny bright light, glossy chains, indirect-dominated interiors, and high-dynamic-range environments; classify which path families dominate variance.", which alternative in naïve recursive path tracing vs next-event estimation/Russian roulette variants on diffuse, glossy, and indirect-only scenes is preferable, and is the change explained by sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Instrument path length, throughput, direct/indirect contribution, and firefly tails; produce convergence curves for diffuse box, glossy indirect, and small-

light scenes. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. For stochastic estimators, use at least 10 independent seeds at 16, 64, 256, and 1024 spp (or equivalent time budgets) on three transport regimes; compare against an analytic or high-sample reference and report confidence intervals/variance tails. The central adversarial condition is: Use a tiny bright light, glossy chains, indirect-dominated interiors, and high-dynamic-range environments; classify which path families dominate variance.

F4. Required evidence. Measure RMSE/variance versus spp/time, contribution-tail quantiles, average path length, firefly frequency, and bias checks. Deliver instrumented renderer code, raw per-seed/per-technique statistics, contribution/PDF diagnostics where applicable, convergence/variance-per-time plots, and one scene where the nominally preferred method loses. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Monte Carlo path tracing. If a stable failure/crossover is found among naïve recursive path tracing vs next-event estimation/Russian roulette variants on diffuse, glossy, and indirect-only scenes, implement one adaptive or corrected variant that directly targets sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Monte Carlo path tracing. Using the measured RMSE/variance versus spp/time, contribution-tail quantiles, average path length, firefly frequency, and bias checks, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Monte Carlo path tracing

- G1. Closed-book derivation/oracle. Derive the path-integral form of light transport and the recursive path-tracing estimator; derive path throughput and Russian-roulette weighting, and state unbiasedness conditions. Then solve this exact micro-case from scratch without opening your implementation: For a path camera->diffuse A->diffuse B->area light, assign numeric BSDFs, cosines, PDFs, and emitted radiance and compute the path contribution and throughput step by step. Then add Russian roulette at B. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement unidirectional path tracer; isolate direct/indirect contributions; run variance studies. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally apply Russian roulette without dividing surviving path throughput by the survival probability. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Instrument path length, throughput, direct/indirect contribution, and firefly tails; produce convergence curves for diffuse box, glossy indirect, and small-light scenes. Then apply this stress without retuning: Use a tiny bright light, glossy chains, indirect-dominated interiors, and high-dynamic-range environments; classify which path families dominate variance. Report RMSE/variance versus spp/time, contribution-tail quantiles, average path length, firefly frequency, and bias checks and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among naïve recursive path tracing vs next-event estimation/Russian roulette variants on diffuse, glossy, and indirect-only scenes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured RMSE/variance versus spp/time, contribution-tail quantiles, average path length, firefly frequency, and bias checks; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Monte Carlo path tracing. Using RMSE/variance versus spp/time, contribution-tail quantiles, average path length, firefly frequency, and bias checks, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 6, Unit 4. Importance sampling and MIS

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Sampling lights/BSDFs, balance/power heuristics, environment map sampling.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 2, 8, 12-13; Appendix A.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: PBRT sampling examples.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- MIS estimator
- balance/power heuristics
- change-of-variables/Jacobians
- Precision requirement: distinguish and justify BSDF-only, light-only, balance-heuristic MIS, and power-heuristic MIS on small-light, glossy, and HDR-environment cases. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the balance and power heuristics for multiple importance sampling; work through a two-proposal integral analytically and show why the denominator must include all proposal densities.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Pair good and deliberately bad proposals on scenes where each technique dominates a different region; plot estimator tails as well as average error. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Compare BSDF-only, light-only and MIS on small lights, glossy surfaces, HDR environments. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Suppose light sampling gives $p_L=0.8$ and BSDF sampling $p_B=0.2$ for the same direction. Compute balance and power-heuristic MIS weights. Repeat with $p_L=0.01, p_B=0.5$ and interpret the change. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Pair good and deliberately bad proposals on scenes where each technique dominates a different region; plot estimator tails as well as average error. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Log per-technique contributions and MIS weights; estimate variance by path family and test whether an online allocation rule predicts the better sampler.
- P5 - Machine-readable evidence. Export variance/RMSE versus time, per-technique contribution variance, MIS weight distribution, firefly rate, and effective samples to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: BSDF-only, light-only, balance-heuristic MIS, and power-heuristic MIS on small-light, glossy, and HDR-environment cases. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Pair good and deliberately bad proposals on scenes where each technique dominates a different region; plot estimator tails as well as average error." without hand-editing code between methods. Record variance/RMSE versus time, per-technique contribution variance, MIS weight distribution, firefly rate, and effective samples.
- P7 - Deliberate bug and regression drill. Intentionally combine light- and BSDF-sampling PDFs expressed in different measures inside the MIS heuristic. Before running, predict the first observable failure in variance/RMSE versus time, per-technique contribution variance, MIS weight distribution, firefly rate, and effective samples. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the balance and power heuristics for multiple importance sampling; work through a two-proposal integral analytically and show why the denominator must include all proposal densities. Then evaluate

the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. Suppose light sampling gives $p_L=0.8$ and BSDF sampling $p_B=0.2$ for the same direction. Compute balance and power-heuristic MIS weights. Repeat with $p_L=0.01, p_B=0.5$ and interpret the change. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Compare BSDF-only, light-only and MIS on small lights, glossy surfaces, HDR environments. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Log per-technique contributions and MIS weights; estimate variance by path family and test whether an online allocation rule predicts the better sampler. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare BSDF-only, light-only, balance-heuristic MIS, and power-heuristic MIS on small-light, glossy, and HDR-environment cases. Use the same scenes, light/material parameters and rendering budget (samples or wall time); compare against an analytic or high-sample reference and keep estimators unbiased unless bias is the variable under study. Record variance/RMSE versus time, per-technique contribution variance, MIS weight distribution, firefly rate, and effective samples. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Pair good and deliberately bad proposals on scenes where each technique dominates a different region; plot estimator tails as well as average error. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: PBRT Ch. 2, 8, 12-13; Appendix A. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Importance sampling and MIS. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among BSDF-only, light-only, balance-heuristic MIS, and power-heuristic MIS on small-light, glossy, and HDR-environment cases. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using variance/RMSE versus time, per-technique contribution variance, MIS weight distribution, firefly rate, and effective samples. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the variance/error-per-time ranking changes reproducibly across distinct light/path regimes and the change is predicted by logged PDFs and contribution statistics without introducing estimator bias.

F. Concrete research project - Importance sampling and MIS

F1. Falsifiable question. Under the controlled failure regime "Pair good and deliberately bad proposals on scenes where each technique dominates a different region; plot estimator tails as well as average error.", characterize the ranking or regime transition across BSDF-only, light-only, balance-heuristic MIS, and power-heuristic MIS on small-light, glossy, and HDR-environment cases. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by proposal support overlap, PDF measure consistency, MIS weighting, contribution tails, and variance per unit time. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Log per-technique contributions and MIS weights; estimate variance by path family and test whether an online allocation rule predicts the better sampler. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. For stochastic estimators, use at least 10 independent seeds at 16, 64, 256, and 1024 spp (or equivalent time budgets) on three transport regimes; compare against an analytic or high-sample reference and report confidence intervals/variance tails. The central adversarial condition is: Pair good and deliberately bad proposals on scenes where each technique dominates a different region; plot estimator tails as well as average error.

F4. Required evidence. Measure variance/RMSE versus time, per-technique contribution variance, MIS weight distribution, firefly rate, and effective samples. Deliver instrumented renderer code, raw per-seed/per-technique statistics, contribution/PDF diagnostics where applicable, convergence/variance-per-time plots, and one scene where the nominally preferred method loses. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Importance sampling and MIS. If a stable failure/crossover is found among BSDF-only, light-only, balance-heuristic MIS, and power-heuristic MIS on small-light, glossy, and HDR-environment cases, implement one adaptive or corrected variant that directly targets proposal support overlap, PDF measure consistency, MIS weighting, contribution tails, and variance per unit time; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Importance sampling and MIS. Using the measured variance/RMSE versus time, per-technique contribution variance, MIS weight distribution, firefly rate, and effective samples, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Importance sampling and MIS

- G1. Closed-book derivation/oracle. Derive the balance and power heuristics for multiple importance sampling; work through a two-proposal integral analytically and show why the denominator must include all proposal densities. Then solve this exact micro-case from scratch without opening your implementation: Suppose light sampling gives $p_L=0.8$ and BSDF sampling $p_B=0.2$ for the same direction. Compute balance and power-heuristic MIS weights. Repeat with $p_L=0.01, p_B=0.5$ and interpret the change. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Compare BSDF-only, light-only and MIS on small lights, glossy surfaces, HDR environments. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally combine light- and BSDF-sampling PDFs expressed in different measures inside the MIS heuristic. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Log per-technique contributions and MIS weights; estimate variance by path family and test whether an online allocation rule predicts the better sampler. Then apply this stress without retuning: Pair good and deliberately bad proposals on scenes where each technique dominates a different region; plot estimator tails as well as average error. Report variance/RMSE versus time, per-technique contribution variance, MIS weight distribution, firefly rate, and effective samples and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across BSDF-only, light-only, balance-heuristic MIS, and power-heuristic MIS on small-light, glossy, and HDR-environment cases for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured variance/RMSE versus time, per-technique contribution variance, MIS weight distribution, firefly rate, and effective samples; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Importance sampling and MIS. Using variance/RMSE versus time, per-technique contribution variance, MIS weight distribution, firefly rate, and effective samples, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 6, Unit 5. Participating media

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Absorption, emission, phase functions, transmittance, free-flight sampling, volume path tracing.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 11 and 14.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: PBRT volume examples.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Radiative transfer equation
- Beer-Lambert law
- Henyey-Greenstein phase function
- Precision requirement: distinguish and justify analytic homogeneous transmittance vs sampled free-flight; isotropic vs Henyey-Greenstein phase functions; delta/ratio tracking for heterogeneous media. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive Beer-Lambert transmittance, the volume rendering equation, and free-flight sampling in homogeneous media; extend the derivation conceptually to heterogeneous extinction.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use optically thin/dense, strongly anisotropic, and heterogeneous media with sharp density changes; compare tracking/sampling behavior. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement homogeneous medium with isotropic/HG scattering; then heterogeneous medium via ratio/delta tracking if ambitious. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For homogeneous $\sigma_a=0.2$, $\sigma_s=0.3$ over distance 4, compute transmittance, extinction-event probability, and the exponential free-flight CDF at $t=1,2,4$. Compare with a sampled-event histogram later. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use optically thin/dense, strongly anisotropic, and heterogeneous media with sharp density changes; compare tracking/sampling behavior. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Validate homogeneous transmittance against $\exp(-\sigma_t d)$, then compare delta/ratio tracking on a heterogeneous density field with known/high-sample reference.
- P5 - Machine-readable evidence. Export transmittance relative error, scattering-position distribution error, RMSE/variance, tracking null-collision rate, and time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: analytic homogeneous transmittance vs sampled free-flight; isotropic vs Henyey-Greenstein phase functions; delta/ratio tracking for heterogeneous media. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use optically thin/dense, strongly anisotropic, and heterogeneous media with sharp density changes; compare tracking/sampling behavior." without hand-editing code between methods. Record transmittance relative error, scattering-position distribution error, RMSE/variance, tracking null-collision rate, and time.
- P7 - Deliberate bug and regression drill. Intentionally use $\exp(+\sigma_t d)$ rather than $\exp(-\sigma_t d)$ in one transmittance path or omit null-collision weights. Before running, predict the first observable failure in transmittance relative error, scattering-position distribution error, RMSE/variance, tracking null-collision rate, and time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive Beer-Lambert transmittance, the volume rendering equation, and free-flight sampling in homogeneous media; extend the derivation conceptually to heterogeneous extinction. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For homogeneous $\sigma_a=0.2$, $\sigma_s=0.3$ over distance 4, compute transmittance, extinction-event probability, and the exponential free-flight CDF at $t=1,2,4$. Compare with a sampled-event histogram later. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement homogeneous medium with isotropic/HG scattering; then heterogeneous medium via ratio/delta tracking if ambitious. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Validate homogeneous transmittance against $\exp(-\sigma_t d)$, then compare

delta/ratio tracking on a heterogeneous density field with known/high-sample reference. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare analytic homogeneous transmittance vs sampled free-flight; isotropic vs Henyey-Greenstein phase functions; delta/ratio tracking for heterogeneous media. Use the same scenes, light/material parameters and rendering budget (samples or wall time); compare against an analytic or high-sample reference and keep estimators unbiased unless bias is the variable under study. Record transmittance relative error, scattering-position distribution error, RMSE/variance, tracking null-collision rate, and time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use optically thin/dense, strongly anisotropic, and heterogeneous media with sharp density changes; compare tracking/sampling behavior. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: PBRT Ch. 11 and 14. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Participating media. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among analytic homogeneous transmittance vs sampled free-flight; isotropic vs Henyey-Greenstein phase functions; delta/ratio tracking for heterogeneous media. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using transmittance relative error, scattering-position distribution error, RMSE/variance, tracking null-collision rate, and time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if analytic homogeneous transmittance vs sampled free-flight; isotropic vs Henyey-Greenstein phase functions; delta/ratio tracking for heterogeneous media improve error/variance per sample or per time in transmittance relative error, scattering-position distribution error, RMSE/variance, tracking null-collision rate, and time across several transport regimes while retaining the intended estimator/physical correctness; explain the change from PDFs, path contributions or transport structure.

F. Concrete research project - Participating media

F1. Falsifiable question. Under the controlled failure regime "Use optically thin/dense, strongly anisotropic, and heterogeneous media with sharp density changes; compare tracking/sampling behavior.", which alternative in analytic homogeneous transmittance vs sampled free-flight; isotropic vs Henyey-Greenstein phase functions; delta/ratio tracking for heterogeneous media is preferable, and is the change explained by sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Validate homogeneous transmittance against $\exp(-\sigma_t d)$, then compare delta/ratio tracking on a heterogeneous density field with known/high-sample reference. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. For stochastic estimators, use at least 10 independent seeds at 16, 64, 256, and 1024 spp (or equivalent time budgets) on three transport regimes; compare against an analytic or high-sample reference and report confidence intervals/variance tails. The central adversarial condition is: Use optically thin/dense, strongly anisotropic, and heterogeneous media with sharp density changes; compare tracking/sampling behavior.

F4. Required evidence. Measure transmittance relative error, scattering-position distribution error, RMSE/variance, tracking null-collision rate, and time. Deliver instrumented renderer code, raw per-seed/per-technique statistics, contribution/PDF diagnostics where applicable, convergence/variance-per-time plots, and one scene where the nominally preferred method loses. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Participating media. If a stable failure/crossover is found among analytic homogeneous transmittance vs sampled free-flight; isotropic vs Henyey-Greenstein phase functions; delta/ratio tracking for heterogeneous media, implement one adaptive or corrected variant that directly targets sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Participating media. Using the measured transmittance relative error, scattering-position distribution error, RMSE/variance, tracking null-collision rate, and time, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Participating media

- G1. Closed-book derivation/oracle. Derive Beer-Lambert transmittance, the volume rendering equation, and free-flight sampling in homogeneous media; extend the derivation conceptually to heterogeneous extinction. Then solve this exact micro-case from scratch without opening your implementation: For homogeneous $\sigma_a=0.2$, $\sigma_s=0.3$ over distance 4, compute transmittance, extinction-event probability, and the exponential free-flight CDF at $t=1,2,4$. Compare with a sampled-event histogram later. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement homogeneous medium with isotropic/HG scattering; then heterogeneous medium via ratio/delta tracking if ambitious. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use $\exp(+\sigma_t d)$ rather than $\exp(-\sigma_t d)$ in one transmittance path or omit null-collision weights. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Validate homogeneous transmittance against $\exp(-\sigma_t d)$, then compare delta/ratio tracking on a heterogeneous density field with known/high-sample reference. Then apply this stress without retuning: Use optically thin/dense, strongly anisotropic, and heterogeneous media with sharp density changes; compare tracking/sampling behavior. Report transmittance relative error, scattering-position distribution error, RMSE/variance, tracking null-collision rate, and time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among analytic homogeneous transmittance vs sampled free-flight; isotropic vs Henyey-Greenstein phase functions; delta/ratio tracking for heterogeneous media for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured transmittance relative error, scattering-position distribution error, RMSE/variance, tracking null-collision rate, and time; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Participating media. Using transmittance relative error, scattering-position distribution error, RMSE/variance, tracking null-collision rate, and time, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 6, Unit 6. Advanced light transport

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Bidirectional path tracing, Metropolis light transport, photon mapping, path-space view, caustics.

A. Reading order

1. Read the primary assignment first: Veach, Robust Monte Carlo Methods for Light Transport Simulation, the bidirectional/path-space chapters; for implementation exposition, use PBRT 3e, Light Transport III: Bidirectional Methods (the 4e book no longer has a BDPT chapter). Add Jensen-style photon-mapping references if pursuing photon methods.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: SIGGRAPH course notes where available.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Path-space formulation
- bidirectional connection strategies

- difficult transport modes
- Precision requirement: distinguish and justify unidirectional path tracing vs simplified BDPT or photon mapping on caustic and difficult-indirect-light scenes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the path-space factors for a bidirectional connection or a simple caustic path and explain why path tracing alone has high variance; compare connection, merging, and Metropolis-style strategies conceptually.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use classic difficult transport patterns such as caustics and specular-diffuse-specular chains; compare contribution histograms across estimators. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement simplified BDPT or progressive photon mapping; compare caustic scenes against path tracing. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Draw all path-connection strategies for a camera path of length 2 and light path of length 2 in a simple diffuse scene. Identify which strategies can construct a specular-diffuse-specular transport path and which cannot. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use classic difficult transport patterns such as caustics and specular-diffuse-specular chains; compare contribution histograms across estimators. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Reproduce a simple caustic (glass sphere over plane) and compare error versus time for path tracing and your BDPT/photon-mapping implementation.
- P5 - Machine-readable evidence. Export RMSE versus wall time in caustic/difficult scenes, path-family coverage, memory, and bias if the chosen method is biased to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: unidirectional path tracing vs simplified BDPT or photon mapping on caustic and difficult-indirect-light scenes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use classic difficult transport patterns such as caustics and specular-diffuse-specular chains; compare contribution histograms across estimators." without hand-editing code between methods. Record RMSE versus wall time in caustic/difficult scenes, path-family coverage, memory, and bias if the chosen method is biased.
- P7 - Deliberate bug and regression drill. Intentionally connect subpaths but omit or mis-index one competing-technique PDF in the MIS weight. Before running, predict the first observable failure in RMSE versus wall time in caustic/difficult scenes, path-family coverage, memory, and bias if the chosen method is biased. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the path-space factors for a bidirectional connection or a simple caustic path and explain why path tracing alone has high variance; compare connection, merging, and Metropolis-style strategies conceptually. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Draw all path-connection strategies for a camera path of length 2 and light path of length 2 in a simple diffuse scene. Identify which strategies can construct a specular-diffuse-specular transport path and which cannot. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement simplified BDPT or progressive photon mapping; compare caustic scenes against path tracing. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Reproduce a simple caustic (glass sphere over plane) and compare error versus time for path tracing and your BDPT/photon-mapping implementation. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare unidirectional path tracing vs simplified BDPT or photon mapping on caustic and difficult-indirect-light scenes. Use the same scenes, light/material parameters and rendering budget (samples or wall time); compare against an analytic or high-sample reference and keep estimators unbiased unless bias is the

variable under study. Record RMSE versus wall time in caustic/difficult scenes, path-family coverage, memory, and bias if the chosen method is biased. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use classic difficult transport patterns such as caustics and specular-diffuse-specular chains; compare contribution histograms across estimators. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Veach, Robust Monte Carlo Methods for Light Transport Simulation, the bidirectional/path-space chapters; for implementation exposition, use PBRT 3e, Light Transport III: Bidirectional Methods (the 4e book no longer has a BDPT chapter). Add Jensen-style photon-mapping references if pursuing photon methods. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Advanced light transport. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among unidirectional path tracing vs simplified BDPT or photon mapping on caustic and difficult-indirect-light scenes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using RMSE versus wall time in caustic/difficult scenes, path-family coverage, memory, and bias if the chosen method is biased. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if unidirectional path tracing vs simplified BDPT or photon mapping on caustic and difficult-indirect-light scenes improve error/variance per sample or per time in RMSE versus wall time in caustic/difficult scenes, path-family coverage, memory, and bias if the chosen method is biased across several transport regimes while retaining the intended estimator/physical correctness; explain the change from PDFs, path contributions or transport structure.

F. Concrete research project - Advanced light transport

F1. Falsifiable question. Under the controlled failure regime "Use classic difficult transport patterns such as caustics and specular-diffuse-specular chains; compare contribution histograms across estimators.", which alternative in unidirectional path tracing vs simplified BDPT or photon mapping on caustic and difficult-indirect-light scenes is preferable, and is the change explained by sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Reproduce a simple caustic (glass sphere over plane) and compare error versus time for path tracing and your BDPT/photon-mapping implementation. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. For stochastic estimators, use at least 10 independent seeds at 16, 64, 256, and 1024 spp (or equivalent time budgets) on three transport regimes; compare against an analytic or high-sample reference and report confidence intervals/variance tails. The central adversarial condition is: Use classic difficult transport patterns such as caustics and specular-diffuse-specular chains; compare contribution histograms across estimators.

F4. Required evidence. Measure RMSE versus wall time in caustic/difficult scenes, path-family coverage, memory, and bias if the chosen method is biased. Deliver instrumented renderer code, raw per-seed/per-technique statistics, contribution/PDF diagnostics where applicable, convergence/variance-per-time plots, and one scene where the nominally preferred method loses. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Advanced light transport. If a stable failure/crossover is found among unidirectional path tracing vs simplified BDPT or photon mapping on caustic and difficult-indirect-light scenes, implement one adaptive or corrected variant that directly targets sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Advanced light transport. Using the measured RMSE versus wall time in caustic/difficult scenes, path-family coverage, memory, and bias if the chosen method is biased, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Continue toward a paper only if the effect survives the

required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Advanced light transport

- G1. Closed-book derivation/oracle. Derive the path-space factors for a bidirectional connection or a simple caustic path and explain why path tracing alone has high variance; compare connection, merging, and Metropolis-style strategies conceptually. Then solve this exact micro-case from scratch without opening your implementation: Draw all path-connection strategies for a camera path of length 2 and light path of length 2 in a simple diffuse scene. Identify which strategies can construct a specular-diffuse-specular transport path and which cannot. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement simplified BDPT or progressive photon mapping; compare caustic scenes against path tracing. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally connect subpaths but omit or mis-index one competing-technique PDF in the MIS weight. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Reproduce a simple caustic (glass sphere over plane) and compare error versus time for path tracing and your BDPT/photon-mapping implementation. Then apply this stress without retuning: Use classic difficult transport patterns such as caustics and specular-diffuse-specular chains; compare contribution histograms across estimators. Report RMSE versus wall time in caustic/difficult scenes, path-family coverage, memory, and bias if the chosen method is biased and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among unidirectional path tracing vs simplified BDPT or photon mapping on caustic and difficult-indirect-light scenes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured RMSE versus wall time in caustic/difficult scenes, path-family coverage, memory, and bias if the chosen method is biased; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Advanced light transport. Using RMSE versus wall time in caustic/difficult scenes, path-family coverage, memory, and bias if the chosen method is biased, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 6, Unit 7. Spectral and polarization rendering

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Spectral sampling, dispersion, fluorescent/polarization concepts.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 4.5-4.6; Mitsuba polarization tutorial later.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Mitsuba polarizer optimization tutorial (later for differentiable case).
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Wavelength-dependent IOR
- tristimulus projection
- Stokes/Mueller intuition optional
- Precision requirement: distinguish and justify RGB rendering vs wavelength sampling on spectra with narrow peaks/dispersion; scalar intensity vs polarization-aware transport where available. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive tristimulus integration from spectral radiance, then derive a wavelength-sampling estimator; for polarization, work through a simple Stokes/Mueller reflection example and identify when scalar RGB is insufficient.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use narrow-band illumination/material response, dispersion, and polarized reflection cases; sweep wavelength/polarization sample count. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement hero-wavelength spectral rendering or a spectral thin-prism experiment. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Trace two wavelengths (450 and 650 nm) through a toy dispersive index $n(\lambda)$ and compute refraction angles. Then show why a three-channel RGB path cannot reproduce wavelength-dependent spatial separation. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use narrow-band illumination/material response, dispersion, and polarized reflection cases; sweep wavelength/polarization sample count. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Render a dispersive prism or thin spectral test with sampled wavelengths; compare RGB approximation and spectral result under two illuminants.
- P5 - Machine-readable evidence. Export spectral/radiometric error, color Delta E under multiple illuminants, dispersion feature error, and wavelength/sample cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: RGB rendering vs wavelength sampling on spectra with narrow peaks/dispersion; scalar intensity vs polarization-aware transport where available. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use narrow-band illumination/material response, dispersion, and polarized reflection cases; sweep wavelength/polarization sample count." without hand-editing code between methods. Record spectral/radiometric error, color Delta E under multiple illuminants, dispersion feature error, and wavelength/sample cost.
- P7 - Deliberate bug and regression drill. Intentionally sample wavelengths nonuniformly but omit division by wavelength-sampling probability. Before running, predict the first observable failure in spectral/radiometric error, color Delta E under multiple illuminants, dispersion feature error, and wavelength/sample cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive tristimulus integration from spectral radiance, then derive a wavelength-sampling estimator; for polarization, work through a simple Stokes/Mueller reflection example and identify when scalar RGB is insufficient. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Trace two wavelengths (450 and 650 nm) through a toy dispersive index $n(\lambda)$ and compute refraction angles. Then show why a three-channel RGB path cannot reproduce wavelength-dependent spatial separation. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement hero-wavelength spectral rendering or a spectral thin-prism experiment. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Render a dispersive prism or thin spectral test with sampled wavelengths; compare RGB approximation and spectral result under two illuminants. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare RGB rendering vs wavelength sampling on spectra with narrow peaks/dispersion; scalar intensity vs polarization-aware transport where available. Use the same scenes, light/material parameters and rendering budget (samples or wall time); compare against an analytic or high-sample reference and keep estimators unbiased unless bias is the variable under study. Record spectral/radiometric error, color Delta E under multiple illuminants, dispersion feature error, and wavelength/sample cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use narrow-band illumination/material response, dispersion, and polarized reflection cases; sweep wavelength/polarization sample count. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: PBRT Ch. 4.5-4.6; Mitsuba polarization tutorial later. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Spectral and polarization rendering. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among RGB rendering vs wavelength sampling on spectra with narrow peaks/dispersion; scalar intensity vs polarization-aware transport where available. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using spectral/radiometric error, color Delta E under multiple illuminants, dispersion feature error, and wavelength/sample cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if RGB rendering vs wavelength sampling on spectra with narrow peaks/dispersion; scalar intensity vs polarization-aware transport where available improve error/variance per sample or per time in spectral/radiometric error, color Delta E under multiple illuminants, dispersion feature error, and wavelength/sample cost across several transport regimes while retaining the intended estimator/physical correctness; explain the change from PDFs, path contributions or transport structure.

F. Concrete research project - Spectral and polarization rendering

- F1. Falsifiable question. Under the controlled failure regime "Use narrow-band illumination/material response, dispersion, and polarized reflection cases; sweep wavelength/polarization sample count.", which alternative in RGB rendering vs wavelength sampling on spectra with narrow peaks/dispersion; scalar intensity vs polarization-aware transport where available is preferable, and is the change explained by sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Render a dispersive prism or thin spectral test with sampled wavelengths; compare RGB approximation and spectral result under two illuminants. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. For stochastic estimators, use at least 10 independent seeds at 16, 64, 256, and 1024 spp (or equivalent time budgets) on three transport regimes; compare against an analytic or high-sample reference and report confidence intervals/variance tails. The central adversarial condition is: Use narrow-band illumination/material response, dispersion, and polarized reflection cases; sweep wavelength/polarization sample count.
- F4. Required evidence. Measure spectral/radiometric error, color Delta E under multiple illuminants, dispersion feature error, and wavelength/sample cost. Deliver instrumented renderer code, raw per-seed/per-technique statistics, contribution/PDF diagnostics where applicable, convergence/variance-per-time plots, and one scene where the nominally preferred method loses. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Spectral and polarization rendering. If a stable failure/crossover is found among RGB rendering vs wavelength sampling on spectra with narrow peaks/dispersion; scalar intensity vs polarization-aware transport where available, implement one adaptive or corrected variant that directly targets sampling measure/PDF correctness, path-family coverage, variance tails, and transport invariants; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Spectral and polarization rendering. Using the measured spectral/radiometric error, color Delta E under multiple illuminants, dispersion feature error, and wavelength/sample cost, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Spectral and polarization rendering

- G1. Closed-book derivation/oracle. Derive tristimulus integration from spectral radiance, then derive a wavelength-sampling estimator; for polarization, work through a simple Stokes/Mueller reflection example and identify when scalar RGB is insufficient. Then solve this exact micro-case from scratch without opening your implementation: Trace two wavelengths (450 and 650 nm) through a toy dispersive index $n(\lambda)$ and compute refraction angles. Then show why a three-channel RGB path cannot reproduce wavelength-dependent spatial separation. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement hero-wavelength spectral rendering or a spectral thin-prism experiment. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally sample wavelengths nonuniformly but omit division by wavelength-sampling probability. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Render a dispersive prism or thin spectral test with sampled wavelengths; compare RGB approximation and spectral result under two illuminants. Then apply this stress without retuning: Use narrow-band illumination/material response, dispersion, and polarized reflection cases; sweep wavelength/polarization sample count. Report spectral/radiometric error, color Delta E under multiple illuminants, dispersion feature error, and wavelength/sample cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among RGB rendering vs wavelength sampling on spectra with narrow peaks/dispersion; scalar intensity vs polarization-aware transport where available for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured spectral/radiometric error, color Delta E under multiple illuminants, dispersion feature error, and wavelength/sample cost; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Spectral and polarization rendering. Using spectral/radiometric error, color Delta E under multiple illuminants, dispersion feature error, and wavelength/sample cost, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 6, Unit 8. GPU wavefront rendering

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Queue-based GPU path tracing, compaction, divergence reduction.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 15.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: PBRT GPU implementation walkthrough.
3. Keep the most useful executable/reference page open while coding: PBRT 4e online book and code exposition - <https://pbr-book.org/4ed/contents>

B. Definitions, results and algorithms to know

- Wavefront decomposition
- persistent work queues
- occupancy tradeoffs
- Precision requirement: distinguish and justify megakernel path tracing vs staged wavefront queues, comparing divergence, queue occupancy, memory traffic, and throughput. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a queue/state-machine decomposition of a path tracer into ray generation, intersection, material evaluation, shadow, and continuation stages; model queue compaction and state traffic.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Start with coherent primary rays then highly divergent secondary paths; record queue sizes, compaction overhead, occupancy, and memory pressure by depth. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Transform a megakernel-style tracer into staged queues conceptually or in code; profile divergence and memory traffic. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For eight rays whose next events are [surface,miss,surface,medium,surface,miss,medium,surface], manually compact them into queues, then perform one more bounce and count inactive lanes for a megakernel versus queues. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Start with coherent primary rays then highly divergent secondary paths; record queue sizes, compaction overhead, occupancy, and memory pressure by depth. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Record queue lengths by bounce and material, branch efficiency, cache/memory metrics, and rays/s; identify the scene that reverses megakernel/wavefront ranking.
- P5 - Machine-readable evidence. Export rays/s, queue length/occupancy, branch efficiency, memory traffic, kernel time by stage, and scene-dependent crossover to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: megakernel path tracing vs staged wavefront queues, comparing divergence, queue occupancy, memory traffic, and throughput. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Start with coherent primary rays then highly divergent secondary paths; record queue sizes, compaction overhead, occupancy, and memory pressure by depth." without hand-editing code between methods. Record rays/s, queue length/occupancy, branch efficiency, memory traffic, kernel time by stage, and scene-dependent crossover.
- P7 - Deliberate bug and regression drill. Intentionally compact path queues with an off-by-one scan so a small fraction of rays are dropped or duplicated. Before running, predict the first observable failure in rays/s, queue length/occupancy, branch efficiency, memory traffic, kernel time by stage, and scene-dependent crossover. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a queue/state-machine decomposition of a path tracer into ray generation, intersection, material evaluation, shadow, and continuation stages; model queue compaction and state traffic. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For eight rays whose next events are [surface,miss,surface,medium,surface,miss,medium,surface], manually compact them into queues, then perform one more bounce and count inactive lanes for a megakernel versus queues. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Transform a megakernel-style tracer into staged queues conceptually or in code; profile divergence and memory traffic. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Record queue lengths by bounce and material, branch efficiency, cache/memory metrics, and rays/s; identify the scene that reverses megakernel/wavefront ranking. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare megakernel path tracing vs staged wavefront queues, comparing divergence, queue occupancy, memory traffic, and throughput. Use the same scenes, light/material parameters and rendering budget (samples or wall time); compare against an analytic or high-sample reference and keep estimators unbiased unless bias is the variable under study. Record rays/s, queue length/occupancy, branch efficiency, memory traffic, kernel time by stage, and scene-dependent crossover. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Start with coherent primary rays then highly divergent secondary paths; record queue sizes, compaction overhead, occupancy, and memory pressure by depth. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: PBRT Ch. 15. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on GPU wavefront rendering. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among megakernel path tracing vs staged wavefront queues, comparing divergence, queue occupancy, memory traffic, and throughput. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using rays/s, queue length/occupancy, branch efficiency, memory traffic, kernel time by stage, and scene-dependent crossover. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if megakernel path tracing vs staged wavefront queues, comparing divergence, queue occupancy, memory traffic, and throughput improve error/variance per sample or per time in rays/s, queue length/occupancy, branch efficiency, memory traffic, kernel time by stage, and scene-dependent crossover across several transport regimes while retaining the intended estimator/physical correctness; explain the change from PDFs, path contributions or transport structure.

F. Concrete research project - GPU wavefront rendering

F1. Falsifiable question. Under the controlled failure regime "Start with coherent primary rays then highly divergent secondary paths; record queue sizes, compaction overhead, occupancy, and memory pressure by depth.", which alternative in megakernel path tracing vs staged wavefront queues, comparing divergence, queue occupancy, memory traffic, and throughput is preferable, and is the change explained by memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Record queue lengths by bounce and material, branch efficiency, cache/memory metrics, and rays/s; identify the scene that reverses megakernel/wavefront ranking. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. For stochastic estimators, use at least 10 independent seeds at 16, 64, 256, and 1024 spp (or equivalent time budgets) on three transport regimes; compare against an analytic or high-sample reference and report confidence intervals/variance tails. The central adversarial condition is: Start with coherent primary rays then highly divergent secondary paths; record queue sizes, compaction overhead, occupancy, and memory pressure by depth.

F4. Required evidence. Measure rays/s, queue length/occupancy, branch efficiency, memory traffic, kernel time by stage, and scene-dependent crossover. Deliver instrumented renderer code, raw per-seed/per-technique statistics, contribution/PDF diagnostics where applicable, convergence/variance-per-time plots, and one scene where the nominally preferred method loses. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for GPU wavefront rendering. If a stable failure/crossover is found among megakernel path tracing vs staged wavefront queues, comparing divergence, queue occupancy, memory traffic, and throughput, implement one adaptive or corrected variant that directly targets memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for GPU wavefront rendering. Using the measured rays/s, queue length/occupancy, branch efficiency, memory traffic, kernel time by stage, and scene-dependent crossover, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for GPU wavefront rendering

- G1. Closed-book derivation/oracle. Derive a queue/state-machine decomposition of a path tracer into ray generation, intersection, material evaluation, shadow, and continuation stages; model queue compaction and state traffic. Then solve this exact micro-case from scratch without opening your implementation: For eight rays whose next events are [surface,miss,surface,medium,surface,miss,medium,surface], manually compact them into queues, then perform one more bounce and count inactive lanes for a megakernel versus queues. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Transform a megakernel-style tracer into staged queues conceptually or in code;

profile divergence and memory traffic. Add the oracle as an automated test and make it pass before importing any optimized/framework code.

- G3. Debugging exam. Intentionally compact path queues with an off-by-one scan so a small fraction of rays are dropped or duplicated. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Record queue lengths by bounce and material, branch efficiency, cache/memory metrics, and rays/s; identify the scene that reverses megakernel/wavefront ranking. Then apply this stress without retuning: Start with coherent primary rays then highly divergent secondary paths; record queue sizes, compaction overhead, occupancy, and memory pressure by depth. Report rays/s, queue length/occupancy, branch efficiency, memory traffic, kernel time by stage, and scene-dependent crossover and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among megakernel path tracing vs staged wavefront queues, comparing divergence, queue occupancy, memory traffic, and throughput for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured rays/s, queue length/occupancy, branch efficiency, memory traffic, kernel time by stage, and scene-dependent crossover; state one counterexample that would reverse your recommendation.
- G6. Pass condition for GPU wavefront rendering. Using rays/s, queue length/occupancy, branch efficiency, memory traffic, kernel time by stage, and scene-dependent crossover, Pass when analytic/high-sample truth is statistically consistent with estimator means, sample/eval/PDF and energy/unit checks relevant to the unit pass, and any claimed variance-per-time ordering survives independent seeds. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 6 Closeout: Project, Review and Mastery Gate

Volume project. Upgrade the ray tracer into a physically based path tracer with MIS, microfacet materials, environment lighting and participating media. Produce convergence and variance plots, not only images.

Cumulative review

- Derive the rendering equation/path estimator, throughput update, BSDF PDF and MIS weights actually used in your renderer.
- Validate BSDF energy/sample-eval-pdf consistency and analytic radiometry cases before trusting complex images.
- Produce convergence/variance-per-time plots for diffuse, glossy/small-light and volume/caustic regimes.
- Instrument path families/contribution tails so you can explain where variance comes from.
- Write a literature synthesis comparing estimator assumptions and failure regimes, not paper chronology.

Mastery gate before moving on

- Derive the rendering-equation/path-throughput estimator, BSDF sampling PDF, and MIS weighting actually used in your renderer.
- Show convergence/variance studies on at least three difficult transport regimes and explain which path families dominate error.
- Demonstrate energy/reciprocity/unit checks for relevant materials/lights and one case where a naïve sampler fails catastrophically.
- Literature gate: from Kajiya through Veach/MIS and GPU wavefront rendering, reconstruct the rendering equation/path estimator and one variance-reduction argument; explain which path families and assumptions each method targets.

Volume 7. Geometric Modeling, CAGD, and Shape Representations

Purpose. Learn the representations used to construct and edit shapes: meshes, splines, subdivision, implicit surfaces, SDFs, and constructive models.

Suggested duration. 14-18 weeks

Prerequisites. Volume 1 and programming from Volume 2. Volume 4/5 help with visualization and intersection tests.

Typical module pace. 2.0-2.6 weeks per module (7 modules total).

Primary references for this volume

- Fundamentals of Computer Graphics, Ch. 15 and 21.
- Farin or Piegl & Tiller for deeper splines/NURBS.
- CS184 Bezier/mesh lectures; libigl/Geometry Central for implementation bridges.

Online spine

Berkeley CS184/284A Spring 2026 course: <https://cs184.eecs.berkeley.edu/sp26/>

libigl tutorial: <https://libigl.github.io/tutorial/>

Geometry Central: <https://geometry-central.net/>

Canonical paper track

- Catmull & Clark (1978), Recursively Generated B-Spline Surfaces on Arbitrary Topological Meshes.
 - Loop (1987), Smooth Subdivision Surfaces Based on Triangles.
 - Osher & Fedkiw / SDF-level-set references for implicit shape representation.
 - Recent procedural/generative modeling papers from SIGGRAPH as specialization reading.
- The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Farin, Curves and Surfaces for CAGD - use for Bezier/B-spline theory and continuity reasoning.
- Piegl & Tiller, The NURBS Book - use for knot vectors, de Boor evaluation, knot insertion and rational representations.
- Catmull & Clark (1978), Recursively Generated B-Spline Surfaces on Arbitrary Topological Meshes - classic subdivision reference.
- Loop (1987), Smooth Subdivision Surfaces Based on Triangles - use as the triangular subdivision companion.
- Perlin (1985), An Image Synthesizer - read when studying procedural noise/modeling; reconstruct one core noise experiment rather than only citing it.
- Berkeley CS184 Spring 2026 Bezier/spline and mesh lectures - executable course bridge for the classical material.

Volume 7, Unit 1. Triangle and polygon meshes

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Topology vs geometry, orientation, manifoldness, adjacency, half-edge structures.

A. Reading order

1. Read the primary assignment first: FCG Ch. 12.1; geometry-central/libigl tutorials.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 mesh representations lecture.
3. Keep the most useful executable/reference page open while coding: libigl tutorial - <https://libigl.github.io/tutorial/> ; Berkeley CS184 geometry lectures - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Euler characteristic
- manifold edge/vertex conditions
- orientability intuition
- Precision requirement: distinguish and justify indexed-face lists vs half-edge connectivity, with explicit validation of boundaries, nonmanifold elements, orientation, and Euler characteristic. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive Euler characteristic for manifold triangle meshes and count vertex/edge/face relations; specify manifold/orientation conditions and show how half-edge adjacency encodes them.

- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use nonuniform triangles, boundaries, holes, flipped faces, and near-degenerate elements; see which quality indicators predict downstream failure. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement mesh loader/validator; detect boundary/nonmanifold elements; compute Euler characteristic/genus for closed orientable meshes. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Build the full half-edge table for two triangles (0,1,2) and (2,1,3). Mark twin/next/vertex/face links, boundary halfedges, vertex one-rings, and verify V-E+F for the resulting disk. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use nonuniform triangles, boundaries, holes, flipped faces, and near-degenerate elements; see which quality indicators predict downstream failure. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement a mesh validator that returns typed errors and locations; test on a corpus with boundaries, flipped faces, duplicates, bow-ties, and nonmanifold edges.
- P5 - Machine-readable evidence. Export topological validity counts, Euler characteristic/genus correctness, boundary/nonmanifold detection accuracy, and memory/traversal time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: indexed-face lists vs half-edge connectivity, with explicit validation of boundaries, nonmanifold elements, orientation, and Euler characteristic. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use nonuniform triangles, boundaries, holes, flipped faces, and near-degenerate elements; see which quality indicators predict downstream failure." without hand-editing code between methods. Record topological validity counts, Euler characteristic/genus correctness, boundary/nonmanifold detection accuracy, and memory/traversal time.
- P7 - Deliberate bug and regression drill. Intentionally construct halfedge twins incorrectly at a boundary/nonmanifold edge while local traversals still appear to work on closed meshes. Before running, predict the first observable failure in topological validity counts, Euler characteristic/genus correctness, boundary/nonmanifold detection accuracy, and memory/traversal time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive Euler characteristic for manifold triangle meshes and count vertex/edge/face relations; specify manifold/orientation conditions and show how half-edge adjacency encodes them. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Build the full half-edge table for two triangles (0,1,2) and (2,1,3). Mark twin/next/vertex/face links, boundary halfedges, vertex one-rings, and verify V-E+F for the resulting disk. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement mesh loader/validator; detect boundary/nonmanifold elements; compute Euler characteristic/genus for closed orientable meshes. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement a mesh validator that returns typed errors and locations; test on a corpus with boundaries, flipped faces, duplicates, bow-ties, and nonmanifold edges. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare indexed-face lists vs half-edge connectivity, with explicit validation of boundaries, nonmanifold elements, orientation, and Euler characteristic. Use the same underlying curve/surface/control data and numerical tolerance; where discretization is involved, compare at matched geometric resolution and verify exact invariants such as interpolation, continuity or shape preservation. Record topological validity counts, Euler characteristic/genus correctness, boundary/nonmanifold detection accuracy, and memory/traversal time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use nonuniform triangles, boundaries, holes, flipped faces, and near-degenerate elements; see which quality indicators predict downstream failure. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 12.1; geometry-central/libigl tutorials. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Triangle and polygon meshes. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among indexed-face lists vs half-edge connectivity, with explicit validation of boundaries, nonmanifold elements, orientation, and Euler characteristic. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using topological validity counts, Euler characteristic/genus correctness, boundary/nonmanifold detection accuracy, and memory/traversal time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if indexed-face lists vs half-edge connectivity, with explicit validation of boundaries, nonmanifold elements, orientation, and Euler characteristic improve or expose a reproducible tradeoff in topological validity counts, Euler characteristic/genus correctness, boundary/nonmanifold detection accuracy, and memory/traversal time across several control configurations/shapes while satisfying the exact geometric invariants of the representation (continuity, interpolation, shape preservation, or distance property as applicable).

F. Concrete research project - Triangle and polygon meshes

- F1. Falsifiable question. Under the controlled failure regime "Use nonuniform triangles, boundaries, holes, flipped faces, and near-degenerate elements; see which quality indicators predict downstream failure.", which alternative in indexed-face lists vs half-edge connectivity, with explicit validation of boundaries, nonmanifold elements, orientation, and Euler characteristic is preferable, and is the change explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement a mesh validator that returns typed errors and locations; test on a corpus with boundaries, flipped faces, duplicates, bow-ties, and nonmanifold edges. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Use nonuniform triangles, boundaries, holes, flipped faces, and near-degenerate elements; see which quality indicators predict downstream failure.
- F4. Required evidence. Measure topological validity counts, Euler characteristic/genus correctness, boundary/nonmanifold detection accuracy, and memory/traversal time. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Triangle and polygon meshes. If a stable failure/crossover is found among indexed-face lists vs half-edge connectivity, with explicit validation of boundaries, nonmanifold elements, orientation, and Euler characteristic, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Triangle and polygon meshes. Using the measured topological validity counts, Euler characteristic/genus correctness, boundary/nonmanifold detection accuracy, and memory/traversal time, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Triangle and polygon meshes

- G1. Closed-book derivation/oracle. Derive Euler characteristic for manifold triangle meshes and count vertex/edge/face relations; specify manifold/orientation conditions and show how half-edge adjacency encodes them. Then solve this exact micro-case from scratch without opening your implementation: Build the full half-edge table for two triangles (0,1,2) and (2,1,3). Mark twin/next/vertex/face links, boundary halfedges, vertex one-rings, and verify $V-E+F$ for the resulting disk. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement mesh loader/validator; detect boundary/nonmanifold elements; compute Euler characteristic/genus for closed orientable meshes. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally construct halfedge twins incorrectly at a boundary/nonmanifold edge while local traversals still appear to work on closed meshes. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement a mesh validator that returns typed errors and locations; test on a corpus with boundaries, flipped faces, duplicates, bow-ties, and nonmanifold edges. Then apply this stress without retuning: Use nonuniform triangles, boundaries, holes, flipped faces, and near-degenerate elements; see which quality indicators predict downstream failure. Report topological validity counts, Euler characteristic/genus correctness, boundary/nonmanifold detection accuracy, and memory/traversal time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among indexed-face lists vs half-edge connectivity, with explicit validation of boundaries, nonmanifold elements, orientation, and Euler characteristic for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured topological validity counts, Euler characteristic/genus correctness, boundary/nonmanifold detection accuracy, and memory/traversal time; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Triangle and polygon meshes. Using topological validity counts, Euler characteristic/genus correctness, boundary/nonmanifold detection accuracy, and memory/traversal time, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 7, Unit 2. Bezier curves and surfaces

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Bernstein basis, de Casteljau, continuity, subdivision, tensor-product patches.

A. Reading order

1. Read the primary assignment first: FCG Ch. 15; CS184 Bezier lectures.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 spline/Bezier lecture sequence.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184 Spring 2026 Bezier/spline lectures and worksheets - <https://cs184.eecs.berkeley.edu/sp26/> ; use your CAGD text as the mathematical oracle.

B. Definitions, results and algorithms to know

- Convex hull property
- affine invariance
- variation diminishing intuition
- C0/C1/G1 continuity
- Precision requirement: distinguish and justify Bernstein evaluation vs de Casteljau, including subdivision, derivative/continuity checks, and numerical behavior near parameter endpoints. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive Bernstein basis properties, de Casteljau subdivision, endpoint derivatives, and the convex-hull property; extend the tensor-product construction to Bezier patches.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use high-curvature control polygons and continuity joins; compare analytic derivatives/curvature and

adaptive subdivision error. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement de Casteljau with animation; split curves; fit cubic Bezier to samples. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For cubic control points (0,0), (1,2),(2,2),(3,0), evaluate at $t=0.5$ using Bernstein polynomials and de Casteljau; compute first derivative and split into two cubics at $t=0.5$. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use high-curvature control polygons and continuity joins; compare analytic derivatives/curvature and adaptive subdivision error. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement de Casteljau subdivision and derivative evaluation; verify endpoint/interpolation/convex-hull properties and C0/C1/C2 joins numerically.
- P5 - Machine-readable evidence. Export point/derivative error, subdivision invariance, continuity residual at joins, fit error, and numerical endpoint behavior to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: Bernstein evaluation vs de Casteljau, including subdivision, derivative/continuity checks, and numerical behavior near parameter endpoints. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use high-curvature control polygons and continuity joins; compare analytic derivatives/curvature and adaptive subdivision error." without hand-editing code between methods. Record point/derivative error, subdivision invariance, continuity residual at joins, fit error, and numerical endpoint behavior.
- P7 - Deliberate bug and regression drill. Intentionally overwrite de Casteljau control points in the wrong order so subdivision fails only near endpoints. Before running, predict the first observable failure in point/derivative error, subdivision invariance, continuity residual at joins, fit error, and numerical endpoint behavior. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive Bernstein basis properties, de Casteljau subdivision, endpoint derivatives, and the convex-hull property; extend the tensor-product construction to Bezier patches. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For cubic control points (0,0),(1,2),(2,2),(3,0), evaluate at $t=0.5$ using Bernstein polynomials and de Casteljau; compute first derivative and split into two cubics at $t=0.5$. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement de Casteljau with animation; split curves; fit cubic Bezier to samples. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement de Casteljau subdivision and derivative evaluation; verify endpoint/interpolation/convex-hull properties and C0/C1/C2 joins numerically. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare Bernstein evaluation vs de Casteljau, including subdivision, derivative/continuity checks, and numerical behavior near parameter endpoints. Use the same underlying curve/surface/control data and numerical tolerance; where discretization is involved, compare at matched geometric resolution and verify exact invariants such as interpolation, continuity or shape preservation. Record point/derivative error, subdivision invariance, continuity residual at joins, fit error, and numerical endpoint behavior. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use high-curvature control polygons and continuity joins; compare analytic derivatives/curvature and adaptive subdivision error. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: FCG Ch. 15; CS184 Bezier lectures. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Bezier curves and surfaces. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among Bernstein evaluation vs de Casteljau, including subdivision, derivative/continuity checks, and numerical behavior near parameter endpoints. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using point/derivative error, subdivision invariance, continuity residual at joins, fit error, and numerical endpoint behavior. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if Bernstein evaluation vs de Casteljau, including subdivision, derivative/continuity checks, and numerical behavior near parameter endpoints improve or expose a reproducible tradeoff in point/derivative error, subdivision invariance, continuity residual at joins, fit error, and numerical endpoint behavior across several control configurations/shapes while satisfying the exact geometric invariants of the representation (continuity, interpolation, shape preservation, or distance property as applicable).

F. Concrete research project - Bezier curves and surfaces

F1. Falsifiable question. Under the controlled failure regime "Use high-curvature control polygons and continuity joins; compare analytic derivatives/curvature and adaptive subdivision error.", which alternative in Bernstein evaluation vs de Casteljau, including subdivision, derivative/continuity checks, and numerical behavior near parameter endpoints is preferable, and is the change explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement de Casteljau subdivision and derivative evaluation; verify endpoint/interpolation/convex-hull properties and C0/C1/C2 joins numerically. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Use high-curvature control polygons and continuity joins; compare analytic derivatives/curvature and adaptive subdivision error.

F4. Required evidence. Measure point/derivative error, subdivision invariance, continuity residual at joins, fit error, and numerical endpoint behavior. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Bezier curves and surfaces. If a stable failure/crossover is found among Bernstein evaluation vs de Casteljau, including subdivision, derivative/continuity checks, and numerical behavior near parameter endpoints, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Bezier curves and surfaces. Using the measured point/derivative error, subdivision invariance, continuity residual at joins, fit error, and numerical endpoint behavior, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Bezier curves and surfaces

- G1. Closed-book derivation/oracle. Derive Bernstein basis properties, de Casteljau subdivision, endpoint derivatives, and the convex-hull property; extend the tensor-product construction to Bezier patches. Then solve this exact micro-case from scratch without opening your implementation: For cubic control points $(0,0),(1,2),(2,2),(3,0)$, evaluate at $t=0.5$ using Bernstein polynomials and de Casteljau; compute first derivative and split into two cubics at $t=0.5$. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement de Casteljau with animation; split curves; fit cubic Bezier to samples. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally overwrite de Casteljau control points in the wrong order so subdivision fails only near endpoints. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement de Casteljau subdivision and derivative evaluation; verify endpoint/interpolation/convex-hull properties and C0/C1/C2 joins numerically. Then apply this stress without retuning: Use high-curvature control polygons and continuity joins; compare analytic derivatives/curvature and adaptive subdivision error. Report point/derivative error, subdivision invariance, continuity residual at joins, fit error, and numerical endpoint behavior and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among Bernstein evaluation vs de Casteljau, including subdivision, derivative/continuity checks, and numerical behavior near parameter endpoints for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured point/derivative error, subdivision invariance, continuity residual at joins, fit error, and numerical endpoint behavior; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Bezier curves and surfaces. Using point/derivative error, subdivision invariance, continuity residual at joins, fit error, and numerical endpoint behavior, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 7, Unit 3. B-splines and NURBS

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Local support, knot vectors, de Boor algorithm, rational curves/surfaces.

A. Reading order

1. Read the primary assignment first: Supplement FCG with a CAGD text such as Piegl & Tiller or Farin, selected spline/NURBS chapters.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: NURBS tutorials in CAD/CAGD notes.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184 Spring 2026 spline material - <https://cs184.eecs.berkeley.edu/sp26/> ; implement de Boor/knot insertion from Piegl & Tiller or Farin, not from an unrelated mesh library.

B. Definitions, results and algorithms to know

- Partition of unity
- local support
- knot multiplicity and continuity
- rational representation of conics
- Precision requirement: distinguish and justify de Boor evaluation, knot insertion, and rational representation tested on exact circles and repeated-knot continuity cases. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive Cox-de Boor recursion, partition of unity, local support, and rational NURBS evaluation in homogeneous coordinates; analyze knot insertion without changing shape.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use clustered/repeated knots and uneven control spacing; test local support, continuity changes, conditioning, and fitting error. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement de Boor curve evaluator; knot insertion; rational circle; tensor-product surface patch. Keep a minimal reference path even if you later optimize it.

- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For quadratic knots $[0,0,0,1,2,3,3,3]$ and five scalar control values, perform one de Boor evaluation at $u=1.5$. Then insert knot 1.5 once and verify the evaluated curve value is unchanged. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use clustered/repeated knots and uneven control spacing; test local support, continuity changes, conditioning, and fitting error. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Verify knot insertion leaves geometry invariant to numerical tolerance; construct an exact rational circle and test continuity as knot multiplicity changes.
- P5 - Machine-readable evidence. Export geometry invariance after knot insertion, continuity residual, exact-circle error, evaluation error/time, and condition/sensitivity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: de Boor evaluation, knot insertion, and rational representation tested on exact circles and repeated-knot continuity cases. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use clustered/repeated knots and uneven control spacing; test local support, continuity changes, conditioning, and fitting error." without hand-editing code between methods. Record geometry invariance after knot insertion, continuity residual, exact-circle error, evaluation error/time, and condition/sensitivity.
- P7 - Deliberate bug and regression drill. Intentionally use the wrong knot-span endpoint convention at a repeated knot, causing a continuity/value jump. Before running, predict the first observable failure in geometry invariance after knot insertion, continuity residual, exact-circle error, evaluation error/time, and condition/sensitivity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive Cox-de Boor recursion, partition of unity, local support, and rational NURBS evaluation in homogeneous coordinates; analyze knot insertion without changing shape. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For quadratic knots $[0,0,0,1,2,3,3,3]$ and five scalar control values, perform one de Boor evaluation at $u=1.5$. Then insert knot 1.5 once and verify the evaluated curve value is unchanged. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement de Boor curve evaluator; knot insertion; rational circle. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Verify knot insertion leaves geometry invariant to numerical tolerance; construct an exact rational circle and test continuity as knot multiplicity changes. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare de Boor evaluation, knot insertion, and rational representation tested on exact circles and repeated-knot continuity cases. Use the same underlying curve/surface/control data and numerical tolerance; where discretization is involved, compare at matched geometric resolution and verify exact invariants such as interpolation, continuity or shape preservation. Record geometry invariance after knot insertion, continuity residual, exact-circle error, evaluation error/time, and condition/sensitivity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use clustered/repeated knots and uneven control spacing; test local support, continuity changes, conditioning, and fitting error. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Supplement FCG with a CAGD text such as Piegl & Tiller or Farin, selected spline/NURBS chapters. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on B-splines and NURBS. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among de Boor evaluation, knot insertion, and rational representation tested on exact circles and repeated-knot continuity cases. State, before running experiments, which

option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using geometry invariance after knot insertion, continuity residual, exact-circle error, evaluation error/time, and condition/sensitivity. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if de Boor evaluation, knot insertion, and rational representation tested on exact circles and repeated-knot continuity cases improve or expose a reproducible tradeoff in geometry invariance after knot insertion, continuity residual, exact-circle error, evaluation error/time, and condition/sensitivity across several control configurations/shapes while satisfying the exact geometric invariants of the representation (continuity, interpolation, shape preservation, or distance property as applicable).

F. Concrete research project - B-splines and NURBS

- F1. Falsifiable question. Under the controlled failure regime "Use clustered/repeated knots and uneven control spacing; test local support, continuity changes, conditioning, and fitting error.", characterize the ranking or regime transition across de Boor evaluation, knot insertion, and rational representation tested on exact circles and repeated-knot continuity cases. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Verify knot insertion leaves geometry invariant to numerical tolerance; construct an exact rational circle and test continuity as knot multiplicity changes. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Use clustered/repeated knots and uneven control spacing; test local support, continuity changes, conditioning, and fitting error.
- F4. Required evidence. Measure geometry invariance after knot insertion, continuity residual, exact-circle error, evaluation error/time, and condition/sensitivity. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for B-splines and NURBS. If a stable failure/crossover is found among de Boor evaluation, knot insertion, and rational representation tested on exact circles and repeated-knot continuity cases, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for B-splines and NURBS. Using the measured geometry invariance after knot insertion, continuity residual, exact-circle error, evaluation error/time, and condition/sensitivity, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for B-splines and NURBS

- G1. Closed-book derivation/oracle. Derive Cox-de Boor recursion, partition of unity, local support, and rational NURBS evaluation in homogeneous coordinates; analyze knot insertion without changing shape. Then solve this exact micro-case from scratch without opening your implementation: For quadratic knots $[0,0,0,1,2,3,3,3]$ and five scalar control values, perform one de Boor evaluation at $u=1.5$. Then insert knot 1.5 once and verify the evaluated curve value is unchanged. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement de Boor curve evaluator; knot insertion; rational circle; tensor-product surface patch. Add the oracle as an automated test and make it pass before importing any optimized/framework code.

- G3. Debugging exam. Intentionally use the wrong knot-span endpoint convention at a repeated knot, causing a continuity/value jump. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Verify knot insertion leaves geometry invariant to numerical tolerance; construct an exact rational circle and test continuity as knot multiplicity changes. Then apply this stress without retuning: Use clustered/repeated knots and uneven control spacing; test local support, continuity changes, conditioning, and fitting error. Report geometry invariance after knot insertion, continuity residual, exact-circle error, evaluation error/time, and condition/sensitivity and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across de Boor evaluation, knot insertion, and rational representation tested on exact circles and repeated-knot continuity cases for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured geometry invariance after knot insertion, continuity residual, exact-circle error, evaluation error/time, and condition/sensitivity; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for B-splines and NURBS. Using geometry invariance after knot insertion, continuity residual, exact-circle error, evaluation error/time, and condition/sensitivity, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 7, Unit 4. Subdivision surfaces

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Catmull-Clark, Loop subdivision, extraordinary vertices, limit surfaces.

A. Reading order

1. Read the primary assignment first: Geometry processing/CAGD reference chapters; CS184 geometry lectures.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Implement Loop subdivision and inspect convergence/normal behavior.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184 mesh/geometry lectures - <https://cs184.eecs.berkeley.edu/sp26/> ; use the original Catmull-Clark/Loop references in the volume paper sequence.

B. Definitions, results and algorithms to know

- Refinement masks
- eigenanalysis near extraordinary vertices concept
- Precision requirement: distinguish and justify Loop/Catmull-Clark regular-region behavior vs extraordinary/boundary cases, with empirical convergence to the limit surface. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive one iteration of Loop or Catmull-Clark subdivision as a linear operator; inspect eigenstructure near a regular/extraordinary vertex and connect it to smoothness.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Place extraordinary vertices near sharp/high-curvature features; inspect curvature and continuity behavior under refinement. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Build subdivision viewer; test boundary rules and extraordinary valence. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a regular Loop vertex of valence 6 with six neighbors on a hexagon, compute the updated center weight and one edge-vertex update. Contrast with the formula needed at a boundary vertex. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Place extraordinary vertices near sharp/high-curvature features; inspect curvature and continuity behavior under refinement. Save the input and config of each qualitatively different failure as a named regression case.

- P4 - Research-grade programming extension. Track vertex positions across 1-8 subdivision levels near regular and extraordinary vertices and estimate empirical convergence/limit behavior.
- P5 - Machine-readable evidence. Export distance to estimated limit surface, continuity/normal variation, convergence rate, and extraordinary/boundary error to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: Loop/Catmull-Clark regular-region behavior vs extraordinary/boundary cases, with empirical convergence to the limit surface. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Place extraordinary vertices near sharp/high-curvature features; inspect curvature and continuity behavior under refinement." without hand-editing code between methods. Record distance to estimated limit surface, continuity/normal variation, convergence rate, and extraordinary/boundary error.
- P7 - Deliberate bug and regression drill. Intentionally apply regular interior masks at an extraordinary or boundary vertex. Before running, predict the first observable failure in distance to estimated limit surface, continuity/normal variation, convergence rate, and extraordinary/boundary error. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive one iteration of Loop or Catmull-Clark subdivision as a linear operator; inspect eigenstructure near a regular/extraordinary vertex and connect it to smoothness. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a regular Loop vertex of valence 6 with six neighbors on a hexagon, compute the updated center weight and one edge-vertex update. Contrast with the formula needed at a boundary vertex. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Build subdivision viewer; test boundary rules and extraordinary valence. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Track vertex positions across 1-8 subdivision levels near regular and extraordinary vertices and estimate empirical convergence/limit behavior. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare Loop/Catmull-Clark regular-region behavior vs extraordinary/boundary cases, with empirical convergence to the limit surface. Use the same underlying curve/surface/control data and numerical tolerance; where discretization is involved, compare at matched geometric resolution and verify exact invariants such as interpolation, continuity or shape preservation. Record distance to estimated limit surface, continuity/normal variation, convergence rate, and extraordinary/boundary error. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Place extraordinary vertices near sharp/high-curvature features; inspect curvature and continuity behavior under refinement. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Geometry processing/CAGD reference chapters; CS184 geometry lectures. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Subdivision surfaces. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among Loop/Catmull-Clark regular-region behavior vs extraordinary/boundary cases, with empirical convergence to the limit surface. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using distance to estimated limit surface, continuity/normal variation, convergence rate, and extraordinary/boundary error. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if Loop/Catmull-Clark regular-region behavior vs extraordinary/boundary cases, with empirical convergence to the limit surface improve or expose a reproducible tradeoff in distance to estimated limit surface, continuity/normal variation, convergence rate, and extraordinary/boundary error across several control configurations/shapes while satisfying the exact geometric invariants of the representation (continuity, interpolation, shape preservation, or distance property as applicable).

F. Concrete research project - Subdivision surfaces

F1. Falsifiable question. Under the controlled failure regime "Place extraordinary vertices near sharp/high-curvature features; inspect curvature and continuity behavior under refinement.", which alternative in Loop/Catmull-Clark regular-region behavior vs extraordinary/boundary cases, with empirical convergence to the limit surface is preferable, and is the change explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Track vertex positions across 1-8 subdivision levels near regular and extraordinary vertices and estimate empirical convergence/limit behavior. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Place extraordinary vertices near sharp/high-curvature features; inspect curvature and continuity behavior under refinement.

F4. Required evidence. Measure distance to estimated limit surface, continuity/normal variation, convergence rate, and extraordinary/boundary error. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Subdivision surfaces. If a stable failure/crossover is found among Loop/Catmull-Clark regular-region behavior vs extraordinary/boundary cases, with empirical convergence to the limit surface, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Subdivision surfaces. Using the measured distance to estimated limit surface, continuity/normal variation, convergence rate, and extraordinary/boundary error, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Subdivision surfaces

- G1. Closed-book derivation/oracle. Derive one iteration of Loop or Catmull-Clark subdivision as a linear operator; inspect eigenstructure near a regular/extraordinary vertex and connect it to smoothness. Then solve this exact micro-case from scratch without opening your implementation: For a regular Loop vertex of valence 6 with six neighbors on a hexagon, compute the updated center weight and one edge-vertex update. Contrast with the formula needed at a boundary vertex. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Build subdivision viewer; test boundary rules and extraordinary valence. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally apply regular interior masks at an extraordinary or boundary vertex. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Track vertex positions across 1-8 subdivision levels near regular and extraordinary vertices and estimate empirical convergence/limit behavior. Then apply this stress without retuning: Place extraordinary vertices near sharp/high-curvature features; inspect curvature and continuity behavior under refinement. Report distance to estimated limit surface, continuity/normal variation, convergence rate, and extraordinary/boundary error and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among Loop/Catmull-Clark regular-region behavior vs extraordinary/boundary cases, with empirical convergence to the limit surface for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured distance to estimated limit surface, continuity/normal

variation, convergence rate, and extraordinary/boundary error; state one counterexample that would reverse your recommendation.

- G6. Pass condition for Subdivision surfaces. Using distance to estimated limit surface, continuity/normal variation, convergence rate, and extraordinary/boundary error, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 7, Unit 5. Implicit surfaces and signed distance fields

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Implicit functions, CSG, blending, level sets, SDF operations, sphere tracing.

A. Reading order

1. Read the primary assignment first: FCG Ch. 21; modern SDF notes as supplement.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: ShaderToy-style signed-distance tutorials as optional practice.
3. Keep the most useful executable/reference page open while coding: libigl signed-distance/geometry utilities - <https://libigl.github.io/tutorial/> ; Geometry Central for mesh-side diagnostics - <https://geometry-central.net/>

B. Definitions, results and algorithms to know

- Implicit function theorem intuition
- SDF gradient as normal
- min/max CSG caveats
- Precision requirement: distinguish and justify analytic SDFs vs approximate blended/CSG fields, checking distance/gradient properties and sphere-tracing safety. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive normals and curvature information from the gradient/Hessian of an implicit function; derive sphere tracing from the Lipschitz/SDF bound and explain failure for non-SDF fields.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use thin shells, sharp corners, close surfaces, and non-distance implicit functions; test distance/gradient accuracy and topology preservation. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement ray marching for SDF primitives, smooth blends and CSG; visualize gradient normals. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Evaluate sphere SDFs for centers $(-1,0,0), (1,0,0)$, radius 1 at $x=(0,0,0)$. Compute union/intersection/difference values and gradients; identify why a smooth-min blend is generally not an exact distance field. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use thin shells, sharp corners, close surfaces, and non-distance implicit functions; test distance/gradient accuracy and topology preservation. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Map $|\text{grad } f| - 1$ and sphere-tracing overshoot for smooth blends/CSG; implement conservative step scaling where the field is not a true SDF.
- P5 - Machine-readable evidence. Export $|\text{grad } f| - 1$ error, signed-distance error, sphere-tracing overshoot/miss rate, CSG/blend validity, and steps/ray to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: analytic SDFs vs approximate blended/CSG fields, checking distance/gradient properties and sphere-tracing safety. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use thin shells, sharp corners, close surfaces, and non-distance implicit functions; test distance/gradient accuracy and topology preservation." without hand-editing code between methods. Record $|\text{grad } f| - 1$ error, signed-distance error, sphere-tracing overshoot/miss rate, CSG/blend validity, and steps/ray.

- P7 - Deliberate bug and regression drill. Intentionally assume a smooth union/blend remains an exact signed-distance field and sphere-trace with the raw blended value. Before running, predict the first observable failure in $|\text{grad } f| - 1$ error, signed-distance error, sphere-tracing overshoot/miss rate, CSG/blend validity, and steps/ray. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive normals and curvature information from the gradient/Hessian of an implicit function; derive sphere tracing from the Lipschitz/SDF bound and explain failure for non-SDF fields. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Evaluate sphere SDFs for centers $(-1,0,0), (1,0,0)$, radius 1 at $x=(0,0,0)$. Compute union/intersection/difference values and gradients; identify why a smooth-min blend is generally not an exact distance field. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement ray marching for SDF primitives, smooth blends and CSG; visualize gradient normals. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Map $|\text{grad } f| - 1$ and sphere-tracing overshoot for smooth blends/CSG; implement conservative step scaling where the field is not a true SDF. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare analytic SDFs vs approximate blended/CSG fields, checking distance/gradient properties and sphere-tracing safety. Use the same underlying curve/surface/control data and numerical tolerance; where discretization is involved, compare at matched geometric resolution and verify exact invariants such as interpolation, continuity or shape preservation. Record $|\text{grad } f| - 1$ error, signed-distance error, sphere-tracing overshoot/miss rate, CSG/blend validity, and steps/ray. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use thin shells, sharp corners, close surfaces, and non-distance implicit functions; test distance/gradient accuracy and topology preservation. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 21; modern SDF notes as supplement. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Implicit surfaces and signed distance fields. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among analytic SDFs vs approximate blended/CSG fields, checking distance/gradient properties and sphere-tracing safety. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using $|\text{grad } f| - 1$ error, signed-distance error, sphere-tracing overshoot/miss rate, CSG/blend validity, and steps/ray. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if analytic SDFs vs approximate blended/CSG fields, checking distance/gradient properties and sphere-tracing safety improve or expose a reproducible tradeoff in $|\text{grad } f| - 1$ error, signed-distance error, sphere-tracing overshoot/miss rate, CSG/blend validity, and steps/ray across several control configurations/shapes while satisfying the exact geometric invariants of the representation (continuity, interpolation, shape preservation, or distance property as applicable).

F. Concrete research project - Implicit surfaces and signed distance fields

- F1. Falsifiable question. Under the controlled failure regime "Use thin shells, sharp corners, close surfaces, and non-distance implicit functions; test distance/gradient accuracy and topology preservation.", which alternative in analytic SDFs vs approximate blended/CSG fields, checking distance/gradient properties and sphere-tracing safety is preferable, and is the change explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Map $|\text{grad } f| - 1$ and sphere-tracing overshoot for smooth blends/CSG; implement conservative step scaling where the field is not a true SDF. Add a single

experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Use thin shells, sharp corners, close surfaces, and non-distance implicit functions; test distance/gradient accuracy and topology preservation.

F4. Required evidence. Measure $|\text{grad } f|$ -1 error, signed-distance error, sphere-tracing overshoot/miss rate, CSG/blend validity, and steps/ray. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Implicit surfaces and signed distance fields. If a stable failure/crossover is found among analytic SDFs vs approximate blended/CSG fields, checking distance/gradient properties and sphere-tracing safety, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Implicit surfaces and signed distance fields. Using the measured $|\text{grad } f|$ -1 error, signed-distance error, sphere-tracing overshoot/miss rate, CSG/blend validity, and steps/ray, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Implicit surfaces and signed distance fields

- G1. Closed-book derivation/oracle. Derive normals and curvature information from the gradient/Hessian of an implicit function; derive sphere tracing from the Lipschitz/SDF bound and explain failure for non-SDF fields. Then solve this exact micro-case from scratch without opening your implementation: Evaluate sphere SDFs for centers $(-1,0,0), (1,0,0)$, radius 1 at $x=(0,0,0)$. Compute union/intersection/difference values and gradients; identify why a smooth-min blend is generally not an exact distance field. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement ray marching for SDF primitives, smooth blends and CSG; visualize gradient normals. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally assume a smooth union/blend remains an exact signed-distance field and sphere-trace with the raw blended value. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Map $|\text{grad } f|$ -1 and sphere-tracing overshoot for smooth blends/CSG; implement conservative step scaling where the field is not a true SDF. Then apply this stress without retuning: Use thin shells, sharp corners, close surfaces, and non-distance implicit functions; test distance/gradient accuracy and topology preservation. Report $|\text{grad } f|$ -1 error, signed-distance error, sphere-tracing overshoot/miss rate, CSG/blend validity, and steps/ray and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among analytic SDFs vs approximate blended/CSG fields, checking distance/gradient properties and sphere-tracing safety for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured $|\text{grad } f|$ -1 error, signed-distance error, sphere-tracing overshoot/miss rate, CSG/blend validity, and steps/ray; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Implicit surfaces and signed distance fields. Using $|\text{grad } f|$ -1 error, signed-distance error, sphere-tracing overshoot/miss rate, CSG/blend validity, and steps/ray, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 7, Unit 6. Procedural modeling

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Noise, grammars, instancing, L-systems, terrain, procedural materials/geometries.

A. Reading order

1. Read the primary assignment first: RTR procedural sections; classic procedural modeling references.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Houdini procedural tutorials optional.
3. Keep the most useful executable/reference page open while coding: Use the implementation/source accompanying the procedural method you selected; keep Perlin (1985) and the relevant modeling text section open as the algorithmic reference.

B. Definitions, results and algorithms to know

- Stationarity/frequency intuition for noise
- hierarchical procedural representations
- Precision requirement: distinguish and justify hand-authored geometry vs parameterized procedural rules, measuring controllability, validity, diversity, and sensitivity to parameters/seeds. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Formalize a procedural generator as a parameterized grammar/function composition, derive how local random choices propagate to geometry statistics, and identify constraints needed for reproducible variation.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Randomly sample parameters near validity boundaries and count self-intersections, disconnected parts, nonmanifold output, and semantic failures. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Generate terrain/vegetation/city-like scene procedurally and expose meaningful parameters. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Generate the first three iterations of L-system $F \rightarrow F[+F]F[-F]F$ and count segments. Change one angle parameter and predict which geometric properties change continuously versus combinatorially. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Randomly sample parameters near validity boundaries and count self-intersections, disconnected parts, nonmanifold output, and semantic failures. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Expose 5-10 interpretable parameters and automatically sample 500 models; reject invalid geometry and quantify diversity/sensitivity rather than showing handpicked outputs.
- P5 - Machine-readable evidence. Export valid-model fraction, parameter sensitivity/Jacobian proxy, geometric diversity, constraint violations, and generation time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: hand-authored geometry vs parameterized procedural rules, measuring controllability, validity, diversity, and sensitivity to parameters/seeds. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Randomly sample parameters near validity boundaries and count self-intersections, disconnected parts, nonmanifold output, and semantic failures." without hand-editing code between methods. Record valid-model fraction, parameter sensitivity/Jacobian proxy, geometric diversity, constraint violations, and generation time.
- P7 - Deliberate bug and regression drill. Intentionally let generation depend on global RNG/order so the same seed no longer reproduces a model after a harmless code refactor. Before running, predict the first observable failure in valid-model fraction, parameter sensitivity/Jacobian proxy, geometric diversity, constraint violations, and generation time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Formalize a procedural generator as a parameterized grammar/function composition, derive how local random choices propagate to geometry statistics, and identify constraints needed for reproducible variation.

Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. Generate the first three iterations of L-system $F \rightarrow F[+F]F[-F]F$ and count segments. Change one angle parameter and predict which geometric properties change continuously versus combinatorially. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Generate terrain/vegetation/city-like scene procedurally and expose meaningful parameters. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Expose 5-10 interpretable parameters and automatically sample 500 models; reject invalid geometry and quantify diversity/sensitivity rather than showing handpicked outputs. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare hand-authored geometry vs parameterized procedural rules, measuring controllability, validity, diversity, and sensitivity to parameters/seeds. Use the same underlying curve/surface/control data and numerical tolerance; where discretization is involved, compare at matched geometric resolution and verify exact invariants such as interpolation, continuity or shape preservation. Record valid-model fraction, parameter sensitivity/Jacobian proxy, geometric diversity, constraint violations, and generation time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Randomly sample parameters near validity boundaries and count self-intersections, disconnected parts, nonmanifold output, and semantic failures. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: RTR procedural sections; classic procedural modeling references. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Procedural modeling. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among hand-authored geometry vs parameterized procedural rules, measuring controllability, validity, diversity, and sensitivity to parameters/seeds. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using valid-model fraction, parameter sensitivity/Jacobian proxy, geometric diversity, constraint violations, and generation time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if hand-authored geometry vs parameterized procedural rules, measuring controllability, validity, diversity, and sensitivity to parameters/seeds improve or expose a reproducible tradeoff in valid-model fraction, parameter sensitivity/Jacobian proxy, geometric diversity, constraint violations, and generation time across several control configurations/shapes while satisfying the exact geometric invariants of the representation (continuity, interpolation, shape preservation, or distance property as applicable).

F. Concrete research project - Procedural modeling

F1. Falsifiable question. Under the controlled failure regime "Randomly sample parameters near validity boundaries and count self-intersections, disconnected parts, nonmanifold output, and semantic failures.", which alternative in hand-authored geometry vs parameterized procedural rules, measuring controllability, validity, diversity, and sensitivity to parameters/seeds is preferable, and is the change explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Expose 5-10 interpretable parameters and automatically sample 500 models; reject invalid geometry and quantify diversity/sensitivity rather than showing handpicked outputs. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Randomly sample parameters near validity boundaries and count self-intersections, disconnected parts, nonmanifold output, and semantic failures.

F4. Required evidence. Measure valid-model fraction, parameter sensitivity/Jacobian proxy, geometric diversity, constraint violations, and generation time. Deliver input/output meshes or control data, invariant/validity checks,

multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Procedural modeling. If a stable failure/crossover is found among hand-authored geometry vs parameterized procedural rules, measuring controllability, validity, diversity, and sensitivity to parameters/seeds, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Procedural modeling. Using the measured valid-model fraction, parameter sensitivity/Jacobian proxy, geometric diversity, constraint violations, and generation time, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Procedural modeling

- G1. Closed-book derivation/oracle. Formalize a procedural generator as a parameterized grammar/function composition, derive how local random choices propagate to geometry statistics, and identify constraints needed for reproducible variation. Then solve this exact micro-case from scratch without opening your implementation: Generate the first three iterations of L-system $F \rightarrow F[+F]F[-F]F$ and count segments. Change one angle parameter and predict which geometric properties change continuously versus combinatorially. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Generate terrain/vegetation/city-like scene procedurally and expose meaningful parameters. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally let generation depend on global RNG/order so the same seed no longer reproduces a model after a harmless code refactor. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Expose 5-10 interpretable parameters and automatically sample 500 models; reject invalid geometry and quantify diversity/sensitivity rather than showing handpicked outputs. Then apply this stress without retuning: Randomly sample parameters near validity boundaries and count self-intersections, disconnected parts, nonmanifold output, and semantic failures. Report valid-model fraction, parameter sensitivity/Jacobian proxy, geometric diversity, constraint violations, and generation time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among hand-authored geometry vs parameterized procedural rules, measuring controllability, validity, diversity, and sensitivity to parameters/seeds for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured valid-model fraction, parameter sensitivity/Jacobian proxy, geometric diversity, constraint violations, and generation time; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Procedural modeling. Using valid-model fraction, parameter sensitivity/Jacobian proxy, geometric diversity, constraint violations, and generation time, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 7, Unit 7. Shape optimization and design variables

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Parameterization, constraints, fairness energies, differentiable shape objectives.

A. Reading order

1. Read the primary assignment first: Connect to Vol. 8 DDG and Vol. 13 inverse rendering.

2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: libigl optimization/deformation examples.
3. Keep the most useful executable/reference page open while coding: Use libigl/Geometry Central for geometry operators and your optimization code from Volume 1; later use Mitsuba only when the objective actually contains rendering.

B. Definitions, results and algorithms to know

- Fairness energies, smoothness penalties, design-space parameterization
- Precision requirement: distinguish and justify direct vertex/control-point parameters vs lower-dimensional geometric parameters under identical interpolation/fairness constraints. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a shape/design objective with explicit constraints and sensitivities for a low-dimensional geometry parameterization; compare direct parameter gradients with finite-difference shape perturbations.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Solve the same design objective with direct vertices, spline/control variables, and a compact latent/implicit parameterization; compare conditioning and constraint violations. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Optimize a curve/surface to satisfy interpolation plus fairness; compare parameterizations. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Optimize the height h of a rectangle of fixed area $A=1$ to minimize perimeter. Solve analytically, then represent the same shape by four free vertices and list the constraints needed to recover the one-parameter problem. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Solve the same design objective with direct vertices, spline/control variables, and a compact latent/implicit parameterization; compare conditioning and constraint violations. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement two parameterizations for the same curve/surface design objective and compare condition number, convergence, feasibility violations, and final fairness.
- P5 - Machine-readable evidence. Export objective/constraint residual, fairness, parameter/geometry condition number, iterations, and feasibility rate to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: direct vertex/control-point parameters vs lower-dimensional geometric parameters under identical interpolation/fairness constraints. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Solve the same design objective with direct vertices, spline/control variables, and a compact latent/implicit parameterization; compare conditioning and constraint violations." without hand-editing code between methods. Record objective/constraint residual, fairness, parameter/geometry condition number, iterations, and feasibility rate.
- P7 - Deliberate bug and regression drill. Intentionally optimize in one parameterization but project constraints in a different coordinate space without accounting for the map. Before running, predict the first observable failure in objective/constraint residual, fairness, parameter/geometry condition number, iterations, and feasibility rate. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a shape/design objective with explicit constraints and sensitivities for a low-dimensional geometry parameterization; compare direct parameter gradients with finite-difference shape perturbations. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Optimize the height h of a rectangle of fixed area $A=1$ to minimize perimeter. Solve analytically, then represent the same shape by four free vertices and list the constraints needed to recover the one-parameter problem. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Optimize a curve/surface to satisfy interpolation plus fairness; compare parameterizations. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement two parameterizations for the same curve/surface design objective and compare condition number, convergence, feasibility violations, and final fairness. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare direct vertex/control-point parameters vs lower-dimensional geometric parameters under identical interpolation/fairness constraints. Use the same underlying curve/surface/control data and numerical tolerance; where discretization is involved, compare at matched geometric resolution and verify exact invariants such as interpolation, continuity or shape preservation. Record objective/constraint residual, fairness, parameter/geometry condition number, iterations, and feasibility rate. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Solve the same design objective with direct vertices, spline/control variables, and a compact latent/implicit parameterization; compare conditioning and constraint violations. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Connect to Vol. 8 DDG and Vol. 13 inverse rendering. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Shape optimization and design variables. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among direct vertex/control-point parameters vs lower-dimensional geometric parameters under identical interpolation/fairness constraints. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using objective/constraint residual, fairness, parameter/geometry condition number, iterations, and feasibility rate. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if direct vertex/control-point parameters vs lower-dimensional geometric parameters under identical interpolation/fairness constraints improve or expose a reproducible tradeoff in objective/constraint residual, fairness, parameter/geometry condition number, iterations, and feasibility rate across several control configurations/shapes while satisfying the exact geometric invariants of the representation (continuity, interpolation, shape preservation, or distance property as applicable).

F. Concrete research project - Shape optimization and design variables

- F1. Falsifiable question. Under the controlled failure regime "Solve the same design objective with direct vertices, spline/control variables, and a compact latent/implicit parameterization; compare conditioning and constraint violations.", which alternative in direct vertex/control-point parameters vs lower-dimensional geometric parameters under identical interpolation/fairness constraints is preferable, and is the change explained by conditioning, local linearization, and loss/parameter scaling? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement two parameterizations for the same curve/surface design objective and compare condition number, convergence, feasibility violations, and final fairness. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Solve the same design objective with direct vertices, spline/control variables, and a compact latent/implicit parameterization; compare conditioning and constraint violations.
- F4. Required evidence. Measure objective/constraint residual, fairness, parameter/geometry condition number, iterations, and feasibility rate. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Shape optimization and design variables. If a stable failure/crossover is found among direct vertex/control-point parameters vs lower-dimensional geometric parameters under identical interpolation/fairness

constraints, implement one adaptive or corrected variant that directly targets conditioning, local linearization, and loss/parameter scaling; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Shape optimization and design variables. Using the measured objective/constraint residual, fairness, parameter/geometry condition number, iterations, and feasibility rate, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Shape optimization and design variables

- G1. Closed-book derivation/oracle. Derive a shape/design objective with explicit constraints and sensitivities for a low-dimensional geometry parameterization; compare direct parameter gradients with finite-difference shape perturbations. Then solve this exact micro-case from scratch without opening your implementation: Optimize the height h of a rectangle of fixed area $A=1$ to minimize perimeter. Solve analytically, then represent the same shape by four free vertices and list the constraints needed to recover the one-parameter problem. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Optimize a curve/surface to satisfy interpolation plus fairness; compare parameterizations. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally optimize in one parameterization but project constraints in a different coordinate space without accounting for the map. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement two parameterizations for the same curve/surface design objective and compare condition number, convergence, feasibility violations, and final fairness. Then apply this stress without retuning: Solve the same design objective with direct vertices, spline/control variables, and a compact latent/implicit parameterization; compare conditioning and constraint violations. Report objective/constraint residual, fairness, parameter/geometry condition number, iterations, and feasibility rate and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among direct vertex/control-point parameters vs lower-dimensional geometric parameters under identical interpolation/fairness constraints for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured objective/constraint residual, fairness, parameter/geometry condition number, iterations, and feasibility rate; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Shape optimization and design variables. Using objective/constraint residual, fairness, parameter/geometry condition number, iterations, and feasibility rate, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 7 Closeout: Project, Review and Mastery Gate

Volume project. Build a shape-modeling sandbox supporting Bezier/B-spline curves, subdivision, mesh IO, implicit/SDF primitives and one procedural generator.

Cumulative review

- Implement and test at least three shape representations: mesh, spline/subdivision and implicit/SDF.
- Verify exact invariants such as knot insertion, partition of unity, continuity class or signed-distance behavior on hand-checkable shapes.
- Run a representation comparison on thin features, topology change, editing locality and conditioning.
- Build a validator/visualizer for control meshes, knot structure, subdivision levels and implicit-field diagnostics.
- Write a modeling synthesis explaining when representation choice constrains later rendering, optimization or simulation.

Mastery gate before moving on

- Evaluate mesh, spline/subdivision, and implicit/SDF representations on the same modeling/editing task and justify the representation choice quantitatively.
- Pass continuity/derivative or signed-distance/gradient tests for your modeling primitives.
- Show one procedural/shape-optimization example where parameter validity and conditioning are measured rather than assumed.

Volume 8. Geometry Processing and Discrete Differential Geometry

Purpose. Build the mathematical and algorithmic foundation for research on meshes and surfaces.

Suggested duration. 18-22 weeks

Prerequisites. Volumes 1, 2 and 7. Sparse linear algebra and differential geometry are central.

Typical module pace. 2.0-2.4 weeks per module (9 modules total).

Primary references for this volume

- Keenan Crane, Discrete Differential Geometry: An Applied Introduction (primary).
- Polygon Mesh Processing (Botsch et al.) as a broad geometry-processing reference.
- libigl tutorial and Geometry Central documentation for executable algorithms.

Online spine

Keenan Crane - Discrete Differential Geometry: <https://www.cs.cmu.edu/~kmc Crane/Projects/DDG/>

libigl tutorial: <https://libigl.github.io/tutorial/>

Geometry Central: <https://geometry-central.net/>

Canonical paper track

- Meyer et al. (2003), Discrete Differential-Geometry Operators for Triangulated 2-Manifolds.
- Desbrun et al. (1999), Implicit Fairing of Irregular Meshes Using Diffusion and Curvature Flow.
- Levy et al. (2002), Least Squares Conformal Maps.
- Sorkine & Alexa (2007), As-Rigid-As-Possible Surface Modeling.
- Crane, Weischedel & Wardetzky (2013), Geodesics in Heat.
- Jacobson et al. (2013), Robust Inside-Outside Segmentation using Generalized Winding Numbers.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Meyer, Desbrun, Schröder & Barr (2003), Discrete Differential-Geometry Operators for Triangulated 2-Manifolds.
- Lévy, Petitjean, Ray & Maillot (2002), Least Squares Conformal Maps for Automatic Texture Atlas Generation.
- Sorkine & Alexa (2007), As-Rigid-As-Possible Surface Modeling.
- Crane, Weischedel & Wardetzky (2013), Geodesics in Heat: A New Approach to Computing Distance Based on Heat Flow.
- Shewchuk (1997), Adaptive Precision Floating-Point Arithmetic and Fast Robust Predicates for Computational Geometry.
- Crane DDG notes + libigl + Geometry Central - use these as the derivation/implementation spine and verify conventions against your own tests.

Volume 8, Unit 1. Discrete curves and surfaces

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Discrete tangents, normals, curvature, metric quantities, mesh quality.

A. Reading order

1. Read the primary assignment first: Keenan Crane DDG lectures on discrete curves/surfaces; libigl Ch. 2 examples.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CMU DDG videos.
3. Keep the most useful executable/reference page open while coding: libigl tutorial - <https://libigl.github.io/tutorial/>; Geometry Central - <https://geometry-central.net/>

B. Definitions, results and algorithms to know

- Angle defect
- cotangent weights
- discrete curvature measures
- Precision requirement: distinguish and justify angle-defect/cotan discrete quantities vs analytic sphere/cylinder/torus quantities under refinement and irregular triangulation. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive discrete curvature quantities for polygonal curves/triangle meshes and verify convergence on sampled analytic shapes; explain which quantities live on vertices, edges, or faces.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Evaluate discrete normal/curvature formulas on analytic shapes under mesh refinement, irregularity, and positional noise. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Compute vertex normals, Gaussian curvature by angle defect, mean curvature normal via cotan Laplacian. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a square split into two triangles, compute cotangent weights on the diagonal and boundary edges, then compare discrete mean-curvature behavior before and after perturbing one vertex out of plane. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Evaluate discrete normal/curvature formulas on analytic shapes under mesh refinement, irregularity, and positional noise. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Generate multiple triangulations of the same sphere/torus and plot Gaussian/mean-curvature error against mesh size and minimum angle.
- P5 - Machine-readable evidence. Export analytic curvature/normal error, convergence rate, triangulation sensitivity, and operator symmetry/conservation checks to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: angle-defect/cotan discrete quantities vs analytic sphere/cylinder/torus quantities under refinement and irregular triangulation. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Evaluate discrete normal/curvature formulas on analytic shapes under mesh refinement, irregularity, and positional noise." without hand-editing code between methods. Record analytic curvature/normal error, convergence rate, triangulation sensitivity, and operator symmetry/conservation checks.
- P7 - Deliberate bug and regression drill. Intentionally assemble cotangent weights with an orientation/sign error that cancels on a symmetric mesh but fails on an irregular one. Before running, predict the first observable failure in analytic curvature/normal error, convergence rate, triangulation sensitivity, and operator symmetry/conservation checks. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive discrete curvature quantities for polygonal curves/triangle meshes and verify convergence on sampled analytic shapes; explain which quantities live on vertices, edges, or faces. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. For a square split into two triangles, compute cotangent weights on the diagonal and boundary edges, then compare discrete mean-curvature behavior before and after perturbing one vertex out of plane. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Compute vertex normals, Gaussian curvature by angle defect, mean curvature normal via cotan Laplacian. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Generate multiple triangulations of the same sphere/torus and plot Gaussian/mean-curvature error against mesh size and minimum angle. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare angle-defect/cotan discrete quantities vs analytic sphere/cylinder/torus quantities under refinement and irregular triangulation. Use multiple samplings/remeshings of the same underlying continuous shape, with matched vertex/element count or approximation error; keep boundary/topology conditions explicit. Record analytic curvature/normal error, convergence rate, triangulation sensitivity, and operator symmetry/conservation checks. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Evaluate discrete normal/curvature formulas on analytic shapes under mesh refinement, irregularity, and positional noise. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Keenan Crane DDG lectures on discrete curves/surfaces; libigl Ch. 2 examples. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Discrete curves and surfaces. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among angle-defect/cotan discrete quantities vs analytic sphere/cylinder/torus quantities under refinement and irregular triangulation. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using analytic curvature/normal error, convergence rate, triangulation sensitivity, and operator symmetry/conservation checks. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among angle-defect/cotan discrete quantities vs analytic sphere/cylinder/torus quantities under refinement and irregular triangulation persists across multiple samplings/resolutions of the same underlying shapes and is explained by a discrete/intrinsic geometric property measured through analytic curvature/normal error, convergence rate, triangulation sensitivity, and operator symmetry/conservation checks, not a single mesh idiosyncrasy.

F. Concrete research project - Discrete curves and surfaces

- F1. Falsifiable question. Under the controlled failure regime "Evaluate discrete normal/curvature formulas on analytic shapes under mesh refinement, irregularity, and positional noise.", which alternative in angle-defect/cotan discrete quantities vs analytic sphere/cylinder/torus quantities under refinement and irregular triangulation is preferable, and is the change explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Generate multiple triangulations of the same sphere/torus and plot Gaussian/mean-curvature error against mesh size and minimum angle. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Evaluate discrete normal/curvature formulas on analytic shapes under mesh refinement, irregularity, and positional noise.
- F4. Required evidence. Measure analytic curvature/normal error, convergence rate, triangulation sensitivity, and operator symmetry/conservation checks. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Discrete curves and surfaces. If a stable failure/crossover is found among angle-defect/cotan discrete quantities vs analytic sphere/cylinder/torus quantities under refinement and irregular triangulation, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Discrete curves and surfaces. Using the measured analytic curvature/normal error, convergence rate, triangulation sensitivity, and operator symmetry/conservation checks, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Discrete curves and surfaces

- G1. Closed-book derivation/oracle. Derive discrete curvature quantities for polygonal curves/triangle meshes and verify convergence on sampled analytic shapes; explain which quantities live on vertices, edges, or faces. Then solve this exact micro-case from scratch without opening your implementation: For a square split into two triangles, compute cotangent weights on the diagonal and boundary edges, then compare discrete mean-curvature behavior before and after perturbing one vertex out of plane. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Compute vertex normals, Gaussian curvature by angle defect, mean curvature normal via cotan Laplacian. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally assemble cotangent weights with an orientation/sign error that cancels on a symmetric mesh but fails on an irregular one. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Generate multiple triangulations of the same sphere/torus and plot Gaussian/mean-curvature error against mesh size and minimum angle. Then apply this stress without retuning: Evaluate discrete normal/curvature formulas on analytic shapes under mesh refinement, irregularity, and positional noise. Report analytic curvature/normal error, convergence rate, triangulation sensitivity, and operator symmetry/conservation checks and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among angle-defect/cotan discrete quantities vs analytic sphere/cylinder/torus quantities under refinement and irregular triangulation for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured analytic curvature/normal error, convergence rate, triangulation sensitivity, and operator symmetry/conservation checks; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Discrete curves and surfaces. Using analytic curvature/normal error, convergence rate, triangulation sensitivity, and operator symmetry/conservation checks, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 8, Unit 2. Exterior algebra and differential forms

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. k-vectors, k-forms, wedge product, exterior derivative, Hodge star, Stokes.

A. Reading order

1. Read the primary assignment first: CMU DDG exterior algebra/calculus lecture sequence.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: DDG lecture videos and assignments.
3. Keep the most useful executable/reference page open while coding: libigl tutorial - <https://libigl.github.io/tutorial/> ; Geometry Central - <https://geometry-central.net/>

B. Definitions, results and algorithms to know

- Generalized Stokes theorem
- primal/dual mesh intuition
- DEC operators
- Precision requirement: distinguish and justify assembled discrete d_0/d_1 /Hodge-star operators with exact $d_1*d_0=0$ and orientation/sign tests on simple complexes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive wedge product antisymmetry, primal/dual k-form placement on a mesh, and the discrete exterior derivative; verify the discrete identity $d^2=0$ on a small complex.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Verify $d*d=0$ and a discrete Stokes relation numerically on meshes with boundaries, holes, and irregular cells. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement a small discrete exterior calculus operator chain and verify $d*d=0$ numerically. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Orient one triangle (0,1,2). Write d_0 and d_1 matrices explicitly and multiply d_1*d_0 to obtain zero. Reverse the triangle orientation and identify exactly which signs change. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Verify $d*d=0$ and a discrete Stokes relation numerically on meshes with boundaries, holes, and irregular cells. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Assemble d_0 and d_1 explicitly, test orientation flips, verify $d_1@d_0$ is numerically zero, and visualize a primal/dual 1-form on a small mesh.
- P5 - Machine-readable evidence. Export $||d_1 d_0||$, orientation/sign consistency, discrete Stokes residual, reconstruction error, and sparse-operator properties to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: assembled discrete d_0/d_1 /Hodge-star operators with exact $d_1*d_0=0$ and orientation/sign tests on simple complexes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Verify $d*d=0$ and a discrete Stokes relation numerically on meshes with boundaries, holes, and irregular cells." without hand-editing code between methods. Record $||d_1 d_0||$, orientation/sign consistency, discrete Stokes residual, reconstruction error, and sparse-operator properties.
- P7 - Deliberate bug and regression drill. Intentionally orient one edge/face inconsistently so the discrete identity $d_1*d_0=0$ is violated. Before running, predict the first observable failure in $||d_1 d_0||$, orientation/sign consistency, discrete Stokes residual, reconstruction error, and sparse-operator properties. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive wedge product antisymmetry, primal/dual k-form placement on a mesh, and the discrete exterior derivative; verify the discrete identity $d^2=0$ on a small complex. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Orient one triangle (0,1,2). Write d_0 and d_1 matrices explicitly and multiply d_1*d_0 to obtain zero. Reverse the triangle orientation and identify exactly which signs change. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement a small discrete exterior calculus operator chain and verify $d*d=0$ numerically. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Assemble d_0 and d_1 explicitly, test orientation flips, verify $d_1@d_0$ is numerically zero, and visualize a primal/dual 1-form on a small mesh. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare assembled discrete d_0/d_1 /Hodge-star operators with exact $d_1*d_0=0$ and orientation/sign tests on simple complexes. Use multiple samplings/remeshings of the same underlying continuous shape, with matched vertex/element count or approximation error; keep boundary/topology conditions explicit. Record $||d_1 d_0||$, orientation/sign consistency, discrete Stokes residual, reconstruction error, and sparse-operator properties. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Verify $d*d=0$ and a discrete Stokes relation numerically on meshes with boundaries, holes, and irregular cells. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: CMU DDG exterior algebra/calculus lecture sequence. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Exterior algebra and differential forms. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among assembled discrete d_0/d_1 /Hodge-star operators with exact $d_1*d_0=0$ and orientation/sign tests on simple complexes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using $||d_1 d_0||$, orientation/sign consistency, discrete Stokes residual, reconstruction error, and sparse-operator properties. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among assembled discrete d_0/d_1 /Hodge-star operators with exact $d_1*d_0=0$ and orientation/sign tests on simple complexes persists across multiple samplings/resolutions of the same underlying shapes and is explained by a discrete/intrinsic geometric property measured through $||d_1 d_0||$, orientation/sign consistency, discrete Stokes residual, reconstruction error, and sparse-operator properties, not a single mesh idiosyncrasy.

F. Concrete research project - Exterior algebra and differential forms

F1. Falsifiable question. Under the controlled failure regime "Verify $d*d=0$ and a discrete Stokes relation numerically on meshes with boundaries, holes, and irregular cells.", characterize the ranking or regime transition across assembled discrete d_0/d_1 /Hodge-star operators with exact $d_1*d_0=0$ and orientation/sign tests on simple complexes. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Assemble d_0 and d_1 explicitly, test orientation flips, verify $d_1@d_0$ is numerically zero, and visualize a primal/dual 1-form on a small mesh. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Verify $d*d=0$ and a discrete Stokes relation numerically on meshes with boundaries, holes, and irregular cells.

F4. Required evidence. Measure $||d_1 d_0||$, orientation/sign consistency, discrete Stokes residual, reconstruction error, and sparse-operator properties. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Exterior algebra and differential forms. If a stable failure/crossover is found among assembled discrete d_0/d_1 /Hodge-star operators with exact $d_1*d_0=0$ and orientation/sign tests on simple complexes, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Exterior algebra and differential forms. Using the measured $||d_1 d_0||$, orientation/sign consistency, discrete Stokes residual, reconstruction error, and sparse-operator properties, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves

correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Exterior algebra and differential forms

- G1. Closed-book derivation/oracle. Derive wedge product antisymmetry, primal/dual k-form placement on a mesh, and the discrete exterior derivative; verify the discrete identity $d^2=0$ on a small complex. Then solve this exact micro-case from scratch without opening your implementation: Orient one triangle (0,1,2). Write d_0 and d_1 matrices explicitly and multiply d_1*d_0 to obtain zero. Reverse the triangle orientation and identify exactly which signs change. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement a small discrete exterior calculus operator chain and verify $d*d=0$ numerically. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally orient one edge/face inconsistently so the discrete identity $d_1*d_0=0$ is violated. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Assemble d_0 and d_1 explicitly, test orientation flips, verify $d_1@d_0$ is numerically zero, and visualize a primal/dual 1-form on a small mesh. Then apply this stress without retuning: Verify $d*d=0$ and a discrete Stokes relation numerically on meshes with boundaries, holes, and irregular cells. Report $||d_1 d_0||$, orientation/sign consistency, discrete Stokes residual, reconstruction error, and sparse-operator properties and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across assembled discrete d_0/d_1 /Hodge-star operators with exact $d_1*d_0=0$ and orientation/sign tests on simple complexes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured $||d_1 d_0||$, orientation/sign consistency, discrete Stokes residual, reconstruction error, and sparse-operator properties; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Exterior algebra and differential forms. Using $||d_1 d_0||$, orientation/sign consistency, discrete Stokes residual, reconstruction error, and sparse-operator properties, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 8, Unit 3. Laplace-Beltrami and smoothing

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Cotangent Laplacian, mass matrices, diffusion, mean curvature flow.

A. Reading order

1. Read the primary assignment first: libigl differential quantities chapter; DDG Laplacian/FEM lectures.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: libigl tutorial examples around cotmatrix and massmatrix.
3. Keep the most useful executable/reference page open while coding: libigl tutorial - <https://libigl.github.io/tutorial/>; Geometry Central - <https://geometry-central.net/>

B. Definitions, results and algorithms to know

- Dirichlet energy
- weak Laplacian
- implicit smoothing stability
- Precision requirement: distinguish and justify uniform vs cotangent Laplacians; explicit vs implicit smoothing; unconstrained vs volume-aware variants. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the cotangent Laplacian from a finite-element or discrete-energy viewpoint and derive mean-curvature flow; explain mass matrices and boundary-condition choices.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use obtuse/skewed meshes and boundaries; inspect shrinkage, sign/maximum-principle issues, spectral drift, and convergence under refinement. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement explicit vs implicit Laplacian smoothing; preserve volume or compare shrinkage. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a path of three vertices at positions [0,1,3], form a simple graph Laplacian and take one explicit smoothing step. Compare with the implicit solve and identify the timestep at which explicit smoothing becomes unstable. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use obtuse/skewed meshes and boundaries; inspect shrinkage, sign/maximum-principle issues, spectral drift, and convergence under refinement. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Compare uniform/cotan and explicit/implicit smoothing while plotting area, volume, spectral energy, and feature shrinkage versus iteration/time.
- P5 - Machine-readable evidence. Export surface error, volume/area shrinkage, spectral attenuation, solver residual/iterations, and stability versus timestep to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: uniform vs cotangent Laplacians; explicit vs implicit smoothing; unconstrained vs volume-aware variants. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use obtuse/skewed meshes and boundaries; inspect shrinkage, sign/maximum-principle issues, spectral drift, and convergence under refinement." without hand-editing code between methods. Record surface error, volume/area shrinkage, spectral attenuation, solver residual/iterations, and stability versus timestep.
- P7 - Deliberate bug and regression drill. Intentionally compare uniform and cotangent Laplacians while omitting the mass matrix in only one method. Before running, predict the first observable failure in surface error, volume/area shrinkage, spectral attenuation, solver residual/iterations, and stability versus timestep. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the cotangent Laplacian from a finite-element or discrete-energy viewpoint and derive mean-curvature flow; explain mass matrices and boundary-condition choices. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a path of three vertices at positions [0,1,3], form a simple graph Laplacian and take one explicit smoothing step. Compare with the implicit solve and identify the timestep at which explicit smoothing becomes unstable. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement explicit vs implicit Laplacian smoothing; preserve volume or compare shrinkage. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Compare uniform/cotan and explicit/implicit smoothing while plotting area, volume, spectral energy, and feature shrinkage versus iteration/time. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare uniform vs cotangent Laplacians; explicit vs implicit smoothing; unconstrained vs volume-aware variants. Use multiple samplings/remeshings of the same underlying continuous shape, with matched vertex/element count or approximation error; keep boundary/topology conditions explicit. Record surface error, volume/area shrinkage, spectral attenuation, solver residual/iterations, and stability versus timestep. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use obtuse/skewed meshes and boundaries; inspect shrinkage, sign/maximum-principle issues, spectral drift, and convergence under refinement. State the predicted failure signature before execution.

Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: libigl differential quantities chapter; DDG Laplacian/FEM lectures. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Laplace-Beltrami and smoothing. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among uniform vs cotangent Laplacians; explicit vs implicit smoothing; unconstrained vs volume-aware variants. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using surface error, volume/area shrinkage, spectral attenuation, solver residual/iterations, and stability versus timestep. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among uniform vs cotangent Laplacians; explicit vs implicit smoothing; unconstrained vs volume-aware variants persists across multiple samplings/resolutions of the same underlying shapes and is explained by a discrete/intrinsic geometric property measured through surface error, volume/area shrinkage, spectral attenuation, solver residual/iterations, and stability versus timestep, not a single mesh idiosyncrasy.

F. Concrete research project - Laplace-Beltrami and smoothing

F1. Falsifiable question. Under the controlled failure regime "Use obtuse/skewed meshes and boundaries; inspect shrinkage, sign/maximum-principle issues, spectral drift, and convergence under refinement.", which alternative in uniform vs cotangent Laplacians; explicit vs implicit smoothing; unconstrained vs volume-aware variants is preferable, and is the change explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Compare uniform/cotan and explicit/implicit smoothing while plotting area, volume, spectral energy, and feature shrinkage versus iteration/time. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Use obtuse/skewed meshes and boundaries; inspect shrinkage, sign/maximum-principle issues, spectral drift, and convergence under refinement.

F4. Required evidence. Measure surface error, volume/area shrinkage, spectral attenuation, solver residual/iterations, and stability versus timestep. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Laplace-Beltrami and smoothing. If a stable failure/crossover is found among uniform vs cotangent Laplacians; explicit vs implicit smoothing; unconstrained vs volume-aware variants, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Laplace-Beltrami and smoothing. Using the measured surface error, volume/area shrinkage, spectral attenuation, solver residual/iterations, and stability versus timestep, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Laplace-Beltrami and smoothing

- G1. Closed-book derivation/oracle. Derive the cotangent Laplacian from a finite-element or discrete-energy viewpoint and derive mean-curvature flow; explain mass matrices and boundary-condition choices. Then solve this exact micro-case from scratch without opening your implementation: For a path of three vertices at positions $[0,1,3]$, form a simple graph Laplacian and take one explicit smoothing step. Compare with the implicit solve and

identify the timestep at which explicit smoothing becomes unstable. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement explicit vs implicit Laplacian smoothing; preserve volume or compare shrinkage. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally compare uniform and cotangent Laplacians while omitting the mass matrix in only one method. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Compare uniform/cotan and explicit/implicit smoothing while plotting area, volume, spectral energy, and feature shrinkage versus iteration/time. Then apply this stress without retuning: Use obtuse/skewed meshes and boundaries; inspect shrinkage, sign/maximum-principle issues, spectral drift, and convergence under refinement. Report surface error, volume/area shrinkage, spectral attenuation, solver residual/iterations, and stability versus timestep and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among uniform vs cotangent Laplacians; explicit vs implicit smoothing; unconstrained vs volume-aware variants for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured surface error, volume/area shrinkage, spectral attenuation, solver residual/iterations, and stability versus timestep; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Laplace-Beltrami and smoothing. Using surface error, volume/area shrinkage, spectral attenuation, solver residual/iterations, and stability versus timestep, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 8, Unit 4. Parameterization and conformal maps

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Harmonic maps, LSCM, angle/area distortion, boundary conditions.

A. Reading order

1. Read the primary assignment first: libigl parameterization chapter; DDG conformal mapping lectures.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: libigl LSCM/harmonic examples.
3. Keep the most useful executable/reference page open while coding: libigl tutorial - <https://libigl.github.io/tutorial/> ; Geometry Central - <https://geometry-central.net/>

B. Definitions, results and algorithms to know

- Dirichlet energy
- conformality
- discrete harmonic functions
- Precision requirement: distinguish and justify harmonic, LSCM/conformal-style, and boundary/seam choices compared by angle distortion, area distortion, flips, and conditioning. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a harmonic/least-squares conformal parameterization energy and its sparse linear system; define angle and area distortion and show how boundary constraints affect the solution.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use long-thin/high-genus shapes and several boundary/seam choices; report flips plus angle/area distortion and conditioning. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Flatten several meshes; quantify angle/area distortion; explore boundary choices. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Map a square mesh of two triangles to the plane with three boundary vertices fixed. Compute angle/area distortion for the identity map and

for a deliberately sheared map; verify what each distortion metric detects. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Use long-thin/high-genus shapes and several boundary/seam choices; report flips plus angle/area distortion and conditioning. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement or call two parameterizers, then export per-triangle angle/area distortion heatmaps, flip count, and solver conditioning for several boundary choices.
- P5 - Machine-readable evidence. Export angle distortion, area distortion, flipped triangles, seam/boundary distortion, solver residual/conditioning to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: harmonic, LSCM/conformal-style, and boundary/seam choices compared by angle distortion, area distortion, flips, and conditioning. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use long-thin/high-genus shapes and several boundary/seam choices; report flips plus angle/area distortion and conditioning." without hand-editing code between methods. Record angle distortion, area distortion, flipped triangles, seam/boundary distortion, solver residual/conditioning.
- P7 - Deliberate bug and regression drill. Intentionally accept UV triangles with negative signed area because only unsigned distortion is reported. Before running, predict the first observable failure in angle distortion, area distortion, flipped triangles, seam/boundary distortion, solver residual/conditioning. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a harmonic/least-squares conformal parameterization energy and its sparse linear system; define angle and area distortion and show how boundary constraints affect the solution. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Map a square mesh of two triangles to the plane with three boundary vertices fixed. Compute angle/area distortion for the identity map and for a deliberately sheared map; verify what each distortion metric detects. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Flatten several meshes; quantify angle/area distortion; explore boundary choices. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement or call two parameterizers, then export per-triangle angle/area distortion heatmaps, flip count, and solver conditioning for several boundary choices. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare harmonic, LSCM/conformal-style, and boundary/seam choices compared by angle distortion, area distortion, flips, and conditioning. Use multiple samplings/remeshings of the same underlying continuous shape, with matched vertex/element count or approximation error; keep boundary/topology conditions explicit. Record angle distortion, area distortion, flipped triangles, seam/boundary distortion, solver residual/conditioning. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use long-thin/high-genus shapes and several boundary/seam choices; report flips plus angle/area distortion and conditioning. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: libigl parameterization chapter; DDG conformal mapping lectures. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Parameterization and conformal maps. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among harmonic, LSCM/conformal-style, and boundary/seam choices compared by angle distortion, area distortion, flips, and conditioning. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using angle distortion, area distortion, flipped triangles, seam/boundary distortion, solver

residual/conditioning. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the boundary/seam/energy choice improves a defined angle-area-flip-conditioning tradeoff on several mesh families; one visually preferred parameterization is insufficient.

F. Concrete research project - Parameterization and conformal maps

F1. Falsifiable question. Under the controlled failure regime "Use long-thin/high-genus shapes and several boundary/seam choices; report flips plus angle/area distortion and conditioning.", characterize the ranking or regime transition across harmonic, LSCM/conformal-style, and boundary/seam choices compared by angle distortion, area distortion, flips, and conditioning. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by boundary/seam constraints, discrete conformal energy, triangle validity/flips, mesh quality, and linear-system conditioning. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement or call two parameterizers, then export per-triangle angle/area distortion heatmaps, flip count, and solver conditioning for several boundary choices. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Use long-thin/high-genus shapes and several boundary/seam choices; report flips plus angle/area distortion and conditioning.

F4. Required evidence. Measure angle distortion, area distortion, flipped triangles, seam/boundary distortion, solver residual/conditioning. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Parameterization and conformal maps. If a stable failure/crossover is found among harmonic, LSCM/conformal-style, and boundary/seam choices compared by angle distortion, area distortion, flips, and conditioning, implement one adaptive or corrected variant that directly targets boundary/seam constraints, discrete conformal energy, triangle validity/flips, mesh quality, and linear-system conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Parameterization and conformal maps. Using the measured angle distortion, area distortion, flipped triangles, seam/boundary distortion, solver residual/conditioning, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Parameterization and conformal maps

- G1. Closed-book derivation/oracle. Derive a harmonic/least-squares conformal parameterization energy and its sparse linear system; define angle and area distortion and show how boundary constraints affect the solution. Then solve this exact micro-case from scratch without opening your implementation: Map a square mesh of two triangles to the plane with three boundary vertices fixed. Compute angle/area distortion for the identity map and for a deliberately sheared map; verify what each distortion metric detects. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Flatten several meshes; quantify angle/area distortion; explore boundary choices. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally accept UV triangles with negative signed area because only unsigned distortion is reported. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement or call two parameterizers, then export per-triangle angle/area distortion heatmaps, flip count, and solver conditioning for several boundary choices. Then apply this stress without retuning: Use long-thin/high-genus shapes and several boundary/seam choices; report flips plus angle/area distortion and conditioning. Report angle distortion, area

distortion, flipped triangles, seam/boundary distortion, solver residual/conditioning and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across harmonic, LSCM/conformal-style, and boundary/seam choices compared by angle distortion, area distortion, flips, and conditioning for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured angle distortion, area distortion, flipped triangles, seam/boundary distortion, solver residual/conditioning; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Parameterization and conformal maps. Using angle distortion, area distortion, flipped triangles, seam/boundary distortion, solver residual/conditioning, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 8, Unit 5. Geodesic distance

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Dijkstra, fast marching intuition, heat method, vector heat method.

A. Reading order

1. Read the primary assignment first: DDG geodesic lectures; geometry-central distance tutorials.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Geometry-central geodesic distance tutorial.
3. Keep the most useful executable/reference page open while coding: libigl tutorial - <https://libigl.github.io/tutorial/> ; Geometry Central - <https://geometry-central.net/>

B. Definitions, results and algorithms to know

- Geodesic equation concept
- heat method pipeline
- Poisson solve
- Precision requirement: distinguish and justify graph Dijkstra, heat method, and exact/analytic references on sphere/flat domains and difficult triangulations. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the Eikonal characterization of geodesic distance and the heat-method pipeline from diffusion to a Poisson solve; analyze the role of the diffusion time parameter.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Compare against exact distances on plane/sphere, then test coarse meshes and narrow handles where graph, heat, and exact methods diverge. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement heat method using sparse solves; compare to graph Dijkstra and ground-truth analytic surfaces. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: On a unit square triangulated by one diagonal, compute graph-edge shortest path between opposite corners and compare with the exact surface geodesic. Use this to predict why edge-graph distance overestimates on coarse meshes. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Compare against exact distances on plane/sphere, then test coarse meshes and narrow handles where graph, heat, and exact methods diverge. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Benchmark Dijkstra and heat method on plane, sphere, and torus; report distance error, convergence with mesh size, and sensitivity to skinny triangles.
- P5 - Machine-readable evidence. Export distance error to analytic/exact reference, convergence rate, runtime, memory, and sensitivity to triangle quality to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.

- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: graph Dijkstra, heat method, and exact/analytic references on sphere/flat domains and difficult triangulations. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Compare against exact distances on plane/sphere, then test coarse meshes and narrow handles where graph, heat, and exact methods diverge." without hand-editing code between methods. Record distance error to analytic/exact reference, convergence rate, runtime, memory, and sensitivity to triangle quality.
- P7 - Deliberate bug and regression drill. Intentionally solve the Poisson stage without fixing the constant nullspace or use an inappropriate heat time scale. Before running, predict the first observable failure in distance error to analytic/exact reference, convergence rate, runtime, memory, and sensitivity to triangle quality. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the Eikonal characterization of geodesic distance and the heat-method pipeline from diffusion to a Poisson solve; analyze the role of the diffusion time parameter. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. On a unit square triangulated by one diagonal, compute graph-edge shortest path between opposite corners and compare with the exact surface geodesic. Use this to predict why edge-graph distance overestimates on coarse meshes. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement heat method using sparse solves; compare to graph Dijkstra and ground-truth analytic surfaces. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Benchmark Dijkstra and heat method on plane, sphere, and torus; report distance error, convergence with mesh size, and sensitivity to skinny triangles. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare graph Dijkstra, heat method, and exact/analytic references on sphere/flat domains and difficult triangulations. Use multiple samplings/remeshings of the same underlying continuous shape, with matched vertex/element count or approximation error; keep boundary/topology conditions explicit. Record distance error to analytic/exact reference, convergence rate, runtime, memory, and sensitivity to triangle quality. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Compare against exact distances on plane/sphere, then test coarse meshes and narrow handles where graph, heat, and exact methods diverge. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: DDG geodesic lectures; geometry-central distance tutorials. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Geodesic distance. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among graph Dijkstra, heat method, and exact/analytic references on sphere/flat domains and difficult triangulations. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using distance error to analytic/exact reference, convergence rate, runtime, memory, and sensitivity to triangle quality. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among graph Dijkstra, heat method, and exact/analytic references on sphere/flat domains and difficult triangulations persists across multiple samplings/resolutions of the same underlying shapes and is explained by a discrete/intrinsic geometric property measured through distance error to analytic/exact reference, convergence rate, runtime, memory, and sensitivity to triangle quality, not a single mesh idiosyncrasy.

F. Concrete research project - Geodesic distance

- F1. Falsifiable question. Under the controlled failure regime "Compare against exact distances on plane/sphere, then test coarse meshes and narrow handles where graph, heat, and exact methods diverge.", characterize the ranking or regime transition across graph Dijkstra, heat method, and exact/analytic references on sphere/flat domains and difficult

triangulations. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Benchmark Dijkstra and heat method on plane, sphere, and torus; report distance error, convergence with mesh size, and sensitivity to skinny triangles. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Compare against exact distances on plane/sphere, then test coarse meshes and narrow handles where graph, heat, and exact methods diverge.

F4. Required evidence. Measure distance error to analytic/exact reference, convergence rate, runtime, memory, and sensitivity to triangle quality. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Geodesic distance. If a stable failure/crossover is found among graph Dijkstra, heat method, and exact/analytic references on sphere/flat domains and difficult triangulations, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Geodesic distance. Using the measured distance error to analytic/exact reference, convergence rate, runtime, memory, and sensitivity to triangle quality, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Geodesic distance

- G1. Closed-book derivation/oracle. Derive the Eikonal characterization of geodesic distance and the heat-method pipeline from diffusion to a Poisson solve; analyze the role of the diffusion time parameter. Then solve this exact micro-case from scratch without opening your implementation: On a unit square triangulated by one diagonal, compute graph-edge shortest path between opposite corners and compare with the exact surface geodesic. Use this to predict why edge-graph distance overestimates on coarse meshes. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement heat method using sparse solves; compare to graph Dijkstra and ground-truth analytic surfaces. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally solve the Poisson stage without fixing the constant nullspace or use an inappropriate heat time scale. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Benchmark Dijkstra and heat method on plane, sphere, and torus; report distance error, convergence with mesh size, and sensitivity to skinny triangles. Then apply this stress without retuning: Compare against exact distances on plane/sphere, then test coarse meshes and narrow handles where graph, heat, and exact methods diverge. Report distance error to analytic/exact reference, convergence rate, runtime, memory, and sensitivity to triangle quality and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across graph Dijkstra, heat method, and exact/analytic references on sphere/flat domains and difficult triangulations for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured distance error to analytic/exact reference, convergence rate, runtime, memory, and sensitivity to triangle quality; state one counterexample that would change the diagnosis or ranking.

- G6. Pass condition for Geodesic distance. Using distance error to analytic/exact reference, convergence rate, runtime, memory, and sensitivity to triangle quality, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 8, Unit 6. Mesh deformation and editing

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Laplacian editing, ARAP, skinning-related deformation, constraints.

A. Reading order

1. Read the primary assignment first: libigl deformation tutorial; geometry processing literature.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: libigl ARAP example.
3. Keep the most useful executable/reference page open while coding: libigl tutorial - <https://libigl.github.io/tutorial/> ; Geometry Central - <https://geometry-central.net/>

B. Definitions, results and algorithms to know

- Local-global optimization
- rotation fitting via polar/SVD
- Precision requirement: distinguish and justify Laplacian editing vs ARAP under large handle motion, rotation, sparse constraints, and poor triangulations. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive Laplacian editing or ARAP local-global optimization, including the per-vertex rotation fit and global linear solve; identify nullspaces and constraint requirements.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Apply large/conflicting handle motions to thin and concave meshes; quantify rigidity distortion, constraint residual, and foldovers. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement ARAP deformation; visualize per-iteration energy; compare to Laplacian editing. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a square of two triangles, pin two left vertices and move the top-right vertex. Write one local-global ARAP iteration: local rotations from current edge vectors and the corresponding global linear system structure. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Apply large/conflicting handle motions to thin and concave meshes; quantify rigidity distortion, constraint residual, and foldovers. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement ARAP local-global iterations with energy logging; compare against Laplacian editing under large rotations and sparse handles.
- P5 - Machine-readable evidence. Export constraint error, ARAP/Laplacian energy, local distortion, iteration convergence, and robustness under large rotations to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: Laplacian editing vs ARAP under large handle motion, rotation, sparse constraints, and poor triangulations. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Apply large/conflicting handle motions to thin and concave meshes; quantify rigidity distortion, constraint residual, and foldovers." without hand-editing code between methods. Record constraint error, ARAP/Laplacian energy, local distortion, iteration convergence, and robustness under large rotations.
- P7 - Deliberate bug and regression drill. Intentionally use the raw least-squares local deformation gradient instead of projecting it to $SO(3)$ in ARAP. Before running, predict the first observable failure in constraint error, ARAP/Laplacian energy, local distortion, iteration convergence, and robustness under large rotations. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive Laplacian editing or ARAP local-global optimization, including the per-vertex rotation fit and global linear solve; identify nullspaces and constraint requirements. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a square of two triangles, pin two left vertices and move the top-right vertex. Write one local-global ARAP iteration: local rotations from current edge vectors and the corresponding global linear system structure. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement ARAP deformation; visualize per-iteration energy; compare to Laplacian editing. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement ARAP local-global iterations with energy logging; compare against Laplacian editing under large rotations and sparse handles. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare Laplacian editing vs ARAP under large handle motion, rotation, sparse constraints, and poor triangulations. Use multiple samplings/remeshings of the same underlying continuous shape, with matched vertex/element count or approximation error; keep boundary/topology conditions explicit. Record constraint error, ARAP/Laplacian energy, local distortion, iteration convergence, and robustness under large rotations. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Apply large/conflicting handle motions to thin and concave meshes; quantify rigidity distortion, constraint residual, and foldovers. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: libigl deformation tutorial; geometry processing literature. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Mesh deformation and editing. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among Laplacian editing vs ARAP under large handle motion, rotation, sparse constraints, and poor triangulations. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using constraint error, ARAP/Laplacian energy, local distortion, iteration convergence, and robustness under large rotations. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among Laplacian editing vs ARAP under large handle motion, rotation, sparse constraints, and poor triangulations persists across multiple samplings/resolutions of the same underlying shapes and is explained by a discrete/intrinsic geometric property measured through constraint error, ARAP/Laplacian energy, local distortion, iteration convergence, and robustness under large rotations, not a single mesh idiosyncrasy.

F. Concrete research project - Mesh deformation and editing

- F1. Falsifiable question. Under the controlled failure regime "Apply large/conflicting handle motions to thin and concave meshes; quantify rigidity distortion, constraint residual, and foldovers.", which alternative in Laplacian editing vs ARAP under large handle motion, rotation, sparse constraints, and poor triangulations is preferable, and is the change explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement ARAP local-global iterations with energy logging; compare against Laplacian editing under large rotations and sparse handles. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Apply large/conflicting handle motions to thin and concave meshes; quantify rigidity distortion, constraint residual, and foldovers.

F4. Required evidence. Measure constraint error, ARAP/Laplacian energy, local distortion, iteration convergence, and robustness under large rotations. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Mesh deformation and editing. If a stable failure/crossover is found among Laplacian editing vs ARAP under large handle motion, rotation, sparse constraints, and poor triangulations, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Mesh deformation and editing. Using the measured constraint error, ARAP/Laplacian energy, local distortion, iteration convergence, and robustness under large rotations, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Mesh deformation and editing

- G1. Closed-book derivation/oracle. Derive Laplacian editing or ARAP local-global optimization, including the per-vertex rotation fit and global linear solve; identify nullspaces and constraint requirements. Then solve this exact micro-case from scratch without opening your implementation: For a square of two triangles, pin two left vertices and move the top-right vertex. Write one local-global ARAP iteration: local rotations from current edge vectors and the corresponding global linear system structure. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement ARAP deformation; visualize per-iteration energy; compare to Laplacian editing. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use the raw least-squares local deformation gradient instead of projecting it to $SO(3)$ in ARAP. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement ARAP local-global iterations with energy logging; compare against Laplacian editing under large rotations and sparse handles. Then apply this stress without retuning: Apply large/conflicting handle motions to thin and concave meshes; quantify rigidity distortion, constraint residual, and foldovers. Report constraint error, ARAP/Laplacian energy, local distortion, iteration convergence, and robustness under large rotations and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among Laplacian editing vs ARAP under large handle motion, rotation, sparse constraints, and poor triangulations for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured constraint error, ARAP/Laplacian energy, local distortion, iteration convergence, and robustness under large rotations; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Mesh deformation and editing. Using constraint error, ARAP/Laplacian energy, local distortion, iteration convergence, and robustness under large rotations, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 8, Unit 7. Remeshing and intrinsic triangulations

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Edge split/collapse/flip, Delaunay triangulations, mesh quality, intrinsic methods.

A. Reading order

1. Read the primary assignment first: geometry-central intrinsic triangulation tutorials; libigl remeshing material.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Geometry-central tutorials.

3. Keep the most useful executable/reference page open while coding: libigl tutorial - <https://libigl.github.io/tutorial/> ; Geometry Central - <https://geometry-central.net/>

B. Definitions, results and algorithms to know

- Delaunay condition
- local mesh operators
- invariants under operations
- Precision requirement: distinguish and justify extrinsic isotropic remeshing vs intrinsic Delaunay/edge-flip strategies, tracking angle quality and downstream operator error. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive quality measures for triangle shape, local edge-flip criteria, and the intrinsic Delaunay condition; reason about how split/collapse/flip operations affect approximation and topology.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Represent the same smooth surface with several intentionally poor triangulations; test whether intrinsic/remeshed downstream results become triangulation-insensitive. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement isotropic remeshing subset or use library and study quality metrics under operations. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Take two triangles forming a convex quadrilateral with a very poor diagonal. Compute the two opposite angle sums and decide whether an intrinsic Delaunay edge flip should occur; compare minimum angles before/after. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Represent the same smooth surface with several intentionally poor triangulations; test whether intrinsic/remeshed downstream results become triangulation-insensitive. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Run edge split/collapse/flip/smooth operations while logging min angle, edge-length variance, Hausdorff distance, and cotan-Laplacian quality.
- P5 - Machine-readable evidence. Export min-angle/quality distribution, Hausdorff distance, edge-length variance, topology preservation, and downstream operator error to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: extrinsic isotropic remeshing vs intrinsic Delaunay/edge-flip strategies, tracking angle quality and downstream operator error. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Represent the same smooth surface with several intentionally poor triangulations; test whether intrinsic/remeshed downstream results become triangulation-insensitive." without hand-editing code between methods. Record min-angle/quality distribution, Hausdorff distance, edge-length variance, topology preservation, and downstream operator error.
- P7 - Deliberate bug and regression drill. Intentionally collapse an edge without checking link/manifold/orientation conditions. Before running, predict the first observable failure in min-angle/quality distribution, Hausdorff distance, edge-length variance, topology preservation, and downstream operator error. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive quality measures for triangle shape, local edge-flip criteria, and the intrinsic Delaunay condition; reason about how split/collapse/flip operations affect approximation and topology. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Take two triangles forming a convex quadrilateral with a very poor diagonal. Compute the two opposite angle sums and decide whether an intrinsic Delaunay edge flip should occur; compare minimum angles before/after. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Implement isotropic remeshing subset or use library and study quality metrics under operations. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Run edge split/collapse/flip/smooth operations while logging min angle, edge-length variance, Hausdorff distance, and cotan-Laplacian quality. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare extrinsic isotropic remeshing vs intrinsic Delaunay/edge-flip strategies, tracking angle quality and downstream operator error. Use multiple samplings/remeshings of the same underlying continuous shape, with matched vertex/element count or approximation error; keep boundary/topology conditions explicit. Record min-angle/quality distribution, Hausdorff distance, edge-length variance, topology preservation, and downstream operator error. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Represent the same smooth surface with several intentionally poor triangulations; test whether intrinsic/remeshed downstream results become triangulation-insensitive. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: geometry-central intrinsic triangulation tutorials; libigl remeshing material. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Remeshing and intrinsic triangulations. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among extrinsic isotropic remeshing vs intrinsic Delaunay/edge-flip strategies, tracking angle quality and downstream operator error. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using min-angle/quality distribution, Hausdorff distance, edge-length variance, topology preservation, and downstream operator error. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among extrinsic isotropic remeshing vs intrinsic Delaunay/edge-flip strategies, tracking angle quality and downstream operator error persists across multiple samplings/resolutions of the same underlying shapes and is explained by a discrete/intrinsic geometric property measured through min-angle/quality distribution, Hausdorff distance, edge-length variance, topology preservation, and downstream operator error, not a single mesh idiosyncrasy.

F. Concrete research project - Remeshing and intrinsic triangulations

- F1. Falsifiable question. Under the controlled failure regime "Represent the same smooth surface with several intentionally poor triangulations; test whether intrinsic/remeshed downstream results become triangulation-insensitive.", which alternative in extrinsic isotropic remeshing vs intrinsic Delaunay/edge-flip strategies, tracking angle quality and downstream operator error is preferable, and is the change explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Run edge split/collapse/flip/smooth operations while logging min angle, edge-length variance, Hausdorff distance, and cotan-Laplacian quality. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Represent the same smooth surface with several intentionally poor triangulations; test whether intrinsic/remeshed downstream results become triangulation-insensitive.
- F4. Required evidence. Measure min-angle/quality distribution, Hausdorff distance, edge-length variance, topology preservation, and downstream operator error. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Remeshing and intrinsic triangulations. If a stable failure/crossover is found among extrinsic isotropic remeshing vs intrinsic Delaunay/edge-flip strategies, tracking angle quality and downstream operator error, implement one adaptive or corrected variant that directly targets representation invariants, mesh

quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Remeshing and intrinsic triangulations. Using the measured min-angle/quality distribution, Hausdorff distance, edge-length variance, topology preservation, and downstream operator error, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Remeshing and intrinsic triangulations

- G1. Closed-book derivation/oracle. Derive quality measures for triangle shape, local edge-flip criteria, and the intrinsic Delaunay condition; reason about how split/collapse/flip operations affect approximation and topology. Then solve this exact micro-case from scratch without opening your implementation: Take two triangles forming a convex quadrilateral with a very poor diagonal. Compute the two opposite angle sums and decide whether an intrinsic Delaunay edge flip should occur; compare minimum angles before/after. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement isotropic remeshing subset or use library and study quality metrics under operations. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally collapse an edge without checking link/manifold/orientation conditions. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Run edge split/collapse/flip/smooth operations while logging min angle, edge-length variance, Hausdorff distance, and cotan-Laplacian quality. Then apply this stress without retuning: Represent the same smooth surface with several intentionally poor triangulations; test whether intrinsic/remeshed downstream results become triangulation-insensitive. Report min-angle/quality distribution, Hausdorff distance, edge-length variance, topology preservation, and downstream operator error and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among extrinsic isotropic remeshing vs intrinsic Delaunay/edge-flip strategies, tracking angle quality and downstream operator error for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured min-angle/quality distribution, Hausdorff distance, edge-length variance, topology preservation, and downstream operator error; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Remeshing and intrinsic triangulations. Using min-angle/quality distribution, Hausdorff distance, edge-length variance, topology preservation, and downstream operator error, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 8, Unit 8. Topology and Hodge decomposition

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Homology intuition, cycles, harmonic forms, vector fields.

A. Reading order

1. Read the primary assignment first: DDG homology/cohomology and Helmholtz-Hodge lectures.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CMU DDG videos.
3. Keep the most useful executable/reference page open while coding: libigl tutorial - <https://libigl.github.io/tutorial/>; Geometry Central - <https://geometry-central.net/>

B. Definitions, results and algorithms to know

- Euler-Poincare relation

- exact/coexact/harmonic decomposition
- Precision requirement: distinguish and justify gradient/curl/harmonic decomposition on simply connected and genus-one meshes with reconstruction and orthogonality residuals. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the relationship among Euler characteristic, genus, and Betti numbers for orientable surfaces, then write the discrete Hodge decomposition of a vector/1-form field into exact, coexact, and harmonic parts.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Add vector-field noise and small handles/holes; verify harmonic dimension and reconstruction residuals. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Compute a harmonic 1-form or vector-field decomposition on torus-like meshes using course scaffold/library. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: On a four-edge cycle graph, assign unit circulation around the loop. Show it has zero divergence but nonzero circulation and explain why it cannot be represented as a pure gradient on the cycle. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Add vector-field noise and small handles/holes; verify harmonic dimension and reconstruction residuals. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. On a torus, construct a known harmonic circulation and verify recovered exact/coexact/harmonic parts, reconstruction residual, and pairwise inner products.
- P5 - Machine-readable evidence. Export Betti/genus checks, reconstruction residual, exact/coexact/harmonic orthogonality, and circulation/flux error to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: gradient/curl/harmonic decomposition on simply connected and genus-one meshes with reconstruction and orthogonality residuals. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Add vector-field noise and small handles/holes; verify harmonic dimension and reconstruction residuals." without hand-editing code between methods. Record Betti/genus checks, reconstruction residual, exact/coexact/harmonic orthogonality, and circulation/flux error.
- P7 - Deliberate bug and regression drill. Intentionally use inconsistent boundary/orientation conventions so the recovered harmonic dimension is wrong. Before running, predict the first observable failure in Betti/genus checks, reconstruction residual, exact/coexact/harmonic orthogonality, and circulation/flux error. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the relationship among Euler characteristic, genus, and Betti numbers for orientable surfaces, then write the discrete Hodge decomposition of a vector/1-form field into exact, coexact, and harmonic parts. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. On a four-edge cycle graph, assign unit circulation around the loop. Show it has zero divergence but nonzero circulation and explain why it cannot be represented as a pure gradient on the cycle. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Compute a harmonic 1-form or vector-field decomposition on torus-like meshes using course scaffold/library. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: On a torus, construct a known harmonic circulation and verify recovered exact/coexact/harmonic parts, reconstruction residual, and pairwise inner products. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare gradient/curl/harmonic decomposition on simply connected and genus-one meshes with reconstruction and orthogonality residuals. Use multiple samplings/remeshings of the same underlying continuous shape, with matched vertex/element count or approximation error; keep boundary/topology conditions

explicit. Record Betti/genus checks, reconstruction residual, exact/coexact/harmonic orthogonality, and circulation/flux error. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Add vector-field noise and small handles/holes; verify harmonic dimension and reconstruction residuals. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: DDG homology/cohomology and Helmholtz-Hodge lectures. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Topology and Hodge decomposition. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among gradient/curl/harmonic decomposition on simply connected and genus-one meshes with reconstruction and orthogonality residuals. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using Betti/genus checks, reconstruction residual, exact/coexact/harmonic orthogonality, and circulation/flux error. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among gradient/curl/harmonic decomposition on simply connected and genus-one meshes with reconstruction and orthogonality residuals persists across multiple samplings/resolutions of the same underlying shapes and is explained by a discrete/intrinsic geometric property measured through Betti/genus checks, reconstruction residual, exact/coexact/harmonic orthogonality, and circulation/flux error, not a single mesh idiosyncrasy.

F. Concrete research project - Topology and Hodge decomposition

F1. Falsifiable question. Under the controlled failure regime "Add vector-field noise and small handles/holes; verify harmonic dimension and reconstruction residuals.", characterize the ranking or regime transition across gradient/curl/harmonic decomposition on simply connected and genus-one meshes with reconstruction and orthogonality residuals. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by conditioning, local linearization, and loss/parameter scaling. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: On a torus, construct a known harmonic circulation and verify recovered exact/coexact/harmonic parts, reconstruction residual, and pairwise inner products. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Add vector-field noise and small handles/holes; verify harmonic dimension and reconstruction residuals.

F4. Required evidence. Measure Betti/genus checks, reconstruction residual, exact/coexact/harmonic orthogonality, and circulation/flux error. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Topology and Hodge decomposition. If a stable failure/crossover is found among gradient/curl/harmonic decomposition on simply connected and genus-one meshes with reconstruction and orthogonality residuals, implement one adaptive or corrected variant that directly targets conditioning, local linearization, and loss/parameter scaling; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Topology and Hodge decomposition. Using the measured Betti/genus checks, reconstruction residual, exact/coexact/harmonic orthogonality, and circulation/flux error, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Topology and Hodge decomposition

- G1. Closed-book derivation/oracle. Derive the relationship among Euler characteristic, genus, and Betti numbers for orientable surfaces, then write the discrete Hodge decomposition of a vector/1-form field into exact, coexact, and harmonic parts. Then solve this exact micro-case from scratch without opening your implementation: On a four-edge cycle graph, assign unit circulation around the loop. Show it has zero divergence but nonzero circulation and explain why it cannot be represented as a pure gradient on the cycle. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Compute a harmonic 1-form or vector-field decomposition on torus-like meshes using course scaffold/library. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use inconsistent boundary/orientation conventions so the recovered harmonic dimension is wrong. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: On a torus, construct a known harmonic circulation and verify recovered exact/coexact/harmonic parts, reconstruction residual, and pairwise inner products. Then apply this stress without retuning: Add vector-field noise and small handles/holes; verify harmonic dimension and reconstruction residuals. Report Betti/genus checks, reconstruction residual, exact/coexact/harmonic orthogonality, and circulation/flux error and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across gradient/curl/harmonic decomposition on simply connected and genus-one meshes with reconstruction and orthogonality residuals for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured Betti/genus checks, reconstruction residual, exact/coexact/harmonic orthogonality, and circulation/flux error; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Topology and Hodge decomposition. Using Betti/genus checks, reconstruction residual, exact/coexact/harmonic orthogonality, and circulation/flux error, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 8, Unit 9. Robust geometry research practices

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Degeneracies, nonmanifold/noisy inputs, exact predicates, intrinsic vs extrinsic formulations.

A. Reading order

1. Read the primary assignment first: Read Shewchuk, Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates (1997), then the current CGAL Polygon Mesh Processing repair/intersection documentation. Use one SGP/TOG paper in your chosen robust-geometry subproblem only after these failure modes are reproducible.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Reproduce one geometry-central/libigl research demo.
3. Keep the most useful executable/reference page open while coding: libigl tutorial - <https://libigl.github.io/tutorial/> ; Geometry Central - <https://geometry-central.net/>

B. Definitions, results and algorithms to know

- Topological invariants
- conditioning
- robust predicate philosophy
- Precision requirement: distinguish and justify two chosen geometry algorithms evaluated across a labeled corpus of nonmanifold, degenerate, self-intersecting, disconnected, and extreme-scale meshes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- For one geometric algorithm, derive the exact predicate/assumption chain that decides success; separate combinatorial correctness, numerical predicate correctness, and approximation error.

- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use a benchmark matrix including nonmanifold edges, duplicates, self-intersections, zero-area faces, disconnected components, and extreme scales. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create adversarial mesh corpus: skinny triangles, duplicated vertices, holes, self-intersections; characterize failure modes of two algorithms. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Construct a triangle with vertices $(0,0),(1,0),(1,10^{-12})$ and a query point near its longest edge. Predict failures in area, barycentric, orientation, and normalization computations under rescaling by 10^8 . The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use a benchmark matrix including nonmanifold edges, duplicates, self-intersections, zero-area faces, disconnected components, and extreme scales. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create a JSON-labeled adversarial mesh suite and an automated runner that records success/failure type, exception, output validity, and metric degradation for two algorithms.
- P5 - Machine-readable evidence. Export success rate by defect class, wrong-output rate, crash/exception rate, metric degradation, and runtime on the adversarial corpus to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: two chosen geometry algorithms evaluated across a labeled corpus of nonmanifold, degenerate, self-intersecting, disconnected, and extreme-scale meshes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use a benchmark matrix including nonmanifold edges, duplicates, self-intersections, zero-area faces, disconnected components, and extreme scales." without hand-editing code between methods. Record success rate by defect class, wrong-output rate, crash/exception rate, metric degradation, and runtime on the adversarial corpus.
- P7 - Deliberate bug and regression drill. Intentionally replace an exact/adaptive predicate with a fixed epsilon that is not scale aware. Before running, predict the first observable failure in success rate by defect class, wrong-output rate, crash/exception rate, metric degradation, and runtime on the adversarial corpus. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. For one geometric algorithm, derive the exact predicate/assumption chain that decides success; separate combinatorial correctness, numerical predicate correctness, and approximation error. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Construct a triangle with vertices $(0,0),(1,0),(1,10^{-12})$ and a query point near its longest edge. Predict failures in area, barycentric, orientation, and normalization computations under rescaling by 10^8 . Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Create adversarial mesh corpus: skinny triangles, duplicated vertices, holes, self-intersections; characterize failure modes of two algorithms. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create a JSON-labeled adversarial mesh suite and an automated runner that records success/failure type, exception, output validity, and metric degradation for two algorithms. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare two chosen geometry algorithms evaluated across a labeled corpus of nonmanifold, degenerate, self-intersecting, disconnected, and extreme-scale meshes. Use multiple samplings/remeshings of the same underlying continuous shape, with matched vertex/element count or approximation error; keep boundary/topology conditions explicit. Record success rate by defect class, wrong-output rate, crash/exception rate, metric degradation, and runtime on the adversarial corpus. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use a benchmark matrix including nonmanifold edges, duplicates, self-intersections, zero-area faces, disconnected components, and extreme scales. State the predicted failure signature before execution.

Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Study modern geometry papers; SIGGRAPH/SGP proceedings. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Robust geometry research practices. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among two chosen geometry algorithms evaluated across a labeled corpus of nonmanifold, degenerate, self-intersecting, disconnected, and extreme-scale meshes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using success rate by defect class, wrong-output rate, crash/exception rate, metric degradation, and runtime on the adversarial corpus. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ranking among two chosen geometry algorithms evaluated across a labeled corpus of nonmanifold, degenerate, self-intersecting, disconnected, and extreme-scale meshes persists across multiple samplings/resolutions of the same underlying shapes and is explained by a discrete/intrinsic geometric property measured through success rate by defect class, wrong-output rate, crash/exception rate, metric degradation, and runtime on the adversarial corpus, not a single mesh idiosyncrasy.

F. Concrete research project - Robust geometry research practices

F1. Falsifiable question. Under the controlled failure regime "Use a benchmark matrix including nonmanifold edges, duplicates, self-intersections, zero-area faces, disconnected components, and extreme scales.", characterize the ranking or regime transition across two chosen geometry algorithms evaluated across a labeled corpus of nonmanifold, degenerate, self-intersecting, disconnected, and extreme-scale meshes. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by representation invariants, mesh quality/topology, discretization error, and solver conditioning. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create a JSON-labeled adversarial mesh suite and an automated runner that records success/failure type, exception, output validity, and metric degradation for two algorithms. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one analytic/simple shape, one irregular real mesh, and one deliberately pathological mesh; run at least three mesh/control resolutions and preserve the worst invalid/degenerate case as a regression asset. The central adversarial condition is: Use a benchmark matrix including nonmanifold edges, duplicates, self-intersections, zero-area faces, disconnected components, and extreme scales.

F4. Required evidence. Measure success rate by defect class, wrong-output rate, crash/exception rate, metric degradation, and runtime on the adversarial corpus. Deliver input/output meshes or control data, invariant/validity checks, multi-resolution metrics, a failure mesh, and scripts that regenerate the principal convergence/distortion plot. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Robust geometry research practices. If a stable failure/crossover is found among two chosen geometry algorithms evaluated across a labeled corpus of nonmanifold, degenerate, self-intersecting, disconnected, and extreme-scale meshes, implement one adaptive or corrected variant that directly targets representation invariants, mesh quality/topology, discretization error, and solver conditioning; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Robust geometry research practices. Using the measured success rate by defect class, wrong-output rate, crash/exception rate, metric degradation, and runtime on the adversarial corpus, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Robust geometry research practices

- G1. Closed-book derivation/oracle. For one geometric algorithm, derive the exact predicate/assumption chain that decides success; separate combinatorial correctness, numerical predicate correctness, and approximation error.

Then solve this exact micro-case from scratch without opening your implementation: Construct a triangle with vertices $(0,0), (1,0), (1,10^{-12})$ and a query point near its longest edge. Predict failures in area, barycentric, orientation, and normalization computations under rescaling by 10^8 . You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create adversarial mesh corpus: skinny triangles, duplicated vertices, holes, self-intersections; characterize failure modes of two algorithms. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally replace an exact/adaptive predicate with a fixed epsilon that is not scale aware. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create a JSON-labeled adversarial mesh suite and an automated runner that records success/failure type, exception, output validity, and metric degradation for two algorithms. Then apply this stress without retuning: Use a benchmark matrix including nonmanifold edges, duplicates, self-intersections, zero-area faces, disconnected components, and extreme scales. Report success rate by defect class, wrong-output rate, crash/exception rate, metric degradation, and runtime on the adversarial corpus and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across two chosen geometry algorithms evaluated across a labeled corpus of nonmanifold, degenerate, self-intersecting, disconnected, and extreme-scale meshes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured success rate by defect class, wrong-output rate, crash/exception rate, metric degradation, and runtime on the adversarial corpus; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Robust geometry research practices. Using success rate by defect class, wrong-output rate, crash/exception rate, metric degradation, and runtime on the adversarial corpus, Pass when the exact/topological/geometric invariants relevant to the unit hold on the oracle case, convergence or distortion behaves correctly under refinement, and the pathological mesh failure is correctly diagnosed. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 8 Closeout: Project, Review and Mastery Gate

Volume project. Build a mesh-processing workbench with curvature, cotan Laplacian, smoothing, parameterization, heat geodesics, deformation, intrinsic/remeshing experiments and a Hodge/vector-field example. Include adversarial meshes.

Cumulative review

- Re-derive cotangent Laplacian/FEM ingredients plus one conformal/geodesic/deformation method and verify sign/convention choices.
- Run refinement studies on analytic shapes and robustness tests on irregular/nonmanifold/degenerate meshes.
- Compare at least two discretizations or boundary/energy choices with distortion/convergence/validity metrics.
- Build an adversarial mesh corpus and preserve failures as regression cases.
- Write a geometry-research synthesis separating continuous theorem, discrete operator, numerical solver and mesh-quality effects.

Mastery gate before moving on

- Implement and convergence-test a cotan Laplacian, one parameterization, one geodesic method, and one deformation method.
- Verify at least one DEC/topological identity numerically and test downstream invariance under remeshing/poor triangulation.
- Maintain an adversarial geometry suite and a failure matrix by defect class.
- Literature gate: reconstruct one discrete operator from Meyer et al., one parameterization/deformation/geodesic method, and one robustness argument; compare continuous identities with their mesh-dependent discrete realization.

Volume 9. Animation, Kinematics, and Character Motion

Purpose. Understand how motion is represented, authored, interpolated, constrained, and controlled before physics enters.

Suggested duration. 12-16 weeks

Prerequisites. Volume 1 transforms/quaternions; programming from Volume 2.

Typical module pace. 1.7-2.3 weeks per module (7 modules total).

Primary references for this volume

- Fundamentals of Computer Graphics, Ch. 16.
- Stanford CS348C readings and notes.
- Classic animation/character papers plus current motion literature.

Online spine

Berkeley CS184/284A Spring 2026 course: <https://cs184.eecs.berkeley.edu/sp26/>

Stanford CS348C Animation and Simulation: <https://graphics.stanford.edu/courses/cs348c/>

Canonical paper track

- Shoemake (1985), Animating Rotation with Quaternion Curves.
- Kavan et al. (2008), Geometric Skinning with Approximate Dual Quaternion Blending.
- Kovar, Gleicher & Pighin (2002), Motion Graphs.
- Holden et al. and later data-driven motion papers as bridge to Volume 15.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Shoemake (1985), Animating Rotation with Quaternion Curves.
- Kovar, Gleicher & Pighin (2002), Motion Graphs.
- Kavan et al. (2008), Geometric Skinning with Approximate Dual Quaternion Blending.
- Treuille, Cooper & Popović (2006), Continuum Crowds.
- Holden et al. (2017), Phase-Functioned Neural Networks for Character Control - a modern contrast to classical motion synthesis.
- Use Berkeley CS184/animation lectures for implementation exercises; treat learned motion as a specialization after classical kinematics and skinning.

Volume 9, Unit 1. Principles and keyframing

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. Timing, spacing, anticipation, arcs, squash/stretch, pose-to-pose, curves.

A. Reading order

1. Read the primary assignment first: FCG Ch. 16.1-16.2; Stanford CS348C traditional animation materials.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 animation lectures.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Animation curves
- interpolation continuity
- ease functions

- Precision requirement: distinguish and justify linear, cubic Hermite/Catmull-Rom, and eased interpolation judged by continuity, overshoot, controllability, and timing intent. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive interpolation curves for position and scalar animation parameters, compare linear, cubic Hermite, and spline timing, and analyze velocity/acceleration continuity.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use sparse keyframes across rapid changes and pauses; compare overshoot, timing, and editability under several interpolants. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Build keyframe animation system with cubic curves and editable tangents. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: With scalar keys $x(0)=0$, $x(1)=1$, $x(2)=0$, compute piecewise linear values at $t=0.5, 1.5$ and one cubic/Hermite alternative with chosen endpoint tangents; compare velocity continuity. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use sparse keyframes across rapid changes and pauses; compare overshoot, timing, and editability under several interpolants. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement editable cubic animation curves with tangent modes and a plot of position/velocity/acceleration; include an overshoot and timing-control test.
- P5 - Machine-readable evidence. Export C0/C1/C2 continuity, overshoot, velocity/acceleration extrema, edit complexity, and timing-target error to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: linear, cubic Hermite/Catmull-Rom, and eased interpolation judged by continuity, overshoot, controllability, and timing intent. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use sparse keyframes across rapid changes and pauses; compare overshoot, timing, and editability under several interpolants." without hand-editing code between methods. Record C0/C1/C2 continuity, overshoot, velocity/acceleration extrema, edit complexity, and timing-target error.
- P7 - Deliberate bug and regression drill. Intentionally interpolate rotations as Euler-angle components and mistake a representation artifact for intended ease-in/ease-out. Before running, predict the first observable failure in C0/C1/C2 continuity, overshoot, velocity/acceleration extrema, edit complexity, and timing-target error. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive interpolation curves for position and scalar animation parameters, compare linear, cubic Hermite, and spline timing, and analyze velocity/acceleration continuity. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. With scalar keys $x(0)=0$, $x(1)=1$, $x(2)=0$, compute piecewise linear values at $t=0.5, 1.5$ and one cubic/Hermite alternative with chosen endpoint tangents; compare velocity continuity. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Build keyframe animation system with cubic curves and editable tangents. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement editable cubic animation curves with tangent modes and a plot of position/velocity/acceleration; include an overshoot and timing-control test. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare linear, cubic Hermite/Catmull-Rom, and eased interpolation judged by continuity, overshoot, controllability, and timing intent. Use the same skeleton, motion clips, keyframes, duration and control targets; compare trajectory/contact errors and, when perception matters, use the same viewer protocol. Record C0/C1/C2 continuity, overshoot, velocity/acceleration extrema, edit complexity, and timing-target error. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use sparse keyframes across rapid changes and pauses; compare overshoot, timing, and editability under several interpolants. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 16.1-16.2; Stanford CS348C traditional animation materials. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Principles and keyframing. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among linear, cubic Hermite/Catmull-Rom, and eased interpolation judged by continuity, overshoot, controllability, and timing intent. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using C0/C1/C2 continuity, overshoot, velocity/acceleration extrema, edit complexity, and timing-target error. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if linear, cubic Hermite/Catmull-Rom, and eased interpolation judged by continuity, overshoot, controllability, and timing intent improve C0/C1/C2 continuity, overshoot, velocity/acceleration extrema, edit complexity, and timing-target error across multiple motions/targets without trading the claimed control or plausibility for hidden contact, retargeting or temporal artifacts.

F. Concrete research project - Principles and keyframing

- F1. Falsifiable question. Under the controlled failure regime "Use sparse keyframes across rapid changes and pauses; compare overshoot, timing, and editability under several interpolants.", characterize the ranking or regime transition across linear, cubic Hermite/Catmull-Rom, and eased interpolation judged by continuity, overshoot, controllability, and timing intent. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement editable cubic animation curves with tangent modes and a plot of position/velocity/acceleration; include an overshoot and timing-control test. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use one hand-computable 2-3 joint/motion case, one medium articulated example, and one contact/retarget stress case. Save synchronized numerical traces with the output animation. The central adversarial condition is: Use sparse keyframes across rapid changes and pauses; compare overshoot, timing, and editability under several interpolants.
- F4. Required evidence. Measure C0/C1/C2 continuity, overshoot, velocity/acceleration extrema, edit complexity, and timing-target error. Deliver source motion/rig, generated motion/video, synchronized joint/contact/trajectory CSV, and one difficult transfer case with a short failure explanation. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Principles and keyframing. If a stable failure/crossover is found among linear, cubic Hermite/Catmull-Rom, and eased interpolation judged by continuity, overshoot, controllability, and timing intent, implement one adaptive or corrected variant that directly targets coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Principles and keyframing. Using the measured C0/C1/C2 continuity, overshoot, velocity/acceleration extrema, edit complexity, and timing-target error, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Principles and keyframing

- G1. Closed-book derivation/oracle. Derive interpolation curves for position and scalar animation parameters, compare linear, cubic Hermite, and spline timing, and analyze velocity/acceleration continuity. Then solve this exact micro-case from scratch without opening your implementation: With scalar keys $x(0)=0$, $x(1)=1$, $x(2)=0$,

compute piecewise linear values at $t=0.5, 1.5$ and one cubic/Hermite alternative with chosen endpoint tangents; compare velocity continuity. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Build keyframe animation system with cubic curves and editable tangents. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally interpolate rotations as Euler-angle components and mistake a representation artifact for intended ease-in/ease-out. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement editable cubic animation curves with tangent modes and a plot of position/velocity/acceleration; include an overshoot and timing-control test. Then apply this stress without retuning: Use sparse keyframes across rapid changes and pauses; compare overshoot, timing, and editability under several interpolants. Report C0/C1/C2 continuity, overshoot, velocity/acceleration extrema, edit complexity, and timing-target error and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across linear, cubic Hermite/Catmull-Rom, and eased interpolation judged by continuity, overshoot, controllability, and timing intent for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured C0/C1/C2 continuity, overshoot, velocity/acceleration extrema, edit complexity, and timing-target error; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Principles and keyframing. Using C0/C1/C2 continuity, overshoot, velocity/acceleration extrema, edit complexity, and timing-target error, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 9, Unit 2. Rotation interpolation

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. Euler angles, quaternions, slerp, squad, rotation splines.

A. Reading order

1. Read the primary assignment first: FCG transforms/animation sections; RTR transform chapter.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Quaternion animation tutorial.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Gimbal lock
- unit quaternion double cover
- geodesic interpolation on S^3
- Precision requirement: distinguish and justify Euler interpolation, normalized lerp, and quaternion slerp under large rotations, repeated composition, and near-antipodal quaternions. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive quaternion representation, composition, normalization, and slerp from the unit sphere in 4D; explain the antipodal ambiguity and compare with Euler-angle interpolation.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use near-180° rotations, long chains, and repeated interpolation; inspect shortest-path choices, angular velocity, and drift. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Animate a camera/object with Euler linear interpolation vs quaternion slerp; expose failure modes. Keep a minimal reference path even if you later optimize it.

- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Interpolate between identity and a 180-degree rotation about z at $t=0.5$ using normalized quaternion SLERP. Compare with linear interpolation of Euler angle triples under two representations of the same endpoint rotation. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use near-180° rotations, long chains, and repeated interpolation; inspect shortest-path choices, angular velocity, and drift. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Plot angular speed and orientation error for Euler lerp, nlerp, and slerp; add tests near 180 degrees and quaternion sign ambiguity.
- P5 - Machine-readable evidence. Export angular-speed variation, geodesic orientation error, norm/orthogonality drift, and failure near singular/antipodal cases to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: Euler interpolation, normalized lerp, and quaternion slerp under large rotations, repeated composition, and near-antipodal quaternions. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use near-180° rotations, long chains, and repeated interpolation; inspect shortest-path choices, angular velocity, and drift." without hand-editing code between methods. Record angular-speed variation, geodesic orientation error, norm/orthogonality drift, and failure near singular/antipodal cases.
- P7 - Deliberate bug and regression drill. Intentionally forget quaternion sign canonicalization before slerp, producing long-path interpolation. Before running, predict the first observable failure in angular-speed variation, geodesic orientation error, norm/orthogonality drift, and failure near singular/antipodal cases. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive quaternion representation, composition, normalization, and slerp from the unit sphere in 4D; explain the antipodal ambiguity and compare with Euler-angle interpolation. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Interpolate between identity and a 180-degree rotation about z at $t=0.5$ using normalized quaternion SLERP. Compare with linear interpolation of Euler angle triples under two representations of the same endpoint rotation. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Animate a camera/object with Euler linear interpolation vs quaternion slerp; expose failure modes. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Plot angular speed and orientation error for Euler lerp, nlerp, and slerp; add tests near 180 degrees and quaternion sign ambiguity. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare Euler interpolation, normalized lerp, and quaternion slerp under large rotations, repeated composition, and near-antipodal quaternions. Use the same skeleton, motion clips, keyframes, duration and control targets; compare trajectory/contact errors and, when perception matters, use the same viewer protocol. Record angular-speed variation, geodesic orientation error, norm/orthogonality drift, and failure near singular/antipodal cases. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use near-180° rotations, long chains, and repeated interpolation; inspect shortest-path choices, angular velocity, and drift. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG transforms/animation sections; RTR transform chapter. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Rotation interpolation. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among Euler interpolation, normalized lerp, and quaternion slerp under large rotations, repeated composition, and near-antipodal quaternions. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol

from Section F, using angular-speed variation, geodesic orientation error, norm/orthogonality drift, and failure near singular/antipodal cases. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if Euler interpolation, normalized lerp, and quaternion slerp under large rotations, repeated composition, and near-antipodal quaternions improve angular-speed variation, geodesic orientation error, norm/orthogonality drift, and failure near singular/antipodal cases across multiple motions/targets without trading the claimed control or plausibility for hidden contact, retargeting or temporal artifacts.

F. Concrete research project - Rotation interpolation

F1. Falsifiable question. Under the controlled failure regime "Use near-180° rotations, long chains, and repeated interpolation; inspect shortest-path choices, angular velocity, and drift.", characterize the ranking or regime transition across Euler interpolation, normalized lerp, and quaternion slerp under large rotations, repeated composition, and near-antipodal quaternions. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Plot angular speed and orientation error for Euler lerp, nlerp, and slerp; add tests near 180 degrees and quaternion sign ambiguity. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one hand-computable 2-3 joint/motion case, one medium articulated example, and one contact/retarget stress case. Save synchronized numerical traces with the output animation. The central adversarial condition is: Use near-180° rotations, long chains, and repeated interpolation; inspect shortest-path choices, angular velocity, and drift.

F4. Required evidence. Measure angular-speed variation, geodesic orientation error, norm/orthogonality drift, and failure near singular/antipodal cases. Deliver source motion/rig, generated motion/video, synchronized joint/contact/trajectory CSV, and one difficult transfer case with a short failure explanation. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Rotation interpolation. If a stable failure/crossover is found among Euler interpolation, normalized lerp, and quaternion slerp under large rotations, repeated composition, and near-antipodal quaternions, implement one adaptive or corrected variant that directly targets coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Rotation interpolation. Using the measured angular-speed variation, geodesic orientation error, norm/orthogonality drift, and failure near singular/antipodal cases, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Rotation interpolation

- G1. Closed-book derivation/oracle. Derive quaternion representation, composition, normalization, and slerp from the unit sphere in 4D; explain the antipodal ambiguity and compare with Euler-angle interpolation. Then solve this exact micro-case from scratch without opening your implementation: Interpolate between identity and a 180-degree rotation about z at $t=0.5$ using normalized quaternion SLERP. Compare with linear interpolation of Euler angle triples under two representations of the same endpoint rotation. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Animate a camera/object with Euler linear interpolation vs quaternion slerp; expose failure modes. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally forget quaternion sign canonicalization before slerp, producing long-path interpolation. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.

- G4. Transfer experiment. On an input/configuration not used during development, execute: Plot angular speed and orientation error for Euler lerp, nlerp, and slerp; add tests near 180 degrees and quaternion sign ambiguity. Then apply this stress without retuning: Use near-180° rotations, long chains, and repeated interpolation; inspect shortest-path choices, angular velocity, and drift. Report angular-speed variation, geodesic orientation error, norm/orthogonality drift, and failure near singular/antipodal cases and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across Euler interpolation, normalized lerp, and quaternion slerp under large rotations, repeated composition, and near-antipodal quaternions for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured angular-speed variation, geodesic orientation error, norm/orthogonality drift, and failure near singular/antipodal cases; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Rotation interpolation. Using angular-speed variation, geodesic orientation error, norm/orthogonality drift, and failure near singular/antipodal cases, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 9, Unit 3. Forward and inverse kinematics

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. Skeletons, joint chains, Jacobian IK, CCD, constraints.

A. Reading order

1. Read the primary assignment first: FCG Ch. 16.4; CS348C kinematics material.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 animation/kinematics discussion.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Forward kinematic chain
- Jacobian transpose/pseudoinverse
- singular configurations
- Precision requirement: distinguish and justify CCD, Jacobian-transpose, damped least squares, and analytic 2-link IK on reachable, near-singular, and unreachable targets. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive forward kinematics for a serial chain using homogeneous transforms, then derive the manipulator Jacobian and a damped least-squares inverse-kinematics step.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use unreachable targets, joint limits, singular configurations, and symmetric solutions; map convergence and pose plausibility. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement 2D/3D articulated chain and at least two IK solvers; compare convergence. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a planar two-link arm $L1=L2=1$ targeting (1,1), solve both IK branches analytically and verify by forward kinematics. Move target to (2.1,0) and explain the unreachable case. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use unreachable targets, joint limits, singular configurations, and symmetric solutions; map convergence and pose plausibility. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement two IK solvers with joint limits; plot end-effector error per iteration and damping behavior near a singular configuration.

- P5 - Machine-readable evidence. Export end-effector error, iterations, joint-limit violations, condition/damping near singularity, and unreachable-target behavior to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: CCD, Jacobian-transpose, damped least squares, and analytic 2-link IK on reachable, near-singular, and unreachable targets. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use unreachable targets, joint limits, singular configurations, and symmetric solutions; map convergence and pose plausibility." without hand-editing code between methods. Record end-effector error, iterations, joint-limit violations, condition/damping near singularity, and unreachable-target behavior.
- P7 - Deliberate bug and regression drill. Intentionally mix world-space and local-space Jacobian columns in the IK solver. Before running, predict the first observable failure in end-effector error, iterations, joint-limit violations, condition/damping near singularity, and unreachable-target behavior. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive forward kinematics for a serial chain using homogeneous transforms, then derive the manipulator Jacobian and a damped least-squares inverse-kinematics step. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a planar two-link arm $L_1=L_2=1$ targeting (1,1), solve both IK branches analytically and verify by forward kinematics. Move target to (2.1,0) and explain the unreachable case. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement 2D/3D articulated chain and at least two IK solvers; compare convergence. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement two IK solvers with joint limits; plot end-effector error per iteration and damping behavior near a singular configuration. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare CCD, Jacobian-transpose, damped least squares, and analytic 2-link IK on reachable, near-singular, and unreachable targets. Use the same skeleton, motion clips, keyframes, duration and control targets; compare trajectory/contact errors and, when perception matters, use the same viewer protocol. Record end-effector error, iterations, joint-limit violations, condition/damping near singularity, and unreachable-target behavior. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use unreachable targets, joint limits, singular configurations, and symmetric solutions; map convergence and pose plausibility. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 16.4; CS348C kinematics material. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Forward and inverse kinematics. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among CCD, Jacobian-transpose, damped least squares, and analytic 2-link IK on reachable, near-singular, and unreachable targets. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using end-effector error, iterations, joint-limit violations, condition/damping near singularity, and unreachable-target behavior. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if CCD, Jacobian-transpose, damped least squares, and analytic 2-link IK on reachable, near-singular, and unreachable targets improve end-effector error, iterations, joint-limit violations, condition/damping near singularity, and unreachable-target behavior across multiple motions/targets without trading the claimed control or plausibility for hidden contact, retargeting or temporal artifacts.

F. Concrete research project - Forward and inverse kinematics

- F1. Falsifiable question. Under the controlled failure regime "Use unreachable targets, joint limits, singular configurations, and symmetric solutions; map convergence and pose plausibility.", characterize the ranking or regime transition across CCD, Jacobian-transpose, damped least squares, and analytic 2-link IK on reachable, near-singular, and

unreachable targets. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement two IK solvers with joint limits; plot end-effector error per iteration and damping behavior near a singular configuration. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one hand-computable 2-3 joint/motion case, one medium articulated example, and one contact/retarget stress case. Save synchronized numerical traces with the output animation. The central adversarial condition is: Use unreachable targets, joint limits, singular configurations, and symmetric solutions; map convergence and pose plausibility.

F4. Required evidence. Measure end-effector error, iterations, joint-limit violations, condition/damping near singularity, and unreachable-target behavior. Deliver source motion/rig, generated motion/video, synchronized joint/contact/trajectory CSV, and one difficult transfer case with a short failure explanation. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Forward and inverse kinematics. If a stable failure/crossover is found among CCD, Jacobian-transpose, damped least squares, and analytic 2-link IK on reachable, near-singular, and unreachable targets, implement one adaptive or corrected variant that directly targets coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Forward and inverse kinematics. Using the measured end-effector error, iterations, joint-limit violations, condition/damping near singularity, and unreachable-target behavior, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Forward and inverse kinematics

- G1. Closed-book derivation/oracle. Derive forward kinematics for a serial chain using homogeneous transforms, then derive the manipulator Jacobian and a damped least-squares inverse-kinematics step. Then solve this exact micro-case from scratch without opening your implementation: For a planar two-link arm $L_1=L_2=1$ targeting $(1,1)$, solve both IK branches analytically and verify by forward kinematics. Move target to $(2.1,0)$ and explain the unreachable case. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement 2D/3D articulated chain and at least two IK solvers; compare convergence. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally mix world-space and local-space Jacobian columns in the IK solver. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement two IK solvers with joint limits; plot end-effector error per iteration and damping behavior near a singular configuration. Then apply this stress without retuning: Use unreachable targets, joint limits, singular configurations, and symmetric solutions; map convergence and pose plausibility. Report end-effector error, iterations, joint-limit violations, condition/damping near singularity, and unreachable-target behavior and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across CCD, Jacobian-transpose, damped least squares, and analytic 2-link IK on reachable, near-singular, and unreachable targets for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured end-effector error, iterations, joint-limit violations, condition/damping near singularity, and unreachable-target behavior; state one counterexample that would change the diagnosis or ranking.

- G6. Pass condition for Forward and inverse kinematics. Using end-effector error, iterations, joint-limit violations, condition/damping near singularity, and unreachable-target behavior, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 9, Unit 4. Skinning and deformation

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. Linear blend skinning, dual quaternions, pose-space deformation.

A. Reading order

1. Read the primary assignment first: Read FCG 5e §§16.3-16.4, Kavan et al., Geometric Skinning with Approximate Dual Quaternion Blending (2008), and Jacobson et al., Bounded Biharmonic Weights for Real-Time Deformation (2011) for weight construction/behavior.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: libigl skinning/deformation demos optional.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Candy-wrapper artifact
- weight normalization
- dual quaternion idea
- Precision requirement: distinguish and justify linear blend skinning vs dual-quaternion skinning on twisting/bending joints with volume/distortion measures. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive linear blend skinning and dual-quaternion skinning for a simple two-bone chain; analytically explain candy-wrapper and volume-loss artifacts.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use twisting limbs, sharp bends, bulging joints, and nonuniform scale; quantify volume and local surface distortion. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement LBS and demonstrate artifacts; optionally implement dual-quaternion skinning. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: A vertex is influenced 50/50 by two bones, one identity and one rotated 180 degrees. Compute linear-blend skinning position for a simple vertex and contrast with a rotation-aware interpretation to expose collapsing. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use twisting limbs, sharp bends, bulging joints, and nonuniform scale; quantify volume and local surface distortion. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Compute per-vertex volume/shape distortion around a twist joint for LBS and dual-quaternion skinning; visualize error as heatmaps.
- P5 - Machine-readable evidence. Export volume/shape distortion, joint collapse/candy-wrapper severity, runtime, and artifact area to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: linear blend skinning vs dual-quaternion skinning on twisting/bending joints with volume/distortion measures. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use twisting limbs, sharp bends, bulging joints, and nonuniform scale; quantify volume and local surface distortion." without hand-editing code between methods. Record volume/shape distortion, joint collapse/candy-wrapper severity, runtime, and artifact area.

- P7 - Deliberate bug and regression drill. Intentionally use unnormalized blend weights or linearly blend rotation matrices without re-orthogonalization. Before running, predict the first observable failure in volume/shape distortion, joint collapse/candy-wrapper severity, runtime, and artifact area. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive linear blend skinning and dual-quaternion skinning for a simple two-bone chain; analytically explain candy-wrapper and volume-loss artifacts. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. A vertex is influenced 50/50 by two bones, one identity and one rotated 180 degrees. Compute linear-blend skinning position for a simple vertex and contrast with a rotation-aware interpretation to expose collapsing. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement LBS and demonstrate artifacts; optionally implement dual-quaternion skinning. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Compute per-vertex volume/shape distortion around a twist joint for LBS and dual-quaternion skinning; visualize error as heatmaps. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare linear blend skinning vs dual-quaternion skinning on twisting/bending joints with volume/distortion measures. Use the same skeleton, motion clips, keyframes, duration and control targets; compare trajectory/contact errors and, when perception matters, use the same viewer protocol. Record volume/shape distortion, joint collapse/candy-wrapper severity, runtime, and artifact area. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use twisting limbs, sharp bends, bulging joints, and nonuniform scale; quantify volume and local surface distortion. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 16.3-16.4; modern skinning papers optional. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Skinning and deformation. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among linear blend skinning vs dual-quaternion skinning on twisting/bending joints with volume/distortion measures. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using volume/shape distortion, joint collapse/candy-wrapper severity, runtime, and artifact area. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if linear blend skinning vs dual-quaternion skinning on twisting/bending joints with volume/distortion measures improve volume/shape distortion, joint collapse/candy-wrapper severity, runtime, and artifact area across multiple motions/targets without trading the claimed control or plausibility for hidden contact, retargeting or temporal artifacts.

F. Concrete research project - Skinning and deformation

- F1. Falsifiable question. Under the controlled failure regime "Use twisting limbs, sharp bends, bulging joints, and nonuniform scale; quantify volume and local surface distortion.", which alternative in linear blend skinning vs dual-quaternion skinning on twisting/bending joints with volume/distortion measures is preferable, and is the change explained by coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Compute per-vertex volume/shape distortion around a twist joint for LBS and dual-quaternion skinning; visualize error as heatmaps. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use one hand-computable 2-3 joint/motion case, one medium articulated example, and one contact/retarget stress case. Save synchronized numerical traces with the output animation. The central adversarial

condition is: Use twisting limbs, sharp bends, bulging joints, and nonuniform scale; quantify volume and local surface distortion.

F4. Required evidence. Measure volume/shape distortion, joint collapse/candy-wrapper severity, runtime, and artifact area. Deliver source motion/rig, generated motion/video, synchronized joint/contact/trajectory CSV, and one difficult transfer case with a short failure explanation. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Skinning and deformation. If a stable failure/crossover is found among linear blend skinning vs dual-quaternion skinning on twisting/bending joints with volume/distortion measures, implement one adaptive or corrected variant that directly targets coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Skinning and deformation. Using the measured volume/shape distortion, joint collapse/candy-wrapper severity, runtime, and artifact area, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Skinning and deformation

- G1. Closed-book derivation/oracle. Derive linear blend skinning and dual-quaternion skinning for a simple two-bone chain; analytically explain candy-wrapper and volume-loss artifacts. Then solve this exact micro-case from scratch without opening your implementation: A vertex is influenced 50/50 by two bones, one identity and one rotated 180 degrees. Compute linear-blend skinning position for a simple vertex and contrast with a rotation-aware interpretation to expose collapsing. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement LBS and demonstrate artifacts; optionally implement dual-quaternion skinning. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use unnormalized blend weights or linearly blend rotation matrices without re-orthogonalization. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Compute per-vertex volume/shape distortion around a twist joint for LBS and dual-quaternion skinning; visualize error as heatmaps. Then apply this stress without retuning: Use twisting limbs, sharp bends, bulging joints, and nonuniform scale; quantify volume and local surface distortion. Report volume/shape distortion, joint collapse/candy-wrapper severity, runtime, and artifact area and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among linear blend skinning vs dual-quaternion skinning on twisting/bending joints with volume/distortion measures for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured volume/shape distortion, joint collapse/candy-wrapper severity, runtime, and artifact area; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Skinning and deformation. Using volume/shape distortion, joint collapse/candy-wrapper severity, runtime, and artifact area, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 9, Unit 5. Motion capture and retargeting

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. Skeleton correspondence, root motion, filtering, retarget constraints.

A. Reading order

1. Read the primary assignment first: Read Gleicher, Retargeting Motion to New Characters (SIGGRAPH 1998), then a modern character-animation course/source for skeleton conventions and contact-aware retargeting. Reconstruct one retargeting objective before using learned motion systems.

2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Use BVH motion-capture data tutorial.

3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026

lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Pose representation
- local/global transforms
- foot-skate problem
- Precision requirement: distinguish and justify direct joint-angle transfer vs scale/contact-aware retargeting across different body proportions. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a coordinate/scale mapping between source and target skeletons, formulate a retargeting objective with end-effector/contact constraints, and identify underdetermined degrees of freedom.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Retarget walking/reaching to skeletons with very different limb ratios; measure foot slip, penetration, end-effector error, and style drift. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Load BVH, visualize skeleton, retarget to second proportions, and measure foot contact error. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Retarget a two-bone leg with source lengths (1,1) to target lengths (1.2,0.8) for one pose. Preserve foot world position if possible and compute the required knee/hip angles or residual. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Retarget walking/reaching to skeletons with very different limb ratios; measure foot slip, penetration, end-effector error, and style drift. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Add foot-contact detection and root-trajectory scaling; compare contact sliding before/after correction across two skeleton proportions.
- P5 - Machine-readable evidence. Export foot sliding/contact error, joint-limit violations, root trajectory error, smoothness, and proportion-change sensitivity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: direct joint-angle transfer vs scale/contact-aware retargeting across different body proportions. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Retarget walking/reaching to skeletons with very different limb ratios; measure foot slip, penetration, end-effector error, and style drift." without hand-editing code between methods. Record foot sliding/contact error, joint-limit violations, root trajectory error, smoothness, and proportion-change sensitivity.
- P7 - Deliberate bug and regression drill. Intentionally retarget positions without reconciling skeleton scale/coordinate-frame conventions. Before running, predict the first observable failure in foot sliding/contact error, joint-limit violations, root trajectory error, smoothness, and proportion-change sensitivity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a coordinate/scale mapping between source and target skeletons, formulate a retargeting objective with end-effector/contact constraints, and identify underdetermined degrees of freedom. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. Retarget a two-bone leg with source lengths (1,1) to target lengths (1.2,0.8) for one pose. Preserve foot world position if possible and compute the required knee/hip angles or residual. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Load BVH, visualize skeleton, retarget to second proportions, and measure foot contact error. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Add foot-contact detection and root-trajectory scaling; compare contact sliding before/after correction across two skeleton proportions. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare direct joint-angle transfer vs scale/contact-aware retargeting across different body proportions. Use the same skeleton, motion clips, keyframes, duration and control targets; compare trajectory/contact errors and, when perception matters, use the same viewer protocol. Record foot sliding/contact error, joint-limit violations, root trajectory error, smoothness, and proportion-change sensitivity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Retarget walking/reaching to skeletons with very different limb ratios; measure foot slip, penetration, end-effector error, and style drift. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: CS348C readings on character animation; selected SIGGRAPH papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Motion capture and retargeting. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among direct joint-angle transfer vs scale/contact-aware retargeting across different body proportions. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using foot sliding/contact error, joint-limit violations, root trajectory error, smoothness, and proportion-change sensitivity. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if direct joint-angle transfer vs scale/contact-aware retargeting across different body proportions improve foot sliding/contact error, joint-limit violations, root trajectory error, smoothness, and proportion-change sensitivity across multiple motions/targets without trading the claimed control or plausibility for hidden contact, retargeting or temporal artifacts.

F. Concrete research project - Motion capture and retargeting

- F1. Falsifiable question. Under the controlled failure regime "Retarget walking/reaching to skeletons with very different limb ratios; measure foot slip, penetration, end-effector error, and style drift.", which alternative in direct joint-angle transfer vs scale/contact-aware retargeting across different body proportions is preferable, and is the change explained by coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Add foot-contact detection and root-trajectory scaling; compare contact sliding before/after correction across two skeleton proportions. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use one hand-computable 2-3 joint/motion case, one medium articulated example, and one contact/retarget stress case. Save synchronized numerical traces with the output animation. The central adversarial condition is: Retarget walking/reaching to skeletons with very different limb ratios; measure foot slip, penetration, end-effector error, and style drift.
- F4. Required evidence. Measure foot sliding/contact error, joint-limit violations, root trajectory error, smoothness, and proportion-change sensitivity. Deliver source motion/rig, generated motion/video, synchronized joint/contact/trajectory CSV, and one difficult transfer case with a short failure explanation. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Motion capture and retargeting. If a stable failure/crossover is found among direct joint-angle transfer vs scale/contact-aware retargeting across different body proportions, implement one adaptive or corrected variant that directly targets coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Motion capture and retargeting. Using the measured foot sliding/contact error, joint-limit violations, root trajectory error, smoothness, and proportion-change sensitivity, Pass when the hand kinematic oracle is

exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Motion capture and retargeting

- G1. Closed-book derivation/oracle. Derive a coordinate/scale mapping between source and target skeletons, formulate a retargeting objective with end-effector/contact constraints, and identify underdetermined degrees of freedom. Then solve this exact micro-case from scratch without opening your implementation: Retarget a two-bone leg with source lengths (1,1) to target lengths (1.2,0.8) for one pose. Preserve foot world position if possible and compute the required knee/hip angles or residual. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Load BVH, visualize skeleton, retarget to second proportions, and measure foot contact error. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally retarget positions without reconciling skeleton scale/coordinate-frame conventions. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Add foot-contact detection and root-trajectory scaling; compare contact sliding before/after correction across two skeleton proportions. Then apply this stress without retuning: Retarget walking/reaching to skeletons with very different limb ratios; measure foot slip, penetration, end-effector error, and style drift. Report foot sliding/contact error, joint-limit violations, root trajectory error, smoothness, and proportion-change sensitivity and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among direct joint-angle transfer vs scale/contact-aware retargeting across different body proportions for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured foot sliding/contact error, joint-limit violations, root trajectory error, smoothness, and proportion-change sensitivity; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Motion capture and retargeting. Using foot sliding/contact error, joint-limit violations, root trajectory error, smoothness, and proportion-change sensitivity, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 9, Unit 6. Motion synthesis and control

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. Blend spaces, motion graphs, optimization, data-driven motion.

A. Reading order

1. Read the primary assignment first: Read Kovar, Gleicher & Pighin, Motion Graphs (SIGGRAPH 2002), and Lee et al., Interactive Control of Avatars Animated with Human Motion Data (SIGGRAPH 2002); use recent learned-control papers only after reproducing one classical transition/contact benchmark.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Reproduce small motion-graph system.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- State graph
- transition cost
- trajectory/control objective
- Precision requirement: distinguish and justify nearest-clip playback vs motion-graph transition search under path/contact/style constraints. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Formulate motion synthesis as trajectory optimization or feedback control on a toy articulated system; derive one tracking/control objective and explain stability versus responsiveness tradeoffs.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Demand abrupt control changes, rare transitions, long horizons, and contact-rich motion; measure responsiveness, contact error, and diversity. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Build 2D motion graph from short clips or toy trajectories; search transitions under a user path. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Blend two 1D cyclic motions $x_1(t)=\sin t$ and $x_2(t)=\sin(t+\pi/2)$ at weight 0.5. Compare naive phase-mismatched blend with a phase-aligned blend at four sample times. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Demand abrupt control changes, rare transitions, long horizons, and contact-rich motion; measure responsiveness, contact error, and diversity. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Construct a motion graph with explicit transition cost terms; visualize chosen transitions and report path error, discontinuity cost, and dead-end rate.
- P5 - Machine-readable evidence. Export transition discontinuity cost, path/control error, contact violations, dead-end rate, and search/runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: nearest-clip playback vs motion-graph transition search under path/contact/style constraints. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Demand abrupt control changes, rare transitions, long horizons, and contact-rich motion; measure responsiveness, contact error, and diversity." without hand-editing code between methods. Record transition discontinuity cost, path/control error, contact violations, dead-end rate, and search/runtime.
- P7 - Deliberate bug and regression drill. Intentionally optimize trajectory error while omitting foot-contact constraints, creating low-error foot skating. Before running, predict the first observable failure in transition discontinuity cost, path/control error, contact violations, dead-end rate, and search/runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Formulate motion synthesis as trajectory optimization or feedback control on a toy articulated system; derive one tracking/control objective and explain stability versus responsiveness tradeoffs. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Blend two 1D cyclic motions $x_1(t)=\sin t$ and $x_2(t)=\sin(t+\pi/2)$ at weight 0.5. Compare naive phase-mismatched blend with a phase-aligned blend at four sample times. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Build 2D motion graph from short clips or toy trajectories; search transitions under a user path. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Construct a motion graph with explicit transition cost terms; visualize chosen transitions and report path error, discontinuity cost, and dead-end rate. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare nearest-clip playback vs motion-graph transition search under path/contact/style constraints. Use the same skeleton, motion clips, keyframes, duration and control targets; compare trajectory/contact errors and, when perception matters, use the same viewer protocol. Record transition discontinuity cost, path/control error, contact violations, dead-end rate, and search/runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Demand abrupt control changes, rare transitions, long horizons, and contact-rich motion; measure responsiveness, contact error, and diversity. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Classic motion graph papers and recent motion-generation papers later in Vol. 15. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Motion synthesis and control. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among nearest-clip playback vs motion-graph transition search under path/contact/style constraints. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using transition discontinuity cost, path/control error, contact violations, dead-end rate, and search/runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if nearest-clip playback vs motion-graph transition search under path/contact/style constraints improve transition discontinuity cost, path/control error, contact violations, dead-end rate, and search/runtime across multiple motions/targets without trading the claimed control or plausibility for hidden contact, retargeting or temporal artifacts.

F. Concrete research project - Motion synthesis and control

F1. Falsifiable question. Under the controlled failure regime "Demand abrupt control changes, rare transitions, long horizons, and contact-rich motion; measure responsiveness, contact error, and diversity.", which alternative in nearest-clip playback vs motion-graph transition search under path/contact/style constraints is preferable, and is the change explained by coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Construct a motion graph with explicit transition cost terms; visualize chosen transitions and report path error, discontinuity cost, and dead-end rate. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one hand-computable 2-3 joint/motion case, one medium articulated example, and one contact/retarget stress case. Save synchronized numerical traces with the output animation. The central adversarial condition is: Demand abrupt control changes, rare transitions, long horizons, and contact-rich motion; measure responsiveness, contact error, and diversity.

F4. Required evidence. Measure transition discontinuity cost, path/control error, contact violations, dead-end rate, and search/runtime. Deliver source motion/rig, generated motion/video, synchronized joint/contact/trajectory CSV, and one difficult transfer case with a short failure explanation. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Motion synthesis and control. If a stable failure/crossover is found among nearest-clip playback vs motion-graph transition search under path/contact/style constraints, implement one adaptive or corrected variant that directly targets coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Motion synthesis and control. Using the measured transition discontinuity cost, path/control error, contact violations, dead-end rate, and search/runtime, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Motion synthesis and control

- G1. Closed-book derivation/oracle. Formulate motion synthesis as trajectory optimization or feedback control on a toy articulated system; derive one tracking/control objective and explain stability versus responsiveness tradeoffs. Then solve this exact micro-case from scratch without opening your implementation: Blend two 1D cyclic motions $x_1(t) = \sin t$ and $x_2(t) = \sin(t + \pi/2)$ at weight 0.5. Compare naive phase-mismatched blend with a phase-aligned blend at four sample times. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Build 2D motion graph from short clips or toy trajectories; search transitions under a user path. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally optimize trajectory error while omitting foot-contact constraints, creating low-error foot skating. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Construct a motion graph with explicit transition cost terms; visualize chosen transitions and report path error, discontinuity cost, and dead-end rate. Then apply this stress without retuning: Demand abrupt control changes, rare transitions, long horizons, and contact-rich motion; measure responsiveness, contact error, and diversity. Report transition discontinuity cost, path/control error, contact violations, dead-end rate, and search/runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among nearest-clip playback vs motion-graph transition search under path/contact/style constraints for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured transition discontinuity cost, path/control error, contact violations, dead-end rate, and search/runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Motion synthesis and control. Using transition discontinuity cost, path/control error, contact violations, dead-end rate, and search/runtime, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 9, Unit 7. Crowds and procedural animation

Recommended time. 1.7-2.3 weeks (about 26-46 focused hours depending on background).

What you are learning. Agent steering, flocking, procedural secondary motion.

A. Reading order

1. Read the primary assignment first: FCG Ch. 16.6-16.7; CS348C optional crowd readings.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Implement Reynolds-style boids.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184/284A Spring 2026 lectures/projects - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Local rules giving global behavior
- collision avoidance intuition
- Precision requirement: distinguish and justify simple boids/social-force-like rules vs obstacle-aware variants across density, bottlenecks, and perturbations. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a local collision-avoidance/steering rule for agents and analyze equilibria/failure modes in symmetric encounters, bottlenecks, and high-density flows.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use narrow exits, bidirectional flow, dense crossings, and sudden obstacles; log collisions, deadlocks, oscillations, throughput, and path inefficiency. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create crowd/boids simulator with obstacles and measurable stability/throughput. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Two disc agents of radius 0.5 start at (-1,0),(1,0) and desire to swap positions. Compute time-to-collision for straight-line velocities and propose one symmetric avoidance velocity pair. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Use narrow exits, bidirectional flow, dense crossings, and sudden obstacles; log collisions, deadlocks, oscillations, throughput, and path inefficiency. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Automate density/bottleneck sweeps and report collision rate, throughput, speed variance, and deadlock/oscillation frequency.
- P5 - Machine-readable evidence. Export collision rate, throughput, speed variance, deadlock/oscillation frequency, and runtime versus agent density to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: simple boids/social-force-like rules vs obstacle-aware variants across density, bottlenecks, and perturbations. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use narrow exits, bidirectional flow, dense crossings, and sudden obstacles; log collisions, deadlocks, oscillations, throughput, and path inefficiency." without hand-editing code between methods. Record collision rate, throughput, speed variance, deadlock/oscillation frequency, and runtime versus agent density.
- P7 - Deliberate bug and regression drill. Intentionally update agents sequentially with newly updated neighbor positions, making results depend on iteration order. Before running, predict the first observable failure in collision rate, throughput, speed variance, deadlock/oscillation frequency, and runtime versus agent density. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a local collision-avoidance/steering rule for agents and analyze equilibria/failure modes in symmetric encounters, bottlenecks, and high-density flows. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Two disc agents of radius 0.5 start at $(-1,0)$, $(1,0)$ and desire to swap positions. Compute time-to-collision for straight-line velocities and propose one symmetric avoidance velocity pair. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Create crowd/boids simulator with obstacles and measurable stability/throughput. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Automate density/bottleneck sweeps and report collision rate, throughput, speed variance, and deadlock/oscillation frequency. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare simple boids/social-force-like rules vs obstacle-aware variants across density, bottlenecks, and perturbations. Use the same skeleton, motion clips, keyframes, duration and control targets; compare trajectory/contact errors and, when perception matters, use the same viewer protocol. Record collision rate, throughput, speed variance, deadlock/oscillation frequency, and runtime versus agent density. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use narrow exits, bidirectional flow, dense crossings, and sudden obstacles; log collisions, deadlocks, oscillations, throughput, and path inefficiency. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: FCG Ch. 16.6-16.7; CS348C optional crowd readings. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Crowds and procedural animation. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among simple boids/social-force-like rules vs obstacle-aware variants across density, bottlenecks, and perturbations. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using collision rate, throughput, speed variance, deadlock/oscillation frequency, and runtime versus agent density. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if simple boids/social-force-like rules vs obstacle-aware variants across density, bottlenecks, and perturbations improve collision rate, throughput, speed variance, deadlock/oscillation frequency, and runtime versus agent density across multiple motions/targets without trading the claimed control or plausibility for hidden contact, retargeting or temporal artifacts.

F. Concrete research project - Crowds and procedural animation

F1. Falsifiable question. Under the controlled failure regime "Use narrow exits, bidirectional flow, dense crossings, and sudden obstacles; log collisions, deadlocks, oscillations, throughput, and path inefficiency.", which alternative in simple boids/social-force-like rules vs obstacle-aware variants across density, bottlenecks, and perturbations is preferable, and is the change explained by coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Automate density/bottleneck sweeps and report collision rate, throughput, speed variance, and deadlock/oscillation frequency. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use one hand-computable 2-3 joint/motion case, one medium articulated example, and one contact/retarget stress case. Save synchronized numerical traces with the output animation. The central adversarial condition is: Use narrow exits, bidirectional flow, dense crossings, and sudden obstacles; log collisions, deadlocks, oscillations, throughput, and path inefficiency.

F4. Required evidence. Measure collision rate, throughput, speed variance, deadlock/oscillation frequency, and runtime versus agent density. Deliver source motion/rig, generated motion/video, synchronized joint/contact/trajectory CSV, and one difficult transfer case with a short failure explanation. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Crowds and procedural animation. If a stable failure/crossover is found among simple boids/social-force-like rules vs obstacle-aware variants across density, bottlenecks, and perturbations, implement one adaptive or corrected variant that directly targets coordinate-frame consistency, constraint/contact satisfaction, interpolation/control parameterization, and transfer error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Crowds and procedural animation. Using the measured collision rate, throughput, speed variance, deadlock/oscillation frequency, and runtime versus agent density, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Crowds and procedural animation

- G1. Closed-book derivation/oracle. Derive a local collision-avoidance/steering rule for agents and analyze equilibria/failure modes in symmetric encounters, bottlenecks, and high-density flows. Then solve this exact micro-case from scratch without opening your implementation: Two disc agents of radius 0.5 start at (-1,0),(1,0) and desire to swap positions. Compute time-to-collision for straight-line velocities and propose one symmetric avoidance velocity pair. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create crowd/boids simulator with obstacles and measurable stability/throughput. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally update agents sequentially with newly updated neighbor positions, making results depend on iteration order. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Automate density/bottleneck sweeps and report collision rate, throughput, speed variance, and deadlock/oscillation frequency. Then apply this stress without retuning: Use narrow exits, bidirectional flow, dense crossings, and sudden obstacles; log collisions, deadlocks, oscillations, throughput, and path inefficiency. Report collision rate, throughput, speed variance, deadlock/oscillation frequency, and runtime versus agent density and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among simple boids/social-force-like rules vs obstacle-aware variants across density, bottlenecks, and perturbations for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured collision rate, throughput, speed variance, deadlock/oscillation frequency, and runtime versus agent density; state one counterexample that would reverse your recommendation.

- G6. Pass condition for Crowds and procedural animation. Using collision rate, throughput, speed variance, deadlock/oscillation frequency, and runtime versus agent density, Pass when the hand kinematic oracle is exact, the transfer case preserves the stated constraints/invariants, and the numerical traces explain any visible artifact such as foot skating, joint error, collision, or oscillation. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 9 Closeout: Project, Review and Mastery Gate

Volume project. Build a character-motion sandbox with skeletons, quaternion interpolation, IK, skinning, motion import/retargeting and one data-driven/procedural motion technique.

Cumulative review

- Build a character/motion sandbox containing quaternion interpolation, FK/IK, skinning and at least one motion-control method.
- Verify rotations/kinematics on hand-computable chains and quantify contact/trajectory errors.
- Compare two skinning or motion-control approaches on a pose/motion designed to expose their characteristic failure.
- Instrument motion with joint-limit, foot-contact, velocity/acceleration and retargeting diagnostics.
- Write a synthesis distinguishing representation/interpolation error from control/data/style error.

Mastery gate before moving on

- Demonstrate quaternion interpolation, IK, skinning, and retargeting on cases intentionally chosen to expose artifacts.
- Measure contact/end-effector error, smoothness, and deformation distortion rather than relying only on visual plausibility.
- Explain how each representation/constraint changes the reachable motion/deformation space.

Volume 10. Physics-Based Simulation I: Particles, Rigid Bodies, Solids, Cloth, and Contact

Purpose. Develop a principled simulation toolkit for deformable and rigid objects, including numerical stability and robust contact.

Suggested duration. 18-22 weeks

Prerequisites. Volume 1 numerical methods; Volumes 2 and 9 are helpful.

Typical module pace. 2.2-2.8 weeks per module (8 modules total).

Primary references for this volume

- Physics-Based Simulation open book (current primary implementation reference).
- Stanford CS348C simulation notes.
- Sifakis & Barbic FEM course notes for deformables.
- Classic PBD/XPBD/IPC/FEM papers.

Online spine

Physics-Based Simulation open book: <https://phys-sim-book.github.io/>

Stanford CS348C Animation and Simulation: <https://graphics.stanford.edu/courses/cs348c/>

Canonical paper track

- Baraff & Witkin (1998), Large Steps in Cloth Simulation.
- Müller et al. (2007), Position Based Dynamics.
- Macklin et al. (2016), XPBD: Position-Based Simulation of Compliant Constrained Dynamics.
- Irving et al. (2004), Invertible Finite Elements for Robust Simulation of Large Deformation.
- Li et al. (2020), Incremental Potential Contact (IPC).

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Baraff & Witkin (1998), Large Steps in Cloth Simulation.
- Bridson, Fedkiw & Anderson (2002), Robust Treatment of Collisions, Contact and Friction for Cloth Animation.
- Irving, Teran & Fedkiw (2004), Invertible Finite Elements for Robust Simulation of Large Deformation.
- Bergou et al. (2008), Discrete Elastic Rods.
- Müller et al. (2007), Position Based Dynamics - compare constraint projection with force/energy-based formulations.
- Li et al. (2020), Incremental Potential Contact: Intersection- and Inversion-free, Large-Deformation Dynamics.
- Physics-Based Simulation online book v1.0.3 (2026) - use as the current executable/theory spine.

Volume 10, Unit 1. Time integration and dynamical systems

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. State-space formulation, explicit/implicit/symplectic schemes, stiffness.

A. Reading order

1. Read the primary assignment first: Physics-Based Simulation book foundations; CS348C integration/cloth notes.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Stanford CS348C implicit integration lecture.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Stability regions
- symplectic behavior
- Newton solve for implicit step
- Precision requirement: distinguish and justify explicit Euler, symplectic Euler, Verlet, and backward Euler on oscillatory and stiff systems with stability/phase/energy plots. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive stability regions/update matrices for explicit, semi-implicit, and implicit integration on a linear oscillator; relate timestep, stiffness, damping, phase error, and energy drift.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Sweep timestep through stability limits for conservative and damped/stiff systems; track phase, energy, and long-horizon drift. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Compare Euler, symplectic Euler, Verlet, implicit Euler on springs; chart stability vs step size. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a mass-spring $m=1, k=100$, $x_0=1, v_0=0$, compute one step at $dt=0.05$ and 0.25 with explicit and implicit Euler. Compare against the explicit stability expectation. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Sweep timestep through stability limits for conservative and damped/stiff systems; track phase, energy, and long-horizon drift. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Produce stability and phase portraits plus long-run energy curves for all integrators on soft and stiff springs.
- P5 - Machine-readable evidence. Export stability boundary, energy drift, phase error, constraint/trajectory error, and runtime per simulated second to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.

- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: explicit Euler, symplectic Euler, Verlet, and backward Euler on oscillatory and stiff systems with stability/phase/energy plots. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Sweep timestep through stability limits for conservative and damped/stiff systems; track phase, energy, and long-horizon drift." without hand-editing code between methods. Record stability boundary, energy drift, phase error, constraint/trajectory error, and runtime per simulated second.
- P7 - Deliberate bug and regression drill. Intentionally label an update as symplectic Euler while actually using the explicit-Euler position/velocity order. Before running, predict the first observable failure in stability boundary, energy drift, phase error, constraint/trajectory error, and runtime per simulated second. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive stability regions/update matrices for explicit, semi-implicit, and implicit integration on a linear oscillator; relate timestep, stiffness, damping, phase error, and energy drift. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a mass-spring $m=1, k=100, x_0=1, v_0=0$, compute one step at $dt=0.05$ and 0.25 with explicit and implicit Euler. Compare against the explicit stability expectation. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Compare Euler, symplectic Euler, Verlet, implicit Euler on springs; chart stability vs step size. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Produce stability and phase portraits plus long-run energy curves for all integrators on soft and stiff springs. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare explicit Euler, symplectic Euler, Verlet, and backward Euler on oscillatory and stiff systems with stability/phase/energy plots. Use the same material parameters, geometry, timestep, mesh resolution and nonlinear/linear solver tolerances; if wall time is compared, report the accuracy/stability change at that cost. Record stability boundary, energy drift, phase error, constraint/trajectory error, and runtime per simulated second. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Sweep timestep through stability limits for conservative and damped/stiff systems; track phase, energy, and long-horizon drift. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Physics-Based Simulation book foundations; CS348C integration/cloth notes. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Time integration and dynamical systems. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among explicit Euler, symplectic Euler, Verlet, and backward Euler on oscillatory and stiff systems with stability/phase/energy plots. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using stability boundary, energy drift, phase error, constraint/trajectory error, and runtime per simulated second. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if explicit Euler, symplectic Euler, Verlet, and backward Euler on oscillatory and stiff systems with stability/phase/energy plots improve stability boundary, energy drift, phase error, constraint/trajectory error, and runtime per simulated second after timestep, mesh and solver-tolerance checks and preserve the physical invariants/constraints relevant to the model; visual smoothness alone is insufficient.

F. Concrete research project - Time integration and dynamical systems

- F1. Falsifiable question. Under the controlled failure regime "Sweep timestep through stability limits for conservative and damped/stiff systems; track phase, energy, and long-horizon drift.", characterize the ranking or regime transition across explicit Euler, symplectic Euler, Verlet, and backward Euler on oscillatory and stiff systems with stability/phase/energy plots. Predict which method/regime should dominate or be diagnosed first as the stress changes,

and test whether the transition is explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Produce stability and phase portraits plus long-run energy curves for all integrators on soft and stiff springs. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a one-element/one-contact oracle, one canonical benchmark, and one large-deformation/contact stress case; run at least four time steps and three spatial/solver resolutions where meaningful. The central adversarial condition is: Sweep timestep through stability limits for conservative and damped/stiff systems; track phase, energy, and long-horizon drift.

F4. Required evidence. Measure stability boundary, energy drift, phase error, constraint/trajectory error, and runtime per simulated second. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Time integration and dynamical systems. If a stable failure/crossover is found among explicit Euler, symplectic Euler, Verlet, and backward Euler on oscillatory and stiff systems with stability/phase/energy plots, implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Time integration and dynamical systems. Using the measured stability boundary, energy drift, phase error, constraint/trajectory error, and runtime per simulated second, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Time integration and dynamical systems

- G1. Closed-book derivation/oracle. Derive stability regions/update matrices for explicit, semi-implicit, and implicit integration on a linear oscillator; relate timestep, stiffness, damping, phase error, and energy drift. Then solve this exact micro-case from scratch without opening your implementation: For a mass-spring $m=1, k=100, x_0=1, v_0=0$, compute one step at $dt=0.05$ and 0.25 with explicit and implicit Euler. Compare against the explicit stability expectation. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Compare Euler, symplectic Euler, Verlet, implicit Euler on springs; chart stability vs step size. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally label an update as symplectic Euler while actually using the explicit-Euler position/velocity order. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Produce stability and phase portraits plus long-run energy curves for all integrators on soft and stiff springs. Then apply this stress without retuning: Sweep timestep through stability limits for conservative and damped/stiff systems; track phase, energy, and long-horizon drift. Report stability boundary, energy drift, phase error, constraint/trajectory error, and runtime per simulated second and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across explicit Euler, symplectic Euler, Verlet, and backward Euler on oscillatory and stiff systems with stability/phase/energy plots for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured stability boundary, energy drift, phase error, constraint/trajectory error, and runtime per simulated second; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Time integration and dynamical systems. Using stability boundary, energy drift, phase error, constraint/trajectory error, and runtime per simulated second, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 10, Unit 2. Particles, springs, and constraints

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Mass-spring systems, damping, constraints, PBD/XPBD ideas.

A. Reading order

1. Read the primary assignment first: Physics-Based Simulation PBD chapters; CS348C.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Official Physics-Based Simulation PBD section.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Constraint projection
- compliance
- position-based dynamics update
- Precision requirement: distinguish and justify mass-spring force integration vs PBD/XPBD constraints as stiffness and timestep vary. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive forces and Jacobians for a mass-spring system and formulate a holonomic distance constraint; compare force-based, penalty, projection, and Lagrange-multiplier enforcement.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Sweep stiffness/compliance, timestep, mass ratios, and solver iterations; check whether apparent material behavior changes with numerical parameters. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement cloth as mass-spring and PBD/XPBD; compare stiffness vs timestep behavior. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Two particles at distance 1.2 are connected by a rest-length-1 spring/constraint. Compute spring force and one projection correction; compare how each treats stiffness/timestep. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Sweep stiffness/compliance, timestep, mass ratios, and solver iterations; check whether apparent material behavior changes with numerical parameters. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement XPBD compliance and demonstrate that effective stiffness is less timestep-dependent than naïve PBD/mass-spring tuning.
- P5 - Machine-readable evidence. Export constraint/distance error, effective stiffness versus timestep, energy behavior, iterations, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: mass-spring force integration vs PBD/XPBD constraints as stiffness and timestep vary. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Sweep stiffness/compliance, timestep, mass ratios, and solver iterations; check whether apparent material behavior changes with numerical parameters." without hand-editing code between methods. Record constraint/distance error, effective stiffness versus timestep, energy behavior, iterations, and runtime.
- P7 - Deliberate bug and regression drill. Intentionally project a constraint without inverse-mass weighting so pinned/heavy particles move incorrectly. Before running, predict the first observable failure in constraint/distance error, effective stiffness versus timestep, energy behavior, iterations, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive forces and Jacobians for a mass-spring system and formulate a holonomic distance constraint; compare force-based, penalty, projection, and Lagrange-multiplier enforcement. Then evaluate the result on

the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. Two particles at distance 1.2 are connected by a rest-length-1 spring/constraint. Compute spring force and one projection correction; compare how each treats stiffness/timestep. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Implement cloth as mass-spring and PBD/XPBD; compare stiffness vs timestep behavior. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement XPBD compliance and demonstrate that effective stiffness is less timestep-dependent than naïve PBD/mass-spring tuning. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare mass-spring force integration vs PBD/XPBD constraints as stiffness and timestep vary. Use the same material parameters, geometry, timestep, mesh resolution and nonlinear/linear solver tolerances; if wall time is compared, report the accuracy/stability change at that cost. Record constraint/distance error, effective stiffness versus timestep, energy behavior, iterations, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Sweep stiffness/compliance, timestep, mass ratios, and solver iterations; check whether apparent material behavior changes with numerical parameters. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Physics-Based Simulation PBD chapters; CS348C. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Particles, springs, and constraints. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among mass-spring force integration vs PBD/XPBD constraints as stiffness and timestep vary. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using constraint/distance error, effective stiffness versus timestep, energy behavior, iterations, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if mass-spring force integration vs PBD/XPBD constraints as stiffness and timestep vary improve constraint/distance error, effective stiffness versus timestep, energy behavior, iterations, and runtime after timestep, mesh and solver-tolerance checks and preserve the physical invariants/constraints relevant to the model; visual smoothness alone is insufficient.

F. Concrete research project - Particles, springs, and constraints

F1. Falsifiable question. Under the controlled failure regime "Sweep stiffness/compliance, timestep, mass ratios, and solver iterations; check whether apparent material behavior changes with numerical parameters.", which alternative in mass-spring force integration vs PBD/XPBD constraints as stiffness and timestep vary is preferable, and is the change explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement XPBD compliance and demonstrate that effective stiffness is less timestep-dependent than naïve PBD/mass-spring tuning. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a one-element/one-contact oracle, one canonical benchmark, and one large-deformation/contact stress case; run at least four time steps and three spatial/solver resolutions where meaningful. The central adversarial condition is: Sweep stiffness/compliance, timestep, mass ratios, and solver iterations; check whether apparent material behavior changes with numerical parameters.

F4. Required evidence. Measure constraint/distance error, effective stiffness versus timestep, energy behavior, iterations, and runtime. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Particles, springs, and constraints. If a stable failure/crossover is found among mass-spring force integration vs PBD/XPBD constraints as stiffness and timestep vary, implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Particles, springs, and constraints. Using the measured constraint/distance error, effective stiffness versus timestep, energy behavior, iterations, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Particles, springs, and constraints

- G1. Closed-book derivation/oracle. Derive forces and Jacobians for a mass-spring system and formulate a holonomic distance constraint; compare force-based, penalty, projection, and Lagrange-multiplier enforcement. Then solve this exact micro-case from scratch without opening your implementation: Two particles at distance 1.2 are connected by a rest-length-1 spring/constraint. Compute spring force and one projection correction; compare how each treats stiffness/timestep. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement cloth as mass-spring and PBD/XPBD; compare stiffness vs timestep behavior. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally project a constraint without inverse-mass weighting so pinned/heavy particles move incorrectly. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement XPBD compliance and demonstrate that effective stiffness is less timestep-dependent than naïve PBD/mass-spring tuning. Then apply this stress without retuning: Sweep stiffness/compliance, timestep, mass ratios, and solver iterations; check whether apparent material behavior changes with numerical parameters. Report constraint/distance error, effective stiffness versus timestep, energy behavior, iterations, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among mass-spring force integration vs PBD/XPBD constraints as stiffness and timestep vary for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured constraint/distance error, effective stiffness versus timestep, energy behavior, iterations, and runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Particles, springs, and constraints. Using constraint/distance error, effective stiffness versus timestep, energy behavior, iterations, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 10, Unit 3. Rigid body dynamics

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Mass properties, angular momentum, inertia tensors, quaternions, impulses.

A. Reading order

1. Read the primary assignment first: Read the rigid-body chapters of the Physics-Based Simulation book and Baraff-style rigid-body/contact course material; derive world-space inertia and impulse response before using an engine.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Stanford CS348C rigid-body/contact lectures.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Euler equations intuition
- world/body inertia transforms

- impulse-momentum
- Precision requirement: distinguish and justify momentum-based free rigid motion and collision response checked for energy/momentum/orientation drift. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive translational and rotational equations, inertia-tensor transforms, and quaternion orientation updates; verify conservation laws in torque-free motion.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use tall stacks, gyroscopic motion, off-center impulses, and long resting periods; track momentum, orientation drift, and constraint error. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement 2D then 3D rigid bodies; verify conservation in collision-free motion. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a unit cube of mass 2, derive diagonal inertia about its center. Apply force (0,1,0) at point (1,0,0), compute torque, linear acceleration, and initial angular acceleration. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use tall stacks, gyroscopic motion, off-center impulses, and long resting periods; track momentum, orientation drift, and constraint error. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Add a free-rotation test and two-body collision test; monitor linear/angular momentum, energy, and rotation-matrix/quaternion norm drift.
- P5 - Machine-readable evidence. Export linear/angular momentum drift, energy drift, orientation norm/orthogonality error, penetration after impacts, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: momentum-based free rigid motion and collision response checked for energy/momentum/orientation drift. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use tall stacks, gyroscopic motion, off-center impulses, and long resting periods; track momentum, orientation drift, and constraint error." without hand-editing code between methods. Record linear/angular momentum drift, energy drift, orientation norm/orthogonality error, penetration after impacts, and runtime.
- P7 - Deliberate bug and regression drill. Intentionally apply the body-space inertia tensor directly in world coordinates without rotating it. Before running, predict the first observable failure in linear/angular momentum drift, energy drift, orientation norm/orthogonality error, penetration after impacts, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive translational and rotational equations, inertia-tensor transforms, and quaternion orientation updates; verify conservation laws in torque-free motion. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a unit cube of mass 2, derive diagonal inertia about its center. Apply force (0,1,0) at point (1,0,0), compute torque, linear acceleration, and initial angular acceleration. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement 2D then 3D rigid bodies; verify conservation in collision-free motion. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Add a free-rotation test and two-body collision test; monitor linear/angular momentum, energy, and rotation-matrix/quaternion norm drift. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare momentum-based free rigid motion and collision response checked for energy/momentum/orientation drift. Use the same material parameters, geometry, timestep, mesh resolution and nonlinear/linear solver tolerances; if wall time is compared, report the accuracy/stability change at that cost. Record

linear/angular momentum drift, energy drift, orientation norm/orthogonality error, penetration after impacts, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use tall stacks, gyroscopic motion, off-center impulses, and long resting periods; track momentum, orientation drift, and constraint error. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: CS348C rigid-body notes and readings. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Rigid body dynamics. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among momentum-based free rigid motion and collision response checked for energy/momentum/orientation drift. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using linear/angular momentum drift, energy drift, orientation norm/orthogonality error, penetration after impacts, and runtime.

Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if momentum-based free rigid motion and collision response checked for energy/momentum/orientation drift improve linear/angular momentum drift, energy drift, orientation norm/orthogonality error, penetration after impacts, and runtime after timestep, mesh and solver-tolerance checks and preserve the physical invariants/constraints relevant to the model; visual smoothness alone is insufficient.

F. Concrete research project - Rigid body dynamics

F1. Falsifiable question. Under the controlled failure regime "Use tall stacks, gyroscopic motion, off-center impulses, and long resting periods; track momentum, orientation drift, and constraint error.", characterize the ranking or regime transition across momentum-based free rigid motion and collision response checked for energy/momentum/orientation drift. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Add a free-rotation test and two-body collision test; monitor linear/angular momentum, energy, and rotation-matrix/quaternion norm drift. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a one-element/one-contact oracle, one canonical benchmark, and one large-deformation/contact stress case; run at least four time steps and three spatial/solver resolutions where meaningful. The central adversarial condition is: Use tall stacks, gyroscopic motion, off-center impulses, and long resting periods; track momentum, orientation drift, and constraint error.

F4. Required evidence. Measure linear/angular momentum drift, energy drift, orientation norm/orthogonality error, penetration after impacts, and runtime. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Rigid body dynamics. If a stable failure/crossover is found among momentum-based free rigid motion and collision response checked for energy/momentum/orientation drift, implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Rigid body dynamics. Using the measured linear/angular momentum drift, energy drift, orientation norm/orthogonality error, penetration after impacts, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Rigid body dynamics

- G1. Closed-book derivation/oracle. Derive translational and rotational equations, inertia-tensor transforms, and quaternion orientation updates; verify conservation laws in torque-free motion. Then solve this exact micro-case from scratch without opening your implementation: For a unit cube of mass 2, derive diagonal inertia about its center. Apply force (0,1,0) at point (1,0,0), compute torque, linear acceleration, and initial angular acceleration. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement 2D then 3D rigid bodies; verify conservation in collision-free motion. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally apply the body-space inertia tensor directly in world coordinates without rotating it. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Add a free-rotation test and two-body collision test; monitor linear/angular momentum, energy, and rotation-matrix/quaternion norm drift. Then apply this stress without retuning: Use tall stacks, gyroscopic motion, off-center impulses, and long resting periods; track momentum, orientation drift, and constraint error. Report linear/angular momentum drift, energy drift, orientation norm/orthogonality error, penetration after impacts, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across momentum-based free rigid motion and collision response checked for energy/momentum/orientation drift for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured linear/angular momentum drift, energy drift, orientation norm/orthogonality error, penetration after impacts, and runtime; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Rigid body dynamics. Using linear/angular momentum drift, energy drift, orientation norm/orthogonality error, penetration after impacts, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 10, Unit 4. Collision detection

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Broad phase, narrow phase, continuous collision detection, signed distance/contact geometry.

A. Reading order

1. Read the primary assignment first: CS348C collision material; Physics-Based Simulation collision chapters.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Collision-processing assignments from graphics courses.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Separating axis intuition
- time of impact
- conservative advancement concept
- Precision requirement: distinguish and justify brute-force narrow phase vs spatial hash/BVH broad phase; discrete vs continuous tests for tunneling cases. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive continuous collision time for two simple moving primitives, explain broad/narrow phases, and analyze robustness of triangle/edge/vertex proximity predicates.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use tunneling/high speeds, nearly coplanar triangles, thin objects, and many object size scales; report missed and false contacts. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement sphere/box/triangle collision set; build spatial hash/BVH broad phase. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: A point moves from $(-1,0)$ to $(1,0)$ in one step toward a vertical segment at $x=0$. Compute discrete start/end collision status and continuous time of impact; show why endpoint-only testing tunnels. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use tunneling/high speeds, nearly coplanar triangles, thin objects, and many object size scales; report missed and false contacts. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build randomized CCD tunneling tests and a broad-phase scaling benchmark with dense, sparse, and clustered objects.
- P5 - Machine-readable evidence. Export miss/false-positive rate, time of impact error, tunneling rate, broad/narrow-phase time, and scaling with object count to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: brute-force narrow phase vs spatial hash/BVH broad phase; discrete vs continuous tests for tunneling cases. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use tunneling/high speeds, nearly coplanar triangles, thin objects, and many object size scales; report missed and false contacts." without hand-editing code between methods. Record miss/false-positive rate, time of impact error, tunneling rate, broad/narrow-phase time, and scaling with object count.
- P7 - Deliberate bug and regression drill. Intentionally use discrete collision detection only and treat high-speed tunneling as a contact-solver failure. Before running, predict the first observable failure in miss/false-positive rate, time of impact error, tunneling rate, broad/narrow-phase time, and scaling with object count. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive continuous collision time for two simple moving primitives, explain broad/narrow phases, and analyze robustness of triangle/edge/vertex proximity predicates. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. A point moves from $(-1,0)$ to $(1,0)$ in one step toward a vertical segment at $x=0$. Compute discrete start/end collision status and continuous time of impact; show why endpoint-only testing tunnels. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement sphere/box/triangle collision set; build spatial hash/BVH broad phase. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build randomized CCD tunneling tests and a broad-phase scaling benchmark with dense, sparse, and clustered objects. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare brute-force narrow phase vs spatial hash/BVH broad phase; discrete vs continuous tests for tunneling cases. Use the same material parameters, geometry, timestep, mesh resolution and nonlinear/linear solver tolerances; if wall time is compared, report the accuracy/stability change at that cost. Record miss/false-positive rate, time of impact error, tunneling rate, broad/narrow-phase time, and scaling with object count. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use tunneling/high speeds, nearly coplanar triangles, thin objects, and many object size scales; report missed and false contacts. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: CS348C collision material; Physics-Based Simulation collision chapters. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Collision detection. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among brute-force narrow phase vs spatial hash/BVH broad phase; discrete vs continuous tests for tunneling cases. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using miss/false-positive rate, time of impact error, tunneling rate, broad/narrow-phase time, and scaling with object count. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if brute-force narrow phase vs spatial hash/BVH broad phase; discrete vs continuous tests for tunneling cases improve miss/false-positive rate, time of impact error, tunneling rate, broad/narrow-phase time, and scaling with object count after timestep, mesh and solver-tolerance checks and preserve the physical invariants/constraints relevant to the model; visual smoothness alone is insufficient.

F. Concrete research project - Collision detection

F1. Falsifiable question. Under the controlled failure regime "Use tunneling/high speeds, nearly coplanar triangles, thin objects, and many object size scales; report missed and false contacts.", which alternative in brute-force narrow phase vs spatial hash/BVH broad phase; discrete vs continuous tests for tunneling cases is preferable, and is the change explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build randomized CCD tunneling tests and a broad-phase scaling benchmark with dense, sparse, and clustered objects. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a one-element/one-contact oracle, one canonical benchmark, and one large-deformation/contact stress case; run at least four time steps and three spatial/solver resolutions where meaningful. The central adversarial condition is: Use tunneling/high speeds, nearly coplanar triangles, thin objects, and many object size scales; report missed and false contacts.

F4. Required evidence. Measure miss/false-positive rate, time of impact error, tunneling rate, broad/narrow-phase time, and scaling with object count. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Collision detection. If a stable failure/crossover is found among brute-force narrow phase vs spatial hash/BVH broad phase; discrete vs continuous tests for tunneling cases, implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Collision detection. Using the measured miss/false-positive rate, time of impact error, tunneling rate, broad/narrow-phase time, and scaling with object count, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Collision detection

- G1. Closed-book derivation/oracle. Derive continuous collision time for two simple moving primitives, explain broad/narrow phases, and analyze robustness of triangle/edge/vertex proximity predicates. Then solve this exact micro-case from scratch without opening your implementation: A point moves from (-1,0) to (1,0) in one step toward a vertical segment at $x=0$. Compute discrete start/end collision status and continuous time of impact; show why endpoint-only testing tunnels. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement sphere/box/triangle collision set; build spatial hash/BVH broad phase. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use discrete collision detection only and treat high-speed tunneling as a contact-solver failure. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.

- G4. Transfer experiment. On an input/configuration not used during development, execute: Build randomized CCD tunneling tests and a broad-phase scaling benchmark with dense, sparse, and clustered objects. Then apply this stress without retuning: Use tunneling/high speeds, nearly coplanar triangles, thin objects, and many object size scales; report missed and false contacts. Report miss/false-positive rate, time of impact error, tunneling rate, broad/narrow-phase time, and scaling with object count and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among brute-force narrow phase vs spatial hash/BVH broad phase; discrete vs continuous tests for tunneling cases for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured miss/false-positive rate, time of impact error, tunneling rate, broad/narrow-phase time, and scaling with object count; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Collision detection. Using miss/false-positive rate, time of impact error, tunneling rate, broad/narrow-phase time, and scaling with object count, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 10, Unit 5. Contact and friction

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Complementarity, impulses, Coulomb friction, penalty/barrier/constraint formulations.

A. Reading order

1. Read the primary assignment first: CS348C contact readings; Physics-Based Simulation IPC/contact chapters.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Stanford CS348C current contact lecture references.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Signorini conditions
- friction cone
- KKT connection
- maximal dissipation intuition
- Precision requirement: distinguish and justify impulse-based contact vs constraint/energy-based formulations on stacks, friction ramps, and resting-contact jitter. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a complementarity or projected-impulse formulation for unilateral contact with Coulomb friction; work through a single block-on-plane example including stick/slip transition.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use stacking, wedging, high friction, shallow contacts, and large timesteps; report penetration, convergence, and sticking/sliding correctness. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement simple impulse-based stacking; then study an IPC example and compare failure cases. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: A block of mass 1 rests on a 30-degree incline with $\mu=0.4$. Compute normal force, required static friction, and whether sticking is feasible; then increase angle until sliding begins. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use stacking, wedging, high friction, shallow contacts, and large timesteps; report penetration, convergence, and sticking/sliding correctness. Save the input and config of each qualitatively different failure as a named regression case.

- P4 - Research-grade programming extension. Run a box-on-incline friction test, resting stack, and impact stack; report penetration, jitter, energy, and static/sliding threshold error.
- P5 - Machine-readable evidence. Export penetration, resting jitter, friction threshold error, energy change, stack stability duration, and iterations/time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: impulse-based contact vs constraint/energy-based formulations on stacks, friction ramps, and resting-contact jitter. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use stacking, wedging, high friction, shallow contacts, and large timesteps; report penetration, convergence, and sticking/sliding correctness." without hand-editing code between methods. Record penetration, resting jitter, friction threshold error, energy change, stack stability duration, and iterations/time.
- P7 - Deliberate bug and regression drill. Intentionally use an incorrect normal-impulse sign or clamp friction independently of the Coulomb cone. Before running, predict the first observable failure in penetration, resting jitter, friction threshold error, energy change, stack stability duration, and iterations/time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a complementarity or projected-impulse formulation for unilateral contact with Coulomb friction; work through a single block-on-plane example including stick/slip transition. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. A block of mass 1 rests on a 30-degree incline with $\mu=0.4$. Compute normal force, required static friction, and whether sticking is feasible; then increase angle until sliding begins. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement simple impulse-based stacking; then study an IPC example and compare failure cases. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Run a box-on-incline friction test, resting stack, and impact stack; report penetration, jitter, energy, and static/sliding threshold error. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare impulse-based contact vs constraint/energy-based formulations on stacks, friction ramps, and resting-contact jitter. Use the same material parameters, geometry, timestep, mesh resolution and nonlinear/linear solver tolerances; if wall time is compared, report the accuracy/stability change at that cost. Record penetration, resting jitter, friction threshold error, energy change, stack stability duration, and iterations/time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use stacking, wedging, high friction, shallow contacts, and large timesteps; report penetration, convergence, and sticking/sliding correctness. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: CS348C contact readings; Physics-Based Simulation IPC/contact chapters. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Contact and friction. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among impulse-based contact vs constraint/energy-based formulations on stacks, friction ramps, and resting-contact jitter. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using penetration, resting jitter, friction threshold error, energy change, stack stability duration, and iterations/time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if impulse-based contact vs constraint/energy-based formulations on stacks, friction ramps, and resting-contact jitter improve penetration, resting jitter, friction threshold error, energy change, stack stability duration, and iterations/time after timestep, mesh and solver-tolerance checks and preserve the physical invariants/constraints relevant to the model; visual smoothness alone is insufficient.

F. Concrete research project - Contact and friction

F1. Falsifiable question. Under the controlled failure regime "Use stacking, wedging, high friction, shallow contacts, and large timesteps; report penetration, convergence, and sticking/sliding correctness.", which alternative in impulse-based contact vs constraint/energy-based formulations on stacks, friction ramps, and resting-contact jitter is preferable, and is the change explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Run a box-on-incline friction test, resting stack, and impact stack; report penetration, jitter, energy, and static/sliding threshold error. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a one-element/one-contact oracle, one canonical benchmark, and one large-deformation/contact stress case; run at least four time steps and three spatial/solver resolutions where meaningful. The central adversarial condition is: Use stacking, wedging, high friction, shallow contacts, and large timesteps; report penetration, convergence, and sticking/sliding correctness.

F4. Required evidence. Measure penetration, resting jitter, friction threshold error, energy change, stack stability duration, and iterations/time. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Contact and friction. If a stable failure/crossover is found among impulse-based contact vs constraint/energy-based formulations on stacks, friction ramps, and resting-contact jitter, implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Contact and friction. Using the measured penetration, resting jitter, friction threshold error, energy change, stack stability duration, and iterations/time, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Contact and friction

- G1. Closed-book derivation/oracle. Derive a complementarity or projected-impulse formulation for unilateral contact with Coulomb friction; work through a single block-on-plane example including stick/slip transition. Then solve this exact micro-case from scratch without opening your implementation: A block of mass 1 rests on a 30-degree incline with $\mu=0.4$. Compute normal force, required static friction, and whether sticking is feasible; then increase angle until sliding begins. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement simple impulse-based stacking; then study an IPC example and compare failure cases. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use an incorrect normal-impulse sign or clamp friction independently of the Coulomb cone. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Run a box-on-incline friction test, resting stack, and impact stack; report penetration, jitter, energy, and static/sliding threshold error. Then apply this stress without retuning: Use stacking, wedging, high friction, shallow contacts, and large timesteps; report penetration, convergence, and sticking/sliding correctness. Report penetration, resting jitter, friction threshold error, energy change, stack stability duration, and iterations/time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among impulse-based contact vs constraint/energy-based formulations on stacks, friction ramps, and resting-contact jitter for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained

case of your choice. For each choice, cite the measured penetration, resting jitter, friction threshold error, energy change, stack stability duration, and iterations/time; state one counterexample that would reverse your recommendation.

- G6. Pass condition for Contact and friction. Using penetration, resting jitter, friction threshold error, energy change, stack stability duration, and iterations/time, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 10, Unit 6. Finite elements for deformable solids

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Deformation gradient, strain, stress, hyperelastic energy, tetrahedral FEM.

A. Reading order

1. Read the primary assignment first: Physics-Based Simulation FEM chapters; Sifakis & Barbic FEM course notes.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: FEM tutorial code from simulation book.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Weak form
- shape functions
- Neo-Hookean/corotational energy
- assembly
- Precision requirement: distinguish and justify linear vs corotated vs Neo-Hookean-style elements under patch, rigid-motion, near-incompressible, and large-deformation tests. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the weak form for linear elasticity, triangle/tetrahedron shape functions, element stiffness/forces, and rigid-motion invariance; then connect to a nonlinear hyperelastic energy.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use near-incompressibility, element inversion, large shear, and mesh refinement; compare stress/energy and solver robustness. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement 2D triangle FEM then small 3D tet FEM; verify patch test/rigid-motion invariance. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For the reference triangle (0,0), (1,0),(0,1) with a linear displacement field $u=(0.1x,0.2y)$, compute deformation gradient/strain and verify a constant-strain triangle reproduces it exactly in a patch test. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use near-incompressibility, element inversion, large shear, and mesh refinement; compare stress/energy and solver robustness. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement 2D triangle FEM with numerical/analytic force checks, patch test, rigid-rotation test, and a compression case exposing locking/inversion.
- P5 - Machine-readable evidence. Export patch/rigid-motion residual, force-gradient check error, displacement/stress error, convergence, inversion/locking rate, and solver iterations to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: linear vs corotated vs Neo-Hookean-style elements under patch, rigid-motion, near-incompressible, and large-deformation tests. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use near-incompressibility, element inversion, large shear, and mesh refinement; compare stress/energy and solver robustness." without hand-editing code between methods. Record patch/rigid-motion

residual, force-gradient check error, displacement/stress error, convergence, inversion/locking rate, and solver iterations.

- P7 - Deliberate bug and regression drill. Intentionally assemble a strain-displacement matrix with an orientation error and fail to reject/handle inverted elements. Before running, predict the first observable failure in patch/rigid-motion residual, force-gradient check error, displacement/stress error, convergence, inversion/locking rate, and solver iterations. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the weak form for linear elasticity, triangle/tetrahedron shape functions, element stiffness/forces, and rigid-motion invariance; then connect to a nonlinear hyperelastic energy. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For the reference triangle $(0,0),(1,0),(0,1)$ with a linear displacement field $u=(0.1x,0.2y)$, compute deformation gradient/strain and verify a constant-strain triangle reproduces it exactly in a patch test. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement 2D triangle FEM then small 3D tet FEM; verify patch test/rigid-motion invariance. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement 2D triangle FEM with numerical/analytic force checks, patch test, rigid-rotation test, and a compression case exposing locking/inversion. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare linear vs corotated vs Neo-Hookean-style elements under patch, rigid-motion, near-incompressible, and large-deformation tests. Use the same material parameters, geometry, timestep, mesh resolution and nonlinear/linear solver tolerances; if wall time is compared, report the accuracy/stability change at that cost. Record patch/rigid-motion residual, force-gradient check error, displacement/stress error, convergence, inversion/locking rate, and solver iterations. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use near-incompressibility, element inversion, large shear, and mesh refinement; compare stress/energy and solver robustness. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Physics-Based Simulation FEM chapters; Sifakis & Barbic FEM course notes. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Finite elements for deformable solids. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among linear vs corotated vs Neo-Hookean-style elements under patch, rigid-motion, near-incompressible, and large-deformation tests. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using patch/rigid-motion residual, force-gradient check error, displacement/stress error, convergence, inversion/locking rate, and solver iterations. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the constitutive/discretization change improves patch/convergence/inversion or near-incompressible behavior after mesh and solver tolerances are controlled, with the failure mechanism identified.

F. Concrete research project - Finite elements for deformable solids

- F1. Falsifiable question. Under the controlled failure regime "Use near-incompressibility, element inversion, large shear, and mesh refinement; compare stress/energy and solver robustness.", which alternative in linear vs corotated vs Neo-Hookean-style elements under patch, rigid-motion, near-incompressible, and large-deformation tests is preferable, and is the change explained by weak-form/element consistency, constitutive conditioning, locking/inversion, temporal integration, and nonlinear-solver convergence? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement 2D triangle FEM with numerical/analytic force checks, patch test, rigid-rotation test, and a compression case exposing locking/inversion. Add

a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a one-element/one-contact oracle, one canonical benchmark, and one large-deformation/contact stress case; run at least four time steps and three spatial/solver resolutions where meaningful. The central adversarial condition is: Use near-incompressibility, element inversion, large shear, and mesh refinement; compare stress/energy and solver robustness.

F4. Required evidence. Measure patch/rigid-motion residual, force-gradient check error, displacement/stress error, convergence, inversion/locking rate, and solver iterations. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Finite elements for deformable solids. If a stable failure/crossover is found among linear vs corotated vs Neo-Hookean-style elements under patch, rigid-motion, near-incompressible, and large-deformation tests, implement one adaptive or corrected variant that directly targets weak-form/element consistency, constitutive conditioning, locking/inversion, temporal integration, and nonlinear-solver convergence; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Finite elements for deformable solids. Using the measured patch/rigid-motion residual, force-gradient check error, displacement/stress error, convergence, inversion/locking rate, and solver iterations, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Finite elements for deformable solids

- G1. Closed-book derivation/oracle. Derive the weak form for linear elasticity, triangle/tetrahedron shape functions, element stiffness/forces, and rigid-motion invariance; then connect to a nonlinear hyperelastic energy. Then solve this exact micro-case from scratch without opening your implementation: For the reference triangle (0,0),(1,0),(0,1) with a linear displacement field $u=(0.1x,0.2y)$, compute deformation gradient/strain and verify a constant-strain triangle reproduces it exactly in a patch test. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement 2D triangle FEM then small 3D tet FEM; verify patch test/rigid-motion invariance. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally assemble a strain-displacement matrix with an orientation error and fail to reject/handle inverted elements. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement 2D triangle FEM with numerical/analytic force checks, patch test, rigid-rotation test, and a compression case exposing locking/inversion. Then apply this stress without retuning: Use near-incompressibility, element inversion, large shear, and mesh refinement; compare stress/energy and solver robustness. Report patch/rigid-motion residual, force-gradient check error, displacement/stress error, convergence, inversion/locking rate, and solver iterations and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among linear vs corotated vs Neo-Hookean-style elements under patch, rigid-motion, near-incompressible, and large-deformation tests for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured patch/rigid-motion residual, force-gradient check error, displacement/stress error, convergence, inversion/locking rate, and solver iterations; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Finite elements for deformable solids. Using patch/rigid-motion residual, force-gradient check error, displacement/stress error, convergence, inversion/locking rate, and solver iterations, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 10, Unit 7. Thin shells and rods

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Discrete shells, bending energies, rods, cloth, anisotropy.

A. Reading order

1. Read the primary assignment first: Read Grinspun et al., Discrete Shells (SCA 2003), and Bergou et al., Discrete Elastic Rods (SIGGRAPH 2008), together with the relevant Physics-Based Simulation sections.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Reproduce cloth or rod example.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Membrane vs bending energy
- curvature discretization
- rod frames
- Precision requirement: distinguish and justify hinge-based bending/discrete rods across mesh refinement, thickness/stiffness, timestep, and large curvature. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive discrete stretching and bending energies for a thin strip/shell or elastic rod; identify invariances and the scaling of bending versus stretching with thickness.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use coarse discretizations, large bending/twist, and anisotropic stiffness; compare with beam/shell analytic or highly refined references. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement a discrete rod or hinge-based cloth bending model and characterize timestep/mesh dependence. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For three equally spaced rod vertices forming a small angle θ at the middle, compute a simple discrete bending energy proportional to θ^2 . Compare energy before and after halving segment length while representing the same curvature. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use coarse discretizations, large bending/twist, and anisotropic stiffness; compare with beam/shell analytic or highly refined references. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Run cantilever/rod-bending tests under refinement and compare deflection/energy across element length and timestep.
- P5 - Machine-readable evidence. Export deflection/curvature/energy convergence, timestep sensitivity, mesh dependence, self-contact error, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: hinge-based bending/discrete rods across mesh refinement, thickness/stiffness, timestep, and large curvature. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use coarse discretizations, large bending/twist, and anisotropic stiffness; compare with beam/shell analytic or highly refined references." without hand-editing code between methods. Record deflection/curvature/energy convergence, timestep sensitivity, mesh dependence, self-contact error, and runtime.
- P7 - Deliberate bug and regression drill. Intentionally mix current and rest curvature/frame conventions, creating spurious bending in the undeformed configuration. Before running, predict the first observable failure in deflection/curvature/energy convergence, timestep sensitivity, mesh dependence, self-contact error, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive discrete stretching and bending energies for a thin strip/shell or elastic rod; identify invariances and the scaling of bending versus stretching with thickness. Then evaluate the result on the explicit micro-

case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. For three equally spaced rod vertices forming a small angle θ at the middle, compute a simple discrete bending energy proportional to θ^2 . Compare energy before and after halving segment length while representing the same curvature. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Implement a discrete rod or hinge-based cloth bending model and characterize timestep/mesh dependence. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Run cantilever/rod-bending tests under refinement and compare deflection/energy across element length and timestep. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare hinge-based bending/discrete rods across mesh refinement, thickness/stiffness, timestep, and large curvature. Use the same material parameters, geometry, timestep, mesh resolution and nonlinear/linear solver tolerances; if wall time is compared, report the accuracy/stability change at that cost. Record deflection/curvature/energy convergence, timestep sensitivity, mesh dependence, self-contact error, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use coarse discretizations, large bending/twist, and anisotropic stiffness; compare with beam/shell analytic or highly refined references. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: CS348C thin-shell/character FX references; recent SIGGRAPH papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Thin shells and rods. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among hinge-based bending/discrete rods across mesh refinement, thickness/stiffness, timestep, and large curvature. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using deflection/curvature/energy convergence, timestep sensitivity, mesh dependence, self-contact error, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if hinge-based bending/discrete rods across mesh refinement, thickness/stiffness, timestep, and large curvature improve deflection/curvature/energy convergence, timestep sensitivity, mesh dependence, self-contact error, and runtime after timestep, mesh and solver-tolerance checks and preserve the physical invariants/constraints relevant to the model; visual smoothness alone is insufficient.

F. Concrete research project - Thin shells and rods

F1. Falsifiable question. Under the controlled failure regime "Use coarse discretizations, large bending/twist, and anisotropic stiffness; compare with beam/shell analytic or highly refined references.", characterize the ranking or regime transition across hinge-based bending/discrete rods across mesh refinement, thickness/stiffness, timestep, and large curvature. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Run cantilever/rod-bending tests under refinement and compare deflection/energy across element length and timestep. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a one-element/one-contact oracle, one canonical benchmark, and one large-deformation/contact stress case; run at least four time steps and three spatial/solver resolutions where meaningful. The central adversarial condition is: Use coarse discretizations, large bending/twist, and anisotropic stiffness; compare with beam/shell analytic or highly refined references.

F4. Required evidence. Measure deflection/curvature/energy convergence, timestep sensitivity, mesh dependence, self-contact error, and runtime. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include

one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Thin shells and rods. If a stable failure/crossover is found among hinge-based bending/discrete rods across mesh refinement, thickness/stiffness, timestep, and large curvature, implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Thin shells and rods. Using the measured deflection/curvature/energy convergence, timestep sensitivity, mesh dependence, self-contact error, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Thin shells and rods

- G1. Closed-book derivation/oracle. Derive discrete stretching and bending energies for a thin strip/shell or elastic rod; identify invariances and the scaling of bending versus stretching with thickness. Then solve this exact micro-case from scratch without opening your implementation: For three equally spaced rod vertices forming a small angle θ at the middle, compute a simple discrete bending energy proportional to θ^2 . Compare energy before and after halving segment length while representing the same curvature. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement a discrete rod or hinge-based cloth bending model and characterize timestep/mesh dependence. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally mix current and rest curvature/frame conventions, creating spurious bending in the undeformed configuration. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Run cantilever/rod-bending tests under refinement and compare deflection/energy across element length and timestep. Then apply this stress without retuning: Use coarse discretizations, large bending/twist, and anisotropic stiffness; compare with beam/shell analytic or highly refined references. Report deflection/curvature/energy convergence, timestep sensitivity, mesh dependence, self-contact error, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across hinge-based bending/discrete rods across mesh refinement, thickness/stiffness, timestep, and large curvature for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured deflection/curvature/energy convergence, timestep sensitivity, mesh dependence, self-contact error, and runtime; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Thin shells and rods. Using deflection/curvature/energy convergence, timestep sensitivity, mesh dependence, self-contact error, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 10, Unit 8. Fracture and topology change

Recommended time. 2.2-2.8 weeks (about 34-55 focused hours depending on background).

What you are learning. Failure criteria, remeshing, cutting, fracture modeling.

A. Reading order

1. Read the primary assignment first: CS348C fracture references; simulation book advanced sections.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Paper reproduction rather than full from-scratch implementation.

3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Stress/strain-based criteria
- energy release intuition
- Precision requirement: distinguish and justify fixed-topology elasticity vs cutting/fracture update, checking energy release, mesh quality, and collision robustness after topology changes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive an energy/stress-based fracture criterion for a toy discretization and reason about how splitting changes connectivity, mass, energy, and collision handling.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Repeat the same loading under mesh refinement and mesh rotation; check whether crack path and energy release converge rather than follow element edges. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Reproduce simplified brittle fracture or mesh-cutting example; document robustness challenges. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a chain of four springs with identical stiffness and one spring at twice the strain threshold, compute stored energies and determine which element breaks first under displacement control; then update connectivity. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Repeat the same loading under mesh refinement and mesh rotation; check whether crack path and energy release converge rather than follow element edges. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Track crack path, released elastic energy, new element quality, and collision failures after each topology update on a controlled notched specimen.
- P5 - Machine-readable evidence. Export crack-path consistency, energy release, element-quality degradation, collision failure rate, and runtime/topology update cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: fixed-topology elasticity vs cutting/fracture update, checking energy release, mesh quality, and collision robustness after topology changes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Repeat the same loading under mesh refinement and mesh rotation; check whether crack path and energy release converge rather than follow element edges." without hand-editing code between methods. Record crack-path consistency, energy release, element-quality degradation, collision failure rate, and runtime/topology update cost.
- P7 - Deliberate bug and regression drill. Intentionally change connectivity without consistently updating mass, collision structures, or released-energy accounting. Before running, predict the first observable failure in crack-path consistency, energy release, element-quality degradation, collision failure rate, and runtime/topology update cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive an energy/stress-based fracture criterion for a toy discretization and reason about how splitting changes connectivity, mass, energy, and collision handling. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a chain of four springs with identical stiffness and one spring at twice the strain threshold, compute stored energies and determine which element breaks first under displacement control; then update connectivity. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Reproduce simplified brittle fracture or mesh-cutting example; document robustness challenges. Before optimizing, make the smallest case agree with Problem 2. Then complete this

deeper task: Track crack path, released elastic energy, new element quality, and collision failures after each topology update on a controlled notched specimen. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare fixed-topology elasticity vs cutting/fracture update, checking energy release, mesh quality, and collision robustness after topology changes. Use the same material parameters, geometry, timestep, mesh resolution and nonlinear/linear solver tolerances; if wall time is compared, report the accuracy/stability change at that cost. Record crack-path consistency, energy release, element-quality degradation, collision failure rate, and runtime/topology update cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Repeat the same loading under mesh refinement and mesh rotation; check whether crack path and energy release converge rather than follow element edges. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: CS348C fracture references; simulation book advanced sections. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Fracture and topology change. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among fixed-topology elasticity vs cutting/fracture update, checking energy release, mesh quality, and collision robustness after topology changes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using crack-path consistency, energy release, element-quality degradation, collision failure rate, and runtime/topology update cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if fixed-topology elasticity vs cutting/fracture update, checking energy release, mesh quality, and collision robustness after topology changes improve crack-path consistency, energy release, element-quality degradation, collision failure rate, and runtime/topology update cost after timestep, mesh and solver-tolerance checks and preserve the physical invariants/constraints relevant to the model; visual smoothness alone is insufficient.

F. Concrete research project - Fracture and topology change

F1. Falsifiable question. Under the controlled failure regime "Repeat the same loading under mesh refinement and mesh rotation; check whether crack path and energy release converge rather than follow element edges.", which alternative in fixed-topology elasticity vs cutting/fracture update, checking energy release, mesh quality, and collision robustness after topology changes is preferable, and is the change explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Track crack path, released elastic energy, new element quality, and collision failures after each topology update on a controlled notched specimen. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a one-element/one-contact oracle, one canonical benchmark, and one large-deformation/contact stress case; run at least four time steps and three spatial/solver resolutions where meaningful. The central adversarial condition is: Repeat the same loading under mesh refinement and mesh rotation; check whether crack path and energy release converge rather than follow element edges.

F4. Required evidence. Measure crack-path consistency, energy release, element-quality degradation, collision failure rate, and runtime/topology update cost. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Fracture and topology change. If a stable failure/crossover is found among fixed-topology elasticity vs cutting/fracture update, checking energy release, mesh quality, and collision robustness after topology changes, implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Fracture and topology change. Using the measured crack-path consistency, energy release, element-quality degradation, collision failure rate, and runtime/topology update cost, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Fracture and topology change

- G1. Closed-book derivation/oracle. Derive an energy/stress-based fracture criterion for a toy discretization and reason about how splitting changes connectivity, mass, energy, and collision handling. Then solve this exact micro-case from scratch without opening your implementation: For a chain of four springs with identical stiffness and one spring at twice the strain threshold, compute stored energies and determine which element breaks first under displacement control; then update connectivity. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Reproduce simplified brittle fracture or mesh-cutting example; document robustness challenges. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally change connectivity without consistently updating mass, collision structures, or released-energy accounting. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Track crack path, released elastic energy, new element quality, and collision failures after each topology update on a controlled notched specimen. Then apply this stress without retuning: Repeat the same loading under mesh refinement and mesh rotation; check whether crack path and energy release converge rather than follow element edges. Report crack-path consistency, energy release, element-quality degradation, collision failure rate, and runtime/topology update cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among fixed-topology elasticity vs cutting/fracture update, checking energy release, mesh quality, and collision robustness after topology changes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured crack-path consistency, energy release, element-quality degradation, collision failure rate, and runtime/topology update cost; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Fracture and topology change. Using crack-path consistency, energy release, element-quality degradation, collision failure rate, and runtime/topology update cost, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 10 Closeout: Project, Review and Mastery Gate

Volume project. Build a simulation suite containing springs/cloth, rigid bodies, collision/contact and a small FEM deformable model. Provide conservation, stability and contact stress tests.

Cumulative review

- Run patch/invariance tests for integrators, rigid bodies, FEM and contact before large scenes.
- Produce timestep/spatial-refinement curves for at least two solid/contact benchmarks.
- Track energy, momentum, penetration/contact residual and inversion/strain diagnostics.
- Compare one explicit/projection-style method with one implicit/energy-based method under matched tolerances.
- Write a failure taxonomy separating constitutive model, discretization, contact model and nonlinear/linear solver effects.

Mastery gate before moving on

- Show timestep/mesh convergence or stability evidence for particles/constraints, rigid bodies, contact, FEM, and thin structures.
- Pass adversarial stacking/contact/inversion tests and identify whether failures come from model, discretization, or solver.

- Track meaningful energy/momentum/constraint quantities through long simulations and explain drift.
- Literature gate: reconstruct the numerical core of one cloth/contact paper and one FEM/rod/contact formulation; identify exactly how timestep, constitutive model, collision treatment and nonlinear solve enter the stability claim.

Volume 11. Physics-Based Simulation II: Fluids, Continuum Methods, MPM, and Multiphysics

Purpose. Learn grid, particle, and hybrid continuum simulation methods used in modern graphics research.

Suggested duration. 18-24 weeks

Prerequisites. Volumes 1, 2 and 10; continuum/PDE comfort is important.

Typical module pace. 2.2-3.0 weeks per module (8 modules total).

Primary references for this volume

- Bridson, Fluid Simulation for Computer Graphics (conceptual reference).
- Physics-Based Simulation open book, current fluid/MPM sections.
- Classic Stable Fluids, FLIP/APIC, PBF and MPM papers.

Online spine

Physics-Based Simulation open book: <https://phys-sim-book.github.io/>

Stanford CS348C Animation and Simulation: <https://graphics.stanford.edu/courses/cs348c/>

Canonical paper track

- Stam (1999), Stable Fluids.
- Zhu & Bridson (2005), Animating Sand as a Fluid (FLIP-style particle/grid transfer context).
- Jiang et al. (2015), The Affine Particle-In-Cell Method (APIC).
- Macklin & Muller (2013), Position Based Fluids.
- Jiang et al. (2016), The Material Point Method for Simulating Continuum Materials (SIGGRAPH course).
- Hu et al. (2018), A Moving Least Squares Material Point Method with Displacement Discontinuity and Two-Way Rigid Body Coupling.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Stam (1999), Stable Fluids.
- Foster & Fedkiw (2001), Practical Animation of Liquids.
- Enright et al. (2002), A Hybrid Particle Level Set Method for Improved Interface Capturing.
- Muller, Charypar & Gross (2003), Particle-Based Fluid Simulation for Interactive Applications.
- Jiang et al. (2015), The Affine Particle-In-Cell Method.
- Stomakhin et al. (2013), A Material Point Method for Snow Simulation.
- Bridson, Fluid Simulation for Computer Graphics + Physics-Based Simulation book - use for derivations and implementation details before frontier papers.

Volume 11, Unit 1. Fluid equations and discretization

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Incompressible Navier-Stokes, advection, pressure projection, boundary conditions.

A. Reading order

1. Read the primary assignment first: Bridson, Fluid Simulation for Computer Graphics, early chapters; Physics-Based Simulation fluid sections.

2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS184 fluid simulation lecture; CS348C fluids.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Material derivative
- divergence-free constraint
- Helmholtz-Hodge decomposition
- Poisson pressure solve
- Precision requirement: distinguish and justify collocated/simple-grid intuition vs MAC-grid projection, with discrete divergence/divergence/gradient/Poisson consistency checks. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive incompressible Navier-Stokes from momentum balance at the level needed for graphics, derive the pressure-projection method, and show why the Poisson solve enforces discrete divergence-free velocity.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use a manufactured or vortex-preservation flow; track divergence, energy, numerical diffusion, and convergence across grid refinements. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement 2D stable-fluid solver; visualize divergence before/after projection. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: On a 2x2 MAC grid, assign four face velocities so one cell has nonzero divergence. Write the discrete divergence and one pressure Poisson equation row; show how subtracting the pressure gradient removes divergence. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use a manufactured or vortex-preservation flow; track divergence, energy, numerical diffusion, and convergence across grid refinements. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Unit-test discrete gradient/divergence adjointness/signs and pressure projection; report max/mean divergence before and after solve.
- P5 - Machine-readable evidence. Export divergence before/after projection, Poisson residual, mass/energy behavior, convergence with grid/timestep, and solver iterations to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: collocated/simple-grid intuition vs MAC-grid projection, with discrete divergence/divergence/gradient/Poisson consistency checks. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use a manufactured or vortex-preservation flow; track divergence, energy, numerical diffusion, and convergence across grid refinements." without hand-editing code between methods. Record divergence before/after projection, Poisson residual, mass/energy behavior, convergence with grid/timestep, and solver iterations.
- P7 - Deliberate bug and regression drill. Intentionally use inconsistent gradient/divergence signs or boundary conditions in the pressure projection. Before running, predict the first observable failure in divergence before/after projection, Poisson residual, mass/energy behavior, convergence with grid/timestep, and solver iterations. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive incompressible Navier-Stokes from momentum balance at the level needed for graphics, derive the pressure-projection method, and show why the Poisson solve enforces discrete divergence-free velocity. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. On a 2x2 MAC grid, assign four face velocities so one cell has nonzero divergence. Write the discrete divergence and one pressure Poisson equation row; show how subtracting the pressure gradient

removes divergence. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Implement 2D stable-fluid solver; visualize divergence before/after projection. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Unit-test discrete gradient/divergence adjointness/signs and pressure projection; report max/mean divergence before and after solve. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare collocated/simple-grid intuition vs MAC-grid projection, with discrete divergence/gradient/Poisson consistency checks. Use the same domain, initial/boundary conditions, grid/particle resolution, timestep and pressure/solver tolerances; separate accuracy from speed by reporting both refinement behavior and wall time. Record divergence before/after projection, Poisson residual, mass/energy behavior, convergence with grid/timestep, and solver iterations. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use a manufactured or vortex-preservation flow; track divergence, energy, numerical diffusion, and convergence across grid refinements. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Bridson, Fluid Simulation for Computer Graphics, early chapters; Physics-Based Simulation fluid sections. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Fluid equations and discretization. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among collocated/simple-grid intuition vs MAC-grid projection, with discrete divergence/gradient/Poisson consistency checks. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using divergence before/after projection, Poisson residual, mass/energy behavior, convergence with grid/timestep, and solver iterations. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if collocated/simple-grid intuition vs MAC-grid projection, with discrete divergence/gradient/Poisson consistency checks improve divergence before/after projection, Poisson residual, mass/energy behavior, convergence with grid/timestep, and solver iterations under grid/particle and timestep refinement, with transfer/advection/pressure/coupling errors separated where possible; speed bought by uncontrolled dissipation is not progress.

F. Concrete research project - Fluid equations and discretization

F1. Falsifiable question. Under the controlled failure regime "Use a manufactured or vortex-preservation flow; track divergence, energy, numerical diffusion, and convergence across grid refinements.", which alternative in collocated/simple-grid intuition vs MAC-grid projection, with discrete divergence/gradient/Poisson consistency checks is preferable, and is the change explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Unit-test discrete gradient/divergence adjointness/signs and pressure projection; report max/mean divergence before and after solve. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a manufactured/analytic field test, one standard visual benchmark, and one high-CFL/free-surface/large-deformation stress case; run four time steps and at least three grid/particle resolutions. The central adversarial condition is: Use a manufactured or vortex-preservation flow; track divergence, energy, numerical diffusion, and convergence across grid refinements.

F4. Required evidence. Measure divergence before/after projection, Poisson residual, mass/energy behavior, convergence with grid/timestep, and solver iterations. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Fluid equations and discretization. If a stable failure/crossover is found among collocated/simple-grid intuition vs MAC-grid projection, with discrete divergence/gradient/Poisson consistency checks,

implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Fluid equations and discretization. Using the measured divergence before/after projection, Poisson residual, mass/energy behavior, convergence with grid/timestep, and solver iterations, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Fluid equations and discretization

- G1. Closed-book derivation/oracle. Derive incompressible Navier-Stokes from momentum balance at the level needed for graphics, derive the pressure-projection method, and show why the Poisson solve enforces discrete divergence-free velocity. Then solve this exact micro-case from scratch without opening your implementation: On a 2x2 MAC grid, assign four face velocities so one cell has nonzero divergence. Write the discrete divergence and one pressure Poisson equation row; show how subtracting the pressure gradient removes divergence. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement 2D stable-fluid solver; visualize divergence before/after projection. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use inconsistent gradient/divergence signs or boundary conditions in the pressure projection. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Unit-test discrete gradient/divergence adjointness/signs and pressure projection; report max/mean divergence before and after solve. Then apply this stress without retuning: Use a manufactured or vortex-preservation flow; track divergence, energy, numerical diffusion, and convergence across grid refinements. Report divergence before/after projection, Poisson residual, mass/energy behavior, convergence with grid/timestep, and solver iterations and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among collocated/simple-grid intuition vs MAC-grid projection, with discrete divergence/gradient/Poisson consistency checks for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured divergence before/after projection, Poisson residual, mass/energy behavior, convergence with grid/timestep, and solver iterations; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Fluid equations and discretization. Using divergence before/after projection, Poisson residual, mass/energy behavior, convergence with grid/timestep, and solver iterations, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 11, Unit 2. Eulerian smoke

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Semi-Lagrangian advection, staggered MAC grids, vorticity, buoyancy.

A. Reading order

1. Read the primary assignment first: Bridson grid fluid chapters.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Bridson/graphics course fluid tutorials.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- MAC discretization
- backtracing

- numerical dissipation
- Precision requirement: distinguish and justify semi-Lagrangian advection vs higher-order/less-dissipative variants while monitoring divergence, scalar loss, and vortical detail. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive semi-Lagrangian advection and pressure projection on a staggered MAC grid; analyze numerical dissipation and boundary-condition treatment.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Advect sharp dye/vorticity structures for many steps; separate advection diffusion from pressure-projection error. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement 2D smoke with MAC grid, pressure solve, buoyancy and vorticity confinement. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Advect a 1D scalar $[0,0,1,0,0]$ rightward by 0.5 cell with semi-Lagrangian interpolation. Compute the new samples and quantify numerical diffusion after two steps. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Advect sharp dye/vorticity structures for many steps; separate advection diffusion from pressure-projection error. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Run a rotating/advection benchmark and a buoyant plume; measure scalar mass, vorticity, divergence, and visual diffusion versus timestep/grid.
- P5 - Machine-readable evidence. Export scalar mass loss, divergence, vorticity/detail loss, advection error, and runtime per simulated second to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: semi-Lagrangian advection vs higher-order/less-dissipative variants while monitoring divergence, scalar loss, and vortical detail. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Advect sharp dye/vorticity structures for many steps; separate advection diffusion from pressure-projection error." without hand-editing code between methods. Record scalar mass loss, divergence, vorticity/detail loss, advection error, and runtime per simulated second.
- P7 - Deliberate bug and regression drill. Intentionally forward-advect samples instead of backward tracing/interpolating the semi-Lagrangian field. Before running, predict the first observable failure in scalar mass loss, divergence, vorticity/detail loss, advection error, and runtime per simulated second. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive semi-Lagrangian advection and pressure projection on a staggered MAC grid; analyze numerical dissipation and boundary-condition treatment. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Advect a 1D scalar $[0,0,1,0,0]$ rightward by 0.5 cell with semi-Lagrangian interpolation. Compute the new samples and quantify numerical diffusion after two steps. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement 2D smoke with MAC grid, pressure solve, buoyancy and vorticity confinement. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Run a rotating/advection benchmark and a buoyant plume; measure scalar mass, vorticity, divergence, and visual diffusion versus timestep/grid. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare semi-Lagrangian advection vs higher-order/less-dissipative variants while monitoring divergence, scalar loss, and vortical detail. Use the same domain, initial/boundary conditions, grid/particle resolution, timestep and pressure/solver tolerances; separate accuracy from speed by reporting both refinement behavior and wall time. Record scalar mass loss, divergence, vorticity/detail loss, advection error, and runtime per

simulated second. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Advect sharp dye/vorticity structures for many steps; separate advection diffusion from pressure-projection error. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Bridson grid fluid chapters. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Eulerian smoke. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among semi-Lagrangian advection vs higher-order/less-dissipative variants while monitoring divergence, scalar loss, and vortical detail. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using scalar mass loss, divergence, vorticity/detail loss, advection error, and runtime per simulated second. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if semi-Lagrangian advection vs higher-order/less-dissipative variants while monitoring divergence, scalar loss, and vortical detail improve scalar mass loss, divergence, vorticity/detail loss, advection error, and runtime per simulated second under grid/particle and timestep refinement, with transfer/advection/pressure/coupling errors separated where possible; speed bought by uncontrolled dissipation is not progress.

F. Concrete research project - Eulerian smoke

F1. Falsifiable question. Under the controlled failure regime "Advect sharp dye/vorticity structures for many steps; separate advection diffusion from pressure-projection error.", which alternative in semi-Lagrangian advection vs higher-order/less-dissipative variants while monitoring divergence, scalar loss, and vortical detail is preferable, and is the change explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Run a rotating/advection benchmark and a buoyant plume; measure scalar mass, vorticity, divergence, and visual diffusion versus timestep/grid. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a manufactured/analytic field test, one standard visual benchmark, and one high-CFL/free-surface/large-deformation stress case; run four time steps and at least three grid/particle resolutions. The central adversarial condition is: Advect sharp dye/vorticity structures for many steps; separate advection diffusion from pressure-projection error.

F4. Required evidence. Measure scalar mass loss, divergence, vorticity/detail loss, advection error, and runtime per simulated second. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Eulerian smoke. If a stable failure/crossover is found among semi-Lagrangian advection vs higher-order/less-dissipative variants while monitoring divergence, scalar loss, and vortical detail, implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Eulerian smoke. Using the measured scalar mass loss, divergence, vorticity/detail loss, advection error, and runtime per simulated second, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Eulerian smoke

- G1. Closed-book derivation/oracle. Derive semi-Lagrangian advection and pressure projection on a staggered MAC grid; analyze numerical dissipation and boundary-condition treatment. Then solve this exact micro-case from scratch without opening your implementation: Advect a 1D scalar $[0,0,1,0,0]$ rightward by 0.5 cell with semi-Lagrangian interpolation. Compute the new samples and quantify numerical diffusion after two steps. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement 2D smoke with MAC grid, pressure solve, buoyancy and vorticity confinement. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally forward-advect samples instead of backward tracing/interpolating the semi-Lagrangian field. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Run a rotating/advection benchmark and a buoyant plume; measure scalar mass, vorticity, divergence, and visual diffusion versus timestep/grid. Then apply this stress without retuning: Advect sharp dye/vorticity structures for many steps; separate advection diffusion from pressure-projection error. Report scalar mass loss, divergence, vorticity/detail loss, advection error, and runtime per simulated second and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among semi-Lagrangian advection vs higher-order/less-dissipative variants while monitoring divergence, scalar loss, and vortical detail for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured scalar mass loss, divergence, vorticity/detail loss, advection error, and runtime per simulated second; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Eulerian smoke. Using scalar mass loss, divergence, vorticity/detail loss, advection error, and runtime per simulated second, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 11, Unit 3. Free surfaces: PIC/FLIP/APIC

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Particles coupled to grids, transfers, level sets/markers, surface tracking.

A. Reading order

1. Read the primary assignment first: Bridson, Fluid Simulation for Computer Graphics, particle/grid transfer chapters; Brackbill & Ruppel (1986) for FLIP origins; and Jiang et al., The Affine Particle-In-Cell Method (SIGGRAPH 2015) for APIC. Reproduce one transfer-only test before coupling the method to pressure projection.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: CS348C particle-based fluids materials.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- PIC damping vs FLIP noise
- affine particle velocity representation
- Precision requirement: distinguish and justify PIC, FLIP, blended PIC/FLIP, and APIC transfers under rotating, splashing, and low-particle-count tests. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive particle-to-grid and grid-to-particle transfer for PIC and FLIP, compare dissipation/noise, and derive the affine velocity term used by APIC.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use a rotating flow and splash; sweep transfer method/blending and track energy, particle noise, surface smoothness, and volume. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement small PIC/FLIP liquid or use scaffold; sweep PIC/FLIP blend and quantify energy/noise. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: One particle with velocity 1 lies halfway between two grid nodes initially at 0. Transfer particle->grid and back for PIC; then compute FLIP update for a grid velocity increment of +0.2 and compare dissipation. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use a rotating flow and splash; sweep transfer method/blending and track energy, particle noise, surface smoothness, and volume. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Use a rotating disk or dam-break benchmark; sweep transfer method/blend and report kinetic-energy decay/growth, particle noise, and volume.
- P5 - Machine-readable evidence. Export kinetic-energy drift, volume loss, particle noise/clumping, transfer error, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: PIC, FLIP, blended PIC/FLIP, and APIC transfers under rotating, splashing, and low-particle-count tests. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use a rotating flow and splash; sweep transfer method/blending and track energy, particle noise, surface smoothness, and volume." without hand-editing code between methods. Record kinetic-energy drift, volume loss, particle noise/clumping, transfer error, and runtime.
- P7 - Deliberate bug and regression drill. Intentionally normalize particle-grid weights incorrectly so constant velocity is not reproduced by P2G/G2P. Before running, predict the first observable failure in kinetic-energy drift, volume loss, particle noise/clumping, transfer error, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive particle-to-grid and grid-to-particle transfer for PIC and FLIP, compare dissipation/noise, and derive the affine velocity term used by APIC. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. One particle with velocity 1 lies halfway between two grid nodes initially at 0. Transfer particle->grid and back for PIC; then compute FLIP update for a grid velocity increment of +0.2 and compare dissipation. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement small PIC/FLIP liquid or use scaffold; sweep PIC/FLIP blend and quantify energy/noise. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Use a rotating disk or dam-break benchmark; sweep transfer method/blend and report kinetic-energy decay/growth, particle noise, and volume. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare PIC, FLIP, blended PIC/FLIP, and APIC transfers under rotating, splashing, and low-particle-count tests. Use the same domain, initial/boundary conditions, grid/particle resolution, timestep and pressure/solver tolerances; separate accuracy from speed by reporting both refinement behavior and wall time. Record kinetic-energy drift, volume loss, particle noise/clumping, transfer error, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use a rotating flow and splash; sweep transfer method/blending and track energy, particle noise, surface smoothness, and volume. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Bridson particle/grid chapters; APIC/FLIP papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Free surfaces: PIC/FLIP/APIC. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among PIC, FLIP, blended PIC/FLIP, and APIC transfers under rotating, splashing, and low-particle-count tests. State, before running experiments, which option should win in

at least two contrasting regimes and why. Execute the controlled protocol from Section F, using kinetic-energy drift, volume loss, particle noise/clumping, transfer error, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if PIC, FLIP, blended PIC/FLIP, and APIC transfers under rotating, splashing, and low-particle-count tests improve kinetic-energy drift, volume loss, particle noise/clumping, transfer error, and runtime under grid/particle and timestep refinement, with transfer/advection/pressure/coupling errors separated where possible; speed bought by uncontrolled dissipation is not progress.

F. Concrete research project - Free surfaces: PIC/FLIP/APIC

F1. Falsifiable question. Under the controlled failure regime "Use a rotating flow and splash; sweep transfer method/blending and track energy, particle noise, surface smoothness, and volume.", characterize the ranking or regime transition across PIC, FLIP, blended PIC/FLIP, and APIC transfers under rotating, splashing, and low-particle-count tests. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Use a rotating disk or dam-break benchmark; sweep transfer method/blend and report kinetic-energy decay/growth, particle noise, and volume. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a manufactured/analytic field test, one standard visual benchmark, and one high-CFL/free-surface/large-deformation stress case; run four time steps and at least three grid/particle resolutions. The central adversarial condition is: Use a rotating flow and splash; sweep transfer method/blending and track energy, particle noise, surface smoothness, and volume.

F4. Required evidence. Measure kinetic-energy drift, volume loss, particle noise/clumping, transfer error, and runtime. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Free surfaces: PIC/FLIP/APIC. If a stable failure/crossover is found among PIC, FLIP, blended PIC/FLIP, and APIC transfers under rotating, splashing, and low-particle-count tests, implement one adaptive or corrected variant that directly targets memory traffic, synchronization, coherence/divergence, and asymptotic/constant-factor cost; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Free surfaces: PIC/FLIP/APIC. Using the measured kinetic-energy drift, volume loss, particle noise/clumping, transfer error, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Free surfaces: PIC/FLIP/APIC

- G1. Closed-book derivation/oracle. Derive particle-to-grid and grid-to-particle transfer for PIC and FLIP, compare dissipation/noise, and derive the affine velocity term used by APIC. Then solve this exact micro-case from scratch without opening your implementation: One particle with velocity 1 lies halfway between two grid nodes initially at 0. Transfer particle->grid and back for PIC; then compute FLIP update for a grid velocity increment of +0.2 and compare dissipation. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement small PIC/FLIP liquid or use scaffold; sweep PIC/FLIP blend and quantify energy/noise. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally normalize particle-grid weights incorrectly so constant velocity is not reproduced by P2G/G2P. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Use a rotating disk or dam-break benchmark; sweep transfer method/blend and report kinetic-energy decay/growth, particle noise, and volume. Then apply this stress without retuning: Use a rotating flow and splash; sweep transfer method/blending and track energy, particle noise, surface smoothness, and volume. Report kinetic-energy drift, volume loss, particle

noise/clumping, transfer error, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across PIC, FLIP, blended PIC/FLIP, and APIC transfers under rotating, splashing, and low-particle-count tests for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured kinetic-energy drift, volume loss, particle noise/clumping, transfer error, and runtime; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Free surfaces: PIC/FLIP/APIC. Using kinetic-energy drift, volume loss, particle noise/clumping, transfer error, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 11, Unit 4. SPH and position-based fluids

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Kernel sums, density constraints, neighborhood search.

A. Reading order

1. Read the primary assignment first: Physics-Based Simulation particle fluid/PBD sections; classic PBF paper.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Position Based Fluids assignment/reference.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Kernel normalization
- density estimate
- pressure/constraint formulations
- Precision requirement: distinguish and justify density-pressure SPH vs position-based fluid constraints under particle spacing, timestep, and boundary variations. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive density estimation and pressure forces in SPH, then derive the position-based density constraint update; discuss consistency, kernel support, and boundary deficiency.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Sweep particle disorder, kernel/support radius, boundaries, and timestep; track density error, clumping, surface artifacts, and stability. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement 2D SPH/PBF toy solver; compare stability and incompressibility. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For two particles separated by $r=h/2$ using a simple compact kernel $W(r)$, compute density contributions and the direction of a pressure-gradient force. Check symmetry/action-reaction. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Sweep particle disorder, kernel/support radius, boundaries, and timestep; track density error, clumping, surface artifacts, and stability. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Sweep smoothing length, neighbors, iterations, and timestep; record density error, boundary deficiency, particle clumping, and runtime.
- P5 - Machine-readable evidence. Export density error, volume/boundary error, clumping, iterations/neighbors, stability limit, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: density-pressure SPH vs position-based fluid constraints under particle spacing, timestep, and boundary variations. The harness

must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Sweep particle disorder, kernel/support radius, boundaries, and timestep; track density error, clumping, surface artifacts, and stability." without hand-editing code between methods. Record density error, volume/boundary error, clumping, iterations/neighbors, stability limit, and runtime.

- P7 - Deliberate bug and regression drill. Intentionally use a kernel/gradient with the wrong normalization or support radius convention. Before running, predict the first observable failure in density error, volume/boundary error, clumping, iterations/neighbors, stability limit, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive density estimation and pressure forces in SPH, then derive the position-based density constraint update; discuss consistency, kernel support, and boundary deficiency. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For two particles separated by $r=h/2$ using a simple compact kernel $W(r)$, compute density contributions and the direction of a pressure-gradient force. Check symmetry/action-reaction. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement 2D SPH/PBF toy solver; compare stability and incompressibility. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Sweep smoothing length, neighbors, iterations, and timestep; record density error, boundary deficiency, particle clumping, and runtime. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare density-pressure SPH vs position-based fluid constraints under particle spacing, timestep, and boundary variations. Use the same domain, initial/boundary conditions, grid/particle resolution, timestep and pressure/solver tolerances; separate accuracy from speed by reporting both refinement behavior and wall time. Record density error, volume/boundary error, clumping, iterations/neighbors, stability limit, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Sweep particle disorder, kernel/support radius, boundaries, and timestep; track density error, clumping, surface artifacts, and stability. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Physics-Based Simulation particle fluid/PBD sections; classic PBF paper. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on SPH and position-based fluids. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among density-pressure SPH vs position-based fluid constraints under particle spacing, timestep, and boundary variations. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using density error, volume/boundary error, clumping, iterations/neighbors, stability limit, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if density-pressure SPH vs position-based fluid constraints under particle spacing, timestep, and boundary variations improve density error, volume/boundary error, clumping, iterations/neighbors, stability limit, and runtime under grid/particle and timestep refinement, with transfer/advection/pressure/coupling errors separated where possible; speed bought by uncontrolled dissipation is not progress.

F. Concrete research project - SPH and position-based fluids

F1. Falsifiable question. Under the controlled failure regime "Sweep particle disorder, kernel/support radius, boundaries, and timestep; track density error, clumping, surface artifacts, and stability.", which alternative in density-pressure SPH vs position-based fluid constraints under particle spacing, timestep, and boundary variations is preferable, and is the change explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Sweep smoothing length, neighbors, iterations, and timestep; record density error, boundary deficiency, particle clumping, and runtime. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a manufactured/analytic field test, one standard visual benchmark, and one high-CFL/free-surface/large-deformation stress case; run four time steps and at least three grid/particle resolutions. The central adversarial condition is: Sweep particle disorder, kernel/support radius, boundaries, and timestep; track density error, clumping, surface artifacts, and stability.

F4. Required evidence. Measure density error, volume/boundary error, clumping, iterations/neighbors, stability limit, and runtime. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for SPH and position-based fluids. If a stable failure/crossover is found among density-pressure SPH vs position-based fluid constraints under particle spacing, timestep, and boundary variations, implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for SPH and position-based fluids. Using the measured density error, volume/boundary error, clumping, iterations/neighbors, stability limit, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for SPH and position-based fluids

- G1. Closed-book derivation/oracle. Derive density estimation and pressure forces in SPH, then derive the position-based density constraint update; discuss consistency, kernel support, and boundary deficiency. Then solve this exact micro-case from scratch without opening your implementation: For two particles separated by $r=h/2$ using a simple compact kernel $W(r)$, compute density contributions and the direction of a pressure-gradient force. Check symmetry/action-reaction. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement 2D SPH/PBF toy solver; compare stability and incompressibility. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use a kernel/gradient with the wrong normalization or support radius convention. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Sweep smoothing length, neighbors, iterations, and timestep; record density error, boundary deficiency, particle clumping, and runtime. Then apply this stress without retuning: Sweep particle disorder, kernel/support radius, boundaries, and timestep; track density error, clumping, surface artifacts, and stability. Report density error, volume/boundary error, clumping, iterations/neighbors, stability limit, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among density-pressure SPH vs position-based fluid constraints under particle spacing, timestep, and boundary variations for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured density error, volume/boundary error, clumping, iterations/neighbors, stability limit, and runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for SPH and position-based fluids. Using density error, volume/boundary error, clumping, iterations/neighbors, stability limit, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 11, Unit 5. Level sets and surface reconstruction

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Signed-distance reinitialization, marching cubes, particle level set idea.

A. Reading order

1. Read the primary assignment first: Bridson level-set chapters; geometry material from Vol. 7.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Marching cubes tutorial/library.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Eikonal equation intuition
- isosurface extraction
- Precision requirement: distinguish and justify level-set advection/reinitialization vs particle-to-surface reconstruction, tracking volume loss and surface error. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the level-set advection equation, normal/curvature from a signed-distance field, and the Eikonal reinitialization condition; connect particle/implicit surface reconstruction to kernel or distance fields.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Advect a slotted disk/thin feature and repeated merging/splitting droplets; track volume, signed-distance error, and normal quality. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Extract liquid surface from grid/particles and analyze volume loss. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For $\phi(x,y)=\sqrt{x^2+y^2}-1$, compute ϕ and gradient at (1.2,0),(0.6,0.8). Perturb ϕ by scaling 2 and show zero set is unchanged but $|\text{grad } \phi| = 1$ property is lost. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Advect a slotted disk/thin feature and repeated merging/splitting droplets; track volume, signed-distance error, and normal quality. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Advect a circle/sphere under a known velocity field and quantify interface error/volume loss; compare reconstruction settings from particles.
- P5 - Machine-readable evidence. Export interface/Hausdorff error, volume loss, signed-distance quality, topology artifacts, and reconstruction/advection cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: level-set advection/reinitialization vs particle-to-surface reconstruction, tracking volume loss and surface error. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Advect a slotted disk/thin feature and repeated merging/splitting droplets; track volume, signed-distance error, and normal quality." without hand-editing code between methods. Record interface/Hausdorff error, volume loss, signed-distance quality, topology artifacts, and reconstruction/advection cost.
- P7 - Deliberate bug and regression drill. Intentionally reinitialize the level set repeatedly without checking that the zero level set has drifted. Before running, predict the first observable failure in interface/Hausdorff error, volume loss, signed-distance quality, topology artifacts, and reconstruction/advection cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the level-set advection equation, normal/curvature from a signed-distance field, and the Eikonal reinitialization condition; connect particle/implicit surface reconstruction to kernel or distance fields. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. For $\phi(x,y)=\sqrt{x^2+y^2}-1$, compute ϕ and gradient at (1.2,0),(0.6,0.8). Perturb ϕ by scaling 2 and show zero set is unchanged but $|\text{grad } \phi|=1$ property is lost. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Extract liquid surface from grid/particles and analyze volume loss. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Advect a circle/sphere under a known velocity field and quantify interface error/volume loss; compare reconstruction settings from particles. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare level-set advection/reinitialization vs particle-to-surface reconstruction, tracking volume loss and surface error. Use the same domain, initial/boundary conditions, grid/particle resolution, timestep and pressure/solver tolerances; separate accuracy from speed by reporting both refinement behavior and wall time. Record interface/Hausdorff error, volume loss, signed-distance quality, topology artifacts, and reconstruction/advection cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Advect a slotted disk/thin feature and repeated merging/splitting droplets; track volume, signed-distance error, and normal quality. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Bridson level-set chapters; geometry material from Vol. 7. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Level sets and surface reconstruction. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among level-set advection/reinitialization vs particle-to-surface reconstruction, tracking volume loss and surface error. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using interface/Hausdorff error, volume loss, signed-distance quality, topology artifacts, and reconstruction/advection cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if level-set advection/reinitialization vs particle-to-surface reconstruction, tracking volume loss and surface error improve interface/Hausdorff error, volume loss, signed-distance quality, topology artifacts, and reconstruction/advection cost under grid/particle and timestep refinement, with transfer/advection/pressure/coupling errors separated where possible; speed bought by uncontrolled dissipation is not progress.

F. Concrete research project - Level sets and surface reconstruction

- F1. Falsifiable question. Under the controlled failure regime "Advect a slotted disk/thin feature and repeated merging/splitting droplets; track volume, signed-distance error, and normal quality.", which alternative in level-set advection/reinitialization vs particle-to-surface reconstruction, tracking volume loss and surface error is preferable, and is the change explained by spectral support, footprint estimation, and aliasing/reconstruction error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Advect a circle/sphere under a known velocity field and quantify interface error/volume loss; compare reconstruction settings from particles. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use a manufactured/analytic field test, one standard visual benchmark, and one high-CFL/free-surface/large-deformation stress case; run four time steps and at least three grid/particle resolutions. The central adversarial condition is: Advect a slotted disk/thin feature and repeated merging/splitting droplets; track volume, signed-distance error, and normal quality.
- F4. Required evidence. Measure interface/Hausdorff error, volume loss, signed-distance quality, topology artifacts, and reconstruction/advection cost. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Level sets and surface reconstruction. If a stable failure/crossover is found among level-set advection/reinitialization vs particle-to-surface reconstruction, tracking volume loss and surface error, implement one adaptive or corrected variant that directly targets spectral support, footprint estimation, and aliasing/reconstruction

error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Level sets and surface reconstruction. Using the measured interface/Hausdorff error, volume loss, signed-distance quality, topology artifacts, and reconstruction/advection cost, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Level sets and surface reconstruction

- G1. Closed-book derivation/oracle. Derive the level-set advection equation, normal/curvature from a signed-distance field, and the Eikonal reinitialization condition; connect particle/implicit surface reconstruction to kernel or distance fields. Then solve this exact micro-case from scratch without opening your implementation: For $\phi(x,y)=\sqrt{x^2+y^2}-1$, compute ϕ and gradient at (1.2,0),(0.6,0.8). Perturb ϕ by scaling 2 and show zero set is unchanged but $|\text{grad } \phi|=1$ property is lost. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Extract liquid surface from grid/particles and analyze volume loss. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally reinitialize the level set repeatedly without checking that the zero level set has drifted. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Advect a circle/sphere under a known velocity field and quantify interface error/volume loss; compare reconstruction settings from particles. Then apply this stress without retuning: Advect a slotted disk/thin feature and repeated merging/splitting droplets; track volume, signed-distance error, and normal quality. Report interface/Hausdorff error, volume loss, signed-distance quality, topology artifacts, and reconstruction/advection cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among level-set advection/reinitialization vs particle-to-surface reconstruction, tracking volume loss and surface error for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured interface/Hausdorff error, volume loss, signed-distance quality, topology artifacts, and reconstruction/advection cost; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Level sets and surface reconstruction. Using interface/Hausdorff error, volume loss, signed-distance quality, topology artifacts, and reconstruction/advection cost, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 11, Unit 6. Material Point Method

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Particles carry material state, grid solves momentum, transfer back; solids and fluids in one framework.

A. Reading order

1. Read the primary assignment first: Read the MPM chapters of Physics-Based Simulation, Stomakhin et al., A Material Point Method for Snow Simulation (SIGGRAPH 2013), and Jiang et al., The Material Point Method for Simulating Continuum Materials (SIGGRAPH course notes, 2016).
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Simulation-book MPM implementations.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Deformation gradient update
- P2G/G2P
- constitutive model

- APIC relation
- Precision requirement: distinguish and justify PIC/FLIP-like grid transfers vs APIC-style MPM transfers and constitutive models on elastic/snow-like benchmarks. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the MPM transfer cycle (particles to grid, grid update, grid to particles) for a simple elastic material and explain how deformation gradients and stresses enter nodal forces.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Move material through grid cells under rotation/shear and large deformation; track cell-crossing artifacts, energy, and stress noise. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement 2D MPM for snow-like or elastic material using reference derivation; run convergence/stability tests. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: In 1D, place one particle halfway between two grid nodes with mass m and velocity v . Compute linear basis weights, particle-to-grid mass/momentum, grid update with one force, and grid-to-particle velocity. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Move material through grid cells under rotation/shear and large deformation; track cell-crossing artifacts, energy, and stress noise. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement a small 2D MPM/APIC solver and track grid/particle momentum transfer, energy, and material response under compression/rotation.
- P5 - Machine-readable evidence. Export momentum/energy transfer residual, deformation/stress error, grid-crossing artifacts, stability, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: PIC/FLIP-like grid transfers vs APIC-style MPM transfers and constitutive models on elastic/snow-like benchmarks. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Move material through grid cells under rotation/shear and large deformation; track cell-crossing artifacts, energy, and stress noise." without hand-editing code between methods. Record momentum/energy transfer residual, deformation/stress error, grid-crossing artifacts, stability, and runtime.
- P7 - Deliberate bug and regression drill. Intentionally mismatch P2G and G2P interpolation gradients or APIC affine terms, creating grid-crossing artifacts. Before running, predict the first observable failure in momentum/energy transfer residual, deformation/stress error, grid-crossing artifacts, stability, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the MPM transfer cycle (particles to grid, grid update, grid to particles) for a simple elastic material and explain how deformation gradients and stresses enter nodal forces. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. In 1D, place one particle halfway between two grid nodes with mass m and velocity v . Compute linear basis weights, particle-to-grid mass/momentum, grid update with one force, and grid-to-particle velocity. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement 2D MPM for snow-like or elastic material using reference derivation; run convergence/stability tests. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement a small 2D MPM/APIC solver and track grid/particle momentum transfer, energy, and material response under compression/rotation. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare PIC/FLIP-like grid transfers vs APIC-style MPM transfers and constitutive models on elastic/snow-like benchmarks. Use the same domain, initial/boundary conditions, grid/particle resolution, timestep and pressure/solver tolerances; separate accuracy from speed by reporting both refinement behavior and wall

time. Record momentum/energy transfer residual, deformation/stress error, grid-crossing artifacts, stability, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Move material through grid cells under rotation/shear and large deformation; track cell-crossing artifacts, energy, and stress noise. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Physics-Based Simulation MPM chapters; Jiang et al. SIGGRAPH course/papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Material Point Method. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among PIC/FLIP-like grid transfers vs APIC-style MPM transfers and constitutive models on elastic/snow-like benchmarks. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using momentum/energy transfer residual, deformation/stress error, grid-crossing artifacts, stability, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if PIC/FLIP-like grid transfers vs APIC-style MPM transfers and constitutive models on elastic/snow-like benchmarks improve momentum/energy transfer residual, deformation/stress error, grid-crossing artifacts, stability, and runtime under grid/particle and timestep refinement, with transfer/advection/pressure/coupling errors separated where possible; speed bought by uncontrolled dissipation is not progress.

F. Concrete research project - Material Point Method

F1. Falsifiable question. Under the controlled failure regime "Move material through grid cells under rotation/shear and large deformation; track cell-crossing artifacts, energy, and stress noise.", which alternative in PIC/FLIP-like grid transfers vs APIC-style MPM transfers and constitutive models on elastic/snow-like benchmarks is preferable, and is the change explained by particle-grid transfer consistency, grid-crossing error, constitutive update, timestep stability, and conservation properties? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement a small 2D MPM/APIC solver and track grid/particle momentum transfer, energy, and material response under compression/rotation. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a manufactured/analytic field test, one standard visual benchmark, and one high-CFL/free-surface/large-deformation stress case; run four time steps and at least three grid/particle resolutions. The central adversarial condition is: Move material through grid cells under rotation/shear and large deformation; track cell-crossing artifacts, energy, and stress noise.

F4. Required evidence. Measure momentum/energy transfer residual, deformation/stress error, grid-crossing artifacts, stability, and runtime. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Material Point Method. If a stable failure/crossover is found among PIC/FLIP-like grid transfers vs APIC-style MPM transfers and constitutive models on elastic/snow-like benchmarks, implement one adaptive or corrected variant that directly targets particle-grid transfer consistency, grid-crossing error, constitutive update, timestep stability, and conservation properties; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Material Point Method. Using the measured momentum/energy transfer residual, deformation/stress error, grid-crossing artifacts, stability, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Material Point Method

- G1. Closed-book derivation/oracle. Derive the MPM transfer cycle (particles to grid, grid update, grid to particles) for a simple elastic material and explain how deformation gradients and stresses enter nodal forces. Then solve this exact micro-case from scratch without opening your implementation: In 1D, place one particle halfway between two grid nodes with mass m and velocity v . Compute linear basis weights, particle-to-grid mass/momentum, grid update with one force, and grid-to-particle velocity. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement 2D MPM for snow-like or elastic material using reference derivation; run convergence/stability tests. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally mismatch P2G and G2P interpolation gradients or APIC affine terms, creating grid-crossing artifacts. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement a small 2D MPM/APIC solver and track grid/particle momentum transfer, energy, and material response under compression/rotation. Then apply this stress without retuning: Move material through grid cells under rotation/shear and large deformation; track cell-crossing artifacts, energy, and stress noise. Report momentum/energy transfer residual, deformation/stress error, grid-crossing artifacts, stability, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among PIC/FLIP-like grid transfers vs APIC-style MPM transfers and constitutive models on elastic/snow-like benchmarks for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured momentum/energy transfer residual, deformation/stress error, grid-crossing artifacts, stability, and runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Material Point Method. Using momentum/energy transfer residual, deformation/stress error, grid-crossing artifacts, stability, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 11, Unit 7. Coupling and multiphysics

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Fluid-solid interaction, two-way coupling, thin structures, granular/continuum mixtures.

A. Reading order

1. Read the primary assignment first: Read the coupling/interface sections of Physics-Based Simulation and one primary paper matching the chosen coupling (fluid-solid, particle-grid, or differentiable multiphysics). Write the interface conditions first; do not treat a generic simulator repository as the theory source.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Choose one recent coupling paper for reproduction.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- Interface conditions
- momentum exchange
- monolithic vs partitioned intuition
- Precision requirement: distinguish and justify one-way vs two-way fluid-solid coupling, tracking force balance, leakage, added-mass/stability effects, and interface resolution. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive interface conditions for one fluid-solid or particle-grid coupling problem and write a partitioned versus monolithic update; identify where added-mass or stability problems can arise.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use density/impedance and timestep mismatches with strong two-way coupling; track exchanged

momentum/energy and interface constraint error. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Reproduce a simplified fluid-solid coupling benchmark; measure conservation and stability. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: A one-dimensional fluid pressure p acts on a rigid piston of area A and mass m attached to spring k . Write coupled force/acceleration equations and compare one explicit staggered step with a monolithic implicit equation. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use density/impedance and timestep mismatches with strong two-way coupling; track exchanged momentum/energy and interface constraint error. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Construct a floating/impacting rigid-body-fluid toy benchmark and monitor exchanged impulse, leakage, stability, and dependence on coupling iterations.
- P5 - Machine-readable evidence. Export interface leakage, exchanged force/impulse residual, mass/energy change, stability versus coupling iterations, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: one-way vs two-way fluid-solid coupling, tracking force balance, leakage, added-mass/stability effects, and interface resolution. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use density/impedance and timestep mismatches with strong two-way coupling; track exchanged momentum/energy and interface constraint error." without hand-editing code between methods. Record interface leakage, exchanged force/impulse residual, mass/energy change, stability versus coupling iterations, and runtime.
- P7 - Deliberate bug and regression drill. Intentionally use the same interface normal/sign on both subsystems instead of equal-and-opposite exchange. Before running, predict the first observable failure in interface leakage, exchanged force/impulse residual, mass/energy change, stability versus coupling iterations, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive interface conditions for one fluid-solid or particle-grid coupling problem and write a partitioned versus monolithic update; identify where added-mass or stability problems can arise. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. A one-dimensional fluid pressure p acts on a rigid piston of area A and mass m attached to spring k . Write coupled force/acceleration equations and compare one explicit staggered step with a monolithic implicit equation. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Reproduce a simplified fluid-solid coupling benchmark; measure conservation and stability. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Construct a floating/impacting rigid-body-fluid toy benchmark and monitor exchanged impulse, leakage, stability, and dependence on coupling iterations. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare one-way vs two-way fluid-solid coupling, tracking force balance, leakage, added-mass/stability effects, and interface resolution. Use the same domain, initial/boundary conditions, grid/particle resolution, timestep and pressure/solver tolerances; separate accuracy from speed by reporting both refinement behavior and wall time. Record interface leakage, exchanged force/impulse residual, mass/energy change, stability versus coupling iterations, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use density/impedance and timestep mismatches with strong two-way coupling; track exchanged momentum/energy and interface constraint error. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Selected SIGGRAPH/TOG papers; simulation book coupled examples. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Coupling and multiphysics. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among one-way vs two-way fluid-solid coupling, tracking force balance, leakage, added-mass/stability effects, and interface resolution. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using interface leakage, exchanged force/impulse residual, mass/energy change, stability versus coupling iterations, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if one-way vs two-way fluid-solid coupling, tracking force balance, leakage, added-mass/stability effects, and interface resolution improve interface leakage, exchanged force/impulse residual, mass/energy change, stability versus coupling iterations, and runtime under grid/particle and timestep refinement, with transfer/advection/pressure/coupling errors separated where possible; speed bought by uncontrolled dissipation is not progress.

F. Concrete research project - Coupling and multiphysics

F1. Falsifiable question. Under the controlled failure regime "Use density/impedance and timestep mismatches with strong two-way coupling; track exchanged momentum/energy and interface constraint error.", which alternative in one-way vs two-way fluid-solid coupling, tracking force balance, leakage, added-mass/stability effects, and interface resolution is preferable, and is the change explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Construct a floating/impacting rigid-body-fluid toy benchmark and monitor exchanged impulse, leakage, stability, and dependence on coupling iterations. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a manufactured/analytic field test, one standard visual benchmark, and one high-CFL/free-surface/large-deformation stress case; run four time steps and at least three grid/particle resolutions. The central adversarial condition is: Use density/impedance and timestep mismatches with strong two-way coupling; track exchanged momentum/energy and interface constraint error.

F4. Required evidence. Measure interface leakage, exchanged force/impulse residual, mass/energy change, stability versus coupling iterations, and runtime. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Coupling and multiphysics. If a stable failure/crossover is found among one-way vs two-way fluid-solid coupling, tracking force balance, leakage, added-mass/stability effects, and interface resolution, implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Coupling and multiphysics. Using the measured interface leakage, exchanged force/impulse residual, mass/energy change, stability versus coupling iterations, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Coupling and multiphysics

- G1. Closed-book derivation/oracle. Derive interface conditions for one fluid-solid or particle-grid coupling problem and write a partitioned versus monolithic update; identify where added-mass or stability problems can arise. Then solve this exact micro-case from scratch without opening your implementation: A one-dimensional fluid pressure p acts on a rigid piston of area A and mass m attached to spring k . Write coupled force/acceleration equations and compare one explicit staggered step with a monolithic implicit equation. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Reproduce a simplified fluid-solid coupling benchmark; measure conservation and stability. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use the same interface normal/sign on both subsystems instead of equal-and-opposite exchange. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Construct a floating/impacting rigid-body-fluid toy benchmark and monitor exchanged impulse, leakage, stability, and dependence on coupling iterations. Then apply this stress without retuning: Use density/impedance and timestep mismatches with strong two-way coupling; track exchanged momentum/energy and interface constraint error. Report interface leakage, exchanged force/impulse residual, mass/energy change, stability versus coupling iterations, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among one-way vs two-way fluid-solid coupling, tracking force balance, leakage, added-mass/stability effects, and interface resolution for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured interface leakage, exchanged force/impulse residual, mass/energy change, stability versus coupling iterations, and runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Coupling and multiphysics. Using interface leakage, exchanged force/impulse residual, mass/energy change, stability versus coupling iterations, and runtime, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 11, Unit 8. Large timestep and high-performance simulation

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Solver conditioning, multigrid/preconditioning, asynchronous/spatiotemporal methods.

A. Reading order

1. Read the primary assignment first: Read the nonlinear/implicit-solver and high-performance chapters/sections of Physics-Based Simulation, then choose one current SIGGRAPH/TOG large-timestep or scalable-simulation paper whose benchmark you can reproduce; record the exact paper in the unit log.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: SIGGRAPH 2026 papers program as frontier reading.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation online book (v1.0.3, 2026) - <https://phys-sim-book.github.io/>

B. Definitions, results and algorithms to know

- CFL limitations
- implicit solves
- preconditioners
- multigrid intuition
- Precision requirement: distinguish and justify baseline small-step solver vs larger-step/implicit or spatiotemporal strategy, with matched error tolerance and wall-clock cost. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the linear/nonlinear system produced by an implicit large-timestep update, estimate conditioning and solver cost, and reason about preconditioning/parallelization tradeoffs.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Increase timestep and problem size until the accelerated formulation fails; separate solver, discretization, contact, and parallelization error sources. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Benchmark solver scaling; reproduce a simpler large-step method or study failure boundaries of a standard solver. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For diffusion $u_t = \nu u_{xx}$ on three interior grid samples, form the explicit update matrix and its timestep restriction; form the backward-Euler matrix for a timestep ten times larger and compare damping. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Increase timestep and problem size until the accelerated formulation fails; separate solver, discretization, contact, and parallelization error sources. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. For one solver, plot error and failure rate versus timestep and wall-clock-to-target-time; include memory and solver-iteration counts.
- P5 - Machine-readable evidence. Export error at target physical time, maximum stable/effective timestep, solver iterations, wall-clock speedup, memory, and failure rate to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: baseline small-step solver vs larger-step/implicit or spatiotemporal strategy, with matched error tolerance and wall-clock cost. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Increase timestep and problem size until the accelerated formulation fails; separate solver, discretization, contact, and parallelization error sources." without hand-editing code between methods. Record error at target physical time, maximum stable/effective timestep, solver iterations, wall-clock speedup, memory, and failure rate.
- P7 - Deliberate bug and regression drill. Intentionally relax the nonlinear/linear solver tolerance as timestep grows and then claim the method itself permits larger stable timesteps. Before running, predict the first observable failure in error at target physical time, maximum stable/effective timestep, solver iterations, wall-clock speedup, memory, and failure rate. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the linear/nonlinear system produced by an implicit large-timestep update, estimate conditioning and solver cost, and reason about preconditioning/parallelization tradeoffs. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For diffusion $u_t = \nu u_{xx}$ on three interior grid samples, form the explicit update matrix and its timestep restriction; form the backward-Euler matrix for a timestep ten times larger and compare damping. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Benchmark solver scaling; reproduce a simpler large-step method or study failure boundaries of a standard solver. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: For one solver, plot error and failure rate versus timestep and wall-clock-to-target-time; include memory and solver-iteration counts. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare baseline small-step solver vs larger-step/implicit or spatiotemporal strategy, with matched error tolerance and wall-clock cost. Use the same domain, initial/boundary conditions, grid/particle resolution, timestep and pressure/solver tolerances; separate accuracy from speed by reporting both refinement behavior and wall time. Record error at target physical time, maximum stable/effective timestep, solver iterations, wall-clock speedup, memory, and failure rate. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Increase timestep and problem size until the accelerated formulation fails; separate solver, discretization, contact, and parallelization error sources. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Current SIGGRAPH simulation papers, including large-timestep FLIP directions. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Large timestep and high-performance simulation. Do not copy reference code. Deliverable:

your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among baseline small-step solver vs larger-step/implicit or spatiotemporal strategy, with matched error tolerance and wall-clock cost. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using error at target physical time, maximum stable/effective timestep, solver iterations, wall-clock speedup, memory, and failure rate. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if baseline small-step solver vs larger-step/implicit or spatiotemporal strategy, with matched error tolerance and wall-clock cost improve error at target physical time, maximum stable/effective timestep, solver iterations, wall-clock speedup, memory, and failure rate under grid/particle and timestep refinement, with transfer/advection/pressure/coupling errors separated where possible; speed bought by uncontrolled dissipation is not progress.

F. Concrete research project - Large timestep and high-performance simulation

F1. Falsifiable question. Under the controlled failure regime "Increase timestep and problem size until the accelerated formulation fails; separate solver, discretization, contact, and parallelization error sources.", which alternative in baseline small-step solver vs larger-step/implicit or spatiotemporal strategy, with matched error tolerance and wall-clock cost is preferable, and is the change explained by time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: For one solver, plot error and failure rate versus timestep and wall-clock-to-target-time; include memory and solver-iteration counts. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a manufactured/analytic field test, one standard visual benchmark, and one high-CFL/free-surface/large-deformation stress case; run four time steps and at least three grid/particle resolutions. The central adversarial condition is: Increase timestep and problem size until the accelerated formulation fails; separate solver, discretization, contact, and parallelization error sources.

F4. Required evidence. Measure error at target physical time, maximum stable/effective timestep, solver iterations, wall-clock speedup, memory, and failure rate. Deliver simulator source/config, oracle and benchmark initial conditions, timestep/resolution CSV, stability/conservation plots, and the first unstable/nonconvergent saved configuration. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Large timestep and high-performance simulation. If a stable failure/crossover is found among baseline small-step solver vs larger-step/implicit or spatiotemporal strategy, with matched error tolerance and wall-clock cost, implement one adaptive or corrected variant that directly targets time-step/spatial discretization stability, conservation/energy behavior, contact/interface treatment, and solver tolerance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Large timestep and high-performance simulation. Using the measured error at target physical time, maximum stable/effective timestep, solver iterations, wall-clock speedup, memory, and failure rate, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Large timestep and high-performance simulation

- G1. Closed-book derivation/oracle. Derive the linear/nonlinear system produced by an implicit large-timestep update, estimate conditioning and solver cost, and reason about preconditioning/parallelization tradeoffs. Then solve this exact micro-case from scratch without opening your implementation: For diffusion $u_t = \nu u_{xx}$ on three interior grid samples, form the explicit update matrix and its timestep restriction; form the backward-Euler matrix for a timestep ten times larger and compare damping. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Benchmark solver scaling; reproduce a simpler large-step method or study failure

boundaries of a standard solver. Add the oracle as an automated test and make it pass before importing any optimized/framework code.

- G3. Debugging exam. Intentionally relax the nonlinear/linear solver tolerance as timestep grows and then claim the method itself permits larger stable timesteps. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: For one solver, plot error and failure rate versus timestep and wall-clock-to-target-time; include memory and solver-iteration counts. Then apply this stress without retuning: Increase timestep and problem size until the accelerated formulation fails; separate solver, discretization, contact, and parallelization error sources. Report error at target physical time, maximum stable/effective timestep, solver iterations, wall-clock speedup, memory, and failure rate and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among baseline small-step solver vs larger-step/implicit or spatiotemporal strategy, with matched error tolerance and wall-clock cost for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured error at target physical time, maximum stable/effective timestep, solver iterations, wall-clock speedup, memory, and failure rate; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Large timestep and high-performance simulation. Using error at target physical time, maximum stable/effective timestep, solver iterations, wall-clock speedup, memory, and failure rate, Pass when the manufactured/hand oracle and timestep/spatial refinement behave as predicted, conservation/energy/divergence/contact diagnostics remain physically interpretable, and the first failure boundary is explained. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 11 Closeout: Project, Review and Mastery Gate

Volume project. Build a fluid/continuum suite containing smoke plus PIC/FLIP/APIC or MPM, and write a numerical error/stability report with refinement studies.

Cumulative review

- Verify advection/projection or particle-grid transfer on manufactured/hand-checkable cases before visual fluids.
- Produce grid/time/particle-resolution studies for smoke, free surface and one particle/MPM method.
- Track divergence, mass/volume, energy/dissipation, interface error and stability limits.
- Create a stress case that distinguishes numerical damping from instability/noise and explain the mechanism.
- Write a fluids synthesis comparing Eulerian, particle-grid, SPH/PBD and MPM representations by conservation, stability and coupling needs.

Mastery gate before moving on

- Track divergence, volume/mass, energy, and refinement behavior for Eulerian and particle-grid methods.
- Explain PIC/FLIP/APIC/SPH/MPM transfer/discretization errors using controlled benchmarks rather than visual impressions.
- Demonstrate one coupling or large-timestep case with a quantified accuracy-stability-performance tradeoff.
- Literature gate: reconstruct one Eulerian advection/projection method and one particle-grid/particle method; compare conservation, numerical dissipation, interface handling and stability assumptions on a common benchmark.

Volume 12. Cameras, Optics, Computational Imaging, and Displays

Purpose. Connect graphics with physical image formation, computational photography, inverse imaging, and immersive display systems.

Suggested duration. 14-18 weeks

Prerequisites. Volumes 1, 3, 5-6; Fourier analysis becomes increasingly important.

Typical module pace. 2.0-2.6 weeks per module (7 modules total).

Primary references for this volume

- PBRT 4e, Ch. 5 plus relevant spectral/radiometric chapters.
- Introductory Fourier optics text (Goodman is the standard deeper reference).
- Computational photography/light-field papers and current display literature.

Online spine

PBRT 4th edition - full online book: <https://pbr-book.org/4ed/contents>

Berkeley CS184/284A Spring 2026 course: <https://cs184.eecs.berkeley.edu/sp26/>

Canonical paper track

- Levoy & Hanrahan (1996), Light Field Rendering.
- Ng et al. (2005), Light Field Photography with a Hand-Held Plenoptic Camera.
- Classic coded-aperture/computational photography papers plus current computational-display literature.
- Current holographic and AR display papers from SIGGRAPH Emerging Technologies/Technical Papers.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Kolb, Mitchell & Hanrahan (1995), A Realistic Camera Model for Computer Graphics.
- Levoy & Hanrahan (1996), Light Field Rendering.
- Gortler et al. (1996), The Lumigraph.
- Debevec & Malik (1997), Recovering High Dynamic Range Radiance Maps from Photographs.
- Ng et al. (2005), Fourier Slice Photography.
- Goodman, Introduction to Fourier Optics - selected diffraction/Fourier-optics chapters for the wave-optics bridge.
- Use PBRT Ch. 5 for camera/film implementation and the Berkeley CS184 VR lectures for display/spatial-computing exercises.

Volume 12, Unit 1. Pinhole and lens cameras

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Projective camera, thin lens, aperture, focus, aberration intuition.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 5; FCG viewing chapters; CS184 cameras if using older offerings.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Camera/lens graphics lectures.
3. Keep the most useful executable/reference page open while coding: PBRT camera/film/radiometry chapters - <https://pbr-book.org/4ed/contents> ; Berkeley CS184 VR material - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Thin-lens equation
- circle of confusion
- radiance-to-sensor mapping
- Precision requirement: distinguish and justify pinhole vs thin-lens vs simple compound-lens ray models on calibration, distortion, depth-of-field, and focus tests. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the intrinsic/extrinsic pinhole camera matrix, perspective projection, and thin-lens ray model; connect focal length, field of view, aperture, and depth of field.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use wide FOV, strong defocus, tilted planes, and calibration perturbations; compare pinhole and thin-

lens predictions with analytic geometry. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Build thin-lens renderer/camera simulator; visualize depth of field and focal-plane effects. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Project points (0,0,-2),(1,0,-2),(1,0,-4) with focal length $f=2$. Compute image coordinates and show inverse-depth scaling. Then use a thin lens to compute focus for one off-axis lens sample. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use wide FOV, strong defocus, tilted planes, and calibration perturbations; compare pinhole and thin-lens predictions with analytic geometry. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Write a camera-calibration/forward-simulation notebook with known 3D points; verify reprojection, distortion, focus, and circle-of-confusion predictions.
- P5 - Machine-readable evidence. Export reprojection error, distortion/focus/CoC error, calibration residual, and ray/sample cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: pinhole vs thin-lens vs simple compound-lens ray models on calibration, distortion, depth-of-field, and focus tests. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use wide FOV, strong defocus, tilted planes, and calibration perturbations; compare pinhole and thin-lens predictions with analytic geometry." without hand-editing code between methods. Record reprojection error, distortion/focus/CoC error, calibration residual, and ray/sample cost.
- P7 - Deliberate bug and regression drill. Intentionally mix focal length in millimeters with sensor/scene units or use an inconsistent camera-to-world convention. Before running, predict the first observable failure in reprojection error, distortion/focus/CoC error, calibration residual, and ray/sample cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the intrinsic/extrinsic pinhole camera matrix, perspective projection, and thin-lens ray model; connect focal length, field of view, aperture, and depth of field. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Project points (0,0,-2),(1,0,-2),(1,0,-4) with focal length $f=2$. Compute image coordinates and show inverse-depth scaling. Then use a thin lens to compute focus for one off-axis lens sample. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Build thin-lens renderer/camera simulator; visualize depth of field and focal-plane effects. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Write a camera-calibration/forward-simulation notebook with known 3D points; verify reprojection, distortion, focus, and circle-of-confusion predictions. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare pinhole vs thin-lens vs simple compound-lens ray models on calibration, distortion, depth-of-field, and focus tests. Use the same scene, optical/sensor/display configuration and exposure/wavelength/FOV conditions; compare to an analytic, calibrated or high-fidelity reference where available. Record reprojection error, distortion/focus/CoC error, calibration residual, and ray/sample cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use wide FOV, strong defocus, tilted planes, and calibration perturbations; compare pinhole and thin-lens predictions with analytic geometry. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: PBRT Ch. 5; FCG viewing chapters; CS184 cameras if using older offerings. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure

that directly bears on Pinhole and lens cameras. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among pinhole vs thin-lens vs simple compound-lens ray models on calibration, distortion, depth-of-field, and focus tests. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using reprojection error, distortion/focus/CoC error, calibration residual, and ray/sample cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if pinhole vs thin-lens vs simple compound-lens ray models on calibration, distortion, depth-of-field, and focus tests improve reprojection error, distortion/focus/CoC error, calibration residual, and ray/sample cost over multiple exposure/wavelength/FOV/depth conditions against an analytic/calibrated reference, rather than overfitting one optical or display configuration.

F. Concrete research project - Pinhole and lens cameras

F1. Falsifiable question. Under the controlled failure regime "Use wide FOV, strong defocus, tilted planes, and calibration perturbations; compare pinhole and thin-lens predictions with analytic geometry.", which alternative in pinhole vs thin-lens vs simple compound-lens ray models on calibration, distortion, depth-of-field, and focus tests is preferable, and is the change explained by calibration, optical/sensor modeling error, sampling, and reconstruction identifiability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Write a camera-calibration/forward-simulation notebook with known 3D points; verify reprojection, distortion, focus, and circle-of-confusion predictions. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Start from synthetic ground truth with known camera/optical parameters, then add at least three noise/calibration levels and one realistic/real capture. Keep calibration error separate from reconstruction/model error. The central adversarial condition is: Use wide FOV, strong defocus, tilted planes, and calibration perturbations; compare pinhole and thin-lens predictions with analytic geometry.

F4. Required evidence. Measure reprojection error, distortion/focus/CoC error, calibration residual, and ray/sample cost. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Pinhole and lens cameras. If a stable failure/crossover is found among pinhole vs thin-lens vs simple compound-lens ray models on calibration, distortion, depth-of-field, and focus tests, implement one adaptive or corrected variant that directly targets calibration, optical/sensor modeling error, sampling, and reconstruction identifiability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Pinhole and lens cameras. Using the measured reprojection error, distortion/focus/CoC error, calibration residual, and ray/sample cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Pinhole and lens cameras

- G1. Closed-book derivation/oracle. Derive the intrinsic/extrinsic pinhole camera matrix, perspective projection, and thin-lens ray model; connect focal length, field of view, aperture, and depth of field. Then solve this exact micro-case from scratch without opening your implementation: Project points (0,0,-2),(1,0,-2),(1,0,-4) with focal length $f=2$. Compute image coordinates and show inverse-depth scaling. Then use a thin lens to compute focus for one off-axis lens sample. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Build thin-lens renderer/camera simulator; visualize depth of field and focal-plane effects. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally mix focal length in millimeters with sensor/scene units or use an inconsistent camera-to-world convention. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.

- G4. Transfer experiment. On an input/configuration not used during development, execute: Write a camera-calibration/forward-simulation notebook with known 3D points; verify reprojection, distortion, focus, and circle-of-confusion predictions. Then apply this stress without retuning: Use wide FOV, strong defocus, tilted planes, and calibration perturbations; compare pinhole and thin-lens predictions with analytic geometry. Report reprojection error, distortion/focus/CoC error, calibration residual, and ray/sample cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among pinhole vs thin-lens vs simple compound-lens ray models on calibration, distortion, depth-of-field, and focus tests for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured reprojection error, distortion/focus/CoC error, calibration residual, and ray/sample cost; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Pinhole and lens cameras. Using reprojection error, distortion/focus/CoC error, calibration residual, and ray/sample cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 12, Unit 2. Image sensors and noise

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Photon shot noise, read noise, raw mosaics, demosaicing, response functions.

A. Reading order

1. Read the primary assignment first: Read Debevec & Malik, Recovering High Dynamic Range Radiance Maps from Photographs (SIGGRAPH 1997), Hasinoff et al., Burst Photography for High Dynamic Range and Low-Light Imaging on Mobile Cameras (TOG 2016), and Mildenhall et al., NeRF in the Dark / RawNeRF (CVPR 2022) only after implementing a simple raw-noise/response pipeline.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Berkeley CS184 image sensor material where available.
3. Keep the most useful executable/reference page open while coding: PBRT camera/film/radiometry chapters - <https://pbr-book.org/4ed/contents> ; Berkeley CS184 VR material - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Poisson-Gaussian noise intuition
- linear raw vs tone-mapped image
- Precision requirement: distinguish and justify Poisson shot + read-noise sensor model vs simplified Gaussian noise, propagated through a minimal ISP. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a simple photon-shot plus read-noise sensor model, convert irradiance/exposure to expected counts, and derive the variance-stabilizing implications for low-light imaging.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Sweep exposure/gain through shot-noise-, read-noise-, clipping-, and quantization-dominated regimes; compare empirical statistics with the assumed noise model. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Simulate noisy raw sensor and simple ISP pipeline. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For an expected photon count $\lambda=4$ plus Gaussian read noise $\sigma=1$, compute mean/variance of the Poisson-Gaussian model. Compare SNR when exposure increases fourfold. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Sweep exposure/gain through shot-noise-, read-noise-, clipping-, and quantization-dominated regimes; compare empirical statistics with the assumed noise model. Save the input and config of each qualitatively different failure as a named regression case.

- P4 - Research-grade programming extension. Simulate a photon-count sweep and show variance-versus-mean for shot/read noise; pass the raw image through demosaic/white balance/tone mapping and track SNR.
- P5 - Machine-readable evidence. Export variance-vs-mean fit, SNR, reconstruction/color error after ISP, saturation fraction, and sensitivity to exposure/read noise to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: Poisson shot + read-noise sensor model vs simplified Gaussian noise, propagated through a minimal ISP. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Sweep exposure/gain through shot-noise-, read-noise-, clipping-, and quantization-dominated regimes; compare empirical statistics with the assumed noise model." without hand-editing code between methods. Record variance-vs-mean fit, SNR, reconstruction/color error after ISP, saturation fraction, and sensitivity to exposure/read noise.
- P7 - Deliberate bug and regression drill. Intentionally add Poisson/read noise after gamma encoding instead of in the sensor/linear domain. Before running, predict the first observable failure in variance-vs-mean fit, SNR, reconstruction/color error after ISP, saturation fraction, and sensitivity to exposure/read noise. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a simple photon-shot plus read-noise sensor model, convert irradiance/exposure to expected counts, and derive the variance-stabilizing implications for low-light imaging. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For an expected photon count $\lambda=4$ plus Gaussian read noise $\sigma=1$, compute mean/variance of the Poisson-Gaussian model. Compare SNR when exposure increases fourfold. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Simulate noisy raw sensor and simple ISP pipeline. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Simulate a photon-count sweep and show variance-versus-mean for shot/read noise; pass the raw image through demosaic/white balance/tone mapping and track SNR. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare Poisson shot + read-noise sensor model vs simplified Gaussian noise, propagated through a minimal ISP. Use the same scene, optical/sensor/display configuration and exposure/wavelength/FOV conditions; compare to an analytic, calibrated or high-fidelity reference where available. Record variance-vs-mean fit, SNR, reconstruction/color error after ISP, saturation fraction, and sensitivity to exposure/read noise. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Sweep exposure/gain through shot-noise-, read-noise-, clipping-, and quantization-dominated regimes; compare empirical statistics with the assumed noise model. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Graphics/computational photography course notes; RawNeRF as later application. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Image sensors and noise. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among Poisson shot + read-noise sensor model vs simplified Gaussian noise, propagated through a minimal ISP. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using variance-vs-mean fit, SNR, reconstruction/color error after ISP, saturation fraction, and sensitivity to exposure/read noise. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if Poisson shot + read-noise sensor model vs simplified Gaussian noise, propagated through a minimal ISP improve variance-vs-mean fit, SNR, reconstruction/color error after ISP, saturation fraction, and sensitivity to exposure/read noise over multiple exposure/wavelength/FOV/depth conditions against an analytic/calibrated reference, rather than overfitting one optical or display configuration.

F. Concrete research project - Image sensors and noise

F1. Falsifiable question. Under the controlled failure regime "Sweep exposure/gain through shot-noise-, read-noise-, clipping-, and quantization-dominated regimes; compare empirical statistics with the assumed noise model.", which alternative in Poisson shot + read-noise sensor model vs simplified Gaussian noise, propagated through a minimal ISP is preferable, and is the change explained by calibration, optical/sensor modeling error, sampling, and reconstruction identifiability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Simulate a photon-count sweep and show variance-versus-mean for shot/read noise; pass the raw image through demosaic/white balance/tone mapping and track SNR. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Start from synthetic ground truth with known camera/optical parameters, then add at least three noise/calibration levels and one realistic/real capture. Keep calibration error separate from reconstruction/model error. The central adversarial condition is: Sweep exposure/gain through shot-noise-, read-noise-, clipping-, and quantization-dominated regimes; compare empirical statistics with the assumed noise model.

F4. Required evidence. Measure variance-vs-mean fit, SNR, reconstruction/color error after ISP, saturation fraction, and sensitivity to exposure/read noise. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Image sensors and noise. If a stable failure/crossover is found among Poisson shot + read-noise sensor model vs simplified Gaussian noise, propagated through a minimal ISP, implement one adaptive or corrected variant that directly targets calibration, optical/sensor modeling error, sampling, and reconstruction identifiability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Image sensors and noise. Using the measured variance-vs-mean fit, SNR, reconstruction/color error after ISP, saturation fraction, and sensitivity to exposure/read noise, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Image sensors and noise

- G1. Closed-book derivation/oracle. Derive a simple photon-shot plus read-noise sensor model, convert irradiance/exposure to expected counts, and derive the variance-stabilizing implications for low-light imaging. Then solve this exact micro-case from scratch without opening your implementation: For an expected photon count $\lambda=4$ plus Gaussian read noise $\sigma=1$, compute mean/variance of the Poisson-Gaussian model. Compare SNR when exposure increases fourfold. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Simulate noisy raw sensor and simple ISP pipeline. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally add Poisson/read noise after gamma encoding instead of in the sensor/linear domain. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Simulate a photon-count sweep and show variance-versus-mean for shot/read noise; pass the raw image through demosaic/white balance/tone mapping and track SNR. Then apply this stress without retuning: Sweep exposure/gain through shot-noise-, read-noise-, clipping-, and quantization-dominated regimes; compare empirical statistics with the assumed noise model. Report variance-vs-mean fit, SNR, reconstruction/color error after ISP, saturation fraction, and sensitivity to exposure/read noise and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among Poisson shot + read-noise sensor model vs simplified Gaussian noise, propagated through a minimal ISP for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured variance-vs-mean fit, SNR, reconstruction/color error after ISP,

saturation fraction, and sensitivity to exposure/read noise; state one counterexample that would reverse your recommendation.

- G6. Pass condition for Image sensors and noise. Using variance-vs-mean fit, SNR, reconstruction/color error after ISP, saturation fraction, and sensitivity to exposure/read noise, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 12, Unit 3. Optics for graphics

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Reflection/refraction, Fresnel, geometric vs wave optics, diffraction limits concept.

A. Reading order

1. Read the primary assignment first: PBRT Ch. 9 for geometric scattering; an introductory Fourier optics text for wave optics.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Mitsuba optics/polarization examples later.
3. Keep the most useful executable/reference page open while coding: PBRT camera/film/radiometry chapters - <https://pbr-book.org/4ed/contents> ; Berkeley CS184 VR material - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Snell law, Fresnel coefficients, Fourier optics bridge
- Precision requirement: distinguish and justify geometric ray optics vs Fourier/diffraction model on aperture/prism/lens cases where wave effects become visible. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive Snell refraction from Fermat or wave-vector continuity, then derive thin-lens imaging and basic aberration consequences for finite apertures.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use apertures/features approaching wavelength scale and high numerical aperture; compare geometric versus Fourier/wave predictions. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Ray trace a compound lens or prism; optional FFT diffraction demo. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Refract a ray from air into glass $n=1.5$ at 30 and 60 degrees. Compute transmitted angles and Fresnel normal-incidence reflectance approximation; identify total-internal-reflection condition in reverse direction. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use apertures/features approaching wavelength scale and high numerical aperture; compare geometric versus Fourier/wave predictions. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement both a ray-based lens/prism trace and an FFT aperture diffraction demo; identify the parameter regime where the ray model misses visible structure.
- P5 - Machine-readable evidence. Export ray/wave prediction mismatch, PSF/diffraction pattern error, focal/dispersion error, and compute cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: geometric ray optics vs Fourier/diffraction model on aperture/prism/lens cases where wave effects become visible. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use apertures/features approaching wavelength scale and high numerical aperture; compare geometric versus Fourier/wave predictions." without hand-editing code between methods. Record ray/wave prediction mismatch, PSF/diffraction pattern error, focal/dispersion error, and compute cost.
- P7 - Deliberate bug and regression drill. Intentionally use the wrong Fourier/Fresnel propagation scaling or omit complex phase while comparing intensity only. Before running, predict the first observable failure in ray/wave

prediction mismatch, PSF/diffraction pattern error, focal/dispersion error, and compute cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive Snell refraction from Fermat or wave-vector continuity, then derive thin-lens imaging and basic aberration consequences for finite apertures. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Refract a ray from air into glass $n=1.5$ at 30 and 60 degrees. Compute transmitted angles and Fresnel normal-incidence reflectance approximation; identify total-internal-reflection condition in reverse direction. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Ray trace a compound lens or prism; optional FFT diffraction demo. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement both a ray-based lens/prism trace and an FFT aperture diffraction demo; identify the parameter regime where the ray model misses visible structure. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare geometric ray optics vs Fourier/diffraction model on aperture/prism/lens cases where wave effects become visible. Use the same scene, optical/sensor/display configuration and exposure/wavelength/FOV conditions; compare to an analytic, calibrated or high-fidelity reference where available. Record ray/wave prediction mismatch, PSF/diffraction pattern error, focal/dispersion error, and compute cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use apertures/features approaching wavelength scale and high numerical aperture; compare geometric versus Fourier/wave predictions. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: PBRT Ch. 9 for geometric scattering; an introductory Fourier optics text for wave optics. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Optics for graphics. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among geometric ray optics vs Fourier/diffraction model on aperture/prism/lens cases where wave effects become visible. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using ray/wave prediction mismatch, PSF/diffraction pattern error, focal/dispersion error, and compute cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if geometric ray optics vs Fourier/diffraction model on aperture/prism/lens cases where wave effects become visible improve ray/wave prediction mismatch, PSF/diffraction pattern error, focal/dispersion error, and compute cost over multiple exposure/wavelength/FOV/depth conditions against an analytic/calibrated reference, rather than overfitting one optical or display configuration.

F. Concrete research project - Optics for graphics

- F1. Falsifiable question. Under the controlled failure regime "Use apertures/features approaching wavelength scale and high numerical aperture; compare geometric versus Fourier/wave predictions.", which alternative in geometric ray optics vs Fourier/diffraction model on aperture/prism/lens cases where wave effects become visible is preferable, and is the change explained by calibration, optical/sensor modeling error, sampling, and reconstruction identifiability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement both a ray-based lens/prism trace and an FFT aperture diffraction demo; identify the parameter regime where the ray model misses visible structure. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Start from synthetic ground truth with known camera/optical parameters, then add at least three noise/calibration levels and one realistic/real capture. Keep calibration error separate from reconstruction/model error. The central adversarial condition is: Use apertures/features approaching wavelength scale and high numerical aperture; compare geometric versus Fourier/wave predictions.

F4. Required evidence. Measure ray/wave prediction mismatch, PSF/diffraction pattern error, focal/dispersion error, and compute cost. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Optics for graphics. If a stable failure/crossover is found among geometric ray optics vs Fourier/diffraction model on aperture/prism/lens cases where wave effects become visible, implement one adaptive or corrected variant that directly targets calibration, optical/sensor modeling error, sampling, and reconstruction identifiability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Optics for graphics. Using the measured ray/wave prediction mismatch, PSF/diffraction pattern error, focal/dispersion error, and compute cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Optics for graphics

- G1. Closed-book derivation/oracle. Derive Snell refraction from Fermat or wave-vector continuity, then derive thin-lens imaging and basic aberration consequences for finite apertures. Then solve this exact micro-case from scratch without opening your implementation: Refract a ray from air into glass $n=1.5$ at 30 and 60 degrees. Compute transmitted angles and Fresnel normal-incidence reflectance approximation; identify total-internal-reflection condition in reverse direction. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Ray trace a compound lens or prism; optional FFT diffraction demo. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use the wrong Fourier/Fresnel propagation scaling or omit complex phase while comparing intensity only. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement both a ray-based lens/prism trace and an FFT aperture diffraction demo; identify the parameter regime where the ray model misses visible structure. Then apply this stress without retuning: Use apertures/features approaching wavelength scale and high numerical aperture; compare geometric versus Fourier/wave predictions. Report ray/wave prediction mismatch, PSF/diffraction pattern error, focal/dispersion error, and compute cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among geometric ray optics vs Fourier/diffraction model on aperture/prism/lens cases where wave effects become visible for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured ray/wave prediction mismatch, PSF/diffraction pattern error, focal/dispersion error, and compute cost; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Optics for graphics. Using ray/wave prediction mismatch, PSF/diffraction pattern error, focal/dispersion error, and compute cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 12, Unit 4. Light fields and plenoptic imaging

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Two-plane parameterization, refocusing, synthetic aperture.

A. Reading order

1. Read the primary assignment first: Read Levoy & Hanrahan, Light Field Rendering (SIGGRAPH 1996), and Gortler et al., The Lumigraph (SIGGRAPH 1996), then a plenoptic-camera/refocusing tutorial or paper for the two-plane parameterization.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Graphics course light-field lectures.
3. Keep the most useful executable/reference page open while coding: PBRT camera/film/radiometry chapters - <https://pbr-book.org/4ed/contents> ; Berkeley CS184 VR material - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- 4D light field
- epipolar-plane image slope
- refocus by shearing/integration
- Precision requirement: distinguish and justify subaperture refocusing and focal-stack reconstruction across baseline/angular sampling and depth. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive two-plane light-field parameterization and refocusing by shearing/integration; connect spatial/angular sampling to depth-dependent aliasing.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Undersample spatial and angular dimensions separately; measure refocus/view-synthesis artifacts as a function of sampling budget. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement digital refocusing from a small light-field dataset. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Using two parallel planes $z=0$ and $z=1$, parameterize the ray through $(u,v)=(0,0)$ and $(s,t)=(0.2,0)$. Compute its intersection with $z=2$ and how that changes when s is shifted. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Undersample spatial and angular dimensions separately; measure refocus/view-synthesis artifacts as a function of sampling budget. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Refocus a public/synthetic light field across depth and measure sharpness/error versus angular sampling and baseline.
- P5 - Machine-readable evidence. Export refocus/depth error, angular aliasing, sharpness, reconstruction PSNR, and sensitivity to baseline/view count to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: subaperture refocusing and focal-stack reconstruction across baseline/angular sampling and depth. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Undersample spatial and angular dimensions separately; measure refocus/view-synthesis artifacts as a function of sampling budget." without hand-editing code between methods. Record refocus/depth error, angular aliasing, sharpness, reconstruction PSNR, and sensitivity to baseline/view count.
- P7 - Deliberate bug and regression drill. Intentionally swap the two-plane parameterization sign/order so refocusing shears in the wrong direction. Before running, predict the first observable failure in refocus/depth error, angular aliasing, sharpness, reconstruction PSNR, and sensitivity to baseline/view count. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive two-plane light-field parameterization and refocusing by shearing/integration; connect spatial/angular sampling to depth-dependent aliasing. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Using two parallel planes $z=0$ and $z=1$, parameterize the ray through $(u,v)=(0,0)$ and $(s,t)=(0.2,0)$. Compute its intersection with $z=2$ and how that changes when s is shifted. Deliverable: every intermediate

value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Implement digital refocusing from a small light-field dataset. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Refocus a public/synthetic light field across depth and measure sharpness/error versus angular sampling and baseline. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare subaperture refocusing and focal-stack reconstruction across baseline/angular sampling and depth. Use the same scene, optical/sensor/display configuration and exposure/wavelength/FOV conditions; compare to an analytic, calibrated or high-fidelity reference where available. Record refocus/depth error, angular aliasing, sharpness, reconstruction PSNR, and sensitivity to baseline/view count. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Undersample spatial and angular dimensions separately; measure refocus/view-synthesis artifacts as a function of sampling budget. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Light-field course notes / classic Levoy-Hanrahan paper. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Light fields and plenoptic imaging. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among subaperture refocusing and focal-stack reconstruction across baseline/angular sampling and depth. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using refocus/depth error, angular aliasing, sharpness, reconstruction PSNR, and sensitivity to baseline/view count. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if subaperture refocusing and focal-stack reconstruction across baseline/angular sampling and depth improve refocus/depth error, angular aliasing, sharpness, reconstruction PSNR, and sensitivity to baseline/view count over multiple exposure/wavelength/FOV/depth conditions against an analytic/calibrated reference, rather than overfitting one optical or display configuration.

F. Concrete research project - Light fields and plenoptic imaging

- F1. Falsifiable question. Under the controlled failure regime "Undersample spatial and angular dimensions separately; measure refocus/view-synthesis artifacts as a function of sampling budget.", characterize the ranking or regime transition across subaperture refocusing and focal-stack reconstruction across baseline/angular sampling and depth. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by calibration, optical/sensor modeling error, sampling, and reconstruction identifiability. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Refocus a public/synthetic light field across depth and measure sharpness/error versus angular sampling and baseline. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Start from synthetic ground truth with known camera/optical parameters, then add at least three noise/calibration levels and one realistic/real capture. Keep calibration error separate from reconstruction/model error. The central adversarial condition is: Undersample spatial and angular dimensions separately; measure refocus/view-synthesis artifacts as a function of sampling budget.
- F4. Required evidence. Measure refocus/depth error, angular aliasing, sharpness, reconstruction PSNR, and sensitivity to baseline/view count. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Light fields and plenoptic imaging. If a stable failure/crossover is found among subaperture refocusing and focal-stack reconstruction across baseline/angular sampling and depth, implement one adaptive or corrected variant that directly targets calibration, optical/sensor modeling error, sampling, and reconstruction identifiability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Light fields and plenoptic imaging. Using the measured refocus/depth error, angular aliasing, sharpness, reconstruction PSNR, and sensitivity to baseline/view count, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Light fields and plenoptic imaging

- G1. Closed-book derivation/oracle. Derive two-plane light-field parameterization and refocusing by shearing/integration; connect spatial/angular sampling to depth-dependent aliasing. Then solve this exact micro-case from scratch without opening your implementation: Using two parallel planes $z=0$ and $z=1$, parameterize the ray through $(u,v)=(0,0)$ and $(s,t)=(0.2,0)$. Compute its intersection with $z=2$ and how that changes when s is shifted. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement digital refocusing from a small light-field dataset. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally swap the two-plane parameterization sign/order so refocusing shears in the wrong direction. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Refocus a public/synthetic light field across depth and measure sharpness/error versus angular sampling and baseline. Then apply this stress without retuning: Undersample spatial and angular dimensions separately; measure refocus/view-synthesis artifacts as a function of sampling budget. Report refocus/depth error, angular aliasing, sharpness, reconstruction PSNR, and sensitivity to baseline/view count and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across subaperture refocusing and focal-stack reconstruction across baseline/angular sampling and depth for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured refocus/depth error, angular aliasing, sharpness, reconstruction PSNR, and sensitivity to baseline/view count; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Light fields and plenoptic imaging. Using refocus/depth error, angular aliasing, sharpness, reconstruction PSNR, and sensitivity to baseline/view count, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 12, Unit 5. Computational photography

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. HDR, deblurring, coded aperture/exposure, burst imaging, inverse problems.

A. Reading order

1. Read the primary assignment first: Use a computational-photography course/text for burst alignment, deconvolution, HDR and inverse problems; reproduce one classical burst/HDR/defocus experiment before reading RawNeRF or neural capture papers.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Implement Richardson-Lucy or regularized deconvolution on synthetic blur.
3. Keep the most useful executable/reference page open while coding: PBRT camera/film/radiometry chapters - <https://pbr-book.org/4ed/contents> ; Berkeley CS184 VR material - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Forward model $y = Ax + n$
- regularization
- deconvolution conditioning
- Precision requirement: distinguish and justify uncoded measurement vs one coded aperture/exposure/illumination design with matched photon/noise budget. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Formulate one computational-photography task (deblurring, HDR, coded aperture, burst reconstruction) as a forward model plus inverse problem; derive the least-squares/MAP objective.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Perturb calibration and noise beyond the design assumptions of a coded capture; inspect inverse conditioning and reconstruction bias. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Design one coded imaging toy problem and solve it numerically. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Merge two linear sensor measurements of the same radiance L with exposures 1 and 4 when the long exposure saturates above 1. Use a simple weighted estimator and identify which measurement should be trusted in dark/bright regimes. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Perturb calibration and noise beyond the design assumptions of a coded capture; inspect inverse conditioning and reconstruction bias. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement one coded-deblurring/HDR/structured-light toy inverse problem with a known synthetic truth and noise sweep.
- P5 - Machine-readable evidence. Export reconstruction/parameter error to known truth, noise amplification/condition number, acquisition budget, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: uncoded measurement vs one coded aperture/exposure/illumination design with matched photon/noise budget. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Perturb calibration and noise beyond the design assumptions of a coded capture; inspect inverse conditioning and reconstruction bias." without hand-editing code between methods. Record reconstruction/parameter error to known truth, noise amplification/condition number, acquisition budget, and runtime.
- P7 - Deliberate bug and regression drill. Intentionally average a burst without alignment and interpret motion blur as a denoising limitation. Before running, predict the first observable failure in reconstruction/parameter error to known truth, noise amplification/condition number, acquisition budget, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Formulate one computational-photography task (deblurring, HDR, coded aperture, burst reconstruction) as a forward model plus inverse problem; derive the least-squares/MAP objective. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Merge two linear sensor measurements of the same radiance L with exposures 1 and 4 when the long exposure saturates above 1. Use a simple weighted estimator and identify which measurement should be trusted in dark/bright regimes. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Design one coded imaging toy problem and solve it numerically. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement one coded-deblurring/HDR/structured-light toy inverse problem with a known synthetic truth and noise sweep. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare uncoded measurement vs one coded aperture/exposure/illumination design with matched photon/noise budget. Use the same scene, optical/sensor/display configuration and exposure/wavelength/FOV conditions; compare to an analytic, calibrated or high-fidelity reference where available. Record reconstruction/parameter error to known truth, noise amplification/condition number, acquisition budget, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Perturb calibration and noise beyond the design assumptions of a coded capture; inspect inverse conditioning and reconstruction bias. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Selected computational photography notes/papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Computational photography. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among uncoded measurement vs one coded aperture/exposure/illumination design with matched photon/noise budget. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using reconstruction/parameter error to known truth, noise amplification/condition number, acquisition budget, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if uncoded measurement vs one coded aperture/exposure/illumination design with matched photon/noise budget improve reconstruction/parameter error to known truth, noise amplification/condition number, acquisition budget, and runtime over multiple exposure/wavelength/FOV/depth conditions against an analytic/calibrated reference, rather than overfitting one optical or display configuration.

F. Concrete research project - Computational photography

- F1. Falsifiable question. Under the controlled failure regime "Perturb calibration and noise beyond the design assumptions of a coded capture; inspect inverse conditioning and reconstruction bias.", which alternative in uncoded measurement vs one coded aperture/exposure/illumination design with matched photon/noise budget is preferable, and is the change explained by calibration, optical/sensor modeling error, sampling, and reconstruction identifiability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement one coded-deblurring/HDR/structured-light toy inverse problem with a known synthetic truth and noise sweep. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Start from synthetic ground truth with known camera/optical parameters, then add at least three noise/calibration levels and one realistic/real capture. Keep calibration error separate from reconstruction/model error. The central adversarial condition is: Perturb calibration and noise beyond the design assumptions of a coded capture; inspect inverse conditioning and reconstruction bias.
- F4. Required evidence. Measure reconstruction/parameter error to known truth, noise amplification/condition number, acquisition budget, and runtime. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Computational photography. If a stable failure/crossover is found among uncoded measurement vs one coded aperture/exposure/illumination design with matched photon/noise budget, implement one adaptive or corrected variant that directly targets calibration, optical/sensor modeling error, sampling, and reconstruction identifiability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Computational photography. Using the measured reconstruction/parameter error to known truth, noise amplification/condition number, acquisition budget, and runtime, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Computational photography

- G1. Closed-book derivation/oracle. Formulate one computational-photography task (deblurring, HDR, coded aperture, burst reconstruction) as a forward model plus inverse problem; derive the least-squares/MAP objective. Then solve this exact micro-case from scratch without opening your implementation: Merge two linear sensor measurements of the same radiance L with exposures 1 and 4 when the long exposure saturates above 1. Use a

simple weighted estimator and identify which measurement should be trusted in dark/bright regimes. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Design one coded imaging toy problem and solve it numerically. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally average a burst without alignment and interpret motion blur as a denoising limitation. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement one coded-deblurring/HDR/structured-light toy inverse problem with a known synthetic truth and noise sweep. Then apply this stress without retuning: Perturb calibration and noise beyond the design assumptions of a coded capture; inspect inverse conditioning and reconstruction bias. Report reconstruction/parameter error to known truth, noise amplification/condition number, acquisition budget, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among uncoded measurement vs one coded aperture/exposure/illumination design with matched photon/noise budget for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured reconstruction/parameter error to known truth, noise amplification/condition number, acquisition budget, and runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Computational photography. Using reconstruction/parameter error to known truth, noise amplification/condition number, acquisition budget, and runtime, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 12, Unit 6. VR/AR rendering and displays

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Stereo geometry, head tracking, latency, distortion, foveation, reprojection.

A. Reading order

1. Read the primary assignment first: CS184 VR lectures; RTR display/VR resource sections.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: OpenXR/WebXR tutorials optional.
3. Keep the most useful executable/reference page open while coding: PBRT camera/film/radiometry chapters - <https://pbr-book.org/4ed/contents> ; Berkeley CS184 VR material - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Binocular disparity
- motion-to-photon latency
- lens warp
- Precision requirement: distinguish and justify mono vs stereo rendering; undistorted vs prewarped output; latency/FOV/resolution tradeoffs in a simple headset model. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive stereo projection and disparity-depth relationships, then derive one reprojection/time-warp transform; connect errors to latency, vergence, and display geometry.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Sweep latency, foveation error, reprojection, and head-motion speed; include a user/task outcome as well as image quality. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Build stereo renderer or WebXR toy scene; simulate lens distortion correction. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For eye separation 6.4 cm and screen/focal distance 1 m, compute binocular disparity for points at 0.5, 1, 2 m in a simplified pinhole stereo model.

Relate sign/magnitude to perceived depth. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Sweep latency, foveation error, reprojection, and head-motion speed; include a user/task outcome as well as image quality. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build stereo projection with configurable IPD/FOV and radial prewarp; quantify disparity, distortion residual, and sensitivity to pose latency.
- P5 - Machine-readable evidence. Export stereo disparity/reprojection error, distortion residual, latency-induced pixel error, FOV/resolution tradeoff, and frame time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: mono vs stereo rendering; undistorted vs prewarped output; latency/FOV/resolution tradeoffs in a simple headset model. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Sweep latency, foveation error, reprojection, and head-motion speed; include a user/task outcome as well as image quality." without hand-editing code between methods. Record stereo disparity/reprojection error, distortion residual, latency-induced pixel error, FOV/resolution tradeoff, and frame time.
- P7 - Deliberate bug and regression drill. Intentionally reuse one projection/view transform for both eyes or evaluate latency without late/head-pose update. Before running, predict the first observable failure in stereo disparity/reprojection error, distortion residual, latency-induced pixel error, FOV/resolution tradeoff, and frame time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive stereo projection and disparity-depth relationships, then derive one reprojection/time-warp transform; connect errors to latency, vergence, and display geometry. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For eye separation 6.4 cm and screen/focal distance 1 m, compute binocular disparity for points at 0.5, 1, 2 m in a simplified pinhole stereo model. Relate sign/magnitude to perceived depth. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Build stereo renderer or WebXR toy scene; simulate lens distortion correction. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build stereo projection with configurable IPD/FOV and radial prewarp; quantify disparity, distortion residual, and sensitivity to pose latency. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare mono vs stereo rendering; undistorted vs prewarped output; latency/FOV/resolution tradeoffs in a simple headset model. Use the same scene, optical/sensor/display configuration and exposure/wavelength/FOV conditions; compare to an analytic, calibrated or high-fidelity reference where available. Record stereo disparity/reprojection error, distortion residual, latency-induced pixel error, FOV/resolution tradeoff, and frame time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Sweep latency, foveation error, reprojection, and head-motion speed; include a user/task outcome as well as image quality. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: CS184 VR lectures; RTR display/VR resource sections. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on VR/AR rendering and displays. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among mono vs stereo rendering; undistorted vs prewarped output; latency/FOV/resolution tradeoffs in a simple headset model. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using stereo disparity/reprojection error, distortion residual, latency-induced pixel error, FOV/resolution tradeoff, and frame time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if mono vs stereo rendering;

undistorted vs prewarped output; latency/FOV/resolution tradeoffs in a simple headset model improve stereo disparity/reprojection error, distortion residual, latency-induced pixel error, FOV/resolution tradeoff, and frame time over multiple exposure/wavelength/FOV/depth conditions against an analytic/calibrated reference, rather than overfitting one optical or display configuration.

F. Concrete research project - VR/AR rendering and displays

F1. Falsifiable question. Under the controlled failure regime "Sweep latency, foveation error, reprojection, and head-motion speed; include a user/task outcome as well as image quality.", which alternative in mono vs stereo rendering; undistorted vs prewarped output; latency/FOV/resolution tradeoffs in a simple headset model is preferable, and is the change explained by calibration, optical/sensor modeling error, sampling, and reconstruction identifiability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build stereo projection with configurable IPD/FOV and radial prewarp; quantify disparity, distortion residual, and sensitivity to pose latency. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Start from synthetic ground truth with known camera/optical parameters, then add at least three noise/calibration levels and one realistic/real capture. Keep calibration error separate from reconstruction/model error. The central adversarial condition is: Sweep latency, foveation error, reprojection, and head-motion speed; include a user/task outcome as well as image quality.

F4. Required evidence. Measure stereo disparity/reprojection error, distortion residual, latency-induced pixel error, FOV/resolution tradeoff, and frame time. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for VR/AR rendering and displays. If a stable failure/crossover is found among mono vs stereo rendering; undistorted vs prewarped output; latency/FOV/resolution tradeoffs in a simple headset model, implement one adaptive or corrected variant that directly targets calibration, optical/sensor modeling error, sampling, and reconstruction identifiability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for VR/AR rendering and displays. Using the measured stereo disparity/reprojection error, distortion residual, latency-induced pixel error, FOV/resolution tradeoff, and frame time, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for VR/AR rendering and displays

- G1. Closed-book derivation/oracle. Derive stereo projection and disparity-depth relationships, then derive one reprojection/time-warp transform; connect errors to latency, vergence, and display geometry. Then solve this exact micro-case from scratch without opening your implementation: For eye separation 6.4 cm and screen/focal distance 1 m, compute binocular disparity for points at 0.5, 1, 2 m in a simplified pinhole stereo model. Relate sign/magnitude to perceived depth. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Build stereo renderer or WebXR toy scene; simulate lens distortion correction. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally reuse one projection/view transform for both eyes or evaluate latency without late/head-pose update. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Build stereo projection with configurable IPD/FOV and radial prewarp; quantify disparity, distortion residual, and sensitivity to pose latency. Then apply this stress without retuning: Sweep latency, foveation error, reprojection, and head-motion speed; include a user/task outcome as well as image quality. Report stereo disparity/reprojection error, distortion residual, latency-induced pixel error, FOV/resolution tradeoff, and frame time and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method-selection defense. Using your own measured results, choose among mono vs stereo rendering; undistorted vs prewarped output; latency/FOV/resolution tradeoffs in a simple headset model for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured stereo disparity/reprojection error, distortion residual, latency-induced pixel error, FOV/resolution tradeoff, and frame time; state one counterexample that would reverse your recommendation.
- G6. Pass condition for VR/AR rendering and displays. Using stereo disparity/reprojection error, distortion residual, latency-induced pixel error, FOV/resolution tradeoff, and frame time, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 12, Unit 7. Holography and computational displays

Recommended time. 2.0-2.6 weeks (about 30-51 focused hours depending on background).

What you are learning. Wavefront encoding, phase-only modulation, inverse design basics.

A. Reading order

1. Read the primary assignment first: Read Goodman, Introduction to Fourier Optics, the chapters on scalar diffraction/Fresnel/Fraunhofer propagation, then one primary computational-display/holography paper with code or reproducible optical parameters.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Start with FFT-based Fresnel propagation notebook.
3. Keep the most useful executable/reference page open while coding: PBRT camera/film/radiometry chapters - <https://pbr-book.org/4ed/contents> ; Berkeley CS184 VR material - <https://cs184.eecs.berkeley.edu/sp26/>

B. Definitions, results and algorithms to know

- Fresnel/Fraunhofer propagation
- phase retrieval intuition
- Precision requirement: distinguish and justify Gerchberg-Saxton phase-only hologram vs naïve phase assignment over propagation distance and spatial-frequency targets. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive scalar diffraction propagation under Fresnel/Fraunhofer approximations and formulate phase-only hologram optimization for a target intensity plane.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use multiple depths, phase quantization, finite aperture, and wavelength mismatch; measure speckle, depth fidelity, and optimization stability. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement Gerchberg-Saxton phase retrieval and reconstruct simple target patterns. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a point source at distance z , compute relative Fresnel phase at two aperture samples $x=0$ and $x=\delta$ using the quadratic phase approximation. Predict fringe spacing trend as z changes. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use multiple depths, phase quantization, finite aperture, and wavelength mismatch; measure speckle, depth fidelity, and optimization stability. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement angular-spectrum/Fresnel propagation plus Gerchberg-Saxton; report reconstruction error versus iterations, propagation distance, and phase quantization.
- P5 - Machine-readable evidence. Export target-plane intensity/phase error, speckle/contrast, convergence versus iterations, quantization sensitivity, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: Gerchberg-Saxton phase-only hologram vs naïve phase assignment over propagation distance and spatial-frequency targets. The

harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use multiple depths, phase quantization, finite aperture, and wavelength mismatch; measure speckle, depth fidelity, and optimization stability." without hand-editing code between methods. Record target-plane intensity/phase error, speckle/contrast, convergence versus iterations, quantization sensitivity, and runtime.

- P7 - Deliberate bug and regression drill. Intentionally wrap/quantize phase inconsistently with the propagation model, creating a false reconstruction improvement. Before running, predict the first observable failure in target-plane intensity/phase error, speckle/contrast, convergence versus iterations, quantization sensitivity, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive scalar diffraction propagation under Fresnel/Fraunhofer approximations and formulate phase-only hologram optimization for a target intensity plane. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a point source at distance z , compute relative Fresnel phase at two aperture samples $x=0$ and $x=\Delta$ using the quadratic phase approximation. Predict fringe spacing trend as z changes. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement Gerchberg-Saxton phase retrieval and reconstruct simple target patterns. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement angular-spectrum/Fresnel propagation plus Gerchberg-Saxton; report reconstruction error versus iterations, propagation distance, and phase quantization. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare Gerchberg-Saxton phase-only hologram vs naïve phase assignment over propagation distance and spatial-frequency targets. Use the same scene, optical/sensor/display configuration and exposure/wavelength/FOV conditions; compare to an analytic, calibrated or high-fidelity reference where available. Record target-plane intensity/phase error, speckle/contrast, convergence versus iterations, quantization sensitivity, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use multiple depths, phase quantization, finite aperture, and wavelength mismatch; measure speckle, depth fidelity, and optimization stability. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Recent SIGGRAPH computational display papers; optics references. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Holography and computational displays. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among Gerchberg-Saxton phase-only hologram vs naïve phase assignment over propagation distance and spatial-frequency targets. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using target-plane intensity/phase error, speckle/contrast, convergence versus iterations, quantization sensitivity, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if Gerchberg-Saxton phase-only hologram vs naïve phase assignment over propagation distance and spatial-frequency targets improve target-plane intensity/phase error, speckle/contrast, convergence versus iterations, quantization sensitivity, and runtime over multiple exposure/wavelength/FOV/depth conditions against an analytic/calibrated reference, rather than overfitting one optical or display configuration.

F. Concrete research project - Holography and computational displays

F1. Falsifiable question. Under the controlled failure regime "Use multiple depths, phase quantization, finite aperture, and wavelength mismatch; measure speckle, depth fidelity, and optimization stability.", which alternative in Gerchberg-Saxton phase-only hologram vs naïve phase assignment over propagation distance and spatial-frequency targets is preferable, and is the change explained by calibration, optical/sensor modeling error, sampling, and

reconstruction identifiability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement angular-spectrum/Fresnel propagation plus Gerchberg-Saxton; report reconstruction error versus iterations, propagation distance, and phase quantization. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Start from synthetic ground truth with known camera/optical parameters, then add at least three noise/calibration levels and one realistic/real capture. Keep calibration error separate from reconstruction/model error. The central adversarial condition is: Use multiple depths, phase quantization, finite aperture, and wavelength mismatch; measure speckle, depth fidelity, and optimization stability.

F4. Required evidence. Measure target-plane intensity/phase error, speckle/contrast, convergence versus iterations, quantization sensitivity, and runtime. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Holography and computational displays. If a stable failure/crossover is found among Gerchberg-Saxton phase-only hologram vs naïve phase assignment over propagation distance and spatial-frequency targets, implement one adaptive or corrected variant that directly targets calibration, optical/sensor modeling error, sampling, and reconstruction identifiability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Holography and computational displays. Using the measured target-plane intensity/phase error, speckle/contrast, convergence versus iterations, quantization sensitivity, and runtime, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Holography and computational displays

- G1. Closed-book derivation/oracle. Derive scalar diffraction propagation under Fresnel/Fraunhofer approximations and formulate phase-only hologram optimization for a target intensity plane. Then solve this exact micro-case from scratch without opening your implementation: For a point source at distance z , compute relative Fresnel phase at two aperture samples $x=0$ and $x=\delta$ using the quadratic phase approximation. Predict fringe spacing trend as z changes. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement Gerchberg-Saxton phase retrieval and reconstruct simple target patterns. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally wrap/quantize phase inconsistently with the propagation model, creating a false reconstruction improvement. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement angular-spectrum/Fresnel propagation plus Gerchberg-Saxton; report reconstruction error versus iterations, propagation distance, and phase quantization. Then apply this stress without retuning: Use multiple depths, phase quantization, finite aperture, and wavelength mismatch; measure speckle, depth fidelity, and optimization stability. Report target-plane intensity/phase error, speckle/contrast, convergence versus iterations, quantization sensitivity, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among Gerchberg-Saxton phase-only hologram vs naïve phase assignment over propagation distance and spatial-frequency targets for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured target-plane intensity/phase error, speckle/contrast, convergence versus iterations, quantization sensitivity, and runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Holography and computational displays. Using target-plane intensity/phase error, speckle/contrast, convergence versus iterations, quantization sensitivity, and runtime, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases,

and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 12 Closeout: Project, Review and Mastery Gate

Volume project. Build a computational imaging notebook covering sensor noise, thin-lens/depth-of-field, deconvolution, light-field refocusing and one Fourier-optics/holography example.

Cumulative review

- Build a calibrated synthetic imaging testbed with known camera/optical parameters and noise.
- Verify pinhole/thin-lens/light-field/Fourier-optics equations on hand-computable or high-accuracy cases.
- Separate sensor/noise/calibration error from reconstruction/model error in at least one computational-imaging task.
- Measure one display/VR pipeline in terms of latency/reprojection/spatial error rather than appearance alone.
- Write an imaging synthesis connecting radiometry, optics, sampling, inverse problems and display constraints.

Mastery gate before moving on

- Validate camera, sensor, and optical forward models against analytic/calibrated synthetic cases.
- Solve one inverse computational-imaging task and map sensitivity to noise and calibration.
- Explain when pinhole/ray optics, Fourier optics, light-field, or display models are required rather than interchangeable approximations.

Volume 13. Differentiable Rendering and Inverse Graphics

Purpose. Learn to differentiate image formation, formulate inverse problems, and use rendering as a differentiable computational layer.

Suggested duration. 16-20 weeks

Prerequisites. Volumes 1, 5-6 and 12; optimization/autodiff basics required.

Typical module pace. 2.0-2.5 weeks per module (8 modules total).

Primary references for this volume

- Mitsuba 3 and Dr.Jit official tutorials (primary executable spine).
- PBRT for the forward model.
- Nocedal & Wright for optimization background.
- Foundational differentiable-rendering papers.

Online spine

Mitsuba 3 documentation: <https://mitsuba.readthedocs.io/en/stable/>

Mitsuba 3 inverse rendering tutorials: https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering_tutorials.html

Canonical paper track

- Loper & Black (2014), OpenDR: An Approximate Differentiable Renderer.
- Li et al. (2018), Differentiable Monte Carlo Ray Tracing through Edge Sampling.
- Nimier-David et al. (2019), Mitsuba 2: A Retargetable Forward and Inverse Renderer.
- Vicini et al. / projective sampling and path-replay style differentiable rendering papers.
- Mitsuba 3 inverse-rendering tutorials as executable companion material.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Loper & Black (2014), OpenDR: An Approximate Differentiable Renderer.

- Kato, Ushiku & Harada (2018), Neural 3D Mesh Renderer.
- Li et al. (2018), Differentiable Monte Carlo Ray Tracing through Edge Sampling.
- Nimier-David et al. (2019), Mitsuba 2: A Retargetable Forward and Inverse Renderer.
- Mitsuba 3 inverse-rendering tutorials - gradient optimization, pose, caustics, projective sampling, shape, volume and radiance-field examples.
- For each paper, explicitly classify which derivatives are pathwise/smooth, which involve visibility/discontinuities, and which approximations can bias optimization.

Volume 13, Unit 1. Inverse problem formulation

Recommended time. 2.0-2.5 weeks (about 30-50 focused hours depending on background).

What you are learning. Unknown geometry/material/light/camera parameters, losses, priors, identifiability.

A. Reading order

1. Read the primary assignment first: Mitsuba 3 inverse-rendering introduction; optimization foundations from Vol. 1.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Mitsuba gradient-based optimization tutorial.
3. Keep the most useful executable/reference page open while coding: Mitsuba 3 inverse-rendering tutorials - https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering_tutorials.html

B. Definitions, results and algorithms to know

- Forward operator
- ill-posedness
- regularization
- gauge ambiguities
- Precision requirement: distinguish and justify direct unconstrained parameters vs physically constrained/reparameterized variables and regularizers on synthetic ground-truth recovery. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a graphics inverse problem from a forward renderer, likelihood/data term, prior/regularizer, and constraints; identify non-identifiability and gauge freedoms on a toy scene.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Construct two or more scene parameterizations that explain nearly the same images; use Hessian/Jacobian structure or perturbation experiments to expose ambiguity. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Fit simple scene color/light/camera parameters from synthetic target images. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For image intensity $I=a*1$ from unknown albedo a and light l , show the scale ambiguity from one observation. Add a second known-light observation and solve for a , explicitly distinguishing identifiability from numerical optimization. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Construct two or more scene parameterizations that explain nearly the same images; use Hessian/Jacobian structure or perturbation experiments to expose ambiguity. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create a synthetic scene with known parameters, solve with unconstrained and constrained parameterizations, and plot recovery error/objective over initializations.
- P5 - Machine-readable evidence. Export parameter recovery error, data residual, constraint violations, condition/identifiability indicators, and success over initializations to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: direct unconstrained parameters vs physically constrained/reparameterized variables and regularizers on synthetic ground-truth recovery. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Construct two or more scene parameterizations that explain nearly the same images; use Hessian/Jacobian structure or perturbation experiments to expose ambiguity." without

hand-editing code between methods. Record parameter recovery error, data residual, constraint violations, condition/identifiability indicators, and success over initializations.

- P7 - Deliberate bug and regression drill. Intentionally compare parameter recovery while leaving an unremoved scale/gauge ambiguity between light, material, or geometry. Before running, predict the first observable failure in parameter recovery error, data residual, constraint violations, condition/identifiability indicators, and success over initializations. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a graphics inverse problem from a forward renderer, likelihood/data term, prior/regularizer, and constraints; identify non-identifiability and gauge freedoms on a toy scene. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For image intensity $I=a \cdot l$ from unknown albedo a and light l , show the scale ambiguity from one observation. Add a second known-light observation and solve for a , explicitly distinguishing identifiability from numerical optimization. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Fit simple scene color/light/camera parameters from synthetic target images. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create a synthetic scene with known parameters, solve with unconstrained and constrained parameterizations, and plot recovery error/objective over initializations. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare direct unconstrained parameters vs physically constrained/reparameterized variables and regularizers on synthetic ground-truth recovery. Use the same observations, parameterization, initialization distribution, optimizer budget and render/gradient samples; report multiple initializations/seeds when the inverse problem is nonconvex or stochastic. Record parameter recovery error, data residual, constraint violations, condition/identifiability indicators, and success over initializations. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Construct two or more scene parameterizations that explain nearly the same images; use Hessian/Jacobian structure or perturbation experiments to expose ambiguity. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Mitsuba 3 inverse-rendering introduction; optimization foundations from Vol. 1. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Inverse problem formulation. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among direct unconstrained parameters vs physically constrained/reparameterized variables and regularizers on synthetic ground-truth recovery. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using parameter recovery error, data residual, constraint violations, condition/identifiability indicators, and success over initializations. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if direct unconstrained parameters vs physically constrained/reparameterized variables and regularizers on synthetic ground-truth recovery improve gradient correctness or parameter recovery in parameter recovery error, data residual, constraint violations, condition/identifiability indicators, and success over initializations across multiple initializations/scenes at matched optimization/render budget, with ambiguity/non-identifiability reported rather than hidden.

F. Concrete research project - Inverse problem formulation

- F1. Falsifiable question. Under the controlled failure regime "Construct two or more scene parameterizations that explain nearly the same images; use Hessian/Jacobian structure or perturbation experiments to expose ambiguity.", which alternative in direct unconstrained parameters vs physically constrained/reparameterized variables and regularizers on synthetic ground-truth recovery is preferable, and is the change explained by identifiability, gradient

- correctness/variance, visibility non-smoothness, and parameter gauge ambiguities? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create a synthetic scene with known parameters, solve with unconstrained and constrained parameterizations, and plot recovery error/objective over initializations. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use synthetic ground truth. Where derivatives are smooth, run central finite-difference checks over at least 7 step sizes; optimize from at least 10 initializations under three noise/visibility settings and report parameter recovery, not only image loss. The central adversarial condition is: Construct two or more scene parameterizations that explain nearly the same images; use Hessian/Jacobian structure or perturbation experiments to expose ambiguity.
- F4. Required evidence. Measure parameter recovery error, data residual, constraint violations, condition/identifiability indicators, and success over initializations. Deliver differentiable code, gradient-check plots where valid, ground-truth parameters, multi-start recovery CSV, parameter-error/identifiability plots, and one saved visibility/ambiguity failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Inverse problem formulation. If a stable failure/crossover is found among direct unconstrained parameters vs physically constrained/reparameterized variables and regularizers on synthetic ground-truth recovery, implement one adaptive or corrected variant that directly targets identifiability, gradient correctness/variance, visibility non-smoothness, and parameter gauge ambiguities; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Inverse problem formulation. Using the measured parameter recovery error, data residual, constraint violations, condition/identifiability indicators, and success over initializations, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.
- G. Mastery checkpoint - self-contained practical for Inverse problem formulation**
- G1. Closed-book derivation/oracle. Derive a graphics inverse problem from a forward renderer, likelihood/data term, prior/regularizer, and constraints; identify non-identifiability and gauge freedoms on a toy scene. Then solve this exact micro-case from scratch without opening your implementation: For image intensity $I=a \cdot l$ from unknown albedo a and light l , show the scale ambiguity from one observation. Add a second known-light observation and solve for a , explicitly distinguishing identifiability from numerical optimization. You pass this part only if the algebra/geometry and the stored oracle agree.
 - G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Fit simple scene color/light/camera parameters from synthetic target images. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
 - G3. Debugging exam. Intentionally compare parameter recovery while leaving an unremoved scale/gauge ambiguity between light, material, or geometry. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
 - G4. Transfer experiment. On an input/configuration not used during development, execute: Create a synthetic scene with known parameters, solve with unconstrained and constrained parameterizations, and plot recovery error/objective over initializations. Then apply this stress without retuning: Construct two or more scene parameterizations that explain nearly the same images; use Hessian/Jacobian structure or perturbation experiments to expose ambiguity. Report parameter recovery error, data residual, constraint violations, condition/identifiability indicators, and success over initializations and explain which invariant should remain stable and which quantity is allowed/expected to change.
 - G5. Method-selection defense. Using your own measured results, choose among direct unconstrained parameters vs physically constrained/reparameterized variables and regularizers on synthetic ground-truth recovery for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured parameter recovery error, data residual,

constraint violations, condition/identifiability indicators, and success over initializations; state one counterexample that would reverse your recommendation.

- G6. Pass condition for Inverse problem formulation. Using parameter recovery error, data residual, constraint violations, condition/identifiability indicators, and success over initializations, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 13, Unit 2. Automatic differentiation for rendering

Recommended time. 2.0-2.5 weeks (about 30-50 focused hours depending on background).

What you are learning. Forward/reverse mode, computation graphs, JVP/VJP, memory tradeoffs.

A. Reading order

1. Read the primary assignment first: Dr.Jit quickstart and Mitsuba developer docs.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Mitsuba/Dr.Jit beginner tutorials.
3. Keep the most useful executable/reference page open while coding: Mitsuba 3 inverse-rendering tutorials - https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering_tutorials.html

B. Definitions, results and algorithms to know

- Chain rule at scale
- adjoint/reverse mode
- checkpointing concept
- Precision requirement: distinguish and justify analytic, finite-difference, PyTorch/JAX autodiff, and Dr.Jit/Mitsuba gradients on smooth test scenes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive forward- and reverse-mode AD on a small renderer computation graph, compute JVPs/VJPs by hand, and compare memory/compute scaling with parameter/output dimension.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Gradient-check smooth and visibility-changing parameters while increasing path depth/resolution; track gradient error, memory, and runtime. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Differentiate a small analytic renderer in PyTorch/JAX and compare gradients to Mitsuba/finite differences. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For toy renderer $y=(\sin x)^2/(1+x^2)$, compute dy/dx analytically at $x=0.5$, by reverse-mode computational graph, and by central differences. Compare numerical errors as h changes. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Gradient-check smooth and visibility-changing parameters while increasing path depth/resolution; track gradient error, memory, and runtime. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build a reusable gradient checker over camera/material/geometry parameters; plot relative gradient error against finite-difference step and parameter scale.
- P5 - Machine-readable evidence. Export relative gradient error, finite-difference convergence window, runtime, peak memory, and parameter-scale sensitivity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: analytic, finite-difference, PyTorch/JAX autodiff, and Dr.Jit/Mitsuba gradients on smooth test scenes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Gradient-check smooth and visibility-changing parameters while increasing path depth/resolution; track gradient error, memory, and runtime." without hand-editing code between methods. Record relative gradient error, finite-difference convergence window, runtime, peak memory, and parameter-scale sensitivity.

- P7 - Deliberate bug and regression drill. Intentionally accidentally detach/stop-gradient through one scene parameter and trust the optimizer loss alone. Before running, predict the first observable failure in relative gradient error, finite-difference convergence window, runtime, peak memory, and parameter-scale sensitivity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive forward- and reverse-mode AD on a small renderer computation graph, compute JVPs/VJPs by hand, and compare memory/compute scaling with parameter/output dimension. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For toy renderer $y = (\sin x)^2 / (1 + x^2)$, compute dy/dx analytically at $x=0.5$, by reverse-mode computational graph, and by central differences. Compare numerical errors as h changes. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Differentiate a small analytic renderer in PyTorch/JAX and compare gradients to Mitsuba/finite differences. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build a reusable gradient checker over camera/material/geometry parameters; plot relative gradient error against finite-difference step and parameter scale. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare analytic, finite-difference, PyTorch/JAX autodiff, and Dr.Jit/Mitsuba gradients on smooth test scenes. Use the same observations, parameterization, initialization distribution, optimizer budget and render/gradient samples; report multiple initializations/seeds when the inverse problem is nonconvex or stochastic. Record relative gradient error, finite-difference convergence window, runtime, peak memory, and parameter-scale sensitivity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Gradient-check smooth and visibility-changing parameters while increasing path depth/resolution; track gradient error, memory, and runtime. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Dr.Jit quickstart and Mitsuba developer docs. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Automatic differentiation for rendering. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among analytic, finite-difference, PyTorch/JAX autodiff, and Dr.Jit/Mitsuba gradients on smooth test scenes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using relative gradient error, finite-difference convergence window, runtime, peak memory, and parameter-scale sensitivity. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if analytic, finite-difference, PyTorch/JAX autodiff, and Dr.Jit/Mitsuba gradients on smooth test scenes improve gradient correctness or parameter recovery in relative gradient error, finite-difference convergence window, runtime, peak memory, and parameter-scale sensitivity across multiple initializations/scenes at matched optimization/render budget, with ambiguity/non-identifiability reported rather than hidden.

F. Concrete research project - Automatic differentiation for rendering

- F1. Falsifiable question. Under the controlled failure regime "Gradient-check smooth and visibility-changing parameters while increasing path depth/resolution; track gradient error, memory, and runtime.", characterize the ranking or regime transition across analytic, finite-difference, PyTorch/JAX autodiff, and Dr.Jit/Mitsuba gradients on smooth test scenes. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by identifiability, gradient correctness/variance, visibility non-smoothness, and parameter gauge ambiguities. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build a reusable gradient checker over camera/material/geometry parameters; plot relative gradient error against finite-difference step and parameter scale.

Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use synthetic ground truth. Where derivatives are smooth, run central finite-difference checks over at least 7 step sizes; optimize from at least 10 initializations under three noise/visibility settings and report parameter recovery, not only image loss. The central adversarial condition is: Gradient-check smooth and visibility-changing parameters while increasing path depth/resolution; track gradient error, memory, and runtime.

F4. Required evidence. Measure relative gradient error, finite-difference convergence window, runtime, peak memory, and parameter-scale sensitivity. Deliver differentiable code, gradient-check plots where valid, ground-truth parameters, multi-start recovery CSV, parameter-error/identifiability plots, and one saved visibility/ambiguity failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Automatic differentiation for rendering. If a stable failure/crossover is found among analytic, finite-difference, PyTorch/JAX autodiff, and Dr.Jit/Mitsuba gradients on smooth test scenes, implement one adaptive or corrected variant that directly targets identifiability, gradient correctness/variance, visibility non-smoothness, and parameter gauge ambiguities; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Automatic differentiation for rendering. Using the measured relative gradient error, finite-difference convergence window, runtime, peak memory, and parameter-scale sensitivity, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Automatic differentiation for rendering

- G1. Closed-book derivation/oracle. Derive forward- and reverse-mode AD on a small renderer computation graph, compute JVPs/VJPs by hand, and compare memory/compute scaling with parameter/output dimension. Then solve this exact micro-case from scratch without opening your implementation: For toy renderer $y = (\sin x)^2 / (1 + x^2)$, compute dy/dx analytically at $x=0.5$, by reverse-mode computational graph, and by central differences. Compare numerical errors as h changes. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Differentiate a small analytic renderer in PyTorch/JAX and compare gradients to Mitsuba/finite differences. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally accidentally detach/stop-gradient through one scene parameter and trust the optimizer loss alone. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Build a reusable gradient checker over camera/material/geometry parameters; plot relative gradient error against finite-difference step and parameter scale. Then apply this stress without retuning: Gradient-check smooth and visibility-changing parameters while increasing path depth/resolution; track gradient error, memory, and runtime. Report relative gradient error, finite-difference convergence window, runtime, peak memory, and parameter-scale sensitivity and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across analytic, finite-difference, PyTorch/JAX autodiff, and Dr.Jit/Mitsuba gradients on smooth test scenes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured relative gradient error, finite-difference convergence window, runtime, peak memory, and parameter-scale sensitivity; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Automatic differentiation for rendering. Using relative gradient error, finite-difference convergence window, runtime, peak memory, and parameter-scale sensitivity, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 13, Unit 3. Visibility discontinuities

Recommended time. 2.0-2.5 weeks (about 30-50 focused hours depending on background).

What you are learning. Why rasterization/ray tracing derivatives are hard at occlusion boundaries; reparameterization/sampling methods.

A. Reading order

1. Read the primary assignment first: Read Li et al., Differentiable Monte Carlo Ray Tracing through Edge Sampling (SIGGRAPH Asia 2018), Mitsuba 2/3 differentiable-rendering material, and the Mitsuba projective-sampling integrator tutorial.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Mitsuba projective sampling tutorial.
3. Keep the most useful executable/reference page open while coding: Mitsuba 3 inverse-rendering tutorials - https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering_tutorials.html

B. Definitions, results and algorithms to know

- Distributional/discontinuous derivative intuition
- bias/variance issues
- Precision requirement: distinguish and justify naïve pathwise/finite-difference gradients vs smoothing/reparameterization/projective sampling as silhouettes cross pixels. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive why image intensity is piecewise smooth but discontinuous as silhouettes/occlusion boundaries move, and analyze the failure of naïve finite differences/reparameterization at an edge event.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Move a silhouette smoothly across a pixel/edge and sweep sample count; compare gradient bias and variance around the visibility event. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Construct a moving-edge scene; compare naïve finite differences, smooth rasterization, and specialized differentiable sampling. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: A vertical occluder edge at screen coordinate $x(\theta) = \theta$ covers a pixel sample at $x=0$. Compute image value as θ crosses zero and sketch the discontinuous derivative; contrast with a softened coverage model. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Move a silhouette smoothly across a pixel/edge and sweep sample count; compare gradient bias and variance around the visibility event. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Animate a silhouette crossing pixels and plot gradient estimate versus parameter for three estimators; estimate bias/variance/cost around the discontinuity.
- P5 - Machine-readable evidence. Export gradient bias/variance around silhouette events, optimization success, samples/time, and image/parameter recovery error to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: naïve pathwise/finite-difference gradients vs smoothing/reparameterization/projective sampling as silhouettes cross pixels. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Move a silhouette smoothly across a pixel/edge and sweep sample count; compare gradient bias and variance around the visibility event." without hand-editing code between methods. Record gradient bias/variance around silhouette events, optimization success, samples/time, and image/parameter recovery error.
- P7 - Deliberate bug and regression drill. Intentionally use central finite differences across an occlusion event and call the unstable derivative a gradient-check failure. Before running, predict the first observable failure in gradient bias/variance around silhouette events, optimization success, samples/time, and image/parameter recovery error. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive why image intensity is piecewise smooth but discontinuous as silhouettes/occlusion boundaries move, and analyze the failure of naïve finite differences/reparameterization at an edge event. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. A vertical occluder edge at screen coordinate $x(\theta)=\theta$ covers a pixel sample at $x=0$. Compute image value as θ crosses zero and sketch the discontinuous derivative; contrast with a softened coverage model. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Construct a moving-edge scene; compare naïve finite differences, smooth rasterization, and specialized differentiable sampling. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Animate a silhouette crossing pixels and plot gradient estimate versus parameter for three estimators; estimate bias/variance/cost around the discontinuity. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare naïve pathwise/finite-difference gradients vs smoothing/reparameterization/projective sampling as silhouettes cross pixels. Use the same observations, parameterization, initialization distribution, optimizer budget and render/gradient samples; report multiple initializations/seeds when the inverse problem is nonconvex or stochastic. Record gradient bias/variance around silhouette events, optimization success, samples/time, and image/parameter recovery error. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Move a silhouette smoothly across a pixel/edge and sweep sample count; compare gradient bias and variance around the visibility event. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Mitsuba projective sampling integrator tutorial; foundational differentiable rendering papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Visibility discontinuities. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among naïve pathwise/finite-difference gradients vs smoothing/reparameterization/projective sampling as silhouettes cross pixels. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using gradient bias/variance around silhouette events, optimization success, samples/time, and image/parameter recovery error. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the gradient treatment improves parameter recovery or bias/variance near visibility events at matched gradient/render cost while smooth-parameter gradient checks continue to pass.

F. Concrete research project - Visibility discontinuities

- F1. Falsifiable question. Under the controlled failure regime "Move a silhouette smoothly across a pixel/edge and sweep sample count; compare gradient bias and variance around the visibility event.", which alternative in naïve pathwise/finite-difference gradients vs smoothing/reparameterization/projective sampling as silhouettes cross pixels is preferable, and is the change explained by occlusion/silhouette events, gradient measure/discontinuity handling, finite-difference validity, and estimator variance? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Animate a silhouette crossing pixels and plot gradient estimate versus parameter for three estimators; estimate bias/variance/cost around the discontinuity. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use synthetic ground truth. Where derivatives are smooth, run central finite-difference checks over at least 7 step sizes; optimize from at least 10 initializations under three noise/visibility settings and report parameter recovery, not only image loss. The central adversarial condition is: Move a silhouette smoothly across a pixel/edge and sweep sample count; compare gradient bias and variance around the visibility event.
- F4. Required evidence. Measure gradient bias/variance around silhouette events, optimization success, samples/time, and image/parameter recovery error. Deliver differentiable code, gradient-check plots where valid, ground-truth

parameters, multi-start recovery CSV, parameter-error/identifiability plots, and one saved visibility/ambiguity failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Visibility discontinuities. If a stable failure/crossover is found among naïve pathwise/finite-difference gradients vs smoothing/reparameterization/projective sampling as silhouettes cross pixels, implement one adaptive or corrected variant that directly targets occlusion/silhouette events, gradient measure/discontinuity handling, finite-difference validity, and estimator variance; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Visibility discontinuities. Using the measured gradient bias/variance around silhouette events, optimization success, samples/time, and image/parameter recovery error, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Visibility discontinuities

- G1. Closed-book derivation/oracle. Derive why image intensity is piecewise smooth but discontinuous as silhouettes/occlusion boundaries move, and analyze the failure of naïve finite differences/reparameterization at an edge event. Then solve this exact micro-case from scratch without opening your implementation: A vertical occluder edge at screen coordinate $x(\theta) = \theta$ covers a pixel sample at $x=0$. Compute image value as θ crosses zero and sketch the discontinuous derivative; contrast with a softened coverage model. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Construct a moving-edge scene; compare naïve finite differences, smooth rasterization, and specialized differentiable sampling. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use central finite differences across an occlusion event and call the unstable derivative a gradient-check failure. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Animate a silhouette crossing pixels and plot gradient estimate versus parameter for three estimators; estimate bias/variance/cost around the discontinuity. Then apply this stress without retuning: Move a silhouette smoothly across a pixel/edge and sweep sample count; compare gradient bias and variance around the visibility event. Report gradient bias/variance around silhouette events, optimization success, samples/time, and image/parameter recovery error and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among naïve pathwise/finite-difference gradients vs smoothing/reparameterization/projective sampling as silhouettes cross pixels for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured gradient bias/variance around silhouette events, optimization success, samples/time, and image/parameter recovery error; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Visibility discontinuities. Using gradient bias/variance around silhouette events, optimization success, samples/time, and image/parameter recovery error, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 13, Unit 4. Material and lighting estimation

Recommended time. 2.0-2.5 weeks (about 30-50 focused hours depending on background).

What you are learning. Optimize BRDFs, textures, environment illumination, spectral parameters.

A. Reading order

1. Read the primary assignment first: Mitsuba forward inverse rendering tutorials; PBRT material/radiometry chapters as theory.

2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Mitsuba inverse-rendering examples.
3. Keep the most useful executable/reference page open while coding: Mitsuba 3 inverse-rendering tutorials - https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering_tutorials.html

B. Definitions, results and algorithms to know

- Parameter constraints
- ambiguity between lighting and reflectance
- Precision requirement: distinguish and justify joint vs staged recovery of albedo/roughness/light parameters under sparse views and correlated ambiguities. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a simple inverse-rendering factorization for diffuse/specular material and illumination, expose scale/color ambiguities, and formulate regularizers or calibration constraints that resolve them.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use albedo-lighting ambiguity, specular highlights, and exposure changes; test which extra views/priors resolve which ambiguity. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Recover roughness/albedo/light direction from synthetic images; study ambiguity under sparse views. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Render a Lambertian sphere point with observed intensity $I = \rho L \max(n \cdot l, 0)$. Given three known normals and measured intensities, solve a small least-squares estimate for light direction/intensity under fixed ρ . The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use albedo-lighting ambiguity, specular highlights, and exposure changes; test which extra views/priors resolve which ambiguity. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Construct a factorial synthetic benchmark varying views, light directions, roughness, and albedo correlation; report parameter identifiability and covariance/error.
- P5 - Machine-readable evidence. Export albedo/roughness/light parameter error, covariance/ambiguity, novel-view/light image error, and success rate to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: joint vs staged recovery of albedo/roughness/light parameters under sparse views and correlated ambiguities. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use albedo-lighting ambiguity, specular highlights, and exposure changes; test which extra views/priors resolve which ambiguity." without hand-editing code between methods. Record albedo/roughness/light parameter error, covariance/ambiguity, novel-view/light image error, and success rate.
- P7 - Deliberate bug and regression drill. Intentionally allow albedo-light intensity scale ambiguity and judge success only from image reconstruction loss. Before running, predict the first observable failure in albedo/roughness/light parameter error, covariance/ambiguity, novel-view/light image error, and success rate. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a simple inverse-rendering factorization for diffuse/specular material and illumination, expose scale/color ambiguities, and formulate regularizers or calibration constraints that resolve them. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Render a Lambertian sphere point with observed intensity $I = \rho L \max(n \cdot l, 0)$. Given three known normals and measured intensities, solve a small least-squares estimate for light direction/intensity under fixed ρ . Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Recover roughness/albedo/light direction from synthetic images; study ambiguity under sparse views. Before optimizing, make the smallest case agree with Problem 2. Then complete

this deeper task: Construct a factorial synthetic benchmark varying views, light directions, roughness, and albedo correlation; report parameter identifiability and covariance/error. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare joint vs staged recovery of albedo/roughness/light parameters under sparse views and correlated ambiguities. Use the same observations, parameterization, initialization distribution, optimizer budget and render/gradient samples; report multiple initializations/seeds when the inverse problem is nonconvex or stochastic. Record albedo/roughness/light parameter error, covariance/ambiguity, novel-view/light image error, and success rate. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use albedo-lighting ambiguity, specular highlights, and exposure changes; test which extra views/priors resolve which ambiguity. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Mitsuba forward inverse rendering tutorials; PBRT material/radiometry chapters as theory. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Material and lighting estimation. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among joint vs staged recovery of albedo/roughness/light parameters under sparse views and correlated ambiguities. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using albedo/roughness/light parameter error, covariance/ambiguity, novel-view/light image error, and success rate. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if joint vs staged recovery of albedo/roughness/light parameters under sparse views and correlated ambiguities improve gradient correctness or parameter recovery in albedo/roughness/light parameter error, covariance/ambiguity, novel-view/light image error, and success rate across multiple initializations/scenes at matched optimization/render budget, with ambiguity/non-identifiability reported rather than hidden.

F. Concrete research project - Material and lighting estimation

F1. Falsifiable question. Under the controlled failure regime "Use albedo-lighting ambiguity, specular highlights, and exposure changes; test which extra views/priors resolve which ambiguity.", which alternative in joint vs staged recovery of albedo/roughness/light parameters under sparse views and correlated ambiguities is preferable, and is the change explained by identifiability, gradient correctness/variance, visibility non-smoothness, and parameter gauge ambiguities? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Construct a factorial synthetic benchmark varying views, light directions, roughness, and albedo correlation; report parameter identifiability and covariance/error. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use synthetic ground truth. Where derivatives are smooth, run central finite-difference checks over at least 7 step sizes; optimize from at least 10 initializations under three noise/visibility settings and report parameter recovery, not only image loss. The central adversarial condition is: Use albedo-lighting ambiguity, specular highlights, and exposure changes; test which extra views/priors resolve which ambiguity.

F4. Required evidence. Measure albedo/roughness/light parameter error, covariance/ambiguity, novel-view/light image error, and success rate. Deliver differentiable code, gradient-check plots where valid, ground-truth parameters, multi-start recovery CSV, parameter-error/identifiability plots, and one saved visibility/ambiguity failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Material and lighting estimation. If a stable failure/crossover is found among joint vs staged recovery of albedo/roughness/light parameters under sparse views and correlated ambiguities, implement one adaptive or corrected variant that directly targets identifiability, gradient correctness/variance, visibility non-smoothness, and parameter gauge ambiguities; compare it against the strongest fixed baseline under the same protocol. If no stable

boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Material and lighting estimation. Using the measured albedo/roughness/light parameter error, covariance/ambiguity, novel-view/light image error, and success rate, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Material and lighting estimation

- G1. Closed-book derivation/oracle. Derive a simple inverse-rendering factorization for diffuse/specular material and illumination, expose scale/color ambiguities, and formulate regularizers or calibration constraints that resolve them. Then solve this exact micro-case from scratch without opening your implementation: Render a Lambertian sphere point with observed intensity $I = \rho L \max(n \cdot l, 0)$. Given three known normals and measured intensities, solve a small least-squares estimate for light direction/intensity under fixed ρ . You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Recover roughness/albedo/light direction from synthetic images; study ambiguity under sparse views. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally allow albedo-light intensity scale ambiguity and judge success only from image reconstruction loss. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Construct a factorial synthetic benchmark varying views, light directions, roughness, and albedo correlation; report parameter identifiability and covariance/error. Then apply this stress without retuning: Use albedo-lighting ambiguity, specular highlights, and exposure changes; test which extra views/priors resolve which ambiguity. Report albedo/roughness/light parameter error, covariance/ambiguity, novel-view/light image error, and success rate and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among joint vs staged recovery of albedo/roughness/light parameters under sparse views and correlated ambiguities for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured albedo/roughness/light parameter error, covariance/ambiguity, novel-view/light image error, and success rate; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Material and lighting estimation. Using albedo/roughness/light parameter error, covariance/ambiguity, novel-view/light image error, and success rate, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 13, Unit 5. Pose and geometry optimization

Recommended time. 2.0-2.5 weeks (about 30-50 focused hours depending on background).

What you are learning. Camera/object pose, shape derivatives, mesh/SDF parameterizations.

A. Reading order

1. Read the primary assignment first: Mitsuba object-pose and shape-optimization tutorials.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Mitsuba pose estimation and shape optimization.
3. Keep the most useful executable/reference page open while coding: Mitsuba 3 inverse-rendering tutorials - https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering_tutorials.html

B. Definitions, results and algorithms to know

- SE(3) parameterization
- shape gradient
- regularized mesh optimization

- Precision requirement: distinguish and justify pose-only, geometry-only, and joint optimization across initialization basins and regularization strengths. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive pose/shape residual Jacobians for a synthetic calibrated scene and formulate Gauss-Newton/LM updates; identify basin-of-attraction and symmetry ambiguities.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use symmetry, occlusion, sparse views, and large initial pose error; map recovery success as a function of initialization. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Recover pose; then deform a coarse shape to match multi-view images with regularization. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: A 3D point (1,0,5) is observed at image $x=0.18$ with unit focal length while predicted pose gives $x=0.2$. Derive reprojection residual derivative with respect to a small camera x -translation and take one Gauss-Newton step. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use symmetry, occlusion, sparse views, and large initial pose error; map recovery success as a function of initialization. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Map success basin over pose perturbation and shape regularization; show cases where lower image loss produces worse geometry.
- P5 - Machine-readable evidence. Export pose angular/translation error, geometry Chamfer/normal error, image loss, basin size, and regularization sensitivity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: pose-only, geometry-only, and joint optimization across initialization basins and regularization strengths. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use symmetry, occlusion, sparse views, and large initial pose error; map recovery success as a function of initialization." without hand-editing code between methods. Record pose angular/translation error, geometry Chamfer/normal error, image loss, basin size, and regularization sensitivity.
- P7 - Deliberate bug and regression drill. Intentionally apply SE(3) increments on the wrong side/frame and compensate by deforming geometry. Before running, predict the first observable failure in pose angular/translation error, geometry Chamfer/normal error, image loss, basin size, and regularization sensitivity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive pose/shape residual Jacobians for a synthetic calibrated scene and formulate Gauss-Newton/LM updates; identify basin-of-attraction and symmetry ambiguities. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. A 3D point (1,0,5) is observed at image $x=0.18$ with unit focal length while predicted pose gives $x=0.2$. Derive reprojection residual derivative with respect to a small camera x -translation and take one Gauss-Newton step. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Recover pose; then deform a coarse shape to match multi-view images with regularization. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Map success basin over pose perturbation and shape regularization; show cases where lower image loss produces worse geometry. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare pose-only, geometry-only, and joint optimization across initialization basins and regularization strengths. Use the same observations, parameterization, initialization distribution, optimizer budget and render/gradient samples; report multiple initializations/seeds when the inverse problem is nonconvex or stochastic. Record pose angular/translation error, geometry Chamfer/normal error, image loss, basin size, and

regularization sensitivity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use symmetry, occlusion, sparse views, and large initial pose error; map recovery success as a function of initialization. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Mitsuba object-pose and shape-optimization tutorials. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Pose and geometry optimization. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among pose-only, geometry-only, and joint optimization across initialization basins and regularization strengths. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using pose angular/translation error, geometry Chamfer/normal error, image loss, basin size, and regularization sensitivity. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if pose-only, geometry-only, and joint optimization across initialization basins and regularization strengths improve gradient correctness or parameter recovery in pose angular/translation error, geometry Chamfer/normal error, image loss, basin size, and regularization sensitivity across multiple initializations/scenes at matched optimization/render budget, with ambiguity/non-identifiability reported rather than hidden.

F. Concrete research project - Pose and geometry optimization

F1. Falsifiable question. Under the controlled failure regime "Use symmetry, occlusion, sparse views, and large initial pose error; map recovery success as a function of initialization.", characterize the ranking or regime transition across pose-only, geometry-only, and joint optimization across initialization basins and regularization strengths. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by conditioning, local linearization, and loss/parameter scaling. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Map success basin over pose perturbation and shape regularization; show cases where lower image loss produces worse geometry. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use synthetic ground truth. Where derivatives are smooth, run central finite-difference checks over at least 7 step sizes; optimize from at least 10 initializations under three noise/visibility settings and report parameter recovery, not only image loss. The central adversarial condition is: Use symmetry, occlusion, sparse views, and large initial pose error; map recovery success as a function of initialization.

F4. Required evidence. Measure pose angular/translation error, geometry Chamfer/normal error, image loss, basin size, and regularization sensitivity. Deliver differentiable code, gradient-check plots where valid, ground-truth parameters, multi-start recovery CSV, parameter-error/identifiability plots, and one saved visibility/ambiguity failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Pose and geometry optimization. If a stable failure/crossover is found among pose-only, geometry-only, and joint optimization across initialization basins and regularization strengths, implement one adaptive or corrected variant that directly targets conditioning, local linearization, and loss/parameter scaling; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Pose and geometry optimization. Using the measured pose angular/translation error, geometry Chamfer/normal error, image loss, basin size, and regularization sensitivity, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Pose and geometry optimization

- G1. Closed-book derivation/oracle. Derive pose/shape residual Jacobians for a synthetic calibrated scene and formulate Gauss-Newton/LM updates; identify basin-of-attraction and symmetry ambiguities. Then solve this exact

micro-case from scratch without opening your implementation: A 3D point (1,0,5) is observed at image $x=0.18$ with unit focal length while predicted pose gives $x=0.2$. Derive reprojection residual derivative with respect to a small camera x -translation and take one Gauss-Newton step. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Recover pose; then deform a coarse shape to match multi-view images with regularization. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally apply SE(3) increments on the wrong side/frame and compensate by deforming geometry. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Map success basin over pose perturbation and shape regularization; show cases where lower image loss produces worse geometry. Then apply this stress without retuning: Use symmetry, occlusion, sparse views, and large initial pose error; map recovery success as a function of initialization. Report pose angular/translation error, geometry Chamfer/normal error, image loss, basin size, and regularization sensitivity and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across pose-only, geometry-only, and joint optimization across initialization basins and regularization strengths for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured pose angular/translation error, geometry Chamfer/normal error, image loss, basin size, and regularization sensitivity; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Pose and geometry optimization. Using pose angular/translation error, geometry Chamfer/normal error, image loss, basin size, and regularization sensitivity, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 13, Unit 6. Volumetric inverse rendering

Recommended time. 2.0-2.5 weeks (about 30-50 focused hours depending on background).

What you are learning. Density/emission/scattering optimization and radiance fields.

A. Reading order

1. Read the primary assignment first: Mitsuba volumetric inverse rendering and radiance-field reconstruction tutorials.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Mitsuba NeRF-like tutorial.
3. Keep the most useful executable/reference page open while coding: Mitsuba 3 inverse-rendering tutorials - https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering_tutorials.html

B. Definitions, results and algorithms to know

- Volume rendering integral
- transmittance derivative
- Precision requirement: distinguish and justify density/radiance recovery under varying view count, initialization, transmittance, and regularization. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive gradients of transmittance/emission accumulation with respect to density and color in a discretized volume; analyze saturation/vanishing-gradient regimes.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Jointly vary density, albedo/scattering, emission, and lighting under limited views; identify parameter compensation/confounding. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Recover a simple 3D density field or radiance field and test sensitivity to initialization/views. Keep a minimal reference path even if you later optimize it.

- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a ray through two voxels with densities σ_1, σ_2 and known colors, write discrete transmittance/weights and derive $dC/d\sigma_1$ and $dC/d\sigma_2$. Identify which voxel has vanishing gradient when the first becomes opaque. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Jointly vary density, albedo/scattering, emission, and lighting under limited views; identify parameter compensation/confounding. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Recover a small voxel/density field from known views and plot error versus view count, initialization, regularization, and noise.
- P5 - Machine-readable evidence. Export density/field error, held-out image error, view-count/noise sensitivity, convergence, and memory/time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: density/radiance recovery under varying view count, initialization, transmittance, and regularization. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Jointly vary density, albedo/scattering, emission, and lighting under limited views; identify parameter compensation/confounding." without hand-editing code between methods. Record density/field error, held-out image error, view-count/noise sensitivity, convergence, and memory/time.
- P7 - Deliberate bug and regression drill. Intentionally let density and emitted/color terms trade off freely without diagnosing the ambiguity. Before running, predict the first observable failure in density/field error, held-out image error, view-count/noise sensitivity, convergence, and memory/time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive gradients of transmittance/emission accumulation with respect to density and color in a discretized volume; analyze saturation/vanishing-gradient regimes. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a ray through two voxels with densities σ_1, σ_2 and known colors, write discrete transmittance/weights and derive $dC/d\sigma_1$ and $dC/d\sigma_2$. Identify which voxel has vanishing gradient when the first becomes opaque. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Recover a simple 3D density field or radiance field and test sensitivity to initialization/views. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Recover a small voxel/density field from known views and plot error versus view count, initialization, regularization, and noise. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare density/radiance recovery under varying view count, initialization, transmittance, and regularization. Use the same observations, parameterization, initialization distribution, optimizer budget and render/gradient samples; report multiple initializations/seeds when the inverse problem is nonconvex or stochastic. Record density/field error, held-out image error, view-count/noise sensitivity, convergence, and memory/time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Jointly vary density, albedo/scattering, emission, and lighting under limited views; identify parameter compensation/confounding. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Mitsuba volumetric inverse rendering and radiance-field reconstruction tutorials. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Volumetric inverse rendering. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among density/radiance recovery under varying view count, initialization, transmittance, and regularization. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using density/field error, held-out image error, view-count/noise sensitivity, convergence, and memory/time. Deliverable: a 2-4 page mini-report

with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if density/radiance recovery under varying view count, initialization, transmittance, and regularization improve gradient correctness or parameter recovery in density/field error, held-out image error, view-count/noise sensitivity, convergence, and memory/time across multiple initializations/scenes at matched optimization/render budget, with ambiguity/non-identifiability reported rather than hidden.

F. Concrete research project - Volumetric inverse rendering

F1. Falsifiable question. Under the controlled failure regime "Jointly vary density, albedo/scattering, emission, and lighting under limited views; identify parameter compensation/confounding.", characterize the ranking or regime transition across density/radiance recovery under varying view count, initialization, transmittance, and regularization. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by identifiability, gradient correctness/variance, visibility non-smoothness, and parameter gauge ambiguities. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Recover a small voxel/density field from known views and plot error versus view count, initialization, regularization, and noise. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use synthetic ground truth. Where derivatives are smooth, run central finite-difference checks over at least 7 step sizes; optimize from at least 10 initializations under three noise/visibility settings and report parameter recovery, not only image loss. The central adversarial condition is: Jointly vary density, albedo/scattering, emission, and lighting under limited views; identify parameter compensation/confounding.

F4. Required evidence. Measure density/field error, held-out image error, view-count/noise sensitivity, convergence, and memory/time. Deliver differentiable code, gradient-check plots where valid, ground-truth parameters, multi-start recovery CSV, parameter-error/identifiability plots, and one saved visibility/ambiguity failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Volumetric inverse rendering. If a stable failure/crossover is found among density/radiance recovery under varying view count, initialization, transmittance, and regularization, implement one adaptive or corrected variant that directly targets identifiability, gradient correctness/variance, visibility non-smoothness, and parameter gauge ambiguities; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Volumetric inverse rendering. Using the measured density/field error, held-out image error, view-count/noise sensitivity, convergence, and memory/time, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Volumetric inverse rendering

- G1. Closed-book derivation/oracle. Derive gradients of transmittance/emission accumulation with respect to density and color in a discretized volume; analyze saturation/vanishing-gradient regimes. Then solve this exact micro-case from scratch without opening your implementation: For a ray through two voxels with densities σ_1, σ_2 and known colors, write discrete transmittance/weights and derive $dC/d\sigma_1$ and $dC/d\sigma_2$. Identify which voxel has vanishing gradient when the first becomes opaque. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Recover a simple 3D density field or radiance field and test sensitivity to initialization/views. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally let density and emitted/color terms trade off freely without diagnosing the ambiguity. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Recover a small voxel/density field from known views and plot error versus view count, initialization, regularization, and noise. Then apply this stress without retuning: Jointly vary density, albedo/scattering, emission, and lighting under

limited views; identify parameter compensation/confounding. Report density/field error, held-out image error, view-count/noise sensitivity, convergence, and memory/time and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across density/radiance recovery under varying view count, initialization, transmittance, and regularization for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured density/field error, held-out image error, view-count/noise sensitivity, convergence, and memory/time; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Volumetric inverse rendering. Using density/field error, held-out image error, view-count/noise sensitivity, convergence, and memory/time, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 13, Unit 7. Differentiable physics/graphics coupling

Recommended time. 2.0-2.5 weeks (about 30-50 focused hours depending on background).

What you are learning. Differentiate through simulation and rendering for system identification/design.

A. Reading order

1. Read the primary assignment first: Read the differentiable-simulation/optimization sections of Physics-Based Simulation plus one primary differentiable-physics paper matching your simulator; pair it with Mitsuba only when the objective contains image formation.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Choose one differentiable simulation framework/tutorial.
3. Keep the most useful executable/reference page open while coding: Mitsuba 3 inverse-rendering tutorials - https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering_tutorials.html

B. Definitions, results and algorithms to know

- Adjoint/sensitivity idea
- differentiable integrators
- long-horizon instability
- Precision requirement: distinguish and justify image/trajectory loss through a differentiable simulator vs finite-difference parameter search for mass/stiffness/friction. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive chain-rule sensitivities through a toy simulator followed by a renderer, then compare direct differentiation, adjoint/backpropagation, and finite differences for a terminal image loss.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Include contact or a long simulation horizon; compare autodiff/adjoint gradients with finite differences on a reduced case before scaling up. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Infer a physical parameter (mass/stiffness/friction) from rendered trajectories. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: A spring with unknown k produces equilibrium $x=F/k$ and an image observes x . Derive $d\text{ loss}/dk$ for squared pixel/position error both directly and by implicit differentiation of $kx-F=0$. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Include contact or a long simulation horizon; compare autodiff/adjoint gradients with finite differences on a reduced case before scaling up. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Infer one stiffness/friction/mass parameter from a rendered trajectory; compare autodiff gradient to finite differences and recovery across initial guesses.

- P5 - Machine-readable evidence. Export physical parameter error, gradient error, trajectory/image residual, optimization success, and simulation/backprop cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: image/trajectory loss through a differentiable simulator vs finite-difference parameter search for mass/stiffness/friction. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Include contact or a long simulation horizon; compare autodiff/adjoint gradients with finite differences on a reduced case before scaling up." without hand-editing code between methods. Record physical parameter error, gradient error, trajectory/image residual, optimization success, and simulation/backprop cost.
- P7 - Deliberate bug and regression drill. Intentionally differentiate through a contact/topology event as though the simulator were globally smooth. Before running, predict the first observable failure in physical parameter error, gradient error, trajectory/image residual, optimization success, and simulation/backprop cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive chain-rule sensitivities through a toy simulator followed by a renderer, then compare direct differentiation, adjoint/backpropagation, and finite differences for a terminal image loss. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. A spring with unknown k produces equilibrium $x=F/k$ and an image observes x . Derive $d \text{ loss}/dk$ for squared pixel/position error both directly and by implicit differentiation of $kx-F=0$. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Infer a physical parameter (mass/stiffness/friction) from rendered trajectories. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Infer one stiffness/friction/mass parameter from a rendered trajectory; compare autodiff gradient to finite differences and recovery across initial guesses. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare image/trajectory loss through a differentiable simulator vs finite-difference parameter search for mass/stiffness/friction. Use the same observations, parameterization, initialization distribution, optimizer budget and render/gradient samples; report multiple initializations/seeds when the inverse problem is nonconvex or stochastic. Record physical parameter error, gradient error, trajectory/image residual, optimization success, and simulation/backprop cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Include contact or a long simulation horizon; compare autodiff/adjoint gradients with finite differences on a reduced case before scaling up. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Physics-Based Simulation differentiable topics and recent papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Differentiable physics/graphics coupling. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among image/trajectory loss through a differentiable simulator vs finite-difference parameter search for mass/stiffness/friction. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using physical parameter error, gradient error, trajectory/image residual, optimization success, and simulation/backprop cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if image/trajectory loss through a differentiable simulator vs finite-difference parameter search for mass/stiffness/friction improve gradient correctness or parameter recovery in physical parameter error, gradient error, trajectory/image residual, optimization success, and simulation/backprop cost across multiple initializations/scenes at matched optimization/render budget, with ambiguity/non-identifiability reported rather than hidden.

F. Concrete research project - Differentiable physics/graphics coupling

F1. Falsifiable question. Under the controlled failure regime "Include contact or a long simulation horizon; compare autodiff/adjoint gradients with finite differences on a reduced case before scaling up.", which alternative in image/trajectory loss through a differentiable simulator vs finite-difference parameter search for mass/stiffness/friction is preferable, and is the change explained by identifiability, gradient correctness/variance, visibility non-smoothness, and parameter gauge ambiguities? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Infer one stiffness/friction/mass parameter from a rendered trajectory; compare autodiff gradient to finite differences and recovery across initial guesses. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use synthetic ground truth. Where derivatives are smooth, run central finite-difference checks over at least 7 step sizes; optimize from at least 10 initializations under three noise/visibility settings and report parameter recovery, not only image loss. The central adversarial condition is: Include contact or a long simulation horizon; compare autodiff/adjoint gradients with finite differences on a reduced case before scaling up.

F4. Required evidence. Measure physical parameter error, gradient error, trajectory/image residual, optimization success, and simulation/backprop cost. Deliver differentiable code, gradient-check plots where valid, ground-truth parameters, multi-start recovery CSV, parameter-error/identifiability plots, and one saved visibility/ambiguity failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Differentiable physics/graphics coupling. If a stable failure/crossover is found among image/trajectory loss through a differentiable simulator vs finite-difference parameter search for mass/stiffness/friction, implement one adaptive or corrected variant that directly targets identifiability, gradient correctness/variance, visibility non-smoothness, and parameter gauge ambiguities; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Differentiable physics/graphics coupling. Using the measured physical parameter error, gradient error, trajectory/image residual, optimization success, and simulation/backprop cost, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Differentiable physics/graphics coupling

- G1. Closed-book derivation/oracle. Derive chain-rule sensitivities through a toy simulator followed by a renderer, then compare direct differentiation, adjoint/backpropagation, and finite differences for a terminal image loss. Then solve this exact micro-case from scratch without opening your implementation: A spring with unknown k produces equilibrium $x=F/k$ and an image observes x . Derive $d \text{ loss}/dk$ for squared pixel/position error both directly and by implicit differentiation of $kx-F=0$. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Infer a physical parameter (mass/stiffness/friction) from rendered trajectories. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally differentiate through a contact/topology event as though the simulator were globally smooth. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Infer one stiffness/friction/mass parameter from a rendered trajectory; compare autodiff gradient to finite differences and recovery across initial guesses. Then apply this stress without retuning: Include contact or a long simulation horizon; compare autodiff/adjoint gradients with finite differences on a reduced case before scaling up. Report physical parameter error, gradient error, trajectory/image residual, optimization success, and simulation/backprop cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among image/trajectory loss through a differentiable simulator vs finite-difference parameter search for mass/stiffness/friction for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured physical parameter error, gradient error, trajectory/image residual,

optimization success, and simulation/backprop cost; state one counterexample that would reverse your recommendation.

- G6. Pass condition for Differentiable physics/graphics coupling. Using physical parameter error, gradient error, trajectory/image residual, optimization success, and simulation/backprop cost, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 13, Unit 8. Research-grade inverse rendering

Recommended time. 2.0-2.5 weeks (about 30-50 focused hours depending on background).

What you are learning. Uncertainty, multi-start optimization, priors, calibration, evaluation protocol.

A. Reading order

1. Read the primary assignment first: Read Mitsuba 3 inverse-rendering tutorials end to end, then one recent TOG/SIGGRAPH inverse-rendering paper in your selected subproblem. Reproduce its forward model and gradient/identifiability diagnostic before attempting the full optimization.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Reproduce one paper baseline in Mitsuba.
3. Keep the most useful executable/reference page open while coding: Mitsuba 3 inverse-rendering tutorials - https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering_tutorials.html

B. Definitions, results and algorithms to know

- Posterior/uncertainty intuition
- identifiability vs optimization failure
- Precision requirement: distinguish and justify baseline inverse renderer vs one proposed parameterization/prior/gradient treatment on a synthetic benchmark with known truth. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Construct the complete identifiability, calibration, optimization, uncertainty, and evaluation model for one inverse-rendering problem; derive which parameters can be recovered from the observations you actually have.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Evaluate recovery on synthetic ground truth then real data; explicitly compare lower image loss with correctness of recovered physical parameters. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create controlled synthetic benchmark with ground-truth parameters and report error/PSNR/SSIM plus failure cases. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Use a two-parameter synthetic scene (roughness, light intensity) with known ground truth. Draw the 2D loss surface on a coarse grid and identify ambiguity/conditioning before running any gradient optimizer. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Evaluate recovery on synthetic ground truth then real data; explicitly compare lower image loss with correctness of recovered physical parameters. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create a benchmark script that generates ground truth, corrupts observations, runs all baselines/seeds, and exports parameter/image errors plus failure mosaics.
- P5 - Machine-readable evidence. Export ground-truth parameter error, held-out image error, gradient/optimization failure categories, seed sensitivity, and compute to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: baseline inverse renderer vs one proposed parameterization/prior/gradient treatment on a synthetic benchmark with known truth. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Evaluate recovery on synthetic ground truth then real data; explicitly compare lower

image loss with correctness of recovered physical parameters." without hand-editing code between methods.

Record ground-truth parameter error, held-out image error, gradient/optimization failure categories, seed sensitivity, and compute.

- P7 - Deliberate bug and regression drill. Intentionally select hyperparameters on the test views and report only image loss, not recovered physical parameters. Before running, predict the first observable failure in ground-truth parameter error, held-out image error, gradient/optimization failure categories, seed sensitivity, and compute. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Construct the complete identifiability, calibration, optimization, uncertainty, and evaluation model for one inverse-rendering problem; derive which parameters can be recovered from the observations you actually have. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. Use a two-parameter synthetic scene (roughness, light intensity) with known ground truth. Draw the 2D loss surface on a coarse grid and identify ambiguity/conditioning before running any gradient optimizer. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Create controlled synthetic benchmark with ground-truth parameters and report error/PSNR/SSIM plus failure cases. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create a benchmark script that generates ground truth, corrupts observations, runs all baselines/seeds, and exports parameter/image errors plus failure mosaics. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare baseline inverse renderer vs one proposed parameterization/prior/gradient treatment on a synthetic benchmark with known truth. Use the same observations, parameterization, initialization distribution, optimizer budget and render/gradient samples; report multiple initializations/seeds when the inverse problem is nonconvex or stochastic. Record ground-truth parameter error, held-out image error, gradient/optimization failure categories, seed sensitivity, and compute. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Evaluate recovery on synthetic ground truth then real data; explicitly compare lower image loss with correctness of recovered physical parameters. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Recent TOG/SIGGRAPH inverse-rendering papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Research-grade inverse rendering. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among baseline inverse renderer vs one proposed parameterization/prior/gradient treatment on a synthetic benchmark with known truth. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using ground-truth parameter error, held-out image error, gradient/optimization failure categories, seed sensitivity, and compute. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if baseline inverse renderer vs one proposed parameterization/prior/gradient treatment on a synthetic benchmark with known truth improve gradient correctness or parameter recovery in ground-truth parameter error, held-out image error, gradient/optimization failure categories, seed sensitivity, and compute across multiple initializations/scenes at matched optimization/render budget, with ambiguity/non-identifiability reported rather than hidden.

F. Concrete research project - Research-grade inverse rendering

F1. Falsifiable question. Under the controlled failure regime "Evaluate recovery on synthetic ground truth then real data; explicitly compare lower image loss with correctness of recovered physical parameters.", which alternative in baseline inverse renderer vs one proposed parameterization/prior/gradient treatment on a synthetic benchmark with known truth is preferable, and is the change explained by identifiability, gradient correctness/variance, visibility non-smoothness, and parameter gauge ambiguities? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create a benchmark script that generates ground truth, corrupts observations, runs all baselines/seeds, and exports parameter/image errors plus failure mosaics. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use synthetic ground truth. Where derivatives are smooth, run central finite-difference checks over at least 7 step sizes; optimize from at least 10 initializations under three noise/visibility settings and report parameter recovery, not only image loss. The central adversarial condition is: Evaluate recovery on synthetic ground truth then real data; explicitly compare lower image loss with correctness of recovered physical parameters.

F4. Required evidence. Measure ground-truth parameter error, held-out image error, gradient/optimization failure categories, seed sensitivity, and compute. Deliver differentiable code, gradient-check plots where valid, ground-truth parameters, multi-start recovery CSV, parameter-error/identifiability plots, and one saved visibility/ambiguity failure. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Research-grade inverse rendering. If a stable failure/crossover is found among baseline inverse renderer vs one proposed parameterization/prior/gradient treatment on a synthetic benchmark with known truth, implement one adaptive or corrected variant that directly targets identifiability, gradient correctness/variance, visibility non-smoothness, and parameter gauge ambiguities; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Research-grade inverse rendering. Using the measured ground-truth parameter error, held-out image error, gradient/optimization failure categories, seed sensitivity, and compute, Pass when smooth gradients agree with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Research-grade inverse rendering

- G1. Closed-book derivation/oracle. Construct the complete identifiability, calibration, optimization, uncertainty, and evaluation model for one inverse-rendering problem; derive which parameters can be recovered from the observations you actually have. Then solve this exact micro-case from scratch without opening your implementation: Use a two-parameter synthetic scene (roughness, light intensity) with known ground truth. Draw the 2D loss surface on a coarse grid and identify ambiguity/conditioning before running any gradient optimizer. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create controlled synthetic benchmark with ground-truth parameters and report error/PSNR/SSIM plus failure cases. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally select hyperparameters on the test views and report only image loss, not recovered physical parameters. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create a benchmark script that generates ground truth, corrupts observations, runs all baselines/seeds, and exports parameter/image errors plus failure mosaics. Then apply this stress without retuning: Evaluate recovery on synthetic ground truth then real data; explicitly compare lower image loss with correctness of recovered physical parameters. Report ground-truth parameter error, held-out image error, gradient/optimization failure categories, seed sensitivity, and compute and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among baseline inverse renderer vs one proposed parameterization/prior/gradient treatment on a synthetic benchmark with known truth for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured ground-truth parameter error, held-out image error, gradient/optimization failure categories, seed sensitivity, and compute; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Research-grade inverse rendering. Using ground-truth parameter error, held-out image error, gradient/optimization failure categories, seed sensitivity, and compute, Pass when smooth gradients agree

with finite differences away from discontinuities, multiple-start parameter recovery is reported under noise/visibility perturbations, and image loss is not used as a substitute for identifiability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 13 Closeout: Project, Review and Mastery Gate

Volume project. Create a synthetic inverse-rendering benchmark in Mitsuba with known camera, shape, BRDF and lighting. Recover different parameter subsets and map ambiguity/failure regions.

Cumulative review

- Gradient-check every smooth parameterization used in one inverse-rendering experiment and explicitly mark visibility-discontinuous cases.
- Recover known synthetic material, pose/geometry and one volumetric parameter from multiple initializations/noise levels.
- Compare at least two gradient/estimator treatments at matched render/gradient cost.
- Report parameter error and identifiability/failure maps in addition to image loss.
- Write a synthesis separating derivative bias/variance, optimization conditioning, calibration/data mismatch and representation ambiguity.

Mastery gate before moving on

- Gradient-check both smooth and visibility-changing scene parameters and explain bias-variance-memory tradeoffs.
- Recover known synthetic parameters and distinguish low image loss from correct physical recovery.
- Map at least one ambiguity/identifiability failure over views, priors, and initialization.
- Literature gate: for OpenDR/edge-sampling/Mitsuba-style methods, classify every derivative used as smooth/pathwise, visibility/discontinuity-related or approximate; reproduce one gradient check and one inverse recovery experiment.

Volume 14. Neural Rendering and Neural Scene Representations

Purpose. Understand radiance fields, Gaussian splatting, implicit neural representations, dynamic scenes, editing, and relighting as graphics representations.

Suggested duration. 18-22 weeks

Prerequisites. Volumes 1, 5-6 and basic deep learning; Volume 13 is strongly recommended.

Typical module pace. 2.0-2.4 weeks per module (9 modules total).

Primary references for this volume

- Original NeRF, Instant-NGP, mip-NeRF and 3DGS papers.
- Nerfstudio Getting Started, Nerfology and developer guides.
- Surface/relighting/dynamic/editing papers selected by module.

Online spine

Nerfstudio documentation: <https://docs.nerf.studio/>

3D Gaussian Splatting official project: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>

NeRF / related project page by Ben Mildenhall: <https://bmild.github.io/>

Canonical paper track

- Mildenhall et al. (2020), NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis.
- Tancik et al. (2020), Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains.
- Müller et al. (2022), Instant Neural Graphics Primitives with a Multiresolution Hash Encoding.
- Barron et al. (2021/2022), mip-NeRF and mip-NeRF 360.
- Kerbl et al. (2023), 3D Gaussian Splatting for Real-Time Radiance Field Rendering.

- Representative dynamic, relighting, editing and compression papers from current SIGGRAPH/CVPR literature.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Tancik et al. (2020), Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains.
- Mildenhall et al. (2020), NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis.
- Barron et al. (2021), Mip-NeRF: A Multiscale Representation for Anti-Aliasing Neural Radiance Fields.
- Müller et al. (2022), Instant Neural Graphics Primitives with a Multiresolution Hash Encoding.
- Kerbl et al. (2023), 3D Gaussian Splatting for Real-Time Radiance Field Rendering.
- Yariv et al. (2021), Volume Rendering of Neural Implicit Surfaces (VolSDF) - contrast radiance fields with surface-oriented representations.
- Nerfstudio/Nerfology - use as the executable framework for controlled, matched-budget experiments.

Volume 14, Unit 1. Implicit neural representations

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Coordinate MLPs, positional/Fourier features, SIREN-style ideas, inductive bias.

A. Reading order

1. Read the primary assignment first: Fourier Features paper; Nerfstudio Nerfology background.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Nerfstudio educational notebooks.
3. Keep the most useful executable/reference page open while coding: Nerfstudio documentation and Nerfology - <https://docs.nerf.studio/>

B. Definitions, results and algorithms to know

- Spectral bias
- Fourier feature mapping
- continuous field representation
- Precision requirement: distinguish and justify plain MLP vs Fourier features vs multiresolution/hash encoding on images/SDFs with increasing frequency content. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the coordinate-MLP training objective and positional/Fourier encoding map; analyze spectral bias on a toy 1D signal and connect frequency content to network/encoding choices.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Fit low/high-frequency and discontinuous signals with different encodings/activations; inspect extrapolation and spectral bias under matched parameter counts. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Fit 2D images/SDFs with coordinate networks and compare encodings. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Fit $f(x)=\sin(\pi x)$ at $x=\{-1,-0.5,0,0.5,1\}$ with a tiny positional encoding basis $[\sin(\pi x), \cos(\pi x)]$ and solve coefficients analytically; compare with an unencoded linear model. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Fit low/high-frequency and discontinuous signals with different encodings/activations; inspect extrapolation and spectral bias under matched parameter counts. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Train identical MLP capacity with three encodings on signals of increasing frequency; report convergence, spectral bias, memory, and gradient smoothness.

- P5 - Machine-readable evidence. Export fit error, convergence time, frequency-wise error/spectral bias, parameter/memory count, and gradient smoothness to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: plain MLP vs Fourier features vs multiresolution/hash encoding on images/SDFs with increasing frequency content. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Fit low/high-frequency and discontinuous signals with different encodings/activations; inspect extrapolation and spectral bias under matched parameter counts." without hand-editing code between methods. Record fit error, convergence time, frequency-wise error/spectral bias, parameter/memory count, and gradient smoothness.
- P7 - Deliberate bug and regression drill. Intentionally change coordinate normalization/encoding frequency while claiming only the network architecture changed. Before running, predict the first observable failure in fit error, convergence time, frequency-wise error/spectral bias, parameter/memory count, and gradient smoothness. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the coordinate-MLP training objective and positional/Fourier encoding map; analyze spectral bias on a toy 1D signal and connect frequency content to network/encoding choices. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Fit $f(x)=\sin(\pi x)$ at $x=\{-1,-0.5,0,0.5,1\}$ with a tiny positional encoding basis $[\sin(\pi x), \cos(\pi x)]$ and solve coefficients analytically; compare with an unencoded linear model. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Fit 2D images/SDFs with coordinate networks and compare encodings. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Train identical MLP capacity with three encodings on signals of increasing frequency; report convergence, spectral bias, memory, and gradient smoothness. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare plain MLP vs Fourier features vs multiresolution/hash encoding on images/SDFs with increasing frequency content. Use identical dataset split, camera calibration, image resolution, training steps/compute and representation budget; evaluate multiple scenes/seeds and report geometry/consistency metrics in addition to view synthesis. Record fit error, convergence time, frequency-wise error/spectral bias, parameter/memory count, and gradient smoothness. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Fit low/high-frequency and discontinuous signals with different encodings/activations; inspect extrapolation and spectral bias under matched parameter counts. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Fourier Features paper; Nerfstudio Nerfology background. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Implicit neural representations. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among plain MLP vs Fourier features vs multiresolution/hash encoding on images/SDFs with increasing frequency content. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using fit error, convergence time, frequency-wise error/spectral bias, parameter/memory count, and gradient smoothness. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if plain MLP vs Fourier features vs multiresolution/hash encoding on images/SDFs with increasing frequency content improve the held-out quality-geometry/consistency-memory-time tradeoff in fit error, convergence time, frequency-wise error/spectral bias, parameter/memory count, and gradient smoothness on multiple scenes/seeds at matched data and compute; separate calibration/data errors from representation errors.

F. Concrete research project - Implicit neural representations

- F1. Falsifiable question. Under the controlled failure regime "Fit low/high-frequency and discontinuous signals with different encodings/activations; inspect extrapolation and spectral bias under matched parameter counts.", which alternative in plain MLP vs Fourier features vs multiresolution/hash encoding on images/SDFs with increasing frequency content is preferable, and is the change explained by representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Train identical MLP capacity with three encodings on signals of increasing frequency; report convergence, spectral bias, memory, and gradient smoothness. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use at least three scenes and three seeds where compute permits. Hold training steps/rays/compute fixed; perturb one factor at a time over at least four levels and keep a held-out view/geometry/deployment test. The central adversarial condition is: Fit low/high-frequency and discontinuous signals with different encodings/activations; inspect extrapolation and spectral bias under matched parameter counts.
- F4. Required evidence. Measure fit error, convergence time, frequency-wise error/spectral bias, parameter/memory count, and gradient smoothness. Deliver fixed training/evaluation configs, seed/scene results, held-out renders plus geometry/temporal/deployment diagnostics, an ablation CSV, and a failure montage. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Implicit neural representations. If a stable failure/crossover is found among plain MLP vs Fourier features vs multiresolution/hash encoding on images/SDFs with increasing frequency content, implement one adaptive or corrected variant that directly targets representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Implicit neural representations. Using the measured fit error, convergence time, frequency-wise error/spectral bias, parameter/memory count, and gradient smoothness, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Implicit neural representations

- G1. Closed-book derivation/oracle. Derive the coordinate-MLP training objective and positional/Fourier encoding map; analyze spectral bias on a toy 1D signal and connect frequency content to network/encoding choices. Then solve this exact micro-case from scratch without opening your implementation: Fit $f(x)=\sin(\pi x)$ at $x=\{-1,-0.5,0,0.5,1\}$ with a tiny positional encoding basis $[\sin(\pi x), \cos(\pi x)]$ and solve coefficients analytically; compare with an unencoded linear model. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Fit 2D images/SDFs with coordinate networks and compare encodings. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally change coordinate normalization/encoding frequency while claiming only the network architecture changed. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Train identical MLP capacity with three encodings on signals of increasing frequency; report convergence, spectral bias, memory, and gradient smoothness. Then apply this stress without retuning: Fit low/high-frequency and discontinuous signals with different encodings/activations; inspect extrapolation and spectral bias under matched parameter counts. Report fit error, convergence time, frequency-wise error/spectral bias, parameter/memory count, and gradient smoothness and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among plain MLP vs Fourier features vs multiresolution/hash encoding on images/SDFs with increasing frequency content for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured fit error, convergence time, frequency-wise error/spectral bias,

parameter/memory count, and gradient smoothness; state one counterexample that would reverse your recommendation.

- G6. Pass condition for Implicit neural representations. Using fit error, convergence time, frequency-wise error/spectral bias, parameter/memory count, and gradient smoothness, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 14, Unit 2. NeRF fundamentals

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Density/color field, volume rendering, camera rays, hierarchical sampling.

A. Reading order

1. Read the primary assignment first: Original NeRF project/paper; Nerfstudio OG NeRF guide.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Nerfstudio training-your-first-model + Nerfology.
3. Keep the most useful executable/reference page open while coding: Nerfstudio documentation and Nerfology - <https://docs.nerf.studio/>

B. Definitions, results and algorithms to know

- NeRF volume rendering weights
- transmittance
- positional encoding
- Precision requirement: distinguish and justify clean calibrated NeRF vs controlled pose, exposure, sparse-view, and specular perturbations on the same scene. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive NeRF alpha compositing from the continuous volume-rendering equation and transmittance, then derive the discrete weights and hierarchical/stratified sampling logic.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Perturb camera poses/exposures, reduce view count, and add specular/translucent regions one factor at a time; attribute each failure to a violated NeRF assumption. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Train small NeRF on synthetic Blender scene; inspect ray samples, density, view dependence. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a ray with two intervals $\Delta=1$, densities $[0.2, 1.0]$, and colors $[1, 0]$, compute alpha values, transmittances, rendering weights, and final scalar color. Increase the first density to 5 and inspect gradient/visibility effects. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Perturb camera poses/exposures, reduce view count, and add specular/translucent regions one factor at a time; attribute each failure to a violated NeRF assumption. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create controlled variants of one dataset with pose noise, exposure offsets, removed views, and specular material; keep architecture/compute fixed and attribute artifacts.
- P5 - Machine-readable evidence. Export held-out PSNR/SSIM/LPIPS, pose/exposure residual, geometry/depth error if available, training time, and failure attribution to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: clean calibrated NeRF vs controlled pose, exposure, sparse-view, and specular perturbations on the same scene. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Perturb camera poses/exposures, reduce view count, and add specular/translucent regions one factor at a time; attribute each failure to a violated NeRF assumption." without hand-editing code between methods. Record held-out PSNR/SSIM/LPIPS, pose/exposure residual, geometry/depth error if available, training time, and failure attribution.

- P7 - Deliberate bug and regression drill. Intentionally mix camera convention or ray scale, then compensate with near/far bounds and attribute residual artifacts to representation capacity. Before running, predict the first observable failure in held-out PSNR/SSIM/LPIPS, pose/exposure residual, geometry/depth error if available, training time, and failure attribution. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive NeRF alpha compositing from the continuous volume-rendering equation and transmittance, then derive the discrete weights and hierarchical/stratified sampling logic. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a ray with two intervals $\delta=1$, densities $[0.2, 1.0]$, and colors $[1, 0]$, compute alpha values, transmittances, rendering weights, and final scalar color. Increase the first density to 5 and inspect gradient/visibility effects. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Train small NeRF on synthetic Blender scene; inspect ray samples, density, view dependence. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create controlled variants of one dataset with pose noise, exposure offsets, removed views, and specular material; keep architecture/compute fixed and attribute artifacts. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare clean calibrated NeRF vs controlled pose, exposure, sparse-view, and specular perturbations on the same scene. Use identical dataset split, camera calibration, image resolution, training steps/compute and representation budget; evaluate multiple scenes/seeds and report geometry/consistency metrics in addition to view synthesis. Record held-out PSNR/SSIM/LPIPS, pose/exposure residual, geometry/depth error if available, training time, and failure attribution. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Perturb camera poses/exposures, reduce view count, and add specular/translucent regions one factor at a time; attribute each failure to a violated NeRF assumption. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Original NeRF project/paper; Nerfstudio OG NeRF guide. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on NeRF fundamentals. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among clean calibrated NeRF vs controlled pose, exposure, sparse-view, and specular perturbations on the same scene. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using held-out PSNR/SSIM/LPIPS, pose/exposure residual, geometry/depth error if available, training time, and failure attribution. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if controlled perturbations let you attribute failures separately to pose, exposure, view sparsity, reflectance or representation capacity; do not claim a representation gain when calibration/data explains it.

F. Concrete research project - NeRF fundamentals

- F1. Falsifiable question. Under the controlled failure regime "Perturb camera poses/exposures, reduce view count, and add specular/translucent regions one factor at a time; attribute each failure to a violated NeRF assumption.", which alternative in clean calibrated NeRF vs controlled pose, exposure, sparse-view, and specular perturbations on the same scene is preferable, and is the change explained by camera/ray calibration, view coverage, frequency representation, volume-rendering ambiguity, and optimization versus representation error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create controlled variants of one dataset with pose noise, exposure offsets, removed views, and specular material; keep architecture/compute fixed and attribute artifacts. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

- F3. Fixed experiment protocol. Use at least three scenes and three seeds where compute permits. Hold training steps/rays/compute fixed; perturb one factor at a time over at least four levels and keep a held-out view/geometry/deployment test. The central adversarial condition is: Perturb camera poses/exposures, reduce view count, and add specular/translucent regions one factor at a time; attribute each failure to a violated NeRF assumption.
- F4. Required evidence. Measure held-out PSNR/SSIM/LPIPS, pose/exposure residual, geometry/depth error if available, training time, and failure attribution. Deliver fixed training/evaluation configs, seed/scene results, held-out renders plus geometry/temporal/deployment diagnostics, an ablation CSV, and a failure montage. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for NeRF fundamentals. If a stable failure/crossover is found among clean calibrated NeRF vs controlled pose, exposure, sparse-view, and specular perturbation on the same scene, implement one adaptive or corrected variant that directly targets camera/ray calibration, view coverage, frequency representation, volume-rendering ambiguity, and optimization versus representation error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for NeRF fundamentals. Using the measured held-out PSNR/SSIM/LPIPS, pose/exposure residual, geometry/depth error if available, training time, and failure attribution, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for NeRF fundamentals

- G1. Closed-book derivation/oracle. Derive NeRF alpha compositing from the continuous volume-rendering equation and transmittance, then derive the discrete weights and hierarchical/stratified sampling logic. Then solve this exact micro-case from scratch without opening your implementation: For a ray with two intervals $\Delta=1$, densities $[0.2, 1.0]$, and colors $[1, 0]$, compute alpha values, transmittances, rendering weights, and final scalar color. Increase the first density to 5 and inspect gradient/visibility effects. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Train small NeRF on synthetic Blender scene; inspect ray samples, density, view dependence. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally mix camera convention or ray scale, then compensate with near/far bounds and attribute residual artifacts to representation capacity. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create controlled variants of one dataset with pose noise, exposure offsets, removed views, and specular material; keep architecture/compute fixed and attribute artifacts. Then apply this stress without retuning: Perturb camera poses/exposures, reduce view count, and add specular/translucent regions one factor at a time; attribute each failure to a violated NeRF assumption. Report held-out PSNR/SSIM/LPIPS, pose/exposure residual, geometry/depth error if available, training time, and failure attribution and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among clean calibrated NeRF vs controlled pose, exposure, sparse-view, and specular perturbation on the same scene for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured held-out PSNR/SSIM/LPIPS, pose/exposure residual, geometry/depth error if available, training time, and failure attribution; state one counterexample that would reverse your recommendation.
- G6. Pass condition for NeRF fundamentals. Using held-out PSNR/SSIM/LPIPS, pose/exposure residual, geometry/depth error if available, training time, and failure attribution, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 14, Unit 3. Acceleration and multiscale representations

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Hash grids, occupancy structures, mip-NeRF, anti-aliasing, unbounded scenes.

A. Reading order

1. Read the primary assignment first: Instant-NGP paper/code; mip-NeRF/mip-NeRF 360; Nerfstudio method docs.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Nerfstudio method walkthroughs.
3. Keep the most useful executable/reference page open while coding: Nerfstudio documentation and Nerfology - <https://docs.nerf.studio/>

B. Definitions, results and algorithms to know

- Integrated positional encoding intuition
- multiresolution hash encoding
- Precision requirement: distinguish and justify vanilla NeRF vs hash/grid and mip-aware variants under matched quality or training-time budgets. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Build a complexity model comparing plain coordinate MLPs with hash grids, tensor/factorized fields, or multiresolution features; derive how resolution, memory, and interpolation affect approximation.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use thin geometry plus large empty space; sweep encoding capacity/resolution under fixed memory and report collision/aliasing effects. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Compare one MLP NeRF and one accelerated method on training time, render speed, quality and memory. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a 1D two-level hash/grid encoding with cell sizes 1 and 0.5, compute interpolated features at $x=0.25$ and $x=0.75$. Introduce one hash collision and identify which samples become coupled. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use thin geometry plus large empty space; sweep encoding capacity/resolution under fixed memory and report collision/aliasing effects. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Benchmark vanilla NeRF, hash/grid, and mip-aware method at matched PSNR or training time; include render FPS, memory, and aliasing on scale changes.
- P5 - Machine-readable evidence. Export quality versus training time, render FPS, memory, antialiasing/scale error, and model size to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: vanilla NeRF vs hash/grid and mip-aware variants under matched quality or training-time budgets. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use thin geometry plus large empty space; sweep encoding capacity/resolution under fixed memory and report collision/aliasing effects." without hand-editing code between methods. Record quality versus training time, render FPS, memory, antialiasing/scale error, and model size.
- P7 - Deliberate bug and regression drill. Intentionally change occupancy/hash/grid resolution and training compute at the same time, invalidating the speed-quality comparison. Before running, predict the first observable failure in quality versus training time, render FPS, memory, antialiasing/scale error, and model size. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Build a complexity model comparing plain coordinate MLPs with hash grids, tensor/factorized fields, or multiresolution features; derive how resolution, memory, and interpolation affect approximation. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. For a 1D two-level hash/grid encoding with cell sizes 1 and 0.5, compute interpolated features at $x=0.25$ and $x=0.75$. Introduce one hash collision and identify which samples become coupled. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Compare one MLP NeRF and one accelerated method on training time, render speed, quality and memory. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Benchmark vanilla NeRF, hash/grid, and mip-aware method at matched PSNR or training time; include render FPS, memory, and aliasing on scale changes. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare vanilla NeRF vs hash/grid and mip-aware variants under matched quality or training-time budgets. Use identical dataset split, camera calibration, image resolution, training steps/compute and representation budget; evaluate multiple scenes/seeds and report geometry/consistency metrics in addition to view synthesis. Record quality versus training time, render FPS, memory, antialiasing/scale error, and model size. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use thin geometry plus large empty space; sweep encoding capacity/resolution under fixed memory and report collision/aliasing effects. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Instant-NGP paper/code; mip-NeRF/mip-NeRF 360; Nerfstudio method docs. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Acceleration and multiscale representations. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among vanilla NeRF vs hash/grid and mip-aware variants under matched quality or training-time budgets. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using quality versus training time, render FPS, memory, antialiasing/scale error, and model size. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if vanilla NeRF vs hash/grid and mip-aware variants under matched quality or training-time budgets improve the held-out quality-geometry/consistency-memory-time tradeoff in quality versus training time, render FPS, memory, antialiasing/scale error, and model size on multiple scenes/seeds at matched data and compute; separate calibration/data errors from representation errors.

F. Concrete research project - Acceleration and multiscale representations

- F1. Falsifiable question. Under the controlled failure regime "Use thin geometry plus large empty space; sweep encoding capacity/resolution under fixed memory and report collision/aliasing effects.", which alternative in vanilla NeRF vs hash/grid and mip-aware variants under matched quality or training-time budgets is preferable, and is the change explained by representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Benchmark vanilla NeRF, hash/grid, and mip-aware method at matched PSNR or training time; include render FPS, memory, and aliasing on scale changes. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use at least three scenes and three seeds where compute permits. Hold training steps/rays/compute fixed; perturb one factor at a time over at least four levels and keep a held-out view/geometry/deployment test. The central adversarial condition is: Use thin geometry plus large empty space; sweep encoding capacity/resolution under fixed memory and report collision/aliasing effects.
- F4. Required evidence. Measure quality versus training time, render FPS, memory, antialiasing/scale error, and model size. Deliver fixed training/evaluation configs, seed/scene results, held-out renders plus geometry/temporal/deployment diagnostics, an ablation CSV, and a failure montage. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Acceleration and multiscale representations. If a stable failure/crossover is found among vanilla NeRF vs hash/grid and mip-aware variants under matched quality or training-time budgets, implement one

adaptive or corrected variant that directly targets representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Acceleration and multiscale representations. Using the measured quality versus training time, render FPS, memory, antialiasing/scale error, and model size, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Acceleration and multiscale representations

- G1. Closed-book derivation/oracle. Build a complexity model comparing plain coordinate MLPs with hash grids, tensor/factorized fields, or multiresolution features; derive how resolution, memory, and interpolation affect approximation. Then solve this exact micro-case from scratch without opening your implementation: For a 1D two-level hash/grid encoding with cell sizes 1 and 0.5, compute interpolated features at $x=0.25$ and $x=0.75$. Introduce one hash collision and identify which samples become coupled. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Compare one MLP NeRF and one accelerated method on training time, render speed, quality and memory. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally change occupancy/hash/grid resolution and training compute at the same time, invalidating the speed-quality comparison. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Benchmark vanilla NeRF, hash/grid, and mip-aware method at matched PSNR or training time; include render FPS, memory, and aliasing on scale changes. Then apply this stress without retuning: Use thin geometry plus large empty space; sweep encoding capacity/resolution under fixed memory and report collision/aliasing effects. Report quality versus training time, render FPS, memory, antialiasing/scale error, and model size and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among vanilla NeRF vs hash/grid and mip-aware variants under matched quality or training-time budgets for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured quality versus training time, render FPS, memory, antialiasing/scale error, and model size; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Acceleration and multiscale representations. Using quality versus training time, render FPS, memory, antialiasing/scale error, and model size, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 14, Unit 4. 3D Gaussian Splatting

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Anisotropic Gaussians, projection/splatting, visibility-aware compositing, densification.

A. Reading order

1. Read the primary assignment first: Kerbl et al. 3D Gaussian Splatting paper/project; Nerfstudio Splatfacto docs.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Official GraphDeco code/project page and Nerfstudio Gaussian workflow.
3. Keep the most useful executable/reference page open while coding: Nerfstudio documentation and Nerfology - <https://docs.nerf.studio/>

B. Definitions, results and algorithms to know

- 3D covariance parameterization

- projected ellipse
- alpha compositing
- density control
- Precision requirement: distinguish and justify NeRF vs 3DGS plus densification/pruning/anisotropy ablations on matched scenes and capture quality. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive projection of an anisotropic 3D Gaussian to screen-space covariance and alpha compositing of splats; connect scale/opacity/densification to representation capacity.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use thin structures, distant backgrounds, specular regions, and extrapolated camera paths; sweep densification/pruning and count Gaussians/memory. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Train 3DGS on same scene as NeRF; compare quality/speed/memory and inspect Gaussian distributions. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Project a 3D Gaussian with diagonal covariance $\text{diag}(0.01, 0.04, 0.09)$ centered on the optical axis. Linearize perspective projection, compute the 2D covariance $J \Sigma J^T$, and identify how depth changes footprint size. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use thin structures, distant backgrounds, specular regions, and extrapolated camera paths; sweep densification/pruning and count Gaussians/memory. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Log Gaussian count, size/anisotropy, opacity, pruning/densification events; relate them to quality, geometry error, memory, and extrapolation failures.
- P5 - Machine-readable evidence. Export held-out image quality, geometry/depth error, Gaussian count/memory, FPS, and extrapolation/floaters failure rate to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: NeRF vs 3DGS plus densification/pruning/anisotropy ablations on matched scenes and capture quality. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use thin structures, distant backgrounds, specular regions, and extrapolated camera paths; sweep densification/pruning and count Gaussians/memory." without hand-editing code between methods. Record held-out image quality, geometry/depth error, Gaussian count/memory, FPS, and extrapolation/floaters failure rate.
- P7 - Deliberate bug and regression drill. Intentionally project anisotropic covariance incorrectly or compose alpha in the wrong depth/order convention. Before running, predict the first observable failure in held-out image quality, geometry/depth error, Gaussian count/memory, FPS, and extrapolation/floaters failure rate. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive projection of an anisotropic 3D Gaussian to screen-space covariance and alpha compositing of splats; connect scale/opacity/densification to representation capacity. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Project a 3D Gaussian with diagonal covariance $\text{diag}(0.01, 0.04, 0.09)$ centered on the optical axis. Linearize perspective projection, compute the 2D covariance $J \Sigma J^T$, and identify how depth changes footprint size. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Train 3DGS on same scene as NeRF; compare quality/speed/memory and inspect Gaussian distributions. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Log Gaussian count, size/anisotropy, opacity, pruning/densification events; relate them to quality, geometry error, memory, and extrapolation failures. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare NeRF vs 3DGS plus densification/pruning/anisotropy ablations on matched scenes and capture quality. Use identical dataset split, camera calibration, image resolution, training steps/compute and representation budget; evaluate multiple scenes/seeds and report geometry/consistency metrics in addition to view synthesis. Record held-out image quality, geometry/depth error, Gaussian count/memory, FPS, and extrapolation/floater failure rate. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use thin structures, distant backgrounds, specular regions, and extrapolated camera paths; sweep densification/pruning and count Gaussians/memory. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Kerbl et al. 3D Gaussian Splatting paper/project; Nerfstudio Splatfacto docs. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on 3D Gaussian Splatting. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among NeRF vs 3DGS plus densification/pruning/anisotropy ablations on matched scenes and capture quality. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using held-out image quality, geometry/depth error, Gaussian count/memory, FPS, and extrapolation/floater failure rate. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if the ablation moves the quality-geometry-memory-speed Pareto frontier on several scenes and survives matched initialization and training budget.

F. Concrete research project - 3D Gaussian Splatting

- F1. Falsifiable question. Under the controlled failure regime "Use thin structures, distant backgrounds, specular regions, and extrapolated camera paths; sweep densification/pruning and count Gaussians/memory.", which alternative in NeRF vs 3DGS plus densification/pruning/anisotropy ablations on matched scenes and capture quality is preferable, and is the change explained by projection/covariance parameterization, opacity compositing, densification/pruning allocation, representation size, and view-dependent geometry error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Log Gaussian count, size/anisotropy, opacity, pruning/densification events; relate them to quality, geometry error, memory, and extrapolation failures. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use at least three scenes and three seeds where compute permits. Hold training steps/rays/compute fixed; perturb one factor at a time over at least four levels and keep a held-out view/geometry/deployment test. The central adversarial condition is: Use thin structures, distant backgrounds, specular regions, and extrapolated camera paths; sweep densification/pruning and count Gaussians/memory.
- F4. Required evidence. Measure held-out image quality, geometry/depth error, Gaussian count/memory, FPS, and extrapolation/floater failure rate. Deliver fixed training/evaluation configs, seed/scene results, held-out renders plus geometry/temporal/deployment diagnostics, an ablation CSV, and a failure montage. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for 3D Gaussian Splatting. If a stable failure/crossover is found among NeRF vs 3DGS plus densification/pruning/anisotropy ablations on matched scenes and capture quality, implement one adaptive or corrected variant that directly targets projection/covariance parameterization, opacity compositing, densification/pruning allocation, representation size, and view-dependent geometry error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for 3D Gaussian Splatting. Using the measured held-out image quality, geometry/depth error, Gaussian count/memory, FPS, and extrapolation/floater failure rate, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for 3D Gaussian Splatting

- G1. Closed-book derivation/oracle. Derive projection of an anisotropic 3D Gaussian to screen-space covariance and alpha compositing of splats; connect scale/opacity/densification to representation capacity. Then solve this exact micro-case from scratch without opening your implementation: Project a 3D Gaussian with diagonal covariance $\text{diag}(0.01, 0.04, 0.09)$ centered on the optical axis. Linearize perspective projection, compute the 2D covariance $J \Sigma J^T$, and identify how depth changes footprint size. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Train 3DGS on same scene as NeRF; compare quality/speed/memory and inspect Gaussian distributions. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally project anisotropic covariance incorrectly or compose alpha in the wrong depth/order convention. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Log Gaussian count, size/anisotropy, opacity, pruning/densification events; relate them to quality, geometry error, memory, and extrapolation failures. Then apply this stress without retuning: Use thin structures, distant backgrounds, specular regions, and extrapolated camera paths; sweep densification/pruning and count Gaussians/memory. Report held-out image quality, geometry/depth error, Gaussian count/memory, FPS, and extrapolation/floaters failure rate and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among NeRF vs 3DGS plus densification/pruning/anisotropy ablations on matched scenes and capture quality for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured held-out image quality, geometry/depth error, Gaussian count/memory, FPS, and extrapolation/floaters failure rate; state one counterexample that would reverse your recommendation.
- G6. Pass condition for 3D Gaussian Splatting. Using held-out image quality, geometry/depth error, Gaussian count/memory, FPS, and extrapolation/floaters failure rate, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 14, Unit 5. Geometry extraction and surface representations

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. SDF/occupancy fields, mesh extraction, hybrid surface-radiance representations.

A. Reading order

1. Read the primary assignment first: Read Wang et al., NeuS (NeurIPS 2021), and Yariv et al., Volume Rendering of Neural Implicit Surfaces / VolSDF (NeurIPS 2021), then compare against the current Nerfstudio/SDFStudio implementation path.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Nerfstudio geometry export workflow.
3. Keep the most useful executable/reference page open while coding: Nerfstudio documentation and Nerfology - <https://docs.nerf.studio/>

B. Definitions, results and algorithms to know

- Eikonal regularization
- marching cubes
- surface/radiance factorization
- Precision requirement: distinguish and justify density threshold/marching cubes vs SDF-based methods, comparing view synthesis to actual surface accuracy. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive an isosurface/mesh extraction objective from a learned density/SDF field and analyze the effect of threshold, gradient quality, and regularization on topology and normals.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Compare extracted geometry with known ground truth, especially thin/occluded regions; contrast image

metrics with geometry metrics. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Extract mesh from neural field; compare geometric metrics and view-synthesis metrics. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For scalar values at the eight corners of one cube with exactly four above iso-value 0, identify a marching-cubes-style edge intersection pattern and compare with the zero set of an analytic plane SDF through the cube. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Compare extracted geometry with known ground truth, especially thin/occluded regions; contrast image metrics with geometry metrics. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Extract meshes at multiple thresholds/resolutions and compare Chamfer/normal/F-score (synthetic truth) alongside held-out view PSNR.
- P5 - Machine-readable evidence. Export Chamfer/F-score/normal error, topology/mesh quality, held-out view quality, and extraction time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: density threshold/marching cubes vs SDF-based methods, comparing view synthesis to actual surface accuracy. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Compare extracted geometry with known ground truth, especially thin/occluded regions; contrast image metrics with geometry metrics." without hand-editing code between methods. Record Chamfer/F-score/normal error, topology/mesh quality, held-out view quality, and extraction time.
- P7 - Deliberate bug and regression drill. Intentionally choose an isovalue/threshold on the evaluation set and interpret threshold sensitivity as geometry improvement. Before running, predict the first observable failure in Chamfer/F-score/normal error, topology/mesh quality, held-out view quality, and extraction time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive an isosurface/mesh extraction objective from a learned density/SDF field and analyze the effect of threshold, gradient quality, and regularization on topology and normals. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For scalar values at the eight corners of one cube with exactly four above iso-value 0, identify a marching-cubes-style edge intersection pattern and compare with the zero set of an analytic plane SDF through the cube. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Extract mesh from neural field; compare geometric metrics and view-synthesis metrics. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Extract meshes at multiple thresholds/resolutions and compare Chamfer/normal/F-score (synthetic truth) alongside held-out view PSNR. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare density threshold/marching cubes vs SDF-based methods, comparing view synthesis to actual surface accuracy. Use identical dataset split, camera calibration, image resolution, training steps/compute and representation budget; evaluate multiple scenes/seeds and report geometry/consistency metrics in addition to view synthesis. Record Chamfer/F-score/normal error, topology/mesh quality, held-out view quality, and extraction time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Compare extracted geometry with known ground truth, especially thin/occluded regions; contrast image metrics with geometry metrics. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: NeuS/VolSDF-style papers plus Nerfstudio SDFStudio references. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Geometry extraction and surface representations. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among density threshold/marching cubes vs SDF-based methods, comparing view synthesis to actual surface accuracy. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using Chamfer/F-score/normal error, topology/mesh quality, held-out view quality, and extraction time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if density threshold/marching cubes vs SDF-based methods, comparing view synthesis to actual surface accuracy improve the held-out quality-geometry/consistency-memory-time tradeoff in Chamfer/F-score/normal error, topology/mesh quality, held-out view quality, and extraction time on multiple scenes/seeds at matched data and compute; separate calibration/data errors from representation errors.

F. Concrete research project - Geometry extraction and surface representations

F1. Falsifiable question. Under the controlled failure regime "Compare extracted geometry with known ground truth, especially thin/occluded regions; contrast image metrics with geometry metrics.", which alternative in density threshold/marching cubes vs SDF-based methods, comparing view synthesis to actual surface accuracy is preferable, and is the change explained by representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Extract meshes at multiple thresholds/resolutions and compare Chamfer/normal/F-score (synthetic truth) alongside held-out view PSNR. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three scenes and three seeds where compute permits. Hold training steps/rays/compute fixed; perturb one factor at a time over at least four levels and keep a held-out view/geometry/deployment test. The central adversarial condition is: Compare extracted geometry with known ground truth, especially thin/occluded regions; contrast image metrics with geometry metrics.

F4. Required evidence. Measure Chamfer/F-score/normal error, topology/mesh quality, held-out view quality, and extraction time. Deliver fixed training/evaluation configs, seed/scene results, held-out renders plus geometry/temporal/deployment diagnostics, an ablation CSV, and a failure montage. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Geometry extraction and surface representations. If a stable failure/crossover is found among density threshold/marching cubes vs SDF-based methods, comparing view synthesis to actual surface accuracy, implement one adaptive or corrected variant that directly targets representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Geometry extraction and surface representations. Using the measured Chamfer/F-score/normal error, topology/mesh quality, held-out view quality, and extraction time, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Geometry extraction and surface representations

- G1. Closed-book derivation/oracle. Derive an isosurface/mesh extraction objective from a learned density/SDF field and analyze the effect of threshold, gradient quality, and regularization on topology and normals. Then solve this exact micro-case from scratch without opening your implementation: For scalar values at the eight corners of one cube with exactly four above iso-value 0, identify a marching-cubes-style edge intersection pattern and compare with the zero set of an analytic plane SDF through the cube. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Extract mesh from neural field; compare geometric metrics and view-synthesis metrics. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally choose an isovalue/threshold on the evaluation set and interpret threshold sensitivity as geometry improvement. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Extract meshes at multiple thresholds/resolutions and compare Chamfer/normal/F-score (synthetic truth) alongside held-out view PSNR. Then apply this stress without retuning: Compare extracted geometry with known ground truth, especially thin/occluded regions; contrast image metrics with geometry metrics. Report Chamfer/F-score/normal error, topology/mesh quality, held-out view quality, and extraction time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among density threshold/marching cubes vs SDF-based methods, comparing view synthesis to actual surface accuracy for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured Chamfer/F-score/normal error, topology/mesh quality, held-out view quality, and extraction time; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Geometry extraction and surface representations. Using Chamfer/F-score/normal error, topology/mesh quality, held-out view quality, and extraction time, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 14, Unit 6. Relighting and material decomposition

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Normals, BRDFs, illumination factorization, inverse rendering with neural scenes.

A. Reading order

1. Read the primary assignment first: Read Bi et al., Neural Reflectance Fields for Appearance Acquisition (2020), and Zhang et al., NeRFactor (2021), then connect the decomposition assumptions back to PBRT radiometry/BSDF theory.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Implement or run a relighting-capable method.
3. Keep the most useful executable/reference page open while coding: Nerfstudio documentation and Nerfology - <https://docs.nerf.studio/>

B. Definitions, results and algorithms to know

- Intrinsic decomposition ambiguities
- physically based neural rendering
- Precision requirement: distinguish and justify view-synthesis-only representation vs explicit decomposition into illumination/material/geometry on synthetic relighting ground truth. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Formulate scene radiance as geometry, material, and illumination factors; derive a simplified relighting model and enumerate the ambiguities that prevent unique decomposition.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Train/fit under one illumination family and relight far outside it; compare recovered normals/BRDF/lighting to synthetic truth. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Evaluate relighting under novel illumination against known synthetic ground truth. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For two Lambertian pixels with known normals observed under two known directional lights, solve albedos. Then make both light intensities

unknown and expose the albedo-light scale ambiguity. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Train/fit under one illumination family and relight far outside it; compare recovered normals/BRDF/lighting to synthetic truth. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Use synthetic known BRDF/light ground truth; evaluate novel-light renders and parameter recovery separately from training-view reconstruction.
- P5 - Machine-readable evidence. Export novel-light image error, BRDF/light parameter error, view consistency, and disentanglement leakage to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: view-synthesis-only representation vs explicit decomposition into illumination/material/geometry on synthetic relighting ground truth. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Train/fit under one illumination family and relight far outside it; compare recovered normals/BRDF/lighting to synthetic truth." without hand-editing code between methods. Record novel-light image error, BRDF/light parameter error, view consistency, and disentanglement leakage.
- P7 - Deliberate bug and regression drill. Intentionally let view-dependent appearance absorb illumination/material effects and call the decomposition physical. Before running, predict the first observable failure in novel-light image error, BRDF/light parameter error, view consistency, and disentanglement leakage. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Formulate scene radiance as geometry, material, and illumination factors; derive a simplified relighting model and enumerate the ambiguities that prevent unique decomposition. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For two Lambertian pixels with known normals observed under two known directional lights, solve albedos. Then make both light intensities unknown and expose the albedo-light scale ambiguity. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Evaluate relighting under novel illumination against known synthetic ground truth. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Use synthetic known BRDF/light ground truth; evaluate novel-light renders and parameter recovery separately from training-view reconstruction. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare view-synthesis-only representation vs explicit decomposition into illumination/material/geometry on synthetic relighting ground truth. Use identical dataset split, camera calibration, image resolution, training steps/compute and representation budget; evaluate multiple scenes/seeds and report geometry/consistency metrics in addition to view synthesis. Record novel-light image error, BRDF/light parameter error, view consistency, and disentanglement leakage. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Train/fit under one illumination family and relight far outside it; compare recovered normals/BRDF/lighting to synthetic truth. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Neural reflectance field / relightable NeRF papers; PBRT theory cross-reference. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Relighting and material decomposition. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among view-synthesis-only representation vs explicit decomposition into illumination/material/geometry on synthetic relighting ground truth. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using novel-light image error, BRDF/light parameter error, view consistency, and disentanglement

leakage. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if view-synthesis-only representation vs explicit decomposition into illumination/material/geometry on synthetic relighting ground truth improve the held-out quality-geometry/consistency-memory-time tradeoff in novel-light image error, BRDF/light parameter error, view consistency, and disentanglement leakage on multiple scenes/seeds at matched data and compute; separate calibration/data errors from representation errors.

F. Concrete research project - Relighting and material decomposition

- F1. Falsifiable question. Under the controlled failure regime "Train/fit under one illumination family and relight far outside it; compare recovered normals/BRDF/lighting to synthetic truth.", which alternative in view-synthesis-only representation vs explicit decomposition into illumination/material/geometry on synthetic relighting ground truth is preferable, and is the change explained by conditioning, local linearization, and loss/parameter scaling? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Use synthetic known BRDF/light ground truth; evaluate novel-light renders and parameter recovery separately from training-view reconstruction. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use at least three scenes and three seeds where compute permits. Hold training steps/rays/compute fixed; perturb one factor at a time over at least four levels and keep a held-out view/geometry/deployment test. The central adversarial condition is: Train/fit under one illumination family and relight far outside it; compare recovered normals/BRDF/lighting to synthetic truth.
- F4. Required evidence. Measure novel-light image error, BRDF/light parameter error, view consistency, and disentanglement leakage. Deliver fixed training/evaluation configs, seed/scene results, held-out renders plus geometry/temporal/deployment diagnostics, an ablation CSV, and a failure montage. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Relighting and material decomposition. If a stable failure/crossover is found among view-synthesis-only representation vs explicit decomposition into illumination/material/geometry on synthetic relighting ground truth, implement one adaptive or corrected variant that directly targets conditioning, local linearization, and loss/parameter scaling; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Relighting and material decomposition. Using the measured novel-light image error, BRDF/light parameter error, view consistency, and disentanglement leakage, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Relighting and material decomposition

- G1. Closed-book derivation/oracle. Formulate scene radiance as geometry, material, and illumination factors; derive a simplified relighting model and enumerate the ambiguities that prevent unique decomposition. Then solve this exact micro-case from scratch without opening your implementation: For two Lambertian pixels with known normals observed under two known directional lights, solve albedos. Then make both light intensities unknown and expose the albedo-light scale ambiguity. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Evaluate relighting under novel illumination against known synthetic ground truth. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally let view-dependent appearance absorb illumination/material effects and call the decomposition physical. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Use synthetic known BRDF/light ground truth; evaluate novel-light renders and parameter recovery separately from training-view reconstruction. Then apply this stress without retuning: Train/fit under one illumination family and relight far outside it; compare recovered normals/BRDF/lighting to synthetic truth. Report novel-light image error, BRDF/light

parameter error, view consistency, and disentanglement leakage and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method-selection defense. Using your own measured results, choose among view-synthesis-only representation vs explicit decomposition into illumination/material/geometry on synthetic relighting ground truth for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured novel-light image error, BRDF/light parameter error, view consistency, and disentanglement leakage; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Relighting and material decomposition. Using novel-light image error, BRDF/light parameter error, view consistency, and disentanglement leakage, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 14, Unit 7. Dynamic and 4D scenes

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Deformation fields, canonical spaces, time-conditioned representations, dynamic Gaussians.

A. Reading order

1. Read the primary assignment first: Read Pumarola et al., D-NeRF (CVPR 2021), Fridovich-Keil et al., K-Planes (CVPR 2023), and one current dynamic Gaussian-splatting paper/project with accessible code; compare temporal representation and deformation assumptions explicitly.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Reproduce a small dynamic scene.
3. Keep the most useful executable/reference page open while coding: Nerfstudio documentation and Nerfology - <https://docs.nerf.studio/>

B. Definitions, results and algorithms to know

- Canonical-to-observed mapping
- temporal regularization
- topology-change challenge
- Precision requirement: distinguish and justify static-per-frame fit vs temporally coupled 4D representation across fast motion, occlusion, topology change, and camera sparsity. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a canonical-space plus deformation-field representation for a dynamic scene and identify the temporal/cycle constraints needed to avoid arbitrary correspondences.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use nonrigid motion, appearance changes, occlusion, and topology change; inspect temporal correspondence, canonical-space distortion, and drift. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Benchmark temporal consistency and motion artifacts; design failure set with fast motion/occlusion. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: A point moves $x(t)=t$ while a camera observes at $t=0,1$ only. Compare independent per-frame positions with a linear temporal model; predict interpolation at $t=0.5$ and what topology change would violate the model. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use nonrigid motion, appearance changes, occlusion, and topology change; inspect temporal correspondence, canonical-space distortion, and drift. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create a synthetic/controlled sequence with occlusion and topology/motion changes; report temporal flicker, trajectory/flow consistency, and held-out-time/view quality.
- P5 - Machine-readable evidence. Export held-out view/time quality, temporal flicker/flow/trajectory error, topology/occlusion failure rate, FPS, and memory to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.

- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: static-per-frame fit vs temporally coupled 4D representation across fast motion, occlusion, topology change, and camera sparsity. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use nonrigid motion, appearance changes, occlusion, and topology change; inspect temporal correspondence, canonical-space distortion, and drift." without hand-editing code between methods. Record held-out view/time quality, temporal flicker/flow/trajectory error, topology/occlusion failure rate, FPS, and memory.
- P7 - Deliberate bug and regression drill. Intentionally leak frame/time identity through camera/view ordering so apparent temporal reconstruction does not generalize. Before running, predict the first observable failure in held-out view/time quality, temporal flicker/flow/trajectory error, topology/occlusion failure rate, FPS, and memory. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a canonical-space plus deformation-field representation for a dynamic scene and identify the temporal/cycle constraints needed to avoid arbitrary correspondences. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. A point moves $x(t)=t$ while a camera observes at $t=0,1$ only. Compare independent per-frame positions with a linear temporal model; predict interpolation at $t=0.5$ and what topology change would violate the model. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Benchmark temporal consistency and motion artifacts; design failure set with fast motion/occlusion. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create a synthetic/controlled sequence with occlusion and topology/motion changes; report temporal flicker, trajectory/flow consistency, and held-out-time/view quality. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare static-per-frame fit vs temporally coupled 4D representation across fast motion, occlusion, topology change, and camera sparsity. Use identical dataset split, camera calibration, image resolution, training steps/compute and representation budget; evaluate multiple scenes/seeds and report geometry/consistency metrics in addition to view synthesis. Record held-out view/time quality, temporal flicker/flow/trajectory error, topology/occlusion failure rate, FPS, and memory. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use nonrigid motion, appearance changes, occlusion, and topology change; inspect temporal correspondence, canonical-space distortion, and drift. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: K-Planes/dynamic NeRF/4DGS representative papers and Nerfstudio extensions. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Dynamic and 4D scenes. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among static-per-frame fit vs temporally coupled 4D representation across fast motion, occlusion, topology change, and camera sparsity. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using held-out view/time quality, temporal flicker/flow/trajectory error, topology/occlusion failure rate, FPS, and memory. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if static-per-frame fit vs temporally coupled 4D representation across fast motion, occlusion, topology change, and camera sparsity improve the held-out quality-geometry/consistency-memory-time tradeoff in held-out view/time quality, temporal flicker/flow/trajectory error, topology/occlusion failure rate, FPS, and memory on multiple scenes/seeds at matched data and compute; separate calibration/data errors from representation errors.

F. Concrete research project - Dynamic and 4D scenes

- F1. Falsifiable question. Under the controlled failure regime "Use nonrigid motion, appearance changes, occlusion, and topology change; inspect temporal correspondence, canonical-space distortion, and drift.", which alternative in static-

per-frame fit vs temporally coupled 4D representation across fast motion, occlusion, topology change, and camera sparsity is preferable, and is the change explained by representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create a synthetic/controlled sequence with occlusion and topology/motion changes; report temporal flicker, trajectory/flow consistency, and held-out-time/view quality. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three scenes and three seeds where compute permits. Hold training steps/rays/compute fixed; perturb one factor at a time over at least four levels and keep a held-out view/geometry/deployment test. The central adversarial condition is: Use nonrigid motion, appearance changes, occlusion, and topology change; inspect temporal correspondence, canonical-space distortion, and drift.

F4. Required evidence. Measure held-out view/time quality, temporal flicker/flow/trajectory error, topology/occlusion failure rate, FPS, and memory. Deliver fixed training/evaluation configs, seed/scene results, held-out renders plus geometry/temporal/deployment diagnostics, an ablation CSV, and a failure montage. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Dynamic and 4D scenes. If a stable failure/crossover is found among static-per-frame fit vs temporally coupled 4D representation across fast motion, occlusion, topology change, and camera sparsity, implement one adaptive or corrected variant that directly targets representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Dynamic and 4D scenes. Using the measured held-out view/time quality, temporal flicker/flow/trajectory error, topology/occlusion failure rate, FPS, and memory, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Dynamic and 4D scenes

- G1. Closed-book derivation/oracle. Derive a canonical-space plus deformation-field representation for a dynamic scene and identify the temporal/cycle constraints needed to avoid arbitrary correspondences. Then solve this exact micro-case from scratch without opening your implementation: A point moves $x(t)=t$ while a camera observes at $t=0,1$ only. Compare independent per-frame positions with a linear temporal model; predict interpolation at $t=0.5$ and what topology change would violate the model. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Benchmark temporal consistency and motion artifacts; design failure set with fast motion/occlusion. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally leak frame/time identity through camera/view ordering so apparent temporal reconstruction does not generalize. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create a synthetic/controlled sequence with occlusion and topology/motion changes; report temporal flicker, trajectory/flow consistency, and held-out-time/view quality. Then apply this stress without retuning: Use nonrigid motion, appearance changes, occlusion, and topology change; inspect temporal correspondence, canonical-space distortion, and drift. Report held-out view/time quality, temporal flicker/flow/trajectory error, topology/occlusion failure rate, FPS, and memory and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among static-per-frame fit vs temporally coupled 4D representation across fast motion, occlusion, topology change, and camera sparsity for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured held-out view/time quality, temporal flicker/flow/trajectory

error, topology/occlusion failure rate, FPS, and memory; state one counterexample that would reverse your recommendation.

- G6. Pass condition for Dynamic and 4D scenes. Using held-out view/time quality, temporal flicker/flow/trajectory error, topology/occlusion failure rate, FPS, and memory, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 14, Unit 8. Editing and semantics

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Language/feature fields, object selection, scene editing, geometry-aware changes.

A. Reading order

1. Read the primary assignment first: Read Haque et al., Instruct-NeRF2NeRF (ICCV 2023), then one current 3DGS/feature-field editing paper with official code; evaluate edit locality and cross-view consistency separately.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Nerfstudio editing methods documentation.
3. Keep the most useful executable/reference page open while coding: Nerfstudio documentation and Nerfology - <https://docs.nerf.studio/>

B. Definitions, results and algorithms to know

- Feature lifting into 3D
- multi-view consistency
- Precision requirement: distinguish and justify 2D-guided edit vs 3D-consistent semantic edit, evaluated across held-out viewpoints and unedited regions. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Formulate a localized semantic edit as constraints on geometry/appearance/latent features, and derive what regularization is needed to preserve unedited regions and multi-view consistency.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Apply localized semantic edits and quantify collateral geometry/appearance change outside the intended object/region across views. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Perform controlled 3D edit and measure view consistency rather than judging a single hero image. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Take three 3D points observed in two cameras, with one semantic label to change. Write the visibility correspondence table and show how an edit applied independently in each 2D view can disagree in the shared 3D point. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Apply localized semantic edits and quantify collateral geometry/appearance change outside the intended object/region across views. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Define an edit mask/target in 3D, render 20 held-out views, and quantify target edit success plus preservation error outside the edited region.
- P5 - Machine-readable evidence. Export edit-target success, preservation error outside target, multiview consistency, semantic localization, and runtime to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: 2D-guided edit vs 3D-consistent semantic edit, evaluated across held-out viewpoints and unedited regions. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Apply localized semantic edits and quantify collateral geometry/appearance change outside the intended object/region across views." without hand-editing code between methods. Record edit-target success, preservation error outside target, multiview consistency, semantic localization, and runtime.

- P7 - Deliberate bug and regression drill. Intentionally edit only training/view-visible features without testing cross-view/occluded consistency. Before running, predict the first observable failure in edit-target success, preservation error outside target, multiview consistency, semantic localization, and runtime. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Formulate a localized semantic edit as constraints on geometry/appearance/latent features, and derive what regularization is needed to preserve unedited regions and multi-view consistency. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Take three 3D points observed in two cameras, with one semantic label to change. Write the visibility correspondence table and show how an edit applied independently in each 2D view can disagree in the shared 3D point. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Perform controlled 3D edit and measure view consistency rather than judging a single hero image. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Define an edit mask/target in 3D, render 20 held-out views, and quantify target edit success plus preservation error outside the edited region. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare 2D-guided edit vs 3D-consistent semantic edit, evaluated across held-out viewpoints and unedited regions. Use identical dataset split, camera calibration, image resolution, training steps/compute and representation budget; evaluate multiple scenes/seeds and report geometry/consistency metrics in addition to view synthesis. Record edit-target success, preservation error outside target, multiview consistency, semantic localization, and runtime. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Apply localized semantic edits and quantify collateral geometry/appearance change outside the intended object/region across views. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: LERF/Instruct-NeRF2NeRF/Instruct-GS2GS and related Nerfstudio methods. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Editing and semantics. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among 2D-guided edit vs 3D-consistent semantic edit, evaluated across held-out viewpoints and unedited regions. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using edit-target success, preservation error outside target, multiview consistency, semantic localization, and runtime. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if 2D-guided edit vs 3D-consistent semantic edit, evaluated across held-out viewpoints and unedited regions improve the held-out quality-geometry/consistency-memory-time tradeoff in edit-target success, preservation error outside target, multiview consistency, semantic localization, and runtime on multiple scenes/seeds at matched data and compute; separate calibration/data errors from representation errors.

F. Concrete research project - Editing and semantics

- F1. Falsifiable question. Under the controlled failure regime "Apply localized semantic edits and quantify collateral geometry/appearance change outside the intended object/region across views.", which alternative in 2D-guided edit vs 3D-consistent semantic edit, evaluated across held-out viewpoints and unedited regions is preferable, and is the change explained by representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Define an edit mask/target in 3D, render 20 held-out views, and quantify target edit success plus preservation error outside the edited region. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three scenes and three seeds where compute permits. Hold training steps/rays/compute fixed; perturb one factor at a time over at least four levels and keep a held-out view/geometry/deployment test. The central adversarial condition is: Apply localized semantic edits and quantify collateral geometry/appearance change outside the intended object/region across views.

F4. Required evidence. Measure edit-target success, preservation error outside target, multiview consistency, semantic localization, and runtime. Deliver fixed training/evaluation configs, seed/scene results, held-out renders plus geometry/temporal/deployment diagnostics, an ablation CSV, and a failure montage. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Editing and semantics. If a stable failure/crossover is found among 2D-guided edit vs 3D-consistent semantic edit, evaluated across held-out viewpoints and unedited regions, implement one adaptive or corrected variant that directly targets representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Editing and semantics. Using the measured edit-target success, preservation error outside target, multiview consistency, semantic localization, and runtime, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Editing and semantics

- G1. Closed-book derivation/oracle. Formulate a localized semantic edit as constraints on geometry/appearance/latent features, and derive what regularization is needed to preserve unedited regions and multi-view consistency. Then solve this exact micro-case from scratch without opening your implementation: Take three 3D points observed in two cameras, with one semantic label to change. Write the visibility correspondence table and show how an edit applied independently in each 2D view can disagree in the shared 3D point. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Perform controlled 3D edit and measure view consistency rather than judging a single hero image. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally edit only training/view-visible features without testing cross-view/occluded consistency. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Define an edit mask/target in 3D, render 20 held-out views, and quantify target edit success plus preservation error outside the edited region. Then apply this stress without retuning: Apply localized semantic edits and quantify collateral geometry/appearance change outside the intended object/region across views. Report edit-target success, preservation error outside target, multiview consistency, semantic localization, and runtime and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among 2D-guided edit vs 3D-consistent semantic edit, evaluated across held-out viewpoints and unedited regions for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured edit-target success, preservation error outside target, multiview consistency, semantic localization, and runtime; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Editing and semantics. Using edit-target success, preservation error outside target, multiview consistency, semantic localization, and runtime, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 14, Unit 9. Uncertainty, compression, and deployment

Recommended time. 2.0-2.4 weeks (about 30-49 focused hours depending on background).

What you are learning. Scene uncertainty, compact representations, streaming, mobile/VR constraints.

A. Reading order

1. Read the primary assignment first: Read Goli et al., Bayes' Rays: Uncertainty Quantification for Neural Radiance Fields (CVPR 2024) for calibrated post-hoc scene uncertainty; Fan et al., LightGaussian: Unbounded 3D Gaussian Compression with 15x Reduction and 200+ FPS for pruning/distillation/quantization; and the current Nerfstudio Splatfacto/NeRF documentation for an executable baseline. Reproduce one uncertainty-error correlation plot and one quality-memory-latency Pareto curve.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Use a scene benchmark and profile memory/bandwidth.
3. Keep the most useful executable/reference page open while coding: Nerfstudio documentation and Nerfology - <https://docs.nerf.studio/>

B. Definitions, results and algorithms to know

- Rate-distortion
- confidence calibration
- pruning/quantization
- Precision requirement: distinguish and justify uncompressed model vs pruning/quantization/distillation/streaming variants, producing quality-memory-latency Pareto curves. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a rate-distortion-style objective for a neural/splat representation and a simple uncertainty/calibration metric; connect pruning/quantization to quality, memory, and latency.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Hold out viewpoints/regions, compare uncertainty with actual error, then allocate pruning/bitrate using uncertainty and measure quality regret. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Compress/prune a NeRF or 3DGS model and produce quality-size-speed Pareto curves. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Start with ten primitives whose importances are [10,9,8,7,6,5,4,3,2,1]. Prune to five under two importance definitions, compute memory reduction, and reason which held-out view could reveal a wrong importance criterion. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Hold out viewpoints/regions, compare uncertainty with actual error, then allocate pruning/bitrate using uncertainty and measure quality regret. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Generate a Pareto frontier over pruning/quantization/LOD levels with quality, memory, bandwidth, startup time, and FPS on a fixed device.
- P5 - Machine-readable evidence. Export quality, calibration/uncertainty if modeled, memory/bytes, bandwidth/startup latency, FPS, and Pareto dominance to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: uncompressed model vs pruning/quantization/distillation/streaming variants, producing quality-memory-latency Pareto curves. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Hold out viewpoints/regions, compare uncertainty with actual error, then allocate pruning/bitrate using uncertainty and measure quality regret." without hand-editing code between methods. Record quality, calibration/uncertainty if modeled, memory/bytes, bandwidth/startup latency, FPS, and Pareto dominance.
- P7 - Deliberate bug and regression drill. Intentionally rank uncertainty by confidence magnitude without checking calibration against actual held-out error. Before running, predict the first observable failure in quality, calibration/uncertainty if modeled, memory/bytes, bandwidth/startup latency, FPS, and Pareto dominance. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a rate-distortion-style objective for a neural/splat representation and a simple uncertainty/calibration metric; connect pruning/quantization to quality, memory, and latency. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Start with ten primitives whose importances are [10,9,8,7,6,5,4,3,2,1]. Prune to five under two importance definitions, compute memory reduction, and reason which held-out view could reveal a wrong importance criterion. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Compress/prune a NeRF or 3DGS model and produce quality-size-speed Pareto curves. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Generate a Pareto frontier over pruning/quantization/LOD levels with quality, memory, bandwidth, startup time, and FPS on a fixed device. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare uncompressed model vs pruning/quantization/distillation/streaming variants, producing quality-memory-latency Pareto curves. Use identical dataset split, camera calibration, image resolution, training steps/compute and representation budget; evaluate multiple scenes/seeds and report geometry/consistency metrics in addition to view synthesis. Record quality, calibration/uncertainty if modeled, memory/bytes, bandwidth/startup latency, FPS, and Pareto dominance. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Hold out viewpoints/regions, compare uncertainty with actual error, then allocate pruning/bitrate using uncertainty and measure quality regret. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Recent SIGGRAPH/CVPR/ICCV papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Uncertainty, compression, and deployment. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among uncompressed model vs pruning/quantization/distillation/streaming variants, producing quality-memory-latency Pareto curves. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using quality, calibration/uncertainty if modeled, memory/bytes, bandwidth/startup latency, FPS, and Pareto dominance. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if uncompressed model vs pruning/quantization/distillation/streaming variants, producing quality-memory-latency Pareto curves improve the held-out quality-geometry/consistency-memory-time tradeoff in quality, calibration/uncertainty if modeled, memory/bytes, bandwidth/startup latency, FPS, and Pareto dominance on multiple scenes/seeds at matched data and compute; separate calibration/data errors from representation errors.

F. Concrete research project - Uncertainty, compression, and deployment

- F1. Falsifiable question. Under the controlled failure regime "Hold out viewpoints/regions, compare uncertainty with actual error, then allocate pruning/bitrate using uncertainty and measure quality regret.", which alternative in uncompressed model vs pruning/quantization/distillation/streaming variants, producing quality-memory-latency Pareto curves is preferable, and is the change explained by representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Generate a Pareto frontier over pruning/quantization/LOD levels with quality, memory, bandwidth, startup time, and FPS on a fixed device. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use at least three scenes and three seeds where compute permits. Hold training steps/rays/compute fixed; perturb one factor at a time over at least four levels and keep a held-out view/geometry/deployment test. The central adversarial condition is: Hold out viewpoints/regions, compare uncertainty with actual error, then allocate pruning/bitrate using uncertainty and measure quality regret.
- F4. Required evidence. Measure quality, calibration/uncertainty if modeled, memory/bytes, bandwidth/startup latency, FPS, and Pareto dominance. Deliver fixed training/evaluation configs, seed/scene results, held-out renders plus

geometry/temporal/deployment diagnostics, an ablation CSV, and a failure montage. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Uncertainty, compression, and deployment. If a stable failure/crossover is found among uncompressed model vs pruning/quantization/distillation/streaming variants, producing quality-memory-latency Pareto curves, implement one adaptive or corrected variant that directly targets representation bias, pose/data quality, capacity/allocation, and optimization versus calibration error; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Uncertainty, compression, and deployment. Using the measured quality, calibration/uncertainty if modeled, memory/bytes, bandwidth/startup latency, FPS, and Pareto dominance, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Uncertainty, compression, and deployment

- G1. Closed-book derivation/oracle. Derive a rate-distortion-style objective for a neural/splat representation and a simple uncertainty/calibration metric; connect pruning/quantization to quality, memory, and latency. Then solve this exact micro-case from scratch without opening your implementation: Start with ten primitives whose importances are [10,9,8,7,6,5,4,3,2,1]. Prune to five under two importance definitions, compute memory reduction, and reason which held-out view could reveal a wrong importance criterion. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Compress/prune a NeRF or 3DGS model and produce quality-size-speed Pareto curves. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally rank uncertainty by confidence magnitude without checking calibration against actual held-out error. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Generate a Pareto frontier over pruning/quantization/LOD levels with quality, memory, bandwidth, startup time, and FPS on a fixed device. Then apply this stress without retuning: Hold out viewpoints/regions, compare uncertainty with actual error, then allocate pruning/bitrate using uncertainty and measure quality regret. Report quality, calibration/uncertainty if modeled, memory/bytes, bandwidth/startup latency, FPS, and Pareto dominance and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among uncompressed model vs pruning/quantization/distillation/streaming variants, producing quality-memory-latency Pareto curves for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured quality, calibration/uncertainty if modeled, memory/bytes, bandwidth/startup latency, FPS, and Pareto dominance; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Uncertainty, compression, and deployment. Using quality, calibration/uncertainty if modeled, memory/bytes, bandwidth/startup latency, FPS, and Pareto dominance, Pass when conclusions survive matched compute/data and multiple seeds/scenes, held-out quality is reported with representation/geometry/deployment diagnostics, and the effect is larger than ordinary seed/config variability. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 14 Closeout: Project, Review and Mastery Gate

Volume project. Train NeRF and 3DGS on identical scenes, compare quality/speed/memory, inspect geometry and implement one controlled editing, relighting or compression experiment.

Cumulative review

- Train a small controlled NeRF/implicit model and a Gaussian/surface-oriented representation on matched scenes/budgets.
- Perturb poses, views, exposure/appearance and representation capacity one factor at a time.
- Report held-out view quality together with geometry/temporal/memory/FPS metrics appropriate to the claim.
- Create a failure-attribution table distinguishing data/calibration, optimization and representation limitations.

- Write a neural-scene synthesis explaining when rendering quality is an insufficient proxy for scene correctness.

Mastery gate before moving on

- Train/evaluate NeRF and 3DGS under matched data and report image quality, geometry fidelity, speed, memory, and OOD behavior.
- Demonstrate one geometry-extraction, relighting/editing, dynamic-scene, or compression experiment with controlled validity metrics.
- Attribute observed failures among representation capacity, optimization, calibration/data, and violated scene assumptions.
- Literature gate: reconstruct NeRF volume-rendering weights and one multiscale/acceleration or Gaussian-splat derivation; compare the papers using held-out scene correctness, representation cost and failure assumptions rather than headline PSNR alone.

Volume 15. Generative 3D/4D and Machine Learning for Graphics

Purpose. Study generative models as tools for creating geometry, appearance, scenes, motion, and physically plausible visual content.

Suggested duration. 18-24 weeks

Prerequisites. Volumes 7, 9-10, 14 and modern deep-learning basics.

Typical module pace. 2.2-3.0 weeks per module (8 modules total).

Primary references for this volume

- A modern diffusion/generative modeling reference for foundations.
- DreamFusion-family and multiview 3D generation papers.
- Current SIGGRAPH/CVPR/ICCV papers for scene, avatar, materials and physics-aware generation.

Online spine

SIGGRAPH 2026 Technical Papers: <https://s2026.siggraph.org/program/technical-papers/>

Nerfstudio documentation: <https://docs.nerf.studio/>

Canonical paper track

- Ho et al. (2020), Denoising Diffusion Probabilistic Models (foundation).
- Poole et al. (2022), DreamFusion: Text-to-3D using 2D Diffusion.
- Representative multiview diffusion and feed-forward 3D generation papers from 2024-2026.
- Representative motion diffusion/avatar papers and physics-aware generation papers.
- Use current SIGGRAPH/CVPR/ICCV proceedings for the final third of this volume.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Ho, Jain & Abbeel (2020), Denoising Diffusion Probabilistic Models.
- Song et al. (2021), Score-Based Generative Modeling through Stochastic Differential Equations.
- Poole et al. (2022), DreamFusion: Text-to-3D using 2D Diffusion.
- Liu et al. (2023), Zero-1-to-3: Zero-shot One Image to 3D Object.
- Shi et al. (2023), MVDream: Multi-view Diffusion for 3D Generation.
- Tevet et al. (2023), Human Motion Diffusion Model.
- For frontier reading, select 2-3 current papers from the SIGGRAPH 2026 program and evaluate them with geometry/physics/task metrics rather than image quality alone.

Volume 15, Unit 1. Deep generative foundations

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Autoencoders, GANs, autoregressive models, diffusion/score models, transformers at the level needed for graphics.

A. Reading order

1. Read the primary assignment first: Read Ho et al., Denoising Diffusion Probabilistic Models (NeurIPS 2020), and Song et al., Score-Based Generative Modeling through Stochastic Differential Equations (ICLR 2021). Implement a small 1D/2D diffusion or score model before moving to 3D.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Official framework diffusion tutorials.
3. Keep the most useful executable/reference page open while coding: PyTorch stable documentation - <https://docs.pytorch.org/docs/stable/> ; implement the small diffusion/score model yourself before using a large 3D generation repository.

B. Definitions, results and algorithms to know

- ELBO intuition
- adversarial objective
- denoising score matching
- classifier-free guidance concept
- Precision requirement: distinguish and justify VAE/GAN intuition vs diffusion/score-based denoising on a tiny controlled dataset where likelihood/objective behavior can be inspected. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the evidence-lower-bound idea for a VAE and the denoising/noise-prediction objective for diffusion in a toy Gaussian setting; compare what is optimized and sampled.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use a multimodal synthetic/3D distribution with known modes; compare quality and coverage under matched compute rather than reporting one aggregate score. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Train a tiny 2D diffusion or use pretrained model and inspect denoising trajectory; focus on objective rather than scale. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a 1D mixture of Gaussians centered at -2 and 2, add one Gaussian-noise level and compute the posterior score direction qualitatively at $x=-3,0,3$. Relate this to one denoising step. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use a multimodal synthetic/3D distribution with known modes; compare quality and coverage under matched compute rather than reporting one aggregate score. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Train a tiny diffusion model on a 2D mixture/manifold dataset and visualize forward/noising, score/denoising vector fields, and failure under insufficient steps/capacity.
- P5 - Machine-readable evidence. Export denoising/reconstruction score on toy data, mode coverage, sample diversity, training/sample steps, and objective diagnostics to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: VAE/GAN intuition vs diffusion/score-based denoising on a tiny controlled dataset where likelihood/objective behavior can be inspected. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use a multimodal synthetic/3D distribution with known modes; compare quality and coverage under matched compute rather than reporting one aggregate score." without hand-editing code between methods. Record denoising/reconstruction score on toy data, mode coverage, sample diversity, training/sample steps, and objective diagnostics.
- P7 - Deliberate bug and regression drill. Intentionally use a different noise/sampling schedule at evaluation than the one assumed by the trained objective. Before running, predict the first observable failure in denoising/reconstruction score on toy data, mode coverage, sample diversity, training/sample steps, and objective

diagnostics. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the evidence-lower-bound idea for a VAE and the denoising/noise-prediction objective for diffusion in a toy Gaussian setting; compare what is optimized and sampled. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a 1D mixture of Gaussians centered at -2 and 2, add one Gaussian-noise level and compute the posterior score direction qualitatively at $x=-3,0,3$. Relate this to one denoising step. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Train a tiny 2D diffusion or use pretrained model and inspect denoising trajectory; focus on objective rather than scale. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Train a tiny diffusion model on a 2D mixture/manifold dataset and visualize forward/noising, score/denoising vector fields, and failure under insufficient steps/capacity. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare VAE/GAN intuition vs diffusion/score-based denoising on a tiny controlled dataset where likelihood/objective behavior can be inspected. Use the same training data/model scale where relevant, prompts/conditions, number of generated samples and compute budget; report semantic quality together with diversity, validity and physical/geometric metrics. Record denoising/reconstruction score on toy data, mode coverage, sample diversity, training/sample steps, and objective diagnostics. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use a multimodal synthetic/3D distribution with known modes; compare quality and coverage under matched compute rather than reporting one aggregate score. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: A modern deep generative modeling reference; original method papers as needed. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Deep generative foundations. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among VAE/GAN intuition vs diffusion/score-based denoising on a tiny controlled dataset where likelihood/objective behavior can be inspected. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using denoising/reconstruction score on toy data, mode coverage, sample diversity, training/sample steps, and objective diagnostics. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if VAE/GAN intuition vs diffusion/score-based denoising on a tiny controlled dataset where likelihood/objective behavior can be inspected improve a joint semantic-diversity-validity/physics tradeoff in denoising/reconstruction score on toy data, mode coverage, sample diversity, training/sample steps, and objective diagnostics across multiple prompts/seeds at matched compute; reject gains caused by mode collapse or a weak validity checker.

F. Concrete research project - Deep generative foundations

- F1. Falsifiable question. Under the controlled failure regime "Use a multimodal synthetic/3D distribution with known modes; compare quality and coverage under matched compute rather than reporting one aggregate score.", which alternative in VAE/GAN intuition vs diffusion/score-based denoising on a tiny controlled dataset where likelihood/objective behavior can be inspected is preferable, and is the change explained by data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Train a tiny diffusion model on a 2D mixture/manifold dataset and visualize forward/noising, score/denoising vector fields, and failure under insufficient steps/capacity. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a fixed batch of at least 50 generated outputs for a compact study (more for high-variance conclusions); keep prompts/seeds/compute matched, annotate at least 20 failures, and evaluate geometry/constraint/task validity separately from perceptual quality. The central adversarial condition is: Use a multimodal synthetic/3D distribution with known modes; compare quality and coverage under matched compute rather than reporting one aggregate score.

F4. Required evidence. Measure denoising/reconstruction score on toy data, mode coverage, sample diversity, training/sample steps, and objective diagnostics. Deliver the generation batch, method-independent evaluator, annotation/failure table, matched-budget comparison, and at least one example where a common perceptual metric gives the wrong research ranking. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Deep generative foundations. If a stable failure/crossover is found among VAE/GAN intuition vs diffusion/score-based denoising on a tiny controlled dataset where likelihood/objective behavior can be inspected, implement one adaptive or corrected variant that directly targets data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Deep generative foundations. Using the measured denoising/reconstruction score on toy data, mode coverage, sample diversity, training/sample steps, and objective diagnostics, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Deep generative foundations

- G1. Closed-book derivation/oracle. Derive the evidence-lower-bound idea for a VAE and the denoising/noise-prediction objective for diffusion in a toy Gaussian setting; compare what is optimized and sampled. Then solve this exact micro-case from scratch without opening your implementation: For a 1D mixture of Gaussians centered at -2 and 2, add one Gaussian-noise level and compute the posterior score direction qualitatively at $x=-3,0,3$. Relate this to one denoising step. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Train a tiny 2D diffusion or use pretrained model and inspect denoising trajectory; focus on objective rather than scale. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally use a different noise/sampling schedule at evaluation than the one assumed by the trained objective. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Train a tiny diffusion model on a 2D mixture/manifold dataset and visualize forward/noising, score/denoising vector fields, and failure under insufficient steps/capacity. Then apply this stress without retuning: Use a multimodal synthetic/3D distribution with known modes; compare quality and coverage under matched compute rather than reporting one aggregate score. Report denoising/reconstruction score on toy data, mode coverage, sample diversity, training/sample steps, and objective diagnostics and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among VAE/GAN intuition vs diffusion/score-based denoising on a tiny controlled dataset where likelihood/objective behavior can be inspected for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured denoising/reconstruction score on toy data, mode coverage, sample diversity, training/sample steps, and objective diagnostics; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Deep generative foundations. Using denoising/reconstruction score on toy data, mode coverage, sample diversity, training/sample steps, and objective diagnostics, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 15, Unit 2. 3D data representations for learning

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Point clouds, voxels, meshes, implicit fields, triplanes, Gaussians, tokens.

A. Reading order

1. Read the primary assignment first: Use representative primary sources: PointNet (CVPR 2017) for point sets, DeepSDF (CVPR 2019) or Occupancy Networks (CVPR 2019) for implicit fields, MeshCNN (SIGGRAPH 2019) for meshes, and Vol. 14 for radiance fields/splats. Compare preprocessing conventions explicitly.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Implement identical simple task over voxel/point/implicit representation if feasible.
3. Keep the most useful executable/reference page open while coding: PyTorch stable documentation plus libigl/Geometry Central for geometry preprocessing; use the official repository of the selected representation paper for comparison only after your own conversion tests.

B. Definitions, results and algorithms to know

- Permutation invariance
- discretization tradeoffs
- topology and memory constraints
- Precision requirement: distinguish and justify voxels, point clouds, meshes, implicit fields, and splats on thin features, topology, rotation, and memory scaling. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive invariance/equivariance requirements and memory/compute scaling for voxels, point clouds, meshes, implicit fields, and splats; show how representation changes neighborhood operations.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Encode the same shapes with several representations under matched memory; compare reconstruction, topology handling, editability, and downstream learning. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Compare representation biases on reconstruction of thin features and topology. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Represent a thin diagonal line in a 4x4 voxel grid, as four points, and as a two-triangle strip. Count storage and identify which rotations/resolutions make each representation lose the feature. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Encode the same shapes with several representations under matched memory; compare reconstruction, topology handling, editability, and downstream learning. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Encode the same synthetic shapes as voxels/points/mesh/SDF/splats and measure thin-feature error, topology, rotation sensitivity, and memory.
- P5 - Machine-readable evidence. Export reconstruction error, topology/thin-feature score, rotation sensitivity, memory scaling, and operator/neighborhood cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: voxels, point clouds, meshes, implicit fields, and splats on thin features, topology, rotation, and memory scaling. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Encode the same shapes with several representations under matched memory; compare reconstruction, topology handling, editability, and downstream learning." without hand-editing code between methods. Record reconstruction error, topology/thin-feature score, rotation sensitivity, memory scaling, and operator/neighborhood cost.
- P7 - Deliberate bug and regression drill. Intentionally compare representations with inconsistent axis, scale, normalization, or point/mesh sampling conventions. Before running, predict the first observable failure in reconstruction error, topology/thin-feature score, rotation sensitivity, memory scaling, and operator/neighborhood

cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive invariance/equivariance requirements and memory/compute scaling for voxels, point clouds, meshes, implicit fields, and splats; show how representation changes neighborhood operations. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Represent a thin diagonal line in a 4x4 voxel grid, as four points, and as a two-triangle strip. Count storage and identify which rotations/resolutions make each representation lose the feature. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Compare representation biases on reconstruction of thin features and topology. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Encode the same synthetic shapes as voxels/points/mesh/SDF/splats and measure thin-feature error, topology, rotation sensitivity, and memory. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare voxels, point clouds, meshes, implicit fields, and splats on thin features, topology, rotation, and memory scaling. Use the same training data/model scale where relevant, prompts/conditions, number of generated samples and compute budget; report semantic quality together with diversity, validity and physical/geometric metrics. Record reconstruction error, topology/thin-feature score, rotation sensitivity, memory scaling, and operator/neighborhood cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Encode the same shapes with several representations under matched memory; compare reconstruction, topology handling, editability, and downstream learning. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Survey/representative papers; connect Vol. 7/14. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on 3D data representations for learning. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among voxels, point clouds, meshes, implicit fields, and splats on thin features, topology, rotation, and memory scaling. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using reconstruction error, topology/thin-feature score, rotation sensitivity, memory scaling, and operator/neighborhood cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if voxels, point clouds, meshes, implicit fields, and splats on thin features, topology, rotation, and memory scaling improve a joint semantic-diversity-validity/physics tradeoff in reconstruction error, topology/thin-feature score, rotation sensitivity, memory scaling, and operator/neighborhood cost across multiple prompts/seeds at matched compute; reject gains caused by mode collapse or a weak validity checker.

F. Concrete research project - 3D data representations for learning

- F1. Falsifiable question. Under the controlled failure regime "Encode the same shapes with several representations under matched memory; compare reconstruction, topology handling, editability, and downstream learning.", characterize the ranking or regime transition across voxels, point clouds, meshes, implicit fields, and splats on thin features, topology, rotation, and memory scaling. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Encode the same synthetic shapes as voxels/points/mesh/SDF/splats and measure thin-feature error, topology, rotation sensitivity, and memory. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use a fixed batch of at least 50 generated outputs for a compact study (more for high-variance conclusions); keep prompts/seeds/compute matched, annotate at least 20 failures, and evaluate

geometry/constraint/task validity separately from perceptual quality. The central adversarial condition is: Encode the same shapes with several representations under matched memory; compare reconstruction, topology handling, editability, and downstream learning.

F4. Required evidence. Measure reconstruction error, topology/thin-feature score, rotation sensitivity, memory scaling, and operator/neighborhood cost. Deliver the generation batch, method-independent evaluator, annotation/failure table, matched-budget comparison, and at least one example where a common perceptual metric gives the wrong research ranking. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for 3D data representations for learning. If a stable failure/crossover is found among voxels, point clouds, meshes, implicit fields, and splats on thin features, topology, rotation, and memory scaling, implement one adaptive or corrected variant that directly targets data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for 3D data representations for learning. Using the measured reconstruction error, topology/thin-feature score, rotation sensitivity, memory scaling, and operator/neighborhood cost, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for 3D data representations for learning

- G1. Closed-book derivation/oracle. Derive invariance/equivariance requirements and memory/compute scaling for voxels, point clouds, meshes, implicit fields, and splats; show how representation changes neighborhood operations. Then solve this exact micro-case from scratch without opening your implementation: Represent a thin diagonal line in a 4x4 voxel grid, as four points, and as a two-triangle strip. Count storage and identify which rotations/resolutions make each representation lose the feature. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Compare representation biases on reconstruction of thin features and topology. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally compare representations with inconsistent axis, scale, normalization, or point/mesh sampling conventions. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Encode the same synthetic shapes as voxels/points/mesh/SDF/splats and measure thin-feature error, topology, rotation sensitivity, and memory. Then apply this stress without retuning: Encode the same shapes with several representations under matched memory; compare reconstruction, topology handling, editability, and downstream learning. Report reconstruction error, topology/thin-feature score, rotation sensitivity, memory scaling, and operator/neighborhood cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across voxels, point clouds, meshes, implicit fields, and splats on thin features, topology, rotation, and memory scaling for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured reconstruction error, topology/thin-feature score, rotation sensitivity, memory scaling, and operator/neighborhood cost; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for 3D data representations for learning. Using reconstruction error, topology/thin-feature score, rotation sensitivity, memory scaling, and operator/neighborhood cost, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 15, Unit 3. Image/text-to-3D

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Score distillation, multiview diffusion, feed-forward 3D reconstruction/generation.

A. Reading order

1. Read the primary assignment first: Read Poole et al., DreamFusion (ICLR 2023) and Lin et al., Magic3D (CVPR 2023) as score-distillation anchors, then one feed-forward 3D reconstruction/generation paper with official code. Keep text/image conditions and evaluation prompts fixed.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Use an open 3D generation system for controlled tests.
3. Keep the most useful executable/reference page open while coding: Use the official code/project page of the selected primary paper plus PyTorch stable documentation - <https://docs.pytorch.org/docs/stable/> ; keep evaluation scripts independent of the method repository.

B. Definitions, results and algorithms to know

- 3D consistency
- SDS-type optimization intuition
- Janus/multi-view ambiguity
- Precision requirement: distinguish and justify optimization-based score distillation vs feed-forward generation evaluated for prompt adherence, multiview consistency, and geometry. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the optimization signal in a score-distillation-style or multi-view generative 3D pipeline at a conceptual/mathematical level and enumerate Janus/multi-view inconsistency failure mechanisms.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use prompts/images requiring backside consistency, thin topology, and material-light separation; score cross-view contradictions and geometry validity. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Evaluate generated objects with multi-view consistency, geometry quality, texture seams, and prompt adherence. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Use a toy sphere parameterized by radius r and color c rendered from two orthographic views. Given target silhouette radius and mean color, derive a two-parameter loss and solve its optimum before considering a learned prior. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use prompts/images requiring backside consistency, thin topology, and material-light separation; score cross-view contradictions and geometry validity. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build a fixed evaluation harness rendering identical canonical views and computing prompt/image alignment plus geometry/multiview diagnostics for several generated objects.
- P5 - Machine-readable evidence. Export prompt/image alignment, multiview consistency, geometry/normal quality, texture seams, diversity, and generation time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: optimization-based score distillation vs feed-forward generation evaluated for prompt adherence, multiview consistency, and geometry. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use prompts/images requiring backside consistency, thin topology, and material-light separation; score cross-view contradictions and geometry validity." without hand-editing code between methods. Record prompt/image alignment, multiview consistency, geometry/normal quality, texture seams, diversity, and generation time.
- P7 - Deliberate bug and regression drill. Intentionally reuse prompts/images or near-duplicates across tuning and evaluation, inflating apparent generalization. Before running, predict the first observable failure in prompt/image alignment, multiview consistency, geometry/normal quality, texture seams, diversity, and generation time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the optimization signal in a score-distillation-style or multi-view generative 3D pipeline at a conceptual/mathematical level and enumerate Janus/multi-view inconsistency failure mechanisms. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Use a toy sphere parameterized by radius r and color c rendered from two orthographic views. Given target silhouette radius and mean color, derive a two-parameter loss and solve its optimum before considering a learned prior. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Evaluate generated objects with multi-view consistency, geometry quality, texture seams, and prompt adherence. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build a fixed evaluation harness rendering identical canonical views and computing prompt/image alignment plus geometry/multiview diagnostics for several generated objects. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare optimization-based score distillation vs feed-forward generation evaluated for prompt adherence, multiview consistency, and geometry. Use the same training data/model scale where relevant, prompts/conditions, number of generated samples and compute budget; report semantic quality together with diversity, validity and physical/geometric metrics. Record prompt/image alignment, multiview consistency, geometry/normal quality, texture seams, diversity, and generation time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use prompts/images requiring backside consistency, thin topology, and material-light separation; score cross-view contradictions and geometry validity. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: DreamFusion-family and modern feed-forward 3D papers; current SIGGRAPH/CVPR literature. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Image/text-to-3D. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among optimization-based score distillation vs feed-forward generation evaluated for prompt adherence, multiview consistency, and geometry. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using prompt/image alignment, multiview consistency, geometry/normal quality, texture seams, diversity, and generation time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if optimization-based score distillation vs feed-forward generation evaluated for prompt adherence, multiview consistency, and geometry improve a joint semantic-diversity-validity/physics tradeoff in prompt/image alignment, multiview consistency, geometry/normal quality, texture seams, diversity, and generation time across multiple prompts/seeds at matched compute; reject gains caused by mode collapse or a weak validity checker.

F. Concrete research project - Image/text-to-3D

- F1. Falsifiable question. Under the controlled failure regime "Use prompts/images requiring backside consistency, thin topology, and material-light separation; score cross-view contradictions and geometry validity.", which alternative in optimization-based score distillation vs feed-forward generation evaluated for prompt adherence, multiview consistency, and geometry is preferable, and is the change explained by data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build a fixed evaluation harness rendering identical canonical views and computing prompt/image alignment plus geometry/multiview diagnostics for several generated objects. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use a fixed batch of at least 50 generated outputs for a compact study (more for high-variance conclusions); keep prompts/seeds/compute matched, annotate at least 20 failures, and evaluate geometry/constraint/task validity separately from perceptual quality. The central adversarial condition is: Use

prompts/images requiring backside consistency, thin topology, and material-light separation; score cross-view contradictions and geometry validity.

F4. Required evidence. Measure prompt/image alignment, multiview consistency, geometry/normal quality, texture seams, diversity, and generation time. Deliver the generation batch, method-independent evaluator, annotation/failure table, matched-budget comparison, and at least one example where a common perceptual metric gives the wrong research ranking. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Image/text-to-3D. If a stable failure/crossover is found among optimization-based score distillation vs feed-forward generation evaluated for prompt adherence, multiview consistency, and geometry, implement one adaptive or corrected variant that directly targets data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Image/text-to-3D. Using the measured prompt/image alignment, multiview consistency, geometry/normal quality, texture seams, diversity, and generation time, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Image/text-to-3D

- G1. Closed-book derivation/oracle. Derive the optimization signal in a score-distillation-style or multi-view generative 3D pipeline at a conceptual/mathematical level and enumerate Janus/multi-view inconsistency failure mechanisms. Then solve this exact micro-case from scratch without opening your implementation: Use a toy sphere parameterized by radius r and color c rendered from two orthographic views. Given target silhouette radius and mean color, derive a two-parameter loss and solve its optimum before considering a learned prior. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Evaluate generated objects with multi-view consistency, geometry quality, texture seams, and prompt adherence. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally reuse prompts/images or near-duplicates across tuning and evaluation, inflating apparent generalization. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Build a fixed evaluation harness rendering identical canonical views and computing prompt/image alignment plus geometry/multiview diagnostics for several generated objects. Then apply this stress without retuning: Use prompts/images requiring backside consistency, thin topology, and material-light separation; score cross-view contradictions and geometry validity. Report prompt/image alignment, multiview consistency, geometry/normal quality, texture seams, diversity, and generation time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among optimization-based score distillation vs feed-forward generation evaluated for prompt adherence, multiview consistency, and geometry for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured prompt/image alignment, multiview consistency, geometry/normal quality, texture seams, diversity, and generation time; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Image/text-to-3D. Using prompt/image alignment, multiview consistency, geometry/normal quality, texture seams, diversity, and generation time, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 15, Unit 4. Scene generation and layout

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Compositional scenes, object placement, semantics, lighting/material generation.

A. Reading order

1. Read the primary assignment first: Read Paschalidou et al., ATISS (NeurIPS 2021) as a structured indoor-scene baseline, then one current diffusion/generative scene-layout paper with official code; evaluate relation, support, collision, and diversity constraints independently.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Create small procedural+learned hybrid benchmark.
3. Keep the most useful executable/reference page open while coding: PyTorch stable documentation and the official repository of the selected scene-generation paper; implement relation/collision checks in a separate evaluator.

B. Definitions, results and algorithms to know

- Factorized scene representations
- constraints and relational layout
- Precision requirement: distinguish and justify unconstrained scene generation vs explicit relational/spatial constraints with measurable satisfaction rates. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Formulate scene generation as structured variables plus geometric/semantic constraints; derive one energy or probabilistic factorization for object identities, poses, support, and collision relationships.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Test crowded scenes, unusual functional relations, and support/collision constraints; count physical and semantic violations. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Generate multi-object scenes under explicit spatial constraints; measure constraint satisfaction. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Place three axis-aligned boxes A,B,C with constraints A left-of B, C on-top-of B, and no overlap. Write inequalities on centers/extents and construct one satisfying and one minimally violating layout. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Test crowded scenes, unusual functional relations, and support/collision constraints; count physical and semantic violations. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Generate at least 500 scenes with explicit relations (left/right/on/inside/non-overlap); parse outputs and report relation satisfaction and collision rates.
- P5 - Machine-readable evidence. Export relation/constraint satisfaction, collision/penetration rate, semantic correctness, diversity, and generation time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: unconstrained scene generation vs explicit relational/spatial constraints with measurable satisfaction rates. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Test crowded scenes, unusual functional relations, and support/collision constraints; count physical and semantic violations." without hand-editing code between methods. Record relation/constraint satisfaction, collision/penetration rate, semantic correctness, diversity, and generation time.
- P7 - Deliberate bug and regression drill. Intentionally score visual plausibility while ignoring object interpenetration, support, and relation constraints. Before running, predict the first observable failure in relation/constraint satisfaction, collision/penetration rate, semantic correctness, diversity, and generation time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Formulate scene generation as structured variables plus geometric/semantic constraints; derive one energy or probabilistic factorization for object identities, poses, support, and collision relationships. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. Place three axis-aligned boxes A,B,C with constraints A left-of B, C on-top-of B, and no overlap. Write inequalities on centers/extents and construct one satisfying and one minimally violating layout. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Generate multi-object scenes under explicit spatial constraints; measure constraint satisfaction. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Generate at least 500 scenes with explicit relations (left/right/on/inside/non-overlap); parse outputs and report relation satisfaction and collision rates. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare unconstrained scene generation vs explicit relational/spatial constraints with measurable satisfaction rates. Use the same training data/model scale where relevant, prompts/conditions, number of generated samples and compute budget; report semantic quality together with diversity, validity and physical/geometric metrics. Record relation/constraint satisfaction, collision/penetration rate, semantic correctness, diversity, and generation time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Test crowded scenes, unusual functional relations, and support/collision constraints; count physical and semantic violations. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Recent graphics/vision scene-generation papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Scene generation and layout. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among unconstrained scene generation vs explicit relational/spatial constraints with measurable satisfaction rates. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using relation/constraint satisfaction, collision/penetration rate, semantic correctness, diversity, and generation time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if unconstrained scene generation vs explicit relational/spatial constraints with measurable satisfaction rates improve a joint semantic-diversity-validity/physics tradeoff in relation/constraint satisfaction, collision/penetration rate, semantic correctness, diversity, and generation time across multiple prompts/seeds at matched compute; reject gains caused by mode collapse or a weak validity checker.

F. Concrete research project - Scene generation and layout

- F1. Falsifiable question. Under the controlled failure regime "Test crowded scenes, unusual functional relations, and support/collision constraints; count physical and semantic violations.", which alternative in unconstrained scene generation vs explicit relational/spatial constraints with measurable satisfaction rates is preferable, and is the change explained by data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Generate at least 500 scenes with explicit relations (left/right/on/inside/non-overlap); parse outputs and report relation satisfaction and collision rates. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use a fixed batch of at least 50 generated outputs for a compact study (more for high-variance conclusions); keep prompts/seeds/compute matched, annotate at least 20 failures, and evaluate geometry/constraint/task validity separately from perceptual quality. The central adversarial condition is: Test crowded scenes, unusual functional relations, and support/collision constraints; count physical and semantic violations.
- F4. Required evidence. Measure relation/constraint satisfaction, collision/penetration rate, semantic correctness, diversity, and generation time. Deliver the generation batch, method-independent evaluator, annotation/failure table, matched-budget comparison, and at least one example where a common perceptual metric gives the wrong research ranking. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Scene generation and layout. If a stable failure/crossover is found among unconstrained scene generation vs explicit relational/spatial constraints with measurable satisfaction rates, implement one adaptive or corrected variant that directly targets data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Scene generation and layout. Using the measured relation/constraint satisfaction, collision/penetration rate, semantic correctness, diversity, and generation time, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Scene generation and layout

- G1. Closed-book derivation/oracle. Formulate scene generation as structured variables plus geometric/semantic constraints; derive one energy or probabilistic factorization for object identities, poses, support, and collision relationships. Then solve this exact micro-case from scratch without opening your implementation: Place three axis-aligned boxes A,B,C with constraints A left-of B, C on-top-of B, and no overlap. Write inequalities on centers/extents and construct one satisfying and one minimally violating layout. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Generate multi-object scenes under explicit spatial constraints; measure constraint satisfaction. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally score visual plausibility while ignoring object interpenetration, support, and relation constraints. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Generate at least 500 scenes with explicit relations (left/right/on/inside/non-overlap); parse outputs and report relation satisfaction and collision rates. Then apply this stress without retuning: Test crowded scenes, unusual functional relations, and support/collision constraints; count physical and semantic violations. Report relation/constraint satisfaction, collision/penetration rate, semantic correctness, diversity, and generation time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among unconstrained scene generation vs explicit relational/spatial constraints with measurable satisfaction rates for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured relation/constraint satisfaction, collision/penetration rate, semantic correctness, diversity, and generation time; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Scene generation and layout. Using relation/constraint satisfaction, collision/penetration rate, semantic correctness, diversity, and generation time, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 15, Unit 5. Generative materials and appearance

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. SVBRDF/material synthesis, textures, relightable appearance, procedural-neural hybrids.

A. Reading order

1. Read the primary assignment first: Read PBRT 4e Chapters 9-10 for physically based reflection/material models and Guo et al., MaterialGAN: Reflectance Capture using a Generative SVBRDF Model (SIGGRAPH Asia 2020) as a generative material prior. Re-render generated SVBRDFs under at least two held-out environment maps and two held-out view angles; do not evaluate only RGB texture plausibility.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Use measured/material datasets where available.
3. Keep the most useful executable/reference page open while coding: PyTorch stable documentation plus PBRT/Mitsuba for physically controlled material renders; keep the generator and evaluator code paths separate.

B. Definitions, results and algorithms to know

- Stationary texture vs spatial material
- tileability
- reflectance consistency
- Precision requirement: distinguish and justify RGB texture generation vs physically structured material-map generation evaluated under novel light/view conditions. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a generative parameterization for spatially varying material maps and a differentiable rendering/evaluation loop; separate perceptual plausibility from physical parameter validity.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Relight generated materials under unseen HDR illumination and inspect tiling, view dependence, spatial coherence, and parameter plausibility. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Generate material maps and evaluate under novel lighting in a reference renderer. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a 2x2 material with albedo and roughness maps, render qualitatively under frontal and grazing light. Swap roughness values while preserving RGB texture and predict which novel-light observations distinguish the materials. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Relight generated materials under unseen HDR illumination and inspect tiling, view dependence, spatial coherence, and parameter plausibility. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Generate albedo/roughness/normal maps, render them under a fixed light/view grid, and detect baked-in lighting, seam, and energy/plausibility failures.
- P5 - Machine-readable evidence. Export novel-light consistency, seam/baked-lighting artifact rate, parameter plausibility, diversity, and render error to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: RGB texture generation vs physically structured material-map generation evaluated under novel light/view conditions. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Relight generated materials under unseen HDR illumination and inspect tiling, view dependence, spatial coherence, and parameter plausibility." without hand-editing code between methods. Record novel-light consistency, seam/baked-lighting artifact rate, parameter plausibility, diversity, and render error.
- P7 - Deliberate bug and regression drill. Intentionally evaluate generated materials under uncontrolled lighting so lighting quality is mistaken for material quality. Before running, predict the first observable failure in novel-light consistency, seam/baked-lighting artifact rate, parameter plausibility, diversity, and render error. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a generative parameterization for spatially varying material maps and a differentiable rendering/evaluation loop; separate perceptual plausibility from physical parameter validity. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a 2x2 material with albedo and roughness maps, render qualitatively under frontal and grazing light. Swap roughness values while preserving RGB texture and predict which novel-light observations distinguish the materials. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Generate material maps and evaluate under novel lighting in a reference renderer. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Generate albedo/roughness/normal maps, render them under a fixed light/view grid, and detect baked-in lighting, seam, and energy/plausibility failures. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare RGB texture generation vs physically structured material-map generation evaluated under novel light/view conditions. Use the same training data/model scale where relevant, prompts/conditions, number of generated samples and compute budget; report semantic quality together with diversity, validity and physical/geometric metrics. Record novel-light consistency, seam/baked-lighting artifact rate, parameter plausibility, diversity, and render error. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Relight generated materials under unseen HDR illumination and inspect tiling, view dependence, spatial coherence, and parameter plausibility. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Recent material-generation papers + PBRT material theory. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Generative materials and appearance. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among RGB texture generation vs physically structured material-map generation evaluated under novel light/view conditions. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using novel-light consistency, seam/baked-lighting artifact rate, parameter plausibility, diversity, and render error. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if RGB texture generation vs physically structured material-map generation evaluated under novel light/view conditions improve a joint semantic-diversity-validity/physics tradeoff in novel-light consistency, seam/baked-lighting artifact rate, parameter plausibility, diversity, and render error across multiple prompts/seeds at matched compute; reject gains caused by mode collapse or a weak validity checker.

F. Concrete research project - Generative materials and appearance

- F1. Falsifiable question. Under the controlled failure regime "Relight generated materials under unseen HDR illumination and inspect tiling, view dependence, spatial coherence, and parameter plausibility.", which alternative in RGB texture generation vs physically structured material-map generation evaluated under novel light/view conditions is preferable, and is the change explained by data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Generate albedo/roughness/normal maps, render them under a fixed light/view grid, and detect baked-in lighting, seam, and energy/plausibility failures. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use a fixed batch of at least 50 generated outputs for a compact study (more for high-variance conclusions); keep prompts/seeds/compute matched, annotate at least 20 failures, and evaluate geometry/constraint/task validity separately from perceptual quality. The central adversarial condition is: Relight generated materials under unseen HDR illumination and inspect tiling, view dependence, spatial coherence, and parameter plausibility.
- F4. Required evidence. Measure novel-light consistency, seam/baked-lighting artifact rate, parameter plausibility, diversity, and render error. Deliver the generation batch, method-independent evaluator, annotation/failure table, matched-budget comparison, and at least one example where a common perceptual metric gives the wrong research ranking. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Generative materials and appearance. If a stable failure/crossover is found among RGB texture generation vs physically structured material-map generation evaluated under novel light/view conditions, implement one adaptive or corrected variant that directly targets data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Generative materials and appearance. Using the measured novel-light consistency, seam/baked-lighting artifact rate, parameter plausibility, diversity, and render error, Pass when the evaluator is

independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Generative materials and appearance

- G1. Closed-book derivation/oracle. Derive a generative parameterization for spatially varying material maps and a differentiable rendering/evaluation loop; separate perceptual plausibility from physical parameter validity. Then solve this exact micro-case from scratch without opening your implementation: For a 2x2 material with albedo and roughness maps, render qualitatively under frontal and grazing light. Swap roughness values while preserving RGB texture and predict which novel-light observations distinguish the materials. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Generate material maps and evaluate under novel lighting in a reference renderer. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally evaluate generated materials under uncontrolled lighting so lighting quality is mistaken for material quality. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Generate albedo/roughness/normal maps, render them under a fixed light/view grid, and detect baked-in lighting, seam, and energy/plausibility failures. Then apply this stress without retuning: Relight generated materials under unseen HDR illumination and inspect tiling, view dependence, spatial coherence, and parameter plausibility. Report novel-light consistency, seam/baked-lighting artifact rate, parameter plausibility, diversity, and render error and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among RGB texture generation vs physically structured material-map generation evaluated under novel light/view conditions for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured novel-light consistency, seam/baked-lighting artifact rate, parameter plausibility, diversity, and render error; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Generative materials and appearance. Using novel-light consistency, seam/baked-lighting artifact rate, parameter plausibility, diversity, and render error, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 15, Unit 6. Generative motion and avatars

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Human motion diffusion/transformers, articulated avatars, dynamic scene synthesis.

A. Reading order

1. Read the primary assignment first: Read Loper et al., SMPL (SIGGRAPH Asia 2015) for the body model and Tevet et al., Human Motion Diffusion Model (ICLR 2023) for diffusion-based motion; add one recent avatar/4D representation paper only after Vol. 9 contact/kinematic diagnostics are working.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Run a motion-generation baseline.
3. Keep the most useful executable/reference page open while coding: PyTorch stable documentation plus the kinematics/contact diagnostics developed in Volume 9; use the official model repository only after the classical motion tests are working.

B. Definitions, results and algorithms to know

- Kinematic constraints, foot contact, physical plausibility
- Precision requirement: distinguish and justify frame-wise or unconstrained motion generation vs temporally/kinematically constrained generation on foot contact, joint limits, and trajectory control. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Formulate conditional motion generation with temporal/kinematic constraints; derive one contact or joint-limit penalty and explain why framewise perceptual metrics miss motion failure.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use long sequences, contacts, style/identity constraints, and user edits; measure foot skate, identity drift, temporal discontinuity, and control adherence. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Quantify foot sliding, joint limits, trajectory control, and perceptual diversity. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a planar two-link leg over three frames, impose foot contact at frame 2 and joint-angle limits. Compare unconstrained interpolation with a constrained correction and compute foot-sliding error. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use long sequences, contacts, style/identity constraints, and user edits; measure foot skate, identity drift, temporal discontinuity, and control adherence. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Automate foot-contact, joint-limit, acceleration/jerk, trajectory, and diversity metrics across generated clips; create a failure montage by metric.
- P5 - Machine-readable evidence. Export foot-contact/sliding error, joint-limit violations, trajectory error, jerk/smoothness, diversity, and user/perceptual score to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: frame-wise or unconstrained motion generation vs temporally/kinematically constrained generation on foot contact, joint limits, and trajectory control. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use long sequences, contacts, style/identity constraints, and user edits; measure foot skate, identity drift, temporal discontinuity, and control adherence." without hand-editing code between methods. Record foot-contact/sliding error, joint-limit violations, trajectory error, jerk/smoothness, diversity, and user/perceptual score.
- P7 - Deliberate bug and regression drill. Intentionally optimize perceptual motion score while omitting foot/contact/penetration diagnostics. Before running, predict the first observable failure in foot-contact/sliding error, joint-limit violations, trajectory error, jerk/smoothness, diversity, and user/perceptual score. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Formulate conditional motion generation with temporal/kinematic constraints; derive one contact or joint-limit penalty and explain why framewise perceptual metrics miss motion failure. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a planar two-link leg over three frames, impose foot contact at frame 2 and joint-angle limits. Compare unconstrained interpolation with a constrained correction and compute foot-sliding error. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Quantify foot sliding, joint limits, trajectory control, and perceptual diversity. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Automate foot-contact, joint-limit, acceleration/jerk, trajectory, and diversity metrics across generated clips; create a failure montage by metric. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare frame-wise or unconstrained motion generation vs temporally/kinematically constrained generation on foot contact, joint limits, and trajectory control. Use the same training data/model scale where relevant, prompts/conditions, number of generated samples and compute budget; report semantic quality together with diversity, validity and physical/geometric metrics. Record foot-contact/sliding error, joint-limit violations, trajectory error, jerk/smoothness, diversity, and user/perceptual score. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use long sequences, contacts, style/identity constraints, and user edits; measure foot skate, identity drift, temporal discontinuity, and control adherence. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Recent SIGGRAPH/CVPR motion/avatars papers; Vol. 9 cross-reference. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Generative motion and avatars. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among frame-wise or unconstrained motion generation vs temporally/kinematically constrained generation on foot contact, joint limits, and trajectory control. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using foot-contact/sliding error, joint-limit violations, trajectory error, jerk/smoothness, diversity, and user/perceptual score. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if frame-wise or unconstrained motion generation vs temporally/kinematically constrained generation on foot contact, joint limits, and trajectory control improve a joint semantic-diversity-validity/physics tradeoff in foot-contact/sliding error, joint-limit violations, trajectory error, jerk/smoothness, diversity, and user/perceptual score across multiple prompts/seeds at matched compute; reject gains caused by mode collapse or a weak validity checker.

F. Concrete research project - Generative motion and avatars

- F1. Falsifiable question. Under the controlled failure regime "Use long sequences, contacts, style/identity constraints, and user edits; measure foot skate, identity drift, temporal discontinuity, and control adherence.", which alternative in frame-wise or unconstrained motion generation vs temporally/kinematically constrained generation on foot contact, joint limits, and trajectory control is preferable, and is the change explained by data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Automate foot-contact, joint-limit, acceleration/jerk, trajectory, and diversity metrics across generated clips; create a failure montage by metric. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use a fixed batch of at least 50 generated outputs for a compact study (more for high-variance conclusions); keep prompts/seeds/compute matched, annotate at least 20 failures, and evaluate geometry/constraint/task validity separately from perceptual quality. The central adversarial condition is: Use long sequences, contacts, style/identity constraints, and user edits; measure foot skate, identity drift, temporal discontinuity, and control adherence.
- F4. Required evidence. Measure foot-contact/sliding error, joint-limit violations, trajectory error, jerk/smoothness, diversity, and user/perceptual score. Deliver the generation batch, method-independent evaluator, annotation/failure table, matched-budget comparison, and at least one example where a common perceptual metric gives the wrong research ranking. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Generative motion and avatars. If a stable failure/crossover is found among frame-wise or unconstrained motion generation vs temporally/kinematically constrained generation on foot contact, joint limits, and trajectory control, implement one adaptive or corrected variant that directly targets data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Generative motion and avatars. Using the measured foot-contact/sliding error, joint-limit violations, trajectory error, jerk/smoothness, diversity, and user/perceptual score, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Generative motion and avatars

- G1. Closed-book derivation/oracle. Formulate conditional motion generation with temporal/kinematic constraints; derive one contact or joint-limit penalty and explain why framewise perceptual metrics miss motion failure. Then solve this exact micro-case from scratch without opening your implementation: For a planar two-link leg over three frames, impose foot contact at frame 2 and joint-angle limits. Compare unconstrained interpolation with a constrained correction and compute foot-sliding error. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Quantify foot sliding, joint limits, trajectory control, and perceptual diversity. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally optimize perceptual motion score while omitting foot/contact/penetration diagnostics. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Automate foot-contact, joint-limit, acceleration/jerk, trajectory, and diversity metrics across generated clips; create a failure montage by metric. Then apply this stress without retuning: Use long sequences, contacts, style/identity constraints, and user edits; measure foot skate, identity drift, temporal discontinuity, and control adherence. Report foot-contact/sliding error, joint-limit violations, trajectory error, jerk/smoothness, diversity, and user/perceptual score and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among frame-wise or unconstrained motion generation vs temporally/kinematically constrained generation on foot contact, joint limits, and trajectory control for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured foot-contact/sliding error, joint-limit violations, trajectory error, jerk/smoothness, diversity, and user/perceptual score; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Generative motion and avatars. Using foot-contact/sliding error, joint-limit violations, trajectory error, jerk/smoothness, diversity, and user/perceptual score, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 15, Unit 7. Physics-aware generation

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Physical constraints, differentiable simulation, stability/feasibility checks.

A. Reading order

1. Read the primary assignment first: Read the relevant constraint/simulation chapters of Physics-Based Simulation, Cao & Kalogerakis, SOPHY: Learning to Generate Simulation-Ready Objects with Physical Materials (2025), and Lin et al., PAT3D: Physics-Augmented Text-to-3D Scene Generation (2025). For design optimization, optionally compare with Yang et al., Physics-guided generative design with gradient-based source-space optimization on flow matching (2026). Keep the independent simulator/checker outside the generator when measuring validity.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Build a simple generate-test-refine loop.
3. Keep the most useful executable/reference page open while coding: Physics-Based Simulation book - <https://phys-sim-book.github.io/> plus PyTorch stable documentation; the constraint/simulator checker must remain independent of the generator.

B. Definitions, results and algorithms to know

- Feasibility as constraint or energy
- simulator-in-the-loop generation
- Precision requirement: distinguish and justify generate-then-filter vs simulator-in-the-loop refinement/conditioning on structural/dynamic validity and diversity. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a constrained or verifier-guided generative objective where physical validity enters as training loss, guidance, rejection, or refinement; compare bias and compute implications.

- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Sweep physical-constraint strength and where the verifier/physics enters the pipeline; plot a validity-diversity-quality Pareto frontier. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Generate structures/animations then reject/refine using simulator; report validity vs diversity. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: A generated cantilever is represented by two beam thickness variables with fixed volume. Use a simple compliance surrogate proportional to $1/I$ to compare two designs and show how a visually plausible thin section can be mechanically invalid. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Sweep physical-constraint strength and where the verifier/physics enters the pipeline; plot a validity-diversity-quality Pareto frontier. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Wrap a simulator/constraint checker around generated structures/motions; compare rejection-only, repair, and constraint-conditioned approaches under equal generation budget.
- P5 - Machine-readable evidence. Export physical validity rate, constraint violation magnitude, downstream simulation success, diversity loss, and compute overhead to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: generate-then-filter vs simulator-in-the-loop refinement/conditioning on structural/dynamic validity and diversity. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Sweep physical-constraint strength and where the verifier/physics enters the pipeline; plot a validity-diversity-quality Pareto frontier." without hand-editing code between methods. Record physical validity rate, constraint violation magnitude, downstream simulation success, diversity loss, and compute overhead.
- P7 - Deliberate bug and regression drill. Intentionally evaluate only the differentiable proxy constraint and never run the independent simulator/geometry checker. Before running, predict the first observable failure in physical validity rate, constraint violation magnitude, downstream simulation success, diversity loss, and compute overhead. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a constrained or verifier-guided generative objective where physical validity enters as training loss, guidance, rejection, or refinement; compare bias and compute implications. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. A generated cantilever is represented by two beam thickness variables with fixed volume. Use a simple compliance surrogate proportional to $1/I$ to compare two designs and show how a visually plausible thin section can be mechanically invalid. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Generate structures/animations then reject/refine using simulator; report validity vs diversity. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Wrap a simulator/constraint checker around generated structures/motions; compare rejection-only, repair, and constraint-conditioned approaches under equal generation budget. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare generate-then-filter vs simulator-in-the-loop refinement/conditioning on structural/dynamic validity and diversity. Use the same training data/model scale where relevant, prompts/conditions, number of generated samples and compute budget; report semantic quality together with diversity, validity and physical/geometric metrics. Record physical validity rate, constraint violation magnitude, downstream simulation success, diversity loss, and compute overhead. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Sweep physical-constraint strength and where the verifier/physics enters the pipeline; plot a validity-diversity-quality Pareto frontier. State the predicted failure signature before execution. Deliverable: a

sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Recent graphics/robotics generative-design papers; Vol. 10-13 methods. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Physics-aware generation. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among generate-then-filter vs simulator-in-the-loop refinement/conditioning on structural/dynamic validity and diversity. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using physical validity rate, constraint violation magnitude, downstream simulation success, diversity loss, and compute overhead. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if generate-then-filter vs simulator-in-the-loop refinement/conditioning on structural/dynamic validity and diversity improve a joint semantic-diversity-validity/physics tradeoff in physical validity rate, constraint violation magnitude, downstream simulation success, diversity loss, and compute overhead across multiple prompts/seeds at matched compute; reject gains caused by mode collapse or a weak validity checker.

F. Concrete research project - Physics-aware generation

F1. Falsifiable question. Under the controlled failure regime "Sweep physical-constraint strength and where the verifier/physics enters the pipeline; plot a validity-diversity-quality Pareto frontier.", which alternative in generate-then-filter vs simulator-in-the-loop refinement/conditioning on structural/dynamic validity and diversity is preferable, and is the change explained by data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Wrap a simulator/constraint checker around generated structures/motions; compare rejection-only, repair, and constraint-conditioned approaches under equal generation budget. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a fixed batch of at least 50 generated outputs for a compact study (more for high-variance conclusions); keep prompts/seeds/compute matched, annotate at least 20 failures, and evaluate geometry/constraint/task validity separately from perceptual quality. The central adversarial condition is: Sweep physical-constraint strength and where the verifier/physics enters the pipeline; plot a validity-diversity-quality Pareto frontier.

F4. Required evidence. Measure physical validity rate, constraint violation magnitude, downstream simulation success, diversity loss, and compute overhead. Deliver the generation batch, method-independent evaluator, annotation/failure table, matched-budget comparison, and at least one example where a common perceptual metric gives the wrong research ranking. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Physics-aware generation. If a stable failure/crossover is found among generate-then-filter vs simulator-in-the-loop refinement/conditioning on structural/dynamic validity and diversity, implement one adaptive or corrected variant that directly targets data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Physics-aware generation. Using the measured physical validity rate, constraint violation magnitude, downstream simulation success, diversity loss, and compute overhead, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Physics-aware generation

- G1. Closed-book derivation/oracle. Derive a constrained or verifier-guided generative objective where physical validity enters as training loss, guidance, rejection, or refinement; compare bias and compute implications. Then solve this exact micro-case from scratch without opening your implementation: A generated cantilever is

represented by two beam thickness variables with fixed volume. Use a simple compliance surrogate proportional to $1/I$ to compare two designs and show how a visually plausible thin section can be mechanically invalid. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Generate structures/animations then reject/refine using simulator; report validity vs diversity. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally evaluate only the differentiable proxy constraint and never run the independent simulator/geometry checker. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Wrap a simulator/constraint checker around generated structures/motions; compare rejection-only, repair, and constraint-conditioned approaches under equal generation budget. Then apply this stress without retuning: Sweep physical-constraint strength and where the verifier/physics enters the pipeline; plot a validity-diversity-quality Pareto frontier. Report physical validity rate, constraint violation magnitude, downstream simulation success, diversity loss, and compute overhead and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among generate-then-filter vs simulator-in-the-loop refinement/conditioning on structural/dynamic validity and diversity for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured physical validity rate, constraint violation magnitude, downstream simulation success, diversity loss, and compute overhead; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Physics-aware generation. Using physical validity rate, constraint violation magnitude, downstream simulation success, diversity loss, and compute overhead, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 15, Unit 8. Evaluation and research methodology

Recommended time. 2.2-3.0 weeks (about 34-60 focused hours depending on background).

What you are learning. Metrics, human studies, 3D consistency, geometry metrics, rendering metrics, benchmark leakage.

A. Reading order

1. Read the primary assignment first: Read the evaluation/ablation sections of at least three strong recent graphics papers in the same subtopic and the current SIGGRAPH Technical Papers evaluation/review guidance. Recompute one table with your own metric script before adopting its protocol.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Reproduce evaluation of one paper.
3. Keep the most useful executable/reference page open while coding: Use the official code/data for the benchmark under study and your own metric harness; use SIGGRAPH 2026 only to find current examples, not as the executable reference.

B. Definitions, results and algorithms to know

- Chamfer/Hausdorff caveats
- FID-like metric limitations
- user-study design basics
- Precision requirement: distinguish and justify standard image/text metrics vs geometry/temporal/physics/task metrics on deliberately constructed ranking reversals. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive an evaluation matrix separating fidelity, diversity, geometry, temporal consistency, physical validity, human preference, and compute; show which conclusions are invalid if only one metric is reported.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Construct cases where popular metrics disagree with geometry/physics/human judgment; test rank

correlation on a small controlled annotation set. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Design an evaluation protocol that catches a failure hidden by a standard image metric. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Construct two hypothetical 3D methods A/B where A has better image PSNR but worse Chamfer/normal consistency. Make a four-scene table that reverses ranking under the two metrics and explain why one scalar leaderboard is insufficient. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Construct cases where popular metrics disagree with geometry/physics/human judgment; test rank correlation on a small controlled annotation set. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Construct at least 20 paired outputs where a popular metric intentionally ranks the worse result higher; test alternative metrics and report rank reversals.
- P5 - Machine-readable evidence. Export metric rank correlation with task/geometry/physics/human judgments, confidence intervals, failure-pair detection rate, and sensitivity to dataset composition to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: standard image/text metrics vs geometry/temporal/physics/task metrics on deliberately constructed ranking reversals. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Construct cases where popular metrics disagree with geometry/physics/human judgment; test rank correlation on a small controlled annotation set." without hand-editing code between methods. Record metric rank correlation with task/geometry/physics/human judgments, confidence intervals, failure-pair detection rate, and sensitivity to dataset composition.
- P7 - Deliberate bug and regression drill. Intentionally select the metric after seeing which one favors the preferred method. Before running, predict the first observable failure in metric rank correlation with task/geometry/physics/human judgments, confidence intervals, failure-pair detection rate, and sensitivity to dataset composition. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive an evaluation matrix separating fidelity, diversity, geometry, temporal consistency, physical validity, human preference, and compute; show which conclusions are invalid if only one metric is reported. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Construct two hypothetical 3D methods A/B where A has better image PSNR but worse Chamfer/normal consistency. Make a four-scene table that reverses ranking under the two metrics and explain why one scalar leaderboard is insufficient. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Design an evaluation protocol that catches a failure hidden by a standard image metric. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Construct at least 20 paired outputs where a popular metric intentionally ranks the worse result higher; test alternative metrics and report rank reversals. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare standard image/text metrics vs geometry/temporal/physics/task metrics on deliberately constructed ranking reversals. Use the same training data/model scale where relevant, prompts/conditions, number of generated samples and compute budget; report semantic quality together with diversity, validity and physical/geometric metrics. Record metric rank correlation with task/geometry/physics/human judgments, confidence intervals, failure-pair detection rate, and sensitivity to dataset composition. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Construct cases where popular metrics disagree with geometry/physics/human judgment; test rank correlation on a small controlled annotation set. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Current benchmark papers and SIGGRAPH author/reviewer guidelines. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Evaluation and research methodology. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among standard image/text metrics vs geometry/temporal/physics/task metrics on deliberately constructed ranking reversals. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using metric rank correlation with task/geometry/physics/human judgments, confidence intervals, failure-pair detection rate, and sensitivity to dataset composition. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if standard image/text metrics vs geometry/temporal/physics/task metrics on deliberately constructed ranking reversals improve a joint semantic-diversity-validity/physics tradeoff in metric rank correlation with task/geometry/physics/human judgments, confidence intervals, failure-pair detection rate, and sensitivity to dataset composition across multiple prompts/seeds at matched compute; reject gains caused by mode collapse or a weak validity checker.

F. Concrete research project - Evaluation and research methodology

F1. Falsifiable question. Under the controlled failure regime "Construct cases where popular metrics disagree with geometry/physics/human judgment; test rank correlation on a small controlled annotation set.", which alternative in standard image/text metrics vs geometry/temporal/physics/task metrics on deliberately constructed ranking reversals is preferable, and is the change explained by data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Construct at least 20 paired outputs where a popular metric intentionally ranks the worse result higher; test alternative metrics and report rank reversals. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use a fixed batch of at least 50 generated outputs for a compact study (more for high-variance conclusions); keep prompts/seeds/compute matched, annotate at least 20 failures, and evaluate geometry/constraint/task validity separately from perceptual quality. The central adversarial condition is: Construct cases where popular metrics disagree with geometry/physics/human judgment; test rank correlation on a small controlled annotation set.

F4. Required evidence. Measure metric rank correlation with task/geometry/physics/human judgments, confidence intervals, failure-pair detection rate, and sensitivity to dataset composition. Deliver the generation batch, method-independent evaluator, annotation/failure table, matched-budget comparison, and at least one example where a common perceptual metric gives the wrong research ranking. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Evaluation and research methodology. If a stable failure/crossover is found among standard image/text metrics vs geometry/temporal/physics/task metrics on deliberately constructed ranking reversals, implement one adaptive or corrected variant that directly targets data/metric validity, geometric/physical constraint satisfaction, diversity, and matched-compute evaluation; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Evaluation and research methodology. Using the measured metric rank correlation with task/geometry/physics/human judgments, confidence intervals, failure-pair detection rate, and sensitivity to dataset composition, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Evaluation and research methodology

- G1. Closed-book derivation/oracle. Derive an evaluation matrix separating fidelity, diversity, geometry, temporal consistency, physical validity, human preference, and compute; show which conclusions are invalid if only one metric is reported. Then solve this exact micro-case from scratch without opening your implementation: Construct

two hypothetical 3D methods A/B where A has better image PSNR but worse Chamfer/normal consistency. Make a four-scene table that reverses ranking under the two metrics and explain why one scalar leaderboard is insufficient. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Design an evaluation protocol that catches a failure hidden by a standard image metric. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally select the metric after seeing which one favors the preferred method. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Construct at least 20 paired outputs where a popular metric intentionally ranks the worse result higher; test alternative metrics and report rank reversals. Then apply this stress without retuning: Construct cases where popular metrics disagree with geometry/physics/human judgment; test rank correlation on a small controlled annotation set. Report metric rank correlation with task/geometry/physics/human judgments, confidence intervals, failure-pair detection rate, and sensitivity to dataset composition and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among standard image/text metrics vs geometry/temporal/physics/task metrics on deliberately constructed ranking reversals for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured metric rank correlation with task/geometry/physics/human judgments, confidence intervals, failure-pair detection rate, and sensitivity to dataset composition; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Evaluation and research methodology. Using metric rank correlation with task/geometry/physics/human judgments, confidence intervals, failure-pair detection rate, and sensitivity to dataset composition, Pass when the evaluator is independent of the generator, geometry/physical/task validity is separated from perceptual quality, and the conclusion survives a fixed batch plus annotated failure analysis. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 15 Closeout: Project, Review and Mastery Gate

Volume project. Run a generative-3D/4D study evaluated by explicit 3D, temporal or physical constraints rather than only visual appeal. Produce a benchmark and ablation table.

Cumulative review

- Implement a small diffusion/score-model toy experiment so generation mathematics is not a black box.
- Evaluate a 3D/motion generator with explicit geometry/constraint/physics/task checks, not only CLIP/image metrics.
- Construct and annotate a failure set large enough to reveal recurring modes and ranking reversals between metrics.
- Run one matched-budget comparison or ablation that separates representation/guidance/evaluation effects.
- Write a generative-graphics synthesis identifying which apparent gains could arise from evaluator bias or 2D appearance shortcuts.

Mastery gate before moving on

- Evaluate 3D/4D generation with explicit geometric, temporal, or physical validity plus diversity-not only attractive renders.
- Run a capacity/compute-aware ablation supporting a causal claim about one conditioning/constraint mechanism.
- Identify one metric failure and propose/test a more task-relevant evaluation.
- Literature gate: reconstruct the DDPM/score-model objective and one 3D guidance/generation mechanism; then audit a recent 3D generative paper for geometry validity, multiview consistency and evaluator dependence.

Volume 16. Visualization, Interaction, VR/AR, Fabrication, and Special Topics

Purpose. Broaden from image synthesis to visual computing: communicating data, interactive systems, immersive interfaces, and graphics-to-physical pipelines.

Suggested duration. 12-18 weeks

Prerequisites. Classical core. Individual topics have their own dependencies.

Typical module pace. 1.7-2.6 weeks per module (7 modules total).

Primary references for this volume

- Munzner, Visualization Analysis and Design (selected chapters).
- OpenXR official material for immersive systems.
- SIGGRAPH technical papers/courses for fabrication, haptics, audio and optics.

Online spine

SIGGRAPH 2026 Technical Papers: <https://s2026.siggraph.org/program/technical-papers/>

Berkeley CS184/284A Spring 2026 course: <https://cs184.eecs.berkeley.edu/sp26/>

Canonical paper track

- Munzner visualization framework readings plus classic volume-rendering papers.
- Lorensen & Cline (1987), Marching Cubes (bridge from visualization to geometry).
- OpenXR specification/tutorials for immersive systems.
- Representative SIGGRAPH fabrication, haptics, audio and optics papers selected by specialization.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Munzner, Visualization Analysis and Design - task/data/encoding/validation framework for scientific and information visualization.
- Lorensen & Cline (1987), Marching Cubes: A High Resolution 3D Surface Construction Algorithm.
- Drebin, Carpenter & Hanrahan (1988), Volume Rendering.
- Khronos OpenXR specification/resources - implementation reference for VR/AR interaction and spatial-computing work.
- For computational fabrication, haptics, audio and optics, choose one classic mechanism paper plus one recent SIGGRAPH/TOG paper in the exact subtopic; do not reuse a paper merely because it belongs to the same volume.
- Use the SIGGRAPH 2026 technical-paper program only as a discovery index; read the actual paper/project page before assigning it to an experiment.

Volume 16, Unit 1. Scientific and information visualization

Recommended time. 1.7-2.6 weeks (about 26-51 focused hours depending on background).

What you are learning. Marks/channels, scalar/vector/tensor fields, volume rendering, transfer functions.

A. Reading order

1. Read the primary assignment first: FCG Ch. 23; visualization textbook selections (e.g., Munzner).
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Visualization course lectures.
3. Keep the most useful executable/reference page open while coding: VTK documentation/tutorials - <https://docs.vtk.org/> ; use the task/encoding framework from Munzner as the design oracle.

B. Definitions, results and algorithms to know

- Perceptual ordering

- direct vs derived quantities
- interpolation artifacts
- Precision requirement: distinguish and justify multiple encodings of the same scalar/vector dataset compared by task accuracy and perceptual ambiguity, not aesthetics alone. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a mapping from data variables to visual channels for a concrete dataset and analyze perceptual effectiveness, scale transforms, uncertainty encoding, and task-dependent error.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use noisy/multiscale data with a specific lookup/compare/trend task; measure error, time, and uncertainty comprehension. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Build scalar field, vector field, and volume visualizations with explicit design rationale. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For scalar values [1,2,4,8] assigned to four regions, encode them once by area and once by aligned position. Compute the ratios viewers must judge and predict which task (ratio vs ordering) each encoding supports better. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use noisy/multiscale data with a specific lookup/compare/trend task; measure error, time, and uncertainty comprehension. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement two encodings for the same task and run a small timed accuracy study; log errors by data regime and revise the design based on observed failure.
- P5 - Machine-readable evidence. Export task accuracy/time, error type, perceptual ambiguity, interaction count, and performance on defined data regimes to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: multiple encodings of the same scalar/vector dataset compared by task accuracy and perceptual ambiguity, not aesthetics alone. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use noisy/multiscale data with a specific lookup/compare/trend task; measure error, time, and uncertainty comprehension." without hand-editing code between methods. Record task accuracy/time, error type, perceptual ambiguity, interaction count, and performance on defined data regimes.
- P7 - Deliberate bug and regression drill. Intentionally normalize/color-map two datasets differently while presenting them as directly comparable. Before running, predict the first observable failure in task accuracy/time, error type, perceptual ambiguity, interaction count, and performance on defined data regimes. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a mapping from data variables to visual channels for a concrete dataset and analyze perceptual effectiveness, scale transforms, uncertainty encoding, and task-dependent error. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For scalar values [1,2,4,8] assigned to four regions, encode them once by area and once by aligned position. Compute the ratios viewers must judge and predict which task (ratio vs ordering) each encoding supports better. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Build scalar field, vector field, and volume visualizations with explicit design rationale. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement two encodings for the same task and run a small timed accuracy study; log errors by data regime and revise the design based on observed failure. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare multiple encodings of the same scalar/vector dataset compared by task accuracy and perceptual ambiguity, not aesthetics alone. Use the same dataset/task, display/hardware and

interaction/viewer protocol appropriate to the subtopic; state explicitly whether the fixed budget is pixels/samples, latency, bandwidth, fabrication resolution or participant time. Record task accuracy/time, error type, perceptual ambiguity, interaction count, and performance on defined data regimes. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use noisy/multiscale data with a specific lookup/compare/trend task; measure error, time, and uncertainty comprehension. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: FCG Ch. 23; visualization textbook selections (e.g., Munzner). Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Scientific and information visualization. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among multiple encodings of the same scalar/vector dataset compared by task accuracy and perceptual ambiguity, not aesthetics alone. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using task accuracy/time, error type, perceptual ambiguity, interaction count, and performance on defined data regimes. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if multiple encodings of the same scalar/vector dataset compared by task accuracy and perceptual ambiguity, not aesthetics alone improve the task-specific tradeoff in task accuracy/time, error type, perceptual ambiguity, interaction count, and performance on defined data regimes under the same data/display/hardware/viewer protocol; use the metric that reflects the actual visualization/interaction/fabrication/optics task rather than generic image quality.

F. Concrete research project - Scientific and information visualization

F1. Falsifiable question. Under the controlled failure regime "Use noisy/multiscale data with a specific lookup/compare/trend task; measure error, time, and uncertainty comprehension.", characterize the ranking or regime transition across multiple encodings of the same scalar/vector dataset compared by task accuracy and perceptual ambiguity, not aesthetics alone. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement two encodings for the same task and run a small timed accuracy study; log errors by data regime and revise the design based on observed failure. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Define the task-success metric before implementation. Include one analytically/synthetically verifiable case and two realistic cases, then sweep at least one fidelity/latency/geometry parameter. The central adversarial condition is: Use noisy/multiscale data with a specific lookup/compare/trend task; measure error, time, and uncertainty comprehension.

F4. Required evidence. Measure task accuracy/time, error type, perceptual ambiguity, interaction count, and performance on defined data regimes. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Scientific and information visualization. If a stable failure/crossover is found among multiple encodings of the same scalar/vector dataset compared by task accuracy and perceptual ambiguity, not aesthetics alone, implement one adaptive or corrected variant that directly targets task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Scientific and information visualization. Using the measured task accuracy/time, error type, perceptual ambiguity, interaction count, and performance on defined data regimes, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives

the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Scientific and information visualization

- G1. Closed-book derivation/oracle. Derive a mapping from data variables to visual channels for a concrete dataset and analyze perceptual effectiveness, scale transforms, uncertainty encoding, and task-dependent error. Then solve this exact micro-case from scratch without opening your implementation: For scalar values [1,2,4,8] assigned to four regions, encode them once by area and once by aligned position. Compute the ratios viewers must judge and predict which task (ratio vs ordering) each encoding supports better. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Build scalar field, vector field, and volume visualizations with explicit design rationale. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally normalize/color-map two datasets differently while presenting them as directly comparable. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement two encodings for the same task and run a small timed accuracy study; log errors by data regime and revise the design based on observed failure. Then apply this stress without retuning: Use noisy/multiscale data with a specific lookup/compare/trend task; measure error, time, and uncertainty comprehension. Report task accuracy/time, error type, perceptual ambiguity, interaction count, and performance on defined data regimes and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across multiple encodings of the same scalar/vector dataset compared by task accuracy and perceptual ambiguity, not aesthetics alone for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured task accuracy/time, error type, perceptual ambiguity, interaction count, and performance on defined data regimes; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Scientific and information visualization. Using task accuracy/time, error type, perceptual ambiguity, interaction count, and performance on defined data regimes, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 16, Unit 2. Volume rendering

Recommended time. 1.7-2.6 weeks (about 26-51 focused hours depending on background).

What you are learning. Ray marching, transfer functions, compositing, preintegration intuition, scientific volumes.

A. Reading order

1. Read the primary assignment first: Read Drebin, Carpenter & Hanrahan, Volume Rendering (SIGGRAPH 1988), and Levoy, Display of Surfaces from Volume Data (IEEE CG&A 1988), then use VTK documentation for a modern executable ray-casting/transfer-function implementation. Keep this distinct from PBRT participating-media transport.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Implement GPU/CPU volume ray marcher.
3. Keep the most useful executable/reference page open while coding: VTK documentation/tutorials for scientific volume rendering - <https://docs.vtk.org/> ; keep physical volume scattering (PBRT) separate unless the task actually models light transport in media.

B. Definitions, results and algorithms to know

- Emission-absorption model
- front-to-back compositing
- Precision requirement: distinguish and justify front-to-back compositing with different step sizes, transfer functions, and empty-space skipping against a high-sample reference. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the emission-absorption volume-rendering integral and its front-to-back discrete compositing form; derive transfer-function effects on opacity/color and early ray termination.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use tiny high-value structures, noisy scalar fields, and overlapping transfer-function ranges; test feature detectability and sampling error. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Visualize CT-like synthetic volume; study sampling step and transfer-function sensitivity. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For two samples with opacities $\alpha=[0.2,0.5]$ and colors $c=[1,0.4]$, compute front-to-back compositing color/opacity. Split each sample into two half-length samples with adjusted alpha and verify approximate step-size consistency. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use tiny high-value structures, noisy scalar fields, and overlapping transfer-function ranges; test feature detectability and sampling error. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement CPU/GPU ray marching with transfer function and early termination; compare error/runtime as step size and empty-space skipping change.
- P5 - Machine-readable evidence. Export image error to high-sample reference, feature visibility, step count, early-termination/empty-space savings, and frame time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: front-to-back compositing with different step sizes, transfer functions, and empty-space skipping against a high-sample reference. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use tiny high-value structures, noisy scalar fields, and overlapping transfer-function ranges; test feature detectability and sampling error." without hand-editing code between methods. Record image error to high-sample reference, feature visibility, step count, early-termination/empty-space savings, and frame time.
- P7 - Deliberate bug and regression drill. Intentionally compare transfer functions with different opacity accumulation/preintegration conventions. Before running, predict the first observable failure in image error to high-sample reference, feature visibility, step count, early-termination/empty-space savings, and frame time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the emission-absorption volume-rendering integral and its front-to-back discrete compositing form; derive transfer-function effects on opacity/color and early ray termination. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For two samples with opacities $\alpha=[0.2,0.5]$ and colors $c=[1,0.4]$, compute front-to-back compositing color/opacity. Split each sample into two half-length samples with adjusted alpha and verify approximate step-size consistency. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Visualize CT-like synthetic volume; study sampling step and transfer-function sensitivity. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement CPU/GPU ray marching with transfer function and early termination; compare error/runtime as step size and empty-space skipping change. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare front-to-back compositing with different step sizes, transfer functions, and empty-space skipping against a high-sample reference. Use the same dataset/task, display/hardware and interaction/viewer protocol appropriate to the subtopic; state explicitly whether the fixed budget is pixels/samples, latency, bandwidth, fabrication resolution or participant time. Record image error to high-sample reference, feature visibility, step count, early-termination/empty-space savings, and frame time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use tiny high-value structures, noisy scalar fields, and overlapping transfer-function ranges; test feature detectability and sampling error. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: For interactive volume visualization, use visualization/volume-rendering notes and classic ray-casting literature; use PBRT Chs. 11 and 14 only to understand physical volume scattering, which is a different problem from scientific volume visualization. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Volume rendering. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among front-to-back compositing with different step sizes, transfer functions, and empty-space skipping against a high-sample reference. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using image error to high-sample reference, feature visibility, step count, early-termination/empty-space savings, and frame time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if front-to-back compositing with different step sizes, transfer functions, and empty-space skipping against a high-sample reference improve the task-specific tradeoff in image error to high-sample reference, feature visibility, step count, early-termination/empty-space savings, and frame time under the same data/display/hardware/viewer protocol; use the metric that reflects the actual visualization/interaction/fabrication/optics task rather than generic image quality.

F. Concrete research project - Volume rendering

- F1. Falsifiable question. Under the controlled failure regime "Use tiny high-value structures, noisy scalar fields, and overlapping transfer-function ranges; test feature detectability and sampling error.", characterize the ranking or regime transition across front-to-back compositing with different step sizes, transfer functions, and empty-space skipping against a high-sample reference. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement CPU/GPU ray marching with transfer function and early termination; compare error/runtime as step size and empty-space skipping change. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Define the task-success metric before implementation. Include one analytically/synthetically verifiable case and two realistic cases, then sweep at least one fidelity/latency/geometry parameter. The central adversarial condition is: Use tiny high-value structures, noisy scalar fields, and overlapping transfer-function ranges; test feature detectability and sampling error.
- F4. Required evidence. Measure image error to high-sample reference, feature visibility, step count, early-termination/empty-space savings, and frame time. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Volume rendering. If a stable failure/crossover is found among front-to-back compositing with different step sizes, transfer functions, and empty-space skipping against a high-sample reference, implement one adaptive or corrected variant that directly targets task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Volume rendering. Using the measured image error to high-sample reference, feature visibility, step count, early-termination/empty-space savings, and frame time, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Volume rendering

- G1. Closed-book derivation/oracle. Derive the emission-absorption volume-rendering integral and its front-to-back discrete compositing form; derive transfer-function effects on opacity/color and early ray termination. Then solve this exact micro-case from scratch without opening your implementation: For two samples with opacities $\alpha=[0.2,0.5]$ and colors $c=[1,0.4]$, compute front-to-back compositing color/opacity. Split each sample into two half-length samples with adjusted alpha and verify approximate step-size consistency. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Visualize CT-like synthetic volume; study sampling step and transfer-function sensitivity. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally compare transfer functions with different opacity accumulation/preintegration conventions. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement CPU/GPU ray marching with transfer function and early termination; compare error/runtime as step size and empty-space skipping change. Then apply this stress without retuning: Use tiny high-value structures, noisy scalar fields, and overlapping transfer-function ranges; test feature detectability and sampling error. Report image error to high-sample reference, feature visibility, step count, early-termination/empty-space savings, and frame time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across front-to-back compositing with different step sizes, transfer functions, and empty-space skipping against a high-sample reference for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured image error to high-sample reference, feature visibility, step count, early-termination/empty-space savings, and frame time; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Volume rendering. Using image error to high-sample reference, feature visibility, step count, early-termination/empty-space savings, and frame time, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 16, Unit 3. Interaction and picking

Recommended time. 1.7-2.6 weeks (about 26-51 focused hours depending on background).

What you are learning. Scene manipulation, ray picking, gizmos, camera controls, direct manipulation.

A. Reading order

1. Read the primary assignment first: Graphics API/UI docs; HCI basics as supplement.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Build an interactive mesh viewer.
3. Keep the most useful executable/reference page open while coding: Berkeley CS184 graphics-pipeline material - <https://cs184.eecs.berkeley.edu/sp26/> ; use the API documentation of the renderer/UI framework you actually implement.

B. Definitions, results and algorithms to know

- Input mappings
- selection
- latency/usability tradeoffs
- Precision requirement: distinguish and justify color-ID, ray-cast, and geometry-accelerated picking plus transform-gizmo interaction under occlusion and dense scenes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive screen-to-world unprojection and ray-based picking; formulate nearest-object/drag constraints and analyze coordinate-space and depth ambiguities.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use tiny, occluded, dense, and depth-ambiguous targets; measure selection error, time, and correction

rate. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Implement picking, transform gizmo, camera orbit/pan, and undo. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Ray-cast from origin along +z against spheres centered (0,0,3) radius 1 and (0.2,0,5) radius 1. Compute nearest hit; then show how color-ID picking could disagree if transparent/front layers are rendered differently. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use tiny, occluded, dense, and depth-ambiguous targets; measure selection error, time, and correction rate. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement ray-based picking with BVH and transform gizmo; create automated tests for occlusion, overlapping objects, tiny targets, and undo/redo state.
- P5 - Machine-readable evidence. Export selection accuracy, latency, occlusion/tiny-target failure rate, undo/state correctness, and frame/update time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: color-ID, ray-cast, and geometry-accelerated picking plus transform-gizmo interaction under occlusion and dense scenes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use tiny, occluded, dense, and depth-ambiguous targets; measure selection error, time, and correction rate." without hand-editing code between methods. Record selection accuracy, latency, occlusion/tiny-target failure rate, undo/state correctness, and frame/update time.
- P7 - Deliberate bug and regression drill. Intentionally mix screen, viewport, NDC, camera, and world coordinates in ray construction/picking. Before running, predict the first observable failure in selection accuracy, latency, occlusion/tiny-target failure rate, undo/state correctness, and frame/update time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive screen-to-world unprojection and ray-based picking; formulate nearest-object/drag constraints and analyze coordinate-space and depth ambiguities. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Ray-cast from origin along +z against spheres centered (0,0,3) radius 1 and (0.2,0,5) radius 1. Compute nearest hit; then show how color-ID picking could disagree if transparent/front layers are rendered differently. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Implement picking, transform gizmo, camera orbit/pan, and undo. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement ray-based picking with BVH and transform gizmo; create automated tests for occlusion, overlapping objects, tiny targets, and undo/redo state. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare color-ID, ray-cast, and geometry-accelerated picking plus transform-gizmo interaction under occlusion and dense scenes. Use the same dataset/task, display/hardware and interaction/viewer protocol appropriate to the subtopic; state explicitly whether the fixed budget is pixels/samples, latency, bandwidth, fabrication resolution or participant time. Record selection accuracy, latency, occlusion/tiny-target failure rate, undo/state correctness, and frame/update time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Use tiny, occluded, dense, and depth-ambiguous targets; measure selection error, time, and correction rate. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Graphics API/UI docs; HCI basics as supplement. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on

Interaction and picking. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among color-ID, ray-cast, and geometry-accelerated picking plus transform-gizmo interaction under occlusion and dense scenes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using selection accuracy, latency, occlusion/tiny-target failure rate, undo/state correctness, and frame/update time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if color-ID, ray-cast, and geometry-accelerated picking plus transform-gizmo interaction under occlusion and dense scenes improve the task-specific tradeoff in selection accuracy, latency, occlusion/tiny-target failure rate, undo/state correctness, and frame/update time under the same data/display/hardware/viewer protocol; use the metric that reflects the actual visualization/interaction/fabrication/optics task rather than generic image quality.

F. Concrete research project - Interaction and picking

F1. Falsifiable question. Under the controlled failure regime "Use tiny, occluded, dense, and depth-ambiguous targets; measure selection error, time, and correction rate.", characterize the ranking or regime transition across color-ID, ray-cast, and geometry-accelerated picking plus transform-gizmo interaction under occlusion and dense scenes. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement ray-based picking with BVH and transform gizmo; create automated tests for occlusion, overlapping objects, tiny targets, and undo/redo state. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Define the task-success metric before implementation. Include one analytically/synthetically verifiable case and two realistic cases, then sweep at least one fidelity/latency/geometry parameter. The central adversarial condition is: Use tiny, occluded, dense, and depth-ambiguous targets; measure selection error, time, and correction rate.

F4. Required evidence. Measure selection accuracy, latency, occlusion/tiny-target failure rate, undo/state correctness, and frame/update time. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Interaction and picking. If a stable failure/crossover is found among color-ID, ray-cast, and geometry-accelerated picking plus transform-gizmo interaction under occlusion and dense scenes, implement one adaptive or corrected variant that directly targets task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Interaction and picking. Using the measured selection accuracy, latency, occlusion/tiny-target failure rate, undo/state correctness, and frame/update time, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Interaction and picking

- G1. Closed-book derivation/oracle. Derive screen-to-world unprojection and ray-based picking; formulate nearest-object/drag constraints and analyze coordinate-space and depth ambiguities. Then solve this exact micro-case from scratch without opening your implementation: Ray-cast from origin along +z against spheres centered (0,0,3) radius 1 and (0.2,0,5) radius 1. Compute nearest hit; then show how color-ID picking could disagree if transparent/front layers are rendered differently. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Implement picking, transform gizmo, camera orbit/pan, and undo. Add the oracle as an automated test and make it pass before importing any optimized/framework code.

- G3. Debugging exam. Intentionally mix screen, viewport, NDC, camera, and world coordinates in ray construction/picking. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement ray-based picking with BVH and transform gizmo; create automated tests for occlusion, overlapping objects, tiny targets, and undo/redo state. Then apply this stress without retuning: Use tiny, occluded, dense, and depth-ambiguous targets; measure selection error, time, and correction rate. Report selection accuracy, latency, occlusion/tiny-target failure rate, undo/state correctness, and frame/update time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across color-ID, ray-cast, and geometry-accelerated picking plus transform-gizmo interaction under occlusion and dense scenes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured selection accuracy, latency, occlusion/tiny-target failure rate, undo/state correctness, and frame/update time; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Interaction and picking. Using selection accuracy, latency, occlusion/tiny-target failure rate, undo/state correctness, and frame/update time, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 16, Unit 4. VR/AR and spatial computing

Recommended time. 1.7-2.6 weeks (about 26-51 focused hours depending on background).

What you are learning. Stereo, tracking, world anchoring, occlusion, interaction, foveation.

A. Reading order

1. Read the primary assignment first: CS184 VR material; OpenXR docs.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: OpenXR/WebXR tutorial.
3. Keep the most useful executable/reference page open while coding: OpenXR specification/resources - <https://www.khronos.org/openxr/>

B. Definitions, results and algorithms to know

- Coordinate calibration
- latency
- stereo comfort
- Precision requirement: distinguish and justify desktop/stereo/6DOF presentation of the same spatial task under latency, FOV, and depth-cue changes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive headset pose-to-view transforms and reprojection; model one registration/calibration error and propagate it to angular/pixel misalignment in the display.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Vary latency, registration error, occlusion correctness, and locomotion/interaction demands; measure task performance and comfort. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Build a simple immersive visualization or emulate stereo/6DOF pipeline without headset. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: A head rotates 30 degrees/s and rendering latency is 50 ms. Compute angular prediction error without pose prediction. Convert it to approximate screen displacement for a 90-degree FOV and discuss reprojection. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Vary latency, registration error, occlusion correctness, and locomotion/interaction demands; measure task performance and comfort. Save the input and config of each qualitatively different failure as a named regression case.

- P4 - Research-grade programming extension. Implement stereo camera matrices and a recorded 6DOF camera path; simulate latency by delayed pose and quantify screen-space reprojection error.
- P5 - Machine-readable evidence. Export task/reprojection error, latency sensitivity, stereo/depth-cue errors, comfort proxy, and frame time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: desktop/stereo/6DOF presentation of the same spatial task under latency, FOV, and depth-cue changes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Vary latency, registration error, occlusion correctness, and locomotion/interaction demands; measure task performance and comfort." without hand-editing code between methods. Record task/reprojection error, latency sensitivity, stereo/depth-cue errors, comfort proxy, and frame time.
- P7 - Deliberate bug and regression drill. Intentionally measure rendering FPS but ignore motion-to-photon latency and tracking/prediction error. Before running, predict the first observable failure in task/reprojection error, latency sensitivity, stereo/depth-cue errors, comfort proxy, and frame time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive headset pose-to-view transforms and reprojection; model one registration/calibration error and propagate it to angular/pixel misalignment in the display. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. A head rotates 30 degrees/s and rendering latency is 50 ms. Compute angular prediction error without pose prediction. Convert it to approximate screen displacement for a 90-degree FOV and discuss reprojection. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Build a simple immersive visualization or emulate stereo/6DOF pipeline without headset. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement stereo camera matrices and a recorded 6DOF camera path; simulate latency by delayed pose and quantify screen-space reprojection error. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare desktop/stereo/6DOF presentation of the same spatial task under latency, FOV, and depth-cue changes. Use the same dataset/task, display/hardware and interaction/viewer protocol appropriate to the subtopic; state explicitly whether the fixed budget is pixels/samples, latency, bandwidth, fabrication resolution or participant time. Record task/reprojection error, latency sensitivity, stereo/depth-cue errors, comfort proxy, and frame time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Vary latency, registration error, occlusion correctness, and locomotion/interaction demands; measure task performance and comfort. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: CS184 VR material; OpenXR docs. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on VR/AR and spatial computing. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among desktop/stereo/6DOF presentation of the same spatial task under latency, FOV, and depth-cue changes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using task/reprojection error, latency sensitivity, stereo/depth-cue errors, comfort proxy, and frame time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if desktop/stereo/6DOF presentation of the same spatial task under latency, FOV, and depth-cue changes improve the task-specific tradeoff in task/reprojection error, latency sensitivity, stereo/depth-cue errors, comfort proxy, and frame time under the same data/display/hardware/viewer protocol; use the metric that reflects the actual visualization/interaction/fabrication/optics task rather than generic image quality.

F. Concrete research project - VR/AR and spatial computing

F1. Falsifiable question. Under the controlled failure regime "Vary latency, registration error, occlusion correctness, and locomotion/interaction demands; measure task performance and comfort.", characterize the ranking or regime transition across desktop/stereo/6DOF presentation of the same spatial task under latency, FOV, and depth-cue changes. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement stereo camera matrices and a recorded 6DOF camera path; simulate latency by delayed pose and quantify screen-space reprojection error. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Define the task-success metric before implementation. Include one analytically/synthetically verifiable case and two realistic cases, then sweep at least one fidelity/latency/geometry parameter. The central adversarial condition is: Vary latency, registration error, occlusion correctness, and locomotion/interaction demands; measure task performance and comfort.

F4. Required evidence. Measure task/reprojection error, latency sensitivity, stereo/depth-cue errors, comfort proxy, and frame time. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for VR/AR and spatial computing. If a stable failure/crossover is found among desktop/stereo/6DOF presentation of the same spatial task under latency, FOV, and depth-cue changes, implement one adaptive or corrected variant that directly targets task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for VR/AR and spatial computing. Using the measured task/reprojection error, latency sensitivity, stereo/depth-cue errors, comfort proxy, and frame time, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for VR/AR and spatial computing

- G1. Closed-book derivation/oracle. Derive headset pose-to-view transforms and reprojection; model one registration/calibration error and propagate it to angular/pixel misalignment in the display. Then solve this exact micro-case from scratch without opening your implementation: A head rotates 30 degrees/s and rendering latency is 50 ms. Compute angular prediction error without pose prediction. Convert it to approximate screen displacement for a 90-degree FOV and discuss reprojection. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Build a simple immersive visualization or emulate stereo/6DOF pipeline without headset. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally measure rendering FPS but ignore motion-to-photon latency and tracking/prediction error. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement stereo camera matrices and a recorded 6DOF camera path; simulate latency by delayed pose and quantify screen-space reprojection error. Then apply this stress without retuning: Vary latency, registration error, occlusion correctness, and locomotion/interaction demands; measure task performance and comfort. Report task/reprojection error, latency sensitivity, stereo/depth-cue errors, comfort proxy, and frame time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across desktop/stereo/6DOF presentation of the same spatial task under latency, FOV, and depth-cue changes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured task/reprojection error, latency sensitivity, stereo/depth-cue errors, comfort proxy, and frame time; state one counterexample that would change the diagnosis or ranking.

- G6. Pass condition for VR/AR and spatial computing. Using task/reprojection error, latency sensitivity, stereo/depth-cue errors, comfort proxy, and frame time, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 16, Unit 5. Computational fabrication

Recommended time. 1.7-2.6 weeks (about 26-51 focused hours depending on background).

What you are learning. Geometry for manufacturing, slicing, supports, toolpaths, structural constraints.

A. Reading order

1. Read the primary assignment first: Use Botsch et al./geometry-processing foundations plus current CGAL Polygon Mesh Processing for repair, offsets, slicing-related geometry and Hausdorff checks; then select one primary SIGGRAPH fabrication paper whose manufacturability constraints match your project.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Implement simple slicer/toolpath generator.
3. Keep the most useful executable/reference page open while coding: CGAL Polygon Mesh Processing - https://doc.cgal.org/latest/Polygon_mesh_processing/index.html ; libigl - <https://libigl.github.io/tutorial/> for prototyping geometry operations.

B. Definitions, results and algorithms to know

- Offset surfaces
- winding numbers
- voxel/SDF representations
- manufacturability
- Precision requirement: distinguish and justify mesh/SDF input through slicing/offset/toolpath generation, checking topology, minimum feature size, and manufacturability. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a geometry-to-manufacturing constraint for a chosen process (minimum feature, overhang, tool reach, slicing), and formulate a design/optimization objective that respects it.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Optimize a design near process/manufacturing limits then perturb material/process tolerances; count infeasible, fragile, or functionally degraded outputs. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Take an implicit/mesh object to contours and G-code-like paths; test topology/robustness. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Offset a unit square outward by nozzle radius 0.2 and compute the new contour. Add a slot of width 0.3 and decide whether it survives a minimum printable feature width of 0.4. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Optimize a design near process/manufacturing limits then perturb material/process tolerances; count infeasible, fragile, or functionally degraded outputs. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement slicing/contouring and offsetting on mesh/SDF inputs; detect self-intersections, tiny islands, and minimum-feature violations before toolpath export.
- P5 - Machine-readable evidence. Export topology/contour correctness, self-intersection/feature-size violations, toolpath length/time, and manufacturable-output rate to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: mesh/SDF input through slicing/offset/toolpath generation, checking topology, minimum feature size, and manufacturability. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Optimize a design near process/manufacturing limits then perturb material/process tolerances; count infeasible, fragile, or functionally degraded outputs." without hand-

editing code between methods. Record topology/contour correctness, self-intersection/feature-size violations, toolpath length/time, and manufacturable-output rate.

- P7 - Deliberate bug and regression drill. Intentionally accept non-watertight/self-intersecting geometry and postpone manufacturability checks until after optimization. Before running, predict the first observable failure in topology/contour correctness, self-intersection/feature-size violations, toolpath length/time, and manufacturable-output rate. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a geometry-to-manufacturing constraint for a chosen process (minimum feature, overhang, tool reach, slicing), and formulate a design/optimization objective that respects it. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Offset a unit square outward by nozzle radius 0.2 and compute the new contour. Add a slot of width 0.3 and decide whether it survives a minimum printable feature width of 0.4. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Take an implicit/mesh object to contours and G-code-like paths; test topology/robustness. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement slicing/contouring and offsetting on mesh/SDF inputs; detect self-intersections, tiny islands, and minimum-feature violations before toolpath export. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare mesh/SDF input through slicing/offset/toolpath generation, checking topology, minimum feature size, and manufacturability. Use the same dataset/task, display/hardware and interaction/viewer protocol appropriate to the subtopic; state explicitly whether the fixed budget is pixels/samples, latency, bandwidth, fabrication resolution or participant time. Record topology/contour correctness, self-intersection/feature-size violations, toolpath length/time, and manufacturable-output rate. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Optimize a design near process/manufacturing limits then perturb material/process tolerances; count infeasible, fragile, or functionally degraded outputs. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Recent SIGGRAPH fabrication papers and geometry references. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Computational fabrication. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among mesh/SDF input through slicing/offset/toolpath generation, checking topology, minimum feature size, and manufacturability. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using topology/contour correctness, self-intersection/feature-size violations, toolpath length/time, and manufacturable-output rate. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if mesh/SDF input through slicing/offset/toolpath generation, checking topology, minimum feature size, and manufacturability improve the task-specific tradeoff in topology/contour correctness, self-intersection/feature-size violations, toolpath length/time, and manufacturable-output rate under the same data/display/hardware/viewer protocol; use the metric that reflects the actual visualization/interaction/fabrication/optics task rather than generic image quality.

F. Concrete research project - Computational fabrication

F1. Falsifiable question. Under the controlled failure regime "Optimize a design near process/manufacturing limits then perturb material/process tolerances; count infeasible, fragile, or functionally degraded outputs.", characterize the ranking or regime transition across mesh/SDF input through slicing/offset/toolpath generation, checking topology, minimum feature size, and manufacturability. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement slicing/contouring and offsetting on mesh/SDF inputs; detect self-intersections, tiny islands, and minimum-feature violations before toolpath export. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Define the task-success metric before implementation. Include one analytically/synthetically verifiable case and two realistic cases, then sweep at least one fidelity/latency/geometry parameter. The central adversarial condition is: Optimize a design near process/manufacturing limits then perturb material/process tolerances; count infeasible, fragile, or functionally degraded outputs.

F4. Required evidence. Measure topology/contour correctness, self-intersection/feature-size violations, toolpath length/time, and manufacturable-output rate. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Computational fabrication. If a stable failure/crossover is found among mesh/SDF input through slicing/offset/toolpath generation, checking topology, minimum feature size, and manufacturability, implement one adaptive or corrected variant that directly targets task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Computational fabrication. Using the measured topology/contour correctness, self-intersection/feature-size violations, toolpath length/time, and manufacturable-output rate, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Computational fabrication

- G1. Closed-book derivation/oracle. Derive a geometry-to-manufacturing constraint for a chosen process (minimum feature, overhang, tool reach, slicing), and formulate a design/optimization objective that respects it. Then solve this exact micro-case from scratch without opening your implementation: Offset a unit square outward by nozzle radius 0.2 and compute the new contour. Add a slot of width 0.3 and decide whether it survives a minimum printable feature width of 0.4. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Take an implicit/mesh object to contours and G-code-like paths; test topology/robustness. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally accept non-watertight/self-intersecting geometry and postpone manufacturability checks until after optimization. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement slicing/contouring and offsetting on mesh/SDF inputs; detect self-intersections, tiny islands, and minimum-feature violations before toolpath export. Then apply this stress without retuning: Optimize a design near process/manufacturing limits then perturb material/process tolerances; count infeasible, fragile, or functionally degraded outputs. Report topology/contour correctness, self-intersection/feature-size violations, toolpath length/time, and manufacturable-output rate and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across mesh/SDF input through slicing/offset/toolpath generation, checking topology, minimum feature size, and manufacturability for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured topology/contour correctness, self-intersection/feature-size violations, toolpath length/time, and manufacturable-output rate; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Computational fabrication. Using topology/contour correctness, self-intersection/feature-size violations, toolpath length/time, and manufacturable-output rate, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-

cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 16, Unit 6. Audio and multimodal graphics

Recommended time. 1.7-2.6 weeks (about 26-51 focused hours depending on background).

What you are learning. Sound synthesis/rendering coupled to animation, audiovisual synchronization.

A. Reading order

1. Read the primary assignment first: Read O'Brien et al., Synthesizing Sounds from Physically Based Motion (SIGGRAPH 2001) or a comparable modal-synthesis primary source, then connect modal/eigen analysis back to Vols. 1 and 10.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Animate impact synchronized with synthesized sound.
3. Keep the most useful executable/reference page open while coding: Use SciPy/NumPy for eigen/signal experiments plus the exact modal/physical-audio paper or notes chosen in the volume sequence; do not use a generic graphics-paper index as an implementation manual.

B. Definitions, results and algorithms to know

- Modal vibration intuition
- spatial audio basics
- Precision requirement: distinguish and justify event-triggered heuristic audio vs modal/physically inspired synthesis for impacts or vibration, compared for timing and perceptual plausibility. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a simple geometric-acoustics or synchronized audiovisual event model and formulate cross-modal alignment in time/space; identify latency and calibration variables.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Introduce controlled audio/visual/haptic latency and cue conflicts; measure localization or task error and perceptual fusion. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create physically-inspired sound for rigid impact or vibrating object. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For damped mode $x(t)=e^{-2t}\sin(20t)$, compute amplitude after 0.1,0.5,1 s and oscillation frequency. Align an impact at $t=0.25$ with animation frames at 30 fps and find nearest frame/time error. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Introduce controlled audio/visual/haptic latency and cue conflicts; measure localization or task error and perceptual fusion. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement simple modal synthesis from a small mass-spring/eigenmode model and compare timing/spectrum with heuristic impact samples.
- P5 - Machine-readable evidence. Export timing error, spectral/modal similarity, perceptual preference/plausibility, and synthesis cost to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: event-triggered heuristic audio vs modal/physically inspired synthesis for impacts or vibration, compared for timing and perceptual plausibility. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Introduce controlled audio/visual/haptic latency and cue conflicts; measure localization or task error and perceptual fusion." without hand-editing code between methods. Record timing error, spectral/modal similarity, perceptual preference/plausibility, and synthesis cost.
- P7 - Deliberate bug and regression drill. Intentionally mix sample rates/phase or modal normalizations between simulation and reconstruction. Before running, predict the first observable failure in timing error, spectral/modal similarity, perceptual preference/plausibility, and synthesis cost. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a simple geometric-acoustics or synchronized audiovisual event model and formulate cross-modal alignment in time/space; identify latency and calibration variables. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For damped mode $x(t)=e^{-2t} \sin(20t)$, compute amplitude after 0.1,0.5,1 s and oscillation frequency. Align an impact at $t=0.25$ with animation frames at 30 fps and find nearest frame/time error. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Create physically-inspired sound for rigid impact or vibrating object. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement simple modal synthesis from a small mass-spring/eigenmode model and compare timing/spectrum with heuristic impact samples. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare event-triggered heuristic audio vs modal/physically inspired synthesis for impacts or vibration, compared for timing and perceptual plausibility. Use the same dataset/task, display/hardware and interaction/viewer protocol appropriate to the subtopic; state explicitly whether the fixed budget is pixels/samples, latency, bandwidth, fabrication resolution or participant time. Record timing error, spectral/modal similarity, perceptual preference/plausibility, and synthesis cost. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Introduce controlled audio/visual/haptic latency and cue conflicts; measure localization or task error and perceptual fusion. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: CS348C audio/animation special topics; selected SIGGRAPH papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Audio and multimodal graphics. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among event-triggered heuristic audio vs modal/physically inspired synthesis for impacts or vibration, compared for timing and perceptual plausibility. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using timing error, spectral/modal similarity, perceptual preference/plausibility, and synthesis cost. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if event-triggered heuristic audio vs modal/physically inspired synthesis for impacts or vibration, compared for timing and perceptual plausibility improve the task-specific tradeoff in timing error, spectral/modal similarity, perceptual preference/plausibility, and synthesis cost under the same data/display/hardware/viewer protocol; use the metric that reflects the actual visualization/interaction/fabrication/optics task rather than generic image quality.

F. Concrete research project - Audio and multimodal graphics

- F1. Falsifiable question. Under the controlled failure regime "Introduce controlled audio/visual/haptic latency and cue conflicts; measure localization or task error and perceptual fusion.", which alternative in event-triggered heuristic audio vs modal/physically inspired synthesis for impacts or vibration, compared for timing and perceptual plausibility is preferable, and is the change explained by task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement simple modal synthesis from a small mass-spring/eigenmode model and compare timing/spectrum with heuristic impact samples. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Define the task-success metric before implementation. Include one analytically/synthetically verifiable case and two realistic cases, then sweep at least one fidelity/latency/geometry parameter. The central adversarial condition is: Introduce controlled audio/visual/haptic latency and cue conflicts; measure localization or task error and perceptual fusion.

F4. Required evidence. Measure timing error, spectral/modal similarity, perceptual preference/plausibility, and synthesis cost. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Audio and multimodal graphics. If a stable failure/crossover is found among event-triggered heuristic audio vs modal/physically inspired synthesis for impacts or vibration, compared for timing and perceptual plausibility, implement one adaptive or corrected variant that directly targets task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Audio and multimodal graphics. Using the measured timing error, spectral/modal similarity, perceptual preference/plausibility, and synthesis cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Audio and multimodal graphics

- G1. Closed-book derivation/oracle. Derive a simple geometric-acoustics or synchronized audiovisual event model and formulate cross-modal alignment in time/space; identify latency and calibration variables. Then solve this exact micro-case from scratch without opening your implementation: For damped mode $x(t)=e^{-2t} \sin(20t)$, compute amplitude after 0.1,0.5,1 s and oscillation frequency. Align an impact at $t=0.25$ with animation frames at 30 fps and find nearest frame/time error. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create physically-inspired sound for rigid impact or vibrating object. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally mix sample rates/phase or modal normalizations between simulation and reconstruction. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement simple modal synthesis from a small mass-spring/eigenmode model and compare timing/spectrum with heuristic impact samples. Then apply this stress without retuning: Introduce controlled audio/visual/haptic latency and cue conflicts; measure localization or task error and perceptual fusion. Report timing error, spectral/modal similarity, perceptual preference/plausibility, and synthesis cost and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among event-triggered heuristic audio vs modal/physically inspired synthesis for impacts or vibration, compared for timing and perceptual plausibility for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured timing error, spectral/modal similarity, perceptual preference/plausibility, and synthesis cost; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Audio and multimodal graphics. Using timing error, spectral/modal similarity, perceptual preference/plausibility, and synthesis cost, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 16, Unit 7. Optics/haptics/robotics crossover

Recommended time. 1.7-2.6 weeks (about 26-51 focused hours depending on background).

What you are learning. Graphics techniques used in design, sensing, haptics, robotics and inverse problems.

A. Reading order

1. Read the primary assignment first: Choose one explicit bridge and read one native primary source from that field plus the graphics source being transferred. For differentiable optics use Mitsuba 3; for geometric contact use Geometry Central/libigl/CGAL; for XR integration use OpenXR. Record both sources in the unit log.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Choose one cross-disciplinary paper each quarter.

3. Keep the most useful executable/reference page open while coding: Choose the executable reference to match the bridge: Mitsuba 3 for differentiable optics, Geometry Central/libigl for geometric contact, OpenXR for XR/haptics integration, or the robotics library used by the selected project.

B. Definitions, results and algorithms to know

- Forward model + optimization as recurring pattern
- Precision requirement: distinguish and justify one graphics representation/solver translated into an optics, haptics, or robotics toy problem and compared with that field's native baseline. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- For one crossover system, derive the coupled forward model from geometry/control input to optical, haptic, or robotic response and identify differentiable/optimizable components.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Close a synthetic or small real control/design loop and perturb model parameters; test whether graphics-derived gradients/plans remain useful. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Write a two-page bridge note translating one graphics method into another field you know. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For signed distance $\phi(x,y)=\sqrt{x^2+y^2}-1$, compute its gradient at (2,0) and use it once as an optical surface normal, once as a haptic restoring direction, and once as a robot obstacle-avoidance gradient. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Close a synthetic or small real control/design loop and perturb model parameters; test whether graphics-derived gradients/plans remain useful. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build one executable bridge prototype-for example signed-distance collision cost for robotics, differentiable optics, or haptic proxy contact-and compare with a native baseline.
- P5 - Machine-readable evidence. Export native-task objective/error, graphics-method approximation error, stability/latency, and runtime relative to field-specific baseline to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: one graphics representation/solver translated into an optics, haptics, or robotics toy problem and compared with that field's native baseline. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Close a synthetic or small real control/design loop and perturb model parameters; test whether graphics-derived gradients/plans remain useful." without hand-editing code between methods. Record native-task objective/error, graphics-method approximation error, stability/latency, and runtime relative to field-specific baseline.
- P7 - Deliberate bug and regression drill. Intentionally compare two subsystems with inconsistent coordinate frames/calibration and attribute the discrepancy to the algorithm. Before running, predict the first observable failure in native-task objective/error, graphics-method approximation error, stability/latency, and runtime relative to field-specific baseline. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. For one crossover system, derive the coupled forward model from geometry/control input to optical, haptic, or robotic response and identify differentiable/optimizable components. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For signed distance $\phi(x,y)=\sqrt{x^2+y^2}-1$, compute its gradient at (2,0) and use it once as an optical surface normal, once as a haptic restoring direction, and once as a robot obstacle-avoidance gradient. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Write a two-page bridge note translating one graphics method into another field you know. Before optimizing, make the smallest case agree with Problem 2. Then complete this

deeper task: Build one executable bridge prototype-for example signed-distance collision cost for robotics, differentiable optics, or haptic proxy contact-and compare with a native baseline. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare one graphics representation/solver translated into an optics, haptics, or robotics toy problem and compared with that field's native baseline. Use the same dataset/task, display/hardware and interaction/viewer protocol appropriate to the subtopic; state explicitly whether the fixed budget is pixels/samples, latency, bandwidth, fabrication resolution or participant time. Record native-task objective/error, graphics-method approximation error, stability/latency, and runtime relative to field-specific baseline. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Close a synthetic or small real control/design loop and perturb model parameters; test whether graphics-derived gradients/plans remain useful. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: SIGGRAPH 2026 technical-paper categories and selected papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Optics/haptics/robotics crossover. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among one graphics representation/solver translated into an optics, haptics, or robotics toy problem and compared with that field's native baseline. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using native-task objective/error, graphics-method approximation error, stability/latency, and runtime relative to field-specific baseline. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if one graphics representation/solver translated into an optics, haptics, or robotics toy problem and compared with that field's native baseline improve the task-specific tradeoff in native-task objective/error, graphics-method approximation error, stability/latency, and runtime relative to field-specific baseline under the same data/display/hardware/viewer protocol; use the metric that reflects the actual visualization/interaction/fabrication/optics task rather than generic image quality.

F. Concrete research project - Optics/haptics/robotics crossover

F1. Falsifiable question. Under the controlled failure regime "Close a synthetic or small real control/design loop and perturb model parameters; test whether graphics-derived gradients/plans remain useful.", characterize the ranking or regime transition across one graphics representation/solver translated into an optics, haptics, or robotics toy problem and compared with that field's native baseline. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build one executable bridge prototype-for example signed-distance collision cost for robotics, differentiable optics, or haptic proxy contact-and compare with a native baseline. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Define the task-success metric before implementation. Include one analytically/synthetically verifiable case and two realistic cases, then sweep at least one fidelity/latency/geometry parameter. The central adversarial condition is: Close a synthetic or small real control/design loop and perturb model parameters; test whether graphics-derived gradients/plans remain useful.

F4. Required evidence. Measure native-task objective/error, graphics-method approximation error, stability/latency, and runtime relative to field-specific baseline. Deliver reference outputs plus difference/diagnostic images, `results.csv`, one quality-versus-cost/error curve, and saved configs for nominal and failure cases. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Optics/haptics/robotics crossover. If a stable failure/crossover is found among one graphics representation/solver translated into an optics, haptics, or robotics toy problem and compared with that field's native baseline, implement one adaptive or corrected variant that directly targets task-specific validity, coordinate/calibration consistency, perceptual/physical error, and latency/fidelity tradeoffs; compare it against the strongest fixed baseline

under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Optics/haptics/robotics crossover. Using the measured native-task objective/error, graphics-method approximation error, stability/latency, and runtime relative to field-specific baseline, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Optics/haptics/robotics crossover

- G1. Closed-book derivation/oracle. For one crossover system, derive the coupled forward model from geometry/control input to optical, haptic, or robotic response and identify differentiable/optimizable components. Then solve this exact micro-case from scratch without opening your implementation: For signed distance $\phi(x,y)=\sqrt{x^2+y^2}-1$, compute its gradient at (2,0) and use it once as an optical surface normal, once as a haptic restoring direction, and once as a robot obstacle-avoidance gradient. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Write a two-page bridge note translating one graphics method into another field you know. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally compare two subsystems with inconsistent coordinate frames/calibration and attribute the discrepancy to the algorithm. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Build one executable bridge prototype-for example signed-distance collision cost for robotics, differentiable optics, or haptic proxy contact-and compare with a native baseline. Then apply this stress without retuning: Close a synthetic or small real control/design loop and perturb model parameters; test whether graphics-derived gradients/plans remain useful. Report native-task objective/error, graphics-method approximation error, stability/latency, and runtime relative to field-specific baseline and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across one graphics representation/solver translated into an optics, haptics, or robotics toy problem and compared with that field's native baseline for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured native-task objective/error, graphics-method approximation error, stability/latency, and runtime relative to field-specific baseline; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Optics/haptics/robotics crossover. Using native-task objective/error, graphics-method approximation error, stability/latency, and runtime relative to field-specific baseline, Pass only when the visual/task conclusion agrees with saved numerical/difference diagnostics on both nominal and adversarial cases, and the quality/error-versus-cost curve is reproducible from raw outputs. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 16 Closeout: Project, Review and Mastery Gate

Volume project. Choose two special topics and build one integrated interactive prototype, e.g. visualization + VR, fabrication + geometry, or optics + inverse graphics.

Cumulative review

- Choose two special-topic projects and define their task-specific ground truth/validation before implementation.
- For visualization, perform a small task-accuracy study; for VR/AR measure spatial/latency error; for fabrication test geometric/manufacturing validity.
- Implement one multimodal or cross-domain bridge and compare against a domain-native baseline.
- Document units/calibration/coordinate conventions explicitly wherever physical devices or signals are involved.
- Write a synthesis explaining which general graphics principles transfer and which assumptions break across visualization, XR, fabrication, audio, optics or haptics.

Mastery gate before moving on

- Build an integrated special-topic prototype and report a task/human/system outcome as well as rendering quality.
- Explain the end-to-end error/latency budget and identify the dominant source experimentally.
- Demonstrate a crossover experiment linking at least two subfields such as optics+inverse graphics or geometry+fabrication.

Volume 17. Mathematical Animation and Visual Explanation

Purpose. Learn mathematical animation from scratch as both a hobby and a way to sharpen geometric intuition and communicate research.

Suggested duration. 16-24 weeks, then ongoing

Prerequisites. None beyond willingness to learn basic Python. Run in parallel from the beginning.

Typical module pace. 1.2-1.8 weeks per module (13 modules total).

Primary references for this volume

- Manim Community stable documentation: Quickstart, Tutorials, Guides, Example Gallery and reference manual.
- Primary mathematics/graphics references for the concept being animated.
- Optional video editing/audio tools only after animation logic is solid.

Online spine

Manim Community documentation: <https://docs.manim.community/en/stable/>

Canonical paper track

- No paper sequence is required at the beginning. Instead, study source code and storyboards from high-quality mathematical explainers.
- For every capstone, read and cite the primary mathematical/graphics source being explained.
- Optional research bridge: literature on automated diagram generation, visual explanation and educational visualization.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Manim Community stable tutorial sequence: Quickstart -> output/configuration -> building blocks -> formula/text guide -> reference examples.
- For every mathematical scene, read the primary mathematical source first and create a storyboard stating the invariant or logical step each transition must preserve.
- Study high-quality mathematical explainers for pacing and visual grammar, but independently implement the scene and verify all displayed numerical quantities from code.
- Use the Manim source/reference documentation when behavior is unclear; do not infer object correspondence from visual appearance alone.
- For the capstone, collect timestamped viewer questions/misconceptions and revise at least one scene based on observed comprehension rather than aesthetic preference.

Volume 17, Unit 1. Python and animation workflow from zero

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. Python scripts, functions/classes, environments, command line, project structure, rendering videos.

A. Reading order

1. Read the primary assignment first: Python basics as needed; Manim Community installation and Quickstart.

2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Manim Community Quickstart (current stable docs).
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Scene as program
- deterministic rendering
- asset pipeline
- Precision requirement: distinguish and justify one-off scripts vs reusable Scene/helpers while reproducing the same ten micro-scenes with deterministic output. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Write the coordinate/time/state model of a minimal animation scene, derive deterministic frame generation from parameters, and explain how code organization affects reproducibility and iteration.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Rebuild the same 30-second scene using ad-hoc code and reusable helpers; compare edit time and correctness after three requested changes. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create 10 micro-scenes: text, circle, square-to-circle, axes, plotted function, moving point, vector, matrix, 3D camera, export. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Write the exact scene timeline for a dot moving from $x=-1$ to 1 in 2 s while a numeric label updates. List object state at $t=0,1,2$ and reproduce the same values in a tiny Python function before Manim. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Rebuild the same 30-second scene using ad-hoc code and reusable helpers; compare edit time and correctness after three requested changes. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create a repository with one command that renders all ten micro-scenes at preview/final quality; add a shared style/config module and deterministic asset paths.
- P5 - Machine-readable evidence. Export successful deterministic renders, code reuse, render time, scene count, and zero manual post-fixes for numerical content to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: one-off scripts vs reusable Scene/helpers while reproducing the same ten micro-scenes with deterministic output. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Rebuild the same 30-second scene using ad-hoc code and reusable helpers; compare edit time and correctness after three requested changes." without hand-editing code between methods. Record successful deterministic renders, code reuse, render time, scene count, and zero manual post-fixes for numerical content.
- P7 - Deliberate bug and regression drill. Intentionally store scene state in globals so rendering scenes in a different order changes the result. Before running, predict the first observable failure in successful deterministic renders, code reuse, render time, scene count, and zero manual post-fixes for numerical content. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Write the coordinate/time/state model of a minimal animation scene, derive deterministic frame generation from parameters, and explain how code organization affects reproducibility and iteration. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Write the exact scene timeline for a dot moving from $x=-1$ to 1 in 2 s while a numeric label updates. List object state at $t=0,1,2$ and reproduce the same values in a tiny Python function before Manim. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Create 10 micro-scenes: text, circle, square-to-circle, axes, plotted function, moving point, vector, matrix, 3D camera, export. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create a repository with one command that renders all ten micro-scenes at preview/final quality; add a shared style/config module and deterministic asset paths. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare one-off scripts vs reusable Scene/helpers while reproducing the same ten micro-scenes with deterministic output. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record successful deterministic renders, code reuse, render time, scene count, and zero manual post-fixes for numerical content. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Rebuild the same 30-second scene using ad-hoc code and reusable helpers; compare edit time and correctness after three requested changes. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Python basics as needed; Manim Community installation and Quickstart. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Python and animation workflow from zero. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among one-off scripts vs reusable Scene/helpers while reproducing the same ten micro-scenes with deterministic output. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using successful deterministic renders, code reuse, render time, scene count, and zero manual post-fixes for numerical content. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if one-off scripts vs reusable Scene/helpers while reproducing the same ten micro-scenes with deterministic output improve mathematical correctness, semantic continuity or viewer comprehension measured by successful deterministic renders, code reuse, render time, scene count, and zero manual post-fixes for numerical content on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - Python and animation workflow from zero

- F1. Falsifiable question. Under the controlled failure regime "Rebuild the same 30-second scene using ad-hoc code and reusable helpers; compare edit time and correctness after three requested changes.", which alternative in one-off scripts vs reusable Scene/helpers while reproducing the same ten micro-scenes with deterministic output is preferable, and is the change explained by mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create a repository with one command that renders all ten micro-scenes at preview/final quality; add a shared style/config module and deterministic asset paths. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Rebuild the same 30-second scene using ad-hoc code and reusable helpers; compare edit time and correctness after three requested changes.
- F4. Required evidence. Measure successful deterministic renders, code reuse, render time, scene count, and zero manual post-fixes for numerical content. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Python and animation workflow from zero. If a stable failure/crossover is found among one-off scripts vs reusable Scene/helpers while reproducing the same ten micro-scenes with deterministic output, implement one adaptive or corrected variant that directly targets mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension; compare it against the strongest fixed baseline under the same

protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Python and animation workflow from zero. Using the measured successful deterministic renders, code reuse, render time, scene count, and zero manual post-fixes for numerical content, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Python and animation workflow from zero

- G1. Closed-book derivation/oracle. Write the coordinate/time/state model of a minimal animation scene, derive deterministic frame generation from parameters, and explain how code organization affects reproducibility and iteration. Then solve this exact micro-case from scratch without opening your implementation: Write the exact scene timeline for a dot moving from $x=-1$ to 1 in 2 s while a numeric label updates. List object state at $t=0,1,2$ and reproduce the same values in a tiny Python function before Manim. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create 10 micro-scenes: text, circle, square-to-circle, axes, plotted function, moving point, vector, matrix, 3D camera, export. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally store scene state in globals so rendering scenes in a different order changes the result. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create a repository with one command that renders all ten micro-scenes at preview/final quality; add a shared style/config module and deterministic asset paths. Then apply this stress without retuning: Rebuild the same 30-second scene using ad-hoc code and reusable helpers; compare edit time and correctness after three requested changes. Report successful deterministic renders, code reuse, render time, scene count, and zero manual post-fixes for numerical content and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among one-off scripts vs reusable Scene/helpers while reproducing the same ten micro-scenes with deterministic output for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured successful deterministic renders, code reuse, render time, scene count, and zero manual post-fixes for numerical content; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Python and animation workflow from zero. Using successful deterministic renders, code reuse, render time, scene count, and zero manual post-fixes for numerical content, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 2. Manim object model and transformations

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. Scene, Mobject, VMobject, groups, positioning, transforms, animation builders.

A. Reading order

1. Read the primary assignment first: Manim tutorials: quickstart and building blocks/reference sections.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Official Manim docs/examples.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Coordinate systems
- object state
- interpolation between states

- Precision requirement: distinguish and justify Transform vs ReplacementTransform vs explicit semantic object reuse on a proof diagram with persistent correspondences. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Trace Manim objects through creation, grouping, coordinate transforms, and animation interpolation; derive how a Transform maps source geometry/style to target state over normalized time.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Transform nested/grouped semantic objects through several states; verify symbol/object identity remains obvious and code structure mirrors the mathematical state. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Recreate a clean geometric proof diagram entirely in code; use reusable helper functions. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Create source objects A,B,C where B must persist while A transforms into C. Decide whether Transform, ReplacementTransform, copies, or persistent semantic objects preserve identity; sketch the object graph before coding. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Transform nested/grouped semantic objects through several states; verify symbol/object identity remains obvious and code structure mirrors the mathematical state. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Write helper functions that preserve semantic object identity across transformations and unit-test bounding boxes/anchors after reuse.
- P5 - Machine-readable evidence. Export semantic correspondence errors, object-identity/reuse correctness, layout collisions, code complexity, and viewer mapping accuracy to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: Transform vs ReplacementTransform vs explicit semantic object reuse on a proof diagram with persistent correspondences. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Transform nested/grouped semantic objects through several states; verify symbol/object identity remains obvious and code structure mirrors the mathematical state." without hand-editing code between methods. Record semantic correspondence errors, object-identity/reuse correctness, layout collisions, code complexity, and viewer mapping accuracy.
- P7 - Deliberate bug and regression drill. Intentionally reuse/mutate one Mobject in two semantic roles so a later transform changes both unexpectedly. Before running, predict the first observable failure in semantic correspondence errors, object-identity/reuse correctness, layout collisions, code complexity, and viewer mapping accuracy. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Trace Manim objects through creation, grouping, coordinate transforms, and animation interpolation; derive how a Transform maps source geometry/style to target state over normalized time. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Create source objects A,B,C where B must persist while A transforms into C. Decide whether Transform, ReplacementTransform, copies, or persistent semantic objects preserve identity; sketch the object graph before coding. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Recreate a clean geometric proof diagram entirely in code; use reusable helper functions. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Write helper functions that preserve semantic object identity across transformations and unit-test bounding boxes/anchors after reuse. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare Transform vs ReplacementTransform vs explicit semantic object reuse on a proof diagram with persistent correspondences. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record semantic correspondence errors, object-identity/reuse correctness, layout collisions, code

complexity, and viewer mapping accuracy. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Transform nested/grouped semantic objects through several states; verify symbol/object identity remains obvious and code structure mirrors the mathematical state. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Manim tutorials: quickstart and building blocks/reference sections. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Manim object model and transformations. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among Transform vs ReplacementTransform vs explicit semantic object reuse on a proof diagram with persistent correspondences. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using semantic correspondence errors, object-identity/reuse correctness, layout collisions, code complexity, and viewer mapping accuracy. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if Transform vs ReplacementTransform vs explicit semantic object reuse on a proof diagram with persistent correspondences improve mathematical correctness, semantic continuity or viewer comprehension measured by semantic correspondence errors, object-identity/reuse correctness, layout collisions, code complexity, and viewer mapping accuracy on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - Manim object model and transformations

F1. Falsifiable question. Under the controlled failure regime "Transform nested/grouped semantic objects through several states; verify symbol/object identity remains obvious and code structure mirrors the mathematical state.", which alternative in Transform vs ReplacementTransform vs explicit semantic object reuse on a proof diagram with persistent correspondences is preferable, and is the change explained by mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Write helper functions that preserve semantic object identity across transformations and unit-test bounding boxes/anchors after reuse. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Transform nested/grouped semantic objects through several states; verify symbol/object identity remains obvious and code structure mirrors the mathematical state.

F4. Required evidence. Measure semantic correspondence errors, object-identity/reuse correctness, layout collisions, code complexity, and viewer mapping accuracy. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Manim object model and transformations. If a stable failure/crossover is found among Transform vs ReplacementTransform vs explicit semantic object reuse on a proof diagram with persistent correspondences, implement one adaptive or corrected variant that directly targets mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Manim object model and transformations. Using the measured semantic correspondence errors, object-identity/reuse correctness, layout collisions, code complexity, and viewer mapping accuracy, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Manim object model and transformations

- G1. Closed-book derivation/oracle. Trace Manim objects through creation, grouping, coordinate transforms, and animation interpolation; derive how a Transform maps source geometry/style to target state over normalized time. Then solve this exact micro-case from scratch without opening your implementation: Create source objects A,B,C where B must persist while A transforms into C. Decide whether Transform, ReplacementTransform, copies, or persistent semantic objects preserve identity; sketch the object graph before coding. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Recreate a clean geometric proof diagram entirely in code; use reusable helper functions. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally reuse/mutate one Mobject in two semantic roles so a later transform changes both unexpectedly. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Write helper functions that preserve semantic object identity across transformations and unit-test bounding boxes/anchors after reuse. Then apply this stress without retuning: Transform nested/grouped semantic objects through several states; verify symbol/object identity remains obvious and code structure mirrors the mathematical state. Report semantic correspondence errors, object-identity/reuse correctness, layout collisions, code complexity, and viewer mapping accuracy and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among Transform vs ReplacementTransform vs explicit semantic object reuse on a proof diagram with persistent correspondences for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured semantic correspondence errors, object-identity/reuse correctness, layout collisions, code complexity, and viewer mapping accuracy; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Manim object model and transformations. Using semantic correspondence errors, object-identity/reuse correctness, layout collisions, code complexity, and viewer mapping accuracy, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 3. LaTeX and equation animation

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. MathTex/TeX, token matching, transforms between equations, color/semantic grouping.

A. Reading order

1. Read the primary assignment first: Manim MathTex documentation.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Official examples for TransformMatchingTex.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Visual continuity
- semantic correspondence between algebraic steps
- Precision requirement: distinguish and justify automatic string/submobject matching vs explicit semantic token mapping through reorder/cancel/factor/duplicate operations. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Build a semantic token correspondence for a multi-step derivation and derive transition rules for reordering, cancellation, factoring, duplication, and substitution without relying on raw glyph positions.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Animate an algebraic derivation containing repeated symbols, cancellation, and reordering; ask a

viewer to map every source term to its destination. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Animate a 6-10 step derivation (quadratic formula, Fourier identity, or rendering equation simplification) with stable symbol tracking. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Animate $x+x=2x \rightarrow x=0$ only as an adversarial symbol-tracking example: label the two source x tokens separately, then track cancellation/combination through a corrected multi-step algebraic derivation with repeated symbols. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Animate an algebraic derivation containing repeated symbols, cancellation, and reordering; ask a viewer to map every source term to its destination. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build a semantic-token mapping helper and an adversarial derivation suite containing repeated symbols, cancellation, factorization, and reordering.
- P5 - Machine-readable evidence. Export term-mapping accuracy, ambiguous transitions per derivation, viewer comprehension/recall, edit time, and semantic-token reuse to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: automatic string/subobject matching vs explicit semantic token mapping through reorder/cancel/factor/duplicate operations. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Animate an algebraic derivation containing repeated symbols, cancellation, and reordering; ask a viewer to map every source term to its destination." without hand-editing code between methods. Record term-mapping accuracy, ambiguous transitions per derivation, viewer comprehension/recall, edit time, and semantic-token reuse.
- P7 - Deliberate bug and regression drill. Intentionally match symbols by raw substring/index when repeated variables are reordered or cancelled. Before running, predict the first observable failure in term-mapping accuracy, ambiguous transitions per derivation, viewer comprehension/recall, edit time, and semantic-token reuse. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Build a semantic token correspondence for a multi-step derivation and derive transition rules for reordering, cancellation, factoring, duplication, and substitution without relying on raw glyph positions. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Animate $x+x=2x \rightarrow x=0$ only as an adversarial symbol-tracking example: label the two source x tokens separately, then track cancellation/combination through a corrected multi-step algebraic derivation with repeated symbols. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Animate a 6-10 step derivation (quadratic formula, Fourier identity, or rendering equation simplification) with stable symbol tracking. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build a semantic-token mapping helper and an adversarial derivation suite containing repeated symbols, cancellation, factorization, and reordering. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare automatic string/subobject matching vs explicit semantic token mapping through reorder/cancel/factor/duplicate operations. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record term-mapping accuracy, ambiguous transitions per derivation, viewer comprehension/recall, edit time, and semantic-token reuse. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Animate an algebraic derivation containing repeated symbols, cancellation, and reordering; ask a viewer to map every source term to its destination. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Manim MathTex documentation. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on LaTeX and equation animation. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among automatic string/subobject matching vs explicit semantic token mapping through reorder/cancel/factor/duplicate operations. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using term-mapping accuracy, ambiguous transitions per derivation, viewer comprehension/recall, edit time, and semantic-token reuse. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if explicit semantic mapping reduces term-correspondence errors or viewer misconceptions on multiple derivations; smoother or prettier motion alone is not evidence.

F. Concrete research project - LaTeX and equation animation

F1. Falsifiable question. Under the controlled failure regime "Animate an algebraic derivation containing repeated symbols, cancellation, and reordering; ask a viewer to map every source term to its destination.", which alternative in automatic string/subobject matching vs explicit semantic token mapping through reorder/cancel/factor/duplicate operations is preferable, and is the change explained by semantic token correspondence, repeated-symbol ambiguity, algebraic invariants across transforms, and viewer comprehension? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build a semantic-token mapping helper and an adversarial derivation suite containing repeated symbols, cancellation, factorization, and reordering. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Animate an algebraic derivation containing repeated symbols, cancellation, and reordering; ask a viewer to map every source term to its destination.

F4. Required evidence. Measure term-mapping accuracy, ambiguous transitions per derivation, viewer comprehension/recall, edit time, and semantic-token reuse. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for LaTeX and equation animation. If a stable failure/crossover is found among automatic string/subobject matching vs explicit semantic token mapping through reorder/cancel/factor/duplicate operations, implement one adaptive or corrected variant that directly targets semantic token correspondence, repeated-symbol ambiguity, algebraic invariants across transforms, and viewer comprehension; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for LaTeX and equation animation. Using the measured term-mapping accuracy, ambiguous transitions per derivation, viewer comprehension/recall, edit time, and semantic-token reuse, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for LaTeX and equation animation

- G1. Closed-book derivation/oracle. Build a semantic token correspondence for a multi-step derivation and derive transition rules for reordering, cancellation, factoring, duplication, and substitution without relying on raw glyph positions. Then solve this exact micro-case from scratch without opening your implementation: Animate $x+x=2x \rightarrow x=0$ only as an adversarial symbol-tracking example: label the two source x tokens separately, then track cancellation/combination through a corrected multi-step algebraic derivation with repeated symbols. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Animate a 6-10 step derivation (quadratic formula, Fourier identity, or rendering

equation simplification) with stable symbol tracking. Add the oracle as an automated test and make it pass before importing any optimized/framework code.

- G3. Debugging exam. Intentionally match symbols by raw substring/index when repeated variables are reordered or cancelled. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Build a semantic-token mapping helper and an adversarial derivation suite containing repeated symbols, cancellation, factorization, and reordering. Then apply this stress without retuning: Animate an algebraic derivation containing repeated symbols, cancellation, and reordering; ask a viewer to map every source term to its destination. Report term-mapping accuracy, ambiguous transitions per derivation, viewer comprehension/recall, edit time, and semantic-token reuse and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among automatic string/subobject matching vs explicit semantic token mapping through reorder/cancel/factor/duplicate operations for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured term-mapping accuracy, ambiguous transitions per derivation, viewer comprehension/recall, edit time, and semantic-token reuse; state one counterexample that would reverse your recommendation.
- G6. Pass condition for LaTeX and equation animation. Using term-mapping accuracy, ambiguous transitions per derivation, viewer comprehension/recall, edit time, and semantic-token reuse, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 4. Updaters and dynamic geometry

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. ValueTracker, always_redraw, updaters, traced paths, parameterized scenes.

A. Reading order

1. Read the primary assignment first: Manim updater/reference docs.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Official updater examples.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- State dependent on a parameter
- continuous animation as sampled trajectory
- Precision requirement: distinguish and justify frame-by-frame precomputed animation vs updater/ValueTracker-driven construction on tangent/secant and moving-frame scenes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Express a dynamic construction as state variables plus invariants, derive updater dependencies, and analyze how frame-rate/time parameterization affects numerical and visual stability.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Drive the construction through degenerate/singular parameter values; dependent objects should remain exact or fail explicitly rather than drift visually. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Animate tangent/secant convergence; moving barycentric coordinates; rotating normal/frame. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For curve $y=x^2$ and moving point $x=a$, compute secant slope from a to $a+h$ and tangent slope $2a$ at $a=1$ for $h=1,0.5,0.1$. These values become test oracles for ValueTracker/updater labels. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Drive the construction through degenerate/singular parameter values; dependent objects should remain exact or fail explicitly rather than drift visually. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create three scenes driven by a single ValueTracker and verify displayed numerical labels/geometric constraints agree at sampled parameter values.
- P5 - Machine-readable evidence. Export constraint/numerical display error over sampled parameter values, dropped/slow frames, code complexity, and viewer understanding to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: frame-by-frame precomputed animation vs updater/ValueTracker-driven construction on tangent/secant and moving-frame scenes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Drive the construction through degenerate/singular parameter values; dependent objects should remain exact or fail explicitly rather than drift visually." without hand-editing code between methods. Record constraint/numerical display error over sampled parameter values, dropped/slow frames, code complexity, and viewer understanding.
- P7 - Deliberate bug and regression drill. Intentionally make an updater depend on frame count rather than dt, so the animation changes with render FPS. Before running, predict the first observable failure in constraint/numerical display error over sampled parameter values, dropped/slow frames, code complexity, and viewer understanding. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Express a dynamic construction as state variables plus invariants, derive updater dependencies, and analyze how frame-rate/time parameterization affects numerical and visual stability. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For curve $y=x^2$ and moving point $x=a$, compute secant slope from a to $a+h$ and tangent slope $2a$ at $a=1$ for $h=1,0.5,0.1$. These values become test oracles for ValueTracker/updater labels. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Animate tangent/secant convergence; moving barycentric coordinates; rotating normal/frame. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create three scenes driven by a single ValueTracker and verify displayed numerical labels/geometric constraints agree at sampled parameter values. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare frame-by-frame precomputed animation vs updater/ValueTracker-driven construction on tangent/secant and moving-frame scenes. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record constraint/numerical display error over sampled parameter values, dropped/slow frames, code complexity, and viewer understanding. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Drive the construction through degenerate/singular parameter values; dependent objects should remain exact or fail explicitly rather than drift visually. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Manim updater/reference docs. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Updaters and dynamic geometry. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among frame-by-frame precomputed animation vs updater/ValueTracker-driven construction on tangent/secant and moving-frame scenes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using constraint/numerical display error over sampled parameter values, dropped/slow frames, code complexity, and viewer understanding. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if

frame-by-frame precomputed animation vs updater/ValueTracker-driven construction on tangent/secant and moving-frame scenes improve mathematical correctness, semantic continuity or viewer comprehension measured by constraint/numerical display error over sampled parameter values, dropped/slow frames, code complexity, and viewer understanding on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - Updaters and dynamic geometry

F1. Falsifiable question. Under the controlled failure regime "Drive the construction through degenerate/singular parameter values; dependent objects should remain exact or fail explicitly rather than drift visually.", which alternative in frame-by-frame precomputed animation vs updater/ValueTracker-driven construction on tangent/secant and moving-frame scenes is preferable, and is the change explained by mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create three scenes driven by a single ValueTracker and verify displayed numerical labels/geometric constraints agree at sampled parameter values. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Drive the construction through degenerate/singular parameter values; dependent objects should remain exact or fail explicitly rather than drift visually.

F4. Required evidence. Measure constraint/numerical display error over sampled parameter values, dropped/slow frames, code complexity, and viewer understanding. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Updaters and dynamic geometry. If a stable failure/crossover is found among frame-by-frame precomputed animation vs updater/ValueTracker-driven construction on tangent/secant and moving-frame scenes, implement one adaptive or corrected variant that directly targets mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Updaters and dynamic geometry. Using the measured constraint/numerical display error over sampled parameter values, dropped/slow frames, code complexity, and viewer understanding, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Updaters and dynamic geometry

- G1. Closed-book derivation/oracle. Express a dynamic construction as state variables plus invariants, derive updater dependencies, and analyze how frame-rate/time parameterization affects numerical and visual stability. Then solve this exact micro-case from scratch without opening your implementation: For curve $y=x^2$ and moving point $x=a$, compute secant slope from a to $a+h$ and tangent slope $2a$ at $a=1$ for $h=1,0.5,0.1$. These values become test oracles for ValueTracker/updater labels. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Animate tangent/secant convergence; moving barycentric coordinates; rotating normal/frame. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally make an updater depend on frame count rather than dt , so the animation changes with render FPS. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create three scenes driven by a single ValueTracker and verify displayed numerical labels/geometric constraints agree at sampled parameter values. Then apply this stress without retuning: Drive the construction through degenerate/singular parameter values; dependent objects should remain exact or fail explicitly rather than drift visually. Report

constraint/numerical display error over sampled parameter values, dropped/slow frames, code complexity, and viewer understanding and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method-selection defense. Using your own measured results, choose among frame-by-frame precomputed animation vs updater/ValueTracker-driven construction on tangent/secant and moving-frame scenes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured constraint/numerical display error over sampled parameter values, dropped/slow frames, code complexity, and viewer understanding; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Updaters and dynamic geometry. Using constraint/numerical display error over sampled parameter values, dropped/slow frames, code complexity, and viewer understanding, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 5. 2D geometry storytelling

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. Camera framing, annotations, braces, labels, proof pacing, color discipline.

A. Reading order

1. Read the primary assignment first: Study high-quality mathematical animation as visual reference; use Manim implementation independently.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Storyboard first; then implement.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Cognitive load
- one visual change per logical step
- invariant highlighting
- Precision requirement: distinguish and justify decorative motion vs invariant-preserving geometric transformations on a proof where viewers must identify the invariant. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a storyboard where each geometric transformation preserves or changes one explicit invariant; connect camera framing, annotation, and object persistence to the proof structure.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Create invariant-first and construction-first storyboards for the same proof; test which allows viewers to reconstruct the argument with fewer prompts. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create an animation of Pythagorean theorem, affine transformations, or projective perspective using no unexplained motion. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For a right triangle with legs 3,4,5, choose one invariant-preserving dissection or similarity proof of Pythagoras. List which lengths/areas must remain visually constant through each transformation. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Create invariant-first and construction-first storyboards for the same proof; test which allows viewers to reconstruct the argument with fewer prompts. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Storyboard one proof into invariant-preserving scenes, then implement a debug overlay that can display equal lengths/angles/areas used by the proof.

- P5 - Machine-readable evidence. Export viewer identification of invariant/goal, misconception rate, unnecessary-motion count, and scene duration/edit complexity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: decorative motion vs invariant-preserving geometric transformations on a proof where viewers must identify the invariant. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Create invariant-first and construction-first storyboards for the same proof; test which allows viewers to reconstruct the argument with fewer prompts." without hand-editing code between methods. Record viewer identification of invariant/goal, misconception rate, unnecessary-motion count, and scene duration/edit complexity.
- P7 - Deliberate bug and regression drill. Intentionally mix screen-space labels with world-space geometry so camera/frame changes break the claimed construction. Before running, predict the first observable failure in viewer identification of invariant/goal, misconception rate, unnecessary-motion count, and scene duration/edit complexity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a storyboard where each geometric transformation preserves or changes one explicit invariant; connect camera framing, annotation, and object persistence to the proof structure. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For a right triangle with legs 3,4,5, choose one invariant-preserving dissection or similarity proof of Pythagoras. List which lengths/areas must remain visually constant through each transformation. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Create an animation of Pythagorean theorem, affine transformations, or projective perspective using no unexplained motion. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Storyboard one proof into invariant-preserving scenes, then implement a debug overlay that can display equal lengths/angles/areas used by the proof. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare decorative motion vs invariant-preserving geometric transformations on a proof where viewers must identify the invariant. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record viewer identification of invariant/goal, misconception rate, unnecessary-motion count, and scene duration/edit complexity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Create invariant-first and construction-first storyboards for the same proof; test which allows viewers to reconstruct the argument with fewer prompts. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Study high-quality mathematical animation as visual reference; use Manim implementation independently. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on 2D geometry storytelling. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among decorative motion vs invariant-preserving geometric transformations on a proof where viewers must identify the invariant. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using viewer identification of invariant/goal, misconception rate, unnecessary-motion count, and scene duration/edit complexity. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if decorative motion vs invariant-preserving geometric transformations on a proof where viewers must identify the invariant improve mathematical correctness, semantic continuity or viewer comprehension measured by viewer identification of invariant/goal, misconception rate, unnecessary-motion count, and scene duration/edit complexity on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - 2D geometry storytelling

F1. Falsifiable question. Under the controlled failure regime "Create invariant-first and construction-first storyboards for the same proof; test which allows viewers to reconstruct the argument with fewer prompts.", which alternative in decorative motion vs invariant-preserving geometric transformations on a proof where viewers must identify the invariant is preferable, and is the change explained by mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Storyboard one proof into invariant-preserving scenes, then implement a debug overlay that can display equal lengths/angles/areas used by the proof. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Create invariant-first and construction-first storyboards for the same proof; test which allows viewers to reconstruct the argument with fewer prompts.

F4. Required evidence. Measure viewer identification of invariant/goal, misconception rate, unnecessary-motion count, and scene duration/edit complexity. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for 2D geometry storytelling. If a stable failure/crossover is found among decorative motion vs invariant-preserving geometric transformations on a proof where viewers must identify the invariant, implement one adaptive or corrected variant that directly targets mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for 2D geometry storytelling. Using the measured viewer identification of invariant/goal, misconception rate, unnecessary-motion count, and scene duration/edit complexity, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for 2D geometry storytelling

- G1. Closed-book derivation/oracle. Derive a storyboard where each geometric transformation preserves or changes one explicit invariant; connect camera framing, annotation, and object persistence to the proof structure. Then solve this exact micro-case from scratch without opening your implementation: For a right triangle with legs 3,4,5, choose one invariant-preserving dissection or similarity proof of Pythagoras. List which lengths/areas must remain visually constant through each transformation. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create an animation of Pythagorean theorem, affine transformations, or projective perspective using no unexplained motion. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally mix screen-space labels with world-space geometry so camera/frame changes break the claimed construction. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Storyboard one proof into invariant-preserving scenes, then implement a debug overlay that can display equal lengths/angles/areas used by the proof. Then apply this stress without retuning: Create invariant-first and construction-first storyboards for the same proof; test which allows viewers to reconstruct the argument with fewer prompts. Report viewer identification of invariant/goal, misconception rate, unnecessary-motion count, and scene duration/edit complexity and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among decorative motion vs invariant-preserving geometric transformations on a proof where viewers must identify the invariant for three concrete

regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured viewer identification of invariant/goal, misconception rate, unnecessary-motion count, and scene duration/edit complexity; state one counterexample that would reverse your recommendation.

- G6. Pass condition for 2D geometry storytelling. Using viewer identification of invariant/goal, misconception rate, unnecessary-motion count, and scene duration/edit complexity, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 6. Linear algebra visualizations

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. Basis change, determinant, eigenvectors, SVD, projections, least squares.

A. Reading order

1. Read the primary assignment first: Vol. 1 math + Manim vector/matrix objects.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Build reusable 2D linear-map component.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Area scaling
- invariant eigendirections
- orthogonal decomposition
- Precision requirement: distinguish and justify entrywise matrix animation vs geometric action on basis/unit circle for eigen/SVD interpretations. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive visual constructions for basis change, determinant/area scaling, eigenvectors, and SVD; make explicit which quantities live in domain, codomain, or coordinate representation.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Include negative determinant, repeated eigenvalues, and a nonnormal matrix; document exactly where a 2D geometric metaphor stops being faithful. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Produce a 3-minute SVD animation: rotate -> scale -> rotate, with unit circle to ellipse. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Apply $A = \text{diag}(2, 0.5)$ to e_1, e_2 and eight unit-circle points. Compute determinant, eigenvectors, and ellipse axes so every animated vector/shape has a numerical oracle. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Include negative determinant, repeated eigenvalues, and a nonnormal matrix; document exactly where a 2D geometric metaphor stops being faithful. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create reusable matrix-as-map scenes for determinant, eigenvectors, and SVD, with numerical unit tests comparing displayed vectors to NumPy.
- P5 - Machine-readable evidence. Export displayed-vs-numeric vector/matrix error, viewer explanation accuracy, semantic correspondence, and reuse across concepts to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: entrywise matrix animation vs geometric action on basis/unit circle for eigen/SVD interpretations. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Include negative determinant, repeated eigenvalues, and a nonnormal matrix; document exactly where a 2D geometric metaphor stops being faithful." without hand-editing code between methods. Record displayed-vs-numeric vector/matrix error, viewer explanation accuracy, semantic correspondence, and reuse across concepts.

- P7 - Deliberate bug and regression drill. Intentionally move drawn arrows without applying the same linear map to the underlying coordinates/data. Before running, predict the first observable failure in displayed-vs-numeric vector/matrix error, viewer explanation accuracy, semantic correspondence, and reuse across concepts. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive visual constructions for basis change, determinant/area scaling, eigenvectors, and SVD; make explicit which quantities live in domain, codomain, or coordinate representation. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Apply $A=\text{diag}(2,0.5)$ to e_1, e_2 and eight unit-circle points. Compute determinant, eigenvectors, and ellipse axes so every animated vector/shape has a numerical oracle. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Produce a 3-minute SVD animation: rotate $\rightarrow$ scale $\rightarrow$ rotate, with unit circle to ellipse. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create reusable matrix-as-map scenes for determinant, eigenvectors, and SVD, with numerical unit tests comparing displayed vectors to NumPy. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare entrywise matrix animation vs geometric action on basis/unit circle for eigen/SVD interpretations. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record displayed-vs-numeric vector/matrix error, viewer explanation accuracy, semantic correspondence, and reuse across concepts. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Include negative determinant, repeated eigenvalues, and a nonnormal matrix; document exactly where a 2D geometric metaphor stops being faithful. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Vol. 1 math + Manim vector/matrix objects. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Linear algebra visualizations. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among entrywise matrix animation vs geometric action on basis/unit circle for eigen/SVD interpretations. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using displayed-vs-numeric vector/matrix error, viewer explanation accuracy, semantic correspondence, and reuse across concepts. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if entrywise matrix animation vs geometric action on basis/unit circle for eigen/SVD interpretations improve mathematical correctness, semantic continuity or viewer comprehension measured by displayed-vs-numeric vector/matrix error, viewer explanation accuracy, semantic correspondence, and reuse across concepts on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - Linear algebra visualizations

- F1. Falsifiable question. Under the controlled failure regime "Include negative determinant, repeated eigenvalues, and a nonnormal matrix; document exactly where a 2D geometric metaphor stops being faithful.", which alternative in entrywise matrix animation vs geometric action on basis/unit circle for eigen/SVD interpretations is preferable, and is the change explained by mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create reusable matrix-as-map scenes for determinant, eigenvectors, and SVD, with numerical unit tests comparing displayed vectors to NumPy. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

- F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Include negative determinant, repeated eigenvalues, and a nonnormal matrix; document exactly where a 2D geometric metaphor stops being faithful.
- F4. Required evidence. Measure displayed-vs-numeric vector/matrix error, viewer explanation accuracy, semantic correspondence, and reuse across concepts. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Linear algebra visualizations. If a stable failure/crossover is found among entrywise matrix animation vs geometric action on basis/unit circle for eigen/SVD interpretations, implement one adaptive or corrected variant that directly targets mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Linear algebra visualizations. Using the measured displayed-vs-numeric vector/matrix error, viewer explanation accuracy, semantic correspondence, and reuse across concepts, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Linear algebra visualizations

- G1. Closed-book derivation/oracle. Derive visual constructions for basis change, determinant/area scaling, eigenvectors, and SVD; make explicit which quantities live in domain, codomain, or coordinate representation. Then solve this exact micro-case from scratch without opening your implementation: Apply $A = \text{diag}(2, 0.5)$ to e_1, e_2 and eight unit-circle points. Compute determinant, eigenvectors, and ellipse axes so every animated vector/shape has a numerical oracle. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Produce a 3-minute SVD animation: rotate $\rightarrow$ scale $\rightarrow$ rotate, with unit circle to ellipse. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally move drawn arrows without applying the same linear map to the underlying coordinates/data. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create reusable matrix-as-map scenes for determinant, eigenvectors, and SVD, with numerical unit tests comparing displayed vectors to NumPy. Then apply this stress without retuning: Include negative determinant, repeated eigenvalues, and a nonnormal matrix; document exactly where a 2D geometric metaphor stops being faithful. Report displayed-vs-numeric vector/matrix error, viewer explanation accuracy, semantic correspondence, and reuse across concepts and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among entrywise matrix animation vs geometric action on basis/unit circle for eigen/SVD interpretations for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured displayed-vs-numeric vector/matrix error, viewer explanation accuracy, semantic correspondence, and reuse across concepts; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Linear algebra visualizations. Using displayed-vs-numeric vector/matrix error, viewer explanation accuracy, semantic correspondence, and reuse across concepts, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 7. Calculus and analysis visualizations

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. Limits, derivatives, integrals, Fourier series, convolution, sampling.

A. Reading order

1. Read the primary assignment first: Vol. 1/3 references.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Manim graphing/axes examples.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Geometric meaning of derivative/integral
- frequency decomposition
- Precision requirement: distinguish and justify static plots vs synchronized domain/frequency/differential views for convolution, aliasing, and limiting processes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive an animation mapping for limits, derivatives, integrals, or convergence in which the varying parameter and limiting object are explicit; identify misleading finite-resolution shortcuts.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Animate a limiting process across several scales and explicitly distinguish finite approximations from the limiting theorem/statement. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Animate aliasing/Nyquist and convolution; compare time/frequency views. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Sample $\sin(2\pi 6x)$ at 8 samples per period interval and compute the aliased discrete sequence/frequency. Plan synchronized time-domain and frequency-domain frames that reveal the same numerical fact. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Animate a limiting process across several scales and explicitly distinguish finite approximations from the limiting theorem/statement. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Synchronize spatial and Fourier plots from the same numerical data; add an automated check that plotted samples/filter outputs match the underlying arrays.
- P5 - Machine-readable evidence. Export numerical plot/filter consistency, synchronized-view alignment, viewer identification of limiting/frequency behavior, and animation complexity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: static plots vs synchronized domain/frequency/differential views for convolution, aliasing, and limiting processes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Animate a limiting process across several scales and explicitly distinguish finite approximations from the limiting theorem/statement." without hand-editing code between methods. Record numerical plot/filter consistency, synchronized-view alignment, viewer identification of limiting/frequency behavior, and animation complexity.
- P7 - Deliberate bug and regression drill. Intentionally hard-code displayed derivative/integral values instead of regenerating them from the numerical source. Before running, predict the first observable failure in numerical plot/filter consistency, synchronized-view alignment, viewer identification of limiting/frequency behavior, and animation complexity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive an animation mapping for limits, derivatives, integrals, or convergence in which the varying parameter and limiting object are explicit; identify misleading finite-resolution shortcuts. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Sample $\sin(2\pi 6x)$ at 8 samples per period interval and compute the aliased discrete sequence/frequency. Plan synchronized time-domain and frequency-domain frames that reveal the same numerical fact. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Animate aliasing/Nyquist and convolution; compare time/frequency views. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Synchronize spatial and Fourier plots from the same numerical data; add an automated check that plotted samples/filter outputs match the underlying arrays. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare static plots vs synchronized domain/frequency/differential views for convolution, aliasing, and limiting processes. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record numerical plot/filter consistency, synchronized-view alignment, viewer identification of limiting/frequency behavior, and animation complexity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Animate a limiting process across several scales and explicitly distinguish finite approximations from the limiting theorem/statement. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Vol. 1/3 references. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Calculus and analysis visualizations. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among static plots vs synchronized domain/frequency/differential views for convolution, aliasing, and limiting processes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using numerical plot/filter consistency, synchronized-view alignment, viewer identification of limiting/frequency behavior, and animation complexity. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if static plots vs synchronized domain/frequency/differential views for convolution, aliasing, and limiting processes improve mathematical correctness, semantic continuity or viewer comprehension measured by numerical plot/filter consistency, synchronized-view alignment, viewer identification of limiting/frequency behavior, and animation complexity on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - Calculus and analysis visualizations

- F1. Falsifiable question. Under the controlled failure regime "Animate a limiting process across several scales and explicitly distinguish finite approximations from the limiting theorem/statement.", which alternative in static plots vs synchronized domain/frequency/differential views for convolution, aliasing, and limiting processes is preferable, and is the change explained by conditioning, local linearization, and loss/parameter scaling? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Synchronize spatial and Fourier plots from the same numerical data; add an automated check that plotted samples/filter outputs match the underlying arrays. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Animate a limiting process across several scales and explicitly distinguish finite approximations from the limiting theorem/statement.
- F4. Required evidence. Measure numerical plot/filter consistency, synchronized-view alignment, viewer identification of limiting/frequency behavior, and animation complexity. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Calculus and analysis visualizations. If a stable failure/crossover is found among static plots vs synchronized domain/frequency/differential views for convolution, aliasing, and limiting processes, implement one adaptive or corrected variant that directly targets conditioning, local linearization, and loss/parameter scaling; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Calculus and analysis visualizations. Using the measured numerical plot/filter consistency, synchronized-view alignment, viewer identification of limiting/frequency behavior, and animation complexity, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Calculus and analysis visualizations

- G1. Closed-book derivation/oracle. Derive an animation mapping for limits, derivatives, integrals, or convergence in which the varying parameter and limiting object are explicit; identify misleading finite-resolution shortcuts. Then solve this exact micro-case from scratch without opening your implementation: Sample $\sin(2\pi 6x)$ at 8 samples per period interval and compute the aliased discrete sequence/frequency. Plan synchronized time-domain and frequency-domain frames that reveal the same numerical fact. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Animate aliasing/Nyquist and convolution; compare time/frequency views. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally hard-code displayed derivative/integral values instead of regenerating them from the numerical source. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Synchronize spatial and Fourier plots from the same numerical data; add an automated check that plotted samples/filter outputs match the underlying arrays. Then apply this stress without retuning: Animate a limiting process across several scales and explicitly distinguish finite approximations from the limiting theorem/statement. Report numerical plot/filter consistency, synchronized-view alignment, viewer identification of limiting/frequency behavior, and animation complexity and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among static plots vs synchronized domain/frequency/differential views for convolution, aliasing, and limiting processes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured numerical plot/filter consistency, synchronized-view alignment, viewer identification of limiting/frequency behavior, and animation complexity; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Calculus and analysis visualizations. Using numerical plot/filter consistency, synchronized-view alignment, viewer identification of limiting/frequency behavior, and animation complexity, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 8. Probability and stochastic-process animations

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. Sampling distributions, LLN/CLT intuition, Monte Carlo convergence, random walks.

A. Reading order

1. Read the primary assignment first: Vol. 1 Monte Carlo readings.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Use seeded randomness for reproducibility.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Estimator convergence
- variance
- sample-path view

- Precision requirement: distinguish and justify single random trajectory vs ensemble/distribution/variance-aware visualization for Monte Carlo and random processes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a sampling/animation procedure for a random variable or process, then connect empirical histograms/paths to theoretical distributions, expectation, variance, or convergence.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Show individual sample paths, an ensemble, empirical distribution, and expectation separately; test whether viewers confuse these objects. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Animate Monte Carlo integration with variance bands and importance sampling. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For four coin tosses, enumerate all 16 outcomes and bin the number of heads. Use the exact binomial probabilities as an oracle for an animated Monte Carlo histogram approaching the distribution. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Show individual sample paths, an ensemble, empirical distribution, and expectation separately; test whether viewers confuse these objects. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Precompute seeded samples and confidence bands, then animate convergence without resampling between camera cuts; verify final displayed statistics numerically.
- P5 - Machine-readable evidence. Export displayed statistic error, seed reproducibility, viewer interpretation of uncertainty/variance, and convergence-story clarity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: single random trajectory vs ensemble/distribution/variance-aware visualization for Monte Carlo and random processes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Show individual sample paths, an ensemble, empirical distribution, and expectation separately; test whether viewers confuse these objects." without hand-editing code between methods. Record displayed statistic error, seed reproducibility, viewer interpretation of uncertainty/variance, and convergence-story clarity.
- P7 - Deliberate bug and regression drill. Intentionally reuse an unseeded RNG and visually compare runs with different sample paths as if only one parameter changed. Before running, predict the first observable failure in displayed statistic error, seed reproducibility, viewer interpretation of uncertainty/variance, and convergence-story clarity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a sampling/animation procedure for a random variable or process, then connect empirical histograms/paths to theoretical distributions, expectation, variance, or convergence. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For four coin tosses, enumerate all 16 outcomes and bin the number of heads. Use the exact binomial probabilities as an oracle for an animated Monte Carlo histogram approaching the distribution. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Animate Monte Carlo integration with variance bands and importance sampling. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Precompute seeded samples and confidence bands, then animate convergence without resampling between camera cuts; verify final displayed statistics numerically. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare single random trajectory vs ensemble/distribution/variance-aware visualization for Monte Carlo and random processes. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic

preference alone. Record displayed statistic error, seed reproducibility, viewer interpretation of uncertainty/variance, and convergence-story clarity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Show individual sample paths, an ensemble, empirical distribution, and expectation separately; test whether viewers confuse these objects. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: Vol. 1 Monte Carlo readings. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Probability and stochastic-process animations. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among single random trajectory vs ensemble/distribution/variance-aware visualization for Monte Carlo and random processes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using displayed statistic error, seed reproducibility, viewer interpretation of uncertainty/variance, and convergence-story clarity. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if single random trajectory vs ensemble/distribution/variance-aware visualization for Monte Carlo and random processes improve mathematical correctness, semantic continuity or viewer comprehension measured by displayed statistic error, seed reproducibility, viewer interpretation of uncertainty/variance, and convergence-story clarity on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - Probability and stochastic-process animations

F1. Falsifiable question. Under the controlled failure regime "Show individual sample paths, an ensemble, empirical distribution, and expectation separately; test whether viewers confuse these objects.", which alternative in single random trajectory vs ensemble/distribution/variance-aware visualization for Monte Carlo and random processes is preferable, and is the change explained by mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Precompute seeded samples and confidence bands, then animate convergence without resampling between camera cuts; verify final displayed statistics numerically. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Show individual sample paths, an ensemble, empirical distribution, and expectation separately; test whether viewers confuse these objects.

F4. Required evidence. Measure displayed statistic error, seed reproducibility, viewer interpretation of uncertainty/variance, and convergence-story clarity. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Probability and stochastic-process animations. If a stable failure/crossover is found among single random trajectory vs ensemble/distribution/variance-aware visualization for Monte Carlo and random processes, implement one adaptive or corrected variant that directly targets mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Probability and stochastic-process animations. Using the measured displayed statistic error, seed reproducibility, viewer interpretation of uncertainty/variance, and convergence-story clarity, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Probability and stochastic-process animations

- G1. Closed-book derivation/oracle. Derive a sampling/animation procedure for a random variable or process, then connect empirical histograms/paths to theoretical distributions, expectation, variance, or convergence. Then solve this exact micro-case from scratch without opening your implementation: For four coin tosses, enumerate all 16 outcomes and bin the number of heads. Use the exact binomial probabilities as an oracle for an animated Monte Carlo histogram approaching the distribution. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Animate Monte Carlo integration with variance bands and importance sampling. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally reuse an unseeded RNG and visually compare runs with different sample paths as if only one parameter changed. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Precompute seeded samples and confidence bands, then animate convergence without resampling between camera cuts; verify final displayed statistics numerically. Then apply this stress without retuning: Show individual sample paths, an ensemble, empirical distribution, and expectation separately; test whether viewers confuse these objects. Report displayed statistic error, seed reproducibility, viewer interpretation of uncertainty/variance, and convergence-story clarity and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among single random trajectory vs ensemble/distribution/variance-aware visualization for Monte Carlo and random processes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured displayed statistic error, seed reproducibility, viewer interpretation of uncertainty/variance, and convergence-story clarity; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Probability and stochastic-process animations. Using displayed statistic error, seed reproducibility, viewer interpretation of uncertainty/variance, and convergence-story clarity, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 9. 3D mathematical animation

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. ThreeDScene, camera orientation, surfaces, vector fields, moving frames.

A. Reading order

1. Read the primary assignment first: Manim 3D examples/reference.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Official 3D scene examples.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Projection and occlusion as explanatory constraints
- Precision requirement: distinguish and justify fixed camera vs intentionally planned camera motion and depth cues on tangent-plane/geodesic/projection scenes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive camera, projection, object-frame, and surface-parameter relationships for one 3D mathematical construction; explain occlusion/depth cues needed to preserve mathematical meaning.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Rotate and section a 3D construction while checking that perspective/occlusion never implies false incidence, orthogonality, or equal length. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Animate sphere tangent plane/normal, torus geodesic intuition, or camera projection frustum. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For unit sphere point $p=(0,0,1)$, write the tangent plane and two tangent basis vectors. Rotate camera around the sphere and specify which geometric quantities must remain invariant despite perspective. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Rotate and section a 3D construction while checking that perspective/occlusion never implies false incidence, orthogonality, or equal length. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create camera-path helpers and depth/reference-grid overlays; test the same 3D idea with two camera paths and evaluate which preserves spatial relations.
- P5 - Machine-readable evidence. Export projected/geometric relation correctness, camera occlusion/depth ambiguity, viewer spatial-relation accuracy, and render time to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: fixed camera vs intentionally planned camera motion and depth cues on tangent-plane/geodesic/projection scenes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Rotate and section a 3D construction while checking that perspective/occlusion never implies false incidence, orthogonality, or equal length." without hand-editing code between methods. Record projected/geometric relation correctness, camera occlusion/depth ambiguity, viewer spatial-relation accuracy, and render time.
- P7 - Deliberate bug and regression drill. Intentionally change camera orientation while judging a geometric invariant from a 2D projection. Before running, predict the first observable failure in projected/geometric relation correctness, camera occlusion/depth ambiguity, viewer spatial-relation accuracy, and render time. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive camera, projection, object-frame, and surface-parameter relationships for one 3D mathematical construction; explain occlusion/depth cues needed to preserve mathematical meaning. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For unit sphere point $p=(0,0,1)$, write the tangent plane and two tangent basis vectors. Rotate camera around the sphere and specify which geometric quantities must remain invariant despite perspective. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Animate sphere tangent plane/normal, torus geodesic intuition, or camera projection frustum. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create camera-path helpers and depth/reference-grid overlays; test the same 3D idea with two camera paths and evaluate which preserves spatial relations. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare fixed camera vs intentionally planned camera motion and depth cues on tangent-plane/geodesic/projection scenes. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record projected/geometric relation correctness, camera occlusion/depth ambiguity, viewer spatial-relation accuracy, and render time. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Rotate and section a 3D construction while checking that perspective/occlusion never implies false incidence, orthogonality, or equal length. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Manim 3D examples/reference. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on 3D mathematical animation. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among fixed camera vs intentionally planned camera motion and depth cues on tangent-plane/geodesic/projection scenes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using projected/geometric relation correctness, camera occlusion/depth ambiguity, viewer spatial-relation accuracy, and render time. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if fixed camera vs intentionally planned camera motion and depth cues on tangent-plane/geodesic/projection scenes improve mathematical correctness, semantic continuity or viewer comprehension measured by projected/geometric relation correctness, camera occlusion/depth ambiguity, viewer spatial-relation accuracy, and render time on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - 3D mathematical animation

F1. Falsifiable question. Under the controlled failure regime "Rotate and section a 3D construction while checking that perspective/occlusion never implies false incidence, orthogonality, or equal length.", which alternative in fixed camera vs intentionally planned camera motion and depth cues on tangent-plane/geodesic/projection scenes is preferable, and is the change explained by mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create camera-path helpers and depth/reference-grid overlays; test the same 3D idea with two camera paths and evaluate which preserves spatial relations. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Rotate and section a 3D construction while checking that perspective/occlusion never implies false incidence, orthogonality, or equal length.

F4. Required evidence. Measure projected/geometric relation correctness, camera occlusion/depth ambiguity, viewer spatial-relation accuracy, and render time. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for 3D mathematical animation. If a stable failure/crossover is found among fixed camera vs intentionally planned camera motion and depth cues on tangent-plane/geodesic/projection scenes, implement one adaptive or corrected variant that directly targets mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for 3D mathematical animation. Using the measured projected/geometric relation correctness, camera occlusion/depth ambiguity, viewer spatial-relation accuracy, and render time, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for 3D mathematical animation

- G1. Closed-book derivation/oracle. Derive camera, projection, object-frame, and surface-parameter relationships for one 3D mathematical construction; explain occlusion/depth cues needed to preserve mathematical meaning. Then solve this exact micro-case from scratch without opening your implementation: For unit sphere point $p=(0,0,1)$, write the tangent plane and two tangent basis vectors. Rotate camera around the sphere and specify which geometric quantities must remain invariant despite perspective. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Animate sphere tangent plane/normal, torus geodesic intuition, or camera projection frustum. Add the oracle as an automated test and make it pass before importing any optimized/framework code.

- G3. Debugging exam. Intentionally change camera orientation while judging a geometric invariant from a 2D projection. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create camera-path helpers and depth/reference-grid overlays; test the same 3D idea with two camera paths and evaluate which preserves spatial relations. Then apply this stress without retuning: Rotate and section a 3D construction while checking that perspective/occlusion never implies false incidence, orthogonality, or equal length. Report projected/geometric relation correctness, camera occlusion/depth ambiguity, viewer spatial-relation accuracy, and render time and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among fixed camera vs intentionally planned camera motion and depth cues on tangent-plane/geodesic/projection scenes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured projected/geometric relation correctness, camera occlusion/depth ambiguity, viewer spatial-relation accuracy, and render time; state one counterexample that would reverse your recommendation.
- G6. Pass condition for 3D mathematical animation. Using projected/geometric relation correctness, camera occlusion/depth ambiguity, viewer spatial-relation accuracy, and render time, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 10. Graphics concepts as animation subjects

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. Turn the graphics curriculum into visual demos: rasterization, BRDFs, path tracing, DDG, fluids.

A. Reading order

1. Read the primary assignment first: Cross-reference Vol. 3-14.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Use each major volume to generate one visual explainer.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Animation as diagnostic model of an algorithm
- Precision requirement: distinguish and justify algorithm trace alone vs simultaneous geometric, algebraic, and diagnostic-state views for rasterization/BVH/path tracing/Laplacians/pressure projection. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- For one graphics algorithm (sampling, rasterization, ray tracing, BRDF, path tracing, mesh Laplacian), derive the computational state transitions that should become visible objects in an explainer.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Expose the algorithm/data state at every step; remove any motion that cannot be mapped to a state update, estimator contribution, or invariant. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create: barycentric rasterization; ray-BVH traversal; Monte Carlo path tree; cotan Laplacian. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For one triangle on a 3x3 sample grid, compute edge functions, covered pixels, and barycentrics for one pixel. Design three synchronized panels - geometry, algebra, and algorithm state - whose displayed numbers match this oracle. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Expose the algorithm/data state at every step; remove any motion that cannot be mapped to a state update, estimator contribution, or invariant. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Instrument a real toy algorithm (rasterizer/BVH/path tracer/Laplacian/pressure solve) to export state frames that Manim consumes, rather than manually faking algorithm state.
- P5 - Machine-readable evidence. Export state-trace fidelity to executable algorithm, viewer ability to predict next step, diagnostic-view usefulness, and scene complexity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: algorithm trace alone vs simultaneous geometric, algebraic, and diagnostic-state views for rasterization/BVH/path tracing/Laplacians/pressure projection. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Expose the algorithm/data state at every step; remove any motion that cannot be mapped to a state update, estimator contribution, or invariant." without hand-editing code between methods. Record state-trace fidelity to executable algorithm, viewer ability to predict next step, diagnostic-view usefulness, and scene complexity.
- P7 - Deliberate bug and regression drill. Intentionally animate an approximate algorithm that disagrees with the reference implementation used elsewhere in the curriculum. Before running, predict the first observable failure in state-trace fidelity to executable algorithm, viewer ability to predict next step, diagnostic-view usefulness, and scene complexity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. For one graphics algorithm (sampling, rasterization, ray tracing, BRDF, path tracing, mesh Laplacian), derive the computational state transitions that should become visible objects in an explainer. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For one triangle on a 3x3 sample grid, compute edge functions, covered pixels, and barycentrics for one pixel. Design three synchronized panels - geometry, algebra, and algorithm state - whose displayed numbers match this oracle. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Create: barycentric rasterization; ray-BVH traversal; Monte Carlo path tree. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Instrument a real toy algorithm (rasterizer/BVH/path tracer/Laplacian/pressure solve) to export state frames that Manim consumes, rather than manually faking algorithm state. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare algorithm trace alone vs simultaneous geometric, algebraic, and diagnostic-state views for rasterization/BVH/path tracing/Laplacians/pressure projection. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record state-trace fidelity to executable algorithm, viewer ability to predict next step, diagnostic-view usefulness, and scene complexity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Expose the algorithm/data state at every step; remove any motion that cannot be mapped to a state update, estimator contribution, or invariant. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Cross-reference Vol. 3-14. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Graphics concepts as animation subjects. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among algorithm trace alone vs simultaneous geometric, algebraic, and diagnostic-state views for rasterization/BVH/path tracing/Laplacians/pressure projection. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using state-trace fidelity to executable algorithm, viewer ability to predict next step, diagnostic-view usefulness, and scene complexity. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines,

uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if algorithm trace alone vs simultaneous geometric, algebraic, and diagnostic-state views for rasterization/BVH/path tracing/Laplacians/pressure projection improve mathematical correctness, semantic continuity or viewer comprehension measured by state-trace fidelity to executable algorithm, viewer ability to predict next step, diagnostic-view usefulness, and scene complexity on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - Graphics concepts as animation subjects

F1. Falsifiable question. Under the controlled failure regime "Expose the algorithm/data state at every step; remove any motion that cannot be mapped to a state update, estimator contribution, or invariant.", which alternative in algorithm trace alone vs simultaneous geometric, algebraic, and diagnostic-state views for rasterization/BVH/path tracing/Laplacians/pressure projection is preferable, and is the change explained by mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Instrument a real toy algorithm (rasterizer/BVH/path tracer/Laplacian/pressure solve) to export state frames that Manim consumes, rather than manually faking algorithm state. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Expose the algorithm/data state at every step; remove any motion that cannot be mapped to a state update, estimator contribution, or invariant.

F4. Required evidence. Measure state-trace fidelity to executable algorithm, viewer ability to predict next step, diagnostic-view usefulness, and scene complexity. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Graphics concepts as animation subjects. If a stable failure/crossover is found among algorithm trace alone vs simultaneous geometric, algebraic, and diagnostic-state views for rasterization/BVH/path tracing/Laplacians/pressure projection, implement one adaptive or corrected variant that directly targets mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Graphics concepts as animation subjects. Using the measured state-trace fidelity to executable algorithm, viewer ability to predict next step, diagnostic-view usefulness, and scene complexity, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Graphics concepts as animation subjects

- G1. Closed-book derivation/oracle. For one graphics algorithm (sampling, rasterization, ray tracing, BRDF, path tracing, mesh Laplacian), derive the computational state transitions that should become visible objects in an explainer. Then solve this exact micro-case from scratch without opening your implementation: For one triangle on a 3x3 sample grid, compute edge functions, covered pixels, and barycentrics for one pixel. Design three synchronized panels - geometry, algebra, and algorithm state - whose displayed numbers match this oracle. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create: barycentric rasterization; ray-BVH traversal; Monte Carlo path tree; cotan Laplacian. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally animate an approximate algorithm that disagrees with the reference implementation used elsewhere in the curriculum. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Instrument a real toy algorithm (rasterizer/BVH/path tracer/Laplacian/pressure solve) to export state frames that Manim consumes, rather than manually faking algorithm state. Then apply this stress without retuning: Expose the algorithm/data

state at every step; remove any motion that cannot be mapped to a state update, estimator contribution, or invariant. Report state-trace fidelity to executable algorithm, viewer ability to predict next step, diagnostic-view usefulness, and scene complexity and explain which invariant should remain stable and which quantity is allowed/expected to change.

- G5. Method-selection defense. Using your own measured results, choose among algorithm trace alone vs simultaneous geometric, algebraic, and diagnostic-state views for rasterization/BVH/path tracing/Laplacians/pressure projection for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured state-trace fidelity to executable algorithm, viewer ability to predict next step, diagnostic-view usefulness, and scene complexity; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Graphics concepts as animation subjects. Using state-trace fidelity to executable algorithm, viewer ability to predict next step, diagnostic-view usefulness, and scene complexity, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 11. Narrative, voice, editing, and sound

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. Storyboard, narration, pacing, cuts, typography, sound/music rights, captions.

A. Reading order

1. Read the primary assignment first: Manim voiceover ecosystem if desired; video editor basics.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Record rough voice first for timing.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Narrative arc: question -> model -> transformation -> insight -> recap
- Precision requirement: distinguish and justify visual-only, narrated, and captioned cuts of the same explanation, comparing comprehension and pacing. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a timing map from mathematical argument to narration beats, visual events, pauses, and emphasis; identify points where narration and visuals compete rather than reinforce each other.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Produce two narration/pacing versions of the same technical segment; test immediate and delayed recall plus timestamped confusion. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Produce a 3-5 minute narrated animation and a silent captioned version. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Take a 30-second explanation with 75 spoken words. Mark exact timestamps for premise, visual transition, key equation, pause, and conclusion; compute speaking rate and design one comprehension question tied to the key equation. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Produce two narration/pacing versions of the same technical segment; test immediate and delayed recall plus timestamped confusion. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Generate timecoded narration/captions from one script and produce two cuts with different pause/transition timing; collect timestamped comprehension feedback.
- P5 - Machine-readable evidence. Export comprehension/recall, timestamped confusion events, words/minute and pause timing, caption coverage, and revision count to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.

- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: visual-only, narrated, and captioned cuts of the same explanation, comparing comprehension and pacing. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Produce two narration/pacing versions of the same technical segment; test immediate and delayed recall plus timestamped confusion." without hand-editing code between methods. Record comprehension/recall, timestamped confusion events, words/minute and pause timing, caption coverage, and revision count.
- P7 - Deliberate bug and regression drill. Intentionally let narration/captions drift from the mathematical transition they explain. Before running, predict the first observable failure in comprehension/recall, timestamped confusion events, words/minute and pause timing, caption coverage, and revision count. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a timing map from mathematical argument to narration beats, visual events, pauses, and emphasis; identify points where narration and visuals compete rather than reinforce each other. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Take a 30-second explanation with 75 spoken words. Mark exact timestamps for premise, visual transition, key equation, pause, and conclusion; compute speaking rate and design one comprehension question tied to the key equation. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Produce a 3-5 minute narrated animation and a silent captioned version. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Generate timecoded narration/captions from one script and produce two cuts with different pause/transition timing; collect timestamped comprehension feedback. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare visual-only, narrated, and captioned cuts of the same explanation, comparing comprehension and pacing. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record comprehension/recall, timestamped confusion events, words/minute and pause timing, caption coverage, and revision count. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Produce two narration/pacing versions of the same technical segment; test immediate and delayed recall plus timestamped confusion. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Manim voiceover ecosystem if desired; video editor basics. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Narrative, voice, editing, and sound. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among visual-only, narrated, and captioned cuts of the same explanation, comparing comprehension and pacing. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using comprehension/recall, timestamped confusion events, words/minute and pause timing, caption coverage, and revision count. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if visual-only, narrated, and captioned cuts of the same explanation, comparing comprehension and pacing improve mathematical correctness, semantic continuity or viewer comprehension measured by comprehension/recall, timestamped confusion events, words/minute and pause timing, caption coverage, and revision count on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - Narrative, voice, editing, and sound

- F1. Falsifiable question. Under the controlled failure regime "Produce two narration/pacing versions of the same technical segment; test immediate and delayed recall plus timestamped confusion.", characterize the ranking or regime

transition across visual-only, narrated, and captioned cuts of the same explanation, comparing comprehension and pacing. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension. Name at least one observation that would falsify your prediction.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Generate timecoded narration/captions from one script and produce two cuts with different pause/transition timing; collect timestamped comprehension feedback. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Produce two narration/pacing versions of the same technical segment; test immediate and delayed recall plus timestamped confusion.

F4. Required evidence. Measure comprehension/recall, timestamped confusion events, words/minute and pause timing, caption coverage, and revision count. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Narrative, voice, editing, and sound. If a stable failure/crossover is found among visual-only, narrated, and captioned cuts of the same explanation, comparing comprehension and pacing, implement one adaptive or corrected variant that directly targets mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Narrative, voice, editing, and sound. Using the measured comprehension/recall, timestamped confusion events, words/minute and pause timing, caption coverage, and revision count, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Narrative, voice, editing, and sound

- G1. Closed-book derivation/oracle. Derive a timing map from mathematical argument to narration beats, visual events, pauses, and emphasis; identify points where narration and visuals compete rather than reinforce each other. Then solve this exact micro-case from scratch without opening your implementation: Take a 30-second explanation with 75 spoken words. Mark exact timestamps for premise, visual transition, key equation, pause, and conclusion; compute speaking rate and design one comprehension question tied to the key equation. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Produce a 3-5 minute narrated animation and a silent captioned version. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally let narration/captions drift from the mathematical transition they explain. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Generate timecoded narration/captions from one script and produce two cuts with different pause/transition timing; collect timestamped comprehension feedback. Then apply this stress without retuning: Produce two narration/pacing versions of the same technical segment; test immediate and delayed recall plus timestamped confusion. Report comprehension/recall, timestamped confusion events, words/minute and pause timing, caption coverage, and revision count and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across visual-only, narrated, and captioned cuts of the same explanation, comparing comprehension and pacing for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured comprehension/recall, timestamped confusion events, words/minute and pause timing, caption coverage, and revision count; state one counterexample that would change the diagnosis or ranking.

- G6. Pass condition for Narrative, voice, editing, and sound. Using comprehension/recall, timestamped confusion events, words/minute and pause timing, caption coverage, and revision count, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 12. Reusable animation library

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. Custom Mobjects, themes, helpers, test renders, templates.

A. Reading order

1. Read the primary assignment first: Manim reference docs and source examples.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Build personal library gradually.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Abstraction without hiding mathematics
- stable APIs for visuals
- Precision requirement: distinguish and justify copy-pasted scene code vs parameterized semantic components tested across three unrelated mathematical topics. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Define abstractions and invariants for reusable axes, vectors, matrices, diagrams, highlights, and transitions; derive an API boundary that separates mathematical semantics from styling.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Apply each proposed abstraction to three unrelated mathematical topics; reject components that require topic-specific hacks or hide semantics. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create reusable components for coordinate frames, matrices, rays, meshes, probability plots, and theorem callouts. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Specify an API for AxesWithTrackedPoint or SemanticEquation that must serve three scenes (calculus, linear algebra, graphics). List parameters, invariants, callbacks, and one regression frame for each use. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Apply each proposed abstraction to three unrelated mathematical topics; reject components that require topic-specific hacks or hide semantics. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Package coordinate-frame, ray, mesh, matrix, probability, and theorem-callout components with examples and minimal API documentation.
- P5 - Machine-readable evidence. Export API reuse count, duplicated lines removed, regression-render success, integration time on new scenes, and documentation completeness to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: copy-pasted scene code vs parameterized semantic components tested across three unrelated mathematical topics. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Apply each proposed abstraction to three unrelated mathematical topics; reject components that require topic-specific hacks or hide semantics." without hand-editing code between methods. Record API reuse count, duplicated lines removed, regression-render success, integration time on new scenes, and documentation completeness.
- P7 - Deliberate bug and regression drill. Intentionally hide mutable global styling/state inside helper functions so scenes are not composable/reproducible. Before running, predict the first observable failure in API reuse count, duplicated lines removed, regression-render success, integration time on new scenes, and documentation

completeness. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Define abstractions and invariants for reusable axes, vectors, matrices, diagrams, highlights, and transitions; derive an API boundary that separates mathematical semantics from styling. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Specify an API for AxesWithTrackedPoint or SemanticEquation that must serve three scenes (calculus, linear algebra, graphics). List parameters, invariants, callbacks, and one regression frame for each use. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Create reusable components for coordinate frames, matrices, rays, meshes, probability plots, and theorem callouts. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Package coordinate-frame, ray, mesh, matrix, probability, and theorem-callout components with examples and minimal API documentation. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare copy-pasted scene code vs parameterized semantic components tested across three unrelated mathematical topics. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record API reuse count, duplicated lines removed, regression-render success, integration time on new scenes, and documentation completeness. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Apply each proposed abstraction to three unrelated mathematical topics; reject components that require topic-specific hacks or hide semantics. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Manim reference docs and source examples. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Reusable animation library. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among copy-pasted scene code vs parameterized semantic components tested across three unrelated mathematical topics. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using API reuse count, duplicated lines removed, regression-render success, integration time on new scenes, and documentation completeness. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if copy-pasted scene code vs parameterized semantic components tested across three unrelated mathematical topics improve mathematical correctness, semantic continuity or viewer comprehension measured by API reuse count, duplicated lines removed, regression-render success, integration time on new scenes, and documentation completeness on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - Reusable animation library

- F1. Falsifiable question. Under the controlled failure regime "Apply each proposed abstraction to three unrelated mathematical topics; reject components that require topic-specific hacks or hide semantics.", which alternative in copy-pasted scene code vs parameterized semantic components tested across three unrelated mathematical topics is preferable, and is the change explained by mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Package coordinate-frame, ray, mesh, matrix, probability, and theorem-callout components with examples and minimal API documentation. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Apply each proposed abstraction to three unrelated mathematical topics; reject components that require topic-specific hacks or hide semantics.

F4. Required evidence. Measure API reuse count, duplicated lines removed, regression-render success, integration time on new scenes, and documentation completeness. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Reusable animation library. If a stable failure/crossover is found among copy-pasted scene code vs parameterized semantic components tested across three unrelated mathematical topics, implement one adaptive or corrected variant that directly targets mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Reusable animation library. Using the measured API reuse count, duplicated lines removed, regression-render success, integration time on new scenes, and documentation completeness, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Reusable animation library

- G1. Closed-book derivation/oracle. Define abstractions and invariants for reusable axes, vectors, matrices, diagrams, highlights, and transitions; derive an API boundary that separates mathematical semantics from styling. Then solve this exact micro-case from scratch without opening your implementation: Specify an API for AxesWithTrackedPoint or SemanticEquation that must serve three scenes (calculus, linear algebra, graphics). List parameters, invariants, callbacks, and one regression frame for each use. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create reusable components for coordinate frames, matrices, rays, meshes, probability plots, and theorem callouts. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally hide mutable global styling/state inside helper functions so scenes are not composable/reproducible. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Package coordinate-frame, ray, mesh, matrix, probability, and theorem-callout components with examples and minimal API documentation. Then apply this stress without retuning: Apply each proposed abstraction to three unrelated mathematical topics; reject components that require topic-specific hacks or hide semantics. Report API reuse count, duplicated lines removed, regression-render success, integration time on new scenes, and documentation completeness and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among copy-pasted scene code vs parameterized semantic components tested across three unrelated mathematical topics for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured API reuse count, duplicated lines removed, regression-render success, integration time on new scenes, and documentation completeness; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Reusable animation library. Using API reuse count, duplicated lines removed, regression-render success, integration time on new scenes, and documentation completeness, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17, Unit 13. Capstone: research-quality mathematical explainer

Recommended time. 1.2-1.8 weeks (about 18-37 focused hours depending on background).

What you are learning. Choose a theorem/algorithm from graphics or mathematics and explain it rigorously to an advanced audience.

A. Reading order

1. Read the primary assignment first: primary sources for chosen topic.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Iterate storyboard -> animatic -> final render.
3. Keep the most useful executable/reference page open while coding: Manim Community stable tutorials - <https://docs.manim.community/en/stable/tutorials/index.html>

B. Definitions, results and algorithms to know

- Accuracy, visual invariants, pacing, citations, reproducibility
- Precision requirement: distinguish and justify first-cut storyboard vs viewer-tested revision, with misconceptions traced to exact timestamps and scene design choices. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Reverse-engineer the chosen theorem/algorithm into prerequisite claims, visual lemmas, transitions, and a final synthesis; derive a storyboard whose dependency graph mirrors the mathematical argument.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Pilot a rough cut with viewers before polishing; log misconceptions by timestamp and revise the conceptual storyboard, not just aesthetics. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Produce 6-10 minute video plus source repository and one-page design retrospective. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Storyboard exactly three consecutive scenes of the capstone. For each, write the mathematical claim, visual invariant, narration sentence, source/code oracle, transition trigger, and one likely viewer misconception. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Pilot a rough cut with viewers before polishing; log misconceptions by timestamp and revise the conceptual storyboard, not just aesthetics. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Publish source, storyboard, references, asset-generation scripts, and a viewer-test log; ensure every numerical animation is generated from executable mathematics.
- P5 - Machine-readable evidence. Export viewer pre/post comprehension, misconception count by timestamp, factual/numerical correctness, source reproducibility, and completion/retention to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: first-cut storyboard vs viewer-tested revision, with misconceptions traced to exact timestamps and scene design choices. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Pilot a rough cut with viewers before polishing; log misconceptions by timestamp and revise the conceptual storyboard, not just aesthetics." without hand-editing code between methods. Record viewer pre/post comprehension, misconception count by timestamp, factual/numerical correctness, source reproducibility, and completion/retention.
- P7 - Deliberate bug and regression drill. Intentionally round or simplify displayed values in a way that changes the mathematical claim without flagging the approximation. Before running, predict the first observable failure in viewer pre/post comprehension, misconception count by timestamp, factual/numerical correctness, source reproducibility, and completion/retention. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Reverse-engineer the chosen theorem/algorithm into prerequisite claims, visual lemmas, transitions, and a final synthesis; derive a storyboard whose dependency graph mirrors the mathematical argument. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. Storyboard exactly three consecutive scenes of the capstone. For each, write the mathematical claim, visual invariant, narration sentence, source/code oracle, transition trigger, and one likely viewer misconception. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Produce 6-10 minute video plus source repository and one-page design retrospective. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Publish source, storyboard, references, asset-generation scripts, and a viewer-test log; ensure every numerical animation is generated from executable mathematics. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare first-cut storyboard vs viewer-tested revision, with misconceptions traced to exact timestamps and scene design choices. Use the same mathematical content, target audience, duration, narration/captions and viewing conditions; compare semantic correctness and comprehension, not aesthetic preference alone. Record viewer pre/post comprehension, misconception count by timestamp, factual/numerical correctness, source reproducibility, and completion/retention. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Pilot a rough cut with viewers before polishing; log misconceptions by timestamp and revise the conceptual storyboard, not just aesthetics. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: primary sources for chosen topic. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Capstone: research-quality mathematical explainer. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among first-cut storyboard vs viewer-tested revision, with misconceptions traced to exact timestamps and scene design choices. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using viewer pre/post comprehension, misconception count by timestamp, factual/numerical correctness, source reproducibility, and completion/retention. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if first-cut storyboard vs viewer-tested revision, with misconceptions traced to exact timestamps and scene design choices improve mathematical correctness, semantic continuity or viewer comprehension measured by viewer pre/post comprehension, misconception count by timestamp, factual/numerical correctness, source reproducibility, and completion/retention on multiple examples; aesthetic preference alone does not count.

F. Concrete research project - Capstone: research-quality mathematical explainer

- F1. Falsifiable question. Under the controlled failure regime "Pilot a rough cut with viewers before polishing; log misconceptions by timestamp and revise the conceptual storyboard, not just aesthetics.", which alternative in first-cut storyboard vs viewer-tested revision, with misconceptions traced to exact timestamps and scene design choices is preferable, and is the change explained by mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Publish source, storyboard, references, asset-generation scripts, and a viewer-test log; ensure every numerical animation is generated from executable mathematics. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Use at least three mathematically distinct examples. For semantic/pacing claims, recruit 5-10 pilot viewers if practical; ask prewritten questions about the intended invariant/step, not aesthetic preference, and keep timestamped confusion notes. The central adversarial condition is: Pilot a rough cut with viewers before polishing; log misconceptions by timestamp and revise the conceptual storyboard, not just aesthetics.
- F4. Required evidence. Measure viewer pre/post comprehension, misconception count by timestamp, factual/numerical correctness, source reproducibility, and completion/retention. Deliver Manim source, data-generation/check scripts, storyboard, final video, two alternative cuts for one key transition, and a small viewer-feedback table. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Capstone: research-quality mathematical explainer. If a stable failure/crossover is found among first-cut storyboard vs viewer-tested revision, with misconceptions traced to exact timestamps and scene design choices, implement one adaptive or corrected variant that directly targets mathematical correctness, semantic correspondence, visual continuity, pacing, and viewer comprehension; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Capstone: research-quality mathematical explainer. Using the measured viewer pre/post comprehension, misconception count by timestamp, factual/numerical correctness, source reproducibility, and completion/retention, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Capstone: research-quality mathematical explainer

- G1. Closed-book derivation/oracle. Reverse-engineer the chosen theorem/algorithm into prerequisite claims, visual lemmas, transitions, and a final synthesis; derive a storyboard whose dependency graph mirrors the mathematical argument. Then solve this exact micro-case from scratch without opening your implementation: Storyboard exactly three consecutive scenes of the capstone. For each, write the mathematical claim, visual invariant, narration sentence, source/code oracle, transition trigger, and one likely viewer misconception. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Produce 6-10 minute video plus source repository and one-page design retrospective. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally round or simplify displayed values in a way that changes the mathematical claim without flagging the approximation. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Publish source, storyboard, references, asset-generation scripts, and a viewer-test log; ensure every numerical animation is generated from executable mathematics. Then apply this stress without retuning: Pilot a rough cut with viewers before polishing; log misconceptions by timestamp and revise the conceptual storyboard, not just aesthetics. Report viewer pre/post comprehension, misconception count by timestamp, factual/numerical correctness, source reproducibility, and completion/retention and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among first-cut storyboard vs viewer-tested revision, with misconceptions traced to exact timestamps and scene design choices for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured viewer pre/post comprehension, misconception count by timestamp, factual/numerical correctness, source reproducibility, and completion/retention; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Capstone: research-quality mathematical explainer. Using viewer pre/post comprehension, misconception count by timestamp, factual/numerical correctness, source reproducibility, and completion/retention, Pass when every displayed mathematical quantity is programmatically verified, the key transition preserves the intended semantic invariant, and pilot-viewer answers (when used) support comprehension rather than aesthetics alone. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 17 Closeout: Project, Review and Mastery Gate

Volume project. Produce a 6-10 minute rigorous mathematical/graphics explainer plus a reusable Manim source library. The capstone must teach a genuinely nontrivial result or algorithm.

Cumulative review

- Publish at least six polished Manim scenes spanning geometry, linear algebra, calculus/probability, 3D and a graphics algorithm.
- For every numerical scene, regenerate plotted/displayed values from executable mathematics and preserve regression tests.

- Create adversarial equation/object transformations with repeated/reordered symbols and verify semantic correspondence.
- Run a small comprehension test on two alternative visual explanations and revise from timestamped misconceptions.
- Complete a 6-10 minute capstone with storyboard, source, references, asset-generation scripts and a viewer-test log.

Mastery gate before moving on

- Produce at least 20 short scenes in which motion carries mathematical state/invariant information rather than decoration.
- Pilot the capstone with viewers, log misunderstandings by timestamp, and revise the conceptual storyboard based on evidence.
- Maintain semantic reusable components and demonstrate reuse across at least three unrelated mathematical topics.

Volume 18. Research Apprenticeship, Paper Reproduction, and Open Problems

Purpose. Convert broad knowledge into research ability: reading, reproduction, benchmarking, identifying gaps, and proposing defensible new work.

Suggested duration. 12-18 months overlapping Vols. 8-17

Prerequisites. Begin informally after Volume 6; become serious after at least one specialization volume (8, 10-11, 13-15).

Typical module pace. 1.3-2.0 weeks per module (9 modules total).

Primary references for this volume

- SIGGRAPH/TOG/SGP proceedings and author/reviewer guidance.
- Reproducibility checklists and official code repositories for selected papers.
- Your own experiment logbook becomes the main text in the final stage.

Online spine

SIGGRAPH 2026 Technical Papers: <https://s2026.siggraph.org/program/technical-papers/>

Canonical paper track

- Read one classic paper, one strong recent paper, and one negative/failure-oriented paper or benchmark each week.
- Use SIGGRAPH/TOG/SGP and graphics-adjacent CVPR/ICCV/ECCV papers for current literature.
- Prefer papers with code for the first reproduction; later reproduce papers without turnkey scripts.

The canonical track supplies historical and conceptual anchors. From Volume 6 onward, supplement it with at least one paper from the most recent 2-3 years in your chosen subtopic.

Research paper/source sequence for this volume

Use this as a coherent sequence across the volume; do not force every paper onto every unit. Reconstruct at least one equation, algorithm, figure, or experimental result from each item you select.

- Each week: one classic graphics paper, one strong recent paper, and one benchmark/failure-oriented paper in the same subtopic.
- Berkeley CS184 2026 final-project workflow - use proposal, milestone, demo and report discipline as a model for short research sprints.
- SIGGRAPH/TOG/SGP papers - extract claim, assumptions, representation, dataset, metric, baselines, ablations, failures and computational cost into a common schema.
- Prefer official code for the first reproduction; later perform at least one partial independent reimplementations so you can separate paper detail from repository detail.
- For each candidate research problem, require a falsifiable hypothesis, a controlled baseline, a held-out failure test and a result that could cause you to abandon the idea.

Volume 18, Unit 1. How to read graphics papers

Recommended time. 1.3-2.0 weeks (about 20-40 focused hours depending on background).

What you are learning. Abstract/intro first, then figures, method assumptions, evaluation, supplementary/code.

A. Reading order

1. Read the primary assignment first: Use three papers on the same narrow question: one classic anchor, one strong 2024-2026 paper, and one competing method. For each, read in the order figures/captions -> abstract/introduction -> method assumptions -> evaluation/ablations -> supplement/code. Use the current SIGGRAPH Technical Papers reviewer criteria as a checklist for novelty, validation, completeness, clarity, and limitations.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Weekly paper club protocol.
3. Keep the most useful executable/reference page open while coding: Keep the actual paper PDF, supplement/project page and official code/data repository open together; the goal is claim-to-evidence tracing, not framework-specific coding.

B. Definitions, results and algorithms to know

- Claim-evidence mapping
- identifying baselines
- separating representation, objective and optimizer
- Precision requirement: distinguish and justify chronological summary vs claim-assumption-evidence-failure decomposition on the same paper. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Decompose one graphics paper into problem statement, assumptions, representation, objective, algorithm, baselines, metrics, ablations, and failure cases; reconstruct the strongest causal claim and the evidence supporting it.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Take three papers in one subfield and predict contribution, assumptions, and likely weakness from abstract/figures; compare with conclusions after full reading. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. For each paper write one page: problem, representation, objective, algorithm, assumptions, baselines, strongest evidence, failure cases, next experiment. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Choose one paper figure. Write a five-row table: claim, independent variable, dependent metric, baseline, hidden assumption. Then state one observation that would falsify the figure-level claim. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Take three papers in one subfield and predict contribution, assumptions, and likely weakness from abstract/figures; compare with conclusions after full reading. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Parse three papers into a structured Markdown/YAML schema (claim, assumption, method, data, metric, evidence, failure, next test) and compare omissions across papers.
- P5 - Machine-readable evidence. Export accuracy of reconstructed claim/assumption/evidence map, missing-variable count, and quality of proposed falsifying experiment to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: chronological summary vs claim-assumption-evidence-failure decomposition on the same paper. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Take three papers in one subfield and predict contribution, assumptions, and likely weakness from abstract/figures; compare with conclusions after full reading." without hand-editing code between methods. Record accuracy of reconstructed claim/assumption/evidence map, missing-variable count, and quality of proposed falsifying experiment.
- P7 - Deliberate bug and regression drill. Intentionally copy the authors' stated conclusion without tracing it to the actual experiment/table and assumptions. Before running, predict the first observable failure in accuracy of

reconstructed claim/assumption/evidence map, missing-variable count, and quality of proposed falsifying experiment. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Decompose one graphics paper into problem statement, assumptions, representation, objective, algorithm, baselines, metrics, ablations, and failure cases; reconstruct the strongest causal claim and the evidence supporting it. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Choose one paper figure. Write a five-row table: claim, independent variable, dependent metric, baseline, hidden assumption. Then state one observation that would falsify the figure-level claim. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: For each paper write one page: problem, representation, objective, algorithm, assumptions, baselines, strongest evidence, failure cases, next experiment. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Parse three papers into a structured Markdown/YAML schema (claim, assumption, method, data, metric, evidence, failure, next test) and compare omissions across papers. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare chronological summary vs claim-assumption-evidence-failure decomposition on the same paper. Use the same paper/data/split/metric and decision question when comparing research workflows; differences must come from the methodological change, not access to different evidence. Record accuracy of reconstructed claim/assumption/evidence map, missing-variable count, and quality of proposed falsifying experiment. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Take three papers in one subfield and predict contribution, assumptions, and likely weakness from abstract/figures; compare with conclusions after full reading. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: SIGGRAPH/TOG papers and author resources. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on How to read graphics papers. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among chronological summary vs claim-assumption-evidence-failure decomposition on the same paper. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using accuracy of reconstructed claim/assumption/evidence map, missing-variable count, and quality of proposed falsifying experiment. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance the workflow only if comparing chronological summary vs claim-assumption-evidence-failure decomposition on the same paper changes a concrete research decision or exposes a reproducibility/evaluation weakness through accuracy of reconstructed claim/assumption/evidence map, missing-variable count, and quality of proposed falsifying experiment on more than one case.

F. Concrete research project - How to read graphics papers

- F1. Falsifiable question. Under the controlled failure regime "Take three papers in one subfield and predict contribution, assumptions, and likely weakness from abstract/figures; compare with conclusions after full reading.", which alternative in chronological summary vs claim-assumption-evidence-failure decomposition on the same paper is preferable, and is the change explained by confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Parse three papers into a structured Markdown/YAML schema (claim, assumption, method, data, metric, evidence, failure, next test) and compare omissions across papers. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Apply the method to at least three independent papers/datasets/failure groups.

Predeclare the prediction, preserve exact versions/configs, and reserve a held-out or independently reconstructed check that can contradict the preferred conclusion. The central adversarial condition is: Take three papers in one subfield and predict contribution, assumptions, and likely weakness from abstract/figures; compare with conclusions after full reading.

F4. Required evidence. Measure accuracy of reconstructed claim/assumption/evidence map, missing-variable count, and quality of proposed falsifying experiment. Deliver exact paper/code/data identifiers, experiment matrix, raw outputs, decision memo, and a counterexample/negative result that constrains the claim. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for How to read graphics papers. If a stable failure/crossover is found among chronological summary vs claim-assumption-evidence-failure decomposition on the same paper, implement one adaptive or corrected variant that directly targets confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for How to read graphics papers. Using the measured accuracy of reconstructed claim/assumption/evidence map, missing-variable count, and quality of proposed falsifying experiment, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for How to read graphics papers

- G1. Closed-book derivation/oracle. Decompose one graphics paper into problem statement, assumptions, representation, objective, algorithm, baselines, metrics, ablations, and failure cases; reconstruct the strongest causal claim and the evidence supporting it. Then solve this exact micro-case from scratch without opening your implementation: Choose one paper figure. Write a five-row table: claim, independent variable, dependent metric, baseline, hidden assumption. Then state one observation that would falsify the figure-level claim. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: For each paper write one page: problem, representation, objective, algorithm, assumptions, baselines, strongest evidence, failure cases, next experiment. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally copy the authors' stated conclusion without tracing it to the actual experiment/table and assumptions. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Parse three papers into a structured Markdown/YAML schema (claim, assumption, method, data, metric, evidence, failure, next test) and compare omissions across papers. Then apply this stress without retuning: Take three papers in one subfield and predict contribution, assumptions, and likely weakness from abstract/figures; compare with conclusions after full reading. Report accuracy of reconstructed claim/assumption/evidence map, missing-variable count, and quality of proposed falsifying experiment and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among chronological summary vs claim-assumption-evidence-failure decomposition on the same paper for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured accuracy of reconstructed claim/assumption/evidence map, missing-variable count, and quality of proposed falsifying experiment; state one counterexample that would reverse your recommendation.
- G6. Pass condition for How to read graphics papers. Using accuracy of reconstructed claim/assumption/evidence map, missing-variable count, and quality of proposed falsifying experiment, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 18, Unit 2. Reproduction ladder

Recommended time. 1.3-2.0 weeks (about 20-40 focused hours depending on background).

What you are learning. Reproduce figures, then metrics, then ablations, then stress tests.

A. Reading order

1. Read the primary assignment first: Choose one canonical executable target from PBRT, libigl/Geometry Central, Physics-Based Simulation, Mitsuba 3, Nerfstudio, or Manim. Reproduce one reported figure/table first; then independently reimplement the smallest central component; finally perturb one assumption and document whether the original conclusion survives.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Start with PBRT/Mitsuba/libigl/Nerfstudio examples then research repos.
3. Keep the most useful executable/reference page open while coding: Use the official repository/environment of the chosen paper for Stage 1, then progressively replace components with your own code; preserve exact commit/config/data identifiers.

B. Definitions, results and algorithms to know

- Replication vs reimplementation
- tolerance for nondeterminism
- Precision requirement: distinguish and justify author-code rerun, figure recreation, metric matching, and independent reimplementation as progressively stronger evidence. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a staged reproduction plan from sanity check to figure recreation to metric matching to full training/evaluation; specify what discrepancy threshold advances or stops each stage.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Choose one paper with turnkey code and one without; document which hidden assumptions emerge at each reproduction level. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Complete 3 reproductions: one classical algorithm, one simulation/geometry paper, one neural/inverse paper. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Take a published curve with three plotted points. Record values from the paper, rerun author code if available, recreate from released outputs, then independently implement the smallest calculation; tabulate discrepancy at each rung. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Choose one paper with turnkey code and one without; document which hidden assumptions emerge at each reproduction level. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Automate environment setup and exact-command logging for one reproduction; store expected outputs/checksums for the smallest sanity stage.
- P5 - Machine-readable evidence. Export difference from reported figure/metric, environment/config discrepancy count, seed variance, and level of independent implementation reached to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Build one common experiment harness covering the methods/regimes in: author-code rerun, figure recreation, metric matching, and independent reimplementation as progressively stronger evidence. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Choose one paper with turnkey code and one without; document which hidden assumptions emerge at each reproduction level." without hand-editing code between methods. Record difference from reported figure/metric, environment/config discrepancy count, seed variance, and level of independent implementation reached.
- P7 - Deliberate bug and regression drill. Intentionally change code commit, dataset preprocessing, or default configuration between stages without recording the change. Before running, predict the first observable failure in difference from reported figure/metric, environment/config discrepancy count, seed variance, and level of independent implementation reached. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a staged reproduction plan from sanity check to figure recreation to metric matching to full training/evaluation; specify what discrepancy threshold advances or stops each stage. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Take a published curve with three plotted points. Record values from the paper, rerun author code if available, recreate from released outputs, then independently implement the smallest calculation; tabulate discrepancy at each rung. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Complete 3 reproductions: one classical algorithm, one simulation/geometry paper, one neural/inverse paper. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Automate environment setup and exact-command logging for one reproduction; store expected outputs/checksums for the smallest sanity stage. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare author-code rerun, figure recreation, metric matching, and independent reimplementations as progressively stronger evidence. Use the same paper/data/split/metric and decision question when comparing research workflows; differences must come from the methodological change, not access to different evidence. Record difference from reported figure/metric, environment/config discrepancy count, seed variance, and level of independent implementation reached. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Choose one paper with turnkey code and one without; document which hidden assumptions emerge at each reproduction level. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Choose papers with official code first. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Reproduction ladder. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among author-code rerun, figure recreation, metric matching, and independent reimplementations as progressively stronger evidence. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using difference from reported figure/metric, environment/config discrepancy count, seed variance, and level of independent implementation reached. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance the workflow only if comparing author-code rerun, figure recreation, metric matching, and independent reimplementations as progressively stronger evidence changes a concrete research decision or exposes a reproducibility/evaluation weakness through difference from reported figure/metric, environment/config discrepancy count, seed variance, and level of independent implementation reached on more than one case.

F. Concrete research project - Reproduction ladder

- F1. Falsifiable question. Under the controlled failure regime "Choose one paper with turnkey code and one without; document which hidden assumptions emerge at each reproduction level.", characterize the ranking or regime transition across author-code rerun, figure recreation, metric matching, and independent reimplementations as progressively stronger evidence. Predict which method/regime should dominate or be diagnosed first as the stress changes, and test whether the transition is explained by confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability. Name at least one observation that would falsify your prediction.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Automate environment setup and exact-command logging for one reproduction; store expected outputs/checksums for the smallest sanity stage. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Apply the method to at least three independent papers/datasets/failure groups. Predeclare the prediction, preserve exact versions/configs, and reserve a held-out or independently reconstructed check that can contradict the preferred conclusion. The central adversarial condition is: Choose one paper with turnkey code and one without; document which hidden assumptions emerge at each reproduction level.
- F4. Required evidence. Measure difference from reported figure/metric, environment/config discrepancy count, seed variance, and level of independent implementation reached. Deliver exact paper/code/data identifiers, experiment

matrix, raw outputs, decision memo, and a counterexample/negative result that constrains the claim. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Reproduction ladder. If a stable failure/crossover is found among author-code rerun, figure recreation, metric matching, and independent reimplementations as progressively stronger evidence, implement one adaptive or corrected variant that directly targets confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Reproduction ladder. Using the measured difference from reported figure/metric, environment/config discrepancy count, seed variance, and level of independent implementation reached, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Reproduction ladder

- G1. Closed-book derivation/oracle. Derive a staged reproduction plan from sanity check to figure recreation to metric matching to full training/evaluation; specify what discrepancy threshold advances or stops each stage. Then solve this exact micro-case from scratch without opening your implementation: Take a published curve with three plotted points. Record values from the paper, rerun author code if available, recreate from released outputs, then independently implement the smallest calculation; tabulate discrepancy at each rung. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Complete 3 reproductions: one classical algorithm, one simulation/geometry paper, one neural/inverse paper. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally change code commit, dataset preprocessing, or default configuration between stages without recording the change. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Automate environment setup and exact-command logging for one reproduction; store expected outputs/checksums for the smallest sanity stage. Then apply this stress without retuning: Choose one paper with turnkey code and one without; document which hidden assumptions emerge at each reproduction level. Report difference from reported figure/metric, environment/config discrepancy count, seed variance, and level of independent implementation reached and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method/regime defense. Using your own measured results, make a diagnosis or ranking across author-code rerun, figure recreation, metric matching, and independent reimplementations as progressively stronger evidence for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. Cite the measured difference from reported figure/metric, environment/config discrepancy count, seed variance, and level of independent implementation reached; state one counterexample that would change the diagnosis or ranking.
- G6. Pass condition for Reproduction ladder. Using difference from reported figure/metric, environment/config discrepancy count, seed variance, and level of independent implementation reached, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 18, Unit 3. Benchmark design

Recommended time. 1.3-2.0 weeks (about 20-40 focused hours depending on background).

What you are learning. Datasets, splits, controlled synthetic scenes, metrics, runtime/memory, statistical reporting.

A. Reading order

1. Read the primary assignment first: Read the evaluation sections of three papers solving the same task plus the current SIGGRAPH Technical Papers criteria. Build a benchmark specification before running models: task, dataset

provenance, train/test separation, nuisance variables, oracle/sanity cases, primary metric, secondary diagnostics, compute budget, seed policy, and a deliberately adversarial split.

2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Build reusable experiment harness.

3. Keep the most useful executable/reference page open while coding: Use the dataset's official documentation plus your own split/leakage scripts; do not inherit split logic blindly from a model repository.

B. Definitions, results and algorithms to know

- Confounding
- train/test leakage
- Pareto comparison
- error bars
- Precision requirement: distinguish and justify random/IID split vs leakage-controlled, nuisance-balanced, and adversarial/failure subsets. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a benchmark sampling model: target population, split rules, nuisance variables, leakage risks, metrics, and uncertainty; show how one seemingly reasonable split can give an invalid conclusion.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Create splits based on a mechanistic variable, then check whether method rankings survive alternative split definitions and nuisance factors. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create a benchmark with at least one adversarial/failure subset, not only average-case scenes. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Create 12 synthetic examples with target label correlated 90% with a nuisance color in training but reversed in test. Construct IID, balanced, and adversarial splits and predict ranking changes for a nuisance-using model. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Create splits based on a mechanistic variable, then check whether method rankings survive alternative split definitions and nuisance factors. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Write split-generation code with leakage checks and nuisance-balance summaries; generate a separate adversarial/failure split from explicit rules.
- P5 - Machine-readable evidence. Export leakage rate, nuisance imbalance, confidence interval width, adversarial-subset coverage, and ranking stability across splits to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: random/IID split vs leakage-controlled, nuisance-balanced, and adversarial/failure subsets. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Create splits based on a mechanistic variable, then check whether method rankings survive alternative split definitions and nuisance factors." without hand-editing code between methods. Record leakage rate, nuisance imbalance, confidence interval width, adversarial-subset coverage, and ranking stability across splits.
- P7 - Deliberate bug and regression drill. Intentionally allow train/test identity or scene/object leakage through near-duplicates or shared captures. Before running, predict the first observable failure in leakage rate, nuisance imbalance, confidence interval width, adversarial-subset coverage, and ranking stability across splits. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a benchmark sampling model: target population, split rules, nuisance variables, leakage risks, metrics, and uncertainty; show how one seemingly reasonable split can give an invalid conclusion. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.

2. Smallest nontrivial hand case. Create 12 synthetic examples with target label correlated 90% with a nuisance color in training but reversed in test. Construct IID, balanced, and adversarial splits and predict ranking changes for a nuisance-

using model. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.

3. Core programming build. Implement the unit core: Create a benchmark with at least one adversarial/failure subset, not only average-case scenes. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Write split-generation code with leakage checks and nuisance-balance summaries; generate a separate adversarial/failure split from explicit rules. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.

4. Matched method comparison. Compare random/IID split vs leakage-controlled, nuisance-balanced, and adversarial/failure subsets. Use the same paper/data/split/metric and decision question when comparing research workflows; differences must come from the methodological change, not access to different evidence. Record leakage rate, nuisance imbalance, confidence interval width, adversarial-subset coverage, and ranking stability across splits. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Create splits based on a mechanistic variable, then check whether method rankings survive alternative split definitions and nuisance factors. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.

6. Reading reconstruction. Using the assigned reading: benchmark sections of strong papers and SIGGRAPH reviewing criteria. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Benchmark design. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among random/IID split vs leakage-controlled, nuisance-balanced, and adversarial/failure subsets. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using leakage rate, nuisance imbalance, confidence interval width, adversarial-subset coverage, and ranking stability across splits. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance the workflow only if comparing random/IID split vs leakage-controlled, nuisance-balanced, and adversarial/failure subsets changes a concrete research decision or exposes a reproducibility/evaluation weakness through leakage rate, nuisance imbalance, confidence interval width, adversarial-subset coverage, and ranking stability across splits on more than one case.

F. Concrete research project - Benchmark design

F1. Falsifiable question. Under the controlled failure regime "Create splits based on a mechanistic variable, then check whether method rankings survive alternative split definitions and nuisance factors.", which alternative in random/IID split vs leakage-controlled, nuisance-balanced, and adversarial/failure subsets is preferable, and is the change explained by confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability?

Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Write split-generation code with leakage checks and nuisance-balance summaries; generate a separate adversarial/failure split from explicit rules. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Apply the method to at least three independent papers/datasets/failure groups. Predeclare the prediction, preserve exact versions/configs, and reserve a held-out or independently reconstructed check that can contradict the preferred conclusion. The central adversarial condition is: Create splits based on a mechanistic variable, then check whether method rankings survive alternative split definitions and nuisance factors.

F4. Required evidence. Measure leakage rate, nuisance imbalance, confidence interval width, adversarial-subset coverage, and ranking stability across splits. Deliver exact paper/code/data identifiers, experiment matrix, raw outputs, decision memo, and a counterexample/negative result that constrains the claim. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Benchmark design. If a stable failure/crossover is found among random/IID split vs leakage-controlled, nuisance-balanced, and adversarial/failure subsets, implement one adaptive or corrected variant that directly targets confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Benchmark design. Using the measured leakage rate, nuisance imbalance, confidence interval width, adversarial-subset coverage, and ranking stability across splits, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Benchmark design

- G1. Closed-book derivation/oracle. Derive a benchmark sampling model: target population, split rules, nuisance variables, leakage risks, metrics, and uncertainty; show how one seemingly reasonable split can give an invalid conclusion. Then solve this exact micro-case from scratch without opening your implementation: Create 12 synthetic examples with target label correlated 90% with a nuisance color in training but reversed in test. Construct IID, balanced, and adversarial splits and predict ranking changes for a nuisance-using model. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create a benchmark with at least one adversarial/failure subset, not only average-case scenes. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally allow train/test identity or scene/object leakage through near-duplicates or shared captures. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Write split-generation code with leakage checks and nuisance-balance summaries; generate a separate adversarial/failure split from explicit rules. Then apply this stress without retuning: Create splits based on a mechanistic variable, then check whether method rankings survive alternative split definitions and nuisance factors. Report leakage rate, nuisance imbalance, confidence interval width, adversarial-subset coverage, and ranking stability across splits and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among random/IID split vs leakage-controlled, nuisance-balanced, and adversarial/failure subsets for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured leakage rate, nuisance imbalance, confidence interval width, adversarial-subset coverage, and ranking stability across splits; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Benchmark design. Using leakage rate, nuisance imbalance, confidence interval width, adversarial-subset coverage, and ranking stability across splits, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 18, Unit 4. Ablation and causal reasoning

Recommended time. 1.3-2.0 weeks (about 20-40 focused hours depending on background).

What you are learning. Remove components, vary hyperparameters, test scaling laws/complexity.

A. Reading order

1. Read the primary assignment first: Select a paper with at least three interacting components. Reconstruct its ablation table, then design a factorial or staged ablation that distinguishes component necessity from interaction effects. Read the method/evaluation/supplement together and predeclare the causal claim each ablation can and cannot support.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Reproduce one ablation from a paper.
3. Keep the most useful executable/reference page open while coding: Use your configuration/experiment framework and the exact baseline repository; keep a machine-readable ablation matrix outside the training code.

B. Definitions, results and algorithms to know

- Necessary vs sufficient evidence
- interaction effects
- Precision requirement: distinguish and justify one-factor-at-a-time ablations vs confounded capacity/compute changes, using a predeclared causal graph. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Draw a causal graph for a graphics method with data, representation, optimization, compute, and evaluation; derive an ablation matrix that changes one causal factor at a time while matching compute.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Use replacement/randomization/capacity-matched ablations so a performance change cannot be explained only by parameter count, compute, or training time. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Design an ablation matrix before implementing your own modification. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Suppose method A adds module X and doubles parameters/compute. Build a four-run 2x2 design controlling X and capacity separately; write which contrasts estimate the effect of X versus capacity. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Use replacement/randomization/capacity-matched ablations so a performance change cannot be explained only by parameter count, compute, or training time. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Implement configuration-driven ablations with matched training steps/parameters where intended; automatically create an ablation matrix and confidence intervals over seeds.
- P5 - Machine-readable evidence. Export effect size with uncertainty, compute/capacity match, interaction/confounding evidence, and seed/split sensitivity to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: one-factor-at-a-time ablations vs confounded capacity/compute changes, using a predeclared causal graph. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Use replacement/randomization/capacity-matched ablations so a performance change cannot be explained only by parameter count, compute, or training time." without hand-editing code between methods. Record effect size with uncertainty, compute/capacity match, interaction/confounding evidence, and seed/split sensitivity.
- P7 - Deliberate bug and regression drill. Intentionally change two coupled factors in one ablation row and then assign the effect to only one of them. Before running, predict the first observable failure in effect size with uncertainty, compute/capacity match, interaction/confounding evidence, and seed/split sensitivity. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Draw a causal graph for a graphics method with data, representation, optimization, compute, and evaluation; derive an ablation matrix that changes one causal factor at a time while matching compute. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Suppose method A adds module X and doubles parameters/compute. Build a four-run 2x2 design controlling X and capacity separately; write which contrasts estimate the effect of X versus capacity. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Design an ablation matrix before implementing your own modification. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Implement configuration-driven ablations with matched training steps/parameters where intended; automatically create an ablation matrix and confidence intervals over seeds. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare one-factor-at-a-time ablations vs confounded capacity/compute changes, using a predeclared causal graph. Use the same paper/data/split/metric and decision question when comparing research workflows; differences must come from the methodological change, not access to different evidence. Record effect size with uncertainty, compute/capacity match, interaction/confounding evidence, and seed/split sensitivity. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.

5. Adversarial experiment. Use replacement/randomization/capacity-matched ablations so a performance change cannot be explained only by parameter count, compute, or training time. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Study ablation tables in exemplary papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Ablation and causal reasoning. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among one-factor-at-a-time ablations vs confounded capacity/compute changes, using a predeclared causal graph. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using effect size with uncertainty, compute/capacity match, interaction/confounding evidence, and seed/split sensitivity. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance the workflow only if comparing one-factor-at-a-time ablations vs confounded capacity/compute changes, using a predeclared causal graph changes a concrete research decision or exposes a reproducibility/evaluation weakness through effect size with uncertainty, compute/capacity match, interaction/confounding evidence, and seed/split sensitivity on more than one case.

F. Concrete research project - Ablation and causal reasoning

- F1. Falsifiable question. Under the controlled failure regime "Use replacement/randomization/capacity-matched ablations so a performance change cannot be explained only by parameter count, compute, or training time.", which alternative in one-factor-at-a-time ablations vs confounded capacity/compute changes, using a predeclared causal graph is preferable, and is the change explained by confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Implement configuration-driven ablations with matched training steps/parameters where intended; automatically create an ablation matrix and confidence intervals over seeds. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Apply the method to at least three independent papers/datasets/failure groups. Predeclare the prediction, preserve exact versions/configs, and reserve a held-out or independently reconstructed check that can contradict the preferred conclusion. The central adversarial condition is: Use replacement/randomization/capacity-matched ablations so a performance change cannot be explained only by parameter count, compute, or training time.
- F4. Required evidence. Measure effect size with uncertainty, compute/capacity match, interaction/confounding evidence, and seed/split sensitivity. Deliver exact paper/code/data identifiers, experiment matrix, raw outputs, decision memo, and a counterexample/negative result that constrains the claim. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for Ablation and causal reasoning. If a stable failure/crossover is found among one-factor-at-a-time ablations vs confounded capacity/compute changes, using a predeclared causal graph, implement one adaptive or corrected variant that directly targets confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for Ablation and causal reasoning. Using the measured effect size with uncertainty, compute/capacity match, interaction/confounding evidence, and seed/split sensitivity, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Ablation and causal reasoning

- G1. Closed-book derivation/oracle. Draw a causal graph for a graphics method with data, representation, optimization, compute, and evaluation; derive an ablation matrix that changes one causal factor at a time while matching compute. Then solve this exact micro-case from scratch without opening your implementation: Suppose method A adds module X and doubles parameters/compute. Build a four-run 2x2 design controlling X and capacity

separately; write which contrasts estimate the effect of X versus capacity. You pass this part only if the algebra/geometry and the stored oracle agree.

- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Design an ablation matrix before implementing your own modification. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally change two coupled factors in one ablation row and then assign the effect to only one of them. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Implement configuration-driven ablations with matched training steps/parameters where intended; automatically create an ablation matrix and confidence intervals over seeds. Then apply this stress without retuning: Use replacement/randomization/capacity-matched ablations so a performance change cannot be explained only by parameter count, compute, or training time. Report effect size with uncertainty, compute/capacity match, interaction/confounding evidence, and seed/split sensitivity and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among one-factor-at-a-time ablations vs confounded capacity/compute changes, using a predeclared causal graph for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured effect size with uncertainty, compute/capacity match, interaction/confounding evidence, and seed/split sensitivity; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Ablation and causal reasoning. Using effect size with uncertainty, compute/capacity match, interaction/confounding evidence, and seed/split sensitivity, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 18, Unit 5. Research coding discipline

Recommended time. 1.3-2.0 weeks (about 20-40 focused hours depending on background).

What you are learning. Version control, configs, logging, seeds, checkpoints, profiling, artifacts.

A. Reading order

1. Read the primary assignment first: Use one existing graphics implementation from PBRT, libigl/Geometry Central, Physics-Based Simulation, Mitsuba 3, or Nerfstudio as a repository-structure reference. Require environment lock, deterministic/configured seeds, immutable raw outputs, machine-readable metrics, unit/regression tests for analytic cases, and a single command that regenerates every reported table/figure.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Automate experiment scripts.
3. Keep the most useful executable/reference page open while coding: Use the project's actual package/tool documentation, environment lock, version control and experiment logger; the executable reference is the codebase you are making reproducible.

B. Definitions, results and algorithms to know

- Reproducible experiment = code + exact config + data version + environment + metric definition
- Precision requirement: distinguish and justify manual experiment state vs fully logged config/seed/data/code/checkpoint pipeline that regenerates a central figure. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive a minimal research software architecture that makes configurations, seeds, data versions, checkpoints, metrics, and figures traceable; identify how silent state can invalidate comparison.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Rerun old experiments after controlled dependency/data changes; verify hashes/configuration tracking catches drift before it changes conclusions. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Make one-click reproduction of a central table/figure from your project. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Define one experiment with two seeds and three settings. Write the directory/config schema and exact command that must regenerate a central CSV and figure from a clean checkout without manual file selection. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Rerun old experiments after controlled dependency/data changes; verify hashes/configuration tracking catches drift before it changes conclusions. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Make a single command regenerate one paper-quality table/figure from raw data; fail loudly if code/data/config hashes differ from the recorded run.
- P5 - Machine-readable evidence. Export one-command rerun success, config/data/code traceability, result checksum/metric match, and time to reproduce from clean environment to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: manual experiment state vs fully logged config/seed/data/code/checkpoint pipeline that regenerates a central figure. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Rerun old experiments after controlled dependency/data changes; verify hashes/configuration tracking catches drift before it changes conclusions." without hand-editing code between methods. Record one-command rerun success, config/data/code traceability, result checksum/metric match, and time to reproduce from clean environment.
- P7 - Deliberate bug and regression drill. Intentionally let the result depend on an untracked config/default/environment variable. Before running, predict the first observable failure in one-command rerun success, config/data/code traceability, result checksum/metric match, and time to reproduce from clean environment. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive a minimal research software architecture that makes configurations, seeds, data versions, checkpoints, metrics, and figures traceable; identify how silent state can invalidate comparison. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Define one experiment with two seeds and three settings. Write the directory/config schema and exact command that must regenerate a central CSV and figure from a clean checkout without manual file selection. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Make one-click reproduction of a central table/figure from your project. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Make a single command regenerate one paper-quality table/figure from raw data; fail loudly if code/data/config hashes differ from the recorded run. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare manual experiment state vs fully logged config/seed/data/code/checkpoint pipeline that regenerates a central figure. Use the same paper/data/split/metric and decision question when comparing research workflows; differences must come from the methodological change, not access to different evidence. Record one-command rerun success, config/data/code traceability, result checksum/metric match, and time to reproduce from clean environment. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Rerun old experiments after controlled dependency/data changes; verify hashes/configuration tracking catches drift before it changes conclusions. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Tool documentation appropriate to project. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Research coding discipline. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.

7. Research mini-study. Predict the failure/crossover boundary among manual experiment state vs fully logged config/seed/data/code/checkpoint pipeline that regenerates a central figure. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using one-command rerun success, config/data/code traceability, result checksum/metric match, and time to reproduce from clean environment. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance the workflow only if comparing manual experiment state vs fully logged config/seed/data/code/checkpoint pipeline that regenerates a central figure changes a concrete research decision or exposes a reproducibility/evaluation weakness through one-command rerun success, config/data/code traceability, result checksum/metric match, and time to reproduce from clean environment on more than one case.

F. Concrete research project - Research coding discipline

F1. Falsifiable question. Under the controlled failure regime "Rerun old experiments after controlled dependency/data changes; verify hashes/configuration tracking catches drift before it changes conclusions.", which alternative in manual experiment state vs fully logged config/seed/data/code/checkpoint pipeline that regenerates a central figure is preferable, and is the change explained by confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Make a single command regenerate one paper-quality table/figure from raw data; fail loudly if code/data/config hashes differ from the recorded run. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Apply the method to at least three independent papers/datasets/failure groups. Predeclare the prediction, preserve exact versions/configs, and reserve a held-out or independently reconstructed check that can contradict the preferred conclusion. The central adversarial condition is: Rerun old experiments after controlled dependency/data changes; verify hashes/configuration tracking catches drift before it changes conclusions.

F4. Required evidence. Measure one-command rerun success, config/data/code traceability, result checksum/metric match, and time to reproduce from clean environment. Deliver exact paper/code/data identifiers, experiment matrix, raw outputs, decision memo, and a counterexample/negative result that constrains the claim. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Research coding discipline. If a stable failure/crossover is found among manual experiment state vs fully logged config/seed/data/code/checkpoint pipeline that regenerates a central figure, implement one adaptive or corrected variant that directly targets confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Research coding discipline. Using the measured one-command rerun success, config/data/code traceability, result checksum/metric match, and time to reproduce from clean environment, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Research coding discipline

- G1. Closed-book derivation/oracle. Derive a minimal research software architecture that makes configurations, seeds, data versions, checkpoints, metrics, and figures traceable; identify how silent state can invalidate comparison. Then solve this exact micro-case from scratch without opening your implementation: Define one experiment with two seeds and three settings. Write the directory/config schema and exact command that must regenerate a central CSV and figure from a clean checkout without manual file selection. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Make one-click reproduction of a central table/figure from your project. Add the oracle as an automated test and make it pass before importing any optimized/framework code.

- G3. Debugging exam. Intentionally let the result depend on an untracked config/default/environment variable. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Make a single command regenerate one paper-quality table/figure from raw data; fail loudly if code/data/config hashes differ from the recorded run. Then apply this stress without retuning: Rerun old experiments after controlled dependency/data changes; verify hashes/configuration tracking catches drift before it changes conclusions. Report one-command rerun success, config/data/code traceability, result checksum/metric match, and time to reproduce from clean environment and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among manual experiment state vs fully logged config/seed/data/code/checkpoint pipeline that regenerates a central figure for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured one-command rerun success, config/data/code traceability, result checksum/metric match, and time to reproduce from clean environment; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Research coding discipline. Using one-command rerun success, config/data/code traceability, result checksum/metric match, and time to reproduce from clean environment, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 18, Unit 6. From anomaly to research question

Recommended time. 1.3-2.0 weeks (about 20-40 focused hours depending on background).

What you are learning. Look for systematic failures, unexplained tradeoffs, missing invariances, unrealistic assumptions.

A. Reading order

1. Read the primary assignment first: Start from at least 30 logged failures produced by one baseline, not from a proposed method. Cluster failures by observable features, formulate at least three competing mechanisms, and design an intervention whose predicted outcomes differ across mechanisms. Read related work only after the first mechanism map so literature labels do not dictate the clustering.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Maintain anomaly notebook.
3. Keep the most useful executable/reference page open while coding: Keep the failure database, plotting scripts and the three hypothesis-specific interventions open; literature search comes after the failure clusters are defined.

B. Definitions, results and algorithms to know

- Research question should imply falsifiable experiment
- Precision requirement: distinguish and justify post-hoc story vs three competing mechanistic hypotheses tested on held-out failures. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Turn a cluster of failures into at least three competing mechanistic hypotheses; derive discriminating predictions whose outcomes differ across hypotheses rather than merely retesting the same effect.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Collect at least 30 failures, cluster by hypothesized mechanism, and test one intervention on held-out failures. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Collect 20 failure cases; cluster them; propose 3 hypotheses; design cheap discriminating experiments. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Take six failures with observed features {small object, grazing view, noisy pose}. Propose three competing mechanisms, assign each failure to predicted clusters, and specify one intervention whose outcomes differ across hypotheses. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.

- P3 - Robustness/diagnostic harness. Collect at least 30 failures, cluster by hypothesized mechanism, and test one intervention on held-out failures. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build a failure database with tags/features, clustering/summary plots, and held-out intervention evaluation for three competing hypotheses.
- P5 - Machine-readable evidence. Export held-out failure reduction by intervention, effect size/uncertainty, competing-hypothesis discrimination, and negative-result informativeness to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: post-hoc story vs three competing mechanistic hypotheses tested on held-out failures. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Collect at least 30 failures, cluster by hypothesized mechanism, and test one intervention on held-out failures." without hand-editing code between methods. Record held-out failure reduction by intervention, effect size/uncertainty, competing-hypothesis discrimination, and negative-result informativeness.
- P7 - Deliberate bug and regression drill. Intentionally form hypotheses after inspecting held-out outcomes, turning the diagnostic test into confirmation. Before running, predict the first observable failure in held-out failure reduction by intervention, effect size/uncertainty, competing-hypothesis discrimination, and negative-result informativeness. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Turn a cluster of failures into at least three competing mechanistic hypotheses; derive discriminating predictions whose outcomes differ across hypotheses rather than merely retesting the same effect. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Take six failures with observed features {small object, grazing view, noisy pose}. Propose three competing mechanisms, assign each failure to predicted clusters, and specify one intervention whose outcomes differ across hypotheses. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Collect 20 failure cases; cluster them; propose 3 hypotheses. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build a failure database with tags/features, clustering/summary plots, and held-out intervention evaluation for three competing hypotheses. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare post-hoc story vs three competing mechanistic hypotheses tested on held-out failures. Use the same paper/data/split/metric and decision question when comparing research workflows; differences must come from the methodological change, not access to different evidence. Record held-out failure reduction by intervention, effect size/uncertainty, competing-hypothesis discrimination, and negative-result informativeness. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Collect at least 30 failures, cluster by hypothesized mechanism, and test one intervention on held-out failures. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: related-work clusters, not isolated papers. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on From anomaly to research question. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among post-hoc story vs three competing mechanistic hypotheses tested on held-out failures. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using held-out failure reduction by intervention, effect size/uncertainty, competing-hypothesis discrimination, and negative-result informativeness. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance only if competing hypotheses make different held-out predictions and an intervention changes the targeted failure cluster more than unrelated clusters; otherwise the hypothesis remains too vague.

F. Concrete research project - From anomaly to research question

- F1. Falsifiable question. Under the controlled failure regime "Collect at least 30 failures, cluster by hypothesized mechanism, and test one intervention on held-out failures.", which alternative in post-hoc story vs three competing mechanistic hypotheses tested on held-out failures is preferable, and is the change explained by failure clustering, competing mechanistic hypotheses, held-out interventions, confounding, and post-hoc confirmation risk? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build a failure database with tags/features, clustering/summary plots, and held-out intervention evaluation for three competing hypotheses. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Apply the method to at least three independent papers/datasets/failure groups. Predeclare the prediction, preserve exact versions/configs, and reserve a held-out or independently reconstructed check that can contradict the preferred conclusion. The central adversarial condition is: Collect at least 30 failures, cluster by hypothesized mechanism, and test one intervention on held-out failures.
- F4. Required evidence. Measure held-out failure reduction by intervention, effect size/uncertainty, competing-hypothesis discrimination, and negative-result informativeness. Deliver exact paper/code/data identifiers, experiment matrix, raw outputs, decision memo, and a counterexample/negative result that constrains the claim. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.
- F5. Research escalation for From anomaly to research question. If a stable failure/crossover is found among post-hoc story vs three competing mechanistic hypotheses tested on held-out failures, implement one adaptive or corrected variant that directly targets failure clustering, competing mechanistic hypotheses, held-out interventions, confounding, and post-hoc confirmation risk; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.
- F6. Go/no-go criterion for From anomaly to research question. Using the measured held-out failure reduction by intervention, effect size/uncertainty, competing-hypothesis discrimination, and negative-result informativeness, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for From anomaly to research question

- G1. Closed-book derivation/oracle. Turn a cluster of failures into at least three competing mechanistic hypotheses; derive discriminating predictions whose outcomes differ across hypotheses rather than merely retesting the same effect. Then solve this exact micro-case from scratch without opening your implementation: Take six failures with observed features {small object, grazing view, noisy pose}. Propose three competing mechanisms, assign each failure to predicted clusters, and specify one intervention whose outcomes differ across hypotheses. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Collect 20 failure cases; cluster them; propose 3 hypotheses; design cheap discriminating experiments. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally form hypotheses after inspecting held-out outcomes, turning the diagnostic test into confirmation. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Build a failure database with tags/features, clustering/summary plots, and held-out intervention evaluation for three competing hypotheses. Then apply this stress without retuning: Collect at least 30 failures, cluster by hypothesized mechanism, and test one intervention on held-out failures. Report held-out failure reduction by intervention, effect size/uncertainty, competing-hypothesis discrimination, and negative-result informativeness and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among post-hoc story vs three competing mechanistic hypotheses tested on held-out failures for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the

measured held-out failure reduction by intervention, effect size/uncertainty, competing-hypothesis discrimination, and negative-result informativeness; state one counterexample that would reverse your recommendation.

- G6. Pass condition for From anomaly to research question. Using held-out failure reduction by intervention, effect size/uncertainty, competing-hypothesis discrimination, and negative-result informativeness, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 18, Unit 7. Literature mapping

Recommended time. 1.3-2.0 weeks (about 20-40 focused hours depending on background).

What you are learning. Build citation graph by method family and assumptions.

A. Reading order

1. Read the primary assignment first: Build a 25-40 paper map for one narrow subfield using exact primary sources from ACM/IEEE/arXiv/project pages. Tag each paper by task, representation, objective, supervision/data, assumptions, solver, metrics, code/data availability, and failure regime; then identify apparent gaps and manually check whether they are genuine or terminology artifacts.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Monthly frontier review.
3. Keep the most useful executable/reference page open while coding: Use DOI/ACM/arXiv/project pages and a machine-readable bibliography; link every node in the map to the exact experiment/claim it supports.

B. Definitions, results and algorithms to know

- Taxonomy by representation/objective/solver/data rather than chronology alone
- Precision requirement: distinguish and justify paper-by-paper notes vs concept-method-evidence graph with explicit task/data/metric/assumption axes. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Construct a concept-method-evidence graph for a subfield, derive inclusion/exclusion criteria, and identify which apparent disagreements are due to different tasks, data, metrics, or assumptions.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Code papers by representation, objective, assumptions, supervision/data, and evaluation; inspect empty combinations and determine whether they are meaningful gaps. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Create one living 2-page map for your chosen specialty. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: For ten papers, make a matrix with axes task, representation, supervision, metric, and key assumption. Identify two cells that look empty and check whether they are true gaps or merely terminology mismatches. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Code papers by representation, objective, assumptions, supervision/data, and evaluation; inspect empty combinations and determine whether they are meaningful gaps. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Maintain a machine-readable bibliography with tags for task, representation, supervision, assumptions, metrics, and code/data availability; generate a concept graph automatically.
- P5 - Machine-readable evidence. Export coverage by venue/year/topic, consistency of tags, number of genuine evidence gaps, and ability to retrieve comparison sets by assumption/task to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: paper-by-paper notes vs concept-method-evidence graph with explicit task/data/metric/assumption axes. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Code papers by representation, objective, assumptions, supervision/data, and evaluation; inspect empty combinations and determine whether they are meaningful gaps." without hand-editing code between methods.

Record coverage by venue/year/topic, consistency of tags, number of genuine evidence gaps, and ability to retrieve comparison sets by assumption/task.

- P7 - Deliberate bug and regression drill. Intentionally merge superficially similar papers without recording different assumptions/tasks/evaluation protocols. Before running, predict the first observable failure in coverage by venue/year/topic, consistency of tags, number of genuine evidence gaps, and ability to retrieve comparison sets by assumption/task. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Construct a concept-method-evidence graph for a subfield, derive inclusion/exclusion criteria, and identify which apparent disagreements are due to different tasks, data, metrics, or assumptions. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. For ten papers, make a matrix with axes task, representation, supervision, metric, and key assumption. Identify two cells that look empty and check whether they are true gaps or merely terminology mismatches. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Create one living 2-page map for your chosen specialty. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Maintain a machine-readable bibliography with tags for task, representation, supervision, assumptions, metrics, and code/data availability; generate a concept graph automatically. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare paper-by-paper notes vs concept-method-evidence graph with explicit task/data/metric/assumption axes. Use the same paper/data/split/metric and decision question when comparing research workflows; differences must come from the methodological change, not access to different evidence. Record coverage by venue/year/topic, consistency of tags, number of genuine evidence gaps, and ability to retrieve comparison sets by assumption/task. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Code papers by representation, objective, assumptions, supervision/data, and evaluation; inspect empty combinations and determine whether they are meaningful gaps. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: SIGGRAPH, TOG, SGP, Eurographics, CVPR/ICCV/ECCV where graphics/vision overlap. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Literature mapping. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among paper-by-paper notes vs concept-method-evidence graph with explicit task/data/metric/assumption axes. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using coverage by venue/year/topic, consistency of tags, number of genuine evidence gaps, and ability to retrieve comparison sets by assumption/task. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance the workflow only if comparing paper-by-paper notes vs concept-method-evidence graph with explicit task/data/metric/assumption axes changes a concrete research decision or exposes a reproducibility/evaluation weakness through coverage by venue/year/topic, consistency of tags, number of genuine evidence gaps, and ability to retrieve comparison sets by assumption/task on more than one case.

F. Concrete research project - Literature mapping

F1. Falsifiable question. Under the controlled failure regime "Code papers by representation, objective, assumptions, supervision/data, and evaluation; inspect empty combinations and determine whether they are meaningful gaps.", which alternative in paper-by-paper notes vs concept-method-evidence graph with explicit task/data/metric/assumption axes is preferable, and is the change explained by confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.

F2. Code target. Start from P1/P2, then complete this unit-specific extension: Maintain a machine-readable bibliography with tags for task, representation, supervision, assumptions, metrics, and code/data availability; generate a concept graph automatically. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Apply the method to at least three independent papers/datasets/failure groups. Predeclare the prediction, preserve exact versions/configs, and reserve a held-out or independently reconstructed check that can contradict the preferred conclusion. The central adversarial condition is: Code papers by representation, objective, assumptions, supervision/data, and evaluation; inspect empty combinations and determine whether they are meaningful gaps.

F4. Required evidence. Measure coverage by venue/year/topic, consistency of tags, number of genuine evidence gaps, and ability to retrieve comparison sets by assumption/task. Deliver exact paper/code/data identifiers, experiment matrix, raw outputs, decision memo, and a counterexample/negative result that constrains the claim. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Literature mapping. If a stable failure/crossover is found among paper-by-paper notes vs concept-method-evidence graph with explicit task/data/metric/assumption axes, implement one adaptive or corrected variant that directly targets confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Literature mapping. Using the measured coverage by venue/year/topic, consistency of tags, number of genuine evidence gaps, and ability to retrieve comparison sets by assumption/task, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Literature mapping

- G1. Closed-book derivation/oracle. Construct a concept-method-evidence graph for a subfield, derive inclusion/exclusion criteria, and identify which apparent disagreements are due to different tasks, data, metrics, or assumptions. Then solve this exact micro-case from scratch without opening your implementation: For ten papers, make a matrix with axes task, representation, supervision, metric, and key assumption. Identify two cells that look empty and check whether they are true gaps or merely terminology mismatches. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Create one living 2-page map for your chosen specialty. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally merge superficially similar papers without recording different assumptions/tasks/evaluation protocols. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Maintain a machine-readable bibliography with tags for task, representation, supervision, assumptions, metrics, and code/data availability; generate a concept graph automatically. Then apply this stress without retuning: Code papers by representation, objective, assumptions, supervision/data, and evaluation; inspect empty combinations and determine whether they are meaningful gaps. Report coverage by venue/year/topic, consistency of tags, number of genuine evidence gaps, and ability to retrieve comparison sets by assumption/task and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among paper-by-paper notes vs concept-method-evidence graph with explicit task/data/metric/assumption axes for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured coverage by venue/year/topic, consistency of tags, number of genuine evidence gaps, and ability to retrieve comparison sets by assumption/task; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Literature mapping. Using coverage by venue/year/topic, consistency of tags, number of genuine evidence gaps, and ability to retrieve comparison sets by assumption/task, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and

survives a held-out/independent check. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 18, Unit 8. Writing and presentation

Recommended time. 1.3-2.0 weeks (about 20-40 focused hours depending on background).

What you are learning. Technical figures, method diagrams, equations, ablations, limitations, reproducibility statement.

A. Reading order

1. Read the primary assignment first: Use the current SIGGRAPH 2026 Technical Papers call/reviewer guidance for standards of novelty, validation and completeness, and study three exemplary papers in your chosen subfield for figure/table organization. Generate every quantitative number directly from raw experiment files and add an automated manuscript-consistency check before writing prose around the result.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Use your mathematical-animation skills for talks/videos.
3. Keep the most useful executable/reference page open while coding: Generate every reported number/plot from raw experiment outputs; keep the manuscript source and result-generation scripts linked by checks rather than manual copy/paste.

B. Definitions, results and algorithms to know

- Every claim should be backed by experiment/theorem/citation
- Precision requirement: distinguish and justify result-first narrative vs claim-evidence-limitation structure where every quantitative claim has comparator and uncertainty. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- Derive the logical dependency of a research argument from claim to evidence to limitation; rewrite one result so every quantitative claim has an explicit comparison, uncertainty, and scope.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Give figures/captions to a colleague without the main text; test whether they infer the claimed result, comparison, and limitation correctly. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Write a 6-page mock paper from a reproduction+extension project and give a 12-minute talk. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Take one hypothetical result: 12.4 vs 11.8 with standard errors 0.3 and 0.4. Write a claim that is supported, one that overclaims significance/generalization, and a figure caption that states comparator, metric, and uncertainty. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Give figures/captions to a colleague without the main text; test whether they infer the claimed result, comparison, and limitation correctly. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Create figures/tables directly from the experiment pipeline and add automated consistency checks that manuscript numbers match source CSV/JSON.
- P5 - Machine-readable evidence. Export claim-to-evidence traceability, numerical consistency with source results, uncertainty/comparator coverage, and reader/audience comprehension to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: result-first narrative vs claim-evidence-limitation structure where every quantitative claim has comparator and uncertainty. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Give figures/captions to a colleague without the main text; test whether they infer the claimed result, comparison, and limitation correctly." without hand-editing code between methods. Record claim-to-evidence traceability, numerical consistency with source results, uncertainty/comparator coverage, and reader/audience comprehension.
- P7 - Deliberate bug and regression drill. Intentionally manually copy a result into the manuscript so the plotted/table value can diverge from raw experiment output. Before running, predict the first observable failure in claim-to-evidence traceability, numerical consistency with source results, uncertainty/comparator coverage, and

reader/audience comprehension. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. Derive the logical dependency of a research argument from claim to evidence to limitation; rewrite one result so every quantitative claim has an explicit comparison, uncertainty, and scope. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Take one hypothetical result: 12.4 vs 11.8 with standard errors 0.3 and 0.4. Write a claim that is supported, one that overclaims significance/generalization, and a figure caption that states comparator, metric, and uncertainty. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Write a 6-page mock paper from a reproduction+extension project and give a 12-minute talk. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Create figures/tables directly from the experiment pipeline and add automated consistency checks that manuscript numbers match source CSV/JSON. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare result-first narrative vs claim-evidence-limitation structure where every quantitative claim has comparator and uncertainty. Use the same paper/data/split/metric and decision question when comparing research workflows; differences must come from the methodological change, not access to different evidence. Record claim-to-evidence traceability, numerical consistency with source results, uncertainty/comparator coverage, and reader/audience comprehension. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Give figures/captions to a colleague without the main text; test whether they infer the claimed result, comparison, and limitation correctly. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Strong recent papers as style references. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Writing and presentation. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among result-first narrative vs claim-evidence-limitation structure where every quantitative claim has comparator and uncertainty. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using claim-to-evidence traceability, numerical consistency with source results, uncertainty/comparator coverage, and reader/audience comprehension. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance the workflow only if comparing result-first narrative vs claim-evidence-limitation structure where every quantitative claim has comparator and uncertainty changes a concrete research decision or exposes a reproducibility/evaluation weakness through claim-to-evidence traceability, numerical consistency with source results, uncertainty/comparator coverage, and reader/audience comprehension on more than one case.

F. Concrete research project - Writing and presentation

- F1. Falsifiable question. Under the controlled failure regime "Give figures/captions to a colleague without the main text; test whether they infer the claimed result, comparison, and limitation correctly.", which alternative in result-first narrative vs claim-evidence-limitation structure where every quantitative claim has comparator and uncertainty is preferable, and is the change explained by confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Create figures/tables directly from the experiment pipeline and add automated consistency checks that manuscript numbers match source CSV/JSON. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.

F3. Fixed experiment protocol. Apply the method to at least three independent papers/datasets/failure groups.

Predeclare the prediction, preserve exact versions/configs, and reserve a held-out or independently reconstructed check that can contradict the preferred conclusion. The central adversarial condition is: Give figures/captions to a colleague without the main text; test whether they infer the claimed result, comparison, and limitation correctly.

F4. Required evidence. Measure claim-to-evidence traceability, numerical consistency with source results, uncertainty/comparator coverage, and reader/audience comprehension. Deliver exact paper/code/data identifiers, experiment matrix, raw outputs, decision memo, and a counterexample/negative result that constrains the claim. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Writing and presentation. If a stable failure/crossover is found among result-first narrative vs claim-evidence-limitation structure where every quantitative claim has comparator and uncertainty, implement one adaptive or corrected variant that directly targets confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Writing and presentation. Using the measured claim-to-evidence traceability, numerical consistency with source results, uncertainty/comparator coverage, and reader/audience comprehension, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Writing and presentation

- G1. Closed-book derivation/oracle. Derive the logical dependency of a research argument from claim to evidence to limitation; rewrite one result so every quantitative claim has an explicit comparison, uncertainty, and scope. Then solve this exact micro-case from scratch without opening your implementation: Take one hypothetical result: 12.4 vs 11.8 with standard errors 0.3 and 0.4. Write a claim that is supported, one that overclaims significance/generalization, and a figure caption that states comparator, metric, and uncertainty. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Write a 6-page mock paper from a reproduction+extension project and give a 12-minute talk. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally manually copy a result into the manuscript so the plotted/table value can diverge from raw experiment output. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Create figures/tables directly from the experiment pipeline and add automated consistency checks that manuscript numbers match source CSV/JSON. Then apply this stress without retuning: Give figures/captions to a colleague without the main text; test whether they infer the claimed result, comparison, and limitation correctly. Report claim-to-evidence traceability, numerical consistency with source results, uncertainty/comparator coverage, and reader/audience comprehension and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among result-first narrative vs claim-evidence-limitation structure where every quantitative claim has comparator and uncertainty for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured claim-to-evidence traceability, numerical consistency with source results, uncertainty/comparator coverage, and reader/audience comprehension; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Writing and presentation. Using claim-to-evidence traceability, numerical consistency with source results, uncertainty/comparator coverage, and reader/audience comprehension, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 18, Unit 9. Research problem portfolio

Recommended time. 1.3-2.0 weeks (about 20-40 focused hours depending on background).

What you are learning. Maintain several problems with different risk levels.

A. Reading order

1. Read the primary assignment first: Use the open-problem portfolio in this document plus the current SIGGRAPH Technical Papers program to maintain five candidate problems. Score each on importance, novelty evidence, tractability, infrastructure readiness, evaluation clarity, falsifiability, and one-week diagnostic value; run a one-week diagnostic sprint on the top two before committing.
2. Use the lecture/tutorial only after a first pass at the mathematics/algorithm: Quarterly problem review.
3. Keep the most useful executable/reference page open while coding: Use the scored project matrix, baseline code and one-week diagnostic-sprint repository as the executable reference for each candidate problem.

B. Definitions, results and algorithms to know

- Low-risk: engineering/robustness
- medium: new combination/analysis
- high-risk: new representation/theory
- Precision requirement: distinguish and justify intuition-only project choice vs scored portfolio using importance, novelty, tractability, infrastructure, evaluation clarity, and falsifiability. For each alternative you actually implement, state its validity conditions and one concrete regime in which it is expected to fail or become inefficient.

C. Derivations and pencil-and-paper work

- For each candidate problem, derive a one-page model of novelty, tractability, required infrastructure, falsifiable hypothesis, minimum publishable experiment, and likely negative result; rank projects using these dimensions.
- Failure-prediction exercise: before running code, derive or reason out the expected signature of this controlled stress test: Score at least five candidate problems before a diagnostic sprint, update scores after one week of evidence, and examine which criteria predicted tractability/value. Write down which measured quantity should change first, and in which direction, if your explanation is correct.

D. Implementation laboratory

- P1 - Core implementation. Rank problems by importance, novelty, tractability, evaluation clarity, and dependency risk. Keep a minimal reference path even if you later optimize it.
- P2 - Oracle regression test. Encode this hand-checkable case as an automated test: Score three candidate projects from 1-5 on importance, novelty, tractability, infrastructure readiness, evaluation clarity, and falsifiability. Change one score after a one-day diagnostic result and show how the ranking changes. The test must fail under at least one intentionally introduced sign/index/frame/boundary bug.
- P3 - Robustness/diagnostic harness. Score at least five candidate problems before a diagnostic sprint, update scores after one week of evidence, and examine which criteria predicted tractability/value. Save the input and config of each qualitatively different failure as a named regression case.
- P4 - Research-grade programming extension. Build a scored project matrix and a one-week diagnostic-sprint template; update scores from evidence rather than intuition and archive the decision rationale.
- P5 - Machine-readable evidence. Export diagnostic-sprint evidence, updated tractability/novelty scores, infrastructure risk, falsifiability, and expected information gain of next experiment to CSV/JSON from the experiment itself; regenerate the principal plot/table from that file with a separate script.
- P6 - Comparative benchmark package. Put the alternatives in one common experiment harness: intuition-only project choice vs scored portfolio using importance, novelty, tractability, infrastructure, evaluation clarity, and falsifiability. The harness must accept a method selector, case/config identifier, random seed (when applicable), and output path; run the exact stress case "Score at least five candidate problems before a diagnostic sprint, update scores after one week of evidence, and examine which criteria predicted tractability/value." without hand-editing code between methods. Record diagnostic-sprint evidence, updated tractability/novelty scores, infrastructure risk, falsifiability, and expected information gain of next experiment.
- P7 - Deliberate bug and regression drill. Intentionally rank candidate projects only by novelty/excitement and ignore baseline accessibility, falsifiability, and diagnostic-sprint evidence. Before running, predict the first observable failure in diagnostic-sprint evidence, updated tractability/novelty scores, infrastructure risk, falsifiability, and expected information gain of next experiment. Verify that the oracle/stress tests detect the bug, minimize or simplify one failing input, fix the code, and keep that case permanently in the regression suite.

E. Curated problem set

1. Derivation with oracle. For each candidate problem, derive a one-page model of novelty, tractability, required infrastructure, falsifiable hypothesis, minimum publishable experiment, and likely negative result; rank projects using these dimensions. Then evaluate the result on the explicit micro-case in Problem 2 before looking at your implementation. Deliverable: derivation, units/sign conventions, and the numerical/geometric oracle.
2. Smallest nontrivial hand case. Score three candidate projects from 1-5 on importance, novelty, tractability, infrastructure readiness, evaluation clarity, and falsifiability. Change one score after a one-day diagnostic result and show how the ranking changes. Deliverable: every intermediate value needed to detect a sign, indexing, interpolation, probability-density, coordinate-frame, or boundary-condition mistake.
3. Core programming build. Implement the unit core: Rank problems by importance, novelty, tractability, evaluation clarity, and dependency risk. Before optimizing, make the smallest case agree with Problem 2. Then complete this deeper task: Build a scored project matrix and a one-week diagnostic-sprint template; update scores from evidence rather than intuition and archive the decision rationale. Deliverable: runnable code plus one named regression test that reproduces the hand oracle.
4. Matched method comparison. Compare intuition-only project choice vs scored portfolio using importance, novelty, tractability, infrastructure, evaluation clarity, and falsifiability. Use the same paper/data/split/metric and decision question when comparing research workflows; differences must come from the methodological change, not access to different evidence. Record diagnostic-sprint evidence, updated tractability/novelty scores, infrastructure risk, falsifiability, and expected information gain of next experiment. Deliverable: a machine-generated comparison table/plot and a short explanation of the observed crossover or dominance regime.
5. Adversarial experiment. Score at least five candidate problems before a diagnostic sprint, update scores after one week of evidence, and examine which criteria predicted tractability/value. State the predicted failure signature before execution. Deliverable: a sweep/failure map, the saved worst-case input, and an explanation linked to the derivation rather than to appearance alone.
6. Reading reconstruction. Using the assigned reading: Use the open-problem section at end of this document. Independently reconstruct one central algorithmic step, numerical example, proof/derivation step, or figure that directly bears on Research problem portfolio. Do not copy reference code. Deliverable: your reconstruction, a discrepancy log, and a statement of which mathematical or implementation details were underspecified by the source.
7. Research mini-study. Predict the failure/crossover boundary among intuition-only project choice vs scored portfolio using importance, novelty, tractability, infrastructure, evaluation clarity, and falsifiability. State, before running experiments, which option should win in at least two contrasting regimes and why. Execute the controlled protocol from Section F, using diagnostic-sprint evidence, updated tractability/novelty scores, infrastructure risk, falsifiability, and expected information gain of next experiment. Deliverable: a 2-4 page mini-report with pre-registered prediction, baselines, uncertainty/refinement where appropriate, negative results, and a final decision using this criterion: Advance a project only after a short diagnostic sprint establishes an accessible baseline, a measurable failure, a falsifiable hypothesis and an evaluation capable of distinguishing mechanism-level progress from cosmetic improvement.

F. Concrete research project - Research problem portfolio

- F1. Falsifiable question. Under the controlled failure regime "Score at least five candidate problems before a diagnostic sprint, update scores after one week of evidence, and examine which criteria predicted tractability/value.", which alternative in intuition-only project choice vs scored portfolio using importance, novelty, tractability, infrastructure, evaluation clarity, and falsifiability is preferable, and is the change explained by confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability? Write a one-paragraph prediction before running the experiment; name at least one observation that would falsify it.
- F2. Code target. Start from P1/P2, then complete this unit-specific extension: Build a scored project matrix and a one-week diagnostic-sprint template; update scores from evidence rather than intuition and archive the decision rationale. Add a single experiment entry point so every compared method uses the same input, seed, precision, stopping rule/sample budget, and metric code.
- F3. Fixed experiment protocol. Apply the method to at least three independent papers/datasets/failure groups. Predeclare the prediction, preserve exact versions/configs, and reserve a held-out or independently reconstructed check that can contradict the preferred conclusion. The central adversarial condition is: Score at least five candidate problems before a diagnostic sprint, update scores after one week of evidence, and examine which criteria predicted tractability/value.

F4. Required evidence. Measure diagnostic-sprint evidence, updated tractability/novelty scores, infrastructure risk, falsifiability, and expected information gain of next experiment. Deliver exact paper/code/data identifiers, experiment matrix, raw outputs, decision memo, and a counterexample/negative result that constrains the claim. Include one case where the method you expected to win does not win, or explicitly report that no crossover/failure was found in the tested domain.

F5. Research escalation for Research problem portfolio. If a stable failure/crossover is found among intuition-only project choice vs scored portfolio using importance, novelty, tractability, infrastructure, evaluation clarity, and falsifiability, implement one adaptive or corrected variant that directly targets confounding, leakage, reproducibility, hypothesis discrimination, and evidence-to-claim traceability; compare it against the strongest fixed baseline under the same protocol. If no stable boundary appears, do not manufacture a novelty claim: broaden the stress range or record the negative result and move on.

F6. Go/no-go criterion for Research problem portfolio. Using the measured diagnostic-sprint evidence, updated tractability/novelty scores, infrastructure risk, falsifiability, and expected information gain of next experiment, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Continue toward a paper only if the effect survives the required controls and the mechanism is supported by the diagnostics; otherwise treat the unit as mastered but the research seed as closed.

G. Mastery checkpoint - self-contained practical for Research problem portfolio

- G1. Closed-book derivation/oracle. For each candidate problem, derive a one-page model of novelty, tractability, required infrastructure, falsifiable hypothesis, minimum publishable experiment, and likely negative result; rank projects using these dimensions. Then solve this exact micro-case from scratch without opening your implementation: Score three candidate projects from 1-5 on importance, novelty, tractability, infrastructure readiness, evaluation clarity, and falsifiability. Change one score after a one-day diagnostic result and show how the ranking changes. You pass this part only if the algebra/geometry and the stored oracle agree.
- G2. Clean-room programming. In a fresh file or small module, rebuild the essential implementation without copying your previous source: Rank problems by importance, novelty, tractability, evaluation clarity, and dependency risk. Add the oracle as an automated test and make it pass before importing any optimized/framework code.
- G3. Debugging exam. Intentionally rank candidate projects only by novelty/excitement and ignore baseline accessibility, falsifiability, and diagnostic-sprint evidence. Predict the failure signature first, run the P2/P3/P7 tests, identify the broken assumption or convention, and demonstrate that the corrected code removes the failure without weakening the test.
- G4. Transfer experiment. On an input/configuration not used during development, execute: Build a scored project matrix and a one-week diagnostic-sprint template; update scores from evidence rather than intuition and archive the decision rationale. Then apply this stress without retuning: Score at least five candidate problems before a diagnostic sprint, update scores after one week of evidence, and examine which criteria predicted tractability/value. Report diagnostic-sprint evidence, updated tractability/novelty scores, infrastructure risk, falsifiability, and expected information gain of next experiment and explain which invariant should remain stable and which quantity is allowed/expected to change.
- G5. Method-selection defense. Using your own measured results, choose among intuition-only project choice vs scored portfolio using importance, novelty, tractability, infrastructure, evaluation clarity, and falsifiability for three concrete regimes: (i) the oracle/nominal case, (ii) the adversarial case above, and (iii) a resource- or accuracy-constrained case of your choice. For each choice, cite the measured diagnostic-sprint evidence, updated tractability/novelty scores, infrastructure risk, falsifiability, and expected information gain of next experiment; state one counterexample that would reverse your recommendation.
- G6. Pass condition for Research problem portfolio. Using diagnostic-sprint evidence, updated tractability/novelty scores, infrastructure risk, falsifiability, and expected information gain of next experiment, Pass when the research decision follows from reproducible claim-to-evidence tracing, includes a competing explanation or negative result, and survives a held-out/independent check. Archive the clean-room code, regression case, raw metrics, and one-page method-selection note; if any of these are missing, the checkpoint is not complete.

Volume 18 Closeout: Project, Review and Mastery Gate

Volume project. Complete one paper reproduction, one systematic failure study and one original extension. Write the result in SIGGRAPH-style structure with reproducible code and a 10-12 minute presentation.

Cumulative review

- Complete one four-stage reproduction, including a partial independent reimplement of a central method component.
- Design a benchmark with leakage/nuisance checks and a held-out failure/adversarial split.
- Run a predeclared ablation matrix with appropriate seeds/uncertainty and identify at least one misleading comparison.
- Turn a clustered anomaly into competing hypotheses and perform an intervention on held-out failures.
- Finish with a scored research portfolio in which every preferred problem has baseline, falsifiable hypothesis, accessible infrastructure and a one-week diagnostic sprint result.

Mastery gate before moving on

- Complete one Level-3-or-higher reproduction and document every meaningful discrepancy from the paper.
- Turn a clustered, systematic failure into a falsifiable hypothesis with baseline, intervention, metric, and a disconfirming outcome.
- Prepare code/configuration/figures/tables that let another researcher audit the central claim without private knowledge.
- Literature gate: complete one classic-current-benchmark triplet in your chosen subfield; reproduce one result, independently rebuild one component, and write a claim-evidence-failure map that changes at least one project decision.

Parallel Mathematical Animation Path: 24 Weeks From Scratch

Volume 17 is already detailed unit-by-unit. This parallel calendar is a hobby-friendly sequence that can begin immediately, even while you are studying Volume 1.

Weeks	Focus	Learn	Deliverable
1-2	Python + Manim installation	Python scripts/functions/classes; Manim Quickstart; Scene/construct; command line; output folders.	10 micro-scenes: text, circle, transform, axes, function, point, vector, matrix, 3D camera, export.
3-4	Mobjects and transforms	VGroup, positioning, Transform, ReplacementTransform, animate syntax, coordinate systems.	Recreate three geometric diagrams with reusable helper functions.
5-6	LaTeX and semantic equation motion	MathTex/Tex, TransformMatchingTex, stable token grouping, color semantics.	Animate a 6-10 step derivation without symbol teleportation.
7-8	Updaters and dynamic geometry	ValueTracker, always_redraw, updaters, traced paths, dependent labels.	Secant-to-tangent, moving barycentric coordinates and one dynamic frame.
9-10	Linear algebra	Vectors, bases, determinant, eigenvectors, projections, SVD.	3-minute SVD or eigenvector explainer.

Weeks	Focus	Learn	Deliverable
11-12	Probability / Monte Carlo	Seeded randomness, samples, histograms, convergence, variance.	Monte Carlo integration with variance comparison.
13-14	Fourier / sampling / graphics	Aliasing, convolution, spectra, texture filtering, mipmapping.	Nyquist/aliasing explainer with spatial and frequency views.
15-16	3D mathematics	ThreeDScene, camera orientation, surfaces, moving frames and occlusion-aware labels.	Tangent plane/normal, sphere geometry or projective camera frustum.
17-18	Graphics algorithms	Ray intersection, BVH traversal, barycentrics, Laplacians, geodesics, path tracing.	One step-by-step algorithm animation with invariant highlighting.
19-20	Simulation visualization	Particles, phase space, energy, vector fields, pressure projection.	Integrator stability or incompressible pressure projection explainer.
21-22	Narration and editing	Storyboard, voice timing, captions, pacing, sound, export pipeline.	90-second polished mini-video with narration.
23-24	Capstone	Choose one nontrivial theorem/algorithm; cite primary sources; script before code.	3-5 minute first capstone; later expand to 6-10 minutes.

Primary tutorial: <https://docs.manim.community/en/stable/>

Recommended practice rule: every time a concept in Volumes 1-16 feels abstract, make a 20-90 second visualization of it. The goal is not social-media polish; it is to force yourself to decide what the mathematical objects are and what must remain invariant while they move.

Research-Level Problem Portfolio

These are directions, not promises of novelty. The research task begins by reproducing a baseline and locating a systematic limitation. Each problem below is paired with a first benchmark, then a sequence that can turn it into a paper-quality question.

Shared research protocol for R1-R12

- Baseline first: reproduce or run at least one strong baseline with documented settings before proposing a new method.
- Controlled benchmark: where feasible, include a synthetic or manufactured case with ground truth plus a realistic case; sweep the hypothesized failure variable rather than comparing one easy and one hard example.
- Matched comparisons: control compute, data, resolution, optimization budget, and preprocessing to the extent relevant; report uncertainty across seeds/instances when stochasticity matters.
- Failure-centered evaluation: predeclare failure cases/selection rules, keep negative results, and include an ablation or intervention capable of falsifying the proposed mechanism.
- Reproducibility: save configurations, exact inputs, code revision, raw machine-readable results, and a one-command script for the central figure/table.
- Publishability gate: by the end of the baseline study, you should be able to state a falsifiable sentence of the form: method family X fails under condition Y for mechanism Z; the proposed change targets Z and moves metric/tradeoff M without unacceptable cost C.

R1. Low-variance differentiable light transport through visibility events

Develop estimators that remain accurate and computationally useful when geometry, shadows, caustics, or occlusion boundaries move. The hard part is not differentiating smooth BSDF algebra but handling visibility discontinuities without intolerable bias or variance.

How to start. Prerequisites: Vols. 1, 5, 6, 13. First experiment: a small Mitsuba benchmark containing moving silhouettes, shadows and specular paths. Compare finite differences, naïve autodiff/smoothing, and a projective/reparameterized estimator. Report gradient error, variance, runtime and optimization success.

Problem-specific success and falsification criteria

- Gradient correctness: on smooth parameters, pass central-difference gradient checks over a step-size sweep; on a moving visibility boundary, report bias and variance as a function of distance to the event rather than one aggregate number.
- Optimization usefulness: recover at least two known scene parameters from multiple initializations and plot error versus gradient-sample/render time; compare finite differences, a naïve/smoothed estimator, and the visibility-aware estimator at matched cost.
- Falsification: abandon the claimed improvement if lower variance is purchased by systematic gradient bias that worsens held-out parameter recovery, or if the benefit disappears outside one hand-picked visibility configuration.

R2. Relightable and editable 3D Gaussian scene representations with physical meaning

Separate geometry, material and illumination in splat-based scene representations so that editing and novel-light rendering are consistent across views rather than merely photorealistic under training illumination.

How to start. Prerequisites: Vols. 6, 13, 14. First experiment: synthetic PBRT/Mitsuba scenes with known BRDFs and lights. Train baseline 3DGS and a factorized variant; evaluate novel-view plus novel-light PSNR/SSIM/LPIPS and material/normal error.

Problem-specific success and falsification criteria

- Decomposition accuracy: use synthetic scenes with known normals, albedo/BRDF parameters, illumination, and geometry; evaluate those latent quantities directly in addition to novel-view PSNR/SSIM/LPIPS.
- Relighting/edit consistency: test at least three held-out lighting directions/environments and two material edits; measure cross-view consistency and unchanged-region error, with model size and rendering speed matched to baseline 3DGS.
- Falsification: do not call the representation physically meaningful if it improves held-out images while normals/materials are wrong, or if edits/relighting require retraining that erases the proposed factorization advantage.

R3. Uncertainty-aware inverse graphics

Most inverse pipelines output one answer even when geometry/material/light are not identifiable. Study calibrated uncertainty or multimodal solutions rather than only point estimates.

How to start. Prerequisites: Vols. 1, 12-14. First experiment: ambiguous single/few-view scenes with known families of equivalent solutions; compare ensembles, Laplace approximations or probabilistic parameterizations and test calibration.

Problem-specific success and falsification criteria

- Calibration: construct both identifiable and deliberately ambiguous scenes. Report empirical interval/credible-set coverage, calibration curves, negative log likelihood or proper scoring rule, and whether distinct solution modes are represented.
- Decision value: compare point estimate, ensemble/Laplace, and multimodal method on downstream decisions such as relighting or geometry tolerance; uncertainty should predict where those decisions fail.
- Falsification: a wider interval is not better uncertainty. Reject the method if nominal coverage is poor, modes collapse in known ambiguous cases, or confidence fails to distinguish identifiable from non-identifiable scenes.

R4. Robust intrinsic geometry processing on pathological meshes

Make geometric algorithms retain guarantees or graceful degradation on meshes with skinny triangles, holes, duplicated elements, poor sampling, mixed scales or nonmanifold features.

How to start. Prerequisites: Vols. 1, 7, 8. First experiment: build a public adversarial mesh suite and benchmark geodesics, curvature, parameterization or vector-field algorithms; identify which failures are numerical, topological or modeling failures.

Problem-specific success and falsification criteria

- Adversarial suite: parameterize skinny-angle, scale, duplicate-element, hole, nonmanifold, and sampling-irregularity severity while retaining analytic/controlled ground truth whenever possible.
- Robustness metrics: report failure rate plus geometry-specific error (distance, curvature, distortion, operator residual), conditioning/predicate fallback counts, and convergence under remeshing/refinement.
- Falsification: a patch is not robust if it merely avoids crashes while returning scale-dependent or nonconvergent answers, or if it damages accuracy on well-conditioned meshes without a documented tradeoff.

R5. Geometry processing beyond fixed triangle meshes

Design operators that transfer cleanly among meshes, point clouds, implicit surfaces and neural/Gaussian representations while preserving intrinsic geometric meaning.

How to start. Prerequisites: Vols. 7, 8, 14. First experiment: reproduce one intrinsic operator on mesh and point/splat representation and compare convergence under resampling.

Problem-specific success and falsification criteria

- Cross-representation test: sample the same continuous sphere/torus/surface family as meshes, point clouds, implicit fields, and splats at several resolutions; evaluate one intrinsic operator against the same analytic quantity.
- Invariance/convergence: measure convergence versus sampling density and sensitivity to rotation, nonuniform sampling, holes, and representation conversion; separate discretization error from representation-conversion error.
- Falsification: reject claims of representation independence if hyperparameters must be retuned per representation or if outputs agree only after converting everything back to a privileged triangle mesh.

R6. Large-timestep free-surface simulation with controlled dissipation and robust topology change

Increase timestep and speed without destroying small-scale vortices, volume, surface detail or collision robustness. Recent graphics work makes this an active direction.

How to start. Prerequisites: Vols. 1, 10, 11. First experiment: standard dam-break and two-phase benchmarks. Compare FLIP/PIC/APIC baselines across timestep; separate pressure-solve, advection and transfer errors.

Problem-specific success and falsification criteria

- Benchmark grid: run a smooth advection/vortex case with reference solution, a dam-break/free-surface case, and a topology-change/splash case across timestep and grid/particle resolution.
- Physical/numerical metrics: volume error, divergence residual, kinetic-energy/vorticity decay, surface position/detail error, pressure iterations, wall time, and failure rate; separate transfer, advection, and pressure-solve contributions.
- Falsification: larger timesteps count only if the method remains stable under refinement and does not obtain speed by erasing the small-scale phenomena the benchmark is intended to preserve.

R7. Contact-robust thin-shell and cloth simulation at production scale

Combine accurate bending/stretching with guaranteed nonintersection, friction, large deformation and practical runtime.

How to start. Prerequisites: Vols. 1, 10. First experiment: reproduce cloth folding/stacking benchmark under IPC/constraint solver and search failure regimes in mesh resolution, friction, thickness and timestep.

Problem-specific success and falsification criteria

- Contact suite: include folding, sliding friction, cloth-on-cloth stacking, thin-gap contact, and high-speed impact across mesh resolution, thickness, friction coefficient, and timestep.
- Pass metrics: zero/controlled interpenetration, contact constraint residual, stretch/bending error, energy behavior, solver iterations, and wall time. Verify convergence or at least monotone behavior as collision tolerance is tightened.
- Falsification: visual nonintersection alone is insufficient; reject the method if collision robustness depends on excessive damping/stiffening, nonphysical thickness inflation, or unresolved timestep-specific artifacts.

R8. Physically constrained generative 3D

Generate shapes/scenes that satisfy geometry, support, collision, mechanics or manufacturability constraints rather than optimizing only image/text similarity.

How to start. Prerequisites: Vols. 7, 10, 13, 15. First experiment: a constrained object family where validity can be checked exactly or with FEM; compare unconstrained generation, rejection, guidance, and simulator-in-the-loop refinement.

Problem-specific success and falsification criteria

- Validity benchmark: choose a family with an executable checker (collision/support), manufacturability rule, or FEM objective. Report validity rate and physical margin, not only pass/fail.
- Generative tradeoff: compare unconstrained generation, rejection/filtering, guidance/conditioning, and simulator-in-the-loop refinement under matched sample/compute budgets; report diversity and semantic/prompt quality alongside physics.
- Falsification: reject a method that raises validity by collapsing to a few trivial shapes, exploiting a weak simulator/checker, or sacrificing the intended design objective without exposing the Pareto tradeoff.

R9. Dynamic 4D scene representations with topology change and editable semantics

Represent fast nonrigid motion, appearance changes and topology changes while maintaining temporal coherence and editability.

How to start. Prerequisites: Vols. 9, 14, 15. First experiment: synthetic sequences with known deformation/topology events; compare canonical-deformation and time-native representations.

Problem-specific success and falsification criteria

- Event-controlled data: use sequences with known nonrigid motion, occlusion, split/merge topology events, and semantic edits at known times; include held-out viewpoints and intermediate times.
- Metrics: reconstruction/view quality, temporal flicker/consistency, geometry/flow error when ground truth exists, topology-event timing, edit propagation accuracy, memory, and training/render time.
- Falsification: a temporally smooth output fails if it smooths over a genuine topology event; an editable output fails if the edit drifts across time/views or corrupts unchanged regions.

R10. Compact neural/splat scene streaming for AR/VR

Optimize quality, memory, bandwidth, latency and update cost jointly for head-tracked immersive rendering.

How to start. Prerequisites: Vols. 2, 4, 12, 14. First experiment: build rate-quality-latency curves under pruning, quantization and LOD; include rapid viewpoint motion and foveated quality metrics.

Problem-specific success and falsification criteria

- Rate-quality-latency curves: sweep pruning, quantization, LOD, and streaming bandwidth; report model memory, transmitted bytes/s, decode/update cost, frame-time distribution (including tail latency), and view-quality metrics.

- Head-motion stress: evaluate rapid viewpoint changes, disocclusions, and foveated eccentricity; a nominal average frame rate is insufficient. Include at least one target such as a 90 Hz / 11.1 ms frame budget on stated hardware.
- Falsification: reject improvements that shift work outside the measured frame (predecode/cache), fail under viewpoint changes, or improve compression only by degrading geometry/temporal stability in peripheral/novel regions.

R11. Differentiable computational optics and display design

Optimize optical surfaces/holograms jointly with image formation while accounting for manufacturing constraints and spectral/field-of-view robustness.

How to start. Prerequisites: Vols. 1, 6, 12, 13. First experiment: phase-mask or relief-surface design under multiple wavelengths/angles; compare ideal optimization with fabrication-aware regularization.

Problem-specific success and falsification criteria

- Robust optical benchmark: optimize over multiple wavelengths, field angles, depths, and fabrication perturbations rather than a single monochromatic on-axis target; keep an ideal unconstrained design as an upper-bound baseline.
- Metrics: PSF/MTF or wavefront error, diffraction efficiency/light throughput, reconstructed-image quality, field-of-view/depth range, sensitivity to quantization/fabrication error, and optimization cost.
- Falsification: reject a design improvement that relies on unrealizable phase/relief resolution, vanishing throughput, or extreme sensitivity to tiny fabrication/wavelength changes not present in the optimization objective.

R12. Automatic mathematical visualization from formal structure

Given an equation, proof outline or algorithm, automatically propose a visual representation and animation plan that preserves mathematical invariants and explanatory continuity.

How to start. Prerequisites: Vols. 1, 17 plus ML if used. First experiment: a dataset of 20-50 hand-designed micro-explainers with explicit semantic objects; evaluate generated storyboards for correctness, correspondence of symbols/objects, and human comprehension.

Problem-specific success and falsification criteria

- Correctness set: create 20-50 examples with explicit semantic objects, invariants, dependencies, and known ambiguous symbol correspondences; include algebra, geometry, calculus, probability, and one graphics algorithm.
- Evaluation: measure semantic-object correspondence accuracy, invariant violations, executable numeric consistency, storyboard edit distance/time, and blinded viewer comprehension/misconception rates against human-authored baselines.
- Falsification: reject a system that produces attractive motion but changes mathematical meaning, relies on manual repair for repeated symbols/topology changes, or fails to improve comprehension/editing effort over a simple template baseline.

18-Volume Programming Capstones - Minimum Research-Ready Builds

These capstones are deliberately more concrete than the unit exercises. Treat each as an integration exam for the corresponding volume. A capstone is complete only when the code, tests, raw outputs, plots and failure cases can be regenerated from a clean checkout; screenshots or a visually convincing demo alone do not count.

Volume 1 Capstone. Numerical Graphics Laboratory

- Build one Python package with subcommands ``transform``, ``solve``, ``sample``, ``spectrum``, ``integrate``, and ``optimize``; include a slow reference implementation for every numerical kernel.
- Required cases: an affine-normal inverse-transpose test; least squares with prescribed condition numbers 10^2 , 10^6 , 10^{10} ; a Monte Carlo integral with analytic truth; a sampled sinusoid above/below Nyquist; explicit vs symplectic/backward integration on a stiff oscillator; a small nonlinear least-squares problem.

- Export forward error, residual, invariant violation, estimator bias/variance, spectral aliasing energy, energy drift, convergence rate and runtime. Plot error against conditioning/sample count/timestep rather than reporting one number.
- Acceptance: all analytic cases pass; at least three deliberately injected bugs are caught by regression tests; you can explain one regime where a numerically cheaper method is preferable and one where it becomes unreliable.

Volume 2 Capstone. Graphics Systems and Performance Harness

- Build a modern C++ scene/mesh core using explicit ownership, Eigen (or equivalent) math, a spatial index, deterministic asset loading, and a small GPU compute/shader benchmark suite.
- Implement and compare AoS/SoA access, kd-tree or BVH queries, exact/adaptive vs epsilon-based geometric predicates, scan/compaction, and one bandwidth-bound plus one compute-bound GPU kernel.
- Run AddressSanitizer/UBSan, property-based geometry tests, stale-bound/update tests, CPU/GPU profiler captures and repeated timing with warm-up. Record compiler flags, GPU/driver, thread/block size, bytes moved and arithmetic intensity where meaningful.
- Acceptance: no sanitizer failures; reference and optimized paths agree; performance claims survive repeated runs; the profiler evidence identifies the bottleneck and predicts the effect of one optimization before you apply it.

Volume 3 Capstone. RAW-to-Display Imaging Pipeline

- Build a small image pipeline: synthetic/raw sensor model -> mosaicing -> shot/read noise -> demosaicing -> linear color transform/white balance -> exposure/HDR merge -> tone map -> display encoding. Keep all filtering in the correct linear/premultiplied domain.
- Test with a color chart, a slanted edge, a high-frequency zone plate, a saturated HDR scene and a moving/noisy burst. Include one ground-truth linear image so every stage can be compared independently.
- Measure PSNR/SSIM only as secondary diagnostics; also report color error, edge MTF/aliasing, clipping fraction, noise statistics and temporal ghosting. Save intermediate buffers for every stage.
- Acceptance: you can localize a deliberately wrong gamma/alpha/white-point/noise-model choice from the diagnostics without looking at the source code; a one-command script regenerates the complete stage-by-stage report.

Volume 4 Capstone. Software Rasterizer and Real-Time Renderer

- Implement triangle setup/rasterization, top-left fill rule, clipping, perspective-correct interpolation, depth, texture sampling with mipmapping, normal mapping, PBR shading, shadow maps and one temporal or hybrid ray-traced feature.
- Use shared-edge triangles, near-plane clipping, extreme perspective foreshortening, grazing microfacet highlights, thin geometry, moving occluders and disocclusions as mandatory test scenes.
- Produce debug buffers for barycentrics, depth, mip level, normals, motion vectors, shadow test and history validity. Measure pixels/triangles per second, GPU timings, temporal error/ghosting and relevant image differences against a reference.
- Acceptance: no cracks/double hits on shared edges; mip selection follows screen-space footprint; temporal history rejects disocclusions; you can trace any artifact to a specific stage using only the debug buffers and counters.

Volume 5 Capstone. CPU Ray-Tracing System

- Build a CPU renderer with robust ray/triangle/sphere intersections, instancing/transforms, camera/film, scene graph, BVH build/traversal, textures and a BSDF sample/eval/pdf interface.
- Required scenes: analytic sphere/plane; Cornell-style box; 1M-triangle mesh; thin/grazing triangles; nested transforms; static vs mildly deforming BVH scene. Compare brute force, median BVH and SAH BVH.
- Measure intersection error, self-intersection count, BVH build/refit time, node/primitive visits, rays/s, memory and image difference. Keep deterministic render regression images.
- Acceptance: analytic intersection oracles pass; transformed instances match world-space reference; BVH and brute-force images agree; you can state when rebuild beats refit from measured deformation/traversal tradeoffs.

Volume 6 Capstone. Physically Based Light-Transport Renderer

- Extend the Volume 5 renderer with radiometry-consistent lights, Lambertian/specular/GGX BSDFs, cosine and light sampling, next-event estimation, Russian roulette, MIS path tracing, participating media, and one advanced transport method or difficult-path experiment.

- Mandatory scenes: diffuse Cornell box with known symmetry/sanity checks; small bright area light; glossy grazing reflection; environment light; homogeneous fog; and a caustic/difficult-specular scene where basic MIS is expected to struggle.
- Record sample/eval/pdf consistency, energy/reciprocity checks where applicable, per-technique contribution histograms, mean/variance across independent seeds, error against a high-sample reference, variance per second and path-length statistics.
- Acceptance: estimator means converge to the reference; Russian roulette is unbiased within uncertainty; MIS PDFs are in consistent measures; you can explain a scene where MIS helps strongly and a scene where changing the path-space strategy matters more than adding samples.

Volume 7 Capstone. Geometric Modeling Workbench

- Build interactive/CLI tools for Bezier and B-spline/NURBS evaluation/subdivision, subdivision surfaces, one implicit/SDF representation with meshing, procedural modeling, and a small shape-optimization problem.
- Use endpoint/repeated-knot cases, extraordinary subdivision vertices, sharp vs smooth implicit blends, thin features and deliberately awkward design-variable parameterizations. Store exact control/knot/parameter files.
- Measure interpolation/continuity error, refinement convergence, SDF/Eikonal deviation, mesh Hausdorff error, topology/feature preservation and optimization objective/constraint history.
- Acceptance: endpoint/knot continuity matches theory; extraordinary/boundary rules are explicit; sphere tracing never assumes a non-SDF blend is exact; an optimization result is reproduced from its parameter file and constraint checks.

Volume 8 Capstone. Discrete Geometry Processing Workbench

- Build a half-edge validator plus cotangent Laplacian/mass matrix, curvature, heat-method geodesics, harmonic/LSCM parameterization, ARAP deformation, remeshing or intrinsic triangulation, and one Hodge/topology computation.
- Create an adversarial mesh suite: clean sphere/torus, irregular triangulation, skinny triangles, boundaries, flipped faces, duplicated/nonmanifold elements and noisy geometry. Include at least one analytic or manufactured ground truth per operator family.
- Report invariant violations (e.g. orientation, $d1*d0$), eigen/spectrum or curvature errors, geodesic error, angle/area distortion and flips, deformation energy, triangle quality, convergence under refinement and failure classification.
- Acceptance: algorithms refuse or diagnose invalid topology instead of silently proceeding; convergence is demonstrated on clean refinement; the failure suite distinguishes numerical, discretization, topology and modeling errors.

Volume 9 Capstone. Character Animation and Motion Laboratory

- Implement skeleton hierarchy/forward kinematics, quaternion interpolation, inverse kinematics, linear-blend and dual-quaternion skinning, and one motion retargeting/synthesis/control component.
- Use a 2-link analytic IK arm, antipodal quaternion interpolation, twisting forearm/candy-wrapper case, retargeting between different limb lengths, foot-contact walking clip and a motion-transition case.
- Export joint/endpoint error, skinning volume/twist distortion, contact/slip error, joint-limit violations, trajectory smoothness and transition discontinuity. Produce synchronized plots alongside rendered motion.
- Acceptance: analytic FK/IK cases are exact within tolerance; quaternion interpolation takes the intended short arc; foot contact is diagnosed numerically rather than visually; retargeting preserves stated constraints on a skeleton not used during development.

Volume 10 Capstone. Solid, Cloth and Contact Simulator

- Build a common time-stepping framework with particle/spring sanity cases, rigid-body impulse/contact tests, FEM deformable solids, cloth or shell discretization, and a robust contact/friction method or clearly scoped baseline.
- Mandatory tests: oscillator with known frequency; free rigid-body conservation; manufactured linear-elastic patch test; nearly incompressible material; cloth drape/fold; stacked/contacting objects; timestep and mesh-refinement sweeps.
- Measure energy/momentum drift, residuals/Newton iterations, displacement/stress error, volume/locking behavior, penetration/contact complementarity, friction work, timestep limit and wall time.
- Acceptance: patch/manufactured tests converge; constitutive/numerical failures are separated; contact does not gain robustness by silently allowing large penetration; the first instability/nonconvergence case is saved as a regression.

Volume 11 Capstone. Fluid, Particle-Grid and Multiphysics Simulator

- Implement MAC-grid incompressible flow with advection and pressure projection, then add particles and PIC/FLIP/APIC transfer or MPM; include one free-surface/smoke case and one coupling or multiphysics extension.
- Required tests: manufactured Poisson solve; divergence-free vortex/advection case; dam break/free surface; transfer-only particle-grid cycle; MPM deformation/material test; one coupled interface case if pursued.
- Sweep grid resolution and timestep/CFL. Report divergence norm, pressure residual, volume/mass, kinetic energy/dissipation, particle-grid transfer error, momentum conservation, interface/coupling residual and runtime.
- Acceptance: projection reduces divergence by the predicted amount; refinement trends are sensible; PIC/FLIP/APIC differences are visible in transfer/dissipation metrics; larger timesteps are not credited if they erase benchmark phenomena.

Volume 12 Capstone. Camera, Optics and Computational-Imaging Laboratory

- Build pinhole/thin-lens camera calibration and ray generation, sensor/noise/response simulation, one computational-photography inverse problem, light-field or plenoptic refocusing, and one Fourier/diffraction optics experiment.
- Use checkerboard/calibration geometry, depth-of-field targets, synthetic noisy RAW burst, HDR exposure stack, two-plane light-field scene and a known aperture/diffraction pattern. Keep geometry and radiometric calibration errors separable.
- Measure reprojection error, recovered intrinsics/extrinsics, depth/focus error, radiometric/noise-model fit, inverse-problem residual vs parameter error, refocusing consistency and diffraction-pattern error.
- Acceptance: camera conventions are validated by rays and reprojection; inverse problems are tested on identifiable and ambiguous cases; Fourier-optics outputs match an analytic/known pattern before moving to learned computational imaging.

Volume 13 Capstone. Differentiable Rendering and Inverse Graphics Suite

- Use Mitsuba 3/Dr.Jit or an equivalent differentiable renderer to solve pose, material/light and shape or volume inverse problems. Implement finite-difference gradient checks on smooth cases and at least one visibility-discontinuity-aware experiment.
- Construct synthetic scenes with known parameters, moving silhouette/shadow boundaries, specular paths and at least one intentionally ambiguous geometry-light-material case. Run multiple starts and controlled noise/calibration perturbations.
- Report relative gradient error away from discontinuities, parameter recovery error, image loss, convergence traces, multi-start dispersion, identifiability/ambiguity diagnostics and runtime/memory.
- Acceptance: image loss is never used as the sole success metric; smooth gradients agree with finite differences; visibility failures are identified; uncertainty/multimodality is reported when distinct parameters explain the same observations.

Volume 14 Capstone. NeRF and Gaussian-Splatting Scene Laboratory

- Train a compact NeRF baseline and a 3D Gaussian Splatting baseline on the same scenes/cameras, then add at least one geometry/relighting/dynamic/editing/compression extension.
- Use one forward-facing/simple scene and one 360-degree scene. Perturb camera poses, exposure, view count and view distribution independently; for 3DGS also vary densification/pruning and SH degree. Hold evaluation views fixed.
- Report PSNR/SSIM/LPIPS plus geometry/depth where available, training time, render FPS, peak memory/storage, primitive/parameter count, uncertainty-error calibration if studied, and failure montages for sparse views/specularities/calibration errors.
- Acceptance: NeRF/3DGS comparisons use matched data and evaluation; representation error is separated from calibration/data error; deployment/compression claims include quality-memory-latency Pareto curves rather than one compressed model.

Volume 15 Capstone. Generative 3D/4D and Physics-Aware Generation Benchmark

- Build one controlled generation benchmark spanning a geometry representation (point/mesh/implicit/splat), conditional generation or reconstruction, and one structured validity checker. Add material, motion, scene-layout or physics-aware constraints depending on specialization.

- Freeze a prompt/image/test set before tuning. Evaluate at least two model families or two guidance strategies with matched compute. For physics-aware work, keep simulator/checker independent and include generate-then-filter versus simulator-in-the-loop refinement.
- Report diversity/coverage and perceptual metrics separately from geometry validity, collisions/support/manufacturability or simulator objective. Include per-category failure annotations and at least one metric-ranking reversal found by human/physical inspection.
- Acceptance: the generator cannot grade itself; improved validity is not purchased by collapse without disclosure; the benchmark includes held-out compositions/conditions; a negative result or failure taxonomy accompanies the best-looking samples.

Volume 16 Capstone. Visualization, XR and Fabrication Prototype

- Choose two tracks: scientific/data visualization, interaction/VR/AR, computational fabrication, audio/haptics, or optics/robotics crossover. Build one interactive prototype and one offline/physical-validity pipeline connected by the same data model.
- Examples: volume renderer with transfer-function design + task study; OpenXR scene with latency/pose logging; mesh repair/offset/slicing/manufacturability checker; modal sound from simulated motion; haptic/geometric contact visualization.
- Define task-specific accuracy and usability/latency/manufacturability metrics before implementation. Save interaction logs or checker outputs, not only videos. Include one stress condition where the prototype fails or becomes unusable.
- Acceptance: the evaluation matches the actual application task; XR claims include latency/tracking evidence, fabrication claims include geometric/manufacturing validity, and visualization claims include a user/task or quantitative interpretation test.

Volume 17 Capstone. Mathematical Animation Toolkit and Explainer

- Build reusable Manim components for coordinate systems, semantic equation transforms, geometric constructions, vector fields, plots, camera moves and data-driven numerical animations. Every displayed numeric value should come from executable code.
- Create at least four short scenes: a proof/geometric invariant, a calculus/linear-algebra transformation, a stochastic/Fourier simulation, and a graphics algorithm (e.g. barycentrics, rasterization, ray tracing, BRDF sampling, Laplacian or pressure projection).
- For a 6-10 minute capstone explainer, storyboard first; implement two alternative versions of the most difficult semantic transition; pilot with at least three viewers and ask content questions, not preference-only questions.
- Acceptance: repeated/reordered/cancelled symbols are mapped semantically rather than by brittle substring indices; animations stay correct after changing numerical parameters; viewer responses support the intended mathematical inference.

Volume 18 Capstone. Research Reproduction and Evidence Pipeline

- Select one paper in your intended specialization with public code/data and one paper whose core component you will independently reimplement. Build a clean repository with environment lock, exact data/code identifiers, configs, immutable raw results and automatic figure/table generation.
- First reproduce one headline figure/table within a declared tolerance; then recreate one central component without copying author code; finally perturb one assumption or failure variable and predeclare what outcome would falsify your extension hypothesis.
- Maintain a failure database with at least 30 cases, a claim-evidence matrix, baseline/ablation table with uncertainty across seeds/instances where applicable, and a literature map of 25-40 closely related papers.
- Acceptance: a fresh environment reproduces the main result from one command; manuscript numbers match source CSV/JSON automatically; at least one alternative explanation is tested; the final go/no-go research memo is supported by held-out or independently reconstructed evidence.

Paper Reading and Reproduction Protocol

Pass 1 - 15 minutes: Read title, abstract, figures, captions, conclusion. Write the problem, claimed novelty and strongest evidence in three sentences.

Pass 2 - 45-90 minutes: Identify representation, objective/equations, algorithm stages, assumptions, data, baselines and evaluation metrics. Reconstruct the pipeline as a diagram.

Pass 3 - derivation: Derive the central estimator/update/energy without looking at code. Mark every approximation and nondifferentiable/discrete step.

Pass 4 - code map: Map paper symbols to code variables/files/functions. Identify hidden defaults and preprocessing.

Pass 5 - reproduction: Reproduce one small result first. Freeze environment, seed, data version and configuration.

Pass 6 - stress test: Create cases outside the authors' polished regime. Search for phase transitions rather than anecdotal failures.

Pass 7 - synthesis: Compare against at least two neighboring methods using the same benchmark and write a failure taxonomy.

Reproduction Ladder

- Level 0: run official demo and save exact environment/configuration.
- Level 1: reproduce one reported metric or figure on authors' data.
- Level 2: rerun on a new seed/scene/dataset and explain discrepancy.
- Level 3: reimplement one important component independently.
- Level 4: reproduce the central claim with independent code or framework.
- Level 5: discover and characterize a systematic failure; propose and test an extension.

Experiment Log Template

- Question/hypothesis; exact commit/configuration; dataset/scene identifiers; seed; hardware/software environment.
- Independent variable and range; dependent metrics; controls; stopping rule.
- Expected result before running the experiment.
- Observed result with plots/tables and raw artifact locations.
- Interpretation; alternative explanations; next discriminating experiment.
- Whether the result changes the research hypothesis.

Core Book and Chapter Map

This appendix is a lookup table. It is not a command to read every book cover-to-cover.

Fundamentals of Computer Graphics, 5e: Ch. 1 introduction; 2 mathematical foundations; 3 raster images; 4 ray tracing; 5 shading; 6 linear algebra; 7 transformations; 8 viewing; 9 graphics pipeline; 10 signal processing; 11 texture mapping; 12 data structures; 13 sampling; 14 physics-based rendering; 15 curves; 16 animation; 17 graphics hardware; 18 color; 19 visual perception; 20 tone reproduction; 21 implicit modeling; 22 games; 23 visualization.

Physically Based Rendering, 4e: Ch. 1 system overview; 2 Monte Carlo integration; 3 geometry/transforms; 4 radiometry/spectra/color; 5 cameras/film; 6 shapes; 7 acceleration; 8 sampling/reconstruction; 9 reflection; 10 textures/materials; 11 participating media; 12 lights; 13 surface light transport; 14 volume light transport; 15 GPU wavefront rendering; 16 retrospective/future; appendices for sampling/math/utilities.

Real-Time Rendering, 4e: Ch. 2 graphics rendering pipeline; 3 GPU; 4 transforms; 5 shading basics; 6 texturing; 7 shadows; 8 light/color; 9 physically based shading; 10 local illumination; 11 global illumination; 12 image-space effects. Use remaining material and the companion resource site selectively for intersections, acceleration, VR and advanced real-time topics.

Trefethen & Bau, Numerical Linear Algebra: Use Lectures 1-11 for core matrix/vector/orthogonality/QR ideas and Lectures 24-31 for eigenvalue algorithms and related numerical viewpoints. Revisit conditioning whenever a graphics solve is unstable.

Nocedal & Wright, Numerical Optimization: Selected Ch. 2-3 for line search and convergence foundations; Ch. 10 for least squares/Gauss-Newton context. Add constrained chapters only as needed for contact/design/inverse problems.

Keenan Crane, Discrete Differential Geometry: Follow the course order rather than page-count goals: smooth/discrete curves and surfaces; curvature; exterior algebra and forms; exterior calculus/Stokes; Laplacians/FEM; parameterization; geodesics; topology/cohomology; Hodge decomposition/vector fields.

Physics-Based Simulation open book: Use the current online section structure by topic: time integration and optimization foundations; PBD/XPBD; FEM/deformables; collision/contact/IPC; fluids; particle-grid methods; MPM; advanced continuum/contact topics. Because the book is actively maintained, prefer section titles over hard-coded page numbers.

Fluid Simulation for Computer Graphics, Bridson: Use as a conceptual companion for incompressible flow, advection, pressure projection, free surfaces, particle/grid coupling and level sets. Pair every chapter with an implementation/refinement experiment.

Munzner, Visualization Analysis and Design: Use selected chapters for task abstraction, marks/channels, data/idiom design and evaluation when studying Volume 16.

Goodman, Introduction to Fourier Optics: Use selected Fourier-optics chapters only in Volume 12 if computational optics/holography becomes a specialization.

Online Resources and Tutorials - Verified 12 August 2026

Berkeley CS184/284A Spring 2026 course: <https://cs184.eecs.berkeley.edu/sp26/>

PBRT 4th edition - full online book: <https://pbr-book.org/4ed/contents>

Keenan Crane - Discrete Differential Geometry: <https://www.cs.cmu.edu/~kmc Crane/Projects/DDG/>

libigl tutorial: <https://libigl.github.io/tutorial/>

Geometry Central: <https://geometry-central.net/>

Physics-Based Simulation open book: <https://phys-sim-book.github.io/>

Stanford CS348C Animation and Simulation: <https://graphics.stanford.edu/courses/cs348c/>

Mitsuba 3 documentation: <https://mitsuba.readthedocs.io/en/stable/>

Mitsuba 3 inverse rendering tutorials: https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering_tutorials.html

Nerfstudio documentation: <https://docs.nerf.studio/>

3D Gaussian Splatting official project: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>

NeRF / related project page by Ben Mildenhall: <https://bmild.github.io/>

Real-Time Rendering 4e resources: <https://www.realtimerendering.com/>

Manim Community documentation: <https://docs.manim.community/en/stable/>

SIGGRAPH 2026 Technical Papers: <https://s2026.siggraph.org/program/technical-papers/>

How to use online resources

- Use official documentation as executable reference, not as a substitute for derivation.
- For course sites, do the worksheets/projects after attempting the problem yourself.
- For libraries such as libigl, Mitsuba and Nerfstudio, trace one algorithm through the source rather than only calling the top-level API.
- For current-literature pages such as SIGGRAPH, maintain a rolling paper queue tagged by representation, objective, algorithmic technique, evaluation type and open failure.
- For Manim, build a reusable local examples repository; every useful pattern should become a small function/class with a rendered demo.

End-of-Curriculum Competency Checklist

Mathematics: Linear algebra/decompositions, projective geometry, multivariable calculus, probability/Monte Carlo, Fourier sampling, ODE/PDE discretization, differential geometry and optimization.

Classical rendering: Rasterization, texture filtering, visibility/shadows, ray tracing/BVH, radiometry, BSDFs, path tracing, MIS, media and GPU rendering architecture.

Geometry: Meshes/splines/subdivision/implicit representations, discrete differential operators, parameterization, geodesics, deformation, remeshing, topology and robustness.

Animation/simulation: Kinematics/skinning/motion plus particles, rigid bodies, collision/contact, FEM/shells, fluids, particle-grid methods and MPM.

Imaging/inverse graphics: Camera/sensor/optics models, computational imaging, differentiable rendering, visibility gradients, material/light/shape/pose estimation and identifiability.

Neural/generative graphics: NeRF/implicit fields, multiscale acceleration, 3DGS, relighting/dynamics/editing/compression, diffusion/generative 3D and physically constrained generation.

Systems: Modern C++, testing, profiling, GPU APIs/shaders, parallel patterns, robust numerics and reproducible environments.

Research: Paper decomposition, benchmark design, reproduction, ablation, failure analysis, literature mapping, research writing and presentation.

Communication: Ability to create precise mathematical diagrams and Manim animations in which every motion corresponds to a mathematical or algorithmic statement.

Minimum Portfolio Before Calling Yourself Research-Ready

- 1 software rasterizer or GPU PBR renderer.
- 1 BVH ray tracer and 1 physically based path tracer with MIS.
- 1 geometry-processing workbench with at least five nontrivial operators.
- 1 rigid/cloth/FEM simulator and 1 fluid/MPM-style simulator with quantitative diagnostics.
- 1 inverse-rendering benchmark and 1 neural scene-representation benchmark.
- 2 full paper reproductions, including at least one with meaningful discrepancy analysis.
- 1 systematic failure study that leads to a falsifiable new research question.
- 1 polished 6-10 minute mathematical/graphics animation explaining a nontrivial concept.
- 1 research-style report with code, configuration, ablations and a presentation.

This plan should remain a living document. Once a specialization becomes clear, replace breadth reading in the final year with a narrower paper-driven queue, but retain the habit of derivation, implementation, stress-testing and visual explanation.